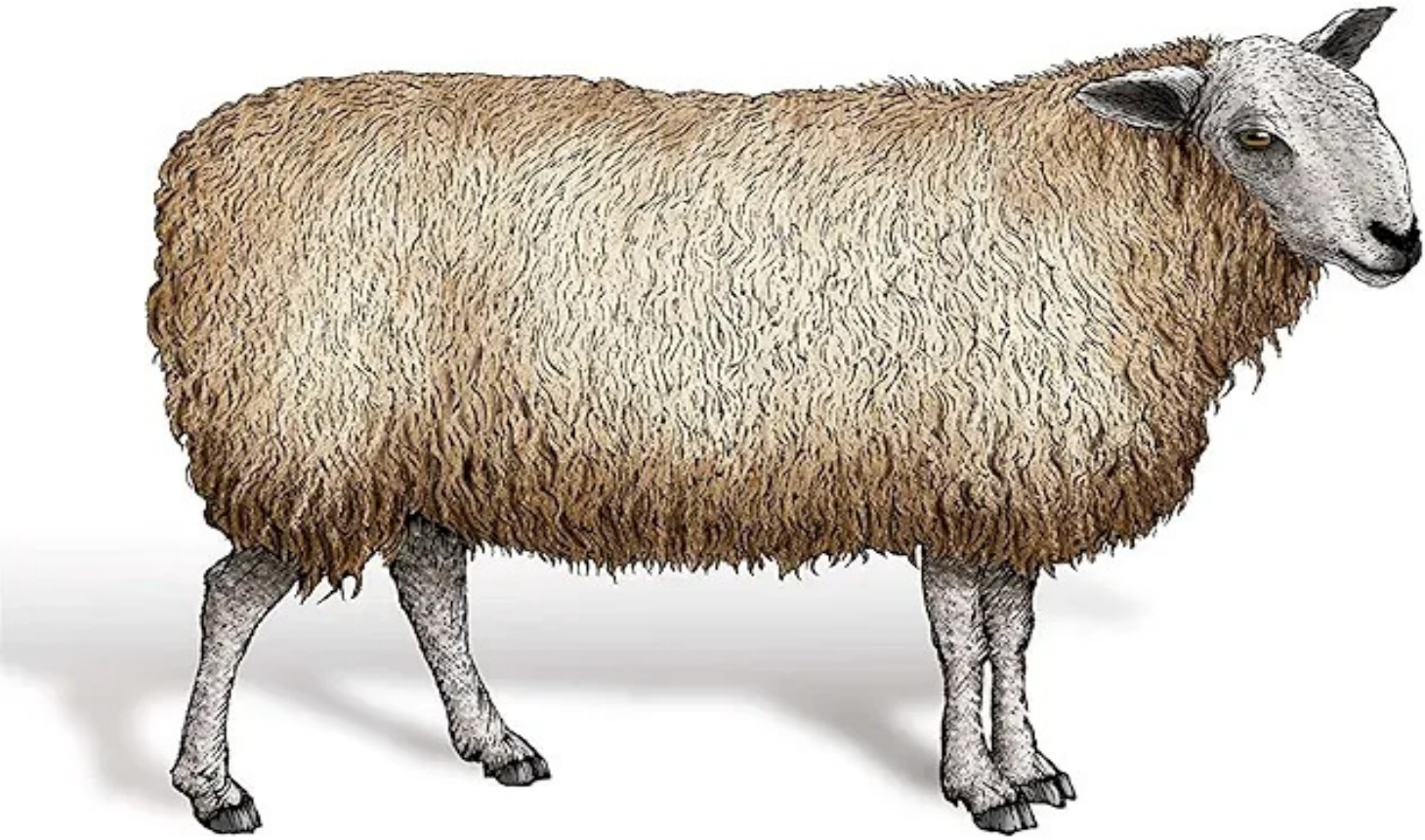
O'REILLY®

Análise Prática de Séries Temporais

Predição com Estatística e Aprendizado de Máquina



ALTA BOOKS
EDITORA

Aileen Nielsen

Índice

direito autoral

Índice

Prefácio

- Quem deveria ler esse livro
  - Antecedentes Esperados
- Por que escrevi este livro
- Navegando neste livro
- Recursos online
- Convenções utilizadas neste livro
- Usando exemplos de código
- Aprendizagem on-line O'Reilly
- Como entrar em contato conosco
- Agradecimentos

Capítulo 1. Séries Temporais: Uma Visão Geral e um Histórico Rápido

- A história das séries temporais em diversas aplicações
  - Medicina como um problema de série temporal
  - Previsão do tempo
  - Previsão de crescimento econômico
  - Astronomia

A análise de série temporal decola

As origens da análise estatística de séries temporais

As origens da análise de série temporal de aprendizado de máquina

Mais recursos

Capítulo 2. Encontrando e organizando dados de séries temporais

Onde encontrar dados de série temporal

Conjuntos de dados preparados

Série temporal encontrada

Adaptando uma coleção de dados de série temporal a partir de uma coleção de tabelas

Um exemplo prático: montando uma coleção de dados de série temporal

Construindo uma série temporal encontrada

Problemas de carimbo de data/hora

De quem é o carimbo de data/hora?

Adivinhando carimbos de data e hora para entender os dados

O que é uma escala de tempo significativa?

Limpando seus dados

Tratamento de dados ausentes

Upsampling e Downsampling

Suavização de dados

Dados sazonais

Fusos horários

Prevenindo a antecipação

Mais recursos

Capítulo 3. Análise Exploratória de Dados para Séries Temporais

Métodos familiares

Plotagem

Histogramas

Gráficos de dispersão

Métodos Exploratórios Específicos de Séries Temporais

Compreendendo a estacionariedade

Aplicando funções de janela

Compreendendo e identificando a autocorrelação

Correlações Espúrias

Algumas visualizações úteis

Visualizações 1D

Visualizações 2D

Visualizações 3D

Mais recursos

Capítulo 4. Simulação de dados de série temporal

O que há de especial na simulação de séries temporais?

Simulação versus previsão

Simulações em código

Fazendo o trabalho sozinho

Construindo um universo de simulação que funciona sozinho

Uma simulação de física

Notas Finais sobre Simulações

Simulações Estatísticas

Simulações de aprendizagem profunda

Mais recursos

Capítulo 5. Armazenando Dados Temporais

Definindo Requisitos

Dados ativos versus dados armazenados

Soluções de banco de dados

SQL versus NoSQL

Soluções populares de banco de dados e arquivos de séries temporais

Soluções de arquivos

NumPy

Pandas

Equivalentes R padrão

Matriz X

- Mais recursos

Capítulo 6. Modelos Estatísticos para Séries Temporais

- Por que não usar uma regressão linear?
- Métodos estatísticos desenvolvidos para séries temporais
  - Modelos Autoregressivos
  - Modelos de média móvel
  - Modelos de média móvel integrados autorregressivos
  - Autoregressão vetorial
  - Variações em modelos estatísticos
- Vantagens e desvantagens dos métodos estatísticos para séries temporais
- Mais recursos

Capítulo 7. Modelos de Espaço de Estados para Séries Temporais

- Modelos de espaço de estado: vantagens e desvantagens
- O Filtro de Kalman
  - Visão geral
  - Código para o Filtro Kalman
- Modelos ocultos de Markov
  - Como funciona o modelo
  - Como ajustamos o modelo

Ajustando um HMM no código

Série temporal estrutural bayesiana

Código para bsts

Mais recursos

Capítulo 8. Gerando e selecionando recursos para uma série temporal

Exemplo introdutório

Considerações gerais ao calcular recursos

A natureza da série temporal

Conhecimento de Domínio

Considerações Externas

Um catálogo de lugares para encontrar recursos de inspiração

Bibliotecas de geração de recursos de série temporal de código aberto

Exemplos de recursos específicos de domínio

Como selecionar recursos depois de gerá-los

Considerações finais

Mais recursos

Capítulo 9. Aprendizado de Máquina para Séries Temporais

Classificação de série temporal

Selecionando e gerando recursos

Métodos de árvore de decisão

Agrupamento

Gerando recursos a partir dos dados

Métricas de distância com reconhecimento temporal

Código de agrupamento

Mais recursos

Capítulo 10. Aprendizado profundo para séries temporais

Conceitos de aprendizagem profunda

Programando uma rede neural

Dados, símbolos, operações, camadas e gráficos

Construindo um pipeline de treinamento

Inspecionando nosso conjunto de dados

Etapas de um pipeline de treinamento

Redes Feed Forward

Um exemplo simples

Usando um mecanismo de atenção para tornar as redes feed forward mais conscientes do tempo

CNN

Um modelo convolucional simples

Modelos Convolucionais Alternativos

RNNs

Continuando nosso exemplo elétrico

A inovação do codificador automático

Arquiteturas de Combinação

Resumindo

Mais recursos

Capítulo 11. Erro de Medição

O básico: como testar previsões

Considerações específicas do modelo para backtesting

Quando sua previsão é boa o suficiente?

Estimando a incerteza em seu modelo com uma simulação

Prevendo vários passos à frente

Ajustar-se diretamente ao horizonte de interesse

Abordagem recursiva para horizontes temporais distantes

Aprendizagem multitarefa aplicada a séries temporais

Dicas de validação de modelo

Mais recursos

Capítulo 12. Considerações de desempenho no ajuste e exibição de modelos de série temporal

Trabalhando com ferramentas desenvolvidas para casos de uso mais gerais

Modelos construídos para dados transversais não “compartilham” dados entre amostras

Modelos que não pré-calculam criam um atraso desnecessário entre a medição dos dados e a

elaboração de uma previsão

Formatos de armazenamento de dados: vantagens e desvantagens

Armazene seus dados em formato binário

Pré-processe seus dados de uma forma que permita “deslizar” sobre eles

Modificando sua análise para atender às considerações de desempenho

Usar todos os seus dados não é necessariamente melhor

Modelos complicados nem sempre funcionam melhor o suficiente

Uma breve menção a ferramentas alternativas de alto desempenho

Mais recursos

Capítulo 13. Aplicações de Saúde

Prevendo a gripe

Um estudo de caso de gripe em uma área metropolitana

O que é o estado da arte na previsão da gripe?

Previsão dos níveis de glicose no sangue

Limpeza e exploração de dados

Gerando Recursos

Ajustando um modelo

Mais recursos

Capítulo 14. Aplicações Financeiras

Obtenção e exploração de dados financeiros

Pré-processamento de dados financeiros para aprendizado profundo

Adicionando Quantidades de Interesse aos Nossos Valores Brutos

Dimensionando quantidades de interesse sem previsão

Formatando nossos dados para uma rede neural

Construindo e treinando um RNN

Mais recursos

Capítulo 15. Séries Temporais para o Governo

Obtenção de dados governamentais

Explorando grandes dados de séries temporais

Faça um upsample e agregue os dados à medida que os iteramos

Classifique os dados

Análise estatística on-line de dados de séries temporais

Perguntas restantes

Melhorias adicionais

Mais recursos

Capítulo 16. Pacotes de Séries Temporais

Previsão em escala

Previsão industrial interna do Google

Pacote Profeta de Código Aberto do Facebook

Detecção de anomalia

Pacote de detecção de anomalias de código aberto do Twitter

Outros pacotes de série temporal

Mais recursos

Capítulo 17. Previsões sobre previsões

Previsão como serviço

Deep Learning aumenta possibilidades probabilísticas

Importância crescente do aprendizado de máquina em vez da estatística

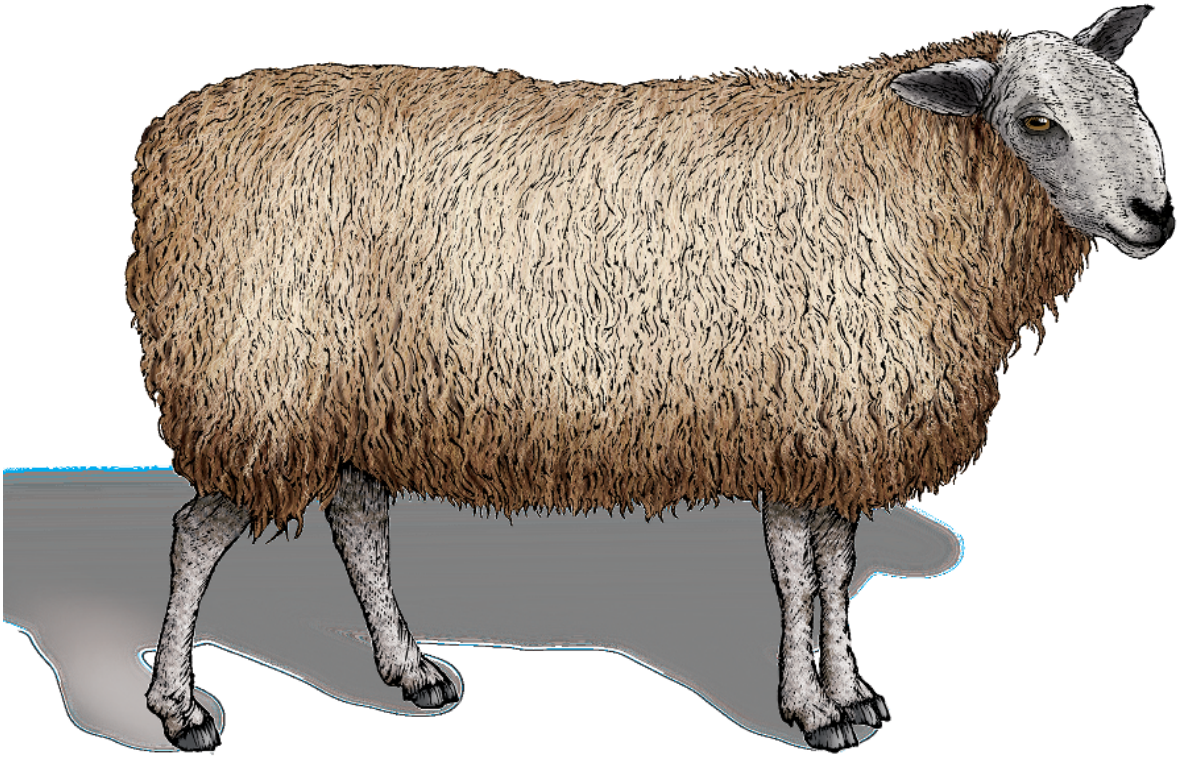
Combinação crescente de metodologias estatísticas e de aprendizado de máquina

Mais previsões para a vida cotidiana

Índice

Sobre o autor

Colofão



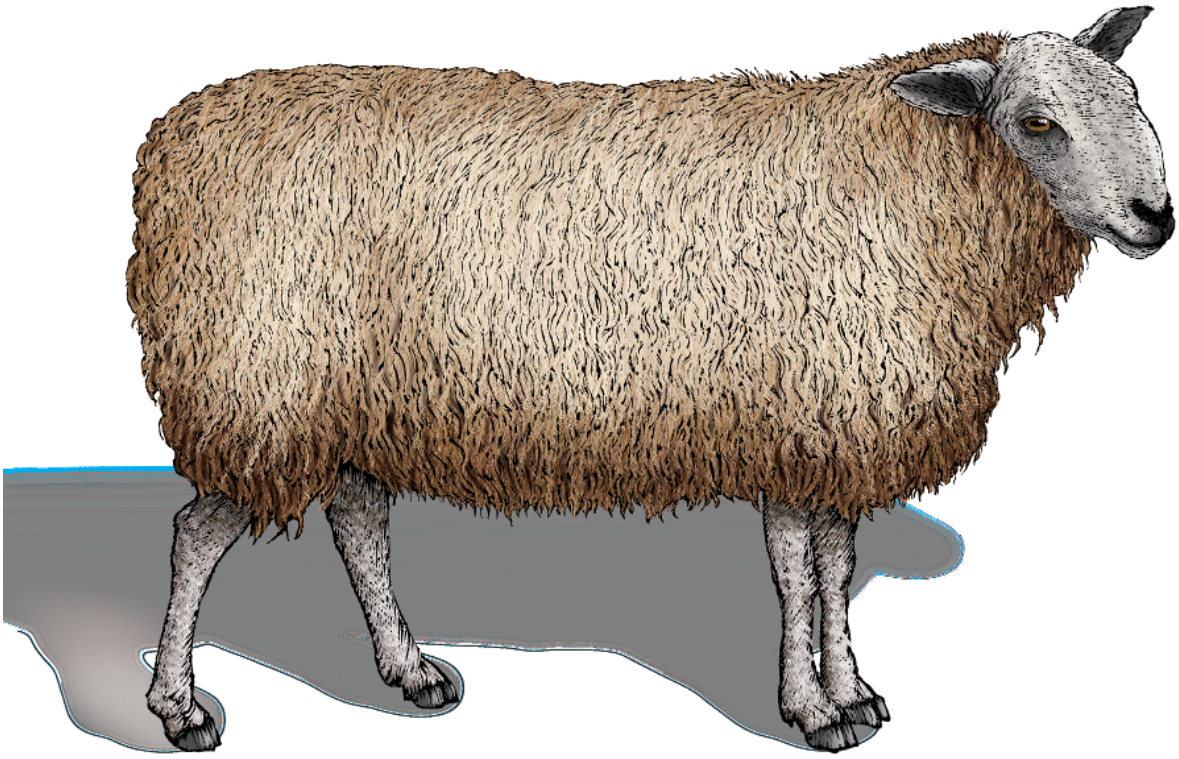
Tempo Prático

Análise de série

Predição com Estatística e Aprendizado de Máquina

Aileen Nielsen

9 781492 041658 56999



Análise prática de séries temporais

Prática

A análise de dados de séries temporais é cada vez mais importante devido à

produção massiva de tais dados através da internet das coisas,

"Um notável

Tempo Prático

a digitalização dos cuidados de saúde e a ascensão das cidades inteligentes.

t

recurso. Em algum ponto,

eu

À medida que o monitoramento contínuo e a coleta de dados se tornam mais

todo cientista de dados

comum, a necessidade de uma análise competente de séries temporais com ambos

al Hora S

começa a trabalhar com o tempo

as técnicas estatísticas e de aprendizado de máquina aumentarão.

série ou outros tipos

Análise de série

Cobrindo inovações em análise de dados de séries temporais e casos de uso

de dados funcionais.

do mundo real, este guia prático irá ajudá-lo a resolver

Este livro fornece uma

os desafios mais comuns de engenharia e análise de dados

em séries temporais, usando estatísticas tradicionais e modernas

visão geral de alto nível de

Predição com Estatística e Aprendizado de Máquina

técnicas de aprendizado de máquina. A autora Aileen Nielsen oferece uma

aprendizado de máquina e

introdução acessível e completa às séries temporais em R

análise de série temporal, e

e Python que contará com cientistas de dados, engenheiros de software,

recursos bem projetados

erie

e pesquisadores em funcionamento rapidamente.

implementação

Você receberá a orientação necessária para:

exemplos."

• Encontrar e organizar dados de séries temporais

— Dr. Andreas W. Kempa-Liehr

Análise

Professor experiente,

• Realizar análise exploratória de dados de séries temporais

Departamento de Ciências da Engenharia,

• Armazenar dados temporais

Universidade de Auckland

• Simular dados de séries temporais

• Gerar e selecionar recursos para uma série temporal

Aileen Nielsen é uma Nova York

Engenheiro de software baseado na cidade

• Medir erro

e analista de dados. Ela trabalhou

em séries temporais em uma variedade de

é

• Prever e classificar séries temporais com aprendizado de máquina ou profundo

disciplinas, de uma área de saúde

é

• Avaliar precisão e desempenho

início de uma campanha política,

de um laboratório de pesquisa em física para um

empresa de comércio financeiro. Ela atualmente

Niel

desenvolve redes neurais para

aplicações de previsão.

sen

MODELAGEM DE IA/DADOS

Twitter: @oreillymedia

facebook.com/oreilly

US$ 69,99

PODE $ 92,99

ISBN: 978-1-492-04165-8

Aileen Nielsen

Análise Prática de Séries Temporais MECH.indd 1

0,972"

20/09/19 08:00

Análise prática de séries temporais

Previsão com estatísticas e

Aprendizado de máquina

Aileen Nielsen

Pequim Boston Farnham Sebastopol Tóquio

Análise prática de séries temporais

por Aileen Nielsen

Direitos autorais © 2020 Aileen Nielsen. Todos os direitos reservados.

Impresso nos Estados Unidos da América.

Publicado por O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

Os livros da O'Reilly podem ser adquiridos para uso educacional, comercial ou promocional de vendas. Edições online também estão disponíveis para a maioria dos títulos (*http://oreilly.com*). Para mais informações, entre em contato com nosso departamento corporativo/institucional

departamento de vendas nacional: 800-998-9938 ou *corporate@oreilly.com* .

Editor de Desenvolvimento: Jeff Bleiel

Indexador: WordCo Indexing Services, Inc.

Editor de aquisições: Jonathan Hassell

Designer de Interiores: David Futato

Editora de produção: Katherine Tozer

Designer da capa: Karen Montgomery

Editora: Rachel Monaghan

Ilustradora: Rebecca Demarest

Revisora: Sonia Saruba

Outubro de 2019:

Primeira edição

Histórico de revisões da primeira edição

20/09/2019: Primeiro lançamento

Ver *http://oreilly.com/catalog/errata.csp?isbn=9781492041658* para obter detalhes da versão.

O logotipo da O'Reilly é uma marca registrada da O'Reilly Media, Inc. *Practical Time Series Analysis* , a imagem da

capa e a imagem comercial relacionada são marcas registradas da O'Reilly Media, Inc.

As opiniões expressas neste trabalho são de responsabilidade do autor e não representam as opiniões do editor.

Embora a editora e o autor tenham envidado esforços de boa fé para garantir que as informações e instruções contidas nesta obra sejam precisas, a editora e o autor isentam-se de qualquer responsabilidade por erros ou omissões, incluindo, sem limitação, responsabilidade por danos resultantes do uso ou confiança neste trabalho. O uso das informações e instruções contidas neste trabalho é por sua conta e risco. Se quaisquer exemplos de código ou outra tecnologia que este trabalho contém ou descreve estão sujeitos a licenças de código aberto ou aos direitos de propriedade intelectual de terceiros, é sua responsabilidade garantir que seu uso esteja em conformidade com tais licenças e/ou direitos.

978-1-492-04165-8

[MBP]

Índice

Prefácio. ix

1. Séries temporais: uma visão geral e um histórico rápido. 1

A história das séries temporais em diversas aplicações 2

Medicina como um problema de série temporal 2

Previsão do tempo 6

Prevendo o Crescimento Econômico 7

Astronomia 9

A análise de série temporal decola 10

As Origens da Análise Estatística de Séries Temporais 12

As origens da análise de série temporal de aprendizado de máquina 13

Mais recursos 13

2. Encontrar e organizar dados de séries temporais. 17

Onde encontrar dados de série temporal 18

Conjuntos de dados preparados 18

Série temporal encontrada 25

Adaptando uma Coleta de Dados de Série Temporal a partir de uma Coleção de Tabelas 26

Um exemplo prático: montando uma coleção de dados de série temporal 27

Construindo uma série temporal encontrada 33

Problemas de carimbo de data e hora 35

De quem é o carimbo de data/hora? 35

Adivinhando carimbos de data e hora para entender os dados 36

O que é uma escala de tempo significativa? 39

Limpando seus dados 40

Tratamento de dados ausentes 40

Aumento e redução da resolução 52

Suavização de dados 55

Dados Sazonais 60

Fusos horários 63

Prevenindo Antecipação 67

Mais recursos 69

3. Análise Exploratória de Dados para Séries Temporais. 73

Métodos Familiares 73

Traçando 74

Histogramas 77

Gráficos de Dispersão 78

Métodos Exploratórios Específicos de Séries Temporais 81

Compreendendo a estacionariedade 82

Aplicando Funções de Janela 86

Compreendendo e identificando a autocorrelação 91

Correlações Espúrias 102

Algumas visualizações úteis 104

Visualizações 1D 104

Visualizações 2D 105

Visualizações 3D 113

Mais recursos 117

4. Simulação de dados de séries temporais. 119

O que há de especial na simulação de séries temporais? 120

Simulação versus Previsão 121

Simulações no Código 121

Fazendo o trabalho sozinho 122

Construindo um universo de simulação que funciona sozinho 128

Uma Simulação de Física 134

Notas Finais sobre Simulações 140

Simulações Estatísticas 141

Simulações de aprendizagem profunda 141

Mais recursos 142

5. Armazenamento de dados temporais. 143

Definindo Requisitos 145

Dados ativos versus dados armazenados 146

Soluções de banco de dados 148

SQL versus NoSQL 149

Soluções populares de banco de dados e arquivos de séries temporais 152

Soluções de Arquivo 157

NumPy 158

Pandas 158

Equivalentes R Padrão 158

Matriz X 159

Mais recursos 160

6. Modelos Estatísticos para Séries Temporais. 163

Por que não usar uma regressão linear? 163

Métodos Estatísticos Desenvolvidos para Séries Temporais 166

Modelos Autoregressivos 166

Modelos de Média Móvel 181

Modelos de Média Móvel Integrada Autoregressiva 186

Autoregressão vetorial 196

Variações nos modelos estatísticos 201

Vantagens e desvantagens dos métodos estatísticos para séries temporais 203

Mais recursos 204

7. Modelos de Espaço de Estados para Séries Temporais. 207

Modelos de espaço de estados: vantagens e desvantagens 209

O Filtro Kalman 210

Visão Geral 210

Código para o Filtro Kalman 212

Modelos ocultos de Markov 218

Como funciona o modelo 218

Como ajustamos o modelo 220

Ajustando um HMM no Código 224

Série temporal estrutural bayesiana 229

Código para bsts 230

Mais recursos 235

8. Gerando e selecionando recursos para uma série temporal. 239

Exemplo introdutório 240

Considerações Gerais ao Calcular Recursos 241

A Natureza da Série Temporal 242

Conhecimento de Domínio 242

Considerações Externas 243

Um catálogo de lugares para encontrar recursos para inspiração 243

Bibliotecas de geração de recursos de série temporal de código aberto 244

Exemplos de recursos específicos de domínio 249

Como selecionar recursos depois de gerá-los 252

Considerações Finais 255

Mais recursos 256

9. Aprendizado de máquina para séries temporais. 259

Classificação de Série Temporal 260

Selecionando e Gerando Recursos 260

Métodos de Árvore de Decisão 264

Agrupamento 272

Gerando Recursos a partir dos Dados 273

Métricas de Distância Temporariamente Conscientes 280

Código de agrupamento 285

Mais recursos 287

10. Aprendizado profundo para séries temporais. 289

Conceitos de aprendizagem profunda 292

Programando uma Rede Neural 294

Dados, símbolos, operações, camadas e gráficos 294

Construindo um pipeline de treinamento 298

Inspecionando Nosso Conjunto de Dados 299

Etapas de um pipeline de treinamento 303

Redes Feed Forward 318

Um exemplo simples 319

Usando um mecanismo de atenção para avançar

Redes mais conscientes do tempo 322

CNN 324

Um modelo convolucional simples 325

Modelos Convolucionais Alternativos 327

RNNs 329

Continuando Nosso Exemplo Elétrico 332

A Inovação do Autoencoder 334

Arquiteturas de Combinação 335

Resumindo 340

Mais recursos 341

11. Erro de medição. 343

O Básico: Como Testar Previsões 344

Considerações Específicas do Modelo para Backtesting 347

Quando sua previsão é boa o suficiente? 348

Estimando a incerteza em seu modelo com uma simulação 350

Prevendo Vários Passos à Frente 353

Ajustar diretamente ao horizonte de interesse 353

Abordagem Recursiva para Horizontes Temporais Distantes 354

Aprendizagem multitarefa aplicada à série temporal 354

Dicas de validação de modelo 355

Mais recursos 355

12. Considerações de desempenho no ajuste e veiculação de modelos de séries temporais. 357

Trabalhando com ferramentas desenvolvidas para casos de uso mais gerais 358

Modelos construídos para dados transversais não “compartilham” dados entre amostras 358

Modelos que não pré-calculam criam atrasos desnecessários entre eles

Medindo dados e fazendo uma previsão 360

Formatos de armazenamento de dados: vantagens e desvantagens 361

Armazene seus dados em formato binário 361

Pré-processe seus dados de uma maneira que permita “deslizar” sobre eles 362

Modificando sua análise para atender às considerações de desempenho 362

Usar todos os seus dados não é necessariamente melhor 363

Modelos complicados nem sempre funcionam melhor o suficiente 363

Uma breve menção a ferramentas alternativas de alto desempenho 364

Mais recursos 365

13. Aplicativos de saúde. 367

Prevendo a gripe 367

Um estudo de caso de gripe em uma área metropolitana 367

O que é o estado da arte na previsão da gripe? 383

Previsão dos níveis de glicose no sangue 384

Limpeza e Exploração de Dados 385

Gerando Recursos 390

Ajustando um Modelo 396

Mais recursos 401

14. Aplicações Financeiras. 403

Obtenção e Exploração de Dados Financeiros 404

Pré-processamento de dados financeiros para aprendizagem profunda 410

Adicionando Quantidades de Interesse aos Nossos Valores Brutos 410

Dimensionando quantidades de interesse sem uma previsão 411

Formatando Nossos Dados para uma Rede Neural 413

Construindo e treinando um RNN 416

Mais recursos 423

15. Séries Temporais para Governo. 425

Obtenção de Dados Governamentais 426

Explorando grandes dados de séries temporais 428

Aumente a amostragem e agregue os dados à medida que iteramos por meio deles 431

Classifique os dados 432

Análise Estatística Online de Dados de Séries Temporais 436

Perguntas restantes 446

Melhorias Adicionais 446

Mais recursos 447

16. Pacotes de séries temporais. 449

Previsão na escala 449

Previsão industrial interna 450 do Google

452 do Facebook

Detecção de anomalia 457

457 do Twitter

Outros Pacotes de Série Temporal 460

Mais recursos 461

17. Previsões sobre previsões. 463

Previsão como Serviço 463

O aprendizado profundo aumenta as possibilidades probabilísticas 464

Aumento da importância do aprendizado de máquina em vez da estatística 465

Combinação crescente de metodologias estatísticas e de aprendizado de máquina 466

Mais previsões para a vida cotidiana 466

Índice. 469

Prefácio

Clima, mercados de ações e batimentos cardíacos. Todos eles formam séries temporais. Se você está interessado em dados diversos e em prever o futuro, você está interessado em análise de série temporal.

Bem-vindo à *Análise Prática de Séries Temporais* ! Se você leu este livro, provavelmente já percebeu que os dados de séries temporais estão em toda parte. Os dados de séries temporais tornam-se cada vez mais onipresentes e importantes à medida que o ecossistema de big data se expande. Para o bem ou para o mal, sensores e mecanismos de rastreamento estão por toda parte e, como resultado, há quantidades sem precedentes de dados de séries temporais de alta qualidade disponíveis. As séries temporais são singularmente interessantes porque podem abordar questões de causalidade, tendências e probabilidade de resultados futuros. Este livro irá guiá-lo através das principais técnicas que são mais comumente aplicadas a séries temporais para responder a essas questões.

Os dados de série temporal abrangem uma ampla variedade de disciplinas e casos de uso. Pode ser qualquer coisa, desde históricos de compras de clientes até medições de condutância de um sistema nanoeletrônico até gravações digitais da linguagem humana. Um ponto que discutimos ao longo do livro é que a análise de séries temporais se aplica a um conjunto de dados surpreendentemente diversificado. Quaisquer dados que tenham um eixo ordenado podem ser analisados com métodos de série temporal, mesmo que esse eixo ordenado não seja o tempo em si. Os dados tradicionais de séries temporais, como dados de stocks e padrões meteorológicos, podem ser analisados com métodos de

séries temporais, mas o mesmo pode acontecer com conjuntos de dados peculiares, como espectrógrafos de vinho, onde o eixo do “tempo” é na verdade um eixo de frequência. As séries temporais estão por toda parte.

Quem deveria ler esse livro

Existem dois tipos de leitores pretendidos para este livro. A primeira e maior categoria de leitor é a de um cientista de dados que raramente trabalhou com dados de séries temporais. Essa pessoa pode ser um veterano do setor ou um analista júnior. O analista de dados mais experiente pode percorrer rapidamente as áreas conceituais introdutórias de cada capítulo, mas ainda assim se beneficiará das discussões deste livro sobre as melhores práticas, bem como as armadilhas de trabalhar com dados de séries temporais. Um analista de dados mais novo pode considerar ler o livro na íntegra, embora eu tenha tentado manter cada tópico o mais independente possível.

A segunda categoria de leitor é alguém que supervisiona análises em uma organização com uma extensa coleta interna de dados. Se você for membro deste grupo, ainda precisará de algum conhecimento técnico, mas não é necessário que você esteja codificando em sua vida profissional. Para esse leitor, este livro é útil para apontar oportunidades para sua organização usar a análise de série temporal, mesmo que ela não seja praticada internamente atualmente. Este livro indicará novos tipos de questões e análises que sua organização pode abordar com seus recursos de dados existentes.

Antecedentes Esperados

Com relação à codificação, você deve ter alguma familiaridade com R e Python, especialmente com certos pacotes fundamentais (em Python: NumPy, Pandas e scikit-learn; e em R: data.table). Os exemplos de código devem ser legíveis mesmo sem todo o histórico, mas nesse caso você pode precisar fazer um pequeno desvio para se familiarizar com esses pacotes. Este é provavelmente o caso do data.table do R, um pacote de quadros de dados

subutilizado, mas de alto desempenho, que possui uma funcionalidade de tempo fantástica.

Em todos os casos, forneci breves visões gerais dos pacotes relacionados, alguns exemplos de código e descrições do que o código faz. Também indico ao leitor visões gerais mais completas dos pacotes mais usados.

Com relação à estatística e ao aprendizado de máquina, você deve ter alguma familiaridade com:

Estatísticas introdutórias

Ideias como variância, correlação e distribuições de probabilidade

Aprendizado de máquina

Clustering e árvores de decisão

Redes neurais

O que são e como são treinados

Para esses casos, forneço uma breve visão geral de tais conceitos no texto, mas os não iniciados deveriam ler mais profundamente sobre eles antes de continuar com alguns capítulos. Para a maioria dos tópicos, forneço links para recursos online gratuitos recomendados para breves tutoriais sobre os fundamentos de um determinado tópico ou técnica.

Por que escrevi este livro

Escrevi este livro por três motivos.

Primeiro, as séries temporais são um aspecto importante da análise de dados, mas não são encontrados no kit de ferramentas padrão de ciência de dados. Isto é lamentável porque os dados de séries temporais estão cada vez mais disponíveis e também porque respondem a questões que os dados transversais

não pode. Um analista que não conhece a análise fundamental de séries temporais não está aproveitando ao máximo seus dados. Eu esperava que este livro pudesse preencher uma lacuna existente e importante.

Em segundo lugar, quando comecei a escrever este livro, não conhecia nenhuma visão geral centralizada dos aspectos mais importantes da análise de séries temporais a partir de uma perspectiva moderna da ciência de dados. Existem muitos recursos excelentes disponíveis para a análise tradicional de séries temporais, principalmente na forma de livros clássicos sobre análise estatística de séries temporais. Há também muitas postagens de blog individuais excelentes sobre métodos estatísticos tradicionais e sobre aprendizado de máquina ou abordagens de redes neurais para séries temporais. No entanto, não consegui identificar um único recurso centralizado para delinear todos estes tópicos e relacioná-los entre si. O objetivo deste livro é fornecer esse recurso: uma visão geral ampla, moderna e prática da análise de séries temporais, abrangendo todo o pipeline para dados e modelagem de séries temporais. Mais uma vez, eu esperava que este livro pudesse preencher uma lacuna existente e importante.

Terceiro, as séries temporais são um tópico interessante com preocupações peculiares sobre dados. Os problemas associados ao vazamento de dados, à antecipação e à causalidade são particularmente divertidos do ponto de vista de séries temporais, assim como muitas técnicas que se aplicam exclusivamente a dados ordenados ao longo de algum tipo de eixo temporal. Estudar esses tópicos de forma ampla e encontrar uma maneira de catalogá-los foi outra motivação para escrever este livro.

Navegando neste livro

Este livro está organizado aproximadamente da seguinte forma:

História

O Capítulo 1 apresenta um histórico da previsão de séries temporais, desde o

gregos antigos até os tempos modernos. Isso coloca nosso aprendizado em contexto para que possamos

entender que disciplina tradicional e histórica estudamos neste livro.

Tudo sobre os dados

Os Capítulos 2, 3, 4 e 5 abordam problemas relacionados à obtenção, limpeza, simulação

armazenar e armazenar dados de séries temporais. Esses capítulos estão interligados pela preocupação com tudo o que vem antes que você possa realmente realizar a análise de séries temporais. Esses tópicos raramente são discutidos nos recursos existentes, mas são importantes na maioria dos pipelines de dados. A identificação e limpeza de dados representam uma grande parte do trabalho realizado pela maioria dos analistas de séries temporais.

Modelos, modelos, modelos

Os Capítulos 6 , 7 , 8 , 9 e 10 cobrem uma variedade de técnicas de modelagem que podem ser

usado para análise de série temporal. Começamos com dois capítulos sobre métodos estatísticos, abrangendo modelos estatísticos padrão, como ARIMA e modelos de espaço de estado Bayesiano. Em seguida, aplicamos métodos desenvolvidos mais recentemente, como

aprendizagem e rede neural, para dados de séries temporais, destacando os desafios do **Prefácio | XI**

processamento de dados e layout de dados quando dados de séries temporais são usados para ajustar modelos que não são inerentemente conscientes do tempo, como árvores de decisão.

Considerações pós-modelagem

Os Capítulos 11 e 12 cobrem métricas de precisão e considerações de desempenho,

respectivamente, para fornecer alguma orientação sobre o que você deve considerar depois de realizar suas primeiras iterações de modelagem de série temporal.

Casos de uso do mundo real

Os Capítulos 13 , 14 e 15 fornecem estudos de caso de saúde, finanças e governo.

dados do governo, respectivamente.

Comentários sobre acontecimentos recentes

Os capítulos 16 e 17 cobrem brevemente desenvolvimentos e previsões recentes de séries temporais

para o futuro. O Capítulo 16 é uma visão geral de uma variedade de pacotes automatizados de séries temporais, alguns de código aberto e desenvolvidos como um esforço acadêmico, e alguns

provenientes de grandes empresas de tecnologia. Essas ferramentas estão em desenvolvimento ativo à medida que os esforços aumentam para melhorar a previsão de séries temporais em escala por meio de processos automatizados.

cessações. O Capítulo 17 discute algumas previsões para o futuro da análise de séries temporais.

Isso ocorre à medida que o ecossistema de big data cresce e à medida que aprendemos mais sobre como o big data pode auxiliar na análise de séries temporais.

Em geral, recomendo a leitura de um capítulo antes de tentar trabalhar com o código. Geralmente há alguns conceitos novos introduzidos em cada capítulo, e pode ser útil dar-lhes alguma atenção antes de recorrer ao teclado. Além disso, na maioria dos casos o código para executar determinados modelos é relativamente simples, de modo que o entendimento conceitual será sua principal habilidade adquirida, sendo o conhecimento das APIs de pacotes importantes um benefício secundário que virá muito mais facilmente se você prestar atenção ao conceitos.

Além disso, o livro foi escrito de forma que seja lido de maneira mais sensata do começo ao fim (com os capítulos posteriores referindo-se a conceitos abordados anteriormente no livro), mas, novamente, mantive propositalmente os capítulos o mais independentes possível para permitir que leitores mais experientes leitores possam pular conforme for conveniente.

Recursos online

O'Reilly hospeda um repositório GitHub para este livro . O repositório inclui grande parte do código discutido neste livro em um formato onde você pode executá-lo em conjuntos de dados, que também são fornecidos no repositório. Em alguns casos, os nomes das variáveis, a formatação e outros não serão idênticos aos produzidos no livro, mas devem ser facilmente vinculados (por exemplo, em alguns casos, os nomes das variáveis foram abreviados no livro devido a restrições de formatação).







Além disso, caso você goste de apresentações em vídeo, tenho dois tutoriais on-line que cobrem parte do conteúdo deste livro com foco em Python. Se quiser complementar

este livro com uma apresentação em vídeo, considere os seguintes recursos:

- Análise de série temporal (SciPy 2016)
- Análise moderna de séries temporais (SciPy 2019)

Convenções utilizadas neste livro

As seguintes convenções tipográficas são usadas neste livro:

Itálico Indica novos termos, URLs, endereços de e-mail, nomes de arquivos e extensões de arquivos.

Largura constante

Usado para listagens de programas, bem como dentro de parágrafos para se referir a elementos de programas, como nomes de variáveis ou funções, bancos de dados, tipos de dados, variáveis de ambiente, instruções e palavras-chave.

Largura constante em negrito

Mostra comandos ou outros textos que devem ser digitados literalmente pelo usuário.

Largura constante em itálico

Mostra o texto que deve ser substituído por valores fornecidos pelo usuário ou por valores determinados pelo contexto.

Este elemento significa uma dica ou sugestão.

Este elemento significa uma nota geral.

Este elemento indica um aviso ou cuidado.

O'REILLY®

Usando exemplos de código

Material suplementar (exemplos de código, exercícios, etc.) está disponível para download em

https://github.com/PracticalTimeSeriesAnalysis/BookRepo .

Este livro está aqui para ajudá-lo a realizar seu trabalho. Em geral, se um código de exemplo for oferecido com este livro, você poderá usá-lo em seus programas e documentação. Você não precisa entrar em contato conosco para obter permissão, a menos que esteja reproduzindo uma parte significativa do código. Por exemplo, escrever um programa que utilize vários trechos de código deste livro não requer permissão. Vender ou distribuir um CD-ROM com exemplos de livros da O'Reilly requer permissão. Responder a uma pergunta citando este livro e citando código de exemplo não requer permissão. Incorporar uma quantidade significativa de código de exemplo deste livro na documentação do seu produto requer permissão.

Agradecemos, mas não exigimos, atribuição. Uma atribuição geralmente inclui título, autor, editora e ISBN. Por exemplo: " *Análise Prática de Séries Temporais* por Aileen Nielsen (O'Reilly). Copyright 2020 Aileen Nielsen, 978-1-492-04165-8."

Se você acha que o uso de exemplos de código está fora do uso justo ou da permissão dada acima, sinta-se à vontade para entrar em contato conosco em *permissions@oreilly.com .*

Aprendizagem on-line O'Reilly

Durante quase 40 anos, *A O'Reilly Media* forneceu tecnologia

e treinamento de negócios, conhecimento e visão para ajudar as empresas

sucesso.

Nossa rede exclusiva de especialistas e inovadores compartilha seu conhecimento e experiência por meio de

livros, artigos, conferências e nossa plataforma de aprendizagem on-line. A plataforma de aprendizagem on-line da O'Reilly oferece acesso sob demanda a cursos de treinamento ao vivo, caminhos de aprendizagem aprofundados, ambientes de codificação interativos e uma vasta coleção de textos e vídeos da O'Reilly e de mais de 200 outros editores. Para mais informações por favor visite *http://oreilly.com* .

Como entrar em contato conosco

Por favor, envie comentários e perguntas sobre este livro à editora: O'Reilly Media, Inc.

Rodovia Gravenstein 1005 Norte

Sebastopol, CA 95472

800-998-9938 (nos Estados Unidos ou Canadá)

707-829-0515 (internacional ou local)

707-829-0104 (fax)

Temos uma página web para este livro, onde listamos erratas, exemplos e qualquer informação adicional. Você pode acessar esta página em *https://oreil.ly/practical-time-series-análise* .

E-mail *bookquestions@oreilly.com* para comentar ou fazer perguntas técnicas sobre este livro.

Para mais informações sobre nossos livros, cursos, conferências e notícias, consulte nosso site

site em *http://www.oreilly.com* .

Encontre-nos no Facebook: *http://facebook.com/oreilly*

Siga-nos no Twitter: *http://twitter.com/oreillymedia*

Assista-nos no YouTube:
http://www.youtube.com/oreillymedia

Agradecimentos

Obrigado aos dois revisores técnicos deste livro, professores Rob Hyndman e David Stoffer. Ambos foram excepcionalmente gentis ao reservarem tempo para revisar este livro e fornecerem feedback abundante e útil e novas ideias sobre o que abordar.

O livro é substancialmente melhor do que seria sem a contribuição deles. Agradeço particularmente a Rob por apontar oportunidades perdidas no rascunho original para destacar metodologias alternativas e muitas fontes interessantes de dados de séries temporais. Agradeço particularmente a David por mostrar ceticismo em relação ao excesso de automação

abordagens de análise e por apontar quando eu estava excessivamente otimista em relação a certas ferramentas. Obrigado por toda sua ajuda, conhecimento e perspectiva profundamente experiente em análise de séries temporais.

Sou grato ao meu editor na O'Reilly, Jeff Bleiel, que tem sido extremamente encorajador, solidário e prestativo na revisão destas muitas páginas durante o ano passado. Também devo muitos agradecimentos à minha editora de produção, Katie Tozer, que foi tão paciente com o processo de limpeza deste livro e me orientou através de falhas técnicas na produção. Obrigado a Rachel Monaghan pela edição cuidadosa e excelente.

Obrigado a Rebecca Demarest, que criou muitas das ilustrações deste livro e me ajudou a limpar muitos gráficos desarrumados. Agradecemos também a Jonathan Hassell, que pegou este projeto e o executou e de alguma forma convenceu O'Reilly a publicá-lo.

Por fim, obrigado a Nicole Tache, que me procurou pela primeira vez há alguns anos para trabalhar com O'Reilly. A todos na O'Reilly, obrigado por me deixarem realizar este projeto e por todo o seu apoio durante todo o processo.

Sou grato a muitos leitores que deram feedback ou assistência na revisão ao longo do caminho, incluindo Wenfei Tong, Richard Krajunus, Zach Bogart, Gabe Fernando, Laura **Prefácio | xv**

Kennedy, Steven Finkelstein, Liana Hitts e Jason Greenberg. Ter a opinião deles foi excepcionalmente útil. Obrigado por ler e por oferecer feedback.

Agradeço à minha mãe (e modelo), Elizabeth, pelo amor, apoio e disciplina ao longo da vida. Agradeço ao meu pai, John, e à tia Claire por todo o amor e apoio à minha educação ao longo de muitos anos. Mais importante ainda, agradeço ao meu marido, Ivan, e ao meu filho, Edmund Hillary, pela paciência e entusiasmo no apoio ao livro e na desculpa pelo tempo que passei longe da família enquanto escrevia.

Quaisquer erros ou bugs que você encontrar neste livro são meus e somente meus. Agradeço comentários em *aileen.a.nielsen@gmail.com* .

xvi | Prefácio

CAPÍTULO 1

Série temporal:

Uma visão geral e um histórico rápido

Os dados de séries temporais e a sua análise são cada vez mais importantes devido à produção massiva de tais dados através, por exemplo, da Internet das coisas, da digitalização dos cuidados de saúde e da ascensão de cidades inteligentes. Nos próximos anos, podemos esperar que a quantidade, a qualidade e a importância dos dados de séries temporais cresçam rapidamente.

À medida que o monitoramento contínuo e a coleta de dados se tornam mais comuns, aumentará a necessidade de análises competentes de séries temporais com técnicas estatísticas e de aprendizado de máquina. Na verdade, os novos modelos mais promissores combinam ambas as metodologias. Por esse motivo, discutiremos

detalhadamente cada um. Estudaremos e usaremos uma ampla gama de técnicas de séries temporais úteis para analisar e prever o comportamento humano, fenômenos científicos e dados do setor privado, já que todas essas áreas oferecem ricos conjuntos de dados de séries temporais.

Vamos começar com uma definição. *A análise de série temporal* é o esforço de extrair informações estatísticas e resumidas significativas de pontos organizados em ordem cronológica. Isso é feito para diagnosticar o comportamento passado, bem como para prever o comportamento futuro. Neste livro usaremos uma variedade de abordagens, desde modelos estatísticos centenários até arquiteturas de redes neurais recentemente desenvolvidas.

Nenhuma das técnicas se desenvolveu no vácuo ou por interesse puramente teórico. As inovações na análise de séries temporais resultam de novas formas de coletar, registrar e visualizar dados. A seguir, discutiremos brevemente o surgimento da análise de séries temporais em diversas aplicações.

1

A história das séries temporais em diversas aplicações

A análise de séries temporais muitas vezes se resume à questão da causalidade: como o passado influenciou o futuro? Às vezes, essas questões (e suas respostas) são tratadas estritamente dentro de sua disciplina, e não como parte da disciplina geral de análise de séries temporais. Como resultado, uma variedade de disciplinas contribuíram com novas formas de pensar sobre conjuntos de dados de séries temporais.

Nesta seção, examinaremos alguns exemplos históricos de dados e análises de séries temporais nestas disciplinas:

- Medicamento
- Clima
- Economia

• Astronomia

Como veremos, o ritmo de desenvolvimento nestas disciplinas e as contribuições originadas em cada área estavam fortemente ligadas à natureza dos dados contemporâneos das séries temporais disponíveis.

Medicina como um problema de série temporal

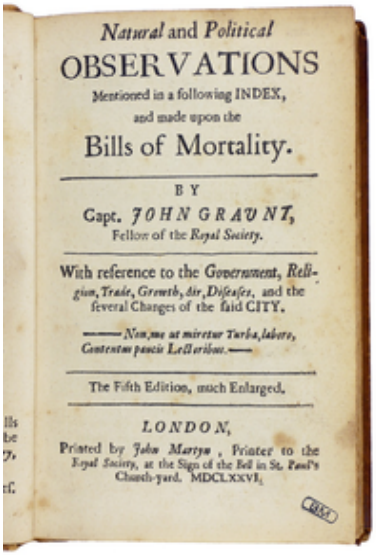
A medicina é um campo orientado por dados que tem contribuído com análises de séries temporais interessantes para o conhecimento humano há alguns séculos. Agora, vamos estudar alguns exemplos de fontes de dados de séries temporais na medicina e como elas surgiram ao longo do tempo.

A medicina começou surpreendentemente lentamente a pensar sobre a matemática da previsão do futuro, apesar do facto de os prognósticos serem uma parte essencial da prática médica.

Este foi o caso por vários motivos. A estatística e uma forma probabilística de pensar o mundo são fenómenos recentes, e estas disciplinas não estiveram disponíveis durante muitos séculos, mesmo com o desenvolvimento da prática da medicina. Além disso, a maioria dos médicos exercia a profissão de forma isolada, sem comunicação profissional fácil e sem uma infra-estrutura formal de manutenção de registos de saúde do paciente ou da população. Portanto, mesmo que os médicos de épocas anteriores tivessem sido treinados como pensadores estatísticos, provavelmente não teriam dados razoáveis dos quais pudessem tirar conclusões.

Isto não visa, de forma alguma, criticar os primeiros médicos, mas explicar por que não é muito surpreendente que uma das primeiras inovações em séries temporais na saúde da população tenha vindo de um vendedor de chapéus e não de um médico. Pensando bem, isso faz sentido: nos séculos anteriores, um vendedor de chapéus urbanos provavelmente teria mais prática na manutenção de registros e na arte de detectar tendências do que um médico.

O inovador foi John Graunt, um armarinho londrino do século XVII. Graunt realizou um estudo dos registros de óbitos mantidos nas paróquias de Londres desde



Natural and *Political*

OBSERVATIONS

Mentioned in a following INDEX,

and made upon the

Bills of Mortality.

BY

Capt. JOHN GRAUNT,

Fellow of the *Royal Society*.

With reference to the *Government, Religion, Trade, Growth, Air, Diseases*, and the several Changes of the said CITY.

——*Non me ut miretur Turba, labore,*
Contentus paucis Lectoribus.——

The Fifth Edition, much Enlarged.

LONDON,

Printed by *John Martyn*, Printer to the *Royal Society*, at the Sign of the Bell in St. *Paul's* Church-yard. MDCLXXVI.

início de 1500. Ao fazer isso, ele originou a disciplina de demografia. Em 1662, publicou *Observações Naturais e Políticas. . . Feito com base nas contas de mortalidade* (ver

Figura 1-1).

Figura 1-1. As tabelas atuariais de John Graunt foram um dos primeiros resultados do pensamento no estilo de série temporal aplicado a questões médicas. Fonte: Wikipédia.

Neste livro, Graunt apresentou as primeiras *tábuas de mortalidade* , que você pode conhecer como *atuariais tabelas* . Essas tabelas mostram a probabilidade de uma pessoa de uma determinada idade morrer antes do próximo aniversário. Graunt, como a primeira pessoa conhecida por formular e publicar tabelas de vida, foi também o primeiro estatístico documentado da saúde humana. Suas tabelas de vida pareciam com a Tabela 1-1, extraída de alguns funcionários da Universidade Rice.

notas do curso de estatística.

Tabela 1-1. Uma amostra das tabelas de vida de John Graunt

Idade

Proporção de mortes no intervalo Proporção de sobreviventes até o início do intervalo 0–6

0,36

1,0

7–16

0,24

0,64

17–26 0,15

0,40

27–36 0,09

0,25

Infelizmente, a maneira de Graunt pensar matematicamente sobre a sobrevivência humana não funcionou. Um mundo mais conectado e orientado por dados começou a se formar – completo com estados-nação, credenciamento, sociedades profissionais, revistas científicas e, muito mais tarde, manutenção de registros de saúde exigida pelo governo – mas a medicina continuou a se concentrar em

fisiologia em vez de estatísticas.

Havia razões compreensíveis para isso. Primeiro, o estudo da anatomia e da fisiologia em um pequeno número de indivíduos proporcionou os maiores avanços na medicina durante séculos, e a maioria dos humanos (até mesmo os cientistas) se dedicam ao que funciona para eles, desde que

possível. Embora o foco na fisiologia tenha sido tão bem-sucedido, não havia razão para ir mais longe. Em segundo lugar, havia muito pouca infra-estrutura de relatórios para os médicos tabularem e partilharem informações numa escala que tornaria os métodos estatísticos superiores às observações clínicas.

A análise de séries temporais tem sido ainda mais lenta a chegar à medicina convencional do que outros ramos da estatística e da análise de dados, provavelmente porque a análise de séries temporais exige mais dos sistemas de manutenção de registos. Os registos devem ser interligados ao longo do tempo e, de preferência, recolhidos em intervalos regulares. Por esta razão, as séries temporais como prática epidemiológica só surgiram muito recentemente e de forma incremental, depois de existir infra-estrutura governamental e científica suficiente para garantir registos temporais razoavelmente bons e extensos.

Da mesma forma, os cuidados de saúde individualizados utilizando análise de séries temporais continuam a ser um campo jovem e desafiante porque pode ser bastante difícil criar conjuntos de dados consistentes ao longo do tempo. Mesmo para pequenas pesquisas baseadas em estudos de caso, manter o contato e a participação de um grupo de indivíduos é terrivelmente difícil e caro. Quando esses estudos são realizados durante longos períodos de tempo, tendem a tornar-se canónicos nas suas áreas - e repetidamente, ou mesmo excessivamente pesquisados - porque os seus dados podem abordar questões importantes, apesar dos desafios de financiamento e

gerenciamento.1

Instrumentos médicos

A análise de séries temporais para pacientes individuais tem uma história muito anterior e mais bem-sucedida do que a dos estudos de saúde em nível populacional. A análise de séries temporais chegou à medicina quando os primeiros eletrocardiogramas (ECGs) práticos, que podem diagnosticar doenças cardíacas registrando os sinais elétricos que passam pelo coração, foram inventados em 1901 (ver Figura 1-2). Outra máquina de séries temporais, o eletroencefalograma (EEG), que mede de forma não

invasiva os impulsos elétricos no cérebro, foi introduzida na medicina em 1924, criando mais oportunidades para os médicos aplicarem a análise de séries temporais ao diagnóstico médico (ver Figura 1-3).

Ambas as máquinas de séries temporais faziam parte de uma tendência maior de aprimorar a medicina com ideias e tecnologias reaproveitadas surgidas do segundo período industrial.

Revolução.

1 Exemplos incluem o British Doctors Study e o Nurses' Health Study.

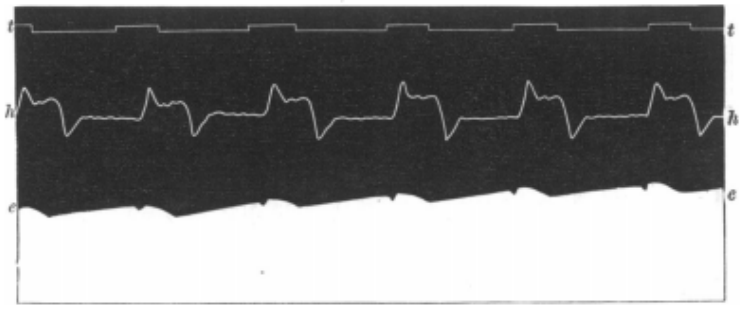


FIG. 1. Man. Heart led off to electrometer from front and back of chest (front to Hg; back to H_2SO_4).
e.e. electrometer. *h.h.* cardiograph. *t.t.* time in seconds.

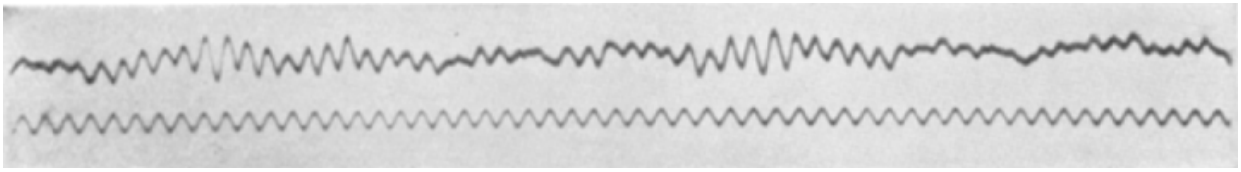


Figura 1-2. Uma gravação de ECG antiga do artigo original de 1877 de Augustus D. Wal-

ler, MD, "Uma demonstração sobre o homem de mudanças eletromotrizes que acompanham o

Batida do Coração". Os primeiros ECGs eram difíceis de construir e usar, por isso foram necessários alguns

mais décadas antes de se tornarem uma ferramenta prática para os médicos.

Figura 1-3. A primeira gravação de EEG humano, de 1924. Fonte: Wikipedia .

As ferramentas de classificação de séries temporais de ECG e EEG continuam sendo áreas ativas de pesquisa para fins muito práticos, como estimar o risco de uma crise cardíaca súbita ou de uma convulsão.

Estas medições são fontes ricas de dados, mas um "problema" com tais dados é que tendem a estar disponíveis apenas para pacientes com doenças específicas. Estas máquinas não geram séries temporais de longo alcance que nos possam informar de forma mais ampla sobre a saúde e o comportamento humano, uma vez que as suas medições raramente são aplicadas durante longos períodos de tempo ou antes de uma doença surgir num paciente.

Felizmente, do ponto de vista da análise de dados, estamos ultrapassando a era em que os ECGs e similares são as séries temporais médicas dominantes disponíveis. Com o advento de sensores vestíveis e de dispositivos médicos eletrónicos "inteligentes", muitos seres humanos saudáveis realizam medições de rotina automaticamente ou com um mínimo de intervenção manual, levando à recolha contínua de bons dados longitudinais sobre pessoas doentes e saudáveis. Isto contrasta fortemente com os dados de séries temporais médicas do século passado, que eram medidos quase exclusivamente sobre pessoas doentes e cujo acesso era muito limitado.

Como mostrou a recente cobertura jornalística, uma variedade de atores não tradicionais estão entrando no campo da medicina, desde enormes empresas de mídia social até empresas financeiras. **A História das Séries Temporais em Diversas Aplicações | 5**

instituições para gigantes do varejo. 2 Provavelmente todos planeiam utilizar grandes conjuntos de dados para agilizar os cuidados de saúde. Não existem apenas novos participantes na área da saúde – também existem novas técnicas. A medicina personalizada baseada no DNA significa que *os dados de séries temporais* são cada vez

mais medidos e valorizados. Graças aos crescentes conjuntos de dados modernos de saúde, tanto a análise de saúde como a análise de séries temporais irão provavelmente evoluir nos próximos anos, particularmente em resposta aos lucrativos conjuntos de dados do sector da saúde. Esperamos que isso aconteça de tal forma que as séries temporais possam beneficiar a todos.

Previsão do tempo

Por razões óbvias, prever o tempo sempre foi uma preocupação para muitos.

O antigo filósofo grego Aristóteles mergulhou no clima com um tratado inteiro (*Meteorologia*), e suas ideias sobre as causas e a sequência do clima permaneceram dominantes até a Renascença. Nessa altura, os cientistas começaram a recolher dados relacionados com o clima com a ajuda de instrumentos recentemente inventados, como o barómetro, para medir o estado da atmosfera. Eles usaram esses instrumentos para registrar séries temporais em intervalos diários ou mesmo de hora em hora. Os registros foram mantidos em vários locais, incluindo diários particulares e diários de bordo da cidade local. Durante séculos, esta foi a única maneira pela qual a civilização ocidental monitorava o clima.

Uma maior formalização e infraestrutura para registro meteorológico chegou na década de 1850, quando Robert FitzRoy foi nomeado chefe de um novo departamento do governo britânico.

mento para registrar e publicar dados meteorológicos para marinheiros.3 FitzRoy cunhou o termo *previsão do tempo* . Na época, ele foi criticado pela qualidade de suas previsões, mas agora é considerado muito à frente de seu tempo na ciência que utilizou para desenvolvê-las. Estabeleceu o costume de imprimir previsões meteorológicas no jornal; foram as primeiras previsões impressas no *The Times* de Londres. FitzRoy é agora celebrado como o “pai da previsão”.

No final do século XIX - centenas de anos depois de muitas medições atmosféricas terem entrado em uso - o telégrafo permitiu compilações rápidas das condições atmosféricas

em séries temporais de muitos locais diferentes. Esta prática tornou-se padrão em muitas partes do mundo na década de 1870 e levou à criação dos primeiros conjuntos de dados significativos para prever o clima local com base no que estava acontecendo em outras localizações geográficas.

2 Veja, por exemplo, Darrell Etherington, Amazon, JPMorgan e Berkshire Hathaway to Build Their Own

Empresa de saúde", TechCrunch, 30 de janeiro de 2018, https://perma.cc/S789-EQGW; Christina Farr, Face-

livro Enviou um médico em uma missão secreta para pedir aos hospitais que compartilhem dados de pacientes", CNBC, 5 de abril de 2018, https://

perma.cc/65GF-M2SJ.

3 Este mesmo Robert FitzRoy foi capitão do HMS *Beagle* durante a viagem que levou Charles Darwin ao redor do mundo. Esta viagem foi fundamental para fornecer evidências a Darwin da teoria da evolução por seleção natural.

Na virada do século XX, a ideia de prever o tempo com métodos computacionais foi vigorosamente perseguida com a ajuda desses conjuntos de dados compilados.

Os primeiros esforços para calcular o clima exigiram um esforço espetacular, mas deram resultados insatisfatórios. Embora os físicos e os químicos tivessem ideias comprovadas sobre as leis naturais relevantes, havia demasiadas leis naturais para serem aplicadas de uma só vez. O sistema de equações resultante era tão complexo que foi um avanço científico notável a primeira vez que alguém tentou fazer os cálculos.

Seguiram-se várias décadas de pesquisa para simplificar as equações físicas de uma forma que aumentasse a precisão e a eficiência computacional. Esses truques foram

transmitidos até mesmo aos atuais modelos de previsão do tempo, que operam com base em uma combinação de princípios físicos conhecidos e heurísticas comprovadas.

Hoje em dia, muitos governos fazem medições meteorológicas altamente granulares a partir de

centenas ou mesmo milhares de estações meteorológicas em todo o mundo, e essas previsões são baseadas em dados com informações precisas sobre a localização e os equipamentos das estações meteorológicas. As raízes destes esforços remontam aos conjuntos de dados coordenados da década de 1870 e, ainda mais cedo, à prática renascentista de manter diários meteorológicos locais.

Infelizmente, a previsão do tempo é um exemplo dos ataques crescentes à ciência que chegam até ao domínio da previsão de séries temporais. Não apenas os debates sobre séries temporais sobre as temperaturas globais foram politizados, mas também o tempo mais mundano

tarefas de previsão em série, como prever a trajetória de um furacão .

Previsão de crescimento econômico

Os indicadores de produção e eficiência nos mercados há muito que fornecem dados interessantes para estudar a partir de uma análise de séries temporais. O mais interessante e urgente tem sido a questão de prever estados económicos futuros com base no passado. Tais previsões não são úteis apenas para ganhar dinheiro – também ajudam a promover a prosperidade e a evitar catástrofes sociais. Vamos discutir alguns desenvolvimentos importantes na história das previsões económicas.

As previsões económicas surgiram da ansiedade desencadeada por crises bancárias episódicas nos Estados Unidos e na Europa no final do século XIX e início do século XX. Nessa altura, tanto empresários como investigadores inspiraram-se na ideia de que a economia poderia ser comparada a um sistema cíclico, tal como se pensava que o clima se comportava. Pensava-se que com as medições corretas seria possível fazer previsões e evitar acidentes.

Até a linguagem das primeiras previsões económicas reflectia a linguagem das previsões meteorológicas. Isso foi involuntariamente adequado. No início do século XX, as previsões económicas e meteorológicas eram de facto semelhantes: ambas eram bastante terríveis. Mas os economistas

aspirações criaram um ambiente no qual o progresso poderia pelo menos ser esperado. **A história das séries temporais em diversas aplicações | 7**

BUSINESS CYCLE REFERENCE DATES		DURATION IN MONTHS			
Peak	Trough	Contraction	Expansion	Cycle	
Quarterly dates are in parentheses		*Peak to Trough*	*Previous trough to this peak*	*Trough from Previous Trough*	*Peak from Previous Peak*
	December 1854 (IV)	--	--	--	--
June 1857(II)	December 1858 (IV)	18	30	48	--
October 1860(III)	June 1861 (III)	8	22	30	40
April 1865(I)	December 1867 (I)	32	46	78	54
June 1869(II)	December 1870 (IV)	18	18	36	50
October 1873(III)	March 1879 (I)	65	34	99	52
March 1882(I)	May 1885 (II)	38	36	74	101
March 1887(II)	April 1888 (I)	13	22	35	60
July 1890(III)	May 1891 (II)	10	27	37	40
January 1893(I)	June 1894 (II)	17	20	37	30
December 1895(IV)	June 1897 (II)	18	18	36	35
June 1899(III)	December 1900 (IV)	18	24	42	42
September 1902(IV)	August 1904 (III)	23	21	44	39

e assim foram formadas diversas instituições públicas e privadas para rastrear dados económicos. Os primeiros esforços de previsão económica levaram à criação de indicadores económicos e de históricos tabulados e publicamente disponíveis desses indicadores que ainda hoje são utilizados. Até usaremos alguns deles neste livro.

Hoje em dia, os Estados Unidos e a maioria das outras nações têm milhares de investigadores governamentais e detentores de registos cuja função é registar dados com a maior precisão possível.

e disponibilizá-lo ao público (ver Figura 1-4). Esta prática provou ser inestimável

capaz de contribuir para o crescimento económico e para evitar catástrofes económicas e dolorosos ciclos de expansão e recessão. Além disso, as empresas beneficiam de uma atmosfera rica em dados, uma vez que estes conjuntos de dados públicos permitem aos fornecedores de transportes, aos fabricantes, aos proprietários de pequenas empresas e até aos agricultores anteciparem as prováveis condições futuras do mercado. Tudo isto surgiu da tentativa de identificar "ciclos económicos" que se pensava serem as causas das falências bancárias cíclicas, uma forma inicial de análise de séries temporais na economia.

Figura 1-4. O governo federal dos EUA financia muitas agências governamentais e organizações sem fins lucrativos que registram estatísticas vitais, bem como formulam indicadores econômicos. Fonte:

Escritório Nacional de Pesquisa Econômica.

Muitos dos dados económicos recolhidos pelo governo, especialmente os mais dignos de notícia, tendem a ser um indicador do bem-estar económico geral da população. Um exemplo dessas informações vitais vem do número de pessoas que solicitam subsídios de desemprego. Os exemplos incluem as estimativas do governo sobre o produto interno bruto e o total das declarações fiscais recebidas num determinado ano.

Graças a este desejo de fazer previsões económicas, o governo tornou-se um curador de dados e também um cobrador de impostos. A coleta desses dados possibilitou o moderno eco-8 | Capítulo 1: Séries Temporais: Uma Visão Geral e um Histórico Rápido

a economia, a indústria financeira moderna e a ciência de dados em geral florescerão. Graças à análise de séries temporais que surge das questões económicas, evitamos agora com segurança muito mais crises bancárias e financeiras do que qualquer governo poderia ter evitado nos séculos passados.

Além disso, centenas de livros didáticos de séries temporais foram escritos na forma de livros didáticos de economia dedicados à compreensão dos ritmos desses indicadores financeiros.

Mercados de negociação

Voltemos ao lado histórico das coisas. À medida que os esforços governamentais na recolha de dados obtiveram grande sucesso, as organizações privadas começaram a copiar os registos governamentais. Com o tempo, as bolsas de mercadorias e de valores tornaram-se cada vez mais técnicas.

Os almanaques financeiros também se tornaram populares. Isto aconteceu porque os participantes no mercado se tornaram mais sofisticados e porque as tecnologias emergentes permitiram uma maior automatização e novas formas de competir e de pensar sobre os preços.

Toda esta manutenção minuciosa de registos deu origem à procura de ganhar dinheiro com os mercados através da matemática e não da intuição, de uma forma inteiramente impulsionada pelas estatísticas (e, mais recentemente, pela aprendizagem automática). Os primeiros pioneiros faziam esse trabalho matemático manualmente, enquanto os "quants" atuais fazem isso por meio de métodos analíticos de séries temporais muito complicados e proprietários.

Um dos pioneiros da *negociação mecânica* , ou previsão de séries temporais via algoritmo, foi Richard Dennis. Dennis foi um milionário que se tornou famoso por transformar pessoas comuns, chamadas de Tartarugas, em traders famosos, ensinando-lhes algumas regras selecionadas sobre como e quando negociar. Estas regras foram desenvolvidas nas décadas de 1970 e 1980 e espelhavam o pensamento de "IA" da década de 1980, em que a heurística ainda governava fortemente o paradigma de como construir máquinas inteligentes para funcionar no mundo real.

Desde então, muitos traders "mecânicos" adaptaram estas regras, o que, como resultado, se tornou menos rentável num mercado automatizado lotado. Os traders mecânicos continuam a crescer em número e riqueza e estão

continuamente em busca da próxima melhor opção porque há muita concorrência.

Astronomia

A astronomia sempre dependeu fortemente da plotagem de objetos, trajetórias e medições ao longo do tempo. Por isso, os astrônomos são mestres em séries temporais, tanto para calibrar instrumentos quanto para estudar seus objetos de interesse. Como exemplo da longa história dos dados de séries temporais, considere que as séries temporais de manchas solares foram registadas na China antiga já em 800 a.C., tornando a recolha de dados de manchas solares um dos fenómenos naturais mais bem registados de sempre.

Algumas das astronomias mais emocionantes do século passado estão relacionadas à análise de séries temporais. A descoberta de estrelas variáveis (que podem ser usadas para deduzir distâncias galácticas) e a observação de eventos transitórios como supernovas (que aprimoram nosso livro **A História das Séries Temporais em Diversas Aplicações | 9**

compreensão de como o universo muda ao longo do tempo) são o resultado do monitoramento de transmissões ao vivo de dados de séries temporais com base nos comprimentos de onda e intensidades da luz.

As séries temporais tiveram um impacto fundamental sobre o que podemos saber e medir sobre o universo.

Aliás, esta monitorização de imagens astronómicas permitiu até aos astrónomos

captar os eventos tal como eles acontecem (ou melhor, como somos capazes de observá-los, o que pode levar milhões de anos).

Nas últimas décadas, a disponibilidade de dados explicitamente marcados com data e hora, como séries temporais formais, explodiu na astronomia com uma ampla gama de novos tipos de telescópios coletando todos os tipos de dados celestes. Alguns astrônomos até se referiram a uma série temporal

“dilúvio de dados”.

A análise de série temporal decola

George Box, um estatístico pioneiro que ajudou a desenvolver um popular modelo de série temporal, foi um grande pragmático. Ele disse a famosa frase: “Todos os modelos estão errados, mas alguns são úteis”.

Box fez essa afirmação em resposta a uma atitude comum de que a modelagem adequada de séries temporais era uma questão de encontrar o melhor modelo para ajustar os dados. Conforme explicou, a ideia de que qualquer modelo possa descrever o mundo real é muito improvável. Box fez este pronunciamento em 1978, o que parece estranhamente tardio na história de um campo tão importante como a análise de séries temporais, mas na verdade a disciplina formal era surpreendentemente jovem.

Por exemplo, uma das conquistas que tornou George Box famoso, o Box-

Método Jenkins — considerado uma contribuição fundamental para a análise de séries temporais —

apareceu apenas em 1970.4 Curiosamente, este método apareceu pela primeira vez não numa revista académica, mas sim num livro de estatística, *Time Series Analysis: Forecasting and Control* (Wiley). Aliás, este livro continua popular e está agora na sua quinta edição.

O modelo Box-Jenkins original foi aplicado a um conjunto de dados de níveis de dióxido de carbono emitidos por uma fornalha a gás. Embora não haja nada de estranho em um forno a gás, o conjunto de dados de 300 pontos que foi usado para demonstrar o método parece um tanto antiquado. Certamente, conjuntos de dados maiores estavam disponíveis na década de 1970, mas lembre-se de que naquela época era excepcionalmente difícil trabalhar com eles. Esta foi uma época anterior a conveniências como R, Python e até C++. Os pesquisadores tinham bons motivos para se concentrar em pequenos conjuntos de dados e métodos que minimizassem os recursos computacionais.

A análise e a previsão de séries temporais foram desenvolvidas como os computadores, com conjuntos de dados maiores e ferramentas de codificação mais fáceis, abrindo caminho para mais experimentação e capacidade de 4 O método Box-Jenkins tornou-se uma técnica canônica para escolher os melhores parâmetros para um modelo ARMA ou ARIMA para modelar uma série temporal. Mais sobre isso no Capítulo 6 .

responda a perguntas mais interessantes. A história das previsões do professor Rob Hyndman

As competições fornecem exemplos adequados de como as competições de previsão de séries temporais se desenvolveram a uma taxa paralela à dos computadores.

O professor Hyndman situa o “estudo não trivial mais antigo sobre a precisão das previsões de séries temporais” como ocorrendo em uma dissertação de doutorado de 1969 na Universidade de Nottingham, apenas um ano antes da publicação do método Box-Jenkins. Esse primeiro esforço foi logo seguido por competições organizadas de previsão de séries temporais, as primeiras apresentando

girando em torno de 100 conjuntos de dados no início da década de 1970.5 Nada mal, mas certamente algo que

poderia ser feito manualmente se for absolutamente necessário.

No final da década de 1970, os investigadores organizaram uma competição com cerca de 1.000 conjuntos de dados, um aumento impressionante. Aliás, esta era também foi marcada pelo primeiro microprocessador comercial, pelo desenvolvimento dos disquetes, pelos primeiros computadores pessoais da Apple e pela linguagem de computador Pascal. É provável que algumas dessas inovações tenham sido úteis. Uma competição de previsão de séries temporais no final da década de 1990 incluiu 3.000 conjuntos de dados. Embora estas colecções de conjuntos de dados tenham sido substanciais e sem dúvida

reflectissem uma enorme quantidade de trabalho e engenhosidade para recolher e curar, são ofuscadas pela quantidade de dados actualmente disponíveis. Os dados de séries temporais estão em toda parte e em breve tudo será uma série temporal.

Este rápido crescimento no tamanho e na qualidade dos conjuntos de dados deve a sua origem aos enormes avanços que foram feitos na computação nas últimas décadas. Os engenheiros de hardware conseguiram dar continuidade à tendência descrita pela Lei de Moore – uma previsão de crescimento exponencial na capacidade computacional – durante esse período. À medida que o hardware se tornou mais pequeno, mais poderoso e mais eficiente, foi fácil ter muito mais dele, a um preço acessível – para criar tudo, desde computadores portáteis em miniatura com sensores ligados a enormes centros de dados que alimentam a Internet moderna na sua forma ávida por dados. Mais recentemente, wearables, técnicas de aprendizado de máquina e GPUs revolucionaram

cionou a quantidade e a qualidade dos dados disponíveis para estudo.6

As séries temporais sem dúvida se beneficiarão à medida que o poder de computação aumentar, porque muitos

aspectos dos dados de séries temporais são computacionalmente exigentes. Com aceleração

recursos computacionais e de dados, espera-se que a análise de séries temporais continue seu rápido ritmo de desenvolvimento.

5 Ou seja, 100 conjuntos de dados separados em diferentes domínios de várias séries temporais de diferentes comprimentos.

6 Dada a variedade de dispositivos que os humanos carregam consigo, bem como os carimbos de data/hora que criam quando fazem compras, iniciam sessão num portal informático no trabalho, navegam na Internet, verificam um indicador de saúde, fazem uma chamada telefónica ou navegam no trânsito. com o GPS, podemos dizer com

segurança que um americano médio provavelmente produz milhares de pontos de dados de séries temporais todos os anos de sua vida.

Análise de série temporal decola | 11

As origens da análise estatística de séries temporais

A estatística é uma ciência muito jovem. O progresso nas estatísticas, na análise de dados e nas séries cronológicas sempre dependeu fortemente de quando, onde e como os dados estavam disponíveis e em que quantidade. O surgimento da análise de séries temporais como disciplina está ligado não apenas aos desenvolvimentos na teoria das probabilidades, mas igualmente ao desenvolvimento de Estados-nação estáveis, onde a manutenção de registos se tornou pela primeira vez um objectivo realizável e interessante. Abordamos isso anteriormente com relação a uma variedade de disciplinas. Agora pensaremos nas próprias séries temporais como uma disciplina.

Uma referência para o início da análise de séries temporais como disciplina é a aplicação de modelos autorregressivos a dados reais. Isso não aconteceu até a década de 1920. Udny Yule, um físico experimental que se tornou professor de estatística na Universidade de Cambridge, aplicou um modelo autorregressivo aos dados de manchas solares, oferecendo uma nova forma de pensar sobre os dados, em contraste com métodos concebidos para ajustar a frequência de uma oscilação. Yule apontou que um modelo autorregressivo não começa com um modelo que assume periodicidade:

Quando a análise do periodograma é aplicada a dados relativos a qualquer fenômeno físico na expectativa de obter uma ou mais periodicidades verdadeiras, geralmente há, como me parece, uma tendência a partir da hipótese inicial de que a periodicidade ou periodicidades são mascaradas apenas por tais flutuações sobrepostas mais ou menos aleatórias – flutuações que não perturbam de forma alguma o curso constante da função ou funções periódicas subjacentes... parece não haver razão para assumir que esta seja a hipótese mais provável a priori.

O pensamento de Yule era o seu, mas é provável que algumas influências históricas o tenham levado a perceber que o modelo tradicional pressupunha o seu próprio resultado. Como antigo físico experimental que tinha trabalhado no estrangeiro, na Alemanha (o epicentro da florescente teoria da mecânica quântica), Yule certamente estaria ciente dos desenvolvimentos recentes que realçaram a natureza probabilística da mecânica quântica.

Ele também teria reconhecido os perigos de restringir o pensamento a um modelo que pressupõe demasiado, como fizeram os físicos clássicos antes da descoberta da mecânica quântica.

À medida que o mundo se tornou um lugar mais ordenado, registado e previsível, especialmente após a Segunda Guerra Mundial, o sector empresarial apresentou problemas iniciais na análise prática de séries temporais. Os problemas de séries temporais orientados para os negócios eram importantes e não eram excessivamente teóricos em suas origens. Estas incluíam a previsão da procura, a estimativa dos preços futuros das matérias-primas e a cobertura dos custos de produção. Nestes casos de uso industrial, as técnicas foram adotadas quando funcionaram e rejeitadas quando não funcionaram.

Provavelmente ajudou o facto de os trabalhadores industriais terem acesso a conjuntos de dados maiores do que os que estavam disponíveis aos académicos na altura (como continua a ser o caso agora). Isso significou que técnicas às vezes práticas, mas teoricamente subexploradas, passaram a ser amplamente utilizadas antes de serem bem compreendidas.

As origens da análise de série temporal de aprendizado de máquina

O aprendizado de máquina inicial na análise de séries temporais remonta a muitas décadas. Um artigo frequentemente citado de 1969, “The Combination of Forecasts”, analisou a ideia de combinar previsões em vez

de escolher a “melhor” como forma de melhorar o desempenho das previsões.

Esta ideia foi, a princípio, repugnante para os estatísticos tradicionais, mas os métodos de conjunto tornaram-se o padrão-ouro em muitos problemas de previsão. O Ensembling rejeita a ideia de um modelo de previsão perfeito ou mesmo significativamente superior em relação a todos os modelos possíveis.

Mais recentemente, usos práticos para análise de séries temporais e aprendizado de máquina surgiram já na década de 1980 e incluíam uma ampla variedade de cenários:

• Especialistas em segurança informática propuseram a detecção de anomalias como método de identificação de hackers/invasões.

• A distorção temporal dinâmica, um dos métodos dominantes para “medir” a similaridade de séries temporais, entrou em uso porque o poder computacional finalmente permitiria o cálculo razoavelmente rápido de “distâncias”, digamos, entre diferentes gravações de áudio.

• Redes neurais recursivas foram inventadas e mostraram-se úteis para extrair padrões de dados corrompidos.

A análise e a previsão de séries temporais ainda não atingiram o seu período áureo e, até à data, a análise de séries temporais continua dominada por métodos estatísticos tradicionais, bem como por técnicas mais simples de aprendizagem automática, tais como conjuntos de árvores e ajustes lineares.

Ainda estamos à espera de um grande salto para prever o futuro.

Mais recursos

• Sobre a história da análise e previsão de séries temporais:

Kenneth F. Wallis, “Revisitando a competição de previsões de Francis Galton”, Ciência Estatística 29, não. 3 (2014): 420–24, https://perma.cc/FJ6V-8HUY.

Esta é uma discussão histórica e estatística de um artigo muito antigo sobre a previsão do peso de um boi abatido enquanto o animal ainda estava vivo em uma feira municipal.

G. Udny Yule, "Sobre um método de investigação de periodicidades em séries perturbadas,

com referência especial aos números de manchas solares de Wolfer", Transações Filosóficas **As origens da análise de séries temporais de aprendizado de máquina | 13**

da Royal Society de Londres. Série A, contendo artigos de matemática ou Caráter Físico 226 (1927): 267–98, https://perma.cc/D6SL-7UZS.

O artigo seminal de Udny Yule, uma das primeiras aplicações de autoregressão

a análise da média móvel para dados reais ilustra uma maneira de remover a suposição de periodicidade da análise de um fenômeno supostamente periódico.

JM Bates e CWJ Granger, "A Combinação de Previsões", Organizacional

Research Quarterly 20, No. 4 (1969): 451–68, https://perma.cc/9AEE-QZ2J.

Este artigo seminal descreve o uso de agrupamento para previsão de séries temporais.

ing. A ideia de que calcular a média dos modelos era melhor para fazer previsões do que olhar

para um modelo perfeito era novo e controverso para muitos

estatísticos.

Jan De Gooijer e Rob Hyndman, "25 anos de previsão de séries temporais", Jornal Internacional de Previsão 22, não. 3 (2006): 443–73, https://perma.cc/

84RG-58BU.

Este é um resumo estatístico completo da pesquisa de previsão de séries temporais no século XX.

Rob Hyndman, "Uma Breve História das Competições de Previsão de Séries Temporais,"

Blog da Hyndsight, 11 de abril de 2018, https://perma.cc/32LJ-RFJW.

Este histórico mais curto e específico fornece números, locais e autores específicos de competições proeminentes de previsão de séries temporais nos últimos 50 anos.

anos.

• Sobre históricos e comentários de séries temporais de domínios específicos:

NASA, "Previsão do tempo através dos tempos", Nasa.gov, 22 de fevereiro de 2002, https://perma.cc/8GK5-JAVT.

NASA conta a história de como surgiu a previsão do tempo, com ênfase

sobre desafios e sucessos específicos da pesquisa no século XX.

Richard C. Cornes, "Dados meteorológicos iniciais de Londres e Paris: Estendendo

a Série de Oscilação do Atlântico Norte", dissertação de doutorado, Escola de Ciências Ambientais,

Universidade de East Anglia, Norwich, Reino Unido, maio de 2010, https://perma.cc/NJ33-

WVXH.

Esta tese de doutorado oferece um relato fascinante dos tipos de clima

informação disponível para duas das cidades mais importantes da Europa, completa

com listagens extensas dos locais e da natureza do clima histórico em formato de série temporal.

14 | Capítulo 1: Séries Temporais: Uma Visão Geral e um Histórico Rápido

Dan Mayer, "Uma Breve História da Medicina e Estatística", em Essential Evidence-

Medicina Baseada (Cambridge, Reino Unido: Cambridge University Press, 2004), https://

perma.cc/WKU3-9SUX.

Este capítulo do livro de Mayer destaca como a relação entre a mídia

O cinema e as estatísticas dependiam muito de fatores sociais e políticos que disponibilizavam dados e treinamento estatístico para os médicos.

Simon Vaughan, "Séries Temporais Aleatórias em Astronomia", Transações Filosóficas

da Royal Society A: Ciências Matemáticas, Físicas e de Engenharia 371, no.

1984 (2013): 1–28, https://perma.cc/J3VS-6JYB.

Vaughan resume as muitas maneiras pelas quais a análise de séries temporais é relevante para

astronomia e alerta sobre o perigo de os astrônomos redescobrirem o tempo

princípios da série ou perdendo colaborações extremamente promissoras com

estatísticos.

CAPÍTULO 2

Encontrar e organizar dados de série temporal

Neste capítulo discutimos problemas que podem surgir enquanto você está pré-processando dados de séries temporais. Alguns destes problemas serão familiares aos analistas de dados experientes, mas existem dificuldades específicas colocadas pelos carimbos de data/hora. Como acontece com qualquer tarefa de análise de dados, a limpeza e o processamento adequado dos dados costumam ser a etapa mais importante de um pipeline de carimbo de data/hora. Técnicas sofisticadas não conseguem corrigir dados confusos.

A maioria dos analistas de dados precisará encontrar, alinhar, limpar e suavizar seus próprios dados para aprender a análise de séries temporais ou para realizar um trabalho significativo em suas organizações. Ao preparar os dados, você precisará realizar diversas tarefas, desde unir colunas diferentes até reamostrar dados irregulares ou ausentes e alinhar séries temporais com diferentes eixos temporais.

Este capítulo ajuda você no caminho para um conjunto de dados de série temporal interessante e devidamente preparado.

Discutimos as seguintes habilidades úteis para localizar e limpar dados de séries temporais:

• Encontrar dados de séries temporais de repositórios on-line

• Descobrir e preparar dados de séries temporais de fontes não originalmente destinadas a séries temporais

• Resolver enigmas comuns que você encontrará com dados de séries temporais, especialmente as dificuldades que surgem com carimbos de data/hora

Depois de ler este capítulo, você terá as habilidades necessárias para identificar e preparar fontes interessantes de dados de séries temporais para análise posterior.

17

Onde encontrar dados de série temporal

Se você estiver interessado em saber onde encontrar dados de séries temporais e como limpá-los, o melhor recurso para você neste capítulo depende de qual destes é seu objetivo principal:

• Encontrar um conjunto de dados apropriado para fins de aprendizagem ou experimentação

• Criar um conjunto de dados de série temporal a partir de dados existentes que não estão armazenados em um formato explicitamente orientado ao tempo

No primeiro caso, você deve encontrar conjuntos de dados existentes com benchmarks conhecidos para poder ver se está fazendo sua análise corretamente. Eles são mais frequentemente encontrados como conjuntos de dados de concurso (como Kaggle) ou conjuntos de dados de repositório. Nesses casos, você provavelmente precisará limpar seus dados para uma finalidade específica, mesmo que algum trabalho preliminar tenha sido feito para você.

No segundo caso, você deve pensar em maneiras eficazes de identificar dados interessantes com carimbo de data e hora, transformá-los em uma série, limpá-los e alinhá-los com outros dados com carimbo de data e hora para criar dados interessantes de série temporal. Vou me referir a eles encontrados nos conjuntos de dados selvagens como *séries temporais encontradas* (este é meu próprio termo e não uma linguagem técnica).

Discutiremos a seguir os conjuntos de dados preparados e as séries temporais encontradas.

Conjuntos de dados preparados

A melhor maneira de aprender uma técnica analítica ou de modelagem é examiná-la em uma variedade de conjuntos de dados e ver como aplicá-la e se ela ajuda você a alcançar um objetivo concreto. Nesses casos, é útil ter opções preparadas.

Embora os dados de séries temporais estejam em toda parte, nem sempre é fácil encontrar o tipo de dados que você deseja, quando você deseja. Se você se encontra frequentemente nesta posição, você vai querer se familiarizar com alguns repositórios de dados de séries temporais comumente usados, por isso discutiremos algumas opções que você deve considerar.

O repositório de aprendizado de máquina UCI

O repositório de aprendizado de máquina da UCI (veja a Figura 2-1) contém cerca de 80 conjuntos de dados de séries temporais, que vão desde amostras horárias de qualidade do ar em uma cidade italiana até registros de acesso a arquivos da Amazon e registros de atividades, alimentação e informações de glicose no sangue de pacientes com diabetes. Esses são tipos de dados muito diferentes, e uma olhada nos arquivos mostra que eles refletem formas distintas de rastrear informações ao longo do tempo, embora cada um seja uma série temporal.



Figura 2-1. O repositório de aprendizado de máquina UCI inclui uma lista anotada de tempo conjuntos de dados em série.

Considere o primeiro conjunto de dados listado na seção Série Temporal no repositório UCI

histórico, que é um conjunto de dados sobre o absentismo no trabalho (ver Figura 2-2).

Uma rápida olhada nos dados revela que as colunas de tempo estão limitadas a “Mês de ausência”, “Dia da semana” e “Estações do ano”, sem coluna para ano. Existem índices de tempo duplicados, mas também existe uma coluna que indica a identidade do funcionário para que possamos diferenciar esses pontos de tempo duplicados. Finalmente, existem várias colunas de atributos de funcionários.

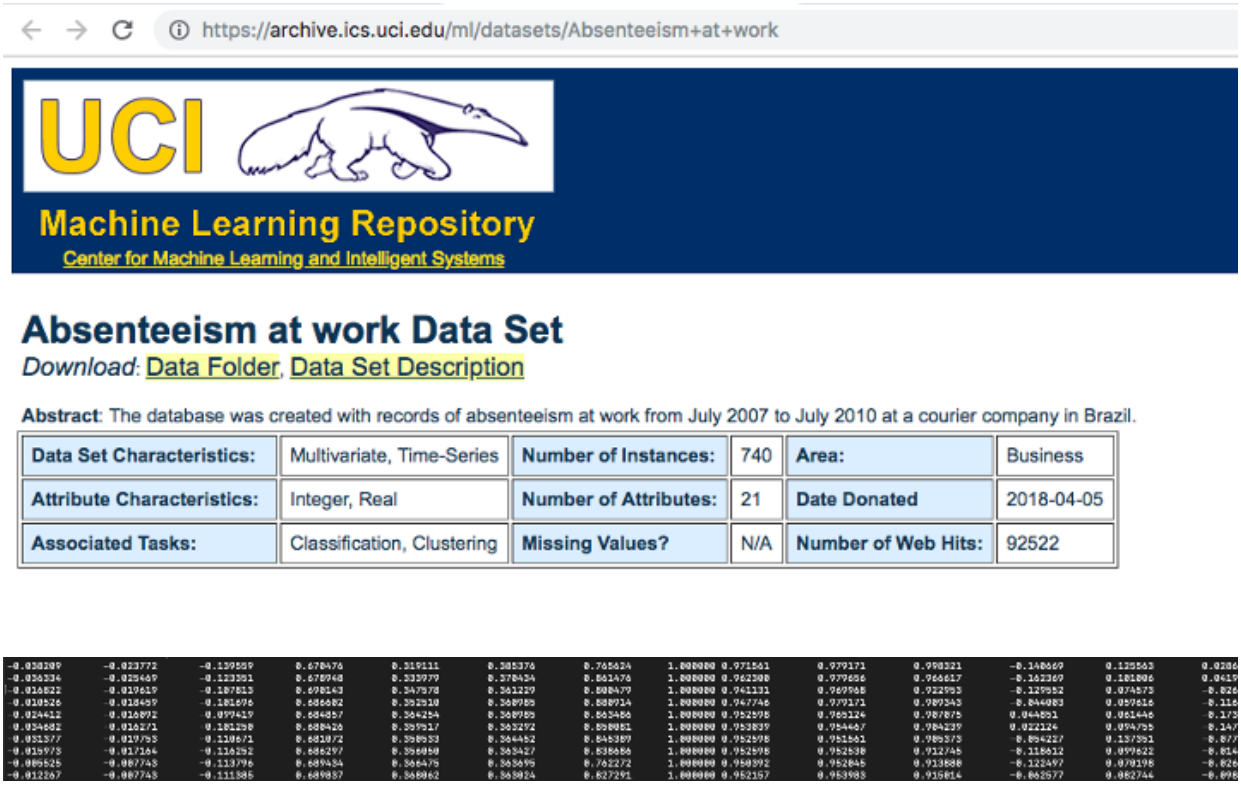


Figura 2-2. O conjunto de dados de absenteísmo no trabalho é o primeiro na lista de dados de séries temporais conjuntos no repositório de aprendizado de máquina UCI.

Este conjunto de dados pode ser bastante difícil de processar, porque primeiro seria necessário determinar se os dados eram todos de um ano ou se o ciclo dos meses de 1 a 12 várias vezes através das progressões de linha indica que o ano está a mudar. Você também precisaria decidir se analisaria o problema de forma agregada, a partir de uma perspectiva de absenteísmo total por unidade de tempo, ou analisaria o absenteísmo.

ism por ID para aqueles relatados no conjunto de dados (veja a Figura 2-3). No primeiro caso, você teria uma única série temporal, enquanto no último caso, você teria várias séries temporais com carimbos de data/hora sobrepostos. A maneira como você analisou os dados dependeria da pergunta que você deseja responder.

Figura 2-3. As primeiras linhas de um arquivo no conjunto de dados da linguagem de sinais australiana. Como você pode ver, este é um formato de arquivo amplo.

Compare o conjunto de dados de absenteísmo com outro conjunto de dados no início da lista, o Aus-

conjunto de dados de sinais de linguagem de sinais tralian, que inclui gravações de um Nintendo

PowerGlove como assuntos assinados em linguagem de sinais australiana. Os dados são formatados como arquivos CSV largos, cada um dentro de uma pasta indicando o indivíduo medido e com um nome de arquivo indicando o sinal.

Neste conjunto de dados, as colunas não são rotuladas e não possuem carimbo de data/hora. No entanto, estes são dados de séries temporais; o eixo do tempo conta os passos de tempo adiante, independentemente de quando os eventos reais aconteceram. Observe que, para pensar nos sinais



como séries temporais, não importa qual seja a unidade de tempo; o ponto é a sequência e não o tempo exato. Nesse caso, tudo o que importa é a ordem do evento e se você poderia assumir ou confirmar, pela leitura da descrição dos dados, que as medições foram feitas em intervalos regulares.

Como podemos ver ao inspecionar esses dois conjuntos de dados, você enfrentará todos os tipos de desafios de manipulação de dados. Alguns dos problemas que já notamos são:

- Carimbos de data e hora incompletos

• Os eixos de tempo podem ser horizontais ou verticais em seus dados

• Diferentes noções de tempo

O Repositório de Classificação de Séries Temporais UEA e UCR

O Repositório de Classificação de Séries Temporais UEA e UCR é um esforço mais recente que fornece um conjunto comum de dados de séries temporais disponíveis para experimentação e pesquisa em tarefas de classificação de séries temporais. Ele também mostra um conjunto muito diversificado de dados. Podemos ver isso observando dois conjuntos de dados.

Um conjunto de dados é uma tarefa de classificação de movimentos de ioga. A tarefa de classificação envolve distinguir entre dois atores individuais que realizaram uma série de transições entre poses de ioga enquanto as imagens eram gravadas. As imagens foram convertidas para uma série unidimensional. Os dados são armazenados em um arquivo CSV, com o rótulo na coluna mais à esquerda e as colunas restantes representando intervalos de tempo. O tempo passa da esquerda para a direita nas colunas, em vez de de cima para baixo nas linhas.

Gráficos de duas séries temporais de amostra por gênero são mostrados na Figura 2-4.

Séries temporais univariadas versus multivariadas

Os conjuntos de dados que examinamos até agora são séries temporais *univariadas ;*

isto é, eles têm apenas uma variável medida em relação ao tempo.

Séries temporais *multivariadas são séries com múltiplas variáveis medidas em cada carimbo de data/hora.* Eles são particularmente ricos para análise porque

muitas vezes as variáveis medidas estão inter-relacionadas e mostram

dependências entre si. Encontraremos multivariações

comeu dados de séries temporais mais tarde.

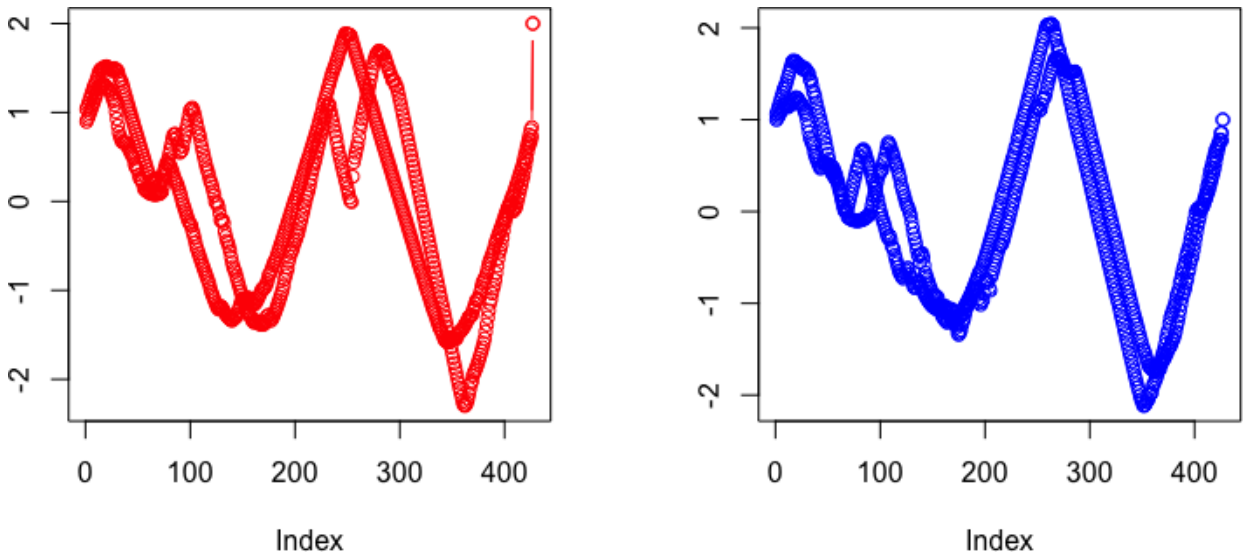


Figura 2-4. Tramas de um ator e uma atriz realizando movimentos de ioga repetidamente. Nós traçar duas séries temporais de amostra por ator. Não há rótulos de tempo explícitos no eixo x.

Em vez da unidade de tempo, o que importa é se os pontos de dados do eixo x são espaçados uniformemente, como são apresentados aqui.

Considere também o conjunto de dados de vinhos , no qual os vinhos foram classificados por região de acordo com as formas dos seus espectros. Então, o que torna isso relevante para a análise de séries temporais? Um *espectro* é um gráfico do comprimento de onda da luz versus intensidade. Aqui vemos uma tarefa de classificação de série temporal onde não há passagem de tempo. A análise de séries temporais aplica-se, contudo, porque existe uma ordenação única e significativa do eixo x, com um significado concreto de distância ao longo desse eixo. A análise de série temporal distingue-se da análise transversal por usar as informações adicionais fornecidas pela ordenação no eixo x, seja tempo, comprimento de onda ou qualquer outra coisa. Podemos ver um

gráfico dessa série “temporal” na Figura 2-5. Não há elemento temporal, mas estamos

ainda assim, olhando para uma série ordenada de dados, então as ideias usuais de séries temporais

aplicar.1

1 Para saber mais sobre esse tipo de análise de dados, veja também as referências em

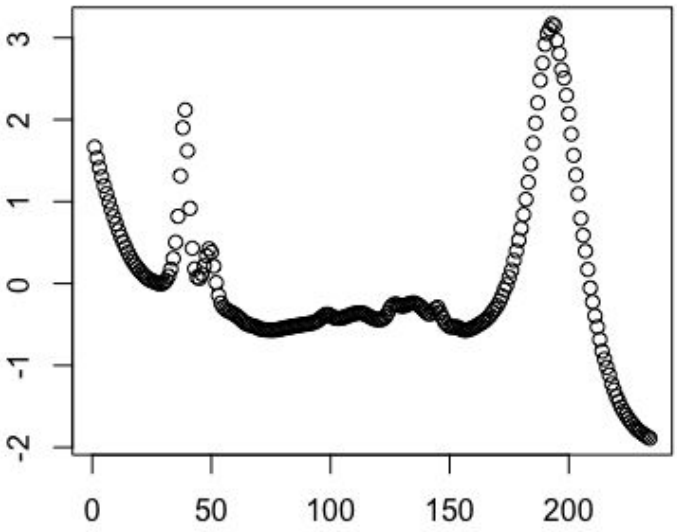


Figura 2-5. Um espectro de amostra do vinho amostrado no conjunto de dados de vinhos da UCI. Picos em a curva indica regiões de comprimento de onda que apresentam taxas de absorção particularmente altas.

Os comprimentos de onda são uniformemente espaçados ao longo do eixo x, enquanto o eixo y indica o taxa de absorção, também em escala linear. Podemos usar análise de série temporal para comparar curvas, como a acima, entre si.

Conjuntos de dados de série temporal do governo

O governo dos EUA tem sido um fornecedor confiável de dados de séries temporais há décadas, ou mesmo séculos.

Por exemplo, os Centros Nacionais de Informação Ambiental da NOAA

A informação publica uma variedade de dados de séries temporais relacionados a temperaturas e precipitação em granularidades tão finas quanto a cada 15 minutos para todas as estações meteorológicas em todo o país. O Bureau of Labor Statistics publica um índice mensal da taxa de desemprego nacional. Os Centros de Controle e Prevenção de Doenças publicam

contagens semanais de casos de gripe durante a temporada de gripe. O Federal Reserve Bank de St.

oferece um conjunto particularmente generoso e útil de dados de séries temporais econômicas.

Para incursões iniciais na análise de séries temporais, recomendo que você acesse esses conjuntos de dados governamentais do mundo real apenas para análise exploratória e visualização. Pode ser difícil aprender com esses conjuntos de dados porque eles apresentam problemas extremamente complicados. Por exemplo, muitos economistas passam toda a sua carreira a tentar prever a taxa de desemprego antes da sua publicação oficial, com sucesso apenas limitado.

Para os problemas importantes, mas intratáveis, enfrentados pelos governos, prever o futuro não seria apenas socialmente benéfico, mas também altamente remunerador. Muitas pessoas inteligentes e bem treinadas estão atacando estes problemas, embora o estado da arte continue um tanto decepcionante. É ótimo trabalhar em problemas difíceis, mas não é uma boa ideia aprender sobre esses problemas.

Onde encontrar dados de série temporal | 23

Séries temporais encontradas em dados governamentais

Não é difícil encontrar uma variedade de conjuntos de dados oportunos e promissores em sites governamentais, como os que acabamos de mencionar. No entanto, também

há muito potencial para séries temporais encontradas em dados governamentais. Por exemplo, imagine estabelecer séries cronológicas paralelas para a economia e o clima, ou correlacionar diferentes crimes com os padrões de gastos do governo. Esses dados viriam de diversas fontes e você precisaria integrá-los.

Você deve ter cuidado ao trabalhar com séries temporais encontradas em conjuntos de dados governamentais por vários motivos. As convenções de gravação, os nomes das colunas ou mesmo as definições das colunas mudam com o tempo sem a documentação que os acompanha. Os projetos começam e param dependendo da política, dos orçamentos e de outras considerações exógenas. Além disso, os formatos de dados dos websites governamentais podem ser muito confusos e são geralmente mais confusos e menos contínuos do que conjuntos de dados comparáveis no setor privado. Isso torna particularmente desafiador construir séries temporais encontradas.

Fontes úteis adicionais

Embora não possamos cobrir extensivamente todas as boas fontes de dados de séries temporais, existem vários outros repositórios que você deve explorar:

CompEngine

Este “banco de dados auto-organizado de dados de séries temporais” tem mais de 25.000 bancos de dados de séries temporais que totalizam quase 140 milhões de pontos de dados individuais. A ênfase deste repositório, e do software associado que ele oferece em sua interface web, é facilitar e promover *análises de séries temporais altamente comparativas* (hctsa). O objetivo dessa análise é gerar insights de alto nível e compreensão de quantos tipos de comportamento temporal podem ser compreendidos sem dados específicos da disciplina.

Mcomp e *M4comp2018* *Pacotes R* Esses pacotes R fornecem os dados da competição M de 1982

(1.001 séries temporais), a competição M3 em 2000 (3.003 séries temporais) e a competição M4

competição em 2018 (100.000 séries históricas). Essas competições de previsão de séries temporais foram discutidas anteriormente na menção do Capítulo 1 da história da previsão de séries temporais do professor Rob Hyndman. Previsão adicional de série temporal

os dados da competição estão incluídos no pacote *tscompdata do R.* Finalmente, conjuntos de dados de séries temporais mais especializados também podem ser encontrados em uma variedade de pacotes descritos no

Listagem do repositório CRAN de pacotes de série temporal em "Dados de série temporal"

cabeçalho.

Série temporal encontrada

No início do capítulo, discutimos o conceito de série temporal encontrada, que são dados de série temporal que reunimos a partir de fontes de dados existentes. Mais especificamente, essas séries temporais seriam reunidas a partir de pontos de dados individuais registados sem quaisquer subsídios especiais para análise de séries temporais, mas com informação suficiente para construir uma série temporal. Reunir uma série temporal de transações para um cliente específico a partir de um banco de dados SQL que armazena as transações de uma empresa é um exemplo claro.

Nesse caso, a série temporal poderia ser construída desde que um carimbo de data/hora, ou algum proxy para um carimbo de data/hora, fosse salvo no banco de dados. 2 Também podemos imaginar outras séries temporais construídas a partir dos mesmos dados, como uma série temporal do volume total de transações por dia para a empresa ou do volume total em dólares para clientes do sexo feminino por semana.

Podemos até imaginar a geração de dados de séries temporais multivariadas, como uma série temporal que

indicaria separadamente o volume total de todos os clientes com menos de 18 anos por semana, o total de dólares gastos por mulheres com mais de 65 anos por semana e o gasto total com publicidade da empresa por semana. Isto nos daria três indicadores em cada intervalo de tempo, uma série temporal multivariada.

Encontrar dados de série temporal em dados estruturados não armazenados explicitamente como uma série temporal pode ser fácil, no sentido de que o carimbo de data/hora é onipresente. Aqui estão alguns exemplos de onde você verá carimbos de data/hora em seu banco de dados:

Gravações de eventos com carimbo de data/hora

Se houver um carimbo de data/hora em seus dados, você terá potencial para construir uma série temporal. Mesmo que tudo o que você faça seja registrar a hora em que um arquivo foi acessado sem nenhuma outra informação, você terá uma série temporal. Por exemplo, nesse caso, você poderia modelar o tempo delta entre carimbos de data/hora, com cada um marcado de acordo com seu carimbo de data/hora posterior, de modo que sua série temporal consistisse em tempo no eixo temporal e tempo delta no eixo de valor. Você poderia ir além, agregando esses tempos delta como médias ou totais em períodos maiores, ou poderia mantê-los individualmente

gravado.

Medições “atemporais” onde outra medição substitui o tempo

Em alguns casos, o tempo não está explícito nos dados, mas é contabilizado na lógica subjacente do conjunto de dados. Por exemplo, você pode pensar em seus dados como “distância versus valor” quando a distância é causada por um parâmetro experimental conhecido,

como retrair um sensor de uma posição a uma taxa conhecida. Se você puder mapear uma de suas variáveis no tempo, você terá uma série temporal. Como alternativa, se um de seus eixos tiver uma distância conhecida e uma relação de ordenação (como comprimento de onda), você

também estará analisando dados de séries temporais, como o caso dos espectros de vinho mencionados anteriormente.

2 Bancos de dados SQL são bancos de dados tradicionais que usam um mecanismo de armazenamento de dados baseado em tabela. Por exemplo, se você quiser armazenar transações de clientes em um banco de dados SQL, poderá ter uma tabela com informações do cliente, incluindo um identificador exclusivo, e outra tabela com transações, cada uma incluindo um desses identificadores exclusivos de cliente.

Onde encontrar dados de série temporal | 25

Vestígios físicos

Muitas disciplinas científicas registram traços físicos, seja na medicina, na audiologia ou no clima. Costumavam ser traços gerados fisicamente e coletados por meio de processos analógicos, mas hoje em dia são armazenados em formato digital. Também são séries temporais, embora possam ser armazenadas em um formato que não torna isso óbvio, como em um arquivo de imagem ou em um único vetor dentro de um único campo de um banco de dados.

Adaptando uma coleta de dados de série temporal de um

Coleção de Tabelas

O exemplo quintessencial de uma série temporal encontrada é aquele extraído de dados de tipo de estado e de evento armazenados em um banco de dados SQL. Este também é o exemplo mais relevante porque muitos dados continuam a ser armazenados em bancos de dados SQL estruturados tradicionais.

Imagine trabalhar para uma grande organização sem fins lucrativos. Você tem acompanhado uma variedade de fatores que podem ser úteis para análise de série temporal:

• A reação de um destinatário de e-mail aos e-mails ao longo do tempo: eles abriram os e-mails ou não?

• Um histórico de associação: Houve períodos em que um membro deixou sua associação expirar?

• Histórico de transações: Quando um indivíduo compra e podemos prever isso?

Você pode observar os dados com várias técnicas de série temporal:

• Você pode gerar um histograma 2D das respostas dos membros aos e-mails ao longo do tempo com um cronograma específico do membro para ter uma ideia se os membros desenvolvem fadiga com os e-mails. (Ilustraremos o uso de histogramas 2D para entender séries temporais no Capítulo 3.)

• Você pode transformar as previsões de doações em um problema de previsão de série temporal. (Bem

discuta a previsão estatística clássica no Capítulo 4.)

• Você poderia examinar se existem padrões típicos de trajetórias para o comportamento dos membros em situações importantes. Por exemplo, existe um padrão típico de eventos que indica quando um membro está prestes a deixar sua organização (talvez três exclusões de e-mail seguidas)? Na análise de série temporal, você poderia enquadrar isso como a detecção do estado subjacente de um membro com base em ações externas. (Abordaremos isso quando discutirmos métodos de espaço de estados para análise de séries temporais no Capítulo 7.) Como podemos ver, há muitas perguntas e respostas sobre séries temporais em um SQL simples.

base de dados. Em muitos casos, as organizações não planejam a análise de séries temporais quando estão projetando seu esquema de banco de dados. Nesses exemplos, precisamos coletar e montar séries temporais de tabelas e fontes distintas.

Um exemplo prático: montando uma coleção de dados de série temporal Se você tiver a sorte de ter diversas fontes de dados relacionadas disponíveis, precisará alinhá-las, possivelmente lidando com convenções de carimbo de data/hora diferentes ou diferentes níveis de granularidade nos dados. Vamos criar alguns números para o exemplo da organização sem fins lucrativos que estávamos usando. Suponha que você tenha os dados mostrados na Tabela 2-1 através

Tabela 2-3:

Tabela 2-1. O ano em que cada membro ingressou e o status atual da associação

ID do membro

Ano de adesão

Status do membro

1

2017

ouro

2

2018

prata

3

2016

inativo

Tabela 2-2. Número de e-mails que você enviou em uma determinada semana e que foram abertos pelo membro **ID do membro**

Semana

E-mails abertos

2

08/01/2017

3

2

15/01/2017

2

1

15/01/2017

1

Tabela 2-3. Tempo que um membro doou para sua organização

Carimbo de data e hora do ID do membro

Valor da doação

2

22/05/2017 11:27:49

1.000

2

13/04/2017 09:19:02

350

1

01/01/2018 00:15:45

25

Você provavelmente já trabalhou com dados neste formato tabular. Com esses dados, você pode responder a muitas perguntas, como a correlação entre o número total de e-mails abertos por um membro e as doações gerais.

Você também pode responder a perguntas específicas, como se um membro doa logo após ingressar ou muito depois. Sem colocar esses dados em um formato mais amigável para séries temporais, no entanto, você não pode obter comportamentos mais granulares que possam ajudá-lo a prever quando alguém provavelmente fará uma doação (digamos, com base no fato de ter aberto e-mails recentemente).

Você precisa colocar esses dados em um formato adequado para análise de série temporal. Há uma série de desafios que você precisará enfrentar.

Você deve começar considerando os eixos temporais dos dados que temos. Nas tabelas anteriores temos três níveis de resolução temporal:

• Um status de membro anual

Adaptando uma coleção de dados de série temporal a partir de uma coleção de tabelas | 27

• Uma contagem semanal de e-mails abertos

• Carimbos de data e hora instantâneos de doações

Você também precisará examinar se os dados significam o que você acha que significam. Por exemplo, você desejaria determinar se o status do membro é anual ou apenas o status mais recente. Uma maneira de responder a isso é verificar se algum membro possui mais de uma entrada:

Pitão

>>> AnoJoined.groupby('memberId').count().

groupby('memberStats').count()

1000

Aqui podemos ver que todos os 1.000 membros têm apenas um status, de modo que o ano em que ingressaram provavelmente será o YearJoined, acompanhado de um status que pode ser o status atual do membro ou o status quando ele ingressou. Essa distinção afeta como você usaria a variável de status; portanto, se você fosse analisar mais detalhadamente esses dados, gostaria de esclarecê-los com alguém que conheça o pipeline de dados. Se você estivesse aplicando o status atual de um membro a uma análise de dados passados, isso seria uma *previsão* porque você estaria inserindo algo em um modelo de série temporal que não poderia ser conhecido naquele momento. É por isso que você não gostaria de usar uma variável de status, como YearJoined, sem saber quando ela foi atribuída.

O que é uma antecipação?

O termo *lookahead* é usado na análise de séries temporais para denotar qualquer conhecimento do futuro. Você não deveria ter esse conhecimento ao projetar, treinar ou avaliar um modelo. Uma antecipação é uma forma, por meio de dados, de descobrir algo sobre o futuro antes do que você deveria saber.

Uma antecipação é qualquer maneira pela qual as informações sobre o que acontecerá no futuro possam se propagar de volta no tempo em sua modelagem e afetar o comportamento de seu modelo anteriormente. Por exemplo, ao escolher hiperparâmetros para um modelo, você pode testar o modelo em vários momentos do seu conjunto de dados, depois escolher o melhor modelo e começar no início dos seus dados para testar esse modelo. Isso é problemático porque você escolheu o modelo para um momento sabendo o que aconteceria no momento seguinte – uma visão antecipada.

Infelizmente, não existe um código automatizado ou teste estatístico para uma previsão, portanto, é algo sobre o qual você deve estar vigilante e cuidadoso.

Observando a tabela de e-mails, tanto o nome da coluna semana quanto seu conteúdo sugerem que os dados são um carimbo de data/hora ou período de tempo semanal. Este deve ser um agregado ao longo dos

semana, portanto, devemos pensar nesses carimbos de data e hora como períodos semanais, em vez de carimbos de data e hora que ocorrem com uma semana de intervalo.

Você deve avaliar algumas características importantes. Por exemplo, você pode começar perguntando como as semanas são informadas no tempo. Embora possamos não ter informações para reestruturar a tabela, se a semana estiver dividida de forma um tanto estranha em relação ao nosso setor, talvez queiramos saber sobre isso também. Para analisar as atividades humanas, geralmente faz sentido olhar para a semana do calendário de domingo a sábado ou de segunda a domingo, em vez de semanas que estão menos alinhadas com o ciclo da atividade humana. Então, por exemplo, não comece sua semana arbitrariamente em 1º de janeiro.

Você também pode perguntar se semanas nulas são informadas? Ou seja, as semanas em que o membro abriu 0 emails têm lugar na tabela? Isso é importante quando queremos fazer modelagem orientada ao tempo. Nesses casos, precisamos sempre ter semanas nulas presentes nos dados porque uma semana 0 ainda é um ponto de dados.

Pitão

>>> e-mails[emails.EmailsOpened <1]

DataFrame vazio

Colunas: [EmailsOpened, membro, semana]

Índice: []

Existem duas possibilidades: ou os nulos não são relatados ou os membros sempre têm pelo menos um evento de email. Qualquer pessoa que já trabalhou com dados de e-mail sabe que é difícil fazer com que as pessoas abram e-mails, então

a hipótese de que os membros sempre abram pelo menos um e-mail por semana é bastante improvável. Nesse caso, podemos resolver isso olhando o histórico de apenas um usuário:

Pitão

>>> e-mails[emails.membro == 998]

EmailsAbertura da semana para membros

25464 1 998 2017-12-04

25465 3 998 11/12/2017

25466 3 998 2017-12-18

25467 3 998 01/01/2018

25468 3 998 2018-01-08

25469 2 998 15/01/2018

25470 3 998 2018-01-22

25471 2 998 29/01/2018

25472 3 998 2018-02-05

25473 3 998 12/02/2018

25474 3 998 19/02/2018

25475 2 998 26/02/2018

25476 2 998 2018-03-05

Podemos ver que faltam algumas semanas. Não há eventos de e-mail de dezembro de 2017 após 18 de dezembro de 2017.

Podemos verificar isso de forma mais matemática calculando quantas observações semanais devemos ter entre o primeiro e o último evento para aquele membro. Primeiro calculamos a duração do mandato do membro, em semanas:

Pitão

>>> (max(emails[emails.membro == 998].semana) -

min(emails[emails.membro == 998].semana)).dias/7

25,0

Então vemos quantas semanas de dados temos para aquele membro:

Pitão

>>> emails[emails.membro == 998].forma

(24, 3)

Temos 24 linhas aqui, mas deveríamos ter 26. Isso mostra que faltam algumas semanas de dados para este membro. Aliás, também poderíamos executar esse cálculo em todos os membros simultaneamente com operações de agrupamento, mas é mais acessível pensar em apenas um membro para fins de exemplo.

Por que 26 linhas?

Você pode se surpreender ao saber que precisamos de 26 em vez de 25, dada a subtração que acabamos de realizar, mas esse foi um cálculo incompleto. Ao trabalhar com dados de séries temporais, uma coisa que você deve sempre se perguntar depois de fazer esse tipo de subtração é se deve adicionar 1 para compensar o deslocamento no final. Em outras palavras, você subtraiu as posições que queria contar?

Considere este exemplo. Digamos que eu tenha informações para os dias 7, 14, 21 e 28 de abril. Quero saber quantos pontos de dados devo ter no total. Subtrair 7 de 28 e dividir

por 7 resulta em 21/7 ou 3. No entanto, obviamente deveria ter quatro pontos de dados. Subtraí o dia 7 de abril e preciso colocá-lo de volta, então o cálculo adequado é a diferença entre o primeiro e o último dia dividida por 7, *mais 1* para contabilizar a data de início subtraída.

Passaremos a preencher os espaços em branco para que tenhamos um conjunto de dados completo, agora que confirmamos que realmente temos semanas faltantes. Não podemos ter a certeza de identificar todas as semanas em falta, uma vez que algumas podem ter ocorrido antes da nossa primeira data registada ou depois da nossa última data registada. O que podemos fazer, entretanto, é preencher os valores faltantes entre a primeira e a última vez que um membro teve um evento não nulo.

É muito mais fácil preencher todas as semanas faltantes para todos os membros explorando o Pandas'

funcionalidade de indexação, em vez de escrever nossa própria solução. Podemos gerar um MultiIndex para um quadro de dados Pandas, que criará todas as combinações de semanas e membros – ou seja, um produto cartesiano:

Pitão

```
>>> complete_idx =
pd.MultiIndex.from_product((set(emails.semana),

set(e-mails.membro)))
```

Usamos esse índice para reindexar a tabela original e preencher os valores ausentes — neste caso, com 0, presumindo que nada registrado significa que não havia nada para registrar. Também redefinimos o índice para disponibilizar as informações do membro e da semana como colunas e, em seguida, nomeamos essas colunas:

Pitão

```
>>> all_email = emails.set_index(['semana', 'membro']).
```

reindexar (complete_idx, fill_value = 0).

reset_index()

>>> all_email.columns = ['semana', 'membro', 'EmailsAbertos']

Vamos dar uma olhada no membro 998 novamente:

Pitão

>>> all_email[all_email.member == 998].sort_values('semana')

membros da semana EmailsOpened

09-02-2015 998 0

16/02/2015 998 0

23/02/2015 998 0

02-03-2015 998 0

09-03-2015 998 0

Pandas de Python

Pandas é um pacote de análise de quadro de dados em Python amplamente utilizado na comunidade de ciência de dados. O próprio nome indica sua adequação para análise de séries temporais: “Pandas” refere-se a “dados em painel”, que é o que os cientistas sociais chamam de dados de séries temporais.

Pandas é baseado em tabelas de dados com índices de linhas e colunas. Possui operações semelhantes a SQL integradas, como agrupar por, seleção de linha e indexação de chave. Ele também possui funcionalidades específicas de séries temporais, como indexação por período de tempo, redução da resolução e operações de agrupamento com base no tempo.

Se você não está familiarizado com Pandas, recomendo fortemente dar uma breve visão geral,

como o fornecido na documentação oficial.

Observe que temos um grande número de zeros no início. Provavelmente, eles ocorreram antes de o membro ingressar na organização, portanto, não estariam em uma lista de e-mail. Não há muitos tipos de análise em que gostaríamos de manter as semanas verdadeiramente nulas do membro - especificamente aquelas semanas antes de o membro indicar a abertura de um e-mail. Se tivéssemos a data precisa em que um membro começou a receber e-mails, teríamos um corte objetivo. Do jeito que está, deixaremos que os dados nos guiem. Para cada membro, determinamos os limites de data_inicial e data_final agrupando o DataFrame de e-mail por membro e selecionando os valores máximo e mínimo da semana:



Pitão

>>> datas_cutoff = emails.groupby('membro').semana.

agg(['min', 'max']).reset_index)

>>> datas_cutoff = datas_cutoff.reset_index()

Eliminamos linhas do DataFrame que não contribuem de forma sensata para a cronologia, especificamente 0 linhas antes da primeira contagem diferente de zero de cada membro:

Pitão

>>> **para** , linha **em** cutoff dates.iterrows():

```
>>> membro = linha['membro']

>>> data_início = linha['min']

>>> data_fim = linha['max']

>>> all_email.drop(

all_email[all_email.membro == membro]

[all_email.week < data_inicial].index, inplace=True)

>>> all_email.drop(all_email[all_email.member ==
membro]

[all_email.week > data_final].index, inplace=True)
```

< ou <= ?

Usamos as operações < e >, sem igualdade, porque o

start_date e end_date incluem os dados significativos

pontos e porque estamos descartando dados, não retendo dados, como nosso

o código está escrito. Neste caso queremos incluir essas semanas no nosso

análise porque foram os primeiros e últimos pontos de dados significativos.

Você fará bem em trabalhar com seus engenheiros de dados e administradores de banco de dados para convencê-los a armazenar os dados de maneira consciente do tempo, especialmente no que diz respeito a como os carimbos de data e hora são criados e o que eles significam. Quanto mais você resolver problemas no upstream, menos trabalho terá que fazer no downstream no pipeline de dados.

Agora que limpamos nossos dados de e-mail, podemos considerar novas questões. Por exemplo, se quisermos

pensar sobre a relação entre o comportamento dos e-mails dos membros e as doações, podemos fazer algumas coisas:

• Agregue o DonationAmount semanalmente para que os períodos sejam comparáveis. Então torna-se razoável perguntar se as doações se correlacionam de alguma forma com as respostas dos membros aos e-mails.

• Trate os EmailsOpened da semana anterior como um preditor da quantia DonationA de uma determinada semana. Observe que temos que usar a semana anterior porque EmailsOpened é uma estatística resumida da semana. Se quisermos prever uma doação de quarta-feira, e nosso EmailsOpened resume o comportamento de abertura de e-mail de segunda a domingo,

então, usar as informações da mesma semana nos fornecerá potencialmente informações

sobre o que o membro fez depois de podermos saber (por

por exemplo, se abriram um e-mail na sexta-feira após a doação).

Construindo uma série temporal encontrada

Considere como relacionar os dados de e-mail e doações entre si. Podemos reduzir a resolução dos dados de doação para transformá-los em uma série temporal semanal, comparável aos dados de e-mail. Como organização, estamos interessados nos valores semanais totais, por isso agregamos os carimbos de data/hora em períodos semanais por meio da soma. É improvável que haja mais de uma doação por semana, portanto os valores das doações semanais refletirão os valores das doações individuais para a maioria dos doadores.

Pitão

```
>>> doações.timestamp =
pd.to_datetime(doações.timestamp)
```

>>> doações.set_index('timestamp', inplace = True)

>>> agg_don = doações.groupby('membro').apply(

lambda df: df.amount.resample("W-MON").sum().dropna())

Neste código, primeiro convertemos um caractere de string em uma classe de dados com carimbo de data/hora adequada para nos beneficiarmos da indexação integrada relacionada à data do Pandas. Definimos o carimbo de data/hora como um índice, conforme necessário para reamostrar um quadro de dados. Finalmente, para o quadro de dados que obtemos subdividindo cada membro, agrupamos e somamos as doações por semana, eliminamos as semanas que não têm doações e depois as coletamos.

Observe que reamostramos com uma semana ancorada para que correspondamos ao mesmo

datas semanais que já temos em nossa tabela de e-mail. Observe também que uma semana ancorada em

“Segunda-feira” faz sentido do ponto de vista humano.

Agora temos informações de doações e informações de e-mail coletadas na mesma frequência e podemos nos juntar a elas. O Pandas torna isso simples, desde que fixemos as semanas no mesmo dia da semana, como já fizemos. Podemos iterar cada membro e mesclar os quadros de dados por membro:

Pitão

>>> **para** membro, member_email **em** all_email.groupby('member'):

>>> member_donations = agg_donations[agg_donations.membro

== membro]

>>>

```
member_donations.set_index('timestamp', inplace = True)

>>>

member_email.set_index ('semana', inplace = True)

>>>

membro_email = all_email[all_email.member == membro]

>>>

member_email.sort_values('semana').set_index('semana')

>>>

df = pd.merge(member_email, member_donations, how =
'left',

índice_esquerdo = Verdadeiro,

índice_direito = Verdadeiro)

>>>

df.fillna(0)

>>>

df['membro'] = df.membro_x

>>>

mesclado_df = mesclado_df.append(df.reset_index()

[['membro', 'semana', 'e-mails abertos', 'quantia']])
```

Adaptando uma coleção de dados de série temporal a partir de uma coleção de tabelas | 33

Agora temos nossos dados de e-mail e doações alinhados por membro. Para cada membro incluímos apenas semanas

significativas e não semanas que parecem ser antes ou depois do período de adesão.

Poderíamos tratar o comportamento do e-mail como uma variável de “estado” relativa ao comportamento de doação, mas provavelmente desejaremos transferir o estado da semana anterior para evitar uma antecipação. Suponha, por exemplo, que estivéssemos construindo um modelo que utiliza o comportamento do e-mail para prever a próxima doação de um membro. Nesse caso, podemos considerar observar o padrão de abertura de e-mail ao longo do tempo como um possível indicador. Precisaríamos alinhar a doação de uma determinada semana com o comportamento do e-mail da semana anterior. Podemos facilmente pegar nossos dados, processá-los para alinhá-los semana a semana e depois alterá-los pelo número apropriado de semanas. Se, digamos, quisermos adiantar a doação uma semana, podemos fazê-lo facilmente com o operador do turno, embora precisemos ter certeza de fazer isso por membro:

Pitão

>>> df = merged_df[merged_df.member == 998]

>>> df['alvo'] = df.amount.shift(1)

>>> df = df.fillna(0)

>>> df

É uma boa prática armazenar essa meta em uma nova coluna em vez de substituir a antiga, principalmente se você não estiver antecipando o carimbo de data e hora do valor da doação separadamente. Mudamos o valor da doação para uma semana no futuro usando a funcionalidade de mudança integrada do Pandas. Você também pode voltar no tempo com números negativos. Geralmente você terá mais preditores do que alvos, então faz sentido mudar seus alvos. Podemos ver o resultado do código aqui:

quantidade de e-mailsOpened member week target

0 1 998 2017-12-04 0

0 3 998 2017-12-11 0

0 3 998 2017-12-18 0

0 0 998 2017-12-25 0

0 3 998 01/01/2018 0

50 3 998 08/01/2018 0

0 2 998 2018-01-15 50

Agora que preenchemos as linhas que faltam, temos as 26 linhas desejadas para o membro 998. Nossos dados agora estão mais limpos e completos.

Para recapitular, estas são as técnicas específicas de séries temporais que usamos para reestruturar os dados:

1. *Recalibre a resolução dos nossos dados* para se adequar à nossa pergunta. Freqüentemente, os dados vêm com informações de tempo mais específicas do que precisamos.

2. Compreender como podemos *evitar a antecipação* ao não usar dados para carimbos de data/hora que produzam a disponibilidade dos dados.

3. Registre *todos os períodos relevantes,* mesmo que “nada tenha acontecido”. Uma contagem zero é tão informativa quanto qualquer outra contagem.

4. *Evite antecipar ,* não usando dados para carimbos de data/hora que produzam informações que ainda não deveríamos conhecer.

Até agora, criamos séries temporais encontradas brutas, alinhando nossas séries temporais de doações e e-mails para serem amostradas nos mesmos momentos e na mesma frequência.

No entanto, não fizemos um trabalho completo de limpeza desses dados ou de exploração completa

antes da análise. Isso é algo que faremos no Capítulo 3.

Problemas de carimbo de data/hora

Os carimbos de data e hora são bastante úteis para análise de séries temporais. A partir dos carimbos de data/hora, podemos extrapolar uma série de recursos interessantes, como hora do dia ou dia da semana.

Esses recursos podem ser importantes para a compreensão dos seus dados, especialmente para dados relativos ao comportamento humano. No entanto, os carimbos de data e hora são complicados. Aqui discutimos algumas das dificuldades dos dados com carimbo de data/hora.

De quem é o carimbo de data/hora?

A primeira pergunta que você deve fazer ao ver um carimbo de data/hora é qual processo gerou o carimbo de data/hora, como e quando. Freqüentemente, um evento que acontece não coincide com um evento que está sendo registrado. Por exemplo, um pesquisador pode anotar algo em seu caderno e depois transcrevê-lo para um arquivo CSV usado como registro. O carimbo de data/hora indica quando eles o escreveram ou quando o transcreveram para o arquivo CSV? Ou um usuário de aplicativo móvel pode realizar ações que acionam o registro quando seu telefone está off-line, para que os dados só sejam carregados posteriormente em seu servidor com alguma combinação de carimbos de data/hora. Esses carimbos de data/hora podem refletir quando o comportamento ocorreu, quando a ação foi registrada pelo aplicativo, quando os metadados foram carregados no servidor ou quando os dados foram acessados pela última vez para download do servidor de volta para o aplicativo (ou qualquer número de outros eventos ao longo do pipeline de dados).

Os carimbos de data e hora podem parecer à primeira vista oferecer clareza, mas isso desaparece rapidamente se faltar a documentação adequada. Quando você analisa novos carimbos de data/hora, seu primeiro passo deve ser sempre

descobrir da melhor maneira possível o que estabeleceu a hora de um evento.

Um exemplo concreto ilustrará estas dificuldades. Suponha que você esteja consultando dados obtidos de um aplicativo móvel para perda de peso e veja um diário de refeições com entradas como as da Tabela 2-4 .

Tabela 2-4. Exemplo de diário de refeições de um aplicativo de perda de peso

Tempo	**Ingestão**
Segunda-feira, 7 de abril, 11h14:32	panquecas
Segunda, 7 de abril, 11h14:32	sanduíche
Segunda, 7 de abril, 11h14:32	pizza

É possível que esse usuário tenha comido panquecas, um sanduíche e uma pizza ao mesmo tempo, mas há cenários mais prováveis. O usuário especificou esse horário ou ele foi criado automaticamente?

A interface talvez ofereça um horário automático que o usuário possa ajustar ou optar por ignorar? Algumas respostas a essas perguntas explicariam melhor os carimbos de data e hora idênticos do que a possibilidade de um usuário tentando perder peso comer panquecas com um acompanhamento de pizza e um sanduíche de aperitivo.

Mesmo que o usuário tivesse todos esses alimentos às 11h14, onde no mundo seriam 11h14?

Esta era a hora local do usuário ou um relógio global? Mesmo no caso improvável de o usuário ter consumido toda essa comida em uma refeição, ainda não sabemos muito sobre o aspecto temporal da refeição apenas com base nessas linhas. Não sabemos se foi café da manhã, almoço, jantar ou lanche. Para dizer algo interessante ao usuário,

gostaríamos de poder falar em termos concretos sobre a hora local do dia, o que não podemos fazer sem informações de fuso horário.

A melhor maneira de responder a essas questões é ver todo o código que coleta e armazena os dados ou conversar com as pessoas que escreveram esse código. Depois de passar por todas as especificações de dados humanos e técnicos disponíveis no sistema, você também deve testar o sistema completo para garantir que os dados se comportem como lhe foi dito. Quanto melhor você entender seu pipeline de dados, menor será a probabilidade de fazer perguntas erradas, porque seus carimbos de data e hora não significam realmente o que você pensa que significam.

Você é o responsável final pela compreensão dos dados. As pessoas que trabalham no upstream do pipeline não sabem o que você tem em mente para análise. Tente ser o mais prático possível ao avaliar como os carimbos de data e hora são gerados. Portanto, se você estiver analisando dados de um pipeline de aplicativo móvel, baixe o aplicativo, acione um evento em vários cenários e veja a aparência dos seus próprios dados. Você provavelmente ficará surpreso com a forma como suas ações foram registradas depois de conversar com aqueles que gerenciam o pipeline de dados. É difícil rastrear múltiplos relógios e contingências, por isso a maioria dos conjuntos de dados achatará as realidades temporais. Você precisa saber exatamente como eles fazem isso.

Adivinhando carimbos de data e hora para entender os dados

Se você estiver lidando com pipelines de dados legados ou dados não documentados, talvez não tenha a opção de explorar um pipeline em funcionamento e conversar com quem o mantém.

Você precisará fazer alguma investigação empírica para entender se é possível fazer inferências sobre o significado dos carimbos de data/hora:

• Lendo os dados como fizemos no exemplo anterior, você pode gerar hipóteses iniciais sobre o significado dos timestamps. No caso anterior, observe os dados de vários usuários para ver se o mesmo padrão (várias linhas com carimbos de data e hora idênticos e conteúdos improváveis de uma única refeição) se manteve ou se foi um problema.

anomalia.

• Usando análises em nível agregado, você pode testar hipóteses sobre o que os carimbos de data/hora significam ou provavelmente significam. Para os dados anteriores, existem algumas questões abertas

questões:

— O carimbo de data/hora é local ou universal?

— O tempo reflete uma ação do usuário ou alguma restrição externa, como

conectividade?

Hora local ou universal?

A maioria dos carimbos de data/hora são armazenados no horário universal (UTC) ou em um único fuso horário, dependendo da localização do servidor, mas independente da localização do usuário. É bastante incomum armazenar dados de acordo com a hora local. No entanto, devemos considerar ambas as possibilidades, porque ambas são encontradas “na natureza”.

Formamos a hipótese de que se a hora for um carimbo de data/hora local (local para cada usuário), deveríamos ver tendências diárias nos dados refletindo o comportamento diurno e noturno. Mais especificamente, não devemos esperar muita atividade durante a noite, quando nossos usuários estão dormindo. Para nosso exemplo de aplicativo móvel, se criarmos um histograma de contagem de horas de refeição por hora, deverá haver horas com um número significativamente menor de refeições registradas, porque na maioria das culturas as pessoas não comem no meio da noite.

Se não víssemos um padrão orientado ao dia nas horas exibidas, poderíamos concluir que os dados provavelmente foram marcados com a data e hora de algum relógio universal e que a base de usuários deve estar espalhada por vários fusos horários. Nesse caso, seria bastante desafiador extrapolar os horários locais dos usuários individuais (assumindo que as informações de fuso horário não estivessem disponíveis). Poderíamos considerar explorações individuais por usuário para ver se poderíamos codificar heurísticas para rotular os usuários aproximadamente por fuso horário, mas esse trabalho é computacionalmente desgastante e nem sempre preciso.

Mesmo que você não consiga identificar o fuso horário exato do usuário, ainda é útil ter um carimbo de data/hora global disponível. Para começar, você pode determinar padrões de uso prováveis para os servidores do seu aplicativo, sabendo quando as refeições são registradas com mais frequência em um determinado horário do dia e dia da semana. Você também pode calcular o diferencial de tempo entre cada refeição registrada pelo usuário, sabendo que como os carimbos de data e hora estão em tempo absoluto, você não precisa se preocupar se o usuário mudou de fuso horário. Além de investigação, isso seria interessante como forma de geração de recursos:

Pitão

```
>>> df['dt'] = df.time - df.time.shift(-1)
```

A coluna dt seria um recurso que você poderia repassar em sua análise. Usar esse diferencial de horário também pode oferecer a oportunidade de estimar o fuso horário de cada usuário.

Você pode observar a hora do dia em que o usuário geralmente tem um dt longo, o que pode indicar quando ocorre a noite para esse usuário. A partir daí você poderia começar a descobrir o “horário noturno” de cada indivíduo sem ter que fazer análises de pico a pico.

Comportamento do usuário ou comportamento da rede?

Voltando a outra questão colocada pela nossa pequena amostra de dados, perguntamos se o nosso usuário fez uma refeição estranha ou se os nossos carimbos de data/hora estão relacionados à atividade de upload.

As mesmas análises usadas para identificar o fuso horário de um usuário são aplicáveis para determinar se os carimbos de data/hora são uma função do comportamento do usuário ou da rede. Depois de ter uma coluna dt (conforme calculado anteriormente), você poderá procurar clusters de 0s e determinar qualitativamente se eles eram provavelmente um único evento comportamental ou um único evento de rede. Você também pode verificar se os dts parecem ser periódicos em dias diferentes. Se fossem uma função do comportamento do usuário, seria mais provável que fossem periódicos do que se fossem uma função da conectividade de rede ou de outros comportamentos relacionados ao software.

Para resumir, aqui estão algumas questões que você provavelmente poderia responder com o conjunto de dados disponível, mesmo com pouca ou nenhuma informação sobre como os carimbos de data/hora são gerados:

• Use diferenças nos carimbos de data/hora por usuário para ter uma noção do espaçamento entre as refeições ou entre as entradas de dados (dependendo da sua hipótese de trabalho sobre o que os horários indicam, um comportamento do usuário ou um comportamento da rede).

• Descreva o comportamento agregado do usuário para determinar quando é mais provável que seus servidores estejam ativos no ciclo de 24 horas.

Fusos horários

Na melhor das hipóteses, seus dados recebem carimbo de data/hora no horário UTC. A maioria dos bancos de dados e outros sistemas de armazenamento usarão isso como padrão. No entanto, há algumas situações em que é

provável que você encontre dados que não tenham carimbo de data/hora UTC:

• Carimbos de data e hora criados de maneiras que não usam objetos de dados específicos de data, como chamadas de API. Essas chamadas de API usam strings em vez de objetos específicos de tempo.

• Dados criados “manualmente” em pequenas organizações locais onde os fusos horários não são relevantes, como planilhas geradas por analistas de negócios ou biólogos de campo.





O que é uma escala de tempo significativa?

Você deve considerar a resolução temporal do carimbo de data / hora recebido com cautela, com base no conhecimento do domínio sobre o comportamento que está estudando e também com base nos detalhes que você pode determinar em relação a como os dados foram coletados.

Por exemplo, imagine que você está analisando dados diários de vendas, mas sabe que, em muitos casos, os gerentes esperam até o final da semana para relatar os números, estimando números diários aproximados, em vez de registrá-los todos os dias. É provável que o erro de medição seja substancial devido a problemas de recordação e vieses cognitivos inatos. Você pode considerar alterar a resolução de seus dados de vendas de diária para semanal

para reduzir ou calcular a média desse erro sistemático. Caso contrário, você deve construir um modelo que leve em consideração a possibilidade de erros tendenciosos em diferentes dias da semana. Por exemplo, pode acontecer que os gestores sobrestimem sistematicamente o seu desempenho na segunda-feira quando reportam os números numa sexta-feira.

Desconto de tempo psicológico

O desconto de tempo é uma manifestação de um fenômeno conhecido como

distância psicológica , que dá nome à nossa tendência de sermos mais optimistas

místico (e menos realista) ao fazer estimativas ou avaliações

que estão mais “distantes” de nós. O desconto de tempo prevê que

os dados relatados no passado serão sistematicamente tendenciosos

em comparação com os dados relatados da memória mais recente. Isto é dis-

matiz do problema mais geral do esquecimento e implica uma

erro não aleatório. Você deve ter isso em mente sempre que estiver

olhando para dados gerados por humanos que foram inseridos manualmente, mas

não simultaneamente com o evento registrado.

Outra situação envolve o conhecimento físico do sistema. Por exemplo, há um limite para a rapidez com que a glicemia de uma pessoa pode mudar, portanto, se você estiver observando uma série de medições de glicemia com intervalos de segundos entre si, provavelmente deverá calcular a média delas em vez de tratá-las como pontos de

dados distintos. Qualquer médico lhe dirá que você está estudando o erro do dispositivo, e não a taxa de variação da glicemia, se estiver observando muitas medições com intervalo de alguns segundos entre si.

Os humanos sabem que o tempo está passando

Sempre que você estiver medindo humanos, tenha em mente que as pessoas

responder de mais de uma maneira à passagem do tempo. Por exemplo,

pesquisas recentes mostram como manipular a velocidade de um relógio que

está na visão de uma pessoa influencia a rapidez com que o sangue dessa pessoa

alterações no nível de glicose.

Problemas de carimbo de data/hora | 39

Limpando seus dados

Nesta seção, abordaremos os seguintes problemas comuns em conjuntos de dados de séries temporais:

• Dados ausentes

• Alteração da frequência de uma série temporal (ou seja, aumento e redução da resolução)

• Suavização de dados

• Abordar a sazonalidade nos dados

• Evitar antecipações não intencionais

Tratamento de dados ausentes

Dados ausentes são surpreendentemente comuns. Por exemplo, na área da saúde, a falta de dados em séries

temporais médicas pode ter várias causas:

• O paciente não cumpriu a medição desejada.

• As estatísticas de saúde do paciente estavam em boas condições, portanto não houve necessidade de realizar nenhuma medição específica.

• O paciente foi esquecido ou subtratado.

• Um dispositivo médico apresentou uma avaria técnica aleatória.

• Ocorreu um erro de entrada de dados.

Generalizando com grande risco, a falta de dados é ainda mais comum na análise de séries temporais do que na análise de dados transversais, porque o fardo da amostragem longitudinal é particularmente pesado: séries temporais incompletas são bastante comuns e, portanto, foram desenvolvidos métodos para lidar com buracos no que está gravado.

Os métodos mais comuns para resolver dados ausentes em séries temporais são:

Imputação

Quando preenchemos os dados faltantes com base em observações sobre todo o conjunto de dados.

Interpolação

Quando usamos pontos de dados vizinhos para estimar o valor ausente. A interpolação também pode ser uma forma de imputação.

Exclusão de períodos afetados

Quando optamos por não usar períodos de tempo que contenham dados ausentes.

Discutiremos imputações e interpolações, bem como ilustraremos a mecânica desses métodos em breve. Nós nos concentramos na preservação dos dados, enquanto um método como a exclusão de períodos com dados ausentes resultará em menos dados para o seu modelo. Seja para **40 | Capítulo 2: Encontrando e organizando dados de séries temporais**



preservar os dados ou descartar períodos de tempo problemáticos dependerá do seu caso de uso e se você pode sacrificar os períodos de tempo em questão, dadas as necessidades de dados do seu modelo.

Preparando um conjunto de dados para testar metodologias de imputação de dados ausentes

Trabalharemos com dados mensais de desemprego que foram divulgados pelos EUA

governo desde 1948 (disponível gratuitamente para download). Geraremos então dois conjuntos de dados a partir desses dados de base: um no qual os dados estão realmente faltando aleatoriamente e outro no qual estão os meses de maior desemprego na história da série temporal. Isso nos dará dois casos de teste para ver como a imputação se comporta na presença de dados ausentes aleatórios e sistemáticos.

Passamos para R para o próximo exemplo. Vamos alternar livremente entre

R e Python ao longo do livro. Presumo que você tenha algum retorno

base no trabalho com quadros de dados, bem como matrizes, tanto em R

e Python.

##R

> require(zoo) *## zoo fornece funcionalidade de série temporal*

> require(data.table) *## data.table é um quadro de dados de alto desempenho*

> unemp <- fread("UNRATE.csv")

> unemp[, DATA := as.Date(DATA)]

> setkey(desembaçar, DATA)

> *## gerar um conjunto de dados onde os dados estão faltando aleatoriamente*

> rand.unemp.idx <- sample(1:nrow(unemp), .1*nrow(unemp))

> rand.unemp <- unemp[-rand.unemp.idx]

> *## gerar um conjunto de dados onde os dados são mais prováveis*

> *## a faltar quando o desemprego é alto*

> high.unemp.idx <- which(unemp$UNRATE > 8)

> num.to.select <- 0,2 * comprimento (high.unemp.idx)

> high.unemp.idx <- amostra(high.unemp.idx,)

> preconceito.unemp <- unemp[-high.unemp.idx]

Como excluímos linhas de nossas tabelas de dados para criar um conjunto de dados com dados ausentes, precisaremos ler as datas ausentes e os valores NA, e para isso a funcionalidade *de junção contínua do pacote data.table* é bastante útil.

Pacote data.table do R

O pacote data.table do R é uma alternativa de alto desempenho, mas lamentavelmente subutilizada, à funcionalidade data.frame principal do R. Embora muitos usuários estejam familiarizados com a seleção organizada de pacotes, data.table é um pacote independente construído apenas sobre data.frame. Possui muitas funções úteis para análise de séries temporais. Provavelmente, isso se deve às suas origens como um pacote desenvolvido por engenheiros de software que trabalhavam em finanças e que precisavam de operações R de alto desempenho para trabalhar com dados financeiros, na maioria das vezes dados de séries temporais.

Embora haja uma curva de aprendizado mais acentuada para data.table do que para data.frame, considero esse pacote inestimável para trabalhar com dados de séries temporais em R e, por isso, uso-o intensamente ao longo do livro. Forneci alguns exemplos rápidos de como data.table funciona e pode ser diferente de data.frame aqui:

```
dt[, novo.col := antigo.col + 7]
```

Neste exemplo, assumimos que o data.table original possui uma coluna antiga chamada old.col. Criamos uma nova coluna, no lugar, o que significa que adicionamos uma coluna ao data.table sem copiar o data.table inteiro para um novo objeto, reduzindo assim o uso de memória. Você verá este operador := usado frequentemente ao longo do livro.

Para acessar um subconjunto de colunas em uma data.table, normalmente simplesmente as colocaremos em uma lista no argumento de indexação de coluna, que é o segundo argumento de uma data.table: dt[, .(col1, col2, col3)]

Isso retornará um subconjunto data.table incluindo apenas os valores col1, col2 e col3. Observe o operador .() em torno dos nomes das colunas. Esta é uma forma abreviada de escrever list().

Se desejarmos fazer a seleção de linhas em vez da seleção de colunas, usamos o primeiro argumento para data.table para criar subconjuntos de linhas da seguinte forma:

dt[col1 < 3 & col2 > 5]

Isso retornará todas as colunas do data.table para as quais as duas condições lógicas foram atendidas. E é claro que podemos combinar a seleção de linhas e colunas, como segue: dt[col1 < 3 & col2 > 5, .(col1, col2, col3)]

Finalmente, as operações de agrupamento são bastante eficientes e rápidas. Por exemplo, para contar o número de linhas em cada grupo entre os grupos designados em col1, poderíamos usar o seguinte código.

dt[, .N, col1]

E se quiséssemos fazer múltiplas operações em nossos objetos agrupados, faríamos novamente uma lista e poderíamos até nomear as colunas:

dt[, .(total = .N, média = média(col2), col1]

O acima, por grupos designados por col1, retornaria o número total de linhas no grupo e a média de col2 para cada grupo.

O pacote data.table está em desenvolvimento ativo e é uma ferramenta inestimável para análise de séries temporais, especialmente para grandes conjuntos de dados. Recomendo fortemente a leitura do

introdução oficial ao data.table.

##R

> all.dates <- seq(from = unemp$DATE[1], to = tail(unemp$DATE, 1),

por = "meses")

> rand.unemp = rand.unemp[J(todas.datas), roll=0]

> preconceito.unemp = preconceito.unemp[J(todas.datas), roll=0]

> rand.unemp[, rpt := is.na(UNRATE)]

aqui rotulamos os dados ausentes para facilitar a plotagem

Com um Rolling Join, geramos a sequência de todas as datas que devem estar disponíveis entre a data de início e de término do conjunto de dados. Isso nos dá linhas no conjunto de dados para preencher como NA.

Agora que temos conjuntos de dados com valores ausentes, daremos uma olhada em algumas maneiras específicas de preencher números para esses valores ausentes:

• Preenchimento direto

• Média móvel

• Interpolação

Compararemos o desempenho desses métodos em conjuntos de dados ausentes aleatoriamente e sistematicamente ausentes. Como geramos esses conjuntos de dados a partir de um conjunto de dados completo, podemos de fato determinar como eles se saíram, em vez de especular. No mundo real, é claro, nunca teremos os dados faltantes para verificar a nossa imputação de dados.

Junções rolantes

Na junção contínua do data.table, aproveitamos o benefício de uma junção especial semelhante a SQL com reconhecimento espacial, projetada para carimbos de data/hora. Embora as tabelas nem sempre correspondam exatamente em seus carimbos de data/hora, as junções contínuas podem lidar com os carimbos de data/hora de maneira inteligente.

##R

> ## temos uma pequena tabela de dados de datas de doação

> doações <- data.table(

> valor = c(99, 100, 5, 15, 11, 1200),

> dt = as.Data(c("2019-2-27", "2019-3-2", "2019-6-13",

> "2019-8-1", "2019-8-31", "2019-9-15"))

>)

> ## também temos informações sobre as datas de cada

> ## campanha publicitária

> publicidade <- data.table(

> identificador = c("q4q42", "4299hj", "bbg2"),

> dt = as.Data(c("2019-1-1",

> "2019-4-1",

> "2019-7-1")))

> ## definimos a chave primária em cada data.table

> setkey(publicidade, "dt")

> setkey(doações, "dt")

> ## desejamos rotular cada doação de acordo com

> ## qual campanha publicitária a precedeu mais recentemente

> ## podemos ver isso facilmente com roll = TRUE

> publicidade[doações, roll = **TRUE**]

Isso produz o seguinte resultado delicioso:

##R

> publicidade[doações, roll = **TRUE**]

identificador dt amt

1: q4q42 2019-02-27 99

2: q4q42 2019-03-02 100

3: 4299hj 13/06/2019 5

4: bbg2 01/08/2019 15

5: bbg2 31/08/2019 11

6: bbg2 15/09/2019 1200

Cada doação agora é associada ao identificador de publicidade que indica a campanha que precedeu mais imediatamente a doação.

Preenchimento direto

Uma das maneiras mais simples de preencher valores ausentes é transportar o último valor conhecido antes do valor ausente, uma abordagem conhecida como *preenchimento direto* . Nenhuma matemática ou lógica complicada é necessária. Basta considerar a experiência de avançar no tempo com os dados que estavam disponíveis e você verá que, em um momento perdido, tudo o que você pode ter certeza é quais dados já foram registrados. Nesse caso, faz sentido usar a medição conhecida mais recente.

O preenchimento direto pode ser realizado facilmente usando na.locf do pacote zoo:

##R

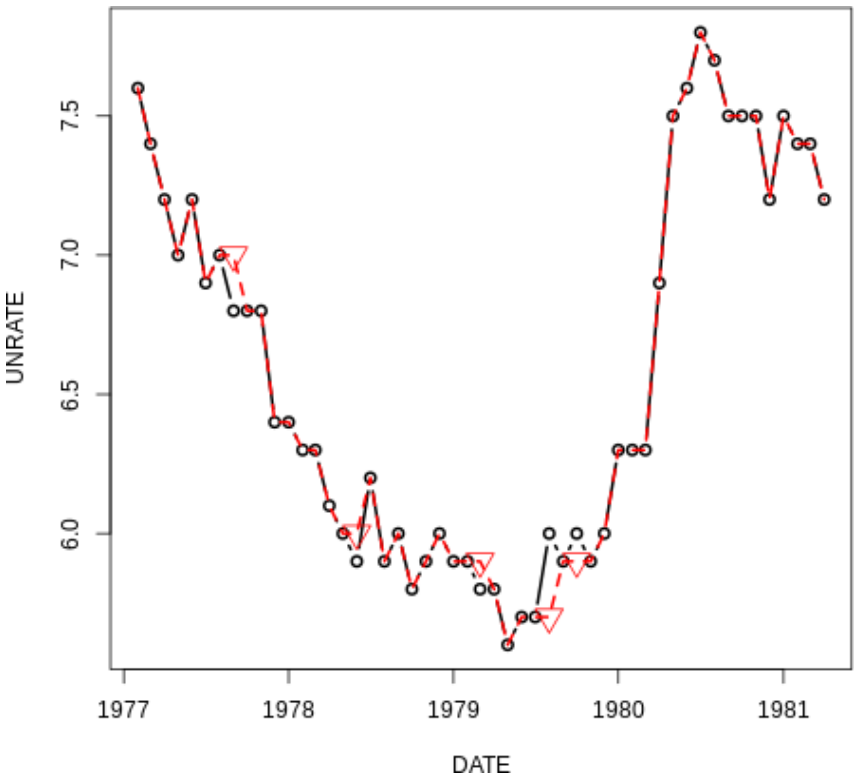
> rand.unemp[, impute.ff := na.locf(UNRATE, na.rm = **FALSE**)]

> preconceito.unemp[, impute.ff := na.locf(UNRATE, na.rm = **FALSE**)]

>

> *## para traçar um gráfico de amostra mostrando as partes planas*

> unemp[350:400, plot (DATA, UNRATE,



col = 1, lwd = 2, tipo = 'b')]

> rand.unemp[350:400, linhas(DATA, impute.ff,

col = 2, lwd = 2, lty = 2)]

> rand.unemp[350:400][rpt == **TRUE** , pontos(DATA, impute.ff,

col = 2, pch = 6, cex = 2)]

Isso resultará em um gráfico que parece natural, exceto onde você vê valores repetidos para compensar dados faltantes, como na Figura 2-6 . Como você notará no gráfico, os valores preenchidos normalmente não se desviam muito dos valores verdadeiros.

Figura 2-6. A série temporal original traçada com uma linha contínua e a série temporal com valores preenchidos adiante para pontos ausentes aleatoriamente plotados com uma linha tracejada. Os valores preenchidos para frente são marcados com triângulos apontando para baixo.

Também podemos comparar os valores da série traçando os valores das séries entre si. Ou seja, para cada intervalo de tempo, traçamos o verdadeiro valor conhecido em relação ao valor ao mesmo tempo da série com valores imputados. A maioria dos valores deve corresponder exatamente, pois a maioria dos dados está presente. Vemos isso manifestado na linha 1:1 em

Figura 2-7 . Também vemos pontos dispersos fora desta linha, mas eles não parecem estar sistematicamente desligados.



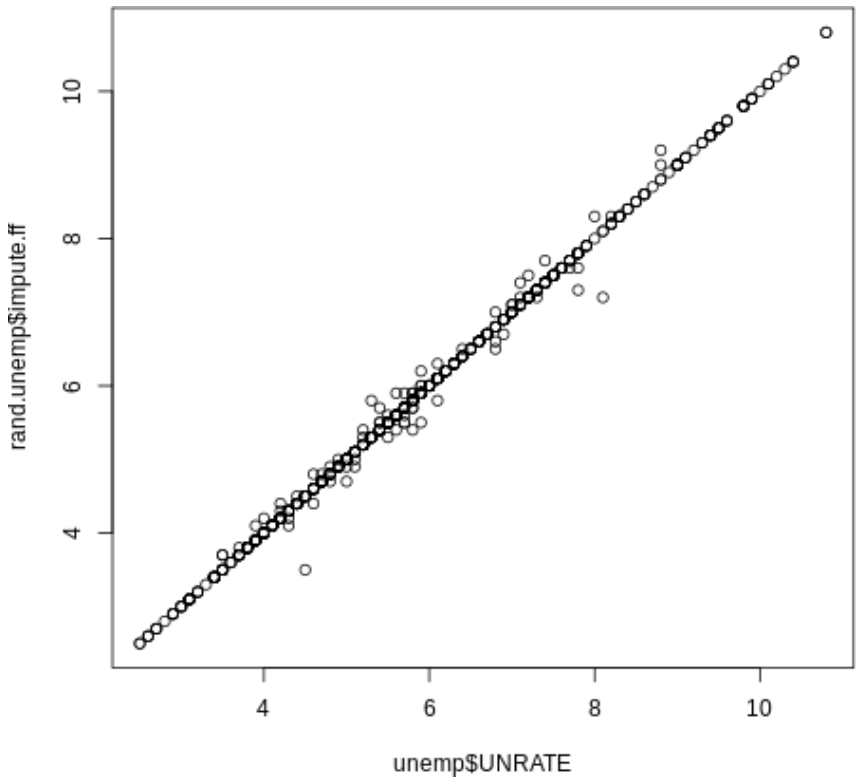


Figura 2-7. Traçando a verdadeira taxa de desemprego versus a série preenchida. Esse O gráfico mostra que o preenchimento direto não distorceu sistematicamente os dados.

Preenchimento para trás

Assim como você pode trazer valores do passado para preencher lacunas

dados, você também pode optar por propagar valores para trás. No entanto,

este é um caso de antecipação, então você só deve fazer isso quando

não procuram prever o futuro a partir dos dados e quando, a partir

conhecimento do domínio, faz mais sentido preencher os dados de trás para frente

em vez de avançar no tempo.

Em algumas configurações, o preenchimento direto faz sentido como a melhor maneira de completar os dados ausentes, mesmo que “métodos mais sofisticados” sejam possíveis. Por exemplo, em ambientes médicos, um valor em falta indica frequentemente que um profissional de saúde não considerou necessário medir novamente um valor, provavelmente porque se esperava que a medição do paciente fosse normal. Em muitos casos médicos, isto significa que poderíamos aplicar o preenchimento antecipado aos valores faltantes com o último valor conhecido, uma vez que esta era a presunção que motivava o profissional médico a não refazer uma medição.

Há muitas vantagens no forward fill: ele não exige muito do ponto de vista computacional, pode ser facilmente aplicado a dados transmitidos ao vivo e faz um trabalho respeitável com imputação. Veremos um exemplo em breve.

Média móvel

Também podemos imputar dados com uma média móvel ou mediana. Conhecido como *movimento média* , isso é semelhante a um preenchimento direto em que você está usando valores anteriores para “prever”



valores futuros ausentes (a imputação pode ser uma forma de previsão). Com uma média móvel, entretanto, você está usando informações de *vários* momentos recentes no passado.

Há muitas situações em que uma imputação de dados de média móvel é mais adequada para a tarefa do que um preenchimento direto. Por exemplo, se os dados apresentarem ruído e você tiver motivos para duvidar do valor de qualquer ponto de dados individual em relação a

uma média geral, deverá usar uma média móvel em vez de um preenchimento direto. O preenchimento direto pode incluir mais ruído aleatório do que a métrica “verdadeira” que lhe interessa, enquanto a média pode remover parte desse ruído.

Para evitar uma antecipação, use apenas os dados que ocorreram antes do ponto de dados ausente. Então, sua implementação seria algo assim:

##R

> ## média rolante sem previsão

> rand.unemp[, impute.rm.nolookahead := rollapply(c(**NA** , **NA** , UNRATE), 3,

> função(x) {

> if (!is.na(x[3])) x[3] else significa(x, na.rm = **TRUE**)

> })]

> preconceito.unemp[, impute.rm.nolookahead := rollapply(c(**NA** , **NA** , UNRATE), 3,

> função(x) {

> if (!is.na(x[3])) x[3] else significa(x, na.rm = **TRUE**)

> })]

Definimos o valor dos dados ausentes como a média dos valores anteriores (porque indexamos o valor final e usamos esse valor para determinar se estão ausentes e como substituí-los).

Uma média móvel não precisa ser uma média aritmética. Para

por exemplo, médias móveis ponderadas exponencialmente dariam mais

peso para dados recentes do que para dados passados. Alternativamente, uma geometria

média pode ser útil para séries temporais que exibem forte correlação serial

lação e nos casos em que os valores aumentam ao longo do tempo.

Ao imputar dados faltantes com uma média móvel, considere se você precisa saber o valor da média móvel apenas com dados prospectivos ou se você se sente confortável em construir uma média móvel. Se você não estiver preocupado com uma previsão antecipada, sua melhor estimativa incluirá pontos antes e depois dos dados ausentes, porque isso maximizará as informações incluídas em suas estimativas. Nesse caso, você pode implementar uma janela contínua, conforme ilustrado aqui com a funcionalidade roll apply() do pacote zoo:

##R

> ## média rolante com uma visão antecipada

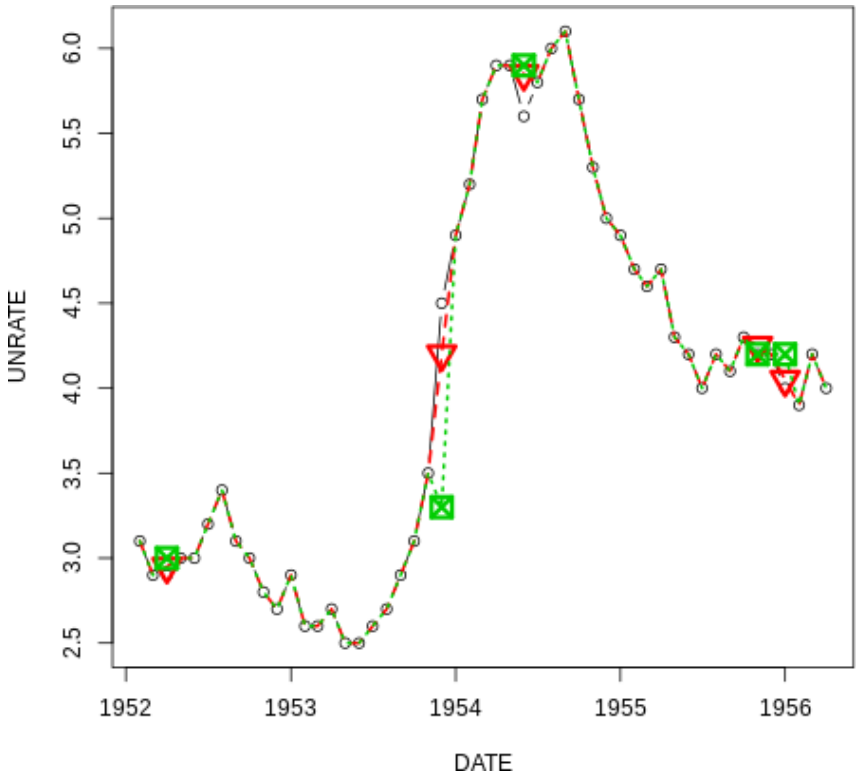
> rand.unemp[, complete.rm := rollapply(c(**NA** , UNRATE, **NA**), 3,

> função(x) {

> se (!is.na(x[2]))

>x[2]





> mais

> média (x, na.rm = **VERDADEIRO**)

> })]

Usar informações passadas e futuras é útil para visualizações e aplicativos de manutenção de registros, mas, como mencionado anteriormente, não é apropriado se você estiver preparando seus dados para serem inseridos em um modelo preditivo.

Os resultados tanto para uma média móvel prospectiva quanto para uma média móvel

Os dados futuros e passados são mostrados na Figura 2-8 .

Figura 2-8. A linha pontilhada mostra a imputação da média móvel sem previsão, enquanto a linha tracejada mostra a imputação da média móvel com uma previsão. Da mesma forma, os quadrados mostram os pontos imputados não antecipados, enquanto os triângulos invertidos mostram a média móvel com antecipação.

Uma imputação de dados de média móvel reduz a variação no conjunto de dados. Isso é algo que você precisa ter em mente ao calcular a precisão, estatísticas R^2 ou outras métricas de erro. Seu cálculo pode superestimar o desempenho do seu modelo, um problema frequente na construção de modelos de séries temporais.

Usando a média de um conjunto de dados para imputar dados ausentes

Num contexto transversal, é comum imputar dados faltantes

preenchendo a média ou mediana para aquela variável onde está faltando

ing. Embora isso possa ser feito com dados de séries temporais, não é apropriado

comi na maioria dos casos. Conhecer a média do conjunto de dados envolve

olhando para o futuro... e isso é uma visão antecipada!

Interpolação

A interpolação é um método para determinar os valores de pontos de dados ausentes com base em restrições geométricas sobre como queremos que os dados gerais se comportem. Por exemplo, uma interpolação linear restringe os dados faltantes a um ajuste linear consistente com pontos vizinhos conhecidos.

A interpolação linear é particularmente útil e interessante porque permite usar o seu conhecimento de como o seu sistema se comporta ao longo do tempo. Por exemplo, se você sabe que um sistema se comporta de maneira linear, você pode incorporar esse conhecimento de modo que apenas tendências lineares sejam usadas para imputar dados ausentes. Na linguagem bayesiana, permite injetar um *anterior* em sua imputação.

Tal como acontece com uma média móvel, a interpolação pode ser feita de forma que observe dados passados e futuros ou apenas em uma direção. As advertências usuais se aplicam: permita que sua interpolação tenha acesso a dados futuros somente se você aceitar que isso cria uma antecipação e tiver certeza de que isso não é um problema para sua tarefa.

Aqui aplicamos interpolação usando pontos de dados passados e futuros (veja a Figura 2-9):

```
##R

> ## interpolação linear

> rand.unemp[, impute.li := na.approx(UNRATE)]

> preconceito.unemp[, impute.li := na.approx(UNRATE)]

>

> ## interpolação polinomial

> rand.unemp[, impute.sp := na.spline(UNRATE)]

> preconceito.unemp[, impute.sp := na.spline(UNRATE)]

>

> usar.idx = 90:120

> unemp[use.idx, plot(DATE, UNRATE, col = 1, type = 'b')]

> rand.unemp[use.idx, linhas(DATA, impute.li, col = 2, lwd
= 2, lty = 2)]

> rand.unemp[use.idx, linhas(DATA, impute.sp, col = 3, lwd
= 2, lty = 3)]
```

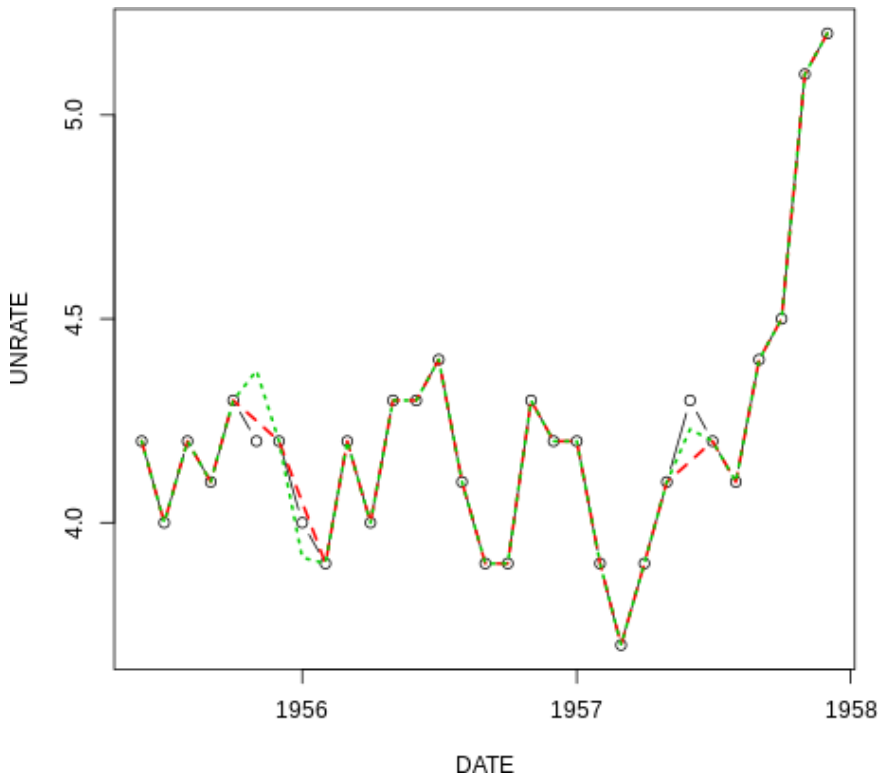


Figura 2-9. A linha tracejada mostra a interpolação linear enquanto a linha pontilhada mostra a interpolação spline.

Existem muitas situações em que uma interpolação linear (ou spline) é apropriada.

Considere a temperatura média semanal onde há uma tendência conhecida de aumento ou queda das temperaturas dependendo da época do ano. Ou considere os dados de vendas anuais de uma empresa em crescimento. Se a tendência tem sido o aumento linear do volume de negócios de ano para ano, é uma imputação de dados razoável preencher os dados em falta com base nessa tendência. Por outras palavras, poderíamos utilizar uma interpolação linear, que levaria em conta a tendência, em vez de uma média móvel, o que não o faria. Se houvesse uma tendência de aumento dos valores, a média móvel subestimaria sistematicamente os valores faltantes.

Existem também muitas situações para as quais a interpolação linear (ou spline) não é apropriada. Por exemplo, se faltarem dados de precipitação em um conjunto de dados meteorológicos, você não deverá extrapolar

linearmente entre os dias conhecidos; como todos sabemos, não é assim que funciona a precipitação. Da mesma forma, se estivermos observando as horas de sono de alguém por dia, mas faltarem alguns dias de dados, não devemos extrapolar linearmente as horas de sono entre os últimos dias conhecidos. Por exemplo, um dos desfechos conhecidos pode incluir uma noite inteira de estudo seguida de uma soneca de 30 minutos. É pouco provável que isto estime os dados em falta.

Comparação geral

Agora que realizamos alguns tipos diferentes de imputações, podemos dar uma olhada nos resultados para comparar como as diferentes imputações de dados se comportaram neste conjunto de dados.

50 | Capítulo 2: Encontrando e organizando dados de séries temporais

Geramos dois conjuntos de dados com dados faltantes, um em que faltavam dados aleatoriamente e outro em que faltavam pontos de dados desfavoráveis (alto desemprego). Quando comparamos os métodos que empregamos para ver qual produz os melhores resultados, podemos ver que o erro quadrático médio pode diferir em uma alta porcentagem:

##R

> sort(rand.unemp[, lapply(.SD, function(x) significa((x - unemp$UNRATE)^2,

> na.rm = **VERDADEIRO**)),

> .SDcols = c("impute.ff", "impute.rm.lookahead",

> "impute.rm.nolookahead", "impute.li",

> "imputar.sp")])

impute.li impute.rm.lookahead impute.sp impute.ff impute.rm.nolookahead 0,0017 0,0019 0,0021 0,0056 0,0080

```
> sort(bias.unemp[ , lapply(.SD, function(x) média((x -
unemp$UNRATE)^2,

> na.rm = VERDADEIRO )),

> .SDcols = c("impute.ff", "impute.rm.lookahead",

> "impute.rm.nolookahead", "impute.li",

> "imputar.sp")])

impute.sp impute.li impute.rm.lookahead
impute.rm.nolookahead impute.ff 0,0012 0,0013 0,0017
0,0030 0,0052
```

Lembre-se de que muitos dos métodos anteriores incluem uma antecipação. Os únicos métodos que não o fazem são o preenchimento direto e a média móvel sem antecipação (há também uma média móvel com antecipação). Por esse motivo, não é surpreendente que haja uma série de diferenças nos erros e que os métodos sem lookahead não tenham um desempenho tão bom quanto os outros.

Notas finais

Aqui cobrimos os métodos mais simples e mais usados de imputação de dados ausentes para aplicações de séries temporais. A imputação de dados continua sendo uma área importante da pesquisa em ciência de dados. Quanto mais significativas forem as decisões que você estiver tomando, mais importante será considerar cuidadosamente os possíveis motivos da falta de dados e as possíveis ramificações de suas correções. Aqui estão algumas dicas preventivas que você deve ter em mente:

• É impossível provar que os dados estão realmente faltando aleatoriamente e é improvável que a falta seja verdadeiramente aleatória na maioria das ocorrências do mundo real.

• Às vezes, a probabilidade de faltar uma medição é explicável pelas variáveis que você mediu, mas às vezes

não. Conjuntos de dados amplos com muitos recursos são a melhor maneira de investigar possíveis explicações para padrões de dados ausentes, mas não são a norma para análise de séries temporais.

• Quando precisar compreender a incerteza introduzida pela imputação de valores aos dados faltantes, você deve analisar uma variedade de cenários e também falar com o maior número possível de pessoas envolvidas no processo de coleta de dados.



• A maneira como você lida com os dados perdidos deve levar em conta o uso posterior desses dados. Você deve se proteger cuidadosamente contra antecipações ou decidir quão seriamente um

a antecipação afetará a validade do seu trabalho subsequente.

Upsampling e Downsampling

Freqüentemente, dados de séries temporais relacionados de fontes diferentes não terão a mesma frequência de amostragem. Este é um dos motivos, entre muitos, pelos quais você pode querer alterar a frequência de amostragem dos seus dados. É claro que você não pode alterar a taxa real com que as informações foram medidas, mas pode alterar a frequência dos carimbos de data/hora na sua coleta de dados. Isso é chamado de *upsampling* e *downsampling* , para aumentar ou diminuir a frequência do timestamp, respectivamente.

Reduzimos a amostragem de dados temporais em [“Retrofitting a Time Series Data Collection from](#)

uma coleção de tabelas” na página 26. Aqui, abordamos o tópico de forma mais geral, aprendemos

saber como e por que tanto o downsampling quanto o upsampling.

Downsampling é subconjunto de dados de modo que os carimbos de data e hora ocorram em

uma frequência mais baixa do que na série temporal original. Aumentar a resolução é

representando dados como se fossem coletados com mais frequência do que era

na verdade é o caso.

Redução da resolução

Sempre que você reduz a frequência de seus dados, você está reduzindo a resolução. Isso é feito com mais frequência nos seguintes casos.

A resolução original dos dados não é sensata. Pode haver vários motivos pelos quais a granularidade original dos dados não é sensata. Por exemplo, você pode estar medindo algo com muita frequência. Suponha que você tenha um conjunto de dados onde alguém mediu a temperatura do ar externo a cada segundo. A experiência comum indica que esta medição é indevidamente frequente e provavelmente oferece muito pouca informação nova relativa à carga adicional de armazenamento e processamento de dados. Na verdade, é provável que o erro de medição seja tão grande quanto a variação da temperatura do ar de segundo a segundo. Portanto, você provavelmente não deseja armazenar dados excessivos e pouco informativos. Neste caso - isto é, para dados amostrados regularmente - a redução da resolução é tão simples quanto selecionar cada *n*- ésimo elemento.

Concentre-se em uma parte específica de um ciclo sazonal. Em vez de se preocupar com dados sazonais em uma série temporal, você pode optar por criar uma subsérie focada em apenas uma temporada.

Por exemplo, podemos aplicar a redução da resolução para criar uma subsérie, como neste caso, onde geramos uma série temporal de medições de janeiro a partir do que era originalmente um **52 | Capítulo 2: Encontrando e organizando dados de séries temporais**

série temporal mensal. No processo, reduzimos a amostragem dos dados para uma frequência anual.

##R

> desemp[seq.int(de = 1, para = nrow(desemp), por = 12)]

DATA NÃO TAXA

01-01-1948 3.4

01-01-1949 4.3

01/01/1950 6,5

01/01/1951 3,7

01-01-1952 3.2

01/01/1953 2,9

01/01/1954 4,9

01/01/1955 4,9

01/01/1956 4,0

01-01-1957 4.2

Compare dados com uma frequência mais baixa. Você pode querer reduzir a resolução dos dados para poder combiná-los com outros dados de baixa frequência. Nesses casos, você provavelmente deseja agregar os dados ou reduzir a resolução, em vez de simplesmente descartar pontos. Pode ser algo simples, como uma média ou uma soma, ou algo mais complicado, como uma média ponderada, com valores posteriores recebendo mais peso.

Vimos anteriormente nos dados de doações a ideia de somar todas as doações de uma única semana, pois o valor total doado provavelmente seria mais interessante.

Em contraste, para os nossos dados económicos, o que provavelmente será interessante é uma média anual. Usamos uma média em vez de uma média móvel porque queremos resumir o ano em vez de obter o valor mais recente desse ano para enfatizar o tempo recente. (Observe a diferença da imputação de dados.) Agrupamos formatando a data em uma string, representando seu ano como um exemplo de como você pode explorar criativamente operações semelhantes a SQL para funcionalidade de série temporal:

##R

> unemp[, média(UNRATE), por = formato(DATA, "%Y")]

formato V1

1948 3,75

1949 6,05

1950 5,21

1951 3,28

1952 3.03

1953 2,93

1954 5,59

1955 4,37

1956 4,13

1957 4h30

Aumento da resolução

O upsampling não é simplesmente o inverso do downsampling. A redução da resolução faz sentido inerente como algo que pode ser feito no mundo real; é simples decidir **limpar seus dados | 53**

medir com menos frequência. Em contraste, aumentar a resolução pode ser como tentar obter algo de graça – isto é, não fazer uma medição, mas ainda assim pensar que é possível obter dados de alta resolução a partir de medições pouco frequentes. Para citar o autor do popular R

pacote de série temporal XTS :

Não é possível converter uma série de periodicidade inferior para periodicidade superior -

por exemplo, barras semanais a diárias ou diárias a 5 minutos, pois isso exigiria mágica.

No entanto, existem razões legítimas para querer rotular os dados com uma frequência superior à frequência padrão. Você simplesmente precisa ter em mente as limitações dos dados ao fazer isso. Lembre-se de que você está adicionando mais rótulos de tempo, mas não mais informações.

Vamos discutir algumas situações em que o aumento da resolução pode fazer sentido.

Séries temporais irregulares. Um motivo muito comum para aumentar a resolução é que você tem uma série temporal amostrada irregularmente e deseja convertê-la em uma série cronometrada regularmente. Esta é uma forma de upsampling porque você está convertendo todos os dados para uma frequência que provavelmente é maior do que a indicada pelas defasagens entre seus dados. Se você está fazendo upsampling por esse motivo, já sabe como fazer isso com Rolling Join, como fizemos no caso de preenchimento de dados econômicos faltantes, via R:

##R

> all.dates <- seq(from = unemp$DATE[1], to = tail(unemp$DATE, 1),

> por = "meses")

> rand.unemp = rand.unemp[J(todas.datas), roll=0]

Entradas amostradas em frequências diferentes. Às vezes, você precisa aumentar a amostragem de informações de baixa frequência simplesmente para carregá-las com suas informações de frequência mais alta em um modelo que exige que suas entradas sejam alinhadas e amostradas

contemporaneamente. Você deve estar vigilante em relação à previsão, mas se assumirmos que os estados conhecidos são verdadeiros até que um novo estado conhecido entre em cena, podemos aumentar a amostragem com segurança e levar nossos dados adiante. Por exemplo, suponha que sabemos que é (relativamente) verdade que a maioria dos novos empregos começa no primeiro dia do mês. Poderíamos decidir que nos sentimos confortáveis em utilizar a taxa de desemprego para um determinado mês indicada pelo relatório de empregos para o mês *inteiro* (sem considerá-la uma previsão porque assumimos que a taxa de desemprego permanece estável durante o mês).

##R

> daily.unemployment = unemp[J(all.dates), roll = 31]

> desemprego.diário

DATA NÃO TAXA

01-01-1948 3.4

02-01-1948 3.4

03-01-1948 3.4

04-01-1948 3.4

05-01-1948 3.4

Conhecimento de dinâmica de séries temporais. Se você tiver conhecimento básico do comportamento temporal normal de uma variável, também poderá tratar um problema de aumento da resolução como um problema de dados ausentes. Nesse caso, todas as técnicas que já discutimos ainda se aplicam. Uma interpolação é a forma mais provável de produzir novos pontos de dados, mas você precisa ter certeza de que a dinâmica do seu sistema pode justificar sua decisão de interpolação.

Conforme discutido anteriormente, o aumento e a redução da resolução ocorrerão rotineiramente mesmo no conjunto de dados mais limpo, porque quase sempre você desejará comparar variáveis de diferentes escalas de tempo. Também deve ser observado que o Pandas possui funcionalidades de aumento e redução da resolução particularmente úteis com o método de reamostragem.

Suavização de dados

A suavização de dados pode ser feita por vários motivos, e muitas vezes os dados de séries temporais do mundo real são suavizados antes da análise, especialmente para visualizações que visam contar uma história compreensível sobre os dados. Nesta seção discutiremos mais detalhadamente por que a suavização é feita, bem como a técnica de suavização de série temporal mais comum: suavização exponencial.

Objetivos de suavização

Embora a detecção de valores discrepantes seja um tópico em si, se você tiver motivos para acreditar que seus dados devem ser suavizados, poderá fazê-lo com uma média móvel para eliminar picos de medição, erros de medição ou ambos. Mesmo que os picos sejam precisos, podem não reflectir o processo subjacente e podem ser mais uma questão de problemas de instrumentação; é por isso que é bastante comum suavizar os dados.

A suavização de dados está fortemente relacionada à imputação de dados ausentes e, portanto, algumas dessas técnicas também são relevantes aqui. Por exemplo, você pode suavizar os dados aplicando uma média móvel, com ou sem antecipação, pois isso é simplesmente uma questão da

posição do ponto em relação à janela usada para calcular seu valor suavizado.

Ao suavizar, você deve pensar em uma série de questões:

• Por que você está suavizando? A suavização pode servir a vários propósitos:

Preparação de dados

Seus dados brutos são inadequados? Por exemplo, você pode conhecer valores muito altos

são improváveis ou não físicos, mas você precisa de uma maneira baseada em princípios para lidar com eles.

A suavização é a solução mais direta.

Geração de recursos

A prática de coletar uma amostra de dados, sejam elas muitas características sobre um

pessoa, imagem ou qualquer outra coisa, e resumindo com algumas métricas. Em

dessa forma, uma amostra mais completa é resumida em algumas dimensões ou em algumas características. A geração de recursos é especialmente importante para o aprendizado de máquina.

Predição

A forma mais simples de previsão para alguns tipos de processos é a reversão à média.

missão, que você obtém fazendo previsões a partir de um recurso suavizado.

Visualização

Você deseja adicionar algum sinal ao que parece ser um gráfico de dispersão barulhento? Se sim, qual é a sua intenção ao fazê-lo?

• Como os seus resultados serão afetados pela suavização ou não?

— O seu modelo assume dados ruidosos e não correlacionados, pelo que o seu modelo suave

poderia comprometer essa suposição?

— Você precisará suavizar um modelo de produção ao vivo? Nesse caso, você precisa escolher um método de suavização que não empregue antecipação.

— Você tem uma maneira de suavizar com princípios ou simplesmente fará uma hiperpara-

pesquisa na grade do medidor? Neste último caso, como você garantirá o uso de uma forma de validação cruzada com reconhecimento de tempo, de modo que os dados futuros não vazem para trás?

em tempo?

O que são hiperparâmetros?

O termo *hiperparâmetros* pode parecer muito assustador, mas são simplesmente os números que você usa para ajustar ou ajustar o desempenho de um modelo estatístico ou de aprendizado de máquina.

Para otimizar um modelo, você tenta diversas variações dos hiperparâmetros desse modelo, geralmente realizando uma *pesquisa em grade* para identificar esses parâmetros. Uma pesquisa em grade é exatamente isso: você constrói uma "grade" de todas as combinações possíveis de hiperparâmetros em um espaço de pesquisa e testa todas elas.

Por exemplo, imagine que um algoritmo usa hiperparâmetros A e B, onde pensamos que os valores razoáveis para A são 0, 1 ou 2, e pensamos que os valores

razoáveis para B são 0,7, 0,75 ou 0,8. Então teríamos nove opções possíveis de hiperparâmetros para nossa grade: cada combinação possível de valores potenciais de A (dos quais existem 3) com todos os valores possíveis de B (dos quais existem 3) para um total de 3 × 3 = 9 possibilidades. Como você pode ver, o ajuste de hiperparâmetros pode rapidamente se tornar bastante trabalhoso.

Suavização exponencial

Freqüentemente, você não desejará tratar todos os pontos de tempo igualmente ao suavizar. Em particular, você pode querer tratar os dados mais recentes como dados mais informativos; nesse caso, a suavização exponencial é uma boa opção. Em contraste com a média móvel que vimos antes – onde cada ponto onde faltavam dados poderia ser imputado à média de **56 | Capítulo 2: Encontrando e organizando dados de séries temporais**

seus pontos circundantes – a suavização exponencial é voltada para ser mais consciente temporalmente, ponderando os pontos mais recentes mais elevados do que os pontos menos recentes. Portanto, para uma determinada janela, o ponto mais próximo no tempo tem o maior peso e cada ponto anterior no tempo tem um peso exponencialmente menor (daí o nome).

A mecânica da suavização exponencial funciona da seguinte maneira. Para um determinado período de tempo t, você encontra o valor suavizado de uma série calculando:

Valor suavizado no tempo $t = St = d \times ; St$ -1 + (1 – d) × xt Pense em como isso se propaga ao longo do tempo. O valor suavizado no tempo (t – 1) é em si um produto da mesma coisa:

St -1 = $d \times St$ -2 + (1 – d) × xt -1

Portanto, podemos ver uma expressão mais complexa para o valor suavizado no tempo t : $d \times$ ($d \times St$ –2 + (1 – d) × xt -1) + (1 - d) × xt Leitores com inclinação matemática perceberão que nós tem uma série da forma: d 3 × xt -3 + d 2 × xt -2 + $d \times xt$ -1

Na verdade, é graças a esta forma que as médias móveis exponenciais são bastante tratáveis.

Mais detalhes estão amplamente disponíveis online e em livros didáticos; veja meus resumos favoritos

em “Mais recursos” na página 69.

Ilustrarei a suavização em Python, já que o Pandas inclui uma variedade de opções de suavização. As opções de suavização também estão amplamente disponíveis em R, incluindo base R, bem como em muitos pacotes de séries temporais.

Embora estejamos analisando os dados da taxa de desemprego nos EUA, passaremos para outro conjunto de dados comumente usado: o conjunto de dados de passageiros de companhias aéreas (que remonta ao famoso livro de séries temporais de Box e Jenkins e está amplamente disponível). O conjunto de dados original é uma conta dos milhares de passageiros mensais de companhias aéreas divididos por mês:

Pitão

>>> ar

Data Passageiros

0 1949-01 112

1 1949-02 118

2 1949-03 132

3 1949-04 129

4 1949-05 121

5 1949-06 135

6 1949-07 148

7 1949-08 148

8 1949-09 136

9 1949-10 119

10 1949-11 104

Podemos suavizar facilmente os valores dos passageiros usando uma variedade de fatores de decaimento e aplicando a função Pandas ewma(), assim:

Pitão

>>> ar['Smooth.5'] = pd.ewma(ar, alfa = 0,5).Passageiros

>>> ar['Smooth.9'] = pd.ewma(ar, alfa = 0,9).Passageiros

Como podemos ver, o nível do parâmetro alfa, também chamado de *fator de suavização* , afeta o quanto o valor é atualizado para seu valor atual versus a retenção de informações da média existente. Quanto maior o valor alfa, mais rapidamente o valor é atualizado para mais perto do seu preço atual. O Pandas aceita vários parâmetros, todos inseridos na mesma equação, mas oferecem diversas maneiras de pensar na especificação de uma média móvel exponencial. 3

Pitão

>>> ar

Data Passageiros Suave.5 Suave.9

0 1949-01 112 112,000000 112,000000

1 1949-02 118 116,000000 117,454545

2 1949-03 132 125.142857 130.558559

3 1949-04 129 127.200000 129.155716

4 1949-05 121 124,000000 121,815498

5 1949-06 135 129.587302 133.681562

6 1949-07 148 138.866142 146.568157

7 1949-08 148 143.450980 147.856816

8 1949-09 136 139.718200 137.185682

9 1949-10 119 129.348974 120.818568

10 1949-11 104 116.668295 105.681857

No entanto, a suavização exponencial simples não funciona bem (para previsão) no caso de dados com tendência de longo prazo. O Método de Holt e a suavização de Holt-Winters são dois métodos de suavização exponencial aplicados a dados com tendência, ou com tendência e sazonalidade.

Existem muitas outras técnicas de suavização amplamente utilizadas. Por exemplo, os filtros de Kalman suavizam os dados modelando um processo de série temporal como uma combinação de dinâmica conhecida e erro de medição. LOESS (abreviação de “suavização de gráfico de dispersão estimada localmente”) é um método não paramétrico de suavização local de dados. Esses métodos, e outros, oferecem gradualmente maneiras mais complexas de compreender a suavização, mas com maior custo computacional. É digno de nota que Kalman e LOESS incorporam dados 3 Você pode especificar o fator de suavização alfa, a meia-vida, a extensão ou o centro de massa de acordo com o qual você se sente confortável. Detalhes estão disponíveis na documentação.

mais cedo e mais tarde, portanto, se você usar esses métodos, tenha em mente o vazamento de informações retroativas no tempo, bem como o fato de que eles geralmente não são apropriados para preparar dados a serem usados em aplicações de previsão.

A suavização é uma forma de previsão comumente usada, e você pode usar uma série temporal suavizada (sem antecipação) como um modelo nulo fácil ao testar se um método mais sofisticado está realmente produzindo uma previsão bem-sucedida.

Como iniciar um cálculo de suavização

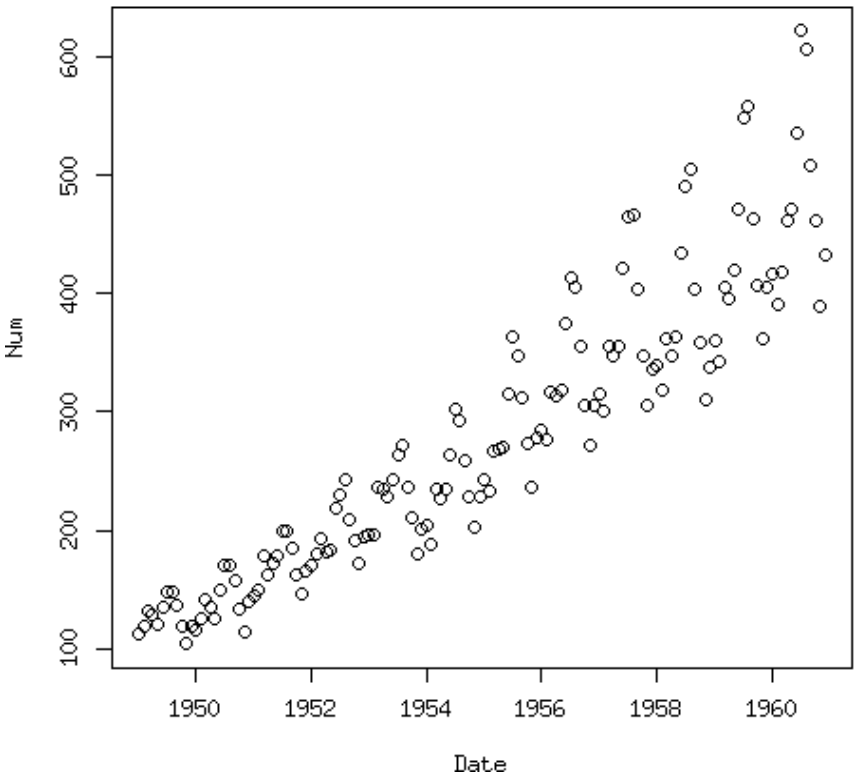
Um motivo importante para usar uma função de suavização exponencial empacotada é que é difícil acertar o início de sua série suavizada. Imagine que o primeiro ponto da sua série seja igual a 3 e o segundo ponto seja igual a 6. Imagine também que o seu fator de desconto (que você define de acordo com a rapidez com que deseja que a média se adapte às informações mais recentes) seja igual a 0,7, de modo que você calcularia o segundo ponto da sua série como:

$$3 \times 0{,}7 + 6 \times (1 - 0{,}7) = 3{,}9$$

Na verdade, você estaria errado. A razão é que multiplicar seu fator de desconto de 0,7 por 3 pressupõe implicitamente que 3 é a soma do seu conhecimento voltando no tempo até o infinito, em vez de um mero ponto de dados que você acabou de medir. Portanto, pesa indevidamente 3 em relação a 6, como se 3 tivesse um conjunto de conhecimentos significativamente mais antigo do que realmente tem.

A álgebra para derivar a expressão correta é muito complicada para uma barra lateral curta, mas verifique os recursos adicionais no final deste livro para obter mais detalhes. Enquanto isso, deixarei a orientação geral de que 6 deve ser pesado um pouco mais do que aqui e 3 um pouco menos. À medida que você avança em uma série temporal, seus valores mais antigos devem ser ponderados mais próximos de seu verdadeiro fator de desconto de 0,7.

Quando isso acontecerá pode ser visto na álgebra, então dê uma olhada nos recursos adicionais se estiver interessado.



Dados sazonais

Sazonalidade nos dados é qualquer tipo de comportamento recorrente em que a frequência do comportamento é estável. Pode ocorrer em muitas frequências diferentes ao mesmo tempo. Por exemplo, o comportamento humano tende a ter uma sazonalidade diária (almoço no mesmo horário todos os dias), uma sazonalidade semanal (as segundas-feiras são semelhantes às outras segundas-feiras) e uma sazonalidade anual (o dia de Ano Novo tem pouco tráfego). Os sistemas físicos também demonstram sazonalidade, como o período que a Terra leva para girar em torno do sol.

Identificar e lidar com a sazonalidade faz parte do processo de modelagem. Por outro lado, é também uma forma de limpeza de dados, como o economicamente importante US Jobs

Relatório . Na verdade, muitas estatísticas governamentais, especialmente as económicas, são dessazonalizadas na sua forma divulgada.

Para ver o que a suavização de dados sazonais pode fazer, voltamos ao conjunto de dados canônicos sobre a contagem de passageiros de companhias aéreas. Um gráfico revela rapidamente que se trata de dados altamente sazonais, mas apenas se você fizer o gráfico correto.

Observe a diferença entre usar o gráfico padrão do R (que usa pontos; consulte

Figura 2-10) versus adicionar o argumento para indicar que você deseja uma linha

(Figura 2-11).

Figura 2-10. A média e a variância crescentes dos dados são aparentes a partir de uma dispersão gráfico, mas não vemos uma tendência sazonal óbvia.



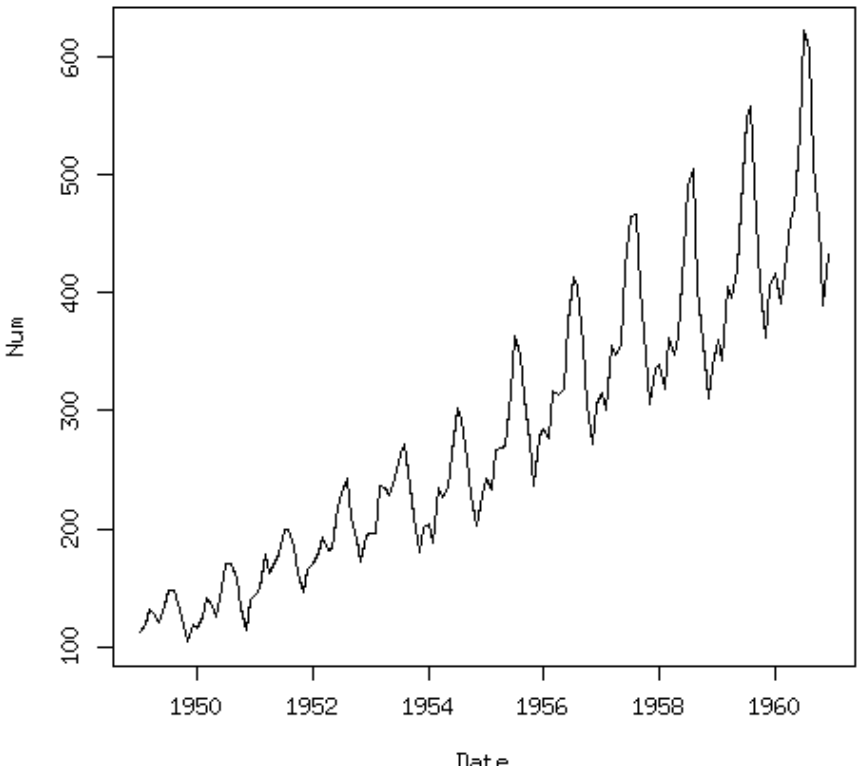


Figura 2-11. Um gráfico de linhas deixa a sazonalidade bastante clara.

Se você estivesse olhando apenas para o gráfico R padrão, a natureza sazonal dos dados poderia lhe escapar. Esperançosamente, isso não permanecerá assim por muito tempo, pois sem dúvida você também faria coisas para explorar seus dados de forma mais completa, talvez com um gráfico de autocorrelação

(discutido no Capítulo 3) ou outros diagnósticos.

Os humanos são criaturas de hábitos

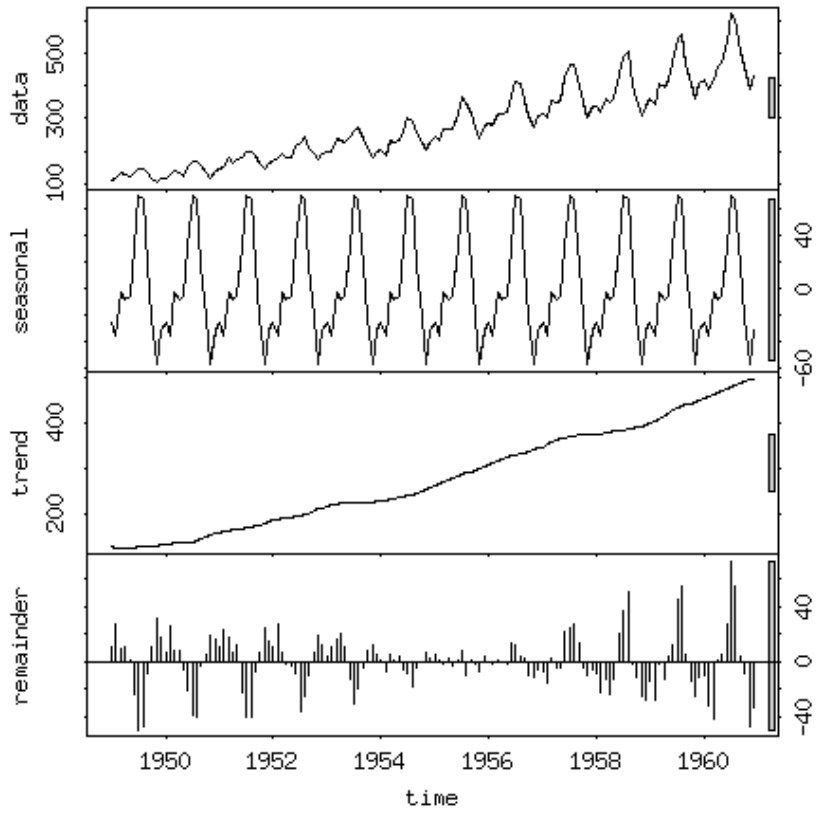
Com os dados do comportamento humano há quase sempre alguma forma de

sazonalidade, mesmo com vários ciclos (um padrão horário, um semanal

padrão, um padrão verão-inverno, etc.).

O gráfico de dispersão mostra algumas informações mais claramente do que o gráfico de linhas. A variância dos nossos dados está a aumentar, tal como a média, o que é mais óbvio quando vemos uma nuvem de pontos de dados a irradiar para fora numa forma cónica, inclinada para cima. Esses dados claramente têm uma tendência e, portanto, provavelmente os transformaríamos com uma transformação logarítmica ou diferenciação, dependendo das demandas do nosso modelo. Esses dados também apresentam claramente uma tendência de aumento da variância. Falaremos mais sobre transformações de dados específicas do modelo em dados sazonais nos capítulos de modelagem, por isso não entraremos em detalhes aqui.

Também obtemos informações úteis além das evidências de sazonalidade do gráfico linear. Obtemos informações sobre que *tipo* de sazonalidade. Ou seja, vemos que os dados não são apenas sazonais, mas sazonais de forma multiplicativa. À medida que os valores gerais aumentam, o mesmo acontece com as oscilações da estação (pense nisso como um dimensionamento do pico ao vale).



Podemos decompor os dados em seus componentes sazonais, de tendência e restantes com muita facilidade, conforme mostrado aqui com apenas uma linha de R:

##R

> plot(stl(AirPassengers, "periódico"))

O gráfico resultante parece muito sensato com base nos dados originais (ver Figura 2-12).

Podemos imaginar a adição dos dados sazonais, de tendência e restantes para obter a série original. Podemos também ver que esta decomposição particular não teve em conta o facto de esta série apresentar uma sazonalidade multiplicativa em vez de aditiva porque os resíduos são maiores no início e no final da série temporal. Parece que esta decomposição se baseou na variância sazonal média como a variância da componente sazonal.

Figura 2-12. Uma decomposição da série temporal original em um componente sazonal, um tendência e os resíduos. Preste atenção ao eixo y de cada gráfico, pois eles são bem diferentes.

Observe que este é o motivo das barras cinzas no lado direito de cada gráfico. Esses cinza as barras têm todas o mesmo tamanho absoluto (em unidades do eixo y), de modo que sua exibição relativamente diferente é um lembrete visual das diferentes escalas do eixo y em diferentes componentes.

Para obter uma compreensão inicial de como isso funciona, podemos dar uma olhada no R oficial

documentação:

A componente sazonal é encontrada por LOESS suavizando as subséries sazonais (a série de todos os valores de janeiro, ...). Se s.window = “periódico”, a suavização é efetivamente substituída pela média. Os valores sazonais são removidos e o restante suavizado para encontrar a tendência. O nível global é retirado da componente sazonal e adicionado à

componente de tendência. Este processo é iterado algumas vezes. O componente restante são os resíduos do ajuste sazonal mais a tendência.

LOESS, apresentado anteriormente, é um método computacionalmente exigente para suavizar pontos de dados que envolve uma janela móvel para estimar o valor suavizado de cada ponto com base em seus vizinhos (espero que seu alarme antecipado esteja soando!).

Dados sazonais e dados cíclicos

Séries temporais sazonais são séries temporais nas quais os comportamentos se repetem durante um período fixo.

Pode haver múltiplas periodicidades que refletem diferentes ritmos de sazonalidade, como a sazonalidade do dia de 24 horas versus a estação do calendário de 12 meses, ambas apresentando características fortes na maioria das séries temporais relacionadas ao comportamento humano.

As séries temporais cíclicas também apresentam comportamentos recorrentes, mas possuem período variável.

Um exemplo comum é um ciclo económico, como os ciclos de expansão e recessão do mercado de ações, que têm uma duração incerta. Da mesma forma, os vulcões apresentam comportamentos cíclicos, mas não sazonais. Conhecemos os períodos aproximados de erupção, mas estes não são precisos e variam ao longo do tempo.

Fusos horários

Os fusos horários são intrinsecamente tediosos, dolorosos e difíceis de acertar, mesmo com muito esforço. É por isso que você nunca deve usar sua própria solução. Desde a sua invenção, os fusos horários tornaram-se complicados, e ficaram ainda mais complicados com o advento dos computadores pessoais. Há muitas razões para isto:

• Os fusos horários são moldados por decisões políticas e sociais.

• Não existe uma forma padrão de transportar informações de fuso horário entre idiomas ou através de um protocolo HTTP.

• Não existe um protocolo único para nomear fusos horários ou para determinar as datas de início e término das compensações de horário de verão.

• Devido ao horário de verão, alguns horários ocorrem duas vezes por ano em seus fusos horários!

A maioria das linguagens depende do sistema operacional subjacente para obter informações de fuso horário. Infelizmente, a função integrada de recuperação automática de hora em Python, date time.datetime.now(), não retorna um carimbo de data/hora com reconhecimento de fuso horário. Parte disso ocorre intencionalmente. Algumas decisões tomadas na biblioteca padrão incluem proibir informações de fuso horário no módulo datetime (porque essas informações mudam com frequência) e permitir objetos datetime com e sem informações de fuso horário. No entanto, comparar um objeto com fuso horário com outro sem fuso horário causará um TypeError.

Alguns blogueiros afirmam que a maioria das bibliotecas são escritas com a suposição de que tzinfo==None. Embora esta seja uma afirmação difícil de fundamentar, é consistente com muita experiência. As pessoas também relatam dificuldade em decapagem de objetos com carimbo de fuso horário, então isso também é algo para verificar com antecedência se você planeja usar decapagem. [4]

Então, vamos dar uma olhada em como trabalhar com fusos horários em Python. As principais bibliotecas que você provavelmente usará são datetime, pytz e dateutil. Além disso, o Pandas oferece funcionalidades convenientes relacionadas ao fuso horário, criadas com base nas duas últimas bibliotecas.

Abordaremos a funcionalidade de fuso horário mais importante a seguir.

Primeiro, observe que quando você recupera um “agora” do módulo datetime, ele não vem com informações de fuso horário, embora forneça a hora apropriada para seu fuso horário. Observe, por exemplo, a diferença nas respostas para now() e utcnow():

Pitão

>>> datetime.datetime.utcnow()

datahora.datehora(2018, 5, 31, 14, 49, 43, 187680)

>>> datetime.datetime.now()

datahora.datehora(2018, 5, 31, 10, 49, 59, 984947)

>>> *# como podemos ver, meu computador não retorna UTC*

>>> *# mesmo que não haja fuso horário anexado*

>>> datetime.datetime.now(datetime.timezone.utc)

datetime.datetime(2018, 5, 31, 14, 51, 35, 601355,

tzinfo=datetime.timezone.utc)

Observe que se passarmos em um fuso horário, obteremos as informações corretas, mas este não é o comportamento padrão. Para trabalhar com fusos horários em Python, criamos um objeto de fuso horário, como ocidental, como o fuso horário do Pacífico dos EUA:

Pitão

>>> ocidental = pytz.timezone('EUA/Pacífico')

>>> zona ocidental.

'EUA/Pacífico'

Podemos então usar esses objetos para localizar um fuso horário da seguinte maneira:

Pitão

>>> ## a API oferece suporte a duas maneiras de criar um horário com reconhecimento de fuso horário,

>>> ## via 'localize' ou para converter um fuso horário de uma localidade

>>> ## para outro

>>> # aqui localizamos

>>> loc_dt = western.localize(datetime.datetime(2018, 5, 15, 12, 34, 0)) 4 Pickling é o processo de armazenamento de objetos Python em formato de byte. Isso é feito por meio do popular módulo pickle integrado. Na maioria das vezes, a decapagem funcionará bem, mas às vezes os objetos relacionados ao tempo podem ser difíceis de decapagem.

datahora.datehora(2018, 5, 15, 12, 34,

tzinfo=<DstTzInfo 'EUA/Pacífico' PDT-1 dia, 17:00:00 DST>)

Observe, entretanto, que passar o fuso horário diretamente para o construtor datetime muitas vezes não produzirá o resultado que esperávamos:

Pitão

>>> londres_tz = pytz.timezone('Europa/Londres')

>>> londres_dt = loc_dt.astimezone(london_tz)

>>> londres_dt

datahora.datehora(2018, 5, 15, 20, 34,

tzinfo=<DstTzInfo 'Europa/Londres' BST+1:00:00 DST>)

>>> f = '%Y-%m-%d %H:%M:%S %Z%z'

>>> datetime.datetime(2018, 5, 12, 12, 15, 0,

tzinfo = londres_tz).strftime(f)

'12/05/2018 12:15:00 LMT-0001'

>>> ## conforme destacado na documentação do pytz usando o tzinfo de

>>> ## o inicializador datetime.datetime nem sempre leva ao

>>> ## resultado desejado, como no exemplo de Londres

>>> ## de acordo com a documentação do pytz, este método leva a

>>> ## os resultados desejados em fusos horários sem horário de verão Isso é importante, como quando você está calculando deltas de horário. O primeiro exemplo dos três seguintes é a pegadinha:

Pitão

>>> # geralmente você deseja armazenar dados em UTC e converter apenas quando

>>> # gerando saída legível por humanos

>>> # você também pode fazer aritmética de datas com fusos horários

>>> evento1 = datetime.datetime(2018, 5, 12, 12, 15, 0,

tzinfo = londres_tz)

>>> evento2 = datetime.datetime(2018, 5, 13, 9, 15, 0,

tzinfo = ocidental)

```
>>> evento2 - evento1

>>> ## isso produzirá o delta de horário errado porque os
fusos horários

>>> ## não foram rotulados corretamente

>>> evento1 = london_tz.localize(

datahora.datehora(2018, 5, 12, 12, 15, 0))

>>> evento2 = western.localize(

datahora.datehora(2018, 5, 13, 9, 15, 0))

>>> evento2 - evento1

>>> evento1 = london_tz.localize(

(datetime.datetime(2018, 5, 12, 12, 15, 0))).

astimezone(datetime.timezone.utc)

>>> evento2 = western.localize(

datetime.datetime(2018, 5, 13, 9, 15, 0)).
```



```
astimezone(datetime.timezone.utc)

>>> evento2 - evento1
```

pytz fornece uma lista de fusos horários comuns e fusos horários por país, os quais podem ser referências úteis:

```
## Pitão

## dê uma olhada em pytz.common_timezones

>>> pytz.common_timezones
```

(saída longa...)

ou específico do país

>>> pytz.country_timezones('RU')

['Europa/Kaliningrado', 'Europa/Moscou',
'Europa/Simferopol',

'Europa/Volgograd', 'Europa/Kirov', 'Europa/Astrakhan',

'Europa/Saratov', 'Europa/Ulyanovsk', 'Europa/Samara',

'Ásia/Ecaterimburgo', 'Ásia/Omsk', 'Ásia/Novosibirsk',

'Ásia/Barnaul', 'Ásia/Tomsk', 'Ásia/Novokuznetsk',

'Ásia/Krasnoyarsk', 'Ásia/Irkutsk', 'Ásia/Chita',

'Ásia/Yakutsk', 'Ásia/Khandyga', 'Ásia/Vladivostok',

'Ásia/Ust-Nera', 'Ásia/Magadan', 'Ásia/Sakhalin',

'Ásia/Srednekolymsk', 'Ásia/Kamchatka', 'Ásia/Anadyr']

>>>

>>> pytz.country_timezones('fr')

>>> ['Europa/Paris']

Uma questão particularmente complicada é a questão do horário de verão. Certos tempos legíveis por humanos existem duas vezes (ficando para trás no outono), enquanto outros nem existem (avançam na primavera):

Pitão

>>> ## fusos horários

>>> ambig_time = western.localize(

datetime.datetime(2002, 10, 27, 1, 30, 00)).

astimezone(datetime.timezone.utc)

>>> ambig_time_earlier = ambig_time -
datetime.timedelta(horas=1)

>>> ambig_time_later = ambig_time +
datetime.timedelta(horas=1)

>>> ambig_time_earlier.astimezone(oeste)

>>> ambig_time.astimezone(oeste)

>>> ambig_time_later.astimezone(oeste)

>>> *#resultados nesta saída*

datahora.datehora(2002, 10, 27, 1, 30,

tzinfo=<DstTzInfo 'EUA/Pacífico' PDT-1 dia, 17:00:00 DST>)

datahora.datehora(2002, 10, 27, 1, 30,

tzinfo=<DstTzInfo 'EUA/Pacífico' PST-1 dia, 16:00:00 STD>)

datahora.datehora(2002, 10, 27, 2, 30,

tzinfo=<DstTzInfo 'EUA/Pacífico' PST-1 dia, 16:00:00 STD>)

>>> *# observe que os dois últimos timestamps são idênticos, não adianta!*

>>> *## neste caso você precisa usar is_dst para indicar se a luz do dia*

>>> *## economia está em vigor*

>>> ambig_time = western.localize(

datetime.datetime(2002, 10, 27, 1, 30, 00), is_dst = Verdadeiro).

astimezone(datetime.timezone.utc)

>>> ambig_time_earlier = ambig_time - datetime.timedelta(horas=1)

>>> ambig_time_later = ambig_time + datetime.timedelta(horas=1)

>>> ambig_time_earlier.astimezone(oeste)

>>> ambig_time.astimezone(oeste)

>>> ambig_time_later.astimezone(oeste)

datahora.datehora(2002, 10, 27, 0, 30,

tzinfo=<DstTzInfo 'EUA/Pacífico' PDT-1 dia, 17:00:00 DST>)

datahora.datehora(2002, 10, 27, 1, 30,

tzinfo=<DstTzInfo 'EUA/Pacífico' PDT-1 dia, 17:00:00 DST>)

datahora.datehora(2002, 10, 27, 1, 30,

tzinfo=<DstTzInfo 'EUA/Pacífico' PST-1 dia, 16:00:00 STD>)

observe que agora não temos o mesmo tempo acontecendo duas vezes.

pode parecer assim até você verificar o deslocamento do UTC

As preocupações com o fuso horário podem não ser importantes para o seu trabalho, portanto a utilidade desse conhecimento dependerá da natureza dos seus dados. Há certamente situações em que fazer isto de forma errada pode ser catastrófico (por exemplo, gerar previsões meteorológicas para aviões comerciais que voam durante a mudança de horário e subitamente encontrar as suas posições drasticamente alteradas).

Prevenindo a antecipação

Lookahead é perigosamente fácil de introduzir em um pipeline de modelagem, especialmente com interfaces de manipulação de dados funcionais vetorizadas, como aquelas oferecidas por R e Python. É fácil deslocar uma variável na direção errada, deslocá-la mais ou menos do que o pretendido ou, de outra forma, encontrar-se com dados que não são totalmente "honestos", pois você tem dados disponíveis antes de serem conhecidos em seu sistema.

Infelizmente, não existe um diagnóstico estatístico definitivo para o futuro – afinal, todo o esforço da análise de séries temporais consiste em modelar o desconhecido. A menos que um sistema seja um tanto determinístico com leis dinâmicas conhecidas, pode ser difícil distinguir um modelo muito bom de um modelo com previsão - isto é, até que você coloque um modelo em produção e perceba que estão faltando dados quando planejou ter isso, ou simplesmente que seus resultados na produção não refletem o que você vê durante o treinamento.

A melhor forma de evitar esse constrangimento é a vigilância constante. Sempre que você estiver mudando o tempo de dados, suavizando dados, imputando dados ou aumentando a resolução de dados, pergunte-se se você poderia saber algo em um determinado momento. Lembre-se de que isso não inclui apenas o tempo do calendário. Também inclui intervalos de tempo realistas para refletir quanto tempo existe entre algo acontecer e a sua organização ter esses dados disponíveis. Por exemplo, se sua organização acessa o Twitter apenas semanalmente para coletar dados de análise de sentimento, você precisa incluir essa periodicidade semanal em sua segmentação de dados de treinamento e validação. Da mesma forma, se você puder treinar novamente seu modelo apenas uma vez, um aviso

mês, você precisa descobrir qual modelo se aplicaria a quais dados ao longo do tempo. Você não pode, por exemplo, treinar um modelo para julho e depois aplicá-lo em julho para teste, porque em uma situação real, você não teria esse modelo treinado a tempo se o treinamento demorasse muito.

Aqui estão algumas outras idéias para usar como uma lista de verificação geral. Tenha-os em mente ao planejar a construção de um modelo e ao auditar seu processo após o fato:

• Se você estiver suavizando dados ou imputando dados faltantes, pense cuidadosamente sobre

se isso pode impactar seus resultados ao introduzir uma previsão. E não pense apenas nisso - experimente como fizemos anteriormente e veja como as imputações e

trabalho de alisamento. Eles parecem estar olhando para o futuro? Se sim, você pode justificar seu uso? (Provavelmente não.)

• Construa todo o seu processo com um conjunto de dados muito pequeno (apenas algumas linhas em um data.table ou algumas etapas de tempo de linha em qualquer formato de dados). Em seguida, faça verificações aleatórias em cada etapa do processo e veja se você acidentalmente transfere alguma informação temporariamente para um local inadequado.

• Para cada tipo de dado, descubra qual é o atraso em relação ao seu próprio carimbo de data/hora.

Por exemplo, se o carimbo de data e hora for quando os dados “aconteceram”, mas não quando foram carregados em seus servidores, você precisa saber disso. Diferentes colunas de um quadro de dados podem ter atrasos diferentes. Para resolver isso, você pode personalizar seu atraso por quadro de dados ou (melhor e mais realista) escolher o maior atraso e aplicá-lo a tudo. Embora você não queira pessimizar indevidamente seu modelo, é um bom ponto de partida, após o qual você pode relaxar essas regras excessivamente restritas, uma de cada vez, com cuidado!

• Use testes de erros (contínuos) com reconhecimento de tempo ou validação cruzada. Isso será discutido

no Capítulo 11 , mas lembre-se de que randomizar seus conjuntos de dados de treinamento versus conjuntos de

dados de teste não funciona com dados de série temporal. Você não quer que informações do futuro vazem para modelos do passado.

• Introduza intencionalmente uma análise antecipada e veja como seu modelo se comporta. Experimente vários graus de antecipação para ter uma ideia de como isso altera a precisão. Se você tem alguma ideia da precisão da previsão, você tem uma ideia do que o teto de um modelo real fará sem um conhecimento injusto do futuro. Lembre-se de que muitos problemas de séries temporais são extremamente difíceis; portanto, um modelo com antecipação pode parecer ótimo até você perceber que está lidando com um conjunto de dados de alto ruído/baixo sinal.

• Adicione recursos lentamente, especialmente recursos que você possa estar processando, para poder procurar saltos. Um sinal de antecipação é quando um recurso específico é inesperadamente bom e não há uma explicação muito boa. No topo da sua lista de explicações deve estar sempre “antecipado”.



Processar e limpar dados relacionados ao tempo pode ser uma tarefa tediosa e

processo detalhado.

Há um perigo tremendo na limpeza e processamento de dados de

apresentando uma *visão antecipada* ! Você deve ter previsões apenas se eles

são intencionais e isso raramente é apropriado.

Mais recursos

• Sobre dados faltantes:

Steffen Moritz et al., "Comparação de diferentes métodos para séries temporais univariadas

Imputação em R," *artigo de pesquisa não publicado, 13 de outubro de 2015, https://perma.cc/*

M4LJ-2DFB.

Este resumo completo de 2015 descreve os métodos disponíveis para imputar

dados de série temporal no caso de dados de série temporal univariados. Dados univariados de séries temporais são um desafio particular porque muitos dados faltantes avançados

métodos de imputação baseiam-se na observação de distribuições entre covariáveis, uma

opção que não está disponível no caso de uma série temporal univariada. Este artigo resume a usabilidade e o desempenho de vários pacotes R como

bem como resultados empíricos dos métodos disponíveis em uma variedade de conjuntos de dados.

James Honaker e Gary King, "O que fazer com valores ausentes em séries temporais

Dados transversais," *American Journal of Political Science 54, não. 2 (2010): 561–*

81, https://perma.cc/8ZLG-SMSX.

Este artigo explora as práticas recomendadas para dados ausentes em séries temporais com amplos painéis de covariáveis.

Léo Belzile, "Notas sobre séries temporais irregulares e valores ausentes", nd https://

perma.cc/8LHP-92FP.

O autor fornece exemplos de como trabalhar com dados irregulares como dados faltantes

problema de dados e uma visão geral de alguns pacotes R comumente usados.

• Nos fusos horários:

Tom Scott, O problema com horário e fusos horários , Vídeo Computerphile, dezembro 30, 2013, https://oreil.ly/iKHkp.

Este vídeo de 10 minutos no YouTube com mais de 1,5 milhão de visualizações descreve os perigos

e desafios de lidar com fusos horários, particularmente no contexto de um

aplicativo da web.

Wikipédia, "Fuso Horário", https://perma.cc/J6PB-232C.

Esta é uma história fascinante resumida que dá algumas dicas sobre como a cronometragem funcionava antes do século passado e como os avanços tecnológicos (começando com a ferrovia) levaram à crescente necessidade de pessoas em

locais diferentes para coordenar seu tempo. Existem também alguns mapas divertidos de fusos horários.

Declan Butler, "Falha no GPS ameaça milhares de instrumentos científicos",

Natureza, 3 de abril de 2019, https://perma.cc/RPT6-AQBC.

Este artigo não está diretamente relacionado aos fusos horários, mas ao problema de carimbo de data / hora de forma mais geral. Ele descreve um problema recente em que um bug no Sistema de Posicionamento Global dos EUA

poderia causar um problema com dados com carimbo de data/hora

porque transmite um “número da semana” binário de 10 dígitos que começou a contar

a partir de 6 de janeiro de 1980. Este sistema pode cobrir apenas 1.024 semanas no total (2 elevadas à potência de 10). Esta contagem foi alcançada pela segunda vez em abril

2019. Dispositivos que não foram projetados para acomodar essa limitação são redefinidos para zero e dados científicos e industriais com registro de data e hora incorretos. Este artigo descreve algumas das dificuldades com uma representação bastante limitada do tempo combinada com dispositivos científicos que não foram preparados para o futuro para dar conta deste problema.

• Sobre suavização e sazonalidade:

Rob J. Hyndman e George Athanasopoulos, “Suavização Exponencial”, em Fore-

elenco: Princípios e Práticas, 2ª ed. (Melbourne: OTexts, 2018), https://

perma.cc/UX4K-2V5N.

Este capítulo do livro acadêmico introdutório de Hyndman e Athanasopoulos

O livro cobre métodos de suavização exponencial para dados de séries temporais, incluindo

uma taxonomia útil de suavização exponencial e extensões de expo-

suavização essencial para aplicações de previsão.

David Owen, “A maneira correta de iniciar uma média móvel exponencial”, Blog Forward Motion, 31 de janeiro de 2017, https://perma.cc/ZPJ4-DJJK.

Conforme destacado anteriormente em uma nota lateral, as médias móveis exponenciais são

conceitualmente simples e fácil de calcular, com a complicação de que como

“iniciar” o cálculo é um pouco complicado. Queremos ter certeza de que nossa média móvel se adapta às novas informações de uma forma que reconheça como

há muito tempo que registra informações. Se iniciarmos uma média móvel sem levar isso em consideração, mesmo uma nova média móvel se comportará como

embora tenha um retrospecto infinito e desconsidere indevidamente novas relações de informação

tivo ao que deveria fazer. Mais detalhes e uma solução computacional são apresentados nesta postagem do blog.

70 | Capítulo 2: Encontrando e organizando dados de séries temporais

Avner Abrami, Aleksandr Arovkin e Younghun Kim, “Séries Temporais Usando Expo-

nential Smoothing Cells”, artigo de pesquisa não publicado, revisado pela última vez em 29 de setembro de

2017, https://perma.cc/2JRX-K2JZ.

Este é um artigo de pesquisa de série temporal extremamente acessível que elabora a ideia de suavização exponencial simples para desenvolver “suavização exponencial

células.”

- Na análise de dados funcionais:

Jane-Ling Wang, Jeng-Min Chiou e Hans-Georg Müller, “Revisão de Funcional

Análise de dados," Revisões Anuais de Estatística e sua Aplicação, 2015, https://

perma.cc/3DNT-J9EZ.

Este artigo de estatística oferece uma visão matemática acessível de

importantes técnicas de análise de dados funcionais. Existem também técnicas úteis de visualização ilustradas ao longo do artigo.

Shahid Ullah e Caroline F. Finch, "Aplicações de Análise Funcional de Dados: A

Revisão sistemática," Metodologia de Pesquisa Médica BMCv, 13, no. 43 (2013), https://perma.cc/VGK5-ZEUX.

Esta revisão adota uma abordagem multidisciplinar para pesquisar pesquisas publicadas recentemente

análises estabelecidas usando análise funcional de dados. Os autores argumentam que

técnicas de análise funcional de dados têm uma aplicabilidade muito mais ampla do que é atualmente considerada e defendem que as ciências biológicas e da saúde poderiam se beneficiar de um maior uso dessas técnicas para analisar o tempo médico

dados da série.

Mais recursos | 71

CAPÍTULO 3

Análise Exploratória de Dados para Séries Temporais

Dividiremos nossa discussão sobre análise exploratória de dados para séries temporais em duas seções distintas. Primeiro, discutimos a aplicação de métodos de dados comumente usados em séries temporais. Em particular, discutimos como histogramas, gráficos e operações de

agrupamento podem ser aplicados a dados de séries temporais.

Em segundo lugar, destacamos métodos fundamentalmente temporais para análise de séries temporais - isto é, métodos desenvolvidos especificamente para dados de séries temporais e que fazem sentido apenas no contexto de pontos de dados que têm uma relação temporal entre si (em vez de uma relação transversal). .

Métodos familiares

Começamos pensando em como aplicar técnicas de exploração de dados comumente usadas a conjuntos de dados de séries temporais. O processo é o mesmo que você executou em dados que não são de série temporal, para começar.

Você desejará saber as colunas disponíveis, seus intervalos de valores e quais unidades lógicas de medida funcionam melhor. Você desejará abordar as mesmas questões exploratórias que faria sobre qualquer novo conjunto de dados, como:

• Alguma das colunas está fortemente correlacionada entre si?

• Qual é a média global de uma variável interessante? Qual é a sua variação?

Para responder a essas perguntas, você pode usar técnicas familiares, como plotagem, obtenção de estatísticas resumidas, aplicação de histogramas e uso de gráficos de dispersão direcionados. Você também vai querer responder a perguntas explicitamente orientadas para o tempo, como:

• Qual é o intervalo de valores que você vê e eles variam de acordo com o período de tempo ou com alguma outra unidade lógica de análise?

• Os dados parecem consistentes e medidos uniformemente ou sugerem mudanças na medição ou no comportamento ao

longo do tempo?

Para resolver isso, você desejará que os métodos mencionados anteriormente – histogramas, gráficos de dispersão e estatísticas resumidas – levem em consideração o eixo temporal. Para implementar este comportamento, iremos incorporar o tempo nas nossas estatísticas, como um eixo nos nossos gráficos ou como um grupo em operações de agrupamento, como as aplicadas a histogramas ou gráficos de dispersão.

No restante desta seção, veremos exemplos de análise exploratória de dados com alguns tipos diferentes de dados de séries temporais para explorar como podemos usar métodos não temporais tradicionais diretamente ou com modificações específicas de tempo. Para demonstrar estes métodos exploratórios, utilizaremos dados dos mercados de ações europeus, um conjunto de dados de séries temporais disponíveis na base R.

Operações de agrupamento

Com dados de séries temporais, existem grupos assim como existem com dados não temporais, portanto, o pensamento de grupo por estilo ainda é aplicável. Por exemplo, em dados transversais você pode usar médias de grupo com base em idade, sexo ou vizinhança. Na análise de série temporal, você pode achar úteis operações exploratórias de agrupamento, como calcular uma média mensal ou medianas semanais. Você também pode combinar grupos temporais e não temporais, como a ingestão média mensal de calorias por gênero em uma população ou a quantidade média semanal de sono por faixa etária em um conjunto de pacientes hospitalares. Há muitas maneiras pelas quais o agrupamento de dados pode permitir que o eixo temporal interaja com outros relacionamentos dentro dos dados. Por exemplo:

• Poderá querer analisar diferentes períodos de tempo separadamente, se tiver razões exógenas para esperar uma mudança de regime. Use sua análise exploratória para encontrar demarcações sensatas para diferentes períodos de tempo significativos.

• Você pode descobrir que os dados apresentados como uma única série (como quando as doações chegam) ficam mais claros quando divididos em muitos processos paralelos (como

cronogramas de doação individual).

Plotagem

Vamos explorar uma série temporal típica fornecida por R, o conjunto de dados EuStockMarkets. Para ter uma ideia, podemos olhar para o cabeçalho do conjunto de dados:

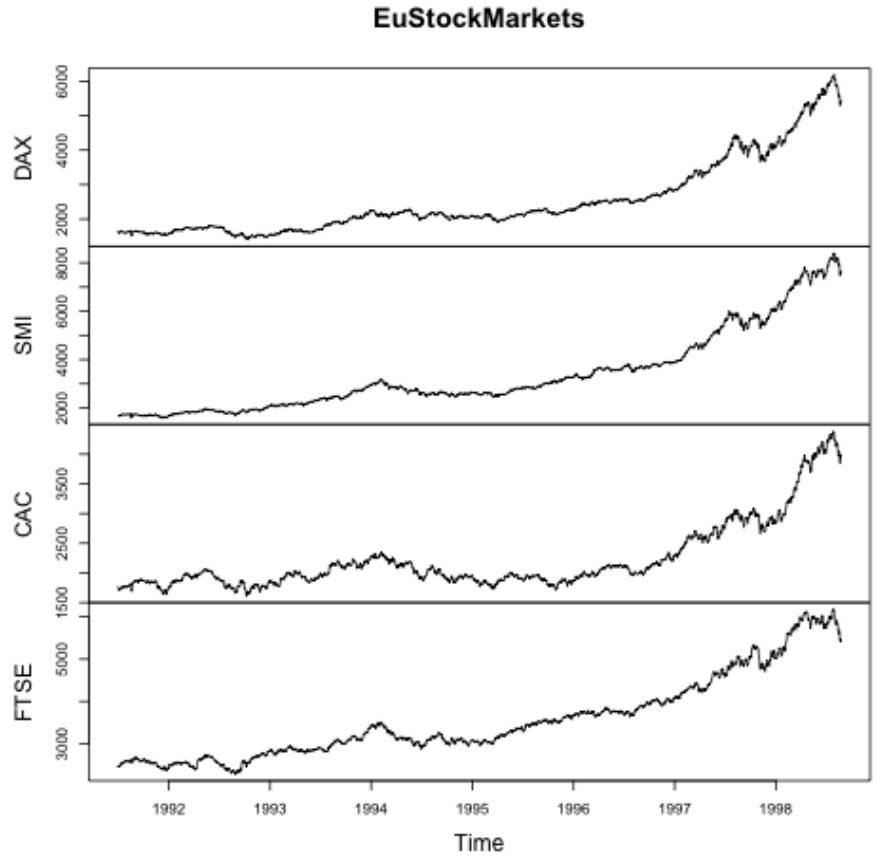
##R

> cabeça(EuStockMarkets)

DAX SMI CAC FTSE

[1,] 1628,75 1678,1 1772,8 2443,6

[2,] 1613,63 1688,5 1750,5 2460,2



[3,] 1606,51 1678,6 1718,0 2448,2

[4,] 1621,04 1684,1 1708,1 2470,4

[5,] 1618,16 1686,6 1723,1 2484,7

[6,] 1610,61 1671,6 1714,3 2466,8

Este é um conjunto de dados integrado na base R e contém os preços de fechamento diários dos quatro principais índices de ações europeus de 1991 a 1998. O conjunto de dados inclui apenas dias úteis.

Este conjunto de dados está melhor preparado do que aqueles que vimos até agora porque já está bem formatado e amostrado para nós. Não precisamos nos preocupar com valores ausentes, fusos horários ou medições incorretas, para que possamos ir direto para a análise exploratória de dados.

Nossos primeiros passos para observar esses dados serão muito semelhantes a uma série temporal não baseada no tempo, embora tenhamos uma opção mais simples com dados temporais do que teríamos com dados transversais. Podemos representar graficamente cada valor individualmente, e tal gráfico faz sentido. Compare isso com dados transversais, onde um gráfico de qualquer recurso individual em relação ao seu índice no conjunto de dados revelará apenas, se houver, a ordem em que os dados foram inseridos em uma tabela de dados ou retornados de um servidor SQL (em outras palavras , uma ordem arbitrária), mas nada sobre a situação subjacente. Entretanto, no caso de séries temporais, a plotagem é bastante informativa, como você pode ver na Figura 3-1.

##R

```
> plot(EuStockMarkets)
```

Figura 3-1. Um gráfico simples dos dados da série temporal.

Observe que a imagem é automaticamente segmentada em diferentes séries temporais com um simples comando plot(). Na verdade, como mostrado aqui, estamos usando um objeto R mts (estaríamos usando um objeto ts se houvesse apenas uma série temporal, mas temos várias em paralelo):

##R

```
> classe(EuStockMarkets)

[1] "mts" "ts" "matriz"
```

Vários pacotes populares fazem uso intenso de objetos ts e classes derivadas.

Esses objetos vêm com ótimas chamadas automáticas para funções de plotagem apropriadas, como vimos no exemplo anterior, onde uma simples chamada para plot() criou um gráfico multipainel bem rotulado. Os objetos ts também possuem algumas funções convenientes:

• frequência para descobrir a frequência anual dos dados:

##R

> frequência(EuStockMarkets)

[1] 260

• início e fim para encontrar o primeiro e o último tempo representado na série:

##R

> iniciar(EuStockMarkets)

[1] 1991 130

> fim(EuStockMarkets)

1] 1998 169

• janela para obter uma seção temporal dos dados:

##R

> janela(EuStockMarkets, início = 1997, fim = 1998)

Série temporal:

Início = c(1997, 1)

Fim = c(1998, 1)

Frequência = 260 DAX SMI CAC FTSE

1.997,000 2.844,09 3.869,8 2.289,6 4.092,5

1997.004 2844,09 3869,8 2289,6 4092,5

1997.008 2844,09 3869,8 2303,8 4092,5

...

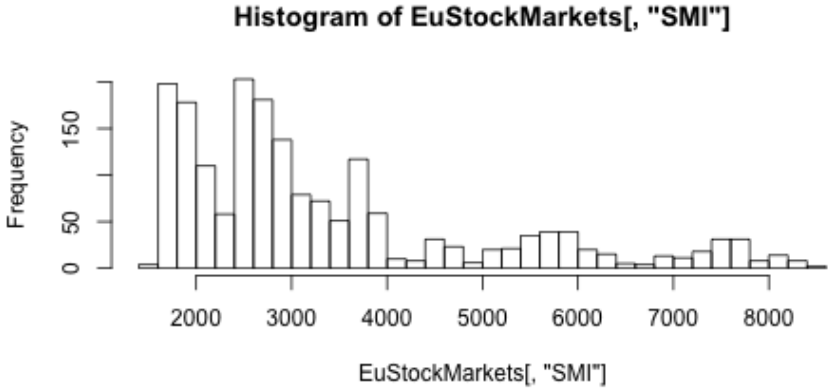
1997,988 4162,92 6115,1 2894,5 5168,3

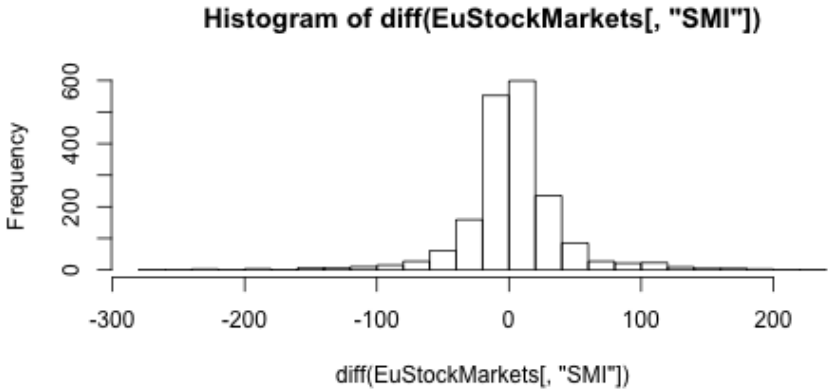
1997,992 4055,35 5989,9 2822,9 5020,2

1997,996 4125,54 6049,3 2869,7 5018,2

1998.000 4132,79 6044,7 2858,1 5049,8

Existem prós e contras na classe ts. Conforme mencionado anteriormente, ts e suas classes derivadas são usadas em muitos pacotes de séries temporais. Além disso, a configuração automática dos parâmetros de plotagem pode ser útil. Contudo, a indexação às vezes pode ser complicada e o **76 | Capítulo 3: Análise Exploratória de Dados para Séries Temporais**





o processo de acesso a subseções de dados com janela pode parecer complicado com o tempo.

Neste livro, você verá diversas maneiras de conter e acessar dados de séries temporais e caberá a você escolher a mais conveniente para seus casos de uso.

Histogramas

Vamos continuar nossa comparação de análise exploratória de dados em séries temporais e dados não-séries temporais. Gostaríamos, por exemplo, de obter um histograma dos dados, assim como acontece com a maioria das análises exploratórias de dados. Adicionamos uma ruga obtendo também um histograma dos dados diferenciados porque queremos usar nosso eixo de tempo (veja a Figura 3-2):

##R

```
> hist( EuStockMarkets[, "SMI"], 30)

> hist(diff(EuStockMarkets[, "SMI"], 30))
```

Figura 3-2. O histograma dos dados não transformados (topo) é extremamente amplo e não não apresentam uma distribuição normal. Isto é de esperar, dado que os dados subjacentes tem uma tendência. Diferenciamos os dados para remover a tendência, e isso transforma os dados em uma distribuição de formato mais normal (parte inferior).

Num contexto de série temporal, um hist() da diferença dos dados é muitas vezes mais interessante do que um hist() dos dados não transformados. Afinal de contas, num contexto de série temporal, muitas vezes (e particularmente em finanças) o que é mais interessante é como um valor muda de uma medição para outra, em vez da medição real do valor. Isto é particularmente verdadeiro para gráficos, porque obter o histograma de dados com uma tendência não produz uma visualização muito informativa.

Observe as novas informações que obtemos do histograma das séries diferenciadas.

Embora os gráficos originais das ações na seção anterior pintassem um quadro econômico muito otimista, com as

ações subindo inexoravelmente, este histograma nos mostra a experiência diária de alguém que segue as ações. O histograma da diferença nos diz que o valor da série temporal aumentou (valores de diferença positivos) e diminuiu (valores de diferença negativos) aproximadamente na mesma proporção ao longo do tempo. Os índices de ações não subiram e desceram exatamente na mesma proporção ao longo do tempo, porque sabemos que os índices de ações têm uma tendência crescente. No entanto, podemos ver neste histograma que apenas um ligeiro viés a favor das diferenças positivas em relação às negativas é o que explica essa tendência.

Este é um bom primeiro exemplo de por que precisamos prestar atenção à escala temporal ao amostrar, resumir e fazer perguntas sobre os dados. Se o desempenho parece ótimo (gráfico de longo prazo) ou apenas mais ou menos (histograma de gráficos de diferenças) dependerá de qual é a nossa escala temporal: nos preocupamos com o dia-a-dia ou temos um horizonte de longo prazo ? Se trabalharmos numa organização que negocia ações para obter lucro anual, precisamos de pensar na experiência de curto prazo que teremos sempre que reportarmos aos nossos chefes uma “diferença” que tende para valores negativos. Contudo, se formos um grande investidor institucional – talvez uma universidade ou um hospital – poderemos ter condições de assumir uma visão de longo prazo, que prevê uma subida ascendente. Este último cenário tem os seus próprios desafios: como demoramos o suficiente para maximizar o nosso lucro, mas não tanto tempo que a festa acabe? Essas são as questões que mantêm o setor financeiro funcionando junto com pesquisas e previsões de séries temporais.

Gráficos de dispersão

O método tradicional de usar gráficos de dispersão é tão útil para dados de séries temporais quanto para outros tipos de dados. Podemos usar gráficos de dispersão para determinar como duas ações estão vinculadas em um momento específico e como suas mudanças de preços estão relacionadas ao longo do tempo.

Neste exemplo, plotamos ambos os casos (veja a Figura 3-3):

• Os valores de duas ações diferentes ao longo do tempo

• Os valores das variações diárias nessas duas ações ao longo do tempo (via diferenciação) com a função diff() de R

##R

> plot(EuStockMarkets[, "SMI"], EuStockMarkets[, "DAX"])

> plot(**diff** (EuStockMarkets[, "SMI"]), **diff** (EuStockMarkets[, "DAX"]))

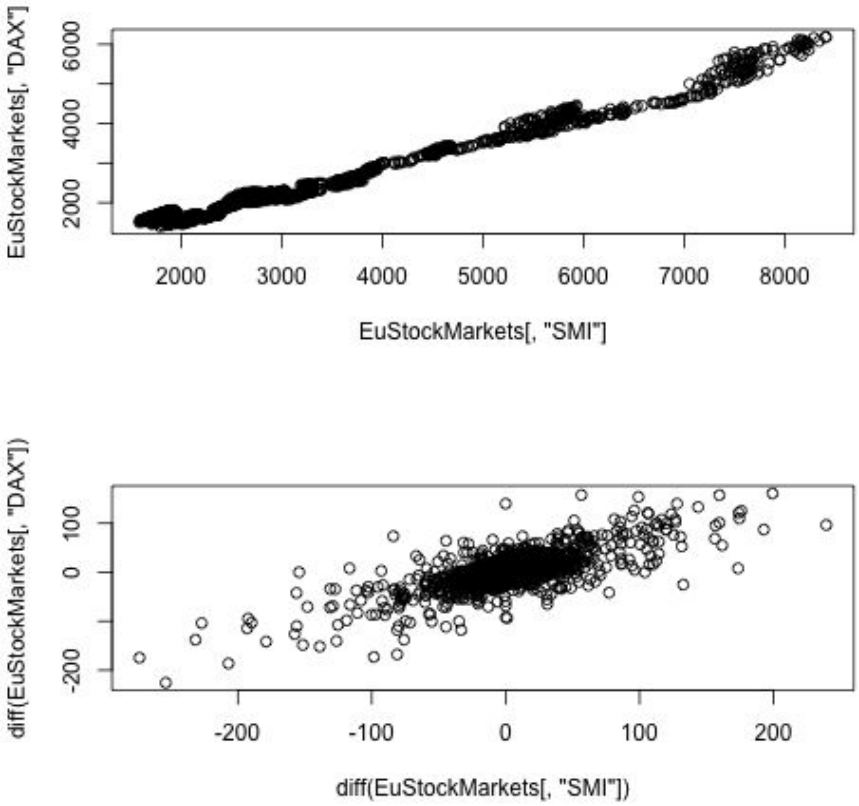


Figura 3-3. Gráficos de dispersão simples de dois índices de ações sugerem correlações fortes. No entanto, há razões para encarar estas conspirações com suspeita.

Como já vimos, os valores reais são menos informativos do que as diferenças entre pontos de tempo adjacentes, por isso traçamos as diferenças em um segundo gráfico de dispersão.

Estas parecem correlações muito fortes, mas as relações não são tão fortes quanto parecem. (Para se aprofundar, vá para “Correlações Espúrias” na página 102.)

As correlações aparentes na Figura 3-3 são interessantes, mas mesmo que sejam verdadeiras

relações (e há razões para duvidar que o sejam), estas não são correlações que possamos rentabilizar como corretores de ações. No momento em que sabemos se uma ação está subindo ou descendo, as ações com as quais ela está correlacionada também terão subido ou descido, uma vez que estamos fazendo correlações de valores em momentos idênticos. O que precisamos fazer é descobrir se a variação de uma ação mais cedo pode prever a variação de outra ação mais tarde. Para fazer isso, deslocamos uma das diferenças das ações em 1 antes de observar o gráfico de dispersão. Leia o código a seguir com atenção; observe que ainda estamos diferenciando, mas agora também estamos aplicando uma defasagem a uma das séries temporais diferenciadas (veja

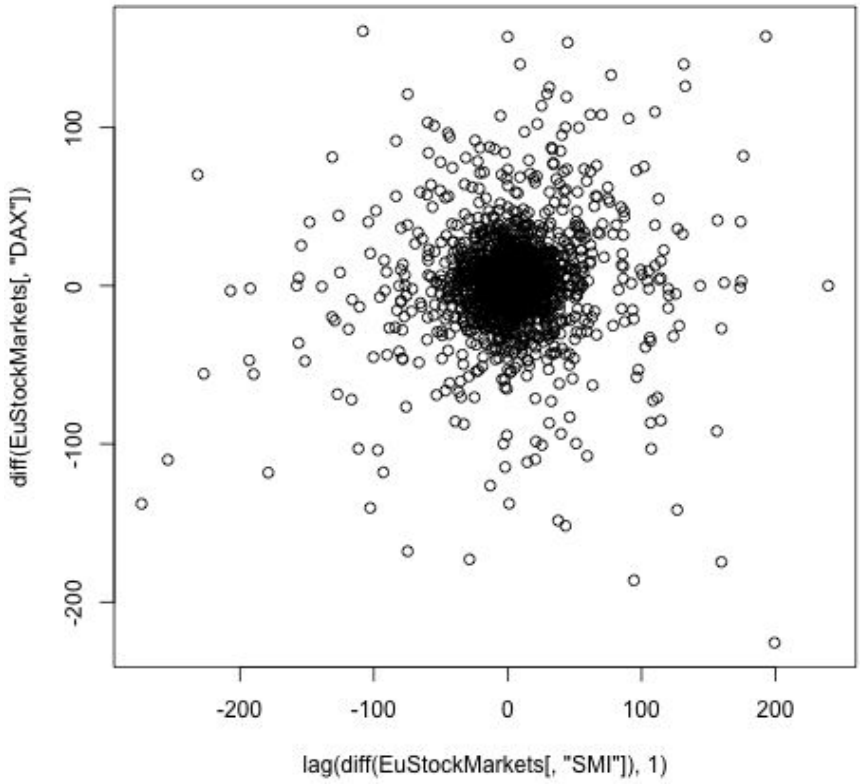
Figura 3-4):

##R

> plot(**lag** (diff(EuStockMarkets[, "SMI"]), 1),

diff(EuStockMarkets[, "DAX"]))





As linhas nesses exemplos de código são fáceis de ler porque cada seção

está alinhada. Pode ser tentador escrever longas linhas de textos ilegíveis

código, particularmente em uma linguagem de programação funcional como R

ou Python, mas você deve evitar isso sempre que possível!

Figura 3-4. A correlação entre as ações desaparece assim que colocamos um tempo lag, indicando que o SMI não parece prever o DAX.

Este resultado nos diz uma série de coisas importantes:

• Com dados de séries temporais, embora possamos usar as mesmas técnicas exploratórias que usamos com dados que não são de séries temporais, a aplicação estúpida não funcionará. Precisamos pensar em como explorar as mesmas técnicas, mas com dados reformulados.

• Muitas vezes é a relação entre os dados em diferentes pontos ou a mudança ao longo do tempo que é mais informativa sobre o comportamento dos seus dados.

• O gráfico da Figura 3-4 mostra por que pode ser difícil ser um corretor de ações. Se você for um investidor passivo e esperar, poderá se beneficiar da tendência ascendente de longo prazo. Porém, se você está tentando fazer previsões sobre o futuro, verá que não é fácil!



A função lag() de R pode não mover dados na direção temporal

você espera. A função lag() é uma mudança de tempo para frente. Isso é

importante lembrar, pois você não gostaria de mover dados no

direção temporal errada, dado um caso de uso específico, e ambos para

mudanças para frente e para trás no tempo são razoáveis para diferentes usos

casos.

Métodos Exploratórios Específicos de Séries Temporais

Vários métodos de análise de dados de séries temporais concentram-se nas relações de valores em momentos diferentes da mesma série, e você provavelmente não os terá visto antes se não tiver trabalhado com dados de séries temporais. No restante deste capítulo, examinaremos

alguns conceitos e técnicas relacionadas usadas para classificar séries temporais.

Os conceitos que exploraremos são:

Estacionaridade

O que significa uma série temporal ser estacionária e um teste estatístico para

estacionariedade

Autocorrelação

O que significa dizer que uma série temporal se correlaciona consigo mesma e o que tal correlação indica sobre a dinâmica subjacente da série temporal

Correlações espúrias

O que significa uma correlação ser espúria e quando você deve esperar encontrar correlações espúrias

Os métodos que aprenderemos a aplicar são:

• Funções de janela rolante e expansível

• Funções de autocorrelação

— A função de autocorrelação

— A função de autocorrelação parcial

Abordaremos os conceitos e seus métodos resultantes, desde a estacionariedade até as autocorrelações e as correlações espúrias. Antes de nos aprofundarmos nos detalhes, vamos discutir a lógica por trás dessa ordem específica.

A primeira pergunta que você provavelmente fará sobre uma série temporal é se ela parece refletir um sistema que é “estável” ou que está em constante mudança. É importante avaliar o nível de estabilidade, ou

estacionariedade , porque precisamos de saber até que ponto devemos esperar que o comportamento passado do sistema a longo prazo reflicta o seu comportamento futuro a longo prazo. Depois de avaliarmos a “estabilidade” (esta palavra não é usada aqui no sentido técnico) de uma série temporal, tentamos determinar se existem dinâmicas internas **Séries Temporais – Métodos Exploratórios Específicos | 81**

naquela série (mudanças sazonais, por exemplo). Ou seja, procuramos *autocorrelações* , respondendo à questão fundamental de quão rigorosamente os dados passados, distantes ou recentes, prevêem os dados futuros. Finalmente, quando pensamos ter encontrado certas dinâmicas comportamentais dentro do sistema, precisamos de ter a certeza de que não estamos a identificar relações baseadas em dinâmicas que não implicam de forma alguma as relações causais que desejamos descobrir; portanto, devemos procurar *correlações espúrias* .

Compreendendo a estacionariedade

Muitos modelos estatísticos tradicionais de séries temporais dependem de uma série temporal estacionária.

De um modo geral, uma série temporal estacionária é aquela que possui propriedades estatísticas bastante estáveis ao longo do tempo, particularmente no que diz respeito à média e à variância. Isto parece relativamente simples.

No entanto, a estacionariedade pode ser um conceito escorregadio, especialmente quando aplicado a dados de séries temporais reais. É muito intuitivo e muito fácil enganar-se e confiar na sua intuição natural. Examinaremos o conceito de forma intuitiva e com uma definição um tanto formal antes de discutir um teste comum de estacionariedade e detalhes práticos sobre como aplicar o conceito.

Intuição

Uma série temporal estacionária é aquela em que uma medição de série temporal reflete um sistema em estado

estacionário. As vezes é difícil afirmar exatamente o que isso significa, e pode ser mais fácil descartar as coisas como *não* estacionárias, em vez de dizer que algo *está* estacionário. Um exemplo fácil de dados que não são estacionários é o conjunto de dados de passageiros de companhias aéreas que examinamos no Capítulo 2, representado na Figura 3-5 (como lembrete, está disponível em R como AirPassengers e também amplamente disponível para download na Internet).

Existem várias características que mostram que esse processo não é estacionário. Primeiro, o valor médio aumenta ao longo do tempo, em vez de permanecer estável. Em segundo lugar, a distância entre o pico e o vale aumenta anualmente, pelo que a variância do processo aumenta ao longo do tempo. Terceiro, o processo apresenta um forte comportamento sazonal, a antítese da estacionariedade.

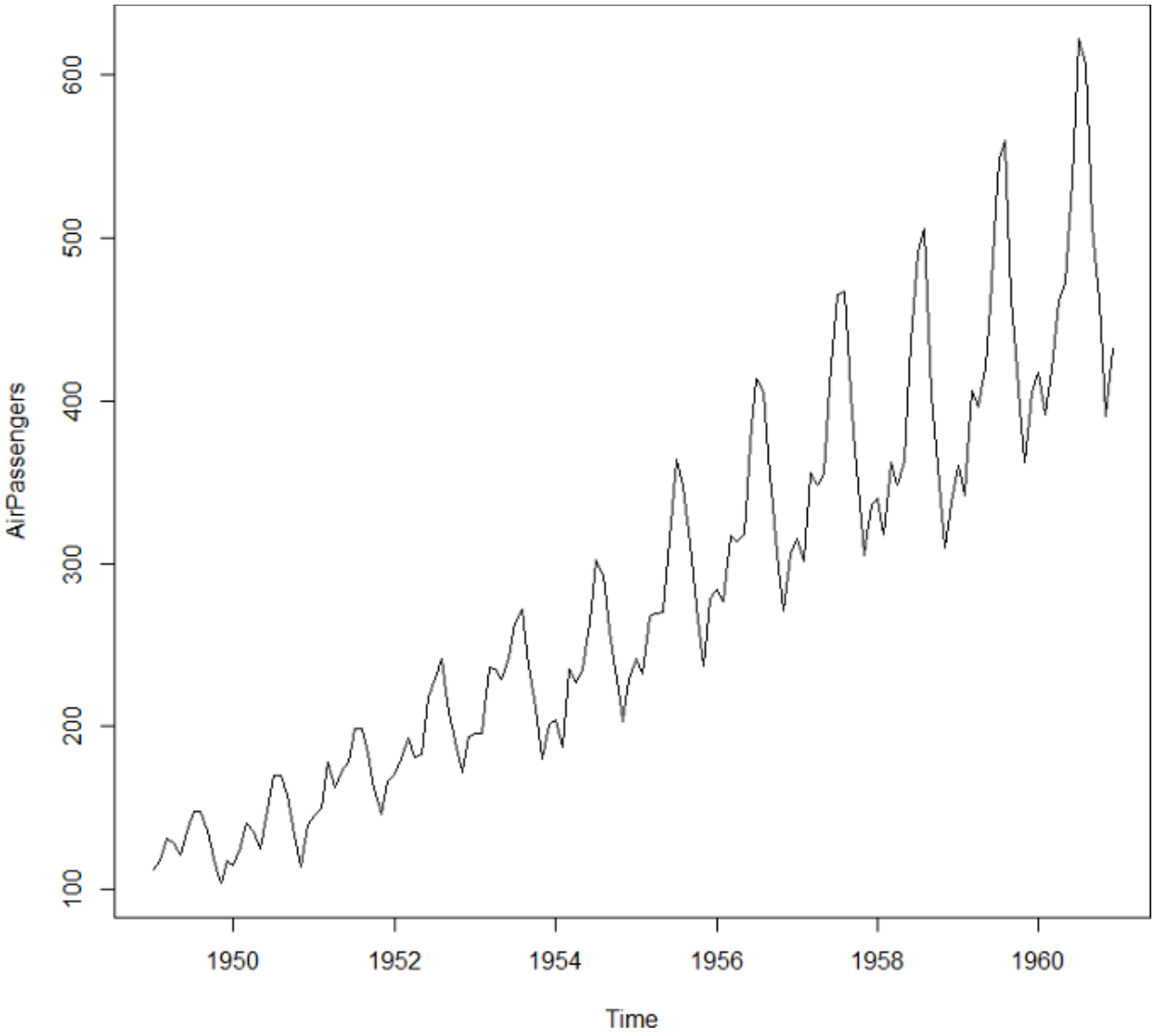


Figura 3-5. O conjunto de dados de passageiros de companhias aéreas oferece um exemplo claro de uma série temporal que é não estacionário. Com o passar do tempo, tanto a média quanto a variância dos dados mudam. Vemos também evidências de sazonalidade, que reflecte intrinsecamente uma tendência não estacionária. processo.

Definição estacionária e teste Dickey-Fuller Aumentado

Uma definição simples de um processo estacionário é a seguinte: um processo é estacionário se para todas as defasagens possíveis, k , a distribuição de yt , $yt+1$,..., $yt+k$, não depende de t.

Os testes estatísticos de estacionariedade muitas vezes se resumem à questão de saber se existe uma

raiz unitária — isto é, se 1 é uma solução da equação característica do processo.1 Uma série temporal linear é não estacionária se houver uma raiz unitária, embora a falta de uma raiz unitária não prove estacionariedade. Abordar a estacionariedade como uma questão geral permanece complicado, e determinar se um processo tem uma raiz unitária continua sendo uma área atual de pesquisa.

1 Se isso não lhe parece familiar, não se preocupe. Você pode ler mais em “Mais recursos” na página 117 , caso deseje aprofundar o assunto.

No entanto, uma intuição simples sobre o que é uma raiz unitária pode ser obtida a partir do exemplo de um passeio aleatório:

$$yt = \phi \times yt\text{-}1 + et$$

Neste processo, o valor de uma série temporal em um determinado momento é uma função de seu valor no momento imediatamente anterior e de algum erro aleatório. Se ϕ for igual a 1, a série tem raiz unitária, “fugirá” e não será estacionária. Interessante notar é que o fato de a série não ser estacionária não significa que tenha que ter tendência. Um passeio aleatório é um bom exemplo de série temporal não estacionária que não possui uma tendência subjacente. 2

Os testes para determinar se um processo é estacionário são chamados *de testes de hipóteses* . O teste Augmented Dickey-Fuller (ADF) é a métrica mais comumente usada para avaliar uma série temporal para problemas de estacionariedade. Este teste postula uma hipótese nula de que uma raiz unitária está presente em uma série temporal. Dependendo dos resultados do teste, esta hipótese nula pode ser rejeitada para um determinado nível de significância, o que significa que a presença de um teste de

raiz unitária pode ser rejeitada para um determinado nível de significância.

Observe que os testes de estacionariedade concentram-se em saber se a média de uma série está mudando. A variação é tratada por transformações em vez de testada formalmente. O teste para saber se uma série é estacionária é, portanto, um teste para saber se uma série é integrada. Uma série integrada de ordem *d* é uma série que deve ser diferenciada *d* vezes para se tornar estacionária.

O enquadramento do teste Dickey-Fuller é o seguinte:

$\Delta yt = yt - yt\text{-}1 = (\phi - 1) \times yt\text{-}1 + \epsilon\ t$ Então o teste para saber se $\phi = 1$ é um teste *t simples* para saber se o parâmetro no defasado *yt–1* é igual a 0. O A diferença que o teste ADF faz é levar em conta mais defasagens, de modo que o modelo subjacente leva em conta a dinâmica de ordem superior, que pode ser escrita como uma série de defasagens diferenciadas:

$Yt - \phi 1 \times yt\text{-}1 - \phi 2 \times yt\text{-}2. . = \epsilon\ t$ Isso requer um pouco mais de álgebra para escrever como uma série de defasagens diferenciadas, e a distribuição esperada contra a qual testar a hipótese nula é um pouco diferente 2 Um determinado processo de série temporal de amostra produzido por um passeio aleatório pode parecer ter uma tendência, e isto inspira muito debate, particularmente em análises de séries temporais de preços de ações.



em comparação com o teste Dickey-Fuller original. O teste ADF é o teste de estacionariedade mais amplamente apresentado na literatura de séries temporais.

Infelizmente, esses testes estão longe de ser uma panacéia para seus problemas de estacionariedade por vários motivos:

• Esses testes têm baixo poder em distinguir raízes unitárias *próximas* de raízes unitárias.

• Com um tamanho de amostra pequeno, os falsos positivos para raízes unitárias são bastante comuns.

• A maioria dos testes não testa a favor ou contra todos os tipos de problemas que podem levar a uma série temporal não estacionária. Por exemplo, algumas vezes procuraremos especificamente testar se a média ou a variância (mas não ambas) são estacionárias. Outros testes analisarão de forma mais geral a distribuição geral. É importante compreender os limites do teste aplicado ao utilizá-lo e garantir que os limites sejam consistentes com suas crenças sobre seus dados.

Definindo uma hipótese nula alternativa: teste KPSS

Embora o teste ADF postule uma hipótese nula de raiz unitária, o

O teste Kwiatkowski-Phillips-Schmidt-Shin (KPSS) postula um nulo

hipótese de um processo estacionário. Ao contrário do ADF, o KPSS não é

disponível na base R, mas ainda é amplamente implementado. Há alguns

nuances sobre para que servem esses testes e como usá-los

anteriormente; estes estão além do escopo deste texto, mas amplamente discutidos

on-line.

Na prática

A estacionariedade é importante na prática por uma série de razões. Primeiro, um grande número de modelos assume um processo estacionário, como os modelos tradicionais com resistências conhecidas

e modelos estatísticos. Abordaremos essas classes de modelos no Capítulo 6.

Um ponto mais amplo é que um modelo de uma série temporal que não é estacionária irá variar em sua precisão à medida que as métricas da série temporal variam. Ou seja, se você estiver procurando um modelo para ajudá-lo a estimar a média de uma série temporal com média e variância não estacionárias, o viés e o erro no seu modelo variarão ao longo do tempo, ponto em que o valor do seu modelo se tornará questionável.

Muitas vezes acontece que uma série temporal pode se tornar estacionária o suficiente com algumas transformações simples. Uma transformação logarítmica e uma transformação de raiz quadrada são duas opções populares, especialmente no caso de mudança de variância ao longo do tempo. Da mesma forma, a remoção de uma tendência é mais comumente feita por diferenciação. Às vezes, uma série deve ser diferenciada mais de uma vez. No entanto, se você estiver diferenciando demais (mais de duas ou três vezes), é improvável que consiga resolver seu problema de estacionariedade com diferenciação.



Log ou sqrt?

Embora uma raiz quadrada tenda a ser menos complexa computacionalmente do que

um logaritmo, você deve explorar ambas as opções. Pense sobre o

intervalo de seus dados e como você deseja compactar valores grandes

em vez de otimizar prematuramente (isto é, pessimizar indevidamente)

seu código e análise.

A estacionariedade não é a única suposição que os modelos de previsão fazem. Outra suposição comum, mas distinta, é a normalidade da distribuição das variáveis de entrada ou variável prevista. Nesses casos, outras transformações podem ser necessárias. Uma delas comum é a transformação Box Cox, que é implementada no pacote de previsão R e em scipy.stats em Python. A transformação faz não normalmente

dados distribuídos (dados distorcidos) mais normais. No entanto, só porque você pode transformar seus dados não significa que deva fazê-lo. Pense cuidadosamente sobre o significado das distâncias entre os pontos de dados no seu conjunto de dados original antes de transformá-los e certifique-se, independentemente da sua tarefa, de que as transformações preservem as informações mais importantes.

As transformações têm suposições

Você pode pensar que está escolhendo apenas transformações que não fazem suposições subjacentes (log e sqrt parecem simples, certo?), mas pense cuidadosamente se isso é verdade.

Conforme observado, as transformações logarítmicas e de raiz quadrada são comumente usadas para reduzir a variação *da variação* ao longo do tempo. Estes fazem uma série de suposições. Uma é que seus dados sempre serão positivos. Como alternativa, se você decidir mudar seus dados antes de calcular a raiz quadrada ou o log, estará novamente adicionando um viés ou presumindo que isso não importa. Finalmente, ao calcular a raiz quadrada ou o log,

você torna os valores maiores menos diferentes uns dos outros, comprimindo efetivamente o espaço entre valores maiores, mas não entre valores menores, diminuindo a ênfase nas diferenças entre valores discrepantes. Isto pode ou não ser apropriado.

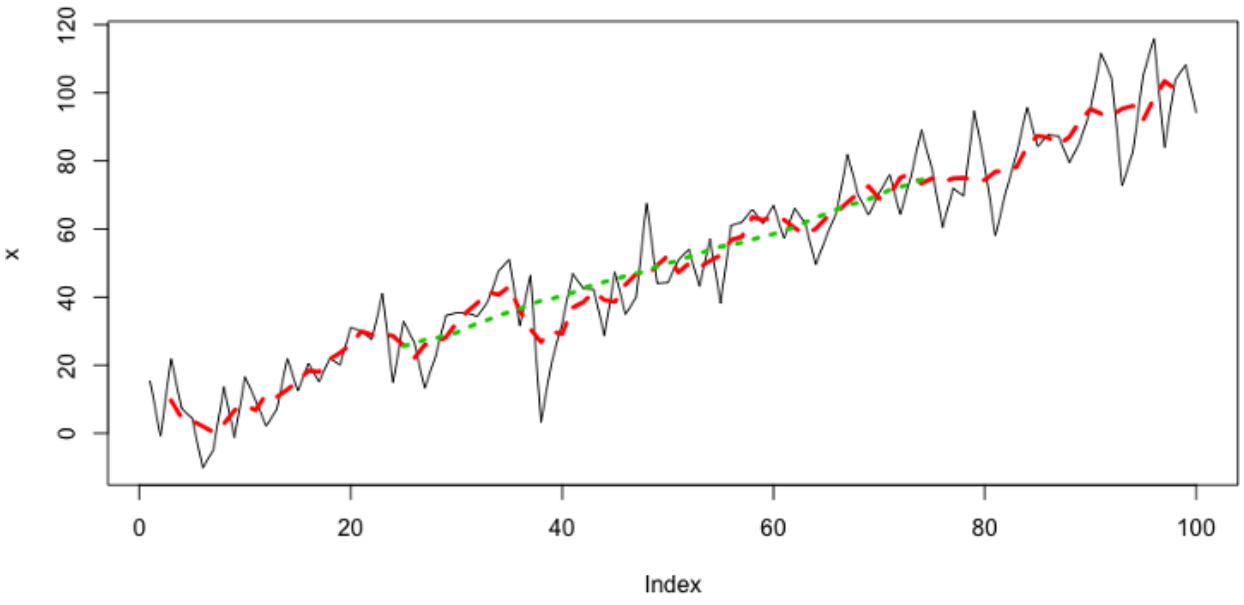
Aplicando funções de janela

Vamos revisar os gráficos exploratórios de séries temporais mais importantes, que você provavelmente usará para a maioria das análises iniciais de séries temporais.

Janelas de enrolar

Uma função comum distinta das séries temporais é uma função de janela, que é qualquer tipo de função onde você agrega dados para compactá-los (como vimos no caso de

redução da resolução no capítulo anterior) ou suavizá-la (também discutido no Capítulo 2).



Além dos aplicativos já discutidos, dados suavizados e janelas

dados agregados proporcionam visualizações exploratórias informativas.

Podemos calcular uma média móvel e outros cálculos que envolvem alguma função linear de uma série de pontos com a função filter() da base R, como segue:

##R

> ## calcular uma média móvel usando a base R

> ## função de filtro

> x <- rnorma(n = 100, média = 0, dp = 10) + 1:100

> mn <- função(n) rep(1/n, n)

> plotar (x, tipo = 'l', lwd = 1)

> linhas(filtro(x, mn(5)), col = 2, lwd = 3, lty = 2)

> linhas(filtro(x, mn(50)), col = 3, lwd = 3, lty = 3)

Este código produz o gráfico da Figura 3-6.

Figura 3-6. Duas curvas exploratórias produzidas por meio de suavização por média rolante. Nós podemos use-os para procurar uma tendência em dados particularmente ruidosos ou para decidir que tipos de desvios do comportamento linear são interessantes de investigar versus quais provavelmente são apenas barulho.

Se estivermos procurando funções que não sejam combinações lineares de todos os pontos da janela, não poderemos usar filter() porque ela depende de uma transformação linear dos dados. No entanto, podemos usar o zoológico. A função rollapply() do pacote zoo é bastante útil (veja a Figura 3-7):

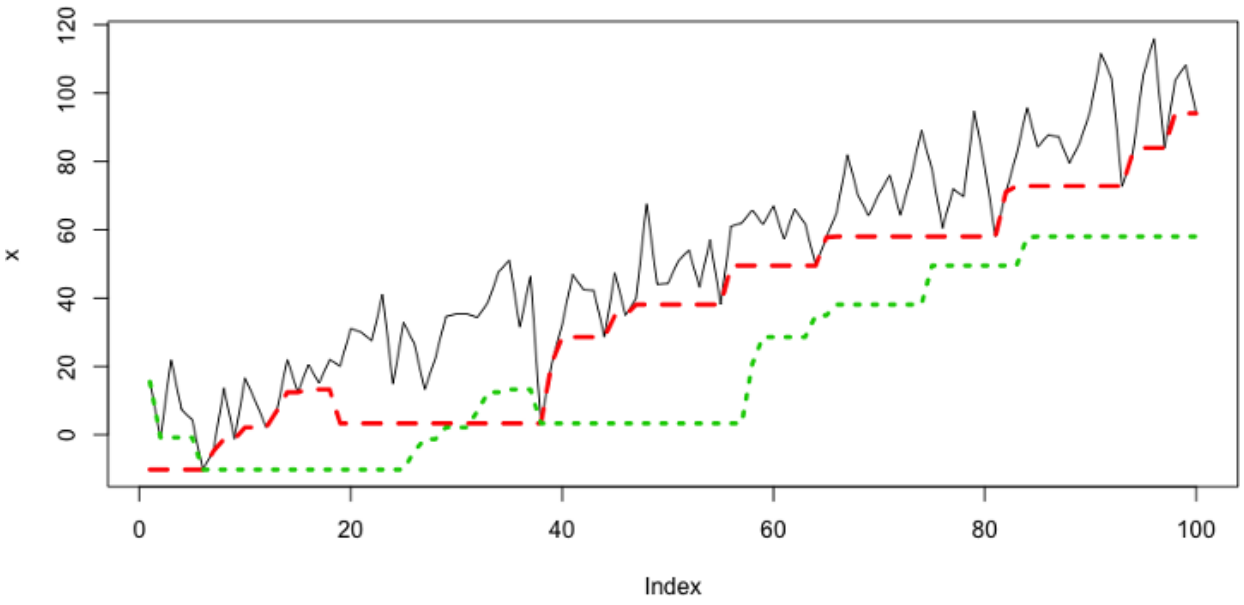
##R

> ## você também pode fazer mais funcionalidades 'personalizadas'

> exigir (zoológico)

> f1 <- rollapply(zoo(x), 20, função(w) min(w),





>alinhar = "esquerda", parcial = **TRUE**)

> f2 <- rollapply(zoo(x), 20, função(w) min(w),

>alinhar = "direita", parcial = **TRUE**)

> gráfico (x, lwd = 1, tipo = 'l')

> linhas (f1, col = 2, lwd = 3, lty = 2)

> linhas (f2, col = 3, lwd = 3, lty = 3)

Figura 3-7. Uma janela móvel alinhada no mínimo à esquerda (traços longos) ou à direita (traços curtos). O

alinhamento à esquerda vê eventos no futuro, enquanto o alinhamento à direita vê apenas eventos do passado. É importante observar isso para evitar a antecipação. No entanto, às vezes, um alinhamento à esquerda pode ser útil para fazer perguntas exploratórias como "se eu soubesse isso com antecedência, seria útil? Às vezes, mesmo uma antecipação não é informativa, o que significa que uma determinada variável não é informativa. Ao conhecer o o futuro de uma medida não é útil, provavelmente não será uma medida útil em nenhum momento contexto da série.

Use objetos de zoológico em funções de zoológico. Se você passar vetores numéricos

diretamente para rollapply(), o argumento de alinhamento não terá efeito.

Você pode confirmar isso excluindo o wrapper zoo() em torno de x em

o código anterior. Você verá que as duas curvas são idênticas

e, na verdade, trata-se de um fracasso silencioso, que é particularmente perigoso

na análise de séries temporais porque pode introduzir um efeito não intencional

olhe para frente. Este é um exemplo de por que você precisa verificar a sanidade

frequentemente, mesmo com sua análise exploratória de dados. Infelizmente,

falha silenciosa não é incomum em muitos pacotes R populares e em

outras linguagens de script, então fique alerta!

Também é possível “fazer o seu próprio” para esse tipo de função, o que pode ser uma boa ideia para limitar dependências. Nesses casos, recomendo fortemente começar com o

código-fonte de um pacote existente amplamente utilizado, como zoo. Isso ocorre porque, mesmo para uma série temporal univariada, há um número surpreendente de casos extremos a serem considerados, como como tratar NAs e como tratar o início e o fim de uma série onde você terá menos pontos do que o especificado. tamanho da janela.

Uma série de opções R para séries temporais

Anteriormente neste capítulo, consideramos a classe ts, que está disponível na base R para lidar com dados de séries temporais. Nesta seção usamos objetos zoológicos. Você também deve estar ciente dos objetos xts. Aqui está um breve resumo de como os objetos zoo e xts melhoram em relação aos objetos ts:

• Um objeto ts assume uma série temporal completa regularmente espaçada e, por esse motivo, não armazena índices temporais individuais, mas apenas o início, o fim e a frequência da série temporal.

• Os objetos ts suportam um ciclo recorrente, como anos ou meses.

• os objetos zoo armazenam carimbos de data e hora como um atributo de índice, de modo que não exijam séries temporais periódicas e regularmente espaçadas.

• os objetos do zoológico podem ser impressos horizontalmente ou verticalmente.

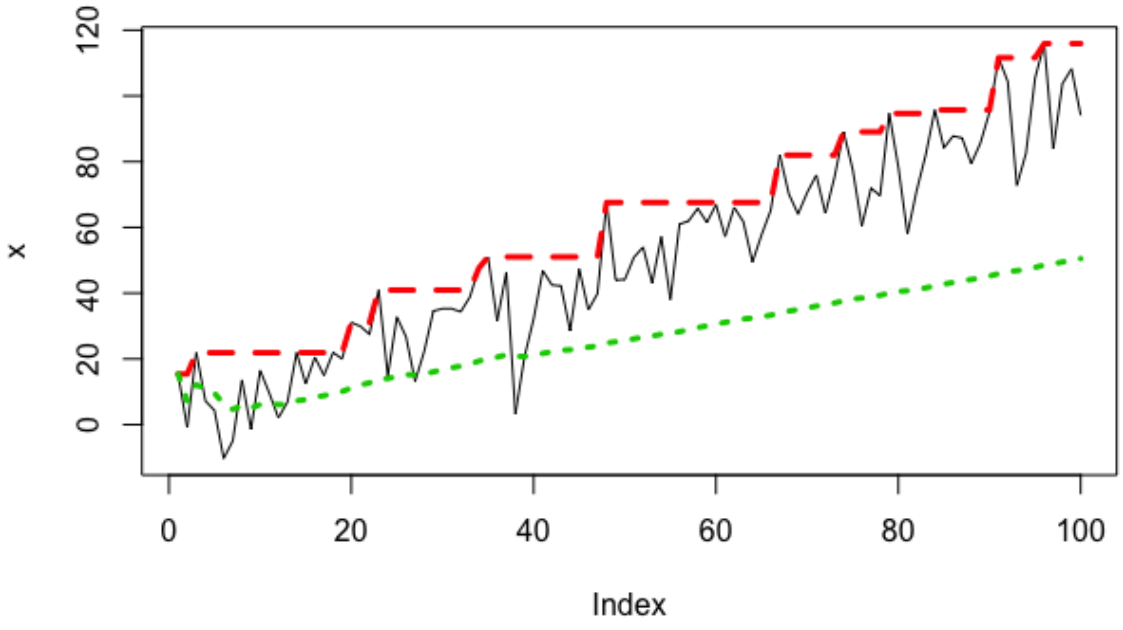
• A parte de dados de um objeto zoológico pode ser um vetor ou uma matriz.

• Os objetos xts são uma extensão dos objetos zoo com ainda mais opções.

Expandindo janelas

As janelas de expansão são menos comumente usadas na análise de séries temporais do que as janelas de rolagem porque sua aplicação adequada é mais limitada. A expansão das janelas só faz sentido nos casos em que você está estimando uma estatística resumida que acredita ser um processo estável, em vez de evoluir ao longo do tempo ou oscilar significativamente. Uma janela em expansão começa com um determinado tamanho mínimo, mas à medida que avança na série temporal, ela cresce para incluir todos os pontos até um determinado momento, em vez de apenas um tamanho finito e constante.

Uma janela em expansão oferece maior certeza em sua estimativa de estatísticas de teste ao longo do tempo, permitindo que você se beneficie de estar “mais aprofundado” em uma série temporal específica. No entanto, só funciona se você levantar a hipótese de que seu sistema subjacente é estacionário. Pode ajudá-lo a manter estatísticas resumidas “online”, pois você as teria estimado em tempo real à medida que coletava mais informações.



Se você olhar para a base R, perceberá que muitas funções já existem como implementações de uma janela em expansão, como cummax e cummin. Você também pode

redirecionar facilmente o cumsum para criar uma média cumulativa. No gráfico a seguir, mostramos um máximo de janela em expansão e uma média de janela em expansão (veja a Figura 3-8):

##R

> *# expandindo janelas*

> plotar (x, tipo = 'l', lwd = 1)

> linhas(cummax(x), col = 2, lwd = 3, lty = 2) *# max*

> linhas(cumsum(x)/1:comprimento(x), col = 3, lwd = 3, lty = 3) *# média*

Figura 3-8. Uma janela expansível "max" (traços longos) e uma janela expansível

"significa" (traços curtos). O uso de uma janela expansível significa que o máximo sempre reflete o máximo global até aquele momento, tornando-a uma função monotônica. Graças a a longa "memória" da janela em expansão, a média da janela em expansão é menor

do que a média móvel (Figura 3-7) porque a tendência subjacente é menos proeminente no

média em expansão. Se isso é bom, ruim ou neutro depende de nossas suposições e conhecimento do sistema subjacente.

Se precisar de uma função personalizada com uma janela contínua, você pode usar rollapply() como fizemos para uma janela contínua. Nesse caso, você precisa especificar uma sequência de tamanhos de janela em vez de um único escalar. A execução do código a seguir produzirá um gráfico idêntico

ao da Figura 3-8 , mas desta vez ele foi criado com rollapply() em vez de funções R integradas:

##R

> plotar (x, tipo = 'l', lwd = 1)

> linhas(rollapply(zoo(x), seq_along(x), função(w) max(w),

> parcial = **TRUE** , alinhar = "direita"),

> col = 2, lwd = 3, lty = 2)

> linhas(rollapply(zoo(x), seq_along(x), função(w) média(w),

> parcial = **TRUE** , alinhar = "direita"),

> col = 3, lwd = 3, lty = 3)

Funções de rolagem personalizadas

Só nos importamos com a opção de aplicar uma função de rolagem personalizada se realmente pudermos imaginar fazê-lo. Na prática, isso é algo que você provavelmente verá ao analisar domínios de séries temporais que conhecem leis fundamentais de comportamento subjacentes ou heurísticas úteis que são necessárias para uma análise adequada.

Por exemplo, podemos estar olhando para janelas que incluem um recurso específico que é informativo, dado o conhecimento do domínio. Poderíamos querer saber quando estamos em um

regime *monotônico* (digamos, o açúcar no sangue está aumentando) versus um regime de altos e baixos que sugere ruído instrumental em vez de uma tendência. Poderíamos escrever uma função personalizada para este cenário e aplicá-la com uma janela móvel ou em expansão.

Compreendendo e identificando a autocorrelação

Na sua forma mais fundamental, a autocorrelação de uma série temporal é a ideia de que um valor em uma série temporal em um determinado momento pode ter uma correlação com o valor em outro momento. Observe que

“autocorrelação” está sendo usada aqui informalmente para descrever uma ideia geral em vez de uma ideia técnica.

Como exemplo de autocorrelação, se você tomar uma série temporal anual de dados diários de temperatura, poderá descobrir que comparar 15 de maio de cada ano com 15 de agosto de cada ano fornecerá alguma correlação, de modo que 15 de maio mais quentes tendem a se correlacionar com 15 de agosto mais quentes (ou tendem a se correlacionar com 15 de agosto mais frios). Você pode sentir que aprendeu um fato potencialmente interessante sobre o sistema de temperatura, indicando que existe uma certa previsibilidade a longo prazo. Por outro lado, poderá encontrar a correlação mais próxima de zero, caso em que também terá encontrado algo interessante, nomeadamente que saber a temperatura no dia 15 de maio não lhe dá, por si só, qualquer informação sobre a provável variação das temperaturas no dia 15 de agosto.

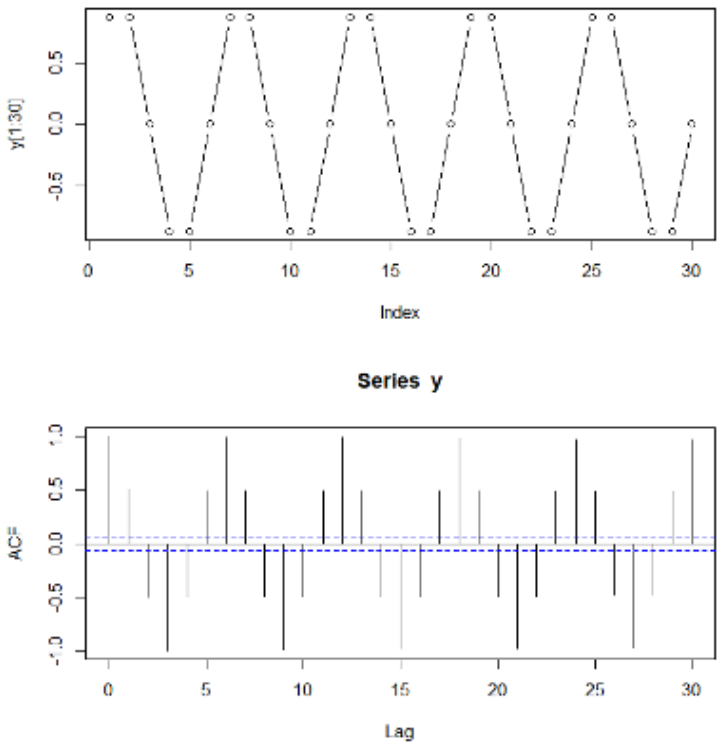
Isso é autocorrelação em poucas palavras anedóticas.

A partir deste exemplo simples, vamos expandir para a autocorrelação, que generaliza a autocorrelação ao não ancorar em um ponto específico no tempo. Em particular, a autocorrelação levanta a questão mais geral de saber se existe uma correlação entre quaisquer dois pontos numa série temporal específica com uma distância fixa específica entre eles.

Veremos isso com mais detalhes a seguir, bem como a elaboração final da autocorrelação parcial.

A função de autocorrelação

Começamos com a excelente definição de autocorrelação da Wikipédia : Autocorrelação, também conhecida como correlação serial, é a correlação de um sinal com uma cópia atrasada de si mesmo em função do atraso. Informalmente, é a semelhança entre as observações em função do intervalo de tempo entre elas.



Vamos traduzir isso para um inglês mais simples. A autocorrelação dá uma ideia de como os pontos de dados em diferentes pontos no tempo estão linearmente relacionados entre si em função da diferença de tempo.

A *função de autocorrelação* (ACF) pode ser compreendida intuitivamente com plotagem. Nós

podemos plotá-lo facilmente em R (veja a Figura 3-9):

##R

```
> x <- 1:100

> y <- sin(x × pi /3)

> plot(y, tipo = "b")

>acf(y)
```

Figura 3-9. Gráfico de uma função seno e seu ACF.

Correlação em um sistema determinístico

Esta série senoidal é uma função simples e um sistema totalmente determinado, dada uma sequência de entrada conhecida. No entanto, não temos uma correlação de 1. Por que isso acontece? Inspecione a série e pense no que o ACF está calculando e você perceberá que, para muitos valores, o valor subsequente pode aumentar ou diminuir dependendo de onde você está no ciclo. Se você conhece alguns pontos seguidos, sabe em que direção o processo está indo, mas se não souber (como acontece com o ACF, que é uma medida de correlação 1:1), terá uma correlação inferior a 1 porque a maioria dos valores não possui um valor subsequente exclusivo, mas sim mais de um. Portanto, lembre-se de que uma correlação não unitária não significa necessariamente que você tenha uma série temporal probabilística ou ruidosa.

No ACF, vemos que os pontos que possuem uma defasagem de 0 entre eles têm uma correlação de 1 (isso é verdade para todas as séries temporais), enquanto os pontos separados por uma defasagem de 1 têm uma correlação de 0,5. Pontos separados por 2 defasagens têm uma correlação de –0,5 e assim por diante.

Calcular o ACF é simples. Podemos fazer isso sozinhos usando a função shift() de data.table:

##R

```
> cor(y, shift(y, 1), use = "pairwise.complete.obs")

[1] 0,5000015

> cor(y, shift(y, 2), use = "pairwise.complete.obs")

[1] -0,5003747
```

Nossos cálculos correspondem aproximadamente aos resultados gráficos da Figura 3-9 . 3 Embora seja simples calcular o ACF com código personalizado, geralmente é melhor usar uma versão pré-instalada, como a função acf do R. Existem várias vantagens em fazer isso:

• Plotagem automática com rótulos úteis

• Um número máximo sensato (geralmente, mas nem sempre) de atrasos para calcular o ACF, bem como a opção de substituir esse máximo

• Uma maneira elegante de lidar com séries temporais multivariadas

Existem alguns fatos importantes sobre o ACF, matematicamente falando:

• O ACF de uma função periódica tem a mesma periodicidade do processo original.

Você pode ver isso nos gráficos de exemplo de seno anteriores.

• A autocorrelação da soma das funções periódicas é a soma das autocorrelações de cada função separadamente. Você pode formular facilmente um exemplo disso com algum código simples.

• Todas as séries temporais têm uma autocorrelação de 1 com defasagem 0.

• A autocorrelação de uma amostra de ruído branco terá um valor de aproximadamente 0 em todos os atrasos diferentes de 0.

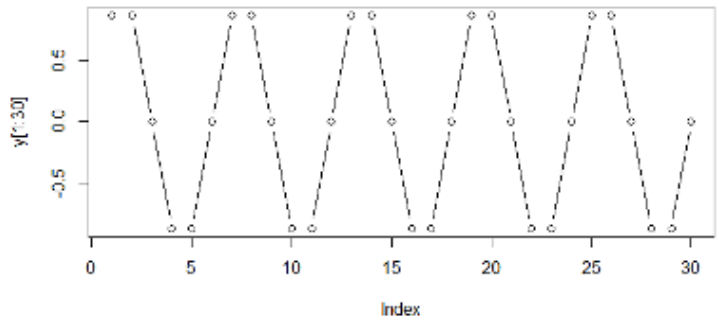
• O ACF é simétrico em relação às defasagens negativas e positivas, portanto apenas as defasagens positivas precisam ser consideradas explicitamente. Você pode tentar traçar um ACF calculado manualmente para provar isso.

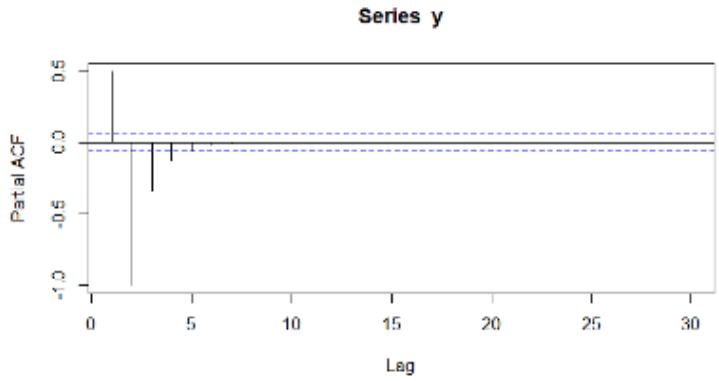
• Uma regra estatística para determinar uma estimativa significativa de ACF diferente de zero é dada por um

“região crítica” com limites em +/–1,96 × sqrt (n). Esta regra depende de um tamanho de amostra suficientemente grande e de uma variância finita para o processo.

3 As correlações de amostra calculadas com a função cor() de R versus a função acf() de R não corresponderão

exatamente porque usam divisores diferentes. Para obter mais informações, consulte StackExchange.





A função de autocorrelação parcial

A função de autocorrelação parcial (PACF) pode ser mais difícil de entender do que a ACF. A autocorrelação parcial de uma série temporal para um determinado desfasamento é a correlação parcial da série temporal consigo mesma naquele desfasamento, dadas todas as informações entre os dois pontos no tempo.

É fácil concordar aqui e pensar que isso parece razoável. Mas o que significa exatamente contabilizar as informações entre dois pontos no tempo? Isso significa que você precisa calcular uma série de correlações condicionais e subtraí-las da correlação total. Calcular o PACF não é simples e há vários métodos para estimá-lo. Não discutiremos isso aqui, mas você pode encontrar discussões na documentação relacionada de R e Python.

O PACF é mais fácil de entender graficamente do que conceitualmente. Sua utilidade também é

muito mais óbvio em um gráfico do que em uma discussão (veja a Figura 3-10):

##R

> y <- sin(x × **pi** /3)

> plot(y[1:30], tipo = "b")

>pacf(y)

Figura 3-10. Gráfico e PACF de um processo sazonal e silencioso.



Para o caso de uma série senoidal, o PACF oferece um contraste marcante com o ACF. O PACF mostra quais pontos de dados são informativos e quais são harmônicos de períodos de tempo mais curtos.

Para um processo sazonal e silencioso, como a função senoidal, com período T, o mesmo valor de ACF será visto em T, 2T, 3T e assim por diante até o infinito. Um ACF não consegue eliminar essas correlações redundantes. O PACF, por outro lado, revela quais correlações são correlações informativas “verdadeiras” para defasagens específicas, em vez de redundâncias. Isto é inestimável para saber quando coletamos informações suficientes para obter uma janela suficientemente longa em uma escala temporal adequada para nossos dados.

A região crítica para o PACF é a mesma do ACF. A região crítica tem limites em ±1,96 n . Quaisquer defasagens com valores PACF calculados dentro da região crítica são efetivamente zero.

Até agora vimos apenas exemplos de processos de frequência única perfeitamente silenciosos. Agora veremos um exemplo um pouco mais complicado. Consideraremos a soma de duas curvas senoidais sob condições de nenhum ruído, baixo ruído e alto ruído.

Primeiro, vamos dar uma olhada nos gráficos sem ruído, cada um individualmente (veja a Figura 3-11):

##R

> y1 <- sin(x × **pi** /3)

> plot(y1, tipo = "b")

>acf (y1)

>pacf(y1)

> y2 <- sen(x × **pi** /10)

> plot(y2, tipo = "b")

>acf (y2)

>pacf(y2)

O ACF dos dados estacionários deve cair rapidamente para zero. Para não

dados estacionários, o valor no atraso 1 é positivo e grande.

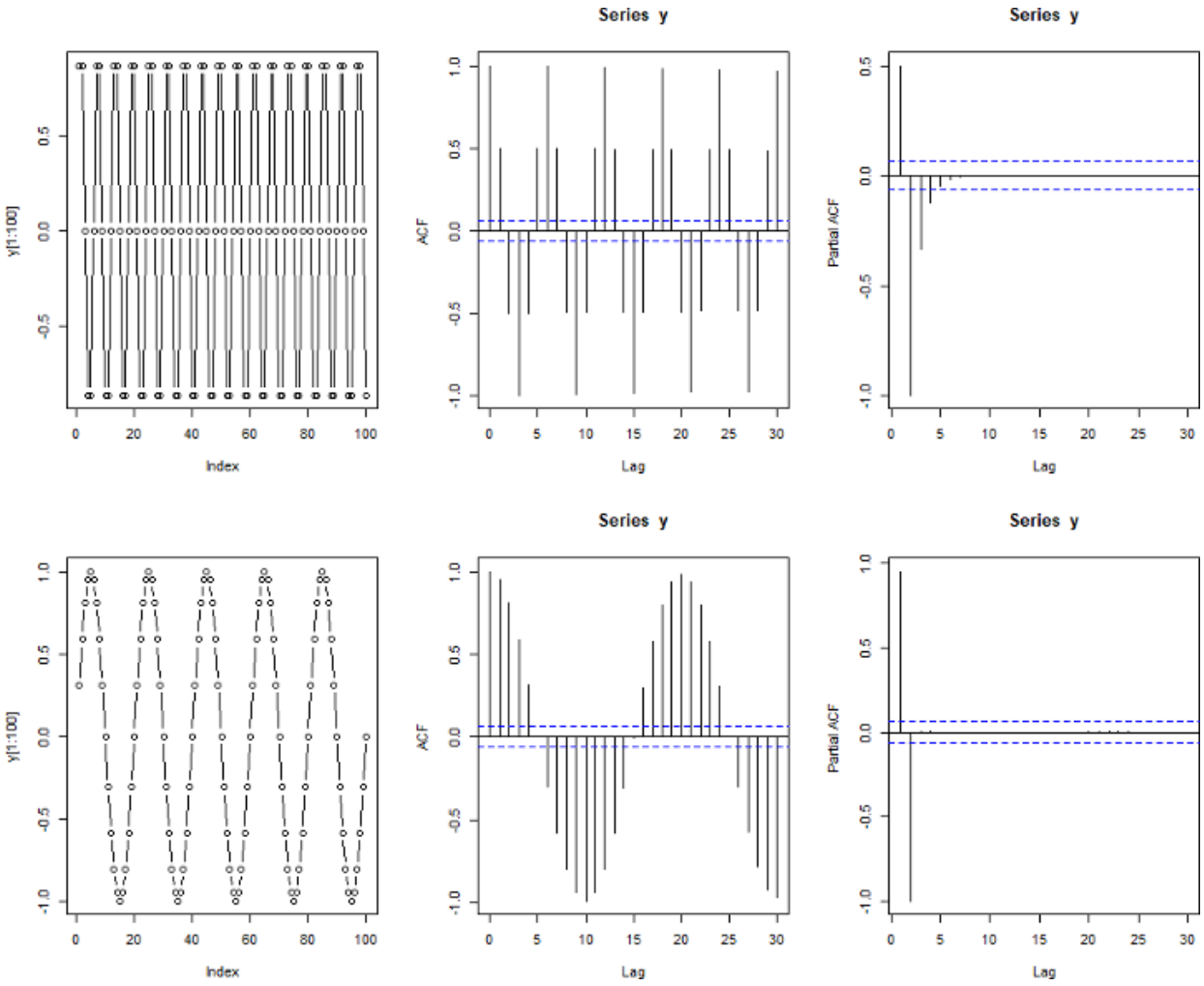


Figura 3-11. Gráficos de duas funções senoidais, seu ACF e seu PACF.

Combinamos as duas séries somando e criando os mesmos gráficos para as séries somadas (veja a Figura 3-12):

##R

```
> y <- y1 + y2

> plot(y, tipo = "b")

>acf (y)

>pacf(y)
```

Como podemos ver, o nosso gráfico ACF é consistente com a propriedade acima mencionada; a ACF

da soma de duas séries periódicas é a soma dos ACFs individuais. Você pode ver isso mais claramente observando as seções positiva → negativa → positiva → negativa do ACF correlacionando-se com o ACF de oscilação mais lenta. Dentro dessas ondas, você pode ver a flutuação mais rápida do ACF de frequência mais alta.

96 | Capítulo 3: Análise Exploratória de Dados para Séries Temporais

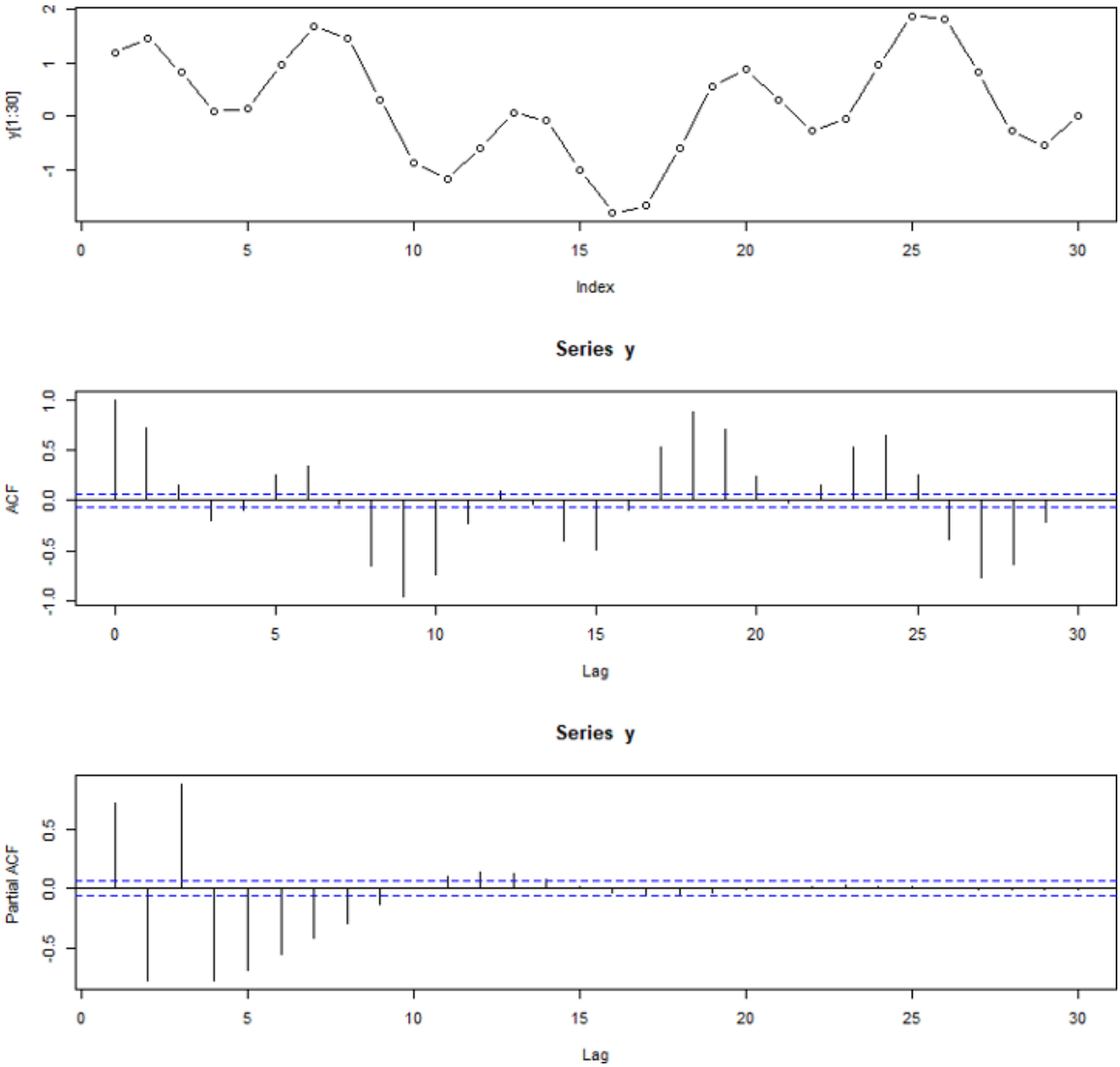
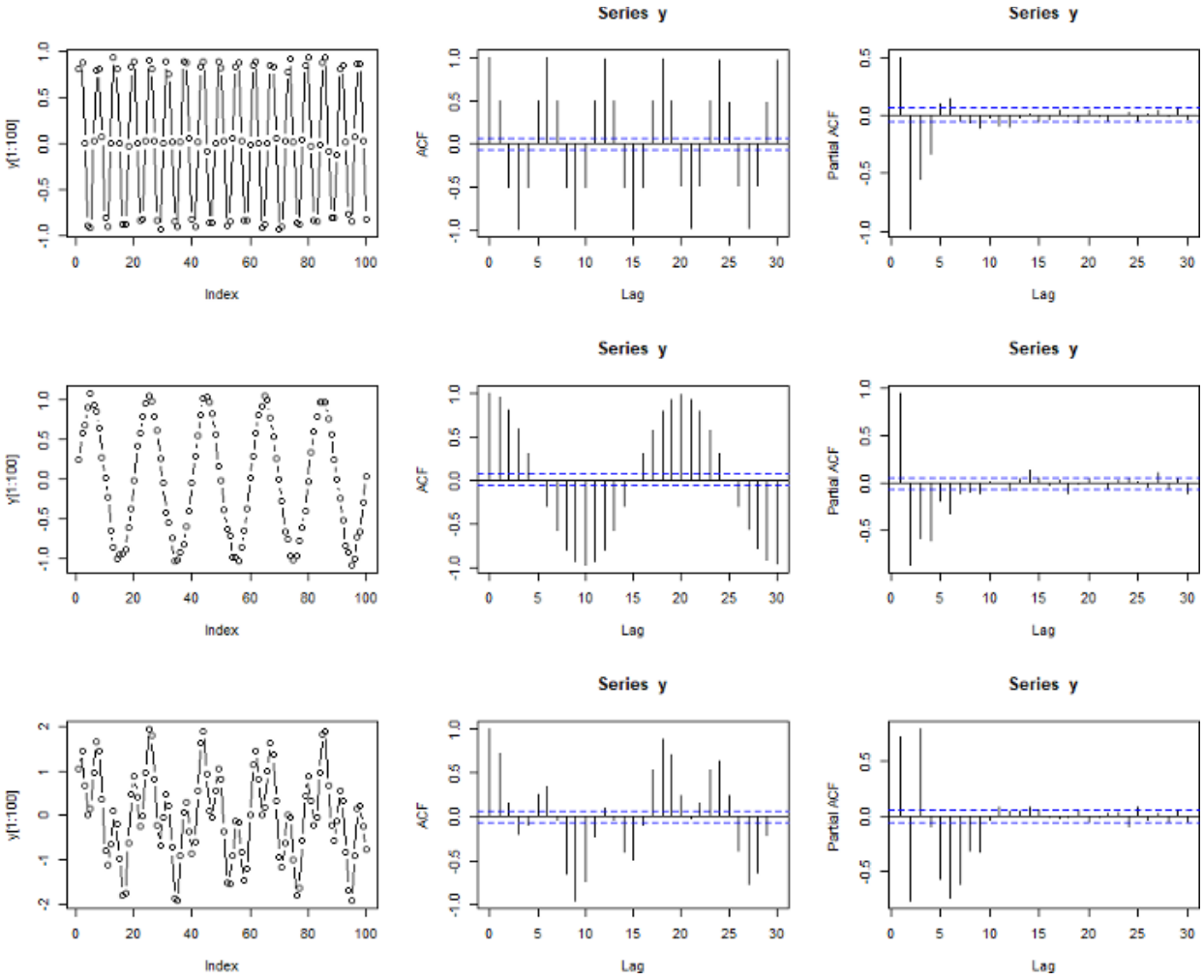


Figura 3-12. Plote, ACF e PACF para a soma das duas séries senoidais.

O PACF não é uma soma direta das funções do PACF dos componentes individuais. Um PACF é bastante simples de entender depois de calculado, mas não é tão fácil de gerar ou prever. Este PACF indica que a autocorrelação parcial é mais substancial na série somada do que em qualquer uma das séries originais. Ou seja, a correlação entre pontos separados por uma certa defasagem, quando contabilizados os valores dos pontos entre eles, é mais informativa na série somada do que na série original. Isto está relacionado com os dois períodos diferentes da série, o que faz com que qualquer ponto seja menos determinado pelos valores dos pontos vizinhos, uma vez que a localização dentro do ciclo dos dois períodos é menos fixa agora, à medida que as oscilações continuam em frequências diferentes.

Vejamos a mesma situação, mas com mais ruído (veja a Figura 3-13): **Métodos Exploratórios Específicos de Séries Temporais | 97**



##R

```
> ruído1 <- rnorma(100, sd = 0,05)
> ruído2 <- rnorma(100, sd = 0,05)
> y1 <- y1 + ruído1
> y2 <- y2 + ruído2
> y <- y1 + y2
> plot(y1, tipo = 'b')
>acf (y1)
```

>pacf(y1)

> plot(y2, tipo = 'b')

>acf (y2)

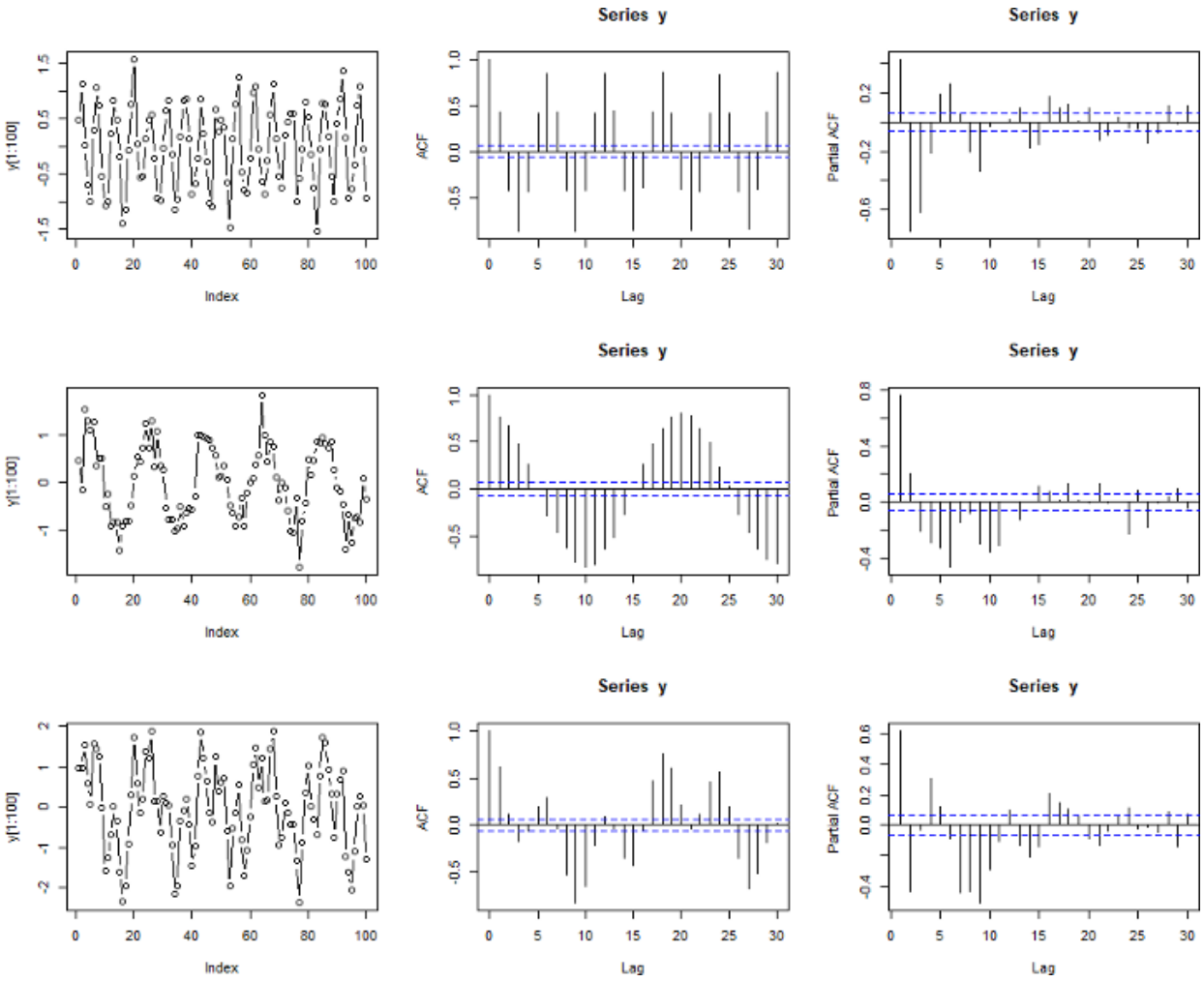
>pacf(y2)

> plot(y, tipo = 'b')

>acf (y)

>pacf(y)

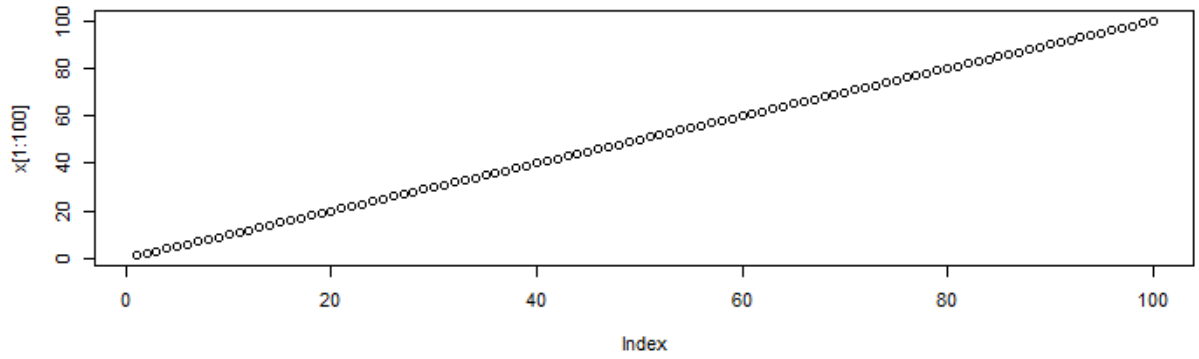
Figura 3-13. Gráfico, ACF e PACF de dois processos senoidais ruidosos e sua soma.

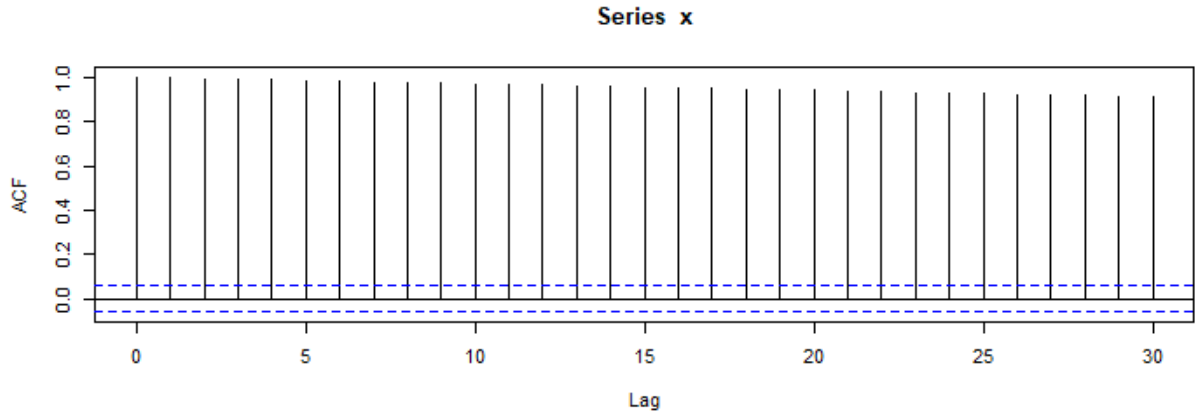


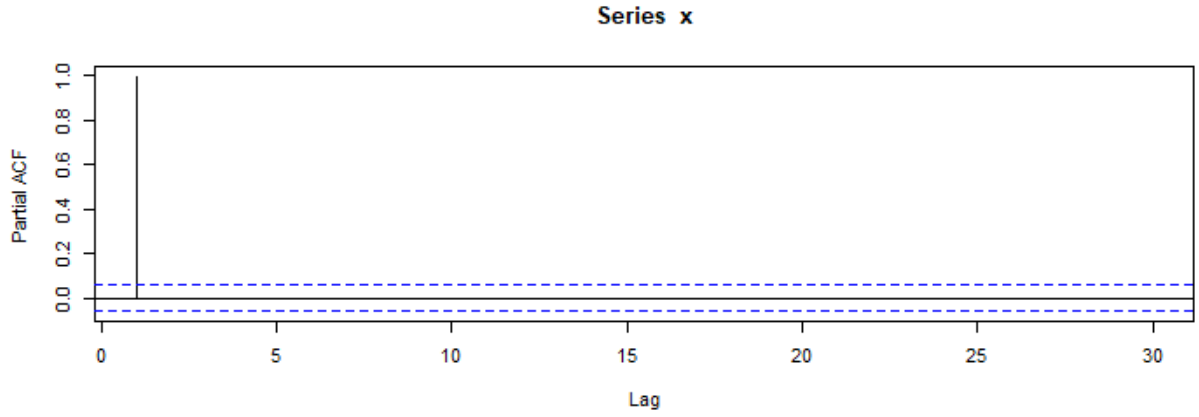
Finalmente, adicionamos mais ruído à série temporal para que os dados iniciais em si nem sequer pareçam particularmente senoidais. (Omitimos o exemplo de código porque é igual ao exemplo anterior, simplesmente com um parâmetro sd maior para rnorm.) Podemos ver que isso acrescenta mais dificuldades de interpretação, particularmente ao PACF. A única diferença

A diferença entre os gráficos da Figura 3-14 e os anteriores é um valor sd maior para as variáveis de ruído.

Figura 3-14. Gráfico, ACF e PACF da soma muito ruidosa de dois processos senoidais.







Dados não estacionários

Vamos considerar como ficariam o ACF e o PACF no caso de uma série com um

tendência, mas sem ciclo (ver Figura 3-15):

##R

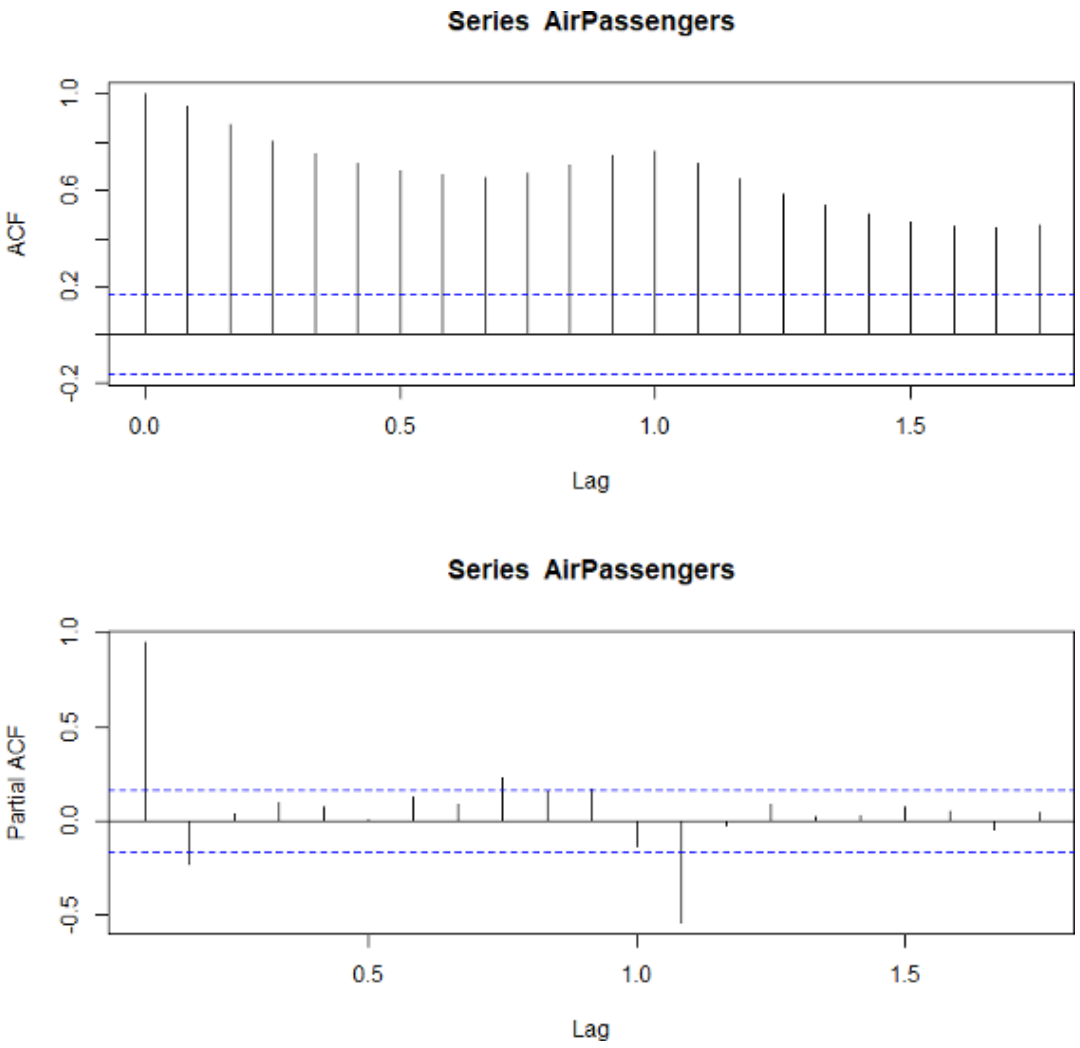
> x <- 1:100

> enredo (x)

>acf (x)

> pacf(x)

Figura 3-15. Gráfico, ACF e PACF de um processo de tendência linear.



O ACF não é informativo. Tem um valor semelhante para cada defasagem, aparentemente implicando que todas as defasagens estão igualmente correlacionadas aos dados. Não está claro o que isso significa, e é mais o caso de ser capaz de computar *algo* , não de ser capaz de tornar essa quantidade computada significativa ou sensata.

Felizmente, o PACF não é tão difícil de contemplar e nos dá a informação que precisamos, que é que a única correlação significativa do PACF está no atraso 1. Por que isso? Para um determinado momento, uma vez que você conheça o ponto imediatamente anterior, você saberá todas as informações que a série pode fornecer sobre o seu momento. Isso se deve ao fato de o próximo ponto no tempo ser 1 mais o ponto antigo.

Vamos concluir dando uma olhada no ACF e no PACF de um conjunto de dados do mundo real. Abaixo, examinamos os dados da AirPassengers. Dado o que vimos até agora, pense por que o ACF tem tantos valores “críticos” (resposta: tem uma tendência) e por que o PACF tem um valor crítico para um grande desfasamento (resposta: o ciclo sazonal anual, que é identificável mesmo com uma tendência nos dados).

Figura 3-16. Gráfico do ACF e PACF dos dados AirPassengers. Os atrasos aqui não são números de unidades porque as defasagens são expressas como frações de um ano. Isso ocorre porque o O conjunto de dados AirPassengers vem na forma de um objeto ts, que possui uma frequência integrada que pode ser usado para plotagem (e outros fins).

Correlações Espúrias

Aqueles que são novos na análise de séries temporais geralmente começarão com práticas exploratórias de dados padrão, como representar graficamente duas variáveis uma contra a outra e calcular sua correlação. O novo analista ficará muito entusiasmado no início do processo de exploração de dados, quando perceber o que parece ser uma correlação muito forte e um relacionamento útil. Eles continuarão procurando e encontrarão outras correlações

surpreendentemente altas; que sistema incrível! Eles desejarão ter começado a trabalhar com dados de séries temporais mais cedo em suas carreiras. Eles pensarão consigo mesmos: “Tudo o que faço é vencer”.

Em seguida, o analista fará uma pequena leitura sobre a análise de séries temporais (melhor cenário) ou apresentará suas descobertas a outra pessoa e perceberá que nem tudo faz sentido (não é o melhor cenário). Um cético apontará ao analista que as correlações são muito altas. Pode parecer que quaisquer dois valores estão relacionados. Mais problemas surgirão à medida que o analista repetir a sua análise com outros conjuntos de variáveis e descobrir que também eles têm correlações surpreendentemente altas. Em algum momento ficará claro que não pode haver tantas correlações verdadeiramente altas.

Essa trajetória é muito parecida com o início da história da econometria. No século XIX, quando os economistas começaram a pensar na ideia de um ciclo económico, alguns deles procuraram impulsionadores externos do ciclo, tais como manchas solares (um ciclo de 11 anos) ou vários ciclos meteorológicos (como um postulado). ciclo de precipitação de 4 anos).

Eles invariavelmente obtiveram resultados muito positivos e fortemente correlacionados, mesmo quando não tinham hipótese causal para explicar esses resultados.

Muitos economistas e estatísticos permaneceram céticos, e com razão. Udny Yule

investigou o problema formalmente com um artigo intitulado “Por que às vezes ficamos

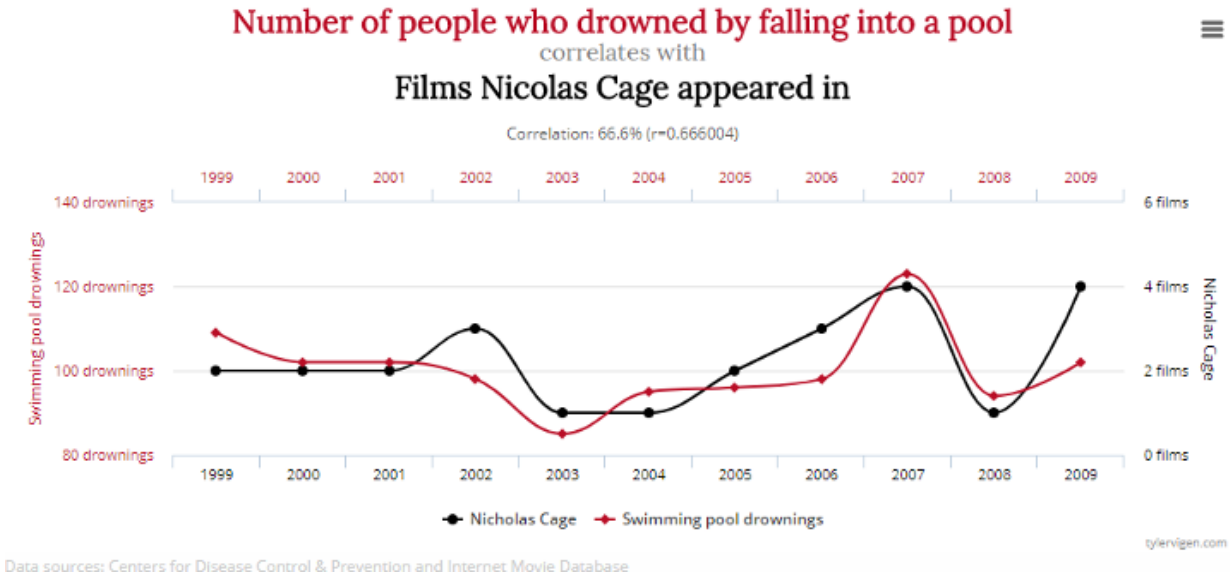
Correlações absurdas?”, e uma área de pesquisa nasceu e continua dando

problemas e prazer para os acadêmicos. As correlações espúrias continuam a ser um problema importante a evitar e são calorosamente debatidas em situações de litígio em que um lado afirma uma relação e o outro lado tenta desacreditá-la. Da mesma forma, uma tentativa de desacreditar os dados sobre alterações climáticas baseia-se

no argumento de que a correlação entre o aumento das emissões de carbono e o aquecimento das temperaturas globais é uma correlação espúria devido às tendências nos dois conjuntos de dados (não considero este argumento convincente).

Os economistas aprenderam ao longo do tempo que os dados com uma tendência subjacente são susceptíveis de produzir correlações espúrias. Esta é uma maneira simples de pensar sobre isso: há mais informações em uma série temporal de tendência do que em uma série temporal estacionária, portanto, há mais oportunidades para os pontos de dados se moverem juntos.

Além das tendências, algumas outras características comuns das séries temporais podem introduzir correlações espúrias:



• Sazonalidade – por exemplo, pense numa correlação espúria entre o consumo de cachorro-quente e a morte por afogamento (verão).

• Mudanças de nível ou inclinação nos dados decorrentes de mudanças de regime ao longo do tempo (produzindo uma

distribuição semelhante a halteres com alta correlação sem sentido).

• Quantidades somadas cumulativamente (esse é um truque usado em certos setores para fazer modelos ou correlações parecerem melhores do que são).

Cointegração

A cointegração refere-se a uma relação real entre duas séries temporais. Um exemplo comumente usado é um pedestre bêbado e seu cachorro. Suas caminhadas medidas individualmente podem parecer aleatórias, tomadas isoladamente, mas eles nunca se afastam muito um do outro.

No caso da cointegração você verá correlações altas. A dificuldade estará em avaliar se dois processos estão cointegrados ou se você está vendo uma correlação espúria porque em ambos os casos você verá correlações surpreendentemente altas. A diferença importante é que não é necessário haver qualquer relação no caso de uma correlação espúria, enquanto as séries temporais cointegradas estão fortemente relacionadas entre si.

Há um blog bem conhecido (e agora um livro) cheio de exemplos maravilhosos de correlações espúrias, e compartilho um deles na Figura 3-17 . Sempre que você se sentir tentado a pensar que encontrou um relacionamento especial e muito forte, certifique-se de verificar seus dados em busca de causas óbvias de problemas, como tendências.

Figura 3-17. Algumas correlações espúrias podem parecer surpreendentemente convincentes. Esta trama foi retirado do site de Tyler Vigen de correlações espúrias.

Algumas visualizações úteis

Os gráficos são fundamentais para uma análise exploratória completa de séries temporais. Você certamente desejará visualizar os dados em relação ao eixo temporal – de

preferência de uma forma que responda às perguntas gerais que você tem sobre o conjunto de dados, como o comportamento de uma variável específica ou a distribuição temporal geral dos pontos de dados.

Anteriormente neste capítulo, examinamos algumas técnicas de plotagem familiares a qualquer analista de dados, como um gráfico de valores em relação ao tempo ou um gráfico de dispersão de diferentes colunas.

valores ao longo do tempo. Nesta seção final sobre análise exploratória de dados, discutimos diversas visualizações que são particularmente úteis para oferecer novos insights sobre o comportamento de séries temporais.

Veremos visualizações de vários graus de complexidade:

• Uma visualização unidimensional para entender a distribuição temporal geral de indivíduos com uma série temporal encontrada que reunimos no Capítulo 2

• Um histograma bidimensional para entender a trajetória típica de um valor ao longo do tempo no caso de muitas medições paralelas (digamos, muitos anos medidos ou

muitas séries temporais do mesmo fenômeno medidas)

• Uma visualização tridimensional onde o tempo pode ocupar até duas das dimensões ou apenas nenhuma das dimensões, mas ainda assim estar implicitamente presente.

Visualizações 1D

Nos casos de muitas unidades de medida (muitos usuários, membros, etc.) consideramos múltiplas séries temporais em paralelo. Pode ser interessante empilhá-los visualmente, enfatizando unidades individuais de análise e seus respectivos prazos. Ignoramos os valores medidos e preferimos considerar a existência de dados em um determinado intervalo como a informação de interesse. O próprio intervalo de tempo se torna a unidade de análise. Aqui usamos o pacote timevis do R, mas existem muitas outras opções disponíveis. Examinamos um pequeno subconjunto de dados sobre doações que preparamos no Capítulo 2 (veja a Figura 3-18):

##R

> exigir (timevis)

> doações <- fread("donations.csv")

> d <- doações[, .(min(timestamp), max(timestamp)), usuário]

> nomes (d) <- c("conteúdo", "início", "fim")

> d <- d[início!= fim]

> timevis(d[amostra(1:nrow(d), 20)])

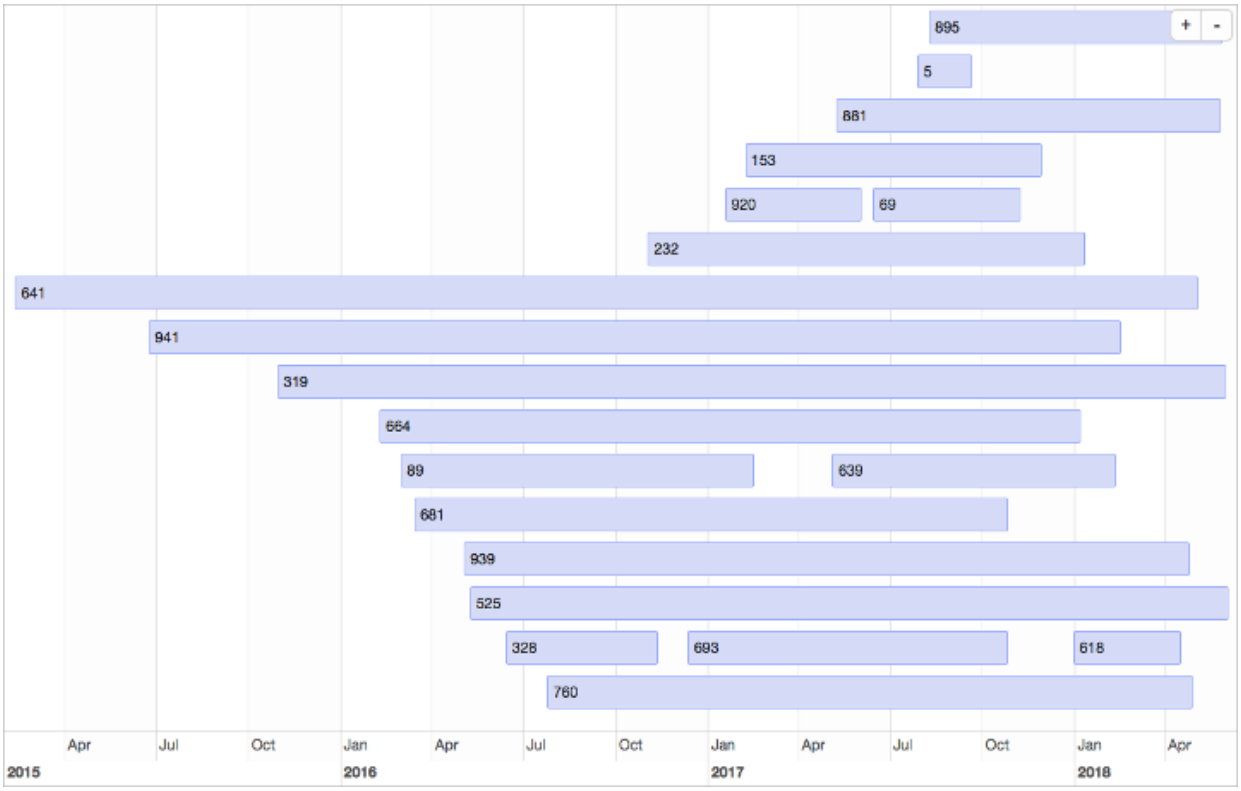


Figura 3-18. Um gráfico de Gantt de uma amostra aleatória de dados pode oferecer uma ideia da distribuição do intervalo de períodos "ativos" para os usuários/doadores.

O gráfico na Figura 3-18 nos ajuda a ver que provavelmente teremos períodos “ocupados” globalmente

em toda a população membro. Também obtemos alguma noção da distribuição dos períodos de doação ativa ao longo da “vida” de um membro em nossa organização.

Os gráficos de Gantt são usados há mais de um século, na maioria das vezes para tarefas de gerenciamento de projetos. Eles surgiram de forma independente em muitos setores diferentes, e a ideia é intuitiva assim que você vê um. Apesar das origens do gerenciamento de projetos, os gráficos de Gantt podem ser úteis na análise de séries temporais onde há muitos atores independentes,

em vez de um único processo sendo medido. O gráfico da Figura 3-18 respondeu rapidamente às perguntas que eu tinha sobre a sobreposição relativa entre toda a base de usuários em relação ao seu histórico de doações, uma distribuição que achei difícil de entender ao apenas ler os dados tabulares.

Visualizações 2D

Agora usaremos os dados da AirPassengers para ver a sazonalidade e a tendência, mas não devemos pensar no tempo como linear. Em particular, o tempo acontece em mais de um eixo.

Existe, é claro, o eixo do tempo avançando de dia para dia e de ano para ano, mas também podemos considerar definir o tempo ao longo do eixo da hora do dia ou do dia da semana, e assim por diante. Dessa forma, podemos pensar mais facilmente na sazonalidade, como determinados comportamentos acontecendo em determinado horário do dia ou mês do ano. Temos **algumas visualizações úteis | 105**



pense em particular em como entender nossos dados de maneira sazonal, em vez de apenas de acordo com visualizações lineares e cronológicas do tempo.

Extraímos os dados do objeto AirPassengers ts e os colocamos no formato de matriz apropriado:

##R

> t(matriz(AirPassengers, nrow = 12, ncol = 12))

[,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12]

[1,] 112 118 132 129 121 135 148 148 136 119 104 118

[2,] 115 126 141 135 125 149 170 170 158 133 114 140

[3,] 145 150 178 163 172 178 199 199 184 162 146 166

[4,] 171 180 193 181 183 218 230 242 209 191 172 194

[5,] 196 196 236 235 229 243 264 272 237 211 180 201

[6,] 204 188 235 227 234 264 302 293 259 229 203 229

[7,] 242 233 267 269 270 315 364 347 312 274 237 278

[8,] 284 277 317 313 318 374 413 405 355 306 271 306

[9,] 315 301 356 348 355 422 465 467 404 347 305 336

[10,] 340 318 362 348 363 435 491 505 404 359 310 337

[11,] 360 342 406 396 420 472 548 559 463 407 362 405

[12,] 417 391 419 461 472 535 622 606 508 461 390 432

Observe que tivemos que transpor os dados para que eles se alinhassem conforme apresentado pelo objeto ts.

Coluna Principal versus Linha Principal

R é a coluna principal por padrão, o que é incomum e diferente do NumPy do Python (linha principal) e também diferente da maioria dos SQL

bancos de dados. É bom estar ciente de quais comportamentos são padrão e

disponível em um determinado idioma, não apenas para fins de exibição, mas para

pense em como gerenciar e acessar a memória de maneira eficaz mais tarde.

Representamos cada ano num conjunto de eixos que refletem a progressão dos meses ao longo do ano (ver Figura 3-19):

##R

> cores <- c("verde", "vermelho", "rosa", "azul",

> "amarelo", "salmão claro", "preto", "cinza",

> "ciano", "azul claro", "marrom", "roxo")

> matplot(matriz(AirPassengers, nrow = 12, ncol = 12),

> tipo = 'l', col = cores, lty = 1, lwd = 2,5,

> xaxt = "n", ylab = "Contagem de passageiros")

> legenda("topo esquerdo", legenda = 1949:1960, lty = 1, lwd = 2,5,

> col = cores)

> eixo(1, às = 1:12, rótulos = c("Jan", "Fevereiro", "Março", "Abr",

> "maio", "junho", "julho", "agosto",

> "Set", "Out", "Nov", "Dez"))

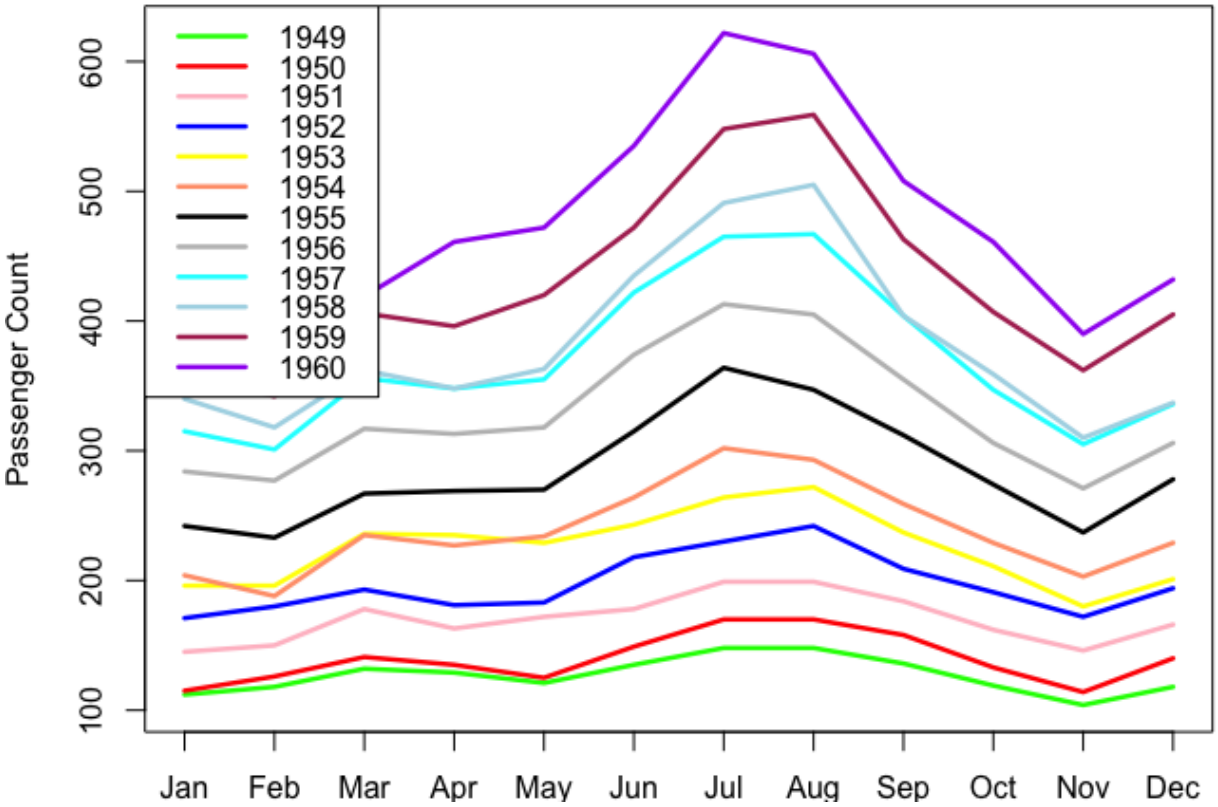


Figura 3-19. Contagens mensais por ano. [4]

Podemos produzir o mesmo gráfico mais facilmente com o pacote de previsão (veja

Figura 3-20):

##R

> exigir (previsão)

> gráfico de temporada (Passageiros Aéreos)

O eixo x é o mês do ano para todos os anos. Todos os anos, o número de passageiros aéreos atinge o pico em julho ou agosto (meses 7 e 8). Também vemos um pico local em março (mês 3) na maioria dos anos. Este gráfico pode assim nos mostrar mais detalhes sobre o comportamento da sazonalidade.

4 Visite o repositório GitHub para visualizar a figura original ou plote você mesmo para obter uma visão mais

detalhada.

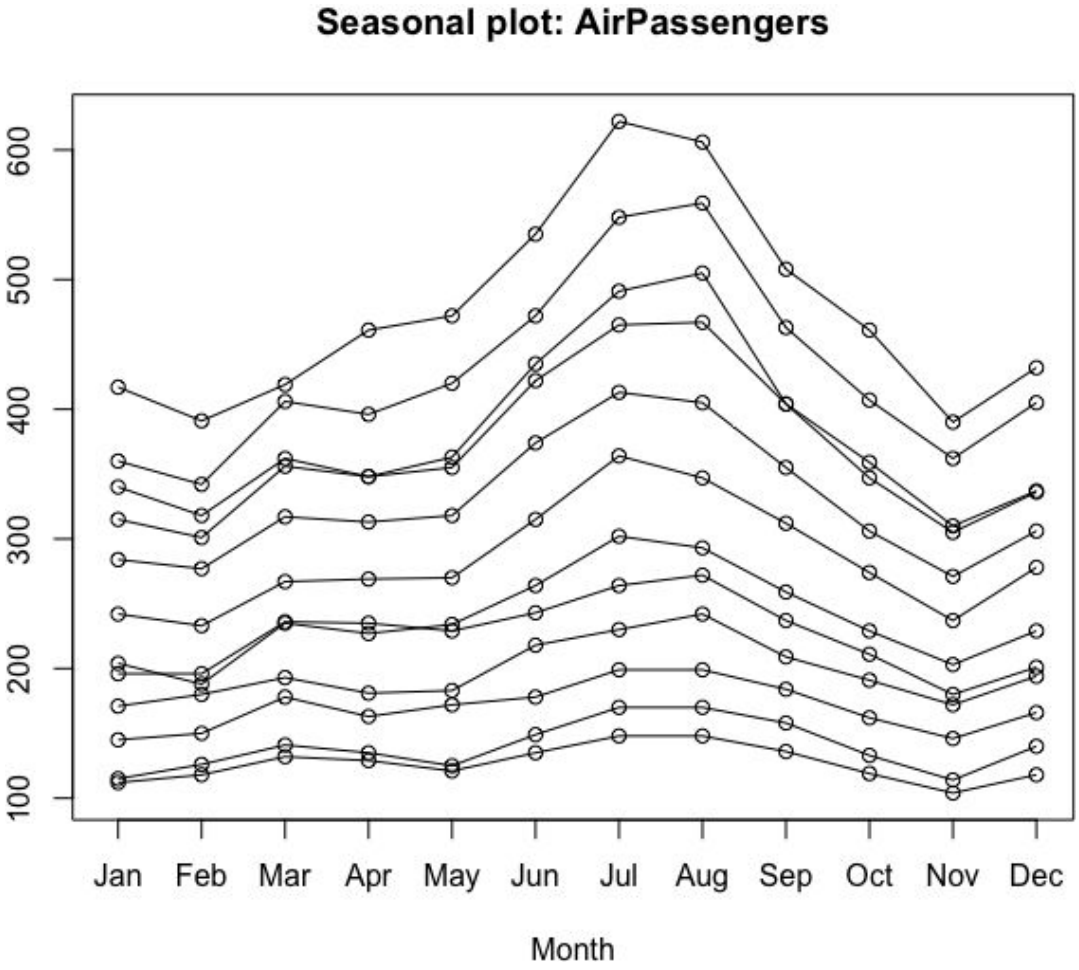


Figura 3-20. Produzimos um gráfico sazonal semelhante mais facilmente com seasonplot() função.

As curvas dos diferentes anos raramente se cruzam. O crescimento foi tão robusto que raramente acontecia que anos diferentes tivessem o mesmo número de passageiros no mesmo mês. Existem algumas exceções, mas não durante os meses de pico. Somente a partir dessas observações, já podemos produzir conselhos para uma empresa de viagens aéreas que busca tomar decisões sobre como planejar o crescimento.

Um gráfico alternativo de curvas mensais em relação aos anos é menos padrão, mas também útil

(veja a Figura 3-21):

##R

> meses <- c("janeiro", "fevereiro", "março", "abril", "maio", "junho",

> "julho", "agosto", "setembro", "outubro", "novembro", "dezembro")

> matplot(t(matriz(AirPassengers, nrow = 12, ncol = 12)),

> tipo = 'l', col = cores, lty = 1, lwd = 2,5)

> legenda("esquerda", legenda = meses,

> col = cores, lty = 1, lwd = 2,5)

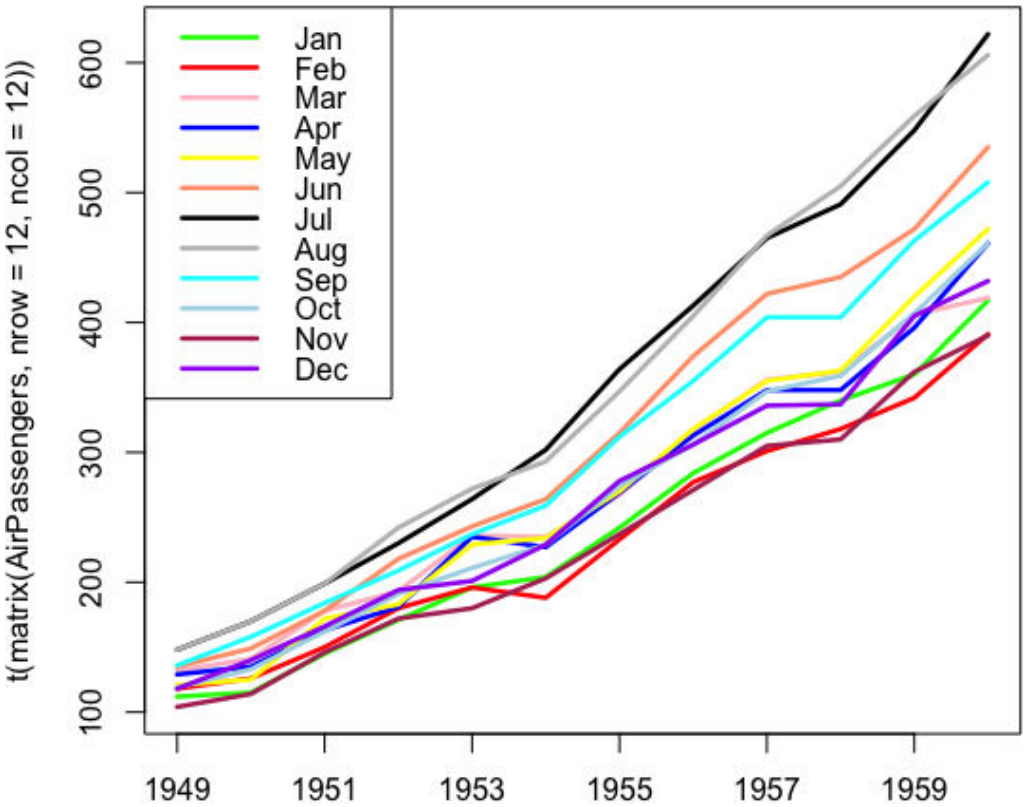


Figura 3-21. Curvas mensais de séries temporais ano a ano.5

Ao longo dos anos, a tendência de crescimento está a acelerar; isto é, a própria taxa de aumento está aumentando. Além disso, dois meses estão a crescer mais rapidamente que os outros, nomeadamente Julho e Agosto. Podemos obter uma visualização semelhante e insights semelhantes com uma função de visualização fácil fornecida pelo pacote de previsão (veja a Figura 3-22):

##R

> gráfico do mês (Passageiros Aéreos)

Existem duas observações gerais que podemos fazer a partir desses gráficos:

• As séries temporais têm mais de um conjunto útil de eixos temporais para serem plotados.

Usamos um eixo do mês do ano (janeiro a dezembro) e um

eixo de anos do conjunto de dados (do primeiro ano até o último/décimo segundo ano).

• Podemos obter muitas informações úteis e detalhes preditivos a partir de visualizações que empilham dados de séries temporais em vez de plotá-los linearmente.

5 Visite o repositório GitHub para visualizar a figura original ou plote você mesmo para obter uma visão mais detalhada.

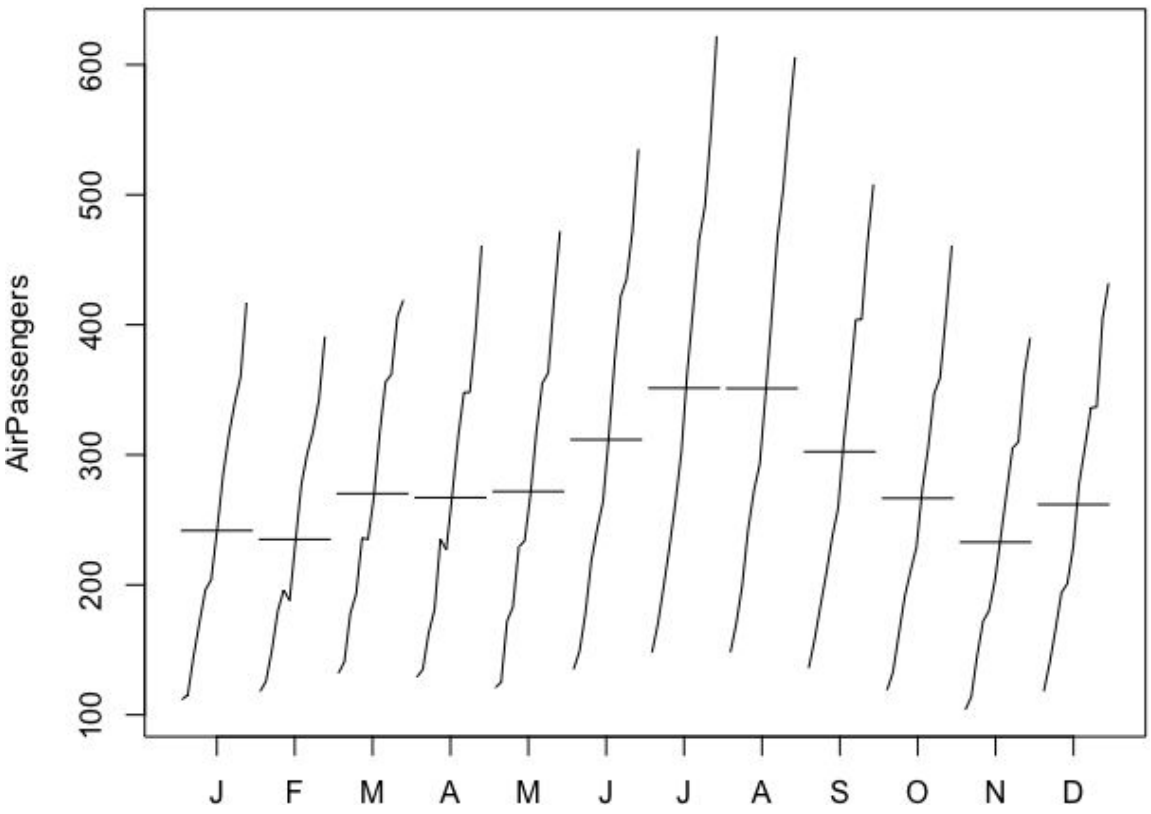


Figura 3-22. Usando a função Monthplot(), podemos ver como o desempenho por mês muda com o passar dos anos.

A seguir consideraremos um histograma bidimensional adequado. Num contexto de série temporal, podemos pensar num histograma bidimensional como tendo um eixo para o tempo (ou um proxy para o tempo) e outro eixo para uma unidade de interesse. Os gráficos “empilhados” que acabamos de fazer estão a caminho de se tornarem histogramas bidimensionais, mas algumas mudanças seriam necessárias.

útil:

• Precisamos agrupar dados tanto no eixo do tempo quanto no número de passageiros.

• Precisamos de mais dados. Um histograma 2D não faz sentido até que as curvas empilhadas se encontrem; eles não podem ser vistos adequadamente sozinhos. Caso contrário, o histograma 2D não consegue transmitir qualquer informação adicional.

Geramos o histograma 2D neste pequeno conjunto de dados para ilustração e depois passamos para um exemplo mais significativo. Construímos nossa própria função de histograma 2D do zero, assim:

##R

> hist2d <- função(dados, nbins.y, xlabels) {

> *## fazemos ybins espaçados uniformemente para incluir*

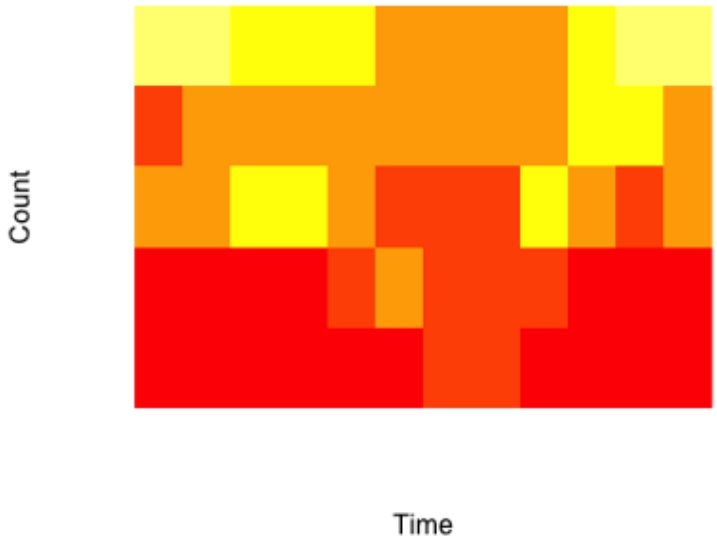
> *## pontos mínimo e máximo*

> ymin = min(dados)

> ymax = max(dados) × 1,0001

> *## a saída preguiçosa para evitar preocupações com inclusão/exclusão*

> ybins = seq(de = ymin, até = ymax, comprimento.out = nbins.y + 1)



> *## faça uma matriz zero do tamanho apropriado*

> hist.matrix = matriz(0, nrow = nbins.y, ncol = ncol(dados))

> *## os dados vêm em forma de matriz onde cada linha*

> *## representa um ponto de dados*

> for (i em 1:nrow(dados)) {

> ts = findInterval(dados[i,], ybins)

> for (j em 1:ncol(dados)) {

> hist.matriz[ts[j], j] = hist.matriz[ts[j], j] + 1

> }

> }

> hist.matriz

> }

Fazemos um histograma com coloração de mapa de calor, da seguinte forma:

##R

> h = hist2d(t(matriz(AirPassengers, nrow = 12, ncol = 12)), 5, meses)

> imagem(1:ncol(h), 1:nrow(h), t(h), col = heat.colors(5),

> eixos = **FALSE** , xlab = "Tempo", ylab = "Contagem de passageiros")

Contudo, a imagem resultante (Figura 3-23) não é muito satisfatória.

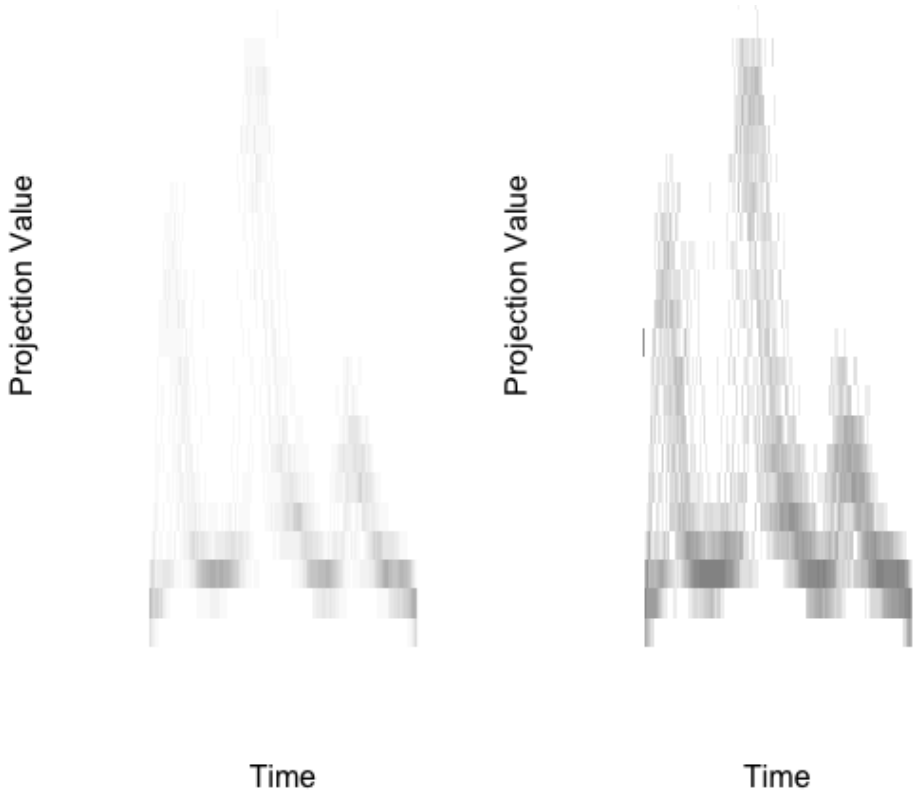
Figura 3-23. Mapa de calor construído a partir de nosso histograma 2D caseiro dos dados do AirPassengers.

Este gráfico não é útil porque não há dados suficientes. Temos apenas 12 curvas e as dividimos em 5 grupos. No entanto, uma questão ainda mais importante é que não temos dados estacionários. O uso de um histograma pressupõe um conjunto de dados estacionário.

Neste caso há uma tendência, então embora gostaríamos de ver a sazonalidade, a tendência necessariamente atrapalhará.

Agora olhamos para um conjunto de dados que apresenta um número maior de amostras e não está poluído por uma tendência. Este é um subconjunto do conjunto de dados FiftyWords retirado do UCR Time Ser-

Arquivo de Classificação ies . Este conjunto de dados inclui uma representação de 50 palavras diferentes registradas por uma série temporal univariada, sendo cada série temporal de igual comprimento.



O subconjunto de dados usado para traçar a Figura 3-24 é do conjunto de dados que usei em um segmento.

discussão sobre classificação de séries temporais em um tutorial geral sobre o assunto. Você pode descer

carregue esse subconjunto, embora para os propósitos deste exercício você possa usar qualquer um deles:

##R

> exigir(data.tabela)

> palavras <- fread(url.str)

> w1 <- palavras[V1 == 1]

> h = hist2d(w1, 25, 1:ncol(w1))

> cores <- cinza.colors(20, início = 1, fim = 0,5)

> par(mfrow = c(1, 2))

> imagem(1:ncol(h), 1:nrow(h), t(h),

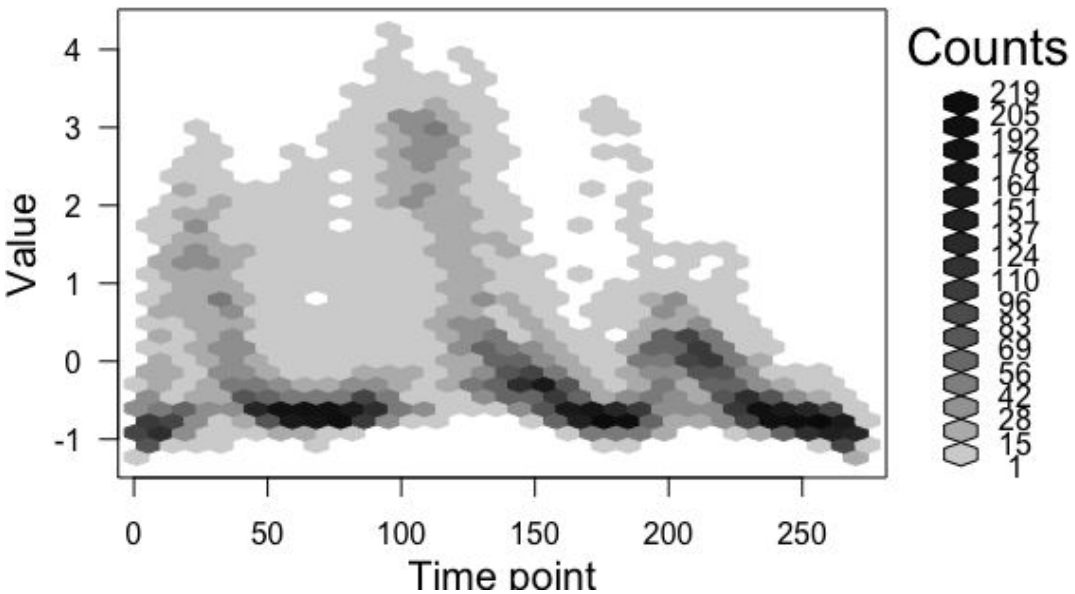
> col = cores, eixos = **FALSE** , xlab = "Tempo", ylab = "Valor da projeção")

> imagem(1:ncol(h), 1:nrow(h), t(**log** (h)),

> col = cores, axes = **FALSE** , xlab = "Tempo", ylab = "Valor da projeção") *Figura 3-24. Histograma bidimensional de uma métrica de áudio para uma única palavra. A esquerda o gráfico tem uma escala de contagem linear, enquanto o gráfico da direita tem uma escala de contagem logarítmica.*

O gráfico à direita da Figura 3-24 parece melhor porque as contagens são coloridas de acordo com uma transformação logarítmica da contagem, em vez da contagem diretamente.

Esta é uma aplicação da mesma ideia de obter o logaritmo de uma série temporal para diminuir a variância e reduzir a importância e a distância entre os valores discrepantes. O uso de uma transformação logarítmica melhora nossa visualização, pois não desperdiça uma grande parte do intervalo com valores de contagem alta relativamente esparsos.



Nossa opção caseira não será tão atraente visualmente quanto muitas opções pré-enlatadas, então devemos dar uma olhada nelas para ver o que há de diferente. Para aproveitar as vantagens das soluções predefinidas, precisamos remodelar nossos dados porque essas opções esperam que pares de valores xy sejam transformados em um histograma 2D. Ao contrário de nossa solução desenvolvida internamente, as opções predefinidas para histogramas 2D não são projetadas especificamente para dados de séries temporais. Mesmo assim, eles oferecem excelentes soluções de visualização (veja a Figura 3-25):

##R

> w1 <- palavras[V1 == 1]

> ## fundir os dados em pares de coordenadas emparelhadas

> ## esperado pela maioria das implementações de histograma 2D

> nomes(w1) <- c("tipo", 1:270)

> w1 <- derreter (w1, id.vars = "tipo")

> w1 <- w1[, -1]

> nomes(w1) <- c("Ponto no tempo", "Valor")

> plotar(hexbin(w1))

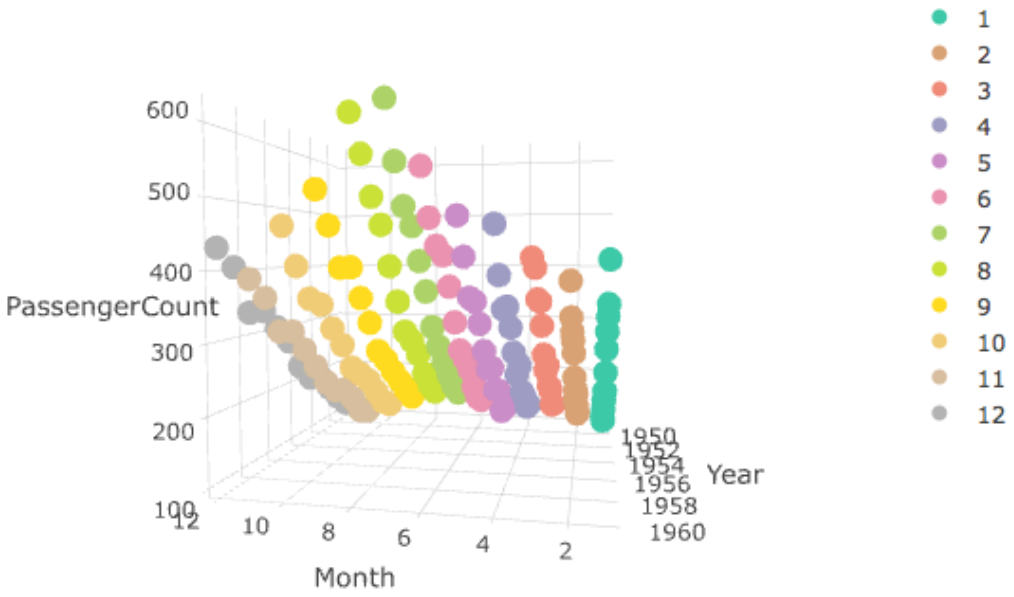
Figura 3-25. Uma visualização alternativa de histograma 2D dos mesmos dados.

Visualizações 3D

As visualizações 3D não vêm com base R, mas existem muitos pacotes disponíveis para elas. Aqui apresento algumas plotagens rápidas feitas com plotly, que escolhi porque produz plotagens que podem ser facilmente giradas no RStudio e exportadas para interfaces web. Além disso, baixar e instalar plotly tende a ser simples, o que não acontece com todos os pacotes de visualização.

Vamos considerar os dados da AirPassengers. Nós o plotamos em três dimensões, usando duas dimensões para tempo (mês e ano) e uma dimensão para valores de dados:

Algumas visualizações úteis | 113



##R

> exigir (enredo)

> exigir(data.tabela)

> meses = 1:12

> ap = data.table(matriz(AirPassengers, nrow = 12, ncol = 12))

> nomes(ap) = as.character(1949:1960)

> ap[, mês := meses]

> ap = derreter(ap, id.vars = 'mês')

>nomes(ap) = c("mês", "ano", "contagem")

```
> p <- plot_ly(ap, x = ~mês, y = ~ano, z = ~contagem,

> cor = ~as.factor(mês)) %>%

> add_markers() %>%

> layout(cena = lista(xaxis = lista(título = 'Mês'),

> yaxis = lista(título = 'Ano'),

> zaxis = lista(title = 'PassengerCount')))
```

Essa visualização 3D nos ajuda a ter uma noção da forma geral dos dados. Já vimos muito disso antes, mas a expansão para um gráfico de dispersão tridimensional prova ser notavelmente melhor do que um histograma bidimensional, talvez devido à escassez de dados (ver Figuras 3-26 e 3-27).

Figura 3-26. Um gráfico de dispersão 3D dos dados do AirPassenger. Essa perspectiva destaca a sazonalidade. 6

6 Visite o repositório GitHub para visualizar a figura original ou plote você mesmo para obter uma visão mais detalhada.

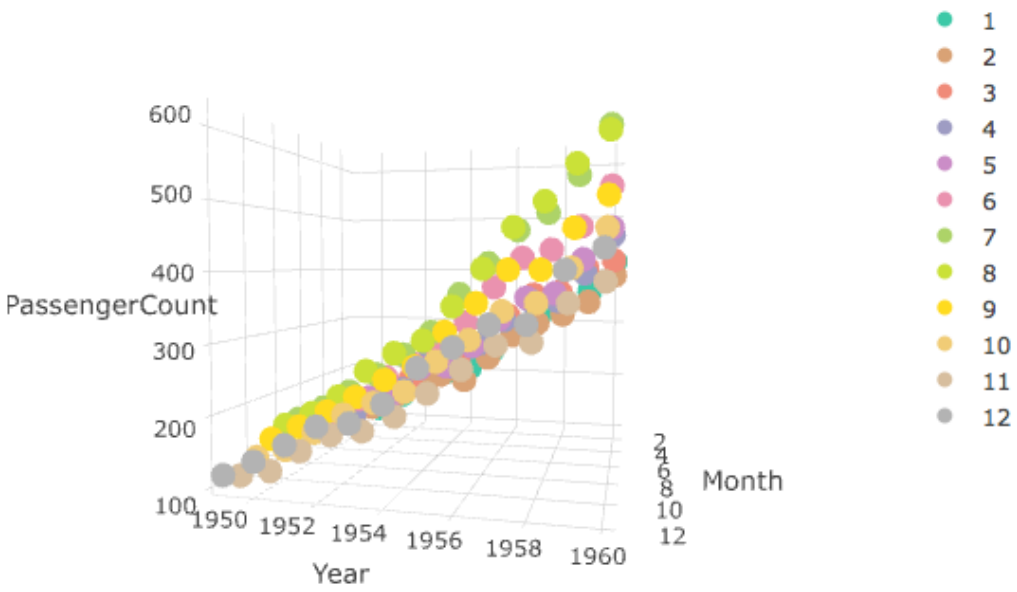


Figura 3-27. Outra perspectiva sobre os mesmos dados ilustra mais claramente a tendência crescente de um ano para o outro. Eu recomendo fortemente executar o código por conta própria computador, onde você mesmo pode girá-lo. [7]

Não precisamos necessariamente gastar dois eixos no tempo. Em vez disso, poderíamos usar dois eixos para localização e um para tempo. Podemos visualizar um passeio aleatório bidimensional como segue, com um exemplo levemente modificado de algum código de demonstração plotada (veja

Figuras 3-28 e 3-29):

##R

```
> arquivo.localização <-
'https://raw.githubusercontent.com/plotly/datasets/master/\
_3d-line-plot.csv'

> dados <- read.csv(arquivo.localização)

> p <- plot_ly(dados, x = ~x1, y = ~y1, z = ~z1,
```

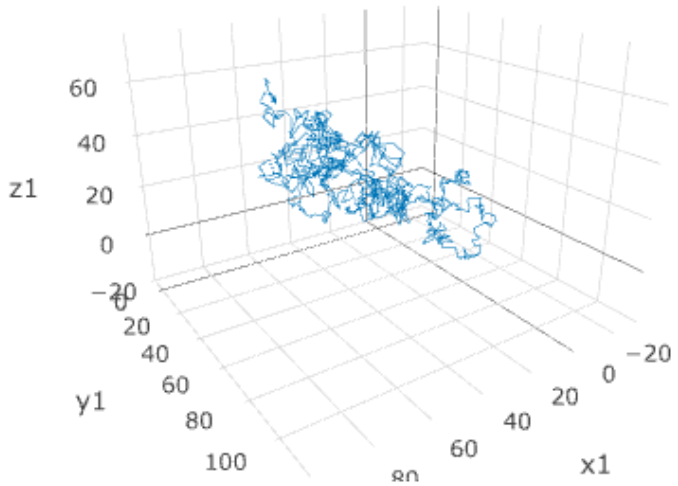
> tipo = 'scatter3d', modo = 'linhas',

> linha = lista(cor = '#1f77b4', largura = 1))

A natureza interativa do enredo é fundamental. Perspectivas diferentes podem ser enganosas ou esclarecedoras de maneiras que não podemos saber até que sejamos capazes de girar os dados.

7 Visite o repositório GitHub para visualizar a figura original ou plote você mesmo para obter uma visão mais detalhada.

Algumas visualizações úteis | 115



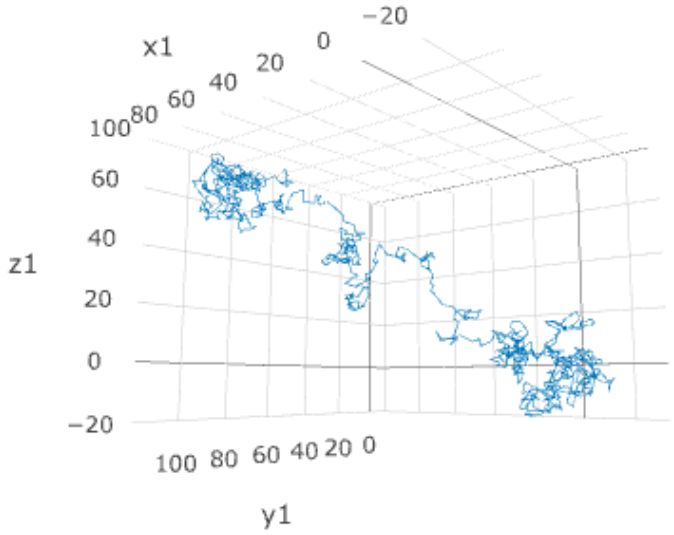


Figura 3-28. Uma perspectiva de um passeio aleatório bidimensional ao longo do tempo.

Figura 3-29. Esta perspectiva do mesmo passeio aleatório é muito mais reveladora. De novo, Eu encorajo você a experimentar este código por si mesmo!

Um bom exercício seria gerar dados de movimento sazonal ruidosos em duas dimensões e visualizá-los da mesma forma que visualizamos o passeio aleatório aqui. Deve haver uma diferença substancial no que você vê nesse gráfico em comparação com os dados do passeio aleatório. Pacotes como o plotly podem ajudá-lo a experimentar rapidamente e com feedback visual abrangente.

Mais recursos

• Sobre correlações espúrias:

Ai Deng, "Uma cartilha sobre significância estatística espúria em regressão de séries temporais

missões", *Boletim do Comitê de Economia 14, no. 1 (2015), https://perma.cc/9CQR-RWHC.*

Este artigo da indústria sobre o que são correlações espúrias e como elas

aparecem nos dados é útil para desenvolver alguns insights práticos sobre quando e onde procurar esse problema em seus próprios conjuntos de dados. O material é escrito em um nível muito acessível.

Tyler Vigen, Correlações Espúrias (Nova York: Hachette, 2015), https://perma.cc/

YY6R-SKWA.

Esta coleção de correlações ridículas de séries temporais é uma leitura essencial para qualquer analista de séries temporais ou pessoa pensante.

Antonio Noriega e Daniel Ventosa-Santaulària, "Regressão espúria sob

Estacionaridade de tendência quebrada", Journal of Time Series Analysis 27, não. 5 (2006): 671–

84, https://perma.cc/V993-SF4F.

Os autores desenvolvem dados teóricos e de simulação para mostrar que as mudanças na

o nível ou tendência de conjuntos de dados gerados de forma independente e aleatória que influenciam

presença de correlações espúrias.

CWJ Granger e P. Newbold, "Regressões Espúrias em Econometria", Diário

de Econometria 2, não. 2 (1974): 111–20, https://perma.cc/M8TE-AL6U.

Este artigo de econometria levou a um Prémio Nobel por identificar as dificuldades inerentes ao tratamento de correlações espúrias e defender uma abordagem mais robusta para identificar séries temporais relacionadas.

• Na análise exploratória de dados:

David R. Brillinger e Mark A. Finney, "Uma Análise Exploratória de Dados do

Flutuações de temperatura em um incêndio em propagação", Environmetrics 25, no. 6 (2014):

443–53, https://perma.cc/QB3D-APKM.

Este é um exemplo muito completo de como alguns dados laboratoriais do mundo real

com uma grade geotemporal foi analisada por acadêmicos e governos

pesquisadores.

Robert H. Shumway e David S. Stoffer, "Regressão de Séries Temporais e Exploratórias

Análise de dados," em análise de série temporal e suas aplicações com exemplos R (Nova York: Springer, 2011), https://perma.cc/UC5B-TPVS.

Este é um capítulo sobre análise exploratória de dados a partir da versão canônica dos autores.

tratado sobre análise de séries temporais para estudantes de pós-graduação.

Mais recursos | 117

• Mais visualizações:

Christian Tominski e Wolfgang Aigner, "O navegador TimeViz", https://

perma.cc/94ND-6ZA5.

Este impressionante catálogo mostra exemplos e códigos-fonte de muitos inter-

estimar visualizações de séries temporais de artigos de pesquisa acadêmica e

casos de uso da indústria.

Oscar Perpiñán Lamigueiro, "Repositório GitHub para exibição de séries temporais, Spa-

dados espaciais e espaço-temporais com R," https://perma.cc/R69Y-5JPL.

Isso inclui código-fonte para uma variedade de visualizações de séries temporais baseadas em R, incluindo dados de séries temporais geoespaciais.

Myles Harrison, "5 maneiras de fazer histogramas 2D em R", R-bloggers, 1º de setembro de

2014, https://perma.cc/ZCX9-FQQY.

Este é um guia prático para uma variedade de opções oferecidas pelos pacotes R para construir histogramas 2D e dar-lhes cores e categorizações significativas. Em

além de uma visão geral básica, um segmento relacionado também fornece um passo a passo

do pacote tidyquant para visualização de dados do mercado de ações, um importante

fonte de séries temporais.

• Em diversas tendências:

Halbert White e Clive WJ Granger, "Considerações sobre tendências em séries temporais",

Journal of Time Series Econometrics 3, não. 1 (2011), https://perma.cc/WF2H-TVTL.

Este recente artigo acadêmico aponta que, embora as tendências sejam onipresentes em

dados, mesmo as estatísticas tradicionais não possuem um conjunto definitivo de definições para descrever diferentes tipos de tendências nos dados. Num estilo acessível, os autores oferecem insights estatísticos sobre diferentes maneiras pelas quais os dados não estacionários podem ter uma tendência subjacente e orientações sobre como melhorar a estatística.

métodos para dados.

CAPÍTULO 4

Simulação de dados de série temporal

Até este ponto, discutimos onde encontrar dados de séries temporais e como processá-los. Agora veremos como criar dados de série temporal por meio de simulação.

Nossa discussão se desenvolve em três partes. Primeiro, comparamos simulações de dados de séries temporais com outros tipos de simulações de dados, observando que novas áreas de particular preocupação emergem quando temos que levar em conta a passagem do tempo. Em segundo lugar, examinamos algumas simulações baseadas em código. Terceiro, discutimos algumas tendências gerais na simulação de séries temporais.

A maior parte deste capítulo se concentrará em exemplos de código específicos para gerar vários tipos de dados de série temporal. Examinaremos os seguintes exemplos:

• Simulamos o comportamento de abertura de e-mail e doação de membros de uma organização sem fins lucrativos

organização ao longo de vários anos. Isso está relacionado aos dados que examinamos em “Retrofiting a Time Series Data Collection from a Collection of Tables” na página 26 .

• Simulamos eventos em uma frota de táxis de 1.000 veículos com vários horários de início de turno e frequências de coleta de passageiros dependentes do horário do dia durante o

curso de um único dia.

• Simulamos a evolução passo a passo do estado de um sólido magnético para uma determinada temperatura e tamanho usando leis relevantes da física.

Esses três exemplos de código estão correlacionados a três classes de simulações de séries temporais: *Simulações heurísticas*

Nós decidimos como o mundo deveria funcionar, garantimos que faz sentido e o codificamos,

uma regra de cada vez.

Simulações de eventos discretos

Construímos atores individuais com certas regras em nosso universo e depois executamos esses atores para ver como o universo evolui ao longo do tempo.

Simulações baseadas em física

Aplicamos leis físicas para ver como um sistema evolui ao longo do tempo.

A simulação de séries temporais pode ser um exercício analítico valioso e também demonstraremos em capítulos posteriores no que se refere a modelos específicos.

O que há de especial na simulação de séries temporais?

A simulação de dados é uma área da ciência de dados raramente ensinada, mas que é uma habilidade particularmente útil para dados de séries temporais. Isto

decorre de uma das desvantagens de ter dados temporais: não existem dois pontos de dados na mesma série temporal que sejam exatamente comparáveis, uma vez que ocorrem em momentos diferentes. Se quisermos pensar no *que poderia ter acontecido em um determinado momento* , entramos no mundo da simulação.

As simulações podem ser simples ou complexas. No lado mais simples, você encontrará dados sintéticos em qualquer livro de estatística sobre séries temporais, como na forma de um passeio aleatório. Geralmente são gerados como somas cumulativas de um processo aleatório (como a norma de R) ou por uma função periódica (como uma curva senoidal). No lado mais complexo, muitos cientistas e engenheiros fazem suas carreiras simulando séries temporais.

As simulações de séries temporais continuam sendo uma área ativa de pesquisa - e computacionalmente exigente - em muitos campos, incluindo:

• Meteorologia

• Finança

• Epidemiologia

• Química Quântica

• Física do plasma

Em alguns destes casos, as regras fundamentais de comportamento são bem compreendidas, mas ainda pode ser difícil explicar tudo o que pode acontecer devido à complexidade das equações (meteorologia, química quântica, física dos plasmas). Noutros casos, nem todas as variáveis preditivas poderão ser conhecidas e os especialistas nem sequer têm a certeza de que possam ser feitas previsões perfeitas devido à natureza estocástica não linear dos sistemas estudados (finanças, epidemiologia).

Simulação versus previsão

Simulação e previsão são exercícios semelhantes. Em ambos os casos, você deve formar hipóteses sobre a dinâmica e os parâmetros subjacentes do sistema e, em seguida, extrapolar a partir dessas hipóteses para gerar pontos de dados.

No entanto, existem diferenças importantes a ter em mente ao aprender e desenvolver simulações em vez de previsões:

• Pode ser mais fácil integrar observações qualitativas numa simulação do que numa previsão.

• As simulações são executadas em escala para que você possa ver muitos cenários alternativos (milhares ou mais), enquanto as previsões devem ser produzidas com mais cuidado.

• As simulações têm riscos menores do que as previsões; não há vidas nem recursos em jogo, então você pode ser mais criativo e exploratório em suas rodadas iniciais de simulações. É claro que você eventualmente deseja ter certeza de que pode justificar como constrói suas simulações, assim como deve justificar suas previsões.

Simulações em código

A seguir, veremos três exemplos de codificação de simulações de séries temporais. Ao ler esses exemplos, considere a grande variedade de dados que pode ser simulada para produzir um

dia das doações, mas também pode ser muito inespecífico e essencialmente não rotulado, como o “ n^{o} passo” de uma simulação de física.

Os três exemplos de simulação que discutiremos nesta seção são:

• Simular um conjunto de dados sintéticos para testar nossas hipóteses sobre como os membros de uma organização podem (ou não) ter um comportamento correlacionado entre a receptividade ao e-mail

organizacional e a disposição para fazer doações. Este é o mais DIY

por exemplo, codificamos relacionamentos e geramos dados tabulares com loops for e similares.

• Simulação do conjunto de dados sintéticos para explorar o comportamento agregado em uma frota de táxis, completo com horários de turno e frequência de passageiros dependente da hora do dia. Neste conjunto de dados, usamos atributos orientados a objetos do Python, bem como geradores, que são bastante úteis quando queremos colocar um sistema em funcionamento e ver o que ele faz.

• Simular o processo físico de um material magnético orientando gradualmente seus elementos magnéticos individuais, que começam desordenados, mas acabam se unindo. **Simulações em Código | 121**

em um sistema bem ordenado. Neste exemplo, vemos como as leis físicas podem conduzir uma simulação de série temporal e inserir escala temporal natural em um processo.

Fazendo o trabalho sozinho

Ao programar simulações, você precisa ter em mente as regras lógicas que se aplicam ao seu sistema. Aqui mostramos um exemplo em que o programador faz a maior parte do trabalho para garantir que os dados façam sentido (por exemplo, não especificando eventos que acontecem em uma ordem ilógica).

Começamos definindo o universo de membros – ou seja, quantos membros temos e quando cada um ingressou na organização. Também associamos cada membro a um status de membro:

```
## Pitão

>>> ## status de membro

>>> anos = ['2014', '2015', '2016', '2017', '2018']

>>> memberStatus = ['bronze', 'prata', 'ouro', 'inativo']
```

```
>>> memberYears = np.random.choice(anos, 1000,

>>> p = [0,1, 0,1, 0,15, 0,30, 0,35])

>>> memberStats = np.random.choice(memberStatus,
1000,

>>> p = [0,5, 0,3, 0,1, 0,1])

>>> anoJoined = pd.DataFrame({'yearJoined':
memberYears,

>>> 'membroStats':membroStats})
```

Observe que já existem muitas regras/suposições incorporadas na simulação apenas a partir dessas linhas de código. Impomos probabilidades específicas dos anos em que os membros aderiram. Também tornamos o status do membro totalmente independente do ano em que ingressou. No mundo real, é provável que já possamos fazer melhor do que isto porque estas duas variáveis devem ter alguma ligação, especialmente se quisermos incentivar as pessoas a permanecerem membros.

Fazemos uma tabela indicando quando os membros abriram emails a cada semana. Neste caso, definimos o comportamento da nossa organização: enviamos três emails por semana. Também definimos diferentes padrões de comportamento dos membros em relação ao email:

• Nunca abrindo e-mail

• Nível constante de engajamento/taxa de abertura de e-mail

• Aumentar ou diminuir o nível de envolvimento

Podemos imaginar maneiras de tornar isso mais complexo e matizado, dependendo das circunstâncias.

observações pontuais de veteranos ou novas hipóteses que temos sobre processos não observáveis que afetam os dados:

```
## Pitão

>>> NUM_EMAILS_SENT_WEEKLY = 3

>>> ## definimos diversas funções para diferentes padrões

>>> def never_opens(período_rng):

>>> retornar []

>>> def constante_open_rate(período_rng):

>>> n, p = NUM_EMAILS_SENT_WEEKLY, np.random.uniform(0, 1)

>>> num_opened = np.random.binomial(n, p, len(período_rng))

>>> retornar num_opened

>>> def aumentando_open_rate(período_rng):

>>> retornar open_rate_with_factor_change(período_rng,

>>> np.random.uniform(1.01,

>>> 1h30))


>>> def taxa_de_abertura decrescente(período_rng):

>>> retornar open_rate_with_factor_change(período_rng,

>>> np.random.uniform(0,5,

>>> 0,99))
```

```
>>> def open_rate_with_factor_change(período_rng, fac):
>>> se len(período_rng) < 1 :
>>> retornar []
>>> vezes = np.random.randint(0, len(período_rng),
>>> int(0,1 * len(período_rng)))
>>> num_opened = np.zeros(len(período_rng))
>>> para prd em range(0, len(period_rng), 2):
>>> tente :
>>> n, p = NUM_EMAILS_SENT_WEEKLY,
np.random.uniform(0,
>>> 1)
>>> num_opened[prd:(prd + 2)] = np.random.binomial(n,
p,
>>> 2)
>>> p = máx(min(1, p * fac), 0)
>>> exceto :
>>> num_opened[prd] = np.random.binomial(n, p, 1)
>>> para t no intervalo(len(vezes)):
>>> num_opened[vezes[t]] = 0
>>> retornar num_opened
```

Definimos funções para simular quatro tipos distintos de comportamento:

Membros que nunca abrem os e-mails que lhes enviamos

(nunca_abre())

Membros que abrem aproximadamente o mesmo número de e-mails por semana

(constante_open_rate())

Membros que abrem um número decrescente de e-mails a cada semana

(diminuindo_open_rate())

Membros que abrem um número crescente de e-mails a cada semana (increasing_open_rate())

Garantimos que aqueles que ficam cada vez mais engajados ou desengajados ao longo do tempo sejam simulados da mesma forma com a função open_rate_with_factor_change() por meio das funções aumentando_open_rate() e diminuindo_open_rate().

Também precisamos criar um sistema para modelar o comportamento de doação. Não queremos ser totalmente ingênuos, ou nossa simulação não nos dará insights sobre o que devemos esperar. Ou seja, queremos incorporar no modelo as nossas hipóteses atuais sobre o comportamento dos membros e depois testar se as simulações baseadas nessas hipóteses correspondem ao que vemos nos nossos dados reais. Aqui, tornamos o comportamento de doação vagamente, mas não deterministicamente relacionado ao número de e-mails que um membro abriu:

Pitão

>>> ## comportamento de doação

>>> **def** produzir_doações(período_rng, comportamento_membro, num_emails,

>>> use_id, member_join_year):

```
>>> valores_doações = np.array([0, 25, 50, 75, 100, 250,
500,

>>> 1000, 1500, 2000])

>>> member_has = np.random.choice(donation_amounts)

>>> email_fraction = num_emails /

>>> (NUM_EMAILS_SENT_WEEKLY * len(período_rng))

>>> membro_gives = membro_has * email_fraction

>>> membro_dá_idx = np.where(membro_dá

>>> >= doações_valores)[0][-1]

>>> membro_dá_idx = max(min(membro_dá_idx,

>>> len(valores_doação) - 2),

>>> 1)

>>> num_times_gave = np.random.poisson(2) *

>>> (2018 - member_join_year)

>>> vezes = np.random.randint(0, len(período_rng),
num_times_gave)

>>> dons = pd.DataFrame({'membro': [],

>>> 'quantia': [],

>>> 'carimbo de data e hora': []})


>>> para n no intervalo (num_times_gave):

>>> doação = doações_valores[member_gives_idx
```

```
>>> + np.random.binomial(1, .3)]

>>> ts = str(período_rng[tempos[n]].start_time

>>> + random_weekly_time_delta())

>>> dons = dons.append(pd.DataFrame(

>>> {'membro': [use_id],

>>> 'quantia': [doação],

>>> 'carimbo de data e hora': [ts]}))

>>>

>>> se dons.shape[0] > 0:

>>> dons = dons[dons.amount != 0]

>>> ## não reportamos nenhum evento de doação, pois isso não aconteceria

>>> ## ser registrado em um banco de dados do mundo real
```



```
>>>

>>> retornar dons
```

Seguimos algumas etapas aqui para garantir que o código produza um comportamento realista:

• Fazemos com que o número total de doações dependa de quanto tempo alguém tem

sido membro.

• Geramos um status de riqueza por membro, construindo uma hipótese sobre o comportamento de que o valor da

doação está relacionado a um valor estável que uma pessoa teria reservado para fazer doações.

Como o comportamento dos nossos membros está vinculado a um carimbo de data/hora específico, temos que escolher em quais semanas cada membro fez doações e também quando durante essa semana eles farão doações.

fez a doação. Escrevemos uma função de utilidade para escolher um horário aleatório durante a semana:

Pitão

>>> **def** random_weekly_time_delta():

>>> dias_da_semana = [d **para** d **no** intervalo (7)]

>>> horas_do_dia = [h **para** h **no** intervalo(11, 23)]

>>> minuto_da_hora = [m **para** m **no** intervalo (60)]

>>> segundo_de_minuto = [s **para** s **no** intervalo(60)]

>>> **retornar**
pd.Timedelta(str(np.random.choice(dias_da_semana))

>>> + "dias") +

>>> pd.Timedelta(str(np.random.choice(horas_do_dia))

>>> + "horas") +

>>> pd.Timedelta(str(np.random.choice(minuto_de_hora))

>>> + "minutos") +

>>>
pd.Timedelta(str(np.random.choice(segundo_de_minuto))

>>> + " segundos")

Você deve ter notado que desenhamos apenas a hora do carimbo de data/hora no intervalo de 11 a 23 (horas_do_dia

= [h para h no intervalo(11, 23)]). Estamos postulando um universo com pessoas em uma faixa muito limitada de fusos horários ou mesmo em um único fuso horário, pois não permitimos horas fora da faixa dada. Aqui estamos construindo mais do nosso modelo subjacente sobre como os usuários se comportam.

Esperamos, portanto, ver um comportamento unificado de nossos usuários, como se estivessem todos em um ou alguns fusos horários adjacentes, e estamos postulando ainda que o comportamento de doação razoável é que as pessoas doem desde o final da manhã até tarde da noite, mas não durante a noite e não primeira coisa quando eles acordam.

Finalmente, juntamos todos os componentes que acabamos de desenvolver para simular um certo número de membros e eventos associados de uma forma que garanta que os eventos aconteçam apenas quando um membro se juntar e que os eventos de email de um membro tenham alguma relação (mas não uma relação irrealisticamente pequena) aos seus eventos de doação:

Pitão

>>> comportamentos = [nunca_abre,

>>> taxa_aberta_constante,

>>> aumentando_open_rate,

>>> diminuindo_taxa_aberta]

>>> member_behaviors =
np.random.choice(comportamentos, 1000,

>>> [0,2, 0,5, 0,1, 0,2])

>>> rng = pd.period_range('2015-02-14', '2018-06-01', freq
= 'W')

>>> emails = pd.DataFrame({'membro': [],

>>> 'semana': [],

>>> 'e-mails abertos': []})

>>> doações = pd.DataFrame({'membro' : [],

>>> 'quantia': [],

>>> 'carimbo de data e hora': []})

>>> **para** idx **no** intervalo (yearJoined.shape[0]):

>>> *## gera aleatoriamente a data em que um membro
teria aderido*

>>> data_de_junção =
pd.Timestamp(anoJoined.iloc[idx].anoJoined) +

>>> pd.Timedelta(str(np.random.randint(0, 365)) +

>>> 'dias')

>>> data_de_junção = min(data_de_junção, pd.Timestamp('01/06/2018'))

>>>

>>> *## membro não deve ter carimbos de data/hora de ação antes de ingressar*

>>> membro_rng = rng[rng > data_de_junção]

>>>

>>> **se** len(membro_rng) <1:

>>> **continuar**

>>>

>>> info = member_behaviors[idx](member_rng)

>>> **if** len(info) == len(membro_rng):

>>> emails = emails.append(pd.DataFrame(

>>> {'membro': [idx] * len(info),

>>> 'semana': [str(r.start_time) **para** r **em** member_rng],

>>> 'e-mails abertos': info}))

>>> doações = doações.append(

>>> produzir_doações(membro_rng, comportamento_membro[idx],

>>> sum(info), idx, join_date.year)) Em seguida, analisamos o comportamento temporal das doações para ter uma noção de como podemos tentar fazer isso para análises ou previsões adicionais. Traçamos a soma total das doações

recebemos para cada mês do conjunto de dados (veja a Figura 4-1):

Pitão

```
>>> df.set_index(pd.to_datetime(df.timestamp), inplace = True)

>>> df.sort_index(inplace = True)

>>> df.groupby(pd.Grouper(freq='M')).amount.sum().plot()
```

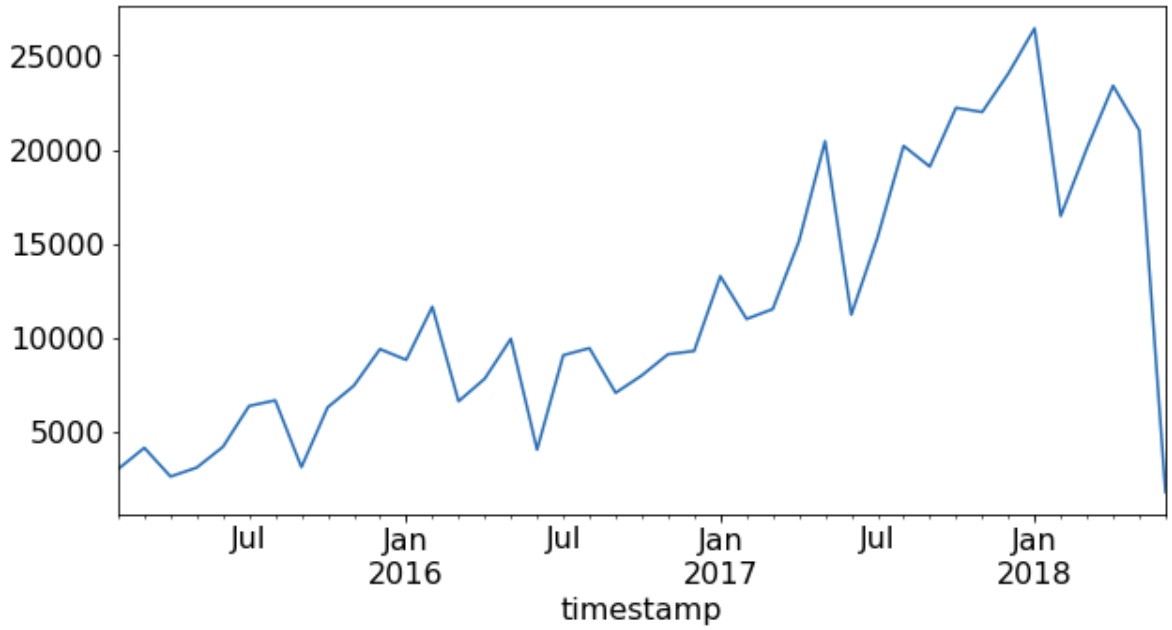



Figura 4-1. Soma total das doações recebidas para cada mês do conjunto de dados.

Parece que o número de donativos e de emails abertos aumentou ao longo do tempo, de 2015 a 2018. Isto não é surpreendente, uma vez que o número de membros também aumentou ao longo do tempo, conforme indicado na soma acumulada de membros e no ano em que aderiram.

Na verdade, uma suposição embutida em nosso modelo era que poderíamos manter um membro indefinidamente após

sua adesão. Não fizemos nenhuma provisão para rescisão, exceto permitir que os membros abram um número decrescente de e-mails. Mesmo nesse caso, porém, deixamos aberta a possibilidade de doações continuadas. Vemos esta suposição de adesão contínua indefinidamente (e comportamento de doação correlato) em

Figura 4-1 . Provavelmente deveríamos voltar e refinar nosso código, já que adesão e doação por tempo indeterminado não são um cenário realista.

Esta não é uma simulação clássica de série temporal, por isso pode parecer um pouco mais com um exercício de geração de dados tabulares. Com certeza é isso também, mas precisávamos estar cientes das séries temporais:

• Tivemos que tomar decisões sobre em quantas séries temporais nossos usuários estavam.

• Tivemos que tomar decisões sobre que tipos de tendências modelaríamos ao longo do tempo:

— No caso do email, decidimos ter três tendências: taxas de abertura de email estáveis, crescentes e decrescentes.

— No caso de doações, fizemos das doações um padrão de comportamento estável

relacionado a quantos e-mails o membro já abriu durante sua vida.

Isso incluía uma previsão, mas como estávamos gerando dados, era uma forma de decidir que a afinidade geral de um membro na organização, o que levaria à abertura de mais e-mails, também aumentaria a frequência das doações.

• Tivemos que ter cuidado para garantir que não tínhamos e-mails abertos ou doações feitas antes do membro ingressar na organização.

• Tivemos que garantir que os nossos dados não fossem para o futuro, para torná-los mais realistas para os

consumidores dos dados. Observe que, para uma simulação, não há problema se nossos dados forem para o futuro.

Mas não é perfeito. O código apresentado aqui é desajeitado e não produz um universo realista. Além do mais, como apenas o programador verificou a lógica, eles poderiam ter perdido casos extremos, de modo que os eventos ocorressem em uma ordem ilógica. Seria bom estabelecer métricas externas e padrões de validade antes de executar a simulação como uma proteção contra tais erros.

Precisamos de software que imponha um universo lógico e consistente. Veremos os geradores Python como uma opção melhor na próxima seção.

Construindo um universo de simulação que funciona sozinho

Às vezes você tem um sistema específico e deseja definir as regras para esse sistema e ver como ele funciona. Talvez você queira imaginar o que um universo de membros independentes que acessam seu aplicativo usará ou queira tentar validar uma teoria interna de tomada de decisão com base no comportamento externo postulado. Nesses casos, você deseja ver como os agentes individuais contribuem para suas métricas agregadas ao longo do tempo. Python é especialmente adequado para este trabalho graças à disponibilidade de geradores. Quando você começa a construir software, em vez de ficar apenas na análise, faz sentido migrar para Python, mesmo que você se sinta mais confortável em R.

Os geradores nos permitem criar uma série de atores independentes (ou dependentes!) e prepará-los para observar o que fazem, sem muito código clichê para acompanhar tudo.

No próximo exemplo de código, exploramos uma simulação de táxi.1 Queremos imaginar como uma frota de táxis, programada para iniciar seus turnos em horários diferentes, poderia se comportar de forma agregada. Para fazer isso, queremos criar muitos táxis individuais, soltá-los em uma cidade cibernética e fazer com que relatem suas atividades.

Tal simulação poderia ser excepcionalmente complicada. Para fins de demonstração, aceitamos que construiremos um mundo mais simples do que imaginamos ser realmente o caso (“Todos os modelos estão errados…”). Começamos tentando entender o que é um gerador Python.

Vamos primeiro considerar um método que escrevi para recuperar um número de identificação de táxi: 1 Este exemplo é fortemente inspirado no livro de Luciano Ramalho, *Python fluente* (O'Reilly 2015). Eu recomendo fortemente

Recomendamos a leitura do capítulo completo sobre simulação desse livro para melhorar suas habilidades de programação em Python e ver oportunidades mais elaboradas para simulação baseada em agente.


```
## Pitão

>>> importar entorpecido como np

>>> def número_id_táxi(num_táxis):

>>> arr = np.arange(num_taxis)

>>> np.random.shuffle(arr)

>>> para i no intervalo (num_taxis):

>>> rendimento arr[i]
```

Para quem não está familiarizado com geradores, aqui está o código anterior em ação:

```
## Pitão

>>> ids = número_id_táxi(10)

>>> imprimir (próximo(ids))
```

>>> **imprimir** (próximo(ids))

>>> **imprimir** (próximo(ids))

que pode imprimir:

7

2

5

Isso irá iterar até emitir 10 números, momento em que sairá do loop for mantido no gerador e emitirá uma exceção StopIteration.

O taxi_id_number() produz objetos de uso único, todos independentes uns dos outros e mantendo seu próprio estado. Esta é uma função geradora. Você pode pensar em geradores como objetos minúsculos que mantêm seu próprio pequeno pacote de variáveis de estado, o que é útil quando você deseja muitos objetos paralelos entre si, cada um cuidando de suas próprias variáveis.

No caso desta simples simulação de táxi, compartimentamos nossos táxis em diferentes turnos, e também utilizamos um gerador para indicar os turnos. Agendamos mais táxis no meio do dia do que nos turnos noturnos ou noturnos, definindo diferentes probabilidades para iniciar um turno em um determinado horário:

Pitão

>>> **def** shift_info():

>>> horários_iniciais_e_freqs = [(0, 8), (8, 30), (16, 15)]

>>> índices = np.arange(len(start_times_and_freqs))

>>> **enquanto** Verdadeiro:

>>> idx = np.random.choice(índices, p = [0,25, 0,5, 0,25])

>>> início = start_times_and_freqs[idx]

>>> **rendimento** (início[0], início[0] + 7,5, início[1])

Preste atenção em start_times_and_freqs. Este é o nosso primeiro código que contribuirá para tornar esta simulação de série temporal. Estamos indicando que diferentes momentos do dia têm diferentes probabilidades de ter um táxi designado para o turno. Além disso, diferentes horários do dia têm um número médio de viagens diferente.

Agora criamos um gerador mais complexo que usará os geradores anteriores para estabelecer parâmetros de táxi individuais, bem como criar cronogramas de táxi individuais:

Pitão

>>> **def** taxi_process(taxi_id_generator, shift_info_generator):

>>> taxi_id = próximo(taxi_id_generator)

>>> shift_start, shift_end, shift_mean_trips =

>>> próximo(shift_info_generator)

>>> viagens_atual = round(np.random.normal(loc = shift_mean_trips,

>>> escala = 2))

>>> tempo_médio_viagem = 6,5 / shift_mean_trips * 60

>>> *# converte o tempo médio de viagem em minutos*

>>> entre_events_time = 1,0 / (shift_mean_trips - 1) * 60

>>> *# esta é uma cidade eficiente onde os táxis raramente ficam sem uso*

```
>>> hora = shift_start

>>> rendimento TimePoint(taxi_id, 'start shift', hora)

>>> deltaT = np.random.poisson(between_events_time) /
60

>>> tempo += deltaT

>>> para i no intervalo (viagens_reais):

>>> rendimento TimePoint(taxi_id, 'pick up', hora)

>>> deltaT = np.random.poisson(average_trip_time) / 60

>>> tempo += deltaT

>>> rendimento TimePoint(taxi_id, 'desembarque', hora)

>>> deltaT = np.random.poisson(between_events_time) /
60

>>> tempo += deltaT

>>> deltaT = np.random.poisson(between_events_time) /
60

>>> tempo += deltaT

>>> rendimento TimePoint(taxi_id, 'end shift', hora)
```

Aqui o táxi acessa geradores para determinar seu número de identificação, horário de início do turno e número médio de viagens para seu horário de início. A partir daí, ele parte em sua própria jornada individual, percorrendo um certo número de viagens em sua própria linha do tempo e as emite para o cliente que chama next() neste gerador. Com efeito, este gerador produz uma série temporal de pontos para um táxi individual.

O gerador de táxi gera TimePoints, que são definidos da seguinte forma:

```
## Pitão

>>> de classes de dados importar classe de dados

>>> @dataclass

>>> aula Ponto de tempo :

>>> taxi_id: int

>>> nome: str

>>> tempo: flutuar

>>> def __lt__ (próprio, outro):

>>> retornar self.time <outro.time
```

Usamos o decorador de classe de dados relativamente novo para simplificar o código (isso requer Python 3.7). Recomendo que todos os cientistas de dados que usam Python se familiarizem com esta adição nova e amigável aos dados do Python.





Métodos Dunder do Python

dunder do Python , cujos nomes começam e terminam com dois

sublinhados, são um conjunto de métodos integrados para cada classe. Dunder

métodos são chamados automaticamente no curso natural usando um

dado objeto. Existem implementações predefinidas que podem ser

substituído quando você mesmo os define para sua classe. Há

muitas razões pelas quais você pode querer fazer isso, como no caso do

código anterior, onde queremos que os TimePoints sejam comparados apenas

com base no horário e não no taxi_id ou nome

atributos.

Dunder originou-se como uma abreviatura de “double under”.

Além do inicializador gerado automaticamente para TimePoint, precisamos apenas de dois outros métodos dunder, __lt__ (para comparar TimePoints) e__str__ (para imprimir TimePoints, não mostrados aqui). Precisamos de comparação porque levaremos todos os Time Points produzidos em uma estrutura de dados que os manterá em ordem: uma fila de prioridade.

Uma *fila de prioridade* é um tipo de dados abstrato no qual os objetos podem ser inseridos em qualquer ordem, mas que emitirá objetos em uma ordem especificada com base em sua prioridade.

Tipo de dados abstrato

Um *tipo de dados abstrato* é um modelo computacional definido por seu

comportamento, que consiste em um conjunto enumerado de ações possíveis

e dados de entrada e quais devem ser os resultados de tais ações

certos conjuntos de dados.

Um tipo de dados abstrato comumente conhecido é o primeiro a entrar, primeiro a sair.

(FIFO) tipo de dados. Isto requer que os objetos sejam emitidos do

estrutura de dados na mesma ordem em que foram inseridos no

estrutura de dados. Como o programador escolhe fazer isso é uma questão

questão de implementação e não de definição.

Temos uma aula de simulação para operar esses geradores de táxi e mantê-los montados.

Esta não é apenas uma classe de dados porque possui bastante funcionalidade, mesmo no inicializador, para organizar as entradas em uma matriz sensata de informações e processamento.

Observe que a única funcionalidade voltada ao público é a função run():

Pitão

>>> **importar fila**

>>> **aula Simulador** :

>>> **def __init__** (self, num_taxis):

>>> self._time_points = queue.PriorityQueue()

>>> taxi_id_generator = taxi_id_number(num_taxis)

>>> shift_info_generator = shift_info()

>>> self._taxis = [taxi_process(taxi_id_generator,

>>> shift_info_generator) **para**

>>> eu **no** intervalo(num_taxis)]

>>> self._prepare_run()

>>> **def** _prepare_run(self):

>>> **para** t **em** self._taxis:

>>> **enquanto** Verdadeiro:

>>> **tente** :

>>> e = próximo(t)

>>> self._time_points.put(e)

>>> **exceto** :

>>> **pausa**

>>> **def** executar(auto):

>>> sim_time = 0

>>> **enquanto** sim_time <24:

>>> **se** self._time_points.empty():

>>> **pausa**

>>> p = self._time_points.get()

>>> sim_time = p.time

>>> **imprimir** (p)

Primeiro, criamos o número de geradores de táxi necessários para representar o número correto de táxis. Em seguida, percorremos cada um desses táxis enquanto ele ainda tem TimePoints e colocamos todos esses TimePoints em uma fila de prioridade. A prioridade do objeto é determinada para uma classe personalizada como TimePoint por nossa implementação de __lt__ de um ponto de tempo, onde comparamos o horário de início. Assim, à medida que os TimePoints são colocados na fila de prioridade, eles serão preparados para serem emitidos em ordem temporal.

Executamos a simulação:

Pitão

>>> sim = Simulador(1000)

>>> sim.run()

Esta é a aparência da saída (sua saída será diferente, pois não definimos uma semente - e toda vez que você executar o código, ele será diferente da última iteração): id: 0539 nome: horário de entrega: 23: 58

id: 0318 nome: horário de saída: 23h58

id: 0759 nome: horário de término do turno: 23:58

id: 0977 nome: horário de saída: 23:58

id: 0693 nome: horário de término do turno: 23h59

id: 0085 nome: horário de término do turno: 23h59

id: 0351 nome: horário de término do turno: 23h59

id: 0036 nome: horário de término do turno: 23h59

id: 0314 nome: horário de entrega: 23:59



Definindo uma semente ao gerar números aleatórios

Ao escrever um código que gera números aleatórios, você

você pode querer garantir que seja reproduzível (por exemplo, se você quiser

configurar testes de unidade para código que normalmente é aleatório ou se você fosse

tentando depurar e queria restringir as fontes de variação para

tornar a depuração mais fácil). Para garantir que números aleatórios apareçam

na mesma ordem não aleatória, você define uma semente. Este é um comum

operação, então existem guias sobre como definir uma semente em qualquer computador

linguagem.

Arredondamos para o minuto mais próximo para simplificar a exibição, embora tenhamos dados mais detalhados disponíveis. A resolução temporal que usaremos dependerá de nossos propósitos:

• Se quisermos fazer uma exibição educativa para as pessoas da nossa cidade sobre como a frota de táxis afeta o tráfego, poderemos exibir agregados por hora.

• Se formos um aplicativo de táxi e precisarmos entender a carga em nosso servidor, provavelmente desejaremos analisar os dados minuto a minuto ou até mesmo dados mais altamente resolvidos para pensar

sobre nosso projeto e capacidade de infraestrutura.

Tomamos a decisão de relatar os TimePoints dos táxis à medida que eles “acontecem”. Ou seja, relatamos o início de uma corrida de táxi (“pick up”) sem o horário em que a corrida terminará, embora pudéssemos facilmente ter condensado isso. Essa é uma maneira de tornar a série temporal mais realista, no sentido de que você provavelmente teria registrado eventos dessa forma em uma transmissão ao vivo.

Observe que, como no caso anterior, nossa simulação de série temporal ainda não produziu uma série temporal. Produzimos um registro e podemos ver como transformá-lo em uma série temporal de várias maneiras:

• Saída para um arquivo CSV ou banco de dados de série temporal à medida que executamos a simulação.

• Execute algum tipo de modelo on-line conectado à nossa simulação para aprender como

desenvolver um pipeline de processamento de dados de streaming em tempo real.

• Salve a saída em um arquivo ou banco de dados e, em seguida, faça mais pós-processamento para empacotar os dados de uma forma conveniente (mas possivelmente arriscada em relação à previsão), como emparelhar os horários de início e término de uma determinada viagem para estudar como se comporta a duração de uma viagem de táxi em diferentes horários do dia.

Existem diversas vantagens em simular esses dados além de poder testar hipóteses sobre a dinâmica de um sistema de táxi. Aqui estão algumas situações em que esses dados sintéticos de série temporal podem ser úteis:



• Testar os méritos de vários modelos de previsão em relação à dinâmica subjacente conhecida da simulação.

• Construir um pipeline para os dados que você espera ter com base em seus dados sintéticos enquanto espera pelos dados reais.

Você será bem servido como analista de séries temporais por sua capacidade de fazer uso de geradores e programação orientada a objetos. Este exemplo oferece apenas um exemplo de como esse conhecimento pode simplificar sua vida e melhorar a qualidade do seu código.

Para simulações extensas, considere a modelagem baseada em agente

A solução que codificamos aqui estava boa, mas era uma quantidade razoável

padrão para garantir que as condições lógicas seriam respeitadas.

Se uma simulação de eventos discretos baseada nas ações de eventos discretos

atores seriam uma fonte útil de dados de séries temporais simuladas, você

deve considerar um módulo orientado para simulação. O módulo SimPy é uma opção útil, com uma API acessível e bastante flexibilidade para realizar os tipos de tarefas de simulação que tratamos nesta seção.

Uma simulação de física

Em outro tipo de cenário de simulação, você pode estar em pleno domínio das leis da física que definem um sistema.

Entretanto, isso não precisa ser física em si; também pode se aplicar a uma série de outras áreas:

• Os investigadores quantitativos em finanças levantam frequentemente hipóteses sobre as regras “físicas” do mercado. O mesmo acontece com os economistas, embora em escalas de tempo diferentes.

• Os psicólogos postulam as regras “psicofísicas” de como os humanos tomam decisões.

Estes podem ser usados para gerar regras “físicas” sobre as respostas humanas esperadas a uma variedade de opções ao longo do tempo.

• Os biólogos pesquisam regras sobre como um sistema se comporta ao longo do tempo em resposta a vários estímulos.

Um caso de conhecer algumas regras para um sistema físico simples é modelar um ímã. É neste caso que vamos trabalhar, através de um mecanismo estatístico muito ensinado.

modelo ics chamado modelo de Ising.2 Veremos uma versão simplificada de como simular seu comportamento ao longo do tempo. Inicializaremos um material magnético de modo que seus componentes magnéticos individuais apontem em direções aleatórias. Iremos então 2 O modelo de Ising é um modelo mecânico estatístico clássico de ímãs bem conhecido e comumente ensinado. Você pode encontrar muitos exemplos de código e discussões adicionais sobre esse modelo on-line, tanto em contextos de programação quanto de física, se estiver interessado em aprender mais.

observe como esse sistema evolui até uma ordem onde todos os componentes magnéticos apontam na mesma direção, sob a ação de leis físicas conhecidas e de algumas linhas de código.

A seguir, discutiremos como tal simulação é realizada através de um método Markov Chain Monte Carlo (MCMC),

discutindo como esse método funciona em geral e como aplicado a este sistema específico.

Simulações de Monte Carlo e Cadeias de Markov

A ideia por trás de uma simulação de Monte Carlo é encontrar maneiras inteligentes de aplicar números aleatórios a situações que deveriam, em teoria, ser solucionáveis com exatidão, mas que na prática são muito mais fáceis de resolver probabilisticamente.

A cadeia de Markov é uma adição útil a uma simulação de Monte Carlo geral e é particularmente aplicável à simulação de séries temporais. Uma simulação de Monte Carlo ajudará você a descobrir como é uma determinada distribuição ou série de termos, mas não como esses termos deveriam evoluir ao longo do tempo. É aqui que entra uma cadeia de Markov. Ela calcula a probabilidade de transição entre estados e, quando levamos isso em consideração, tomamos

“etapas” em vez de simplesmente calcular uma integral global. Agora podemos ter uma simulação de série temporal em vez de apenas o cálculo de uma integral.

Na física, uma simulação MCMC pode ser usada, por exemplo, para entender como as transições quânticas em moléculas individuais podem afetar as medições agregadas desse sistema ao longo do tempo. Neste caso, precisamos aplicar algumas regras específicas: 1. Num processo de Markov, a probabilidade de uma transição para um estado no futuro depende apenas do estado presente (não da informação passada).

2. Iremos impor uma condição específica da física de exigir uma distribuição de energia de Boltzmann; isto é, T /

$ij\ T\ ji = e - b\ (\ Ej - Ei\ )$. Para a maioria de nós, isso é apenas um detalhe de implementação e não algo com que os não-físicos precisem se preocupar.

Implementamos uma simulação MCMC da seguinte forma:

1. Selecione aleatoriamente o estado inicial de cada site de rede individual.

2. Para cada intervalo de tempo individual, escolha um local individual da rede e inverta sua direção.

3. Calcule a variação de energia que resultaria dessa inversão, dadas as leis físicas com as quais você está trabalhando. Neste caso isso significa:

• Se a mudança na energia for negativa, você está fazendo a transição para um estado de energia mais baixo, que sempre será favorecido, então você mantém a chave e passa para o próximo passo de tempo.

• Se a mudança na energia não for negativa, você a aceita com a probabilidade de aceitação de e (-mudança de energia). Isto é consistente com a regra 2.

Continue as etapas 2 e 3 indefinidamente até a convergência para determinar o estado mais provável para qualquer medição agregada que você esteja fazendo.

Vamos dar uma olhada nos detalhes específicos do modelo de Ising. Imagine que temos um material bidimensional composto por uma grade de objetos, cada um tendo o que se resume a um miniímã que pode apontar para cima ou para baixo. Colocamos esses miniímãs aleatoriamente em rotação para cima ou para baixo no tempo zero e, em seguida, registramos o sistema à medida que ele evolui de um estado aleatório para um estado ordenado em baixa temperatura. [3]

Primeiro configuramos nosso sistema da seguinte forma:

```
## Pitão

>>> ### CONFIGURAÇÃO

>>> ## layout físico

>>> N = 5 # largura da rede

>>> M = 5 # altura da treliça
```

>>> *## configurações de temperatura*

>>> temperatura = 0,5

>>> BETA = 1 / temperatura

Depois temos alguns métodos utilitários, como a inicialização aleatória do nosso bloco inicial:

>>> def initRandState(N, M):

>>> bloco = np.random.choice([-1, 1], tamanho = (N, M))

>>> bloco de retorno

Também calculamos a energia para um determinado alinhamento do estado central em relação aos seus vizinhos:

Pitão

>>> **def** costForCenterState(estado, i, j, n, m):

>>> centroS = estado[i, j]

>>> vizinhos = [((i + 1) % n, j), ((i - 1) % n, j),

>>> (i, (j + 1) % m), (i, (j - 1) % m)]

>>> *## observe o % n porque impomos limite periódico cond*

>>> *## ignore isso se não fizer sentido - é apenas um*

>>> *## restrição física no sistema dizendo que o sistema 2D é como*

>>> *## a superfície de um donut*

>>> interaçãoE = [estado[x, y] * centroS **para** (x, y) **em** vizinhos]

>>> **retornar** np.sum(interaçãoE)

E queremos determinar a magnetização de todo o bloco para um determinado estado: 3 O modelo de Ising é mais frequentemente usado para entender qual é o estado de equilíbrio de um ferromagneto, em vez de considerar o aspecto temporal de como um ferromagneto pode chegar ao equilíbrio. estado. No entanto, tratamos a evolução ao longo do tempo como uma série temporal.

Pitão

```
>>> def magnetizaçãoForState(estado):

>>> retornar np.sum(estado)
```

É aqui que apresentamos as etapas do MCMC discutidas anteriormente:

Pitão

```
>>> def mcmcAdjust(estado):

>>> n = estado.forma[0]

>>> m = estado.forma[1]

>>> x, y = np.random.randint(0, n), np.random.randint(0, m)

>>> centroS = estado[x, y]

>>> custo = custoForCenterState(estado, x, y, n, m)

>>> se custo < 0:

>>> centroS *= -1

>>> elif np.random.random() <np.exp(-custo * BETA):

>>> centroS *= -1
```

>>> estado[x, y] = centroS

>>> estado **de retorno**

Agora, para realmente executar uma simulação, precisamos de alguns registros, bem como de chamadas repetidas para o ajuste do MCMC:

Pitão

>>> **def** runState(estado, n_steps, snapsteps = Nenhum):

>>> **se** snapsteps **for** Nenhum:

>>> snapsteps = np.linspace(0, n_steps, num = round(n_steps / (M * N * 100)),

>>> dtype = np.int32)

>>> estados_salvos = []

>>> sp = 0

>>> magnet_hist = []

>>> **para** i **no** intervalo (n_steps):

>>> estado = mcmcAdjust(estado)

>>> magnet_hist.append(magnetizationForState(estado))

>>> **se** sp < len(snapsteps) **e** i == snapsteps[sp]:

>>> salvos_states.append(np.copy(estado))

>>> sp += 1

>>> estado **de retorno** , estados_salvos, magnet_hist

E executamos a simulação:

Pitão

~

>>> *### EXECUTAR UMA SIMULAÇAO*

>>> estado_inicial = estadoRandInit(N, M)

>>> **imprimir** (estado_inicialização)

>>> estado_final = runState(np.copy(init_state), 1000)

Podemos obter alguns insights desta simulação olhando para o início e o fim.

estados de controle (veja a Figura 4-2).

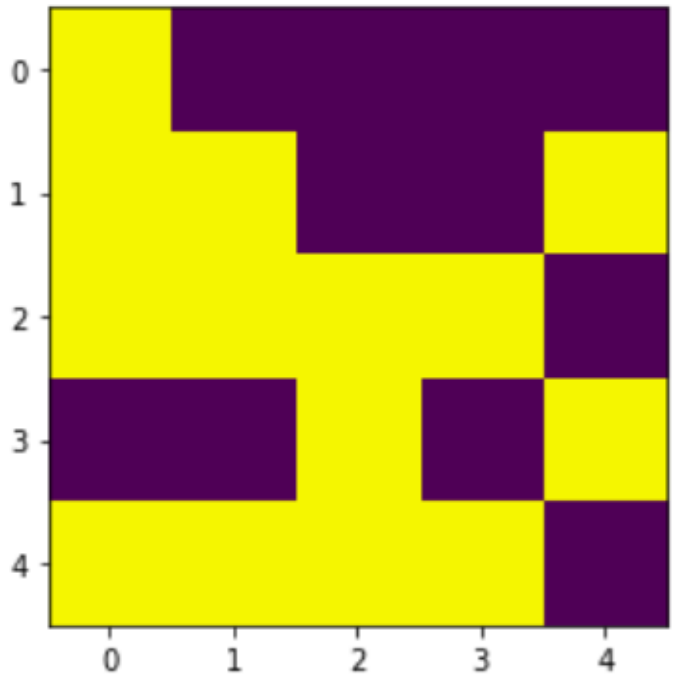


Figura 4-2. Estado inicial de um material ferromagnético simulado 5 × 5, inicializado com cada estado selecionado aleatoriamente para ser girado para cima ou para baixo com igual probabilidade.

Na Figura 4-2 examinamos um estado inicial gerado aleatoriamente. Embora você possa esperar ver os dois estados mais misturados, lembre-se de que,

probabilisticamente, não é muito provável que se obtenha um efeito xadrez perfeito. Tente gerar o estado inicial muitas vezes e você verá que o estado aparentemente "aleatório" ou "50/50" do tabuleiro de xadrez não é nada provável. Observe, entretanto, que começamos com aproximadamente metade de nossas unidades em cada estado. Perceba também que qualquer padrão que você encontrar nos estados iniciais provavelmente é o seu cérebro seguindo a tendência humana de ver padrões, mesmo onde não há nenhum.

Em seguida, passamos o estado inicial para a função runState(), permitindo 1.000 passos de tempo para

passar e, em seguida, examinar o resultado na Figura 4-3.

Este é um instantâneo do estado obtido na etapa 1.000. Há pelo menos duas observações interessantes neste ponto. Primeiro, o estado dominante inverteu-se em relação ao passo 1.000. Em segundo lugar, o estado dominante não é mais dominante numericamente do que o outro estado dominante no passo 1.000. Isto sugere que a temperatura pode continuar a tirar os locais do estado dominante, mesmo quando, de outra forma, poderia ser favorecida. Para entender melhor essa dinâmica, devemos considerar traçar medições agregadas gerais, como magnetização, ou fazer filmes onde possamos visualizar nossos dados bidimensionais em formato de série temporal.

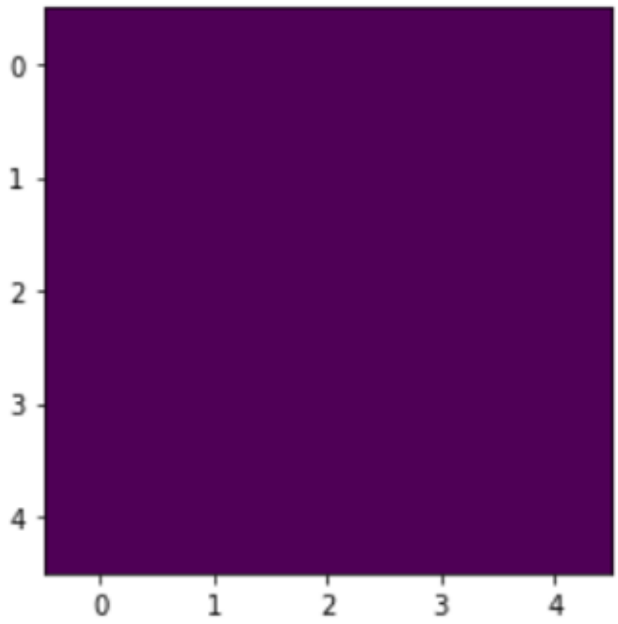


Figura 4-3. Estado final de baixa temperatura em uma execução de nossa simulação, como visto em 1.000

passos de tempo.

Fazemos isso com magnetização ao longo do tempo para muitas execuções independentes do anterior

simulação, conforme ilustrado na Figura 4-4:

Pitão

>>> coletamos cada série temporal **como** um elemento separado **na** lista de resultados

>>> resultados = []

>>> **para** i **no** intervalo (100):

>>> estado_inicial = estadoRandInit(N, M)

>>> estado_final, estados, magnet_hist = runState(init_state, 1000)

>>> resultados.append(magnet_hist)

```
>>>

>>> ## traçamos cada curva com alguma transparência para que possamos ver

>>> ## curvas que se sobrepõem

>>> para mh nos resultados:

>>> plt.plot(mh,'r', alfa=0,2)
```

As curvas de magnetização são apenas um exemplo de como poderíamos imaginar a evolução do sistema ao longo do tempo. Também podemos considerar o registro de séries temporais 2D, como um instantâneo do estado geral em cada momento. Ou pode haver outras variáveis agregadas interessantes para medir em cada etapa, como uma medida de entropia de layout ou uma medida de energia total. Quantidades como magnetização ou entropia são quantidades relacionadas, pois são uma função do layout geométrico do estado em cada local da rede, mas cada quantidade é uma medida ligeiramente diferente.

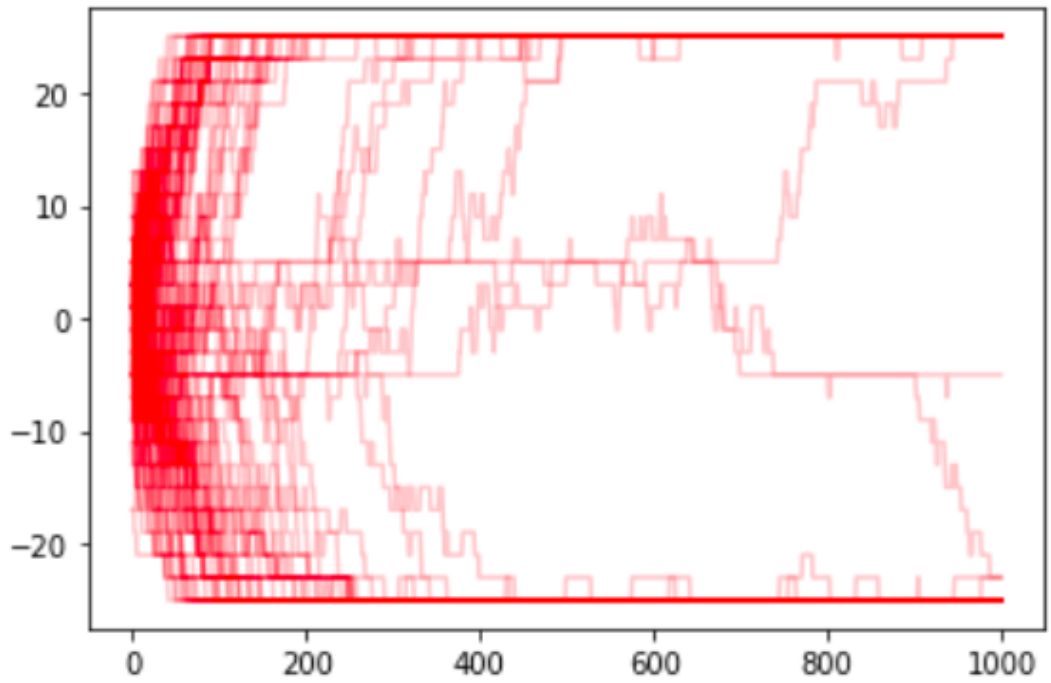


Figura 4-4. 100 simulações independentes de possíveis maneiras pelas quais o sistema poderia entrar em um estado magnetizado a uma temperatura baixa, mesmo quando cada local da rede original foi inicializado aleatoriamente.

Podemos usar esses dados de maneira semelhante à que discutimos com os dados do táxi, embora o sistema subjacente seja bem diferente. Por exemplo, poderíamos:

• Use os dados simulados como impulso para configurar um pipeline.

• Teste métodos de aprendizado de máquina nesses dados sintéticos para ver se eles podem ser úteis em dados físicos antes de nos darmos ao trabalho de limpar dados do mundo real para tal modelagem.

• Assista a imagens semelhantes a filmes de métricas importantes para desenvolver melhores intuições físicas sobre o sistema.

Notas Finais sobre Simulações

Vimos vários exemplos muito diferentes de simulação de medições que descrevem o comportamento ao longo do tempo. Analisamos dados de simulação relacionados ao comportamento do consumidor (filiação a ONGs e doações), infraestrutura da cidade (padrões de coleta de táxis) e as leis da física (a ordenação gradual de um material magnético aleatório). Esses exemplos devem deixar você confortável o suficiente para começar a ler exemplos de código de dados simulados e também ter ideias de como seu próprio trabalho pode se beneficiar das simulações.

É provável que, no passado, você tenha feito suposições sobre seus dados sem saber como testá-las ou possibilidades alternativas. As simulações fornecem uma rota para

faça isso, o que significa que suas conversas sobre dados podem se expandir para incluir exemplos hipotéticos combinados com métricas quantitativas de simulações. Isso

fundamentará suas discussões e, ao mesmo tempo, abrirá novas possibilidades, tanto no domínio das séries temporais quanto em outros ramos da ciência de dados.

Simulações Estatísticas

Simulações estatísticas são o caminho mais tradicional para dados simulados de séries temporais.

Eles são particularmente úteis quando conhecemos a dinâmica subjacente de um sistema estocástico e queremos estimar alguns parâmetros desconhecidos ou ver como diferentes suposições impactariam o processo de estimativa de parâmetros (veremos um exemplo disso mais adiante neste livro). Mesmo para sistemas físicos, às vezes a simulação estatística é melhor.

Simulações estatísticas de dados de séries temporais também são bastante valiosas quando precisamos de uma métrica quantitativa definitiva para definir a nossa própria incerteza sobre a precisão das nossas simulações. Em simulações estatísticas tradicionais, como um modelo ARIMA (a ser discutido no Capítulo 6), as fórmulas para o erro estão bem estabelecidas,

o que significa que para compreender um sistema com um modelo estatístico subjacente postulado, não é necessário executar muitas simulações para fazer afirmações numéricas sobre erro e variância.

Simulações de aprendizagem profunda

Simulações de aprendizagem profunda para séries temporais são um campo incipiente, mas promissor. As vantagens do aprendizado profundo são que dinâmicas não lineares muito complicadas podem ser capturadas em dados de séries temporais, mesmo sem que o profissional compreenda totalmente a dinâmica. Isto também é uma desvantagem, no entanto, na medida em que o profissional não tem uma base de princípios para compreender a dinâmica do sistema.

Simulações de aprendizagem profunda também são promissoras quando a privacidade é uma preocupação. Por

exemplo, a aprendizagem profunda tem sido usada para gerar dados sintéticos de séries temporais heterogêneas para aplicações médicas com base em dados de séries temporais reais, mas sem potencial de vazamento de informações privadas. Tal conjunto de dados, se puder realmente ser produzido sem quaisquer fugas de privacidade, seria inestimável porque os investigadores poderiam ter acesso a uma grande variedade de dados médicos (de outra forma caros e que violam a privacidade).

Notas Finais sobre Simulações | 141

Mais recursos

Cristóbal Esteban, Stephanie L. Hyland e Gunnar Rätsch, "Valor Real (Médico)

Geração de série temporal com GANs condicionais recorrentes," manuscrito não publicado, última revisão em 4 de dezembro de 2017, https://perma.cc/Q69W-L44Z.

Os autores demonstram como redes adversárias generativas podem ser usadas para

produzir dados de séries temporais médicas heterogêneas e de aparência realista. Isto é um

exemplo de como a simulação de aprendizagem profunda pode ser usada para criar conjuntos de dados médicos éticos, legais e (esperançosamente) que preservam a privacidade para permitir um acesso mais amplo a dados úteis para aprendizagem automática e aprendizagem profunda no contexto da saúde.

Gordon Reikard e W. Erick Rogers, "Previsão das Ondas Oceânicas: Comparando um

Modelo Baseado em Física com Modelos Estatísticos," Engenharia Costeira 58 (2011): 409–16, https://perma.cc/89DJ-ZENZ.

Este artigo oferece uma comparação acessível e prática de duas formas drasticamente diferentes de modelar um sistema, com física ou com estatística. Os investigadores

concluem que, para o problema específico que abordam, a escala temporal de interesse para o analista deve orientar as decisões sobre qual o paradigma a aplicar. Embora este artigo seja sobre previsão, a simulação está fortemente relacionada e os mesmos insights se aplicam.

Wolfgang Härdle, Joel Horowitz e Jens-Peter Kreiss, "Métodos Bootstrap para o Tempo

Series," Revisão Estatística Internacional / Revue Internationale de Statistique 71, no. 2

(2003): 435–59, https://perma.cc/6CQA-EG2E.

Uma revisão clássica de 2005 das dificuldades de simulação estatística de dados de séries temporais dadas as dependências temporais. Os autores explicam, em um periódico de estatística altamente técnico, por que os métodos para inicializar dados de séries temporais ficam atrás dos métodos para outros tipos de dados, bem como quais métodos promissores estavam disponíveis no momento da redação deste artigo. O estado da arte não mudou muito, então esta é uma leitura útil, embora desafiadora.

CAPÍTULO 5

Armazenando Dados Temporais

Muitas vezes, o valor dos dados de séries temporais surge em cenários retrospectivos, e não em cenários de transmissão ao vivo. Por esse motivo, o armazenamento de dados de séries temporais é necessário para a maioria das análises de séries temporais.

Uma boa solução de armazenamento é aquela que permite facilidade de acesso e confiabilidade dos dados sem exigir um grande investimento de recursos computacionais. Neste capítulo, discutiremos quais aspectos de um conjunto de dados você deve considerar ao projetar o armazenamento

de dados de séries temporais. Também discutiremos as vantagens dos bancos de dados SQL, NoSQL

bancos de dados e uma variedade de formatos de arquivo simples.

Projetar uma solução geral de armazenamento de série temporal é um desafio porque há muitos tipos diferentes de dados de série temporal, cada um com diferentes padrões de armazenamento, leitura/gravação e análise. Alguns dados serão armazenados e examinados repetidamente, enquanto outros dados serão úteis apenas por um curto período de tempo, após o qual poderão ser totalmente excluídos.

Aqui estão alguns casos de uso para armazenamento de série temporal que possuem diferentes padrões de leitura, gravação e consulta:

1. Você está coletando métricas de desempenho em um sistema de produção. Você precisa armazenar essas métricas de desempenho durante anos, mas quanto mais antigos os dados ficam, menos detalhados eles precisam ser. Portanto, você precisa de uma forma de armazenamento que reduza automaticamente a amostragem e selecione os dados à medida que as informações envelhecem.

2. Você tem acesso a um repositório remoto de dados de série temporal de código aberto, mas precisa manter uma cópia local em seu computador para reduzir o tráfego de rede. O repositório remoto armazena cada série temporal em uma pasta de arquivos disponíveis para download em um servidor web, mas você gostaria de reunir todos esses arquivos em um único banco de dados para simplificar. Os dados devem ser imutáveis e capazes de serem armazenados

indefinidamente, pois pretende ser uma cópia confiável do repositório remoto.

3. Você criou seus próprios dados de série temporal integrando uma variedade de fontes de dados em diferentes

escalas de tempo e com diferentes pré-processamento e formatação. A coleta e o processamento de dados foram trabalhosos e demorados. Você gostaria de armazenar os dados em sua forma final, em vez de executar uma etapa de pré-processamento repetidamente, mas também gostaria de manter os dados brutos caso você investigue posteriormente alternativas de pré-processamento. Você espera revisitar os dados processados e brutos com frequência à medida que desenvolve novos modelos de aprendizado de máquina, reajustando novos modelos nos mesmos dados

e também aumentando seus dados ao longo do tempo, à medida que dados brutos novos e mais recentes se tornam

disponível. Você nunca reduzirá a amostragem ou selecionará dados armazenados, mesmo que faça isso para suas análises.

Esses casos de uso são bastante variados em relação às principais demandas de um sistema: *Importância de como o desempenho se adapta ao tamanho*

No primeiro caso de uso, procuraríamos uma solução que pudesse incorporar scripts automatizados para excluir dados antigos. Não estaríamos preocupados com a forma como o sistema foi dimensionado para grandes conjuntos de dados porque planejaríamos manter o conjunto de dados pequeno. Por outro lado, no segundo e terceiro casos de uso, esperaríamos ter uma grande e estável coleção de dados (caso de uso 2) ou uma grande e crescente coleção de dados (caso de uso 3).

Importância do acesso aleatório versus acesso sequencial de pontos de dados

No caso de uso 2, esperaríamos que todos os dados fossem acessados em quantidades iguais, já que esses dados de série temporal teriam todos a mesma “idade” na inserção e fariam referência a conjuntos de dados interessantes. Por outro lado, nos casos de uso 1 e 3 (e especialmente no 1, dada a descrição anterior), esperaríamos que os dados mais recentes fossem acessados com mais frequência.

Importância dos scripts de automação

O caso de uso 1 parece poder ser automatizado, enquanto o caso de uso 2 não exigiria automação (uma vez que os dados seriam imutáveis). O caso de uso 3 sugere pouca automação, mas muita busca e processamento de dados de todas as partes dos dados, não apenas das mais recentes. No caso de uso 1, desejaríamos uma solução de armazenamento que pudesse ser integrada com scripts ou procedimentos armazenados, enquanto no caso de uso 3, desejaríamos uma solução que permitisse uma grande personalização fácil do processamento de dados.

Apenas três exemplos já oferecem uma boa ideia dos muitos casos de uso que uma solução de série temporal multifuncional precisa atender.

Em casos de uso do mundo real, você poderá personalizar sua solução de armazenamento e não se preocupar em encontrar uma ferramenta adequada a todos os casos de uso. Dito isto, você sempre escolherá entre uma gama semelhante de tecnologias disponíveis, que tendem a se resumir a:

• Bancos de dados SQL

• Bancos de dados NoSQL

• Formatos de arquivo simples

Neste capítulo, cobrimos todas as três opções e discutimos as vantagens e desvantagens de cada uma. É claro que os detalhes dependerão do caso de uso em questão, mas este capítulo fornecerá a base necessária quando você começar sua busca por uma opção de armazenamento de série temporal que funcione para seu caso de uso.

Primeiro discutimos quais perguntas você deve fazer no início ao escolher uma solução de armazenamento. Em seguida, examinaremos o grande debate SQL versus NoSQL e examinaremos algumas das soluções de armazenamento de série temporal mais populares. Por fim, consideramos a

criação de políticas para permitir que dados antigos de séries temporais expirem e sejam excluídos.

Definindo Requisitos

Ao pensar em armazenamento de dados de séries temporais, você precisa se perguntar uma série de perguntas:

• *Quantos dados de série temporal você armazenará? Com que rapidez esses dados crescerão?*

Você deseja escolher uma solução de armazenamento adequada à taxa de crescimento dos dados esperada. Os administradores de banco de dados que passam para séries temporais trabalham a partir de

conjuntos de dados orientados a transações muitas vezes ficam surpresos com a rapidez com que os conjuntos de dados de séries temporais podem crescer.

• *Suas medições tendem a canais infinitos de atualizações (por exemplo, uma constante fluxo de atualizações de tráfego da web) ou eventos distintos (por exemplo, uma série temporal de tráfego aéreo de hora em hora para cada feriado importante nos EUA nos últimos 10 anos)?* Se seus dados forem como um canal infinito, você analisará principalmente dados recentes. Por outro lado, se seus dados forem uma coleção de séries temporais distintas divididas em eventos separados, então eventos mais distantes no tempo ainda poderão ser bastante interessantes. Neste último caso, o acesso aleatório é um padrão mais provável.

• *Os seus dados serão espaçados regular ou irregularmente?* Se seus dados forem regularmente espaçados, você poderá calcular antecipadamente com mais precisão a quantidade de dados que espera coletar e com que frequência esses dados serão inseridos. Se seus dados estiverem espaçados irregularmente, você deverá se preparar para um estilo menos previsível de acesso aos dados que possa permitir com eficiência tanto períodos mortos quanto períodos de atividade de gravação.

• *Você coletará dados continuamente ou há um final bem definido para o seu projeto?* Se você tiver um fim bem definido para a coleta de dados, será mais fácil saber o tamanho do conjunto de dados que você precisa acomodar. No entanto, muitas organizações

descobrem que, uma vez que começam a coletar um determinado tipo de série temporal, eles não querem mais parar!



• *O que você fará com sua série temporal? Você precisa de visualizações em tempo real? Dados pré-processados para uma rede neural iterar milhares de vezes?*

Dados fragmentados altamente disponíveis para uma grande base de usuários móveis? Seu caso de uso principal indicará se é mais provável que você precise de acesso sequencial ou aleatório aos seus dados e qual deve ser a importância do fator latência para a seleção do formato de armazenamento.

• *Como você selecionará ou reduzirá a resolução dos dados? Como você evitará o crescimento infinito?*

Qual deve ser o ciclo de vida de um ponto de dados individual em uma série temporal? É impossível armazenar todos os eventos de todos os tempos. É melhor tomar decisões sobre políticas de eliminação de dados de forma sistemática e antecipada, em vez de numa base ad hoc.

Quanto mais você se comprometer antecipadamente, melhor será a escolha que poderá fazer em relação aos formatos de armazenamento. Mais sobre isso na próxima seção.

As respostas a estas perguntas indicarão se você deve armazenar dados brutos ou processados, se os dados devem ser localizados na memória de acordo com o tempo ou de acordo com algum outro eixo, e se você precisa armazenar seus dados de uma forma que facilite o armazenamento. ler e escrever. Os casos de uso variam, portanto, você deve fazer um novo inventário com cada novo conjunto de dados.

Dados *fragmentados* são dados que fazem parte de um grande sistema de dados, mas estão espalhados

em pedaços menores e mais gerenciáveis, geralmente em vários servidores

em uma rede.

Dados ativos versus dados armazenados

Ao pensar sobre quais opções de armazenamento são desejáveis para seus dados, é fundamental compreender o ciclo de vida dos seus dados. Quanto mais realista você puder ser sobre seus casos reais de uso de dados, menos dados você precisará salvar e menos tempo precisará se preocupar em encontrar o sistema de armazenamento ideal, porque você não dimensionará rapidamente uma quantidade intratável de dados. Muitas vezes, as organizações registam demasiado eventos de interesse e têm medo de perder os seus armazenamentos de dados, mas ter mais dados armazenados num formato intratável é muito menos útil do que ter dados agregados armazenados em escalas de tempo significativas.

Para dados de curta duração, como dados de desempenho que serão analisados apenas para garantir que nada está errado, é possível que você nunca precise armazenar os dados na forma em que foram coletados, pelo menos não por muito tempo. Isto é mais importante para dados orientados por eventos, onde nenhum evento único é importante e, em vez disso, as estatísticas agregadas são os valores de interesse.

Suponha que você esteja executando um servidor da web que registra e informa quanto tempo cada dispositivo móvel

levou para carregar totalmente uma determinada página da web. O resultado

séries temporais irregularmente espaçadas podem ser semelhantes à Tabela 5-1.

Tabela 5-1. Série temporal do servidor Web

Carimbo de data e hora	**Hora de carregar a página**
5 de abril de 2018 22h22:24	23s
5 de abril de 2018 22h22:28	15s
5 de abril de 2018 22h22:41	14s
5 de abril de 2018 22h23:02	11s

Por vários motivos, você não estaria interessado em nenhuma medição individual do tempo de carregamento de uma página. Você gostaria de agregar os dados (por exemplo, tempo médio de carregamento por minuto) e até mesmo as estatísticas agregadas seriam interessantes apenas por um curto período. Suponha que você estivesse de plantão durante a noite para esse servidor. Você gostaria de ter certeza de que poderia mostrar que o desempenho foi bom enquanto estava no comando. Você poderia simplificar isso para um ponto de dados para suas 12 horas de plantão e teria a maior parte

as informações que você poderia desejar, conforme mostrado na Tabela 5-2.

Tabela 5-2. Um ponto de dados simplificado para horas de plantão

Período de tempo	**Hora de acesso mais popular**	**Num cargas**	**Tempo médio de carregamento**	**Tempo máximo de carregamento**
5 de abril de 2018, das 20h às 8h	23h	3.470	21 anos	45 anos

Nesse caso, você não deve planejar armazenar indefinidamente os eventos individuais. Em vez disso, você deve criar uma solução de armazenamento que forneça a preparação de eventos individuais apenas como armazenamento temporário até que os dados cheguem à sua forma final. Você evitará muito sofrimento para você e seus colegas, evitando o crescimento descontrolado de dados antes mesmo de começar. Em vez de ter 3.470 eventos individuais, nenhum dos quais interessa a ninguém, você terá números de interesse compactos e facilmente acessíveis. Você deve simplificar o armazenamento de dados por meio de agregação e desduplicação sempre que possível.

A seguir, consideramos algumas oportunidades para reduzir dados sem perder informações.

Variáveis mudando lentamente

Se você estiver armazenando uma variável de estado, considere registrar apenas os pontos de dados onde o valor foi alterado. Por exemplo, se você estiver registrando a

temperatura em incrementos de cinco minutos, sua curva de temperatura poderá parecer uma função escalonada, principalmente se você se importar apenas com um valor como o grau mais próximo. Nesse caso, não é necessário armazenar valores repetitivos e você economizará espaço de armazenamento se não fizer isso.

Dados barulhentos e de alta frequência

Se seus dados apresentarem ruído, há motivos para não se importar muito com nenhum ponto de dados específico. Você pode considerar agregar pontos de dados antes de registrá-los, pois o alto nível de ruído torna qualquer medição individual menos valiosa. Esta vontade, de **Definir Requisitos | 147**

É claro que você deve ser bastante específico do domínio e precisará garantir que os usuários a jusante ainda sejam capazes de avaliar o ruído nas medições para seus propósitos.

Dados desatualizados

Quanto mais antigos forem os dados, menor será a probabilidade de sua organização utilizá-los, exceto de uma forma muito geral. Sempre que você estiver começando a registrar um novo conjunto de dados de série temporal, você deve pensar preventivamente sobre quando os dados da série temporal provavelmente se tornarão irrelevantes:

• Existe uma data de validade natural?

• Caso contrário, você pode examinar pesquisas anteriores do seu departamento de análise e ver até onde elas realmente remontam? Quando algum script em um repositório Git realmente acessou os dados mais antigos do seu conjunto de dados?

Se você puder automatizar a exclusão de dados de uma forma que não impeça os esforços de análise de dados, você melhorará suas opções de armazenamento de dados, reduzindo a importância da escalabilidade ou o arrastamento de consultas lentas em conjuntos de dados inchados.

Considerações legais

Se você trabalha em uma organização grande ou que lida com dados exigidos por terceiros (como para auditorias regulatórias), considerações legais devem ser levadas em consideração nos requisitos do seu sistema. Você deve avaliar com que frequência provavelmente receberá solicitações de dados de terceiros, a quantidade de dados que eles exigiriam e se você seria capaz de escolher o formato ou se os regulamentos impõem um formulário específico.

Pode parecer excessivamente cauteloso citar requisitos legais ao avaliar as especificações de armazenamento de dados, mas os desenvolvimentos recentes na Europa e nos Estados Unidos sugerem um interesse crescente nos conjuntos de dados utilizados para alimentar a aprendizagem automática (por exemplo, o Regulamento Geral de Proteção de Dados da União Europeia).

Até agora, discutimos a gama geral de casos de uso para armazenamento de série temporal. Também revisamos um conjunto geral de consultas relacionadas a como um conjunto de dados de série temporal será produzido e analisado para que essas consultas possam informar nossa seleção de um formato de armazenamento. Agora revisamos duas opções comuns para armazenar séries temporais: bancos de dados e arquivos.

Soluções de banco de dados

Para quase qualquer analista ou engenheiro de dados, um banco de dados é uma solução intuitiva e familiar para a questão de como armazenar dados. Tal como acontece com os dados relacionais, um banco de dados geralmente será uma boa opção de armazenamento para dados de séries temporais. Isto é particularmente verdadeiro se você deseja uma solução pronta para uso com qualquer uma destas características clássicas de banco de dados:

• Um sistema de armazenamento que pode ser expandido para vários servidores

• Um sistema de leitura/gravação de baixa latência

• Funções já implementadas para calcular métricas comumente usadas (como calcular a média em uma consulta agrupar por, onde o agrupamento pode ser aplicado ao tempo

Métricas)

• Ferramentas de solução de problemas e monitoramento que podem ser usadas para ajustar o desempenho do sistema e analisar gargalos

Estas são boas razões entre muitas para optar por um banco de dados em vez de um sistema de arquivos,

e você deve sempre considerar uma solução de banco de dados para armazenamento de dados, principalmente ao trabalhar com um novo conjunto de dados. Um banco de dados, especialmente um banco de dados NoSQL, pode ajudar a preservar a flexibilidade. Além disso, um banco de dados colocará seu projeto em funcionamento mais cedo do que se você fosse trabalhar com arquivos individuais, porque grande parte do padrão necessário já está em vigor. Mesmo que você decida por uma solução de armazenamento de arquivos (e a maioria das pessoas não o fará), trabalhar primeiro com um banco de dados pode ajudá-lo a determinar como estruturar seus próprios arquivos quando seus novos processos de dados amadurecerem.

No restante desta seção, abordaremos as respectivas vantagens dos bancos de dados SQL e NoSQL para séries temporais e, em seguida, discutiremos as opções de banco de dados atualmente populares para aplicações de séries temporais.

A boa notícia é que os gráficos de séries temporais parecem ser a categoria de crescimento mais rápido

banco de dados de série temporal no momento, então você pode esperar ver mais e ainda melhores opções para soluções de banco de dados de série temporal no futuro.

SQL versus NoSQL

SQL versus NoSQL é um debate tão animado na comunidade de bancos de dados de série temporal quanto mais amplamente. Muitos administradores de banco de dados especialistas insistem que SQL é o único caminho a seguir e que não há dados em qualquer formato que não possam ser bem descritos por um bom conjunto de tabelas relacionais. No entanto, na prática, muitas vezes há uma queda no desempenho quando as organizações tentam dimensionar soluções SQL para acomodar grandes quantidades de dados de séries temporais e, por esse motivo, sempre vale a pena considerar também uma solução NoSQL, especialmente se você estiver procurando uma solução aberta. solução que pode ser dimensionada para acomodar casos em que a coleta de dados de séries temporais começa sem um horizonte de tempo finito à vista.

Embora as soluções SQL e NoSQL possam ser boas para dados de séries temporais, primeiro motivamos nossa discussão sobre as dificuldades de aplicação da lógica de banco de dados a dados de séries temporais explorando como os dados de séries temporais diferem do tipo de dados para os quais os bancos de dados SQL foram desenvolvidos.

Características dos dados que inspiraram originalmente os bancos de dados SQL Podemos compreender melhor a incompatibilidade entre o pensamento semelhante ao SQL e os dados de série temporal revisando o histórico das soluções SQL. As soluções SQL foram baseadas em dados transacionais, que são quaisquer dados necessários para descrever completamente um evento discreto.

Uma transação é composta de atributos que refletem muitas chaves primárias, como produto, participantes, horário e valor de uma transação. Observe que o tempo pode estar presente como chave primária, mas apenas como uma entre muitas, e não como um eixo privilegiado de informação.

Existem dois recursos importantes de dados transacionais que são bastante diferentes das necessidades de séries temporais:

• Os pontos de dados existentes são frequentemente atualizados.

• Os dados são acessados de forma um tanto aleatória, pois não há ordem subjacente

obrigatório.

Características dos dados de série temporal

Os dados de séries temporais elaboram toda a história de algo, enquanto um registro de transação nos informa apenas o estado final. Conseqüentemente, os dados de séries temporais geralmente não requerem atualização, o que significa que o acesso aleatório para operações de gravação tem baixa prioridade.

Isso significa que os objetivos de desempenho que foram fundamentais em décadas de SQL

o design do banco de dados não é muito importante para bancos de dados de séries temporais. Na verdade, quando consideramos os objetivos ao projetar um banco de dados para séries temporais, temos prioridades bastante diferentes devido à forma como usaremos nossos dados de séries temporais. Os principais recursos do nosso caso de uso de dados de série temporal são:

• As operações de gravação predominam sobre as operações de leitura.

• Os dados não são escritos, lidos ou atualizados em ordem aleatória, mas sim na ordem relacionada ao sequenciamento temporal.

• Leituras simultâneas são muito mais prováveis do que para dados de transação.

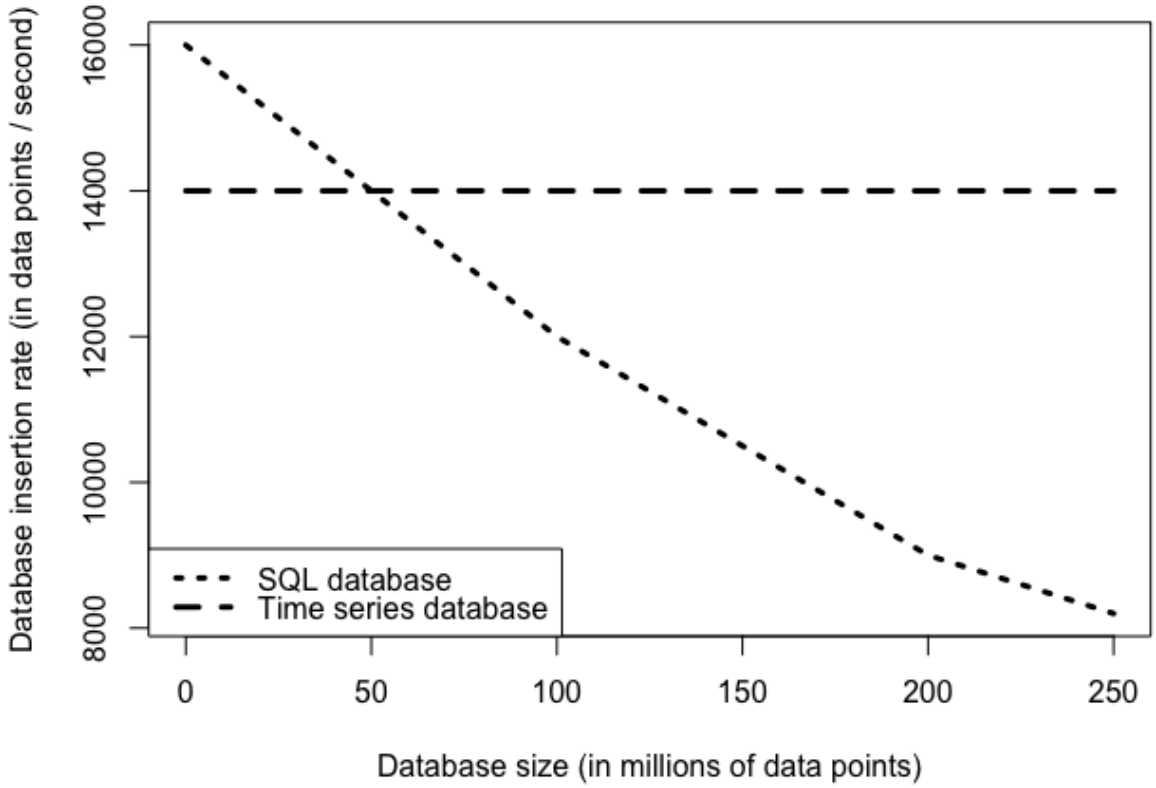
• Existem poucas chaves primárias, se houver alguma, além do próprio tempo.

• As exclusões em massa são muito mais comuns do que a exclusão de pontos de dados individuais.

Esses recursos suportam o uso de um banco de dados NoSQL, porque muitos

Os bancos de dados NoSQL de aplicação oferecem muito do que desejamos para bancos de dados de série temporal, particularmente a ênfase em operações de gravação em vez de operações de leitura. Conceitualmente, os bancos de dados NoSQL mapeiam bem os dados de série temporal, pois refletem nativamente aspectos da coleta de dados de série temporal, como nem todos os campos sendo coletados para todos os pontos de dados.

Os esquemas flexíveis do NoSQL são naturais com dados de série temporal. Muitos dos dados que motivaram a popularidade atual dos bancos de dados NoSQL são dados de séries temporais.



Por esse motivo, bancos de dados NoSQL prontos para uso tendem a ter melhor desempenho que SQL

bancos de dados para operações de gravação. A Figura 5-1 representa o desempenho de um banco de dados SQL comumente usado versus um banco de dados NoSQL comumente usado para o caso de inserções de pontos de dados de séries temporais (operações de gravação).

Figura 5-1. Uma característica comum de um banco de dados projetado para uso em séries temporais é que o a taxa de inserção de dados é constante em relação ao tamanho do banco de dados. Este é um impressionante vantagem em comparação com uma solução de armazenamento SQL tradicional.

Como escolher entre SQL e NoSQL

Pode parecer que estou tentando empurrá-lo para o caso de uso NoSQL, mas também existem muitos bons casos de uso para bancos de dados SQL. Ao pensar nos seus próprios dados, tenha em mente o princípio que sempre se aplica ao armazenamento de dados, seja em SQL

banco de dados, um banco de dados NoSQL ou um arquivo de texto simples: *dados que tendem a ser solicitados em ao mesmo tempo deve ser armazenado no mesmo local* . Esse é o fator mais importante, independentemente do seu caso de uso.

Muitas apresentações em conferências e postagens em blogs anunciam soluções NoSQL que atendem precisamente à situação de um cenário de alta gravação e baixa atualização, onde os dados não são ordenados aleatoriamente. No entanto, os bancos de dados SQL são uma opção de armazenamento de série temporal viável e usada regularmente. Em particular, com algumas mudanças arquitetônicas com ênfase nos bancos de dados SQL tradicionais e em sua estrutura de memória interna, esses desafios podem ser superados mantendo algumas das vantagens do SQL. Por exemplo, algo como sim

simples e aparentemente óbvio, pois estruturar a representação da memória interna das tabelas SQL para levar em conta o tempo pode fazer uma diferença substancial no desempenho.

Em última análise, as distinções entre NoSQL e SQL são altamente dependentes da implementação e não são tão sistemáticas ou importantes como parecem ser. Deixe que seus dados orientem sua seleção de uma implementação específica de uma dessas tecnologias. Ao considerar os atributos dos seus dados de série temporal e padrões de uso de acesso, você pode ter em mente algumas limitações gerais:

Vantagens do SQL para séries temporais

• Se sua série temporal estiver armazenada em um banco de dados SQL, você poderá relacioná-la facilmente com dados transversais relevantes também armazenados nesse banco de dados.

• Os dados hierárquicos de séries temporais se ajustam naturalmente às tabelas relacionais. Um conjunto apropriado de esquema SQL ajudará você a agrupar séries temporais relacionadas e delinear claramente a hierarquia, embora elas possam ser facilmente dispersas de forma menos sistemática em uma solução NoSQL.

• Se você estiver criando uma série temporal baseada em dados transacionais, onde os dados são melhor armazenados em um banco de dados SQL, será vantajoso armazenar também sua série temporal no mesmo banco de dados para facilitar a verificação, referência cruzada e assim por diante .

Vantagens do NoSQL para séries temporais

• As gravações são rápidas.

• Eles são bons se você não tiver conhecimento suficiente sobre dados futuros para projetar um esquema inteligente e robusto.

• Para o usuário inexperiente, esses bancos de dados costumam ser a solução pronta para uso com melhor desempenho porque é menos provável que você projete um

esquema estranho ou fique preso a um design de esquema medíocre.

Soluções populares de banco de dados e arquivos de séries temporais

Agora discutiremos algumas soluções de banco de dados populares para dados de séries temporais. Eles dão uma ideia do que está disponível além das soluções SQL tradicionais. Observe que as tecnologias discutidas aqui ocupam um cenário tecnológico lotado e fragmentado.

O que é comumente usado este ano pode não ser tão popular no próximo ano. Assim, esta discussão não deve ser vista como uma recomendação tecnológica específica, mas sim como um conjunto de amostras que ajudam a ilustrar o estado atual do mercado.

Bancos de dados específicos de séries temporais e ferramentas de monitoramento relacionadas

Primeiro, discutiremos ferramentas criadas especificamente para armazenar e monitorar dados de séries temporais. Em particular, daremos uma olhada em um banco de dados de série temporal (InfluxDB) e em outro produto que é uma ferramenta de monitoramento de desempenho, que pode

funcionar como uma solução de armazenamento de série temporal (Prometheus). As vantagens de cada ferramenta reflectem necessariamente a sua ênfase e padrões de utilização distintos.

InfluxoDB. InfluxDB é um banco de dados específico de série temporal, conforme sua própria descrição na página da web do projeto GitHub:

InfluxDB é um banco de dados de série temporal de código aberto...útil para registrar métricas, eventos e realizar análises.

No InfluxDB, os dados são organizados por séries temporais. Um ponto de dados no InfluxDB consiste em:

• Um carimbo de data/hora

• Uma etiqueta indicando em que consiste a medição

• Um ou mais campos de chave/valor (como temperatura=25,3)

• Pares chave/valor contendo tags de metadados

O InfluxDB, como um banco de dados com reconhecimento de tempo, marca automaticamente a data e hora de qualquer ponto de dados que chega sem carimbo de data e hora. Além disso, o InfluxDB usa consultas semelhantes a SQL, como: SELECT * FROM access_counts WHERE value > 10000

Outras vantagens do InfluxDB incluem:

• Opções de retenção de dados que permitem automatizar facilmente a designação e exclusão de dados obsoletos

• Alta velocidade de ingestão de dados e compactação agressiva de dados

• Capacidade de marcar séries temporais individuais para permitir a indexação rápida de séries temporais que correspondam a um critério específico

• Membro da pilha TICK madura , que é uma plataforma para capturar, armazenar, monitorar e exibir dados de séries temporais

Existem muitos outros bancos de dados específicos de séries temporais; no momento, o InfluxDB é o mais popular, então é o que você provavelmente encontrará. As opções que oferece são aquelas comumente oferecidas por bancos de dados especializados em séries temporais, pois essas opções refletem os atributos mais comumente desejados para armazenamento de dados de séries temporais.

Como banco de dados, o InfluxDB é um sistema *baseado em push* , o que significa que, ao usá-lo, você envia dados para o banco de dados para ingestão. Isso é diferente da próxima opção que discutiremos, Prometheus.

Dadas essas especificações, uma solução como o InfluxDB fornece todas as funcionalidades gerais prontas para uso, de maneira consciente do tempo. Isso significa que você pode usar suas habilidades SQL existentes e, ao mesmo tempo, se beneficiar das vantagens relacionadas à necessidade de capturar, mas controlar, o crescimento de dados de séries temporais. Finalmente, você tem a gravação rápida necessária com frequência ao capturar dados de séries temporais à medida que são produzidos.

Prometeu. O Prometheus se descreve como um “sistema de monitoramento e séries temporais

banco de dados” que funciona via HTTP. Esta descrição indica sua ênfase geral: monitoramento primeiro e armazenamento depois. A grande vantagem do Prometheus é que ele é um sistema *baseado em pull* , o que significa que a lógica de como os dados são extraídos para a criação de uma série temporal e com que frequência eles são centralizados pode ser facilmente ajustada e inspecionada.

O Prometheus é um ótimo recurso durante emergências porque é atômico e autossuficiente. No entanto, também não é garantido que esteja totalmente atualizado ou preciso, devido à sua arquitetura baseada em pull. Embora deva ser a tecnologia ideal para monitoramento de desempenho rápido e sujo, não é apropriada para aplicações onde os dados precisam ser 100% precisos.

O Prometheus usa uma linguagem de expressão funcional chamada PromQL para sua consulta: access_counts > 10000

O Prometheus também oferece, via PromQL, uma API para muitas tarefas comuns de séries temporais, incluindo até mesmo funções sofisticadas, como fazer uma previsão (predict_lin ear()) e calcular a taxa de aumento por unidade de tempo para uma série temporal (taxa ()).

A agregação por períodos de tempo também é oferecida com uma interface simples. O Prometheus tende a enfatizar

o monitoramento e a análise em vez da manutenção, portanto, comparado ao

No InfluxDB, há menos funcionalidades automatizadas de curadoria de dados.

O Prometheus é uma solução útil de armazenamento de série temporal, especialmente para aplicativos de transmissão ao vivo e quando a disponibilidade de dados é fundamental. Ele tem uma curva de aprendizado mais acentuada devido à sua linguagem de script personalizada e à arquitetura e API menos parecidas com bancos de dados, mas mesmo assim é amplamente utilizado e apreciado por muitos desenvolvedores.

Bancos de dados NoSQL gerais

Embora os bancos de dados específicos de séries temporais ofereçam muitas vantagens, você também pode considerar o caso de bancos de dados NoSQL de uso mais geral. Esses tipos de bancos de dados são baseados em uma estrutura *de documento ,* e não em uma estrutura *de tabela ,* e geralmente não possuem muitas funções explícitas especificamente para séries temporais.

No entanto, os esquemas flexíveis dos bancos de dados NoSQL continuam úteis para dados de séries temporais, especialmente para novos projetos onde o ritmo da coleta de dados e o número de canais de entrada podem mudar ao longo da vida útil de um conjunto de dados. Por exemplo, uma série temporal pode inicialmente começar como apenas um canal de dados, mas crescer gradualmente para incluir mais tipos de dados, todos com registro de data e hora juntos. Mais tarde, pode ser decidido que alguns dos canais de entrada não são particularmente úteis e podem ser descontinuados.

Nesse caso, armazenar os dados em uma tabela SQL seria difícil por vários motivos e resultaria em muitos NaNs onde os dados não estavam disponíveis. Dados NoSQL-154 | Capítulo 5: Armazenando Dados Temporais

base simplesmente deixaria os canais ausentes quando não estivessem disponíveis, em vez de marcar um

armazenamento de dados retangular com vários NaNs.

Um banco de dados de série temporal NoSQL popular e de alto desempenho é o MongoDB. Mongo é

particularmente consciente de seu valor como banco de dados de séries temporais e tem um forte impulso para desenvolver arquitetura e instruções amigáveis à IoT. Ele oferece recursos de agregação de alto nível que podem ser aplicados a operações de agregação por tempo e agrupamentos relacionados ao tempo, e também oferece muitos processos automatizados para dividir o tempo em marcações relevantes para humanos, como dia da semana ou dia do mês:

• $diaDaSemana

• $diaDoMês

• $hora

Além disso, o Mongo dedicou extensos esforços de documentação para demonstrar como as séries temporais podem ser tratadas. 1 Um conjunto pronto de documentação e foco institucional em

os dados de série temporal significam explicitamente que os usuários podem esperar que esse banco de dados continue desenvolvendo recursos ainda mais amigáveis.

Mais útil do que toda essa funcionalidade, entretanto, é a flexibilidade do Mongo na evolução do esquema ao longo do tempo. Essa flexibilidade de esquema evitará muitos problemas se você trabalhar em um conjunto de dados de série temporal em rápida evolução com práticas de coleta de dados em constante mudança.

Por exemplo, imagine uma sequência de práticas de coleta de dados de série temporal, como o que pode ocorrer em uma startup de saúde:

1. Sua startup é lançada como um aplicativo de pressão arterial e coleta apenas duas métricas, pressão arterial

sistólica e diastólica, que você incentiva os usuários a medir várias vezes ao dia…

2. Mas seus usuários desejam conselhos aprimorados sobre estilo de vida e ficarão felizes em fornecer mais dados. Os usuários fornecem tudo, desde datas de nascimento até registros mensais de peso, contagens de passos por hora e contagens de calorias. Obviamente, esses diferentes tipos de dados são coletados em ritmos drasticamente diferentes…

3. Mas mais tarde você percebe que alguns desses dados não são muito úteis, então você para de coletá-los...

4. Mas seus usuários sentem falta da exibição dos dados, mesmo que você não os use, então você reinicia a coleta dos dados mais populares…

1 Consulte, por exemplo, “Design de esquema para dados de séries temporais no MongoDB” , “Dados de séries temporais e MongoDB: Parte

1 – Uma introdução” e “Dados de série temporal e MongoDB: Parte 2 – Melhores práticas de design de esquema”. Para

visão contrária, consulte “Como armazenar dados de série temporal no MongoDB e por que isso é uma má ideia” .

5. E então o governo de um dos seus maiores mercados aprova uma lei relativa ao armazenamento de dados de saúde e você precisa limpar esses dados ou criptografá-los, então você precisa de um novo campo criptografado…

6. E as mudanças continuam.

Quando você precisar da flexibilidade de consulta e esquema descrita em um aplicativo como este, você estará bem atendido por um banco de dados NoSQL geral que oferece um equilíbrio razoável entre flexibilidade específica de tempo e flexibilidade mais geral.

Outra vantagem de um banco de dados NoSQL de uso geral em vez de uma série temporal:

banco de dados específico é que você pode integrar mais facilmente dados que não sejam de séries temporais no mesmo banco de dados para facilitar a referência cruzada dos conjuntos de dados relacionados. Às vezes, um banco de dados NoSQL de uso geral é a combinação certa de considerações de desempenho e funcionalidade semelhante ao SQL, sem a inteligência necessária para otimizar um esquema de banco de dados SQL para funcionalidade de série temporal.

Nesta seção, examinamos soluções de banco de dados NoSQL2 e pesquisamos alguns

opções agora em uso popular. Embora possamos esperar que as tecnologias específicas que dominam o mercado evoluam ao longo do tempo, os princípios gerais sobre os quais estas tecnologias operam e as vantagens que oferecem permanecerão os mesmos. Recapitulando rapidamente, algumas das vantagens que encontramos em diferentes variedades de bancos de dados são:

• Altas capacidades de leitura ou gravação (ou ambas)

• Estruturas de dados flexíveis

• Ingestão de dados push ou pull

• Processos automatizados de seleção de dados

Um banco de dados específico de série temporal oferecerá a automação mais pronta para uso para suas tarefas específicas de série temporal, mas oferecerá menos flexibilidade de esquema e menos oportunidades para integrar dados transversais relacionados do que um banco de dados mais geral e completo. banco de dados NoSQL de propósito.

De modo geral, os bancos de dados oferecerão mais flexibilidade do que os formatos de armazenamento de arquivos simples, mas essa flexibilidade significa que os bancos de dados são menos simplificados e têm menos

desempenho de E/S do que arquivos simples, que discutiremos a seguir.

2 A otimização de bancos de dados SQL para dados de séries temporais está além do escopo deste livro e tende a ser muito específica para cada caso de uso. Alguns recursos no final do capítulo abordam essa questão.

Soluções de arquivos

No final das contas, um banco de dados é um software que integra scripts e armazenamento de dados. É essencialmente um arquivo simples envolvido em um software especial responsável por tornar esse arquivo o mais seguro e fácil de usar possível.

Às vezes faz sentido retirar essa camada externa e assumir nós mesmos a responsabilidade total pelo armazenamento dos dados. Embora isto não seja muito comum em aplicações empresariais, é frequentemente feito em investigação científica e nas raras aplicações industriais (como a negociação de alta frequência) em que a velocidade é fundamental. Nesses casos, é o analista quem projetará um pipeline de dados muito mais complexo que envolve a alocação de espaço de armazenamento, abertura de arquivos, leitura de arquivos, fechamento de arquivos, proteção de arquivos e assim por diante, em vez de simplesmente escrever algumas consultas ao banco de dados.

Uma solução de arquivo simples é uma boa opção se alguma destas condições se aplicar:

• Seu formato de dados está maduro para que você possa se comprometer com uma especificação por um período de tempo razoavelmente longo.

• Seu processamento de dados está vinculado à E/S, por isso faz sentido gastar tempo de desenvolvimento acelerando-o.

• Você não precisa de acesso aleatório, mas pode ler os dados sequencialmente.

Nesta seção, examinamos brevemente algumas soluções comuns de arquivos simples. Você também deveria

lembre-se de que você sempre pode criar seu próprio formato de armazenamento de arquivos, embora isso seja

bastante complexo e geralmente só vale a pena se você estiver trabalhando em uma linguagem de alto desempenho, como C++ ou Java.

Se o seu sistema for maduro e sensível ao desempenho o suficiente para se adequar a um sistema de arquivos simples, há diversas vantagens em implementar um, mesmo que isso signifique a tarefa de migrar seus dados de um banco de dados. Essas vantagens incluem:

• Um formato de arquivo simples é independente do sistema. Se precisar compartilhar dados, basta disponibilizar os arquivos em um formato compartilhável conhecido. Não há necessidade de pedir a um colaborador que acesse remotamente seu banco de dados ou configure seu próprio banco de dados espelhado.

• A sobrecarga de E/S de um formato de arquivo simples é necessariamente menor do que a de um banco de dados, porque isso equivale a uma operação simples de leitura para um arquivo simples, em vez de uma recuperação e leitura com um banco de dados.

• Um formato de arquivo simples codifica a ordem em que os dados devem ser lidos, embora nem todos os bancos de dados o façam. Isto impõe a leitura serial dos dados, o que pode ser desejável, como para regimes de treinamento de aprendizagem profunda.

• Seus dados ocuparão uma quantidade muito menor de memória do que ocupariam em um

banco de dados porque você pode maximizar as oportunidades de compactação. Relacionado, você

pode ajustar o grau de compactação de dados para equilibrar explicitamente seu desejo de **File Solutions | 157**

minimize o volume de armazenamento de dados e o tempo de E/S em seu aplicativo. Mais compactação significará menor volume de dados, mas maior espera de E/S.

NumPy

Se seus dados forem puramente numéricos, uma opção amplamente usada para armazenar esses dados é um array Python NumPy. Os arrays NumPy podem ser facilmente salvos em vários formatos, e há muitas publicações de benchmarking comparando seu desempenho relativo.

Por exemplo, o repositório GitHub array_storage_benchmark foi projetado para testar a eficiência

eficiência e velocidade de vários formatos de arquivo NumPy.

Uma desvantagem dos arrays NumPy é que eles têm um único tipo de dados, o que significa que você não pode armazenar dados heterogêneos de séries temporais automaticamente, mas deve pensar se ter apenas um tipo de dados pode funcionar para seus dados na forma bruta ou processada (embora existam formas de contornar esta restrição). Outra desvantagem dos arrays NumPy é que não é natural adicionar rótulos a linhas ou colunas, portanto, não há uma maneira direta de registrar a data e hora de cada linha de um array, por exemplo.

A vantagem de usar um array NumPy é que há muitas opções para salvá-lo, incluindo um formato binário compactado que ocupa menos espaço e tem E/S mais rápida do que você verá com uma solução de banco de dados. Também é o mais inovador possível em termos de uma estrutura de dados de alto desempenho para análise e armazenamento.

Pandas

Se você deseja fácil rotulagem de dados ou fácil armazenamento de dados heterogêneos de séries temporais (ou ambos), considere o quadro de dados Pandas menos simplificado, mas mais flexível. Pode ser particularmente útil quando você tem uma série temporal que consiste em muitos tipos distintos de dados, talvez incluindo contagens de eventos (ints), medidas de estado (flutuantes) e rótulos (strings ou codificações one-hot). Nesses casos, você provavelmente desejará se ater aos quadros de dados do Pandas (lembre-se também de que o nome “pandas”, na verdade, vem de uma elisão de

“dados em painel”, portanto este é um formato natural para muitos casos de uso).

Os data frames do Pandas são amplamente usados e existem vários recursos online para com-

compare os vários formatos usados para armazenar esses dados.

Equivalentes R padrão

Os formatos nativos para armazenar objetos R são objetos .Rds e .Rdata. Esses são formatos de arquivo binários, portanto serão necessariamente mais eficientes tanto para compactação quanto para E/S do que formatos baseados em texto e, dessa forma, são semelhantes aos quadros de dados Pandas em Python. Da mesma forma, o formato Feather pode ser usado em R e Python para salvar quadros de dados em um formato de arquivo independente de linguagem. Para usuários de R, os formatos binários nativos serão (é claro) os de melhor desempenho. A Tabela 5-3 compara as opções de formato de arquivo.

Tabela 5-3. Comparação de tamanho e desempenho de opções de formato de arquivo **Nome do formato Tamanho relativo Tempo relativo para carregar**

.RDS

1x

1x

pena

2x

1x

csv

3x

20x

Como podemos ver, o formato nativo é um vencedor claro para minimizar tanto o espaço de armazenamento quanto a E/S, e aqueles que usam um formato de arquivo baseado em texto em vez de binário pagarão um preço alto tanto em lentidão de armazenamento quanto de E/S.

Matriz X

Quando seus dados de série temporal atingem muitas dimensões, talvez seja hora de pensar

sobre uma solução de dados mais industrial, chamada Xarray , que é útil por vários motivos:

• Dimensões nomeadas

• Operações matemáticas vetorizadas, como NumPy

• Operações agrupadas, como Pandas

• Funcionalidade semelhante a banco de dados que permite a indexação com base em um intervalo de tempo

• Uma variedade de opções de armazenamento de arquivos

Xarray é uma estrutura de dados que oferece suporte a muitas operações específicas de séries temporais, como indexação e reamostragem no tempo, interpolação de dados e acesso a componentes individuais de uma data e hora. O

Xarray foi construído como um instrumento de computação científica de alto desempenho e é lamentavelmente subutilizado e subestimado para aplicações de análise de séries temporais.

Xarray é uma estrutura de dados em Python e vem com várias opções de armazenamento. Ele implementa decapagem e outro formato de arquivo binário chamado netCDF, um formato universal de dados científicos compatível com muitas plataformas e idiomas. Se você deseja aprimorar seu jogo de série temporal em Python, o Xarray é um bom lugar para começar.

Como você viu, há muitas opções para armazenamento de arquivos simples de dados de séries temporais, algumas com funcionalidade associada (Xarray) e outras com formatos puramente numéricos muito simplificados (NumPy). Ao migrar um conjunto de dados de um banco de dados orientado

pipeline para um pipeline orientado a arquivos, haverá algumas dificuldades crescentes relacionadas à simplificação de dados e à reescrita de scripts para mover a lógica para fora do banco de dados e para os scripts ETL (extrair-transformar-carregar) explícitos. Nos casos em que o desempenho é fundamental, mover os dados para arquivos é provavelmente a etapa mais importante para reduzir a latência. Os casos comuns em que isso é desejável incluem:

• Previsão sensível à latência, como para software voltado ao usuário

• Situações de acesso a dados repetitivos com uso intensivo de E/S, como treinamento de modelos de aprendizado profundo

Por outro lado, para muitas aplicações, a conveniência, a escalabilidade e a flexibilidade de um banco de dados compensam a latência mais alta. Sua situação ideal de armazenamento dependerá muito da natureza dos dados que você está armazenando e do que você deseja fazer com eles.

Mais recursos

• Em tecnologias de banco de dados de série temporal:

Jason Moiron, "Reflexões sobre bancos de dados de séries temporais", blog jmoiron, 30 de junho de 2015, https://perma.cc/8GDC-6CTX.

Esta postagem clássica do blog de 2015 oferece uma visão de uma época anterior

período com bancos de dados de séries temporais e o culto de registrar tudo. Esta visão geral de alto nível de opções para armazenar bancos de dados de série temporal e casos de uso típicos é altamente informativa para iniciantes em administração e engenharia de banco de dados.

Preetam Jinka, "Lista de bancos de dados de séries temporais," Blog Misframe, 9 de abril de 2016, https://perma.cc/9SCQ-9G57.

Esta longa lista de bancos de dados de séries temporais é frequentemente atualizada para mostrar os bancos de dados atualmente no mercado. Cada entrada do banco de dados vem com destaques como

à forma como um determinado banco de dados se relaciona com seus concorrentes e antecessores.

Peter Zaitsev, "Pesquisa do Blog Percona: Qual mecanismo de banco de dados você está usando para armazenar

Dados de série temporal?" Blog Percona, 10 de fevereiro de 2017, https://perma.cc/5PXF-BF7L.

Esta pesquisa de 2017 com engenheiros de banco de dados indicou que bancos de dados relacionais (SQL

bancos de dados) continuaram a dominar como um grupo, com 35% dos entrevistados indi-

Eles usam esses bancos de dados para armazenar dados de séries temporais. ElasticSearch,

InfluxDB, MongoDB e Prometheus também eram favoritos.

Rachel Stephens, "O Estado do Mercado de Banco de Dados de Séries Temporais", Monk Vermelho, abril 3, 2018, https://perma.cc/WLA7-ABRU.

Este recente artigo baseado em dados, escrito por um analista de tecnologia, descreve as áreas mais populosas.

soluções lar para armazenamento de banco de dados de séries temporais por meio de uma investigação empírica da atividade do GitHub e do Stack Overflow. O relatório também indica um alto grau

de fragmentação no domínio de armazenamento de séries temporais devido a uma ampla variedade de

casos de uso para dados de séries temporais, bem como uma tendência maior de segmentação em bancos de dados.

Prometheus.io, "Documentação do Prometheus: comparação com alternativas", nd,

https://perma.cc/M83E-NBHQ.

Esta lista extremamente detalhada e abrangente compara Prometheus a um

uma série de outras soluções populares de armazenamento de séries temporais. Você pode usar isso

modelo como uma referência rápida para as estruturas de dados e armazenamento dominantes

estruturas usadas pelas alternativas ao Prometheus. Este é um bom lugar para obter uma visão geral do que é oferecido e quais são as vantagens e desvantagens entre vários

bancos de dados projetados para dados temporais.

• Sobre a adaptação de tecnologias gerais de banco de dados:

Gregory Trubetskoy, "Armazenando séries temporais no PostreSQL com eficiência", notas pessoais

blog, 23 de setembro de 2015, https://perma.cc/QP2D-YBTS.

Esta postagem de blog mais antiga, mas ainda relevante, explica como armazenar séries temporais no Postgres de maneira focada no desempenho. Trubetskoy explica as dificuldades

da abordagem "ingênua" de uma coluna de valores e uma coluna de carimbos de data/hora,

e dá conselhos práticos para abordagens baseadas em matrizes Postgres.

Josiah Carlson, "Usando Redis como um banco de dados de série temporal", InfoQ, 2 de janeiro de 2016,

https://perma.cc/RDZ2-YM22.

Este artigo fornece conselhos detalhados e exemplos relacionados ao uso do Redis como um banco de dados de série temporal. Embora o Redis tenha sido usado para dados de séries temporais desde sua criação, o autor aponta várias dicas, bem como estruturas de dados vantajosas que podem ser aplicadas em muitos cenários de casos de uso de séries temporais.

narios. Este artigo é útil para aprender mais sobre como usar o Redis com dados de séries temporais, mas também como um exemplo de como ferramentas mais gerais podem ser

reaproveitados para usos específicos de séries temporais.

Mike Freedman, "Dados de série temporal: por que (e como) usar um banco de dados relacional

Em vez de NoSQL", blog Timescale, 20 de abril de 2017, https://perma.cc/A6CU-6XTZ.

Esta postagem do blog do fundador do TimescaleDB descreve as maneiras pelas quais seu

A equipe construiu um banco de dados de série temporal como um banco de dados relacional com modificações de layout de memória específicas para dados temporais. A alegação do TimescaleDB é que o principal problema com os bancos de dados SQL tradicionais para dados de séries temporais é que esses bancos de dados não são escalonáveis, resultando em desempenho lento à medida que os dados são transferidos para dentro e para fora da memória para realizar consultas relacionadas ao tempo. TimescaleDB propõe

traçando memória e mapeamentos de memória para refletir a natureza temporal de

dados e reduza a troca de dados díspares dentro e fora da memória.

Hilmar Buchta, "Combinando múltiplas tabelas com intervalos de datas válidos de/até em

uma dimensão única", Blog Oraylis, 17 de novembro de 2014, https://perma.cc/B8CT-BCEK.

Embora o título não seja muito atraente, esta é uma postagem de blog extremamente útil para aqueles de nós que não são especialistas em banco de dados SQL. Ele ilustra uma boa maneira de

lidar com o complicado problema dos dados marcados como válidos apenas para um

determinado intervalo de tempo e como combinar esses dados e juntá-los em várias tabelas. Isto é surpreendentemente comum e surpreendentemente difícil

tarefa. O exemplo coberto envolve alguns gráficos de recursos humanos, incluindo

as datas válidas de emprego de um funcionário, as datas válidas para um determinado departamento, as datas válidas para um local de escritório específico e as datas

válidas para um carro da empresa. Uma tarefa muito provável seria determinar o status de um funcionário em um determinado

dia - essa pessoa estava empregada e, em caso afirmativo, em que departamento e em que

local e o funcionário possuía carro da empresa? Isso parece intuitivo,

mas o SQL para responder a essa pergunta é difícil e bem abordado neste post.

CAPÍTULO 6

Modelos Estatísticos para Séries Temporais

Neste capítulo, estudamos alguns modelos estatísticos lineares para séries temporais. Esses modelos estão relacionados à regressão linear, mas levam em conta as correlações que surgem entre pontos de dados na mesma série temporal, em contraste com os métodos padrão aplicados a dados transversais, nos quais se assume que cada ponto de dados é independente dos outros na amostra.

Os modelos específicos que discutiremos são:

• Modelos autoregressivos (AR), modelos de média móvel (MA) e modelos autoregressivos de média móvel integrada (ARIMA)

• Autoregressão vetorial (VAR)

• Modelos hierárquicos.

Esses modelos têm sido tradicionalmente o carro-chefe da previsão de séries temporais e continuam a ser aplicados em uma ampla gama de situações, desde pesquisas acadêmicas até modelagem industrial.

Por que não usar uma regressão linear?

Como analista de dados, você provavelmente já está familiarizado com regressões lineares. Caso contrário, eles podem ser definidos da seguinte forma: uma regressão linear pressupõe que você tenha dados *distribuídos de forma independente e idêntica* (iid). Como discutimos detalhadamente nos capítulos anteriores, este não é o caso dos dados de séries temporais. Em dados de séries temporais, os pontos próximos no tempo tendem a estar fortemente correlacionados entre si. Na verdade, quando não há correlações temporais, os dados de séries temporais dificilmente são úteis para tarefas tradicionais de séries temporais, como prever o futuro ou compreender a dinâmica temporal.

163

Às vezes, tutoriais e livros didáticos de séries temporais dão a impressão indevida de que a regressão linear não é útil para séries temporais. Eles fazem os alunos pensarem que regressões lineares simples simplesmente não serão suficientes. Felizmente, este não é o caso. A regressão linear de mínimos quadrados comum pode ser aplicada a dados de séries temporais, desde que as seguintes condições sejam mantidas:

Suposições a respeito do comportamento da série temporal

• A série temporal tem uma resposta linear aos seus preditores.

• Nenhuma variável de entrada é constante ao longo do tempo ou perfeitamente correlacionada com outra variável de entrada. Isto simplesmente estende o requisito tradicional de regressão linear de variáveis independentes para levar em conta a dimensão temporal dos dados.

Suposições a respeito do erro

• Para cada ponto no tempo, o valor esperado do erro, dadas todas as variáveis explicativas para todos os períodos de tempo (para frente e para trás), é 0.

• O erro em qualquer período de tempo não está correlacionado com as entradas em qualquer período de

tempo no passado ou no futuro. Portanto, um gráfico da função de autocorrelação dos erros não indicará nenhum padrão.

• A variação do erro é independente do tempo.

Se essas suposições forem válidas, então a regressão de mínimos quadrados ordinários é uma estimativa imparcial.

principal dos coeficientes dados as entradas, mesmo para dados de séries temporais.1 Neste caso, o

as variâncias amostrais das estimativas têm a mesma forma matemática da regressão linear padrão. Portanto, se seus dados atenderem às suposições listadas, você poderá aplicar uma regressão linear, que sem dúvida ajudará a oferecer intuições claras e simples para o comportamento de sua série temporal.

Os requisitos de dados que acabamos de descrever são semelhantes aos da regressão linear padrão aplicada a dados transversais. O que adicionamos é uma ênfase nas qualidades temporais do conjunto de dados.

1 Observe que os mínimos quadrados ordinários são imparciais mesmo no caso de remoção de algumas dessas condições. Por exemplo, quando os erros são correlacionados e heterocedásticos, os mínimos quadrados ordinários ainda podem fornecer uma estimativa imparcial dos coeficientes, embora haja preocupações de eficiência. Para obter mais informações sobre eficiência, comece com

Wikipédia .





Não force a regressão linear. Algumas das consequências da aplicação

fazer regressão linear quando seus dados não atendem aos requisitos

suposições são:

• Seus coeficientes não minimizarão o erro do seu modelo.

• Seus valores *p* para determinar se seus coeficientes são

diferente de zero será incorreto porque se baseia em suposições

que não são atendidos. Isso significa que suas avaliações do coeficiente

significância pode estar errada.

As regressões lineares podem ser úteis para oferecer simplicidade e transparência.

parência quando apropriado, mas um modelo incorreto certamente não é

transparente!

É justo questionar se os analistas de séries temporais são excessivamente rígidos na aplicação das suposições exigidas pela regressão linear padrão de forma tão rigorosa que não possam usar a técnica de regressão linear. Os analistas do mundo real tomam liberdade com as suposições do modelo de tempos em tempos. Isto pode ser produtivo, desde que as potenciais desvantagens de fazê-lo sejam compreendidas.

A importância de aderir às suposições de um modelo depende fortemente do

domínio. Por vezes, um modelo é aplicado com pleno conhecimento de que os seus pressupostos de base não são cumpridos porque as consequências não são demasiado graves em relação ao retorno.

Por exemplo, na negociação de alta frequência, os modelos lineares são bastante populares por uma série de razões, apesar de ninguém acreditar que os dados seguem rigorosamente todos os padrões.

suposições.2

O que é um estimador imparcial?

Se uma estimativa não for superestimada ou subestimada, ela está usando

um estimador imparcial. Isso tende a ser uma coisa boa, embora você

deve estar ciente do *trade-off viés-variância* , que é uma descrição

ção do problema tanto para problemas estatísticos quanto de aprendizado de máquina

problemas em que modelos com menor viés em suas estimativas de parâmetros

tendem a ter maior variância da estimativa do parâmetro.

A variância da estimativa do parâmetro reflete o quão variável um

estimativa será em diferentes amostras de dados.

2 Existem alguns fatores atenuantes que justificam o uso da regressão linear padrão nesse caso. Primeiro, alguns acreditam que em escalas suficientemente curtas os movimentos dos mercados financeiros são independentes

uns dos outros (iid). Em segundo lugar, como as regressões lineares são tão eficientes do ponto de vista computacional, um modelo rápido, mesmo que impreciso nos seus pressupostos, é um bom modelo numa indústria impulsionada pela velocidade. Terceiro, as empresas que utilizam estes modelos conseguem ganhar dinheiro, por isso devem estar a fazer algo certo.

Por que não usar uma regressão linear? | 165

Se você se encontrar em uma situação em que uma regressão linear possa ser uma boa opção para sua tarefa de previsão, considere aproveitar tslm(), uma função na previsão

pacote projetado para fornecer metodologias fáceis de regressão linear para séries temporais

dados.

Métodos estatísticos desenvolvidos para séries temporais

Consideramos métodos estatísticos desenvolvidos especificamente para dados de séries temporais. Primeiro estudamos métodos desenvolvidos para dados de séries temporais univariadas, começando com o caso muito simples de um modelo autoregressivo, que é um modelo que diz que os valores futuros de uma série temporal são uma função de seus valores passados. Em seguida, avançamos para modelos cada vez mais complexos, concluindo com uma discussão sobre auto-regressão vetorial para séries temporais multivariadas e alguns métodos adicionais especializados de séries temporais, como modelos GARCH e modelagem hierárquica.

Modelos Autoregressivos

O modelo autorregressivo (AR) baseia-se na intuição de que o passado prevê o futuro e, portanto, postula um processo de série temporal no qual o valor em um ponto no tempo t é uma função dos valores da série em pontos anteriores no tempo.

Nossa discussão deste modelo será mais detalhada para lhe dar uma ideia de como os estatísticos pensam sobre esses modelos e suas propriedades. Por esta razão, começamos com uma visão geral bastante extensa da teoria. Você pode dar uma olhada nisso se não estiver interessado na mecânica por trás de como as propriedades do modelo estatístico são derivadas para séries temporais.

Usando álgebra para entender as restrições nos processos de AR

A autorregressão se parece com o que muitas pessoas usariam como uma primeira tentativa de ajustar uma série temporal, especialmente se não tivessem nenhuma informação além da própria série temporal. É exatamente o que o seu nome indica: uma regressão a valores passados para prever valores futuros.

O modelo AR mais simples, um modelo AR(1), descreve um sistema da seguinte forma:

$yt = b\,0 + b\,1 \times yt\,\text{-}1 + et$

O valor da série no tempo t é função de uma constante $b0$, seu valor no intervalo de tempo anterior multiplicado por outra constante $b\,1 \times yt\,\text{-}1$ e um termo de erro que também varia com o tempo et . Presume-se que este termo de erro tenha uma variância constante e uma média de 0.

Denotamos um termo autorregressivo que olha apenas para o tempo imediatamente anterior como um modelo AR(1) porque inclui uma retrospectiva de um atraso.

Aliás, o modelo AR(1) tem forma idêntica a um modelo de regressão linear simples com apenas uma variável explicativa. Ou seja, ele mapeia para:

$Y = b\,0 + b\,1 \times x + e$

Podemos calcular tanto o valor esperado de yt quanto sua variância, dado yt –1, se soubermos o valor de $b\,0$ e $b\,1$.

Consulte a Equação 6-1.3

Equação 6-1. E(yt | yt-1) = b0 + b1 × yt-1 + et

$Var(yt \mid yt\text{-}1) = Var(et) = Var(e)$ **O que é | Símbolo?**

Nas estatísticas, você pode condicionar probabilidades, valores esperados e assim por diante, de uma variável a outra. Tal condicionamento é indicado com o | símbolo, ou barra vertical, que deve ser lido como “dado”. Assim, por exemplo, P(A|B) deve ser lido como “a probabilidade de A dado B”.

Por que o usamos no caso anterior? Na equação (0) queremos determinar o valor esperado do processo no tempo t, yt dado qual é o valor no passo de tempo anterior. Esta é uma forma de expressar que conhecemos informações até o tempo t – 1. O valor esperado de yt, se não tivermos nenhuma informação, é o valor esperado que podemos calcular de forma mais geral a partir do processo. No entanto, se soubermos o valor um passo antes, poderemos ser mais específicos sobre o valor esperado no momento t.

A generalização desta notação permite que o valor presente de um processo AR dependa dos p valores mais recentes, produzindo um processo AR(p).

Agora mudamos para uma notação mais tradicional, que usa ϕ para denotar os coeficientes de autorregressão:

$yt = \phi 0 + \phi 1 \times yt\text{-}1 + \phi 2 \times yt\text{-}2 + \ldots + \phi p \times yt\text{-}p + et$

Conforme discutido no Capítulo 3, a estacionariedade é um conceito-chave na análise de séries temporais porque é exigida por muitos modelos de séries temporais, incluindo modelos AR.

3 Apenas as equações referenciadas posteriormente são numeradas.

Podemos determinar as condições para que um modelo AR seja estacionário a partir da definição de estacionariedade. Continuamos nosso foco no modelo AR mais simples, AR(1) em

Equação 6-2 .

Equação 6-2. $yt = \phi 0 + \phi 1 \times yt\text{-}1 + et$

Assumimos que o processo é estacionário e depois trabalhamos “para trás” para ver o que isso implica sobre os coeficientes. Primeiro, a partir da suposição de estacionariedade, sabemos que o valor esperado do processo deve ser sempre o mesmo. Podemos reescrever *yt* pela equação para um processo AR(1):

$$E (yt) = \mu = E (yt\text{-}1)$$

Por definição, *et* tem um valor esperado de 0. Além disso, os phis são constantes, portanto seus valores esperados são valores constantes. A Equação 6-2 reduz à esquerda

lado para:

$$E (yt) = E (\phi 0 + \phi 1 \times yt\text{-}1 + et) \; E (yt) = \mu$$

E no lado direito para:

$$\phi 0 + \phi 1 \times \mu + 0$$

Isso simplifica para:

$$\mu = \phi 0 + \phi 1 \times \mu$$

o que por sua vez implica isso (Equação 6-3).

Equação 6-3. $\mu = \frac{\phi 0}{1 - \phi 1}$

Portanto, encontramos uma relação entre a média do processo e os coeficientes AR(1) subjacentes.

Podemos tomar medidas semelhantes para observar como uma variância e uma covariância constantes impõem condições aos coeficientes ϕ . Começamos substituindo o valor de ϕ 0, que temos

pode derivar a Equação 6-4 da Equação 6-3 .

Equação 6-4. $\phi 0 = \mu \times (1 - \phi 1)$

na Equação 6-2 :

$yt = \phi 0 + \phi 1 \times yt\text{-}1 + et$

$yt = (\mu - \mu \times \phi 1) + \phi 1 \times yt\text{-}1 + et \; yt - \mu = \phi 1(yt\text{-}1 - \mu) + et$

Se você inspecionar a Equação 6.4 , o que deve chamar sua atenção é que ela tem expressões muito semelhantes nos lados esquerdo e direito, ou seja, $yt - \mu$ e $yt\text{-}1 - \mu$. Dado que esta série temporal é estacionária, sabemos que a matemática no tempo $t - 1$ deve ser a mesma que a matemática no tempo t . Reescrevemos a Equação 6-4 no quadro de um passo

anteriormente como Equação 6-5 .

Equação 6-5. $yt\text{-}1 - \mu = \phi 1(yt\text{-}2 - \mu) + et\text{-}1$

Podemos então substituir isso na Equação 6-4 da seguinte forma:

$yt - \mu = \phi 1(\phi 1(yt\text{-}2 - \mu) + et\text{-}1) + et$ Reorganizamos para maior clareza na Equação 6-6.

Equação 6-6. $yt - \mu = et + \phi 1(et\text{-}1 + \phi 1(yt\text{-}2 - \mu))$

Deve chamar sua atenção que outra substituição é possível na Equação 6-6 para $yt\text{-}2 - \mu$ usando a mesma substituição recursiva que usamos anteriormente, mas em vez de trabalhar em yt –1, trabalharemos em yt –2. Se você fizer

essa substituição, o padrão fica claro: yt - μ = et + ϕ 1(et -1 + ϕ 1(et -2 + ϕ 1(yt -3 - μ)))

= et + ϕ × et -1 + ϕ 2 × et -2 + ϕ 3 × et -3 + (expressões ainda a serem substituídas) Portanto, podemos concluir de forma mais geral que y

∞

eu

t - μ = $\sum$ i =1 ϕ 1 × et - i .

Em linguagem simples, yt menos a média do processo é uma função linear dos termos de erro.

Métodos Estatísticos Desenvolvidos para Séries Temporais | 169

Este resultado pode então ser usado para calcular a expectativa de E (yt - μ) × et +1 = 0 dado que os valores de et em diferentes valores de t são independentes. Disto podemos concluir que a covariância de yt –1 e et é 0, como deveria ser. Podemos aplicar uma lógica semelhante para calcular a variância de yt elevando ao quadrado esta equação:

yt - μ = ϕ 1(yt -1 - μ) + et

$variar$

2

t) = ϕ 1 var (yt -1) + var (et) Porque as quantidades de variância em cada lado da equação devem ser iguais devido à estacionariedade, tal que (var (y t) = var (y t – 1), isso implica que: var (yt) = var (et)

1 – ϕ 2

1

Dado que a variância deve ser maior ou igual a 0 por definição, podemos ver que $\phi 21$ deve ser menor que 1 para garantir um valor positivo no lado direito da equação anterior. Isto implica que para um processo estacionário devemos ter $-1 < \phi 1 < 1$. Esta é uma condição necessária e suficiente para estacionariedade fraca.

Estacionaridade fraca versus forte

Quando discutimos a estacionariedade neste livro, estamos sempre nos referindo à estacionariedade fraca. A estacionariedade fraca requer apenas que a média e a variância de um processo sejam invariantes no tempo.

A estacionariedade forte exige que a distribuição das variáveis aleatórias produzidas por um processo permaneça a mesma ao longo do tempo, então, por exemplo, exige que a distribuição estatística de $y\,1$, $y\,2$, $y\,3$ seja a mesma que $y\,101$, $y\,102$, $y\,103$ para qualquer medida dessa distribuição em vez do primeiro e segundo momentos (a média e a variância).

Curiosidade: embora a estacionariedade forte pareça abranger totalmente o caso de estacionariedade fraca, isso não é inteiramente assim. A estacionariedade forte inclui a estacionariedade fraca onde a média e a variância de um processo existem, mas se um processo não tiver uma média ou uma variância, ele pode, no entanto, ser fortemente estacionário, mesmo que não possa ser fracamente estacionário. O exemplo clássico disso é a *distribuição de Cauchy* , que possui média indefinida e variância infinita. Um processo de Cauchy não pode atender aos requisitos de estacionariedade fraca, mas pode atender aos requisitos de estacionariedade forte.



Estudamos o processo AR(1) porque é o processo autorregressivo mais simples. Na prática, você vai encaixar modelos mais complexos o tempo todo. É possível derivar condições semelhantes de estacionariedade para uma ordem arbitrária AR(*p*), e há muitos livros onde isso é demonstrado. Se você estiver interessado em ver isso resolvido de forma mais explícita, dê uma olhada nos recursos listados no final do capítulo. A conclusão mais importante desta discussão é que as séries temporais são bastante acessíveis com alguma álgebra e estatística, e que a estacionariedade não é simplesmente uma questão de representar graficamente um modelo, mas sim um conceito matemático que pode ser trabalhado com respeito às especificidades de um modelo. qualquer modelo estatístico.

Uma *distribuição* é uma função estatística que descreve a probabilidade, para todos os valores possíveis, de que um determinado valor seja gerado por um

processo. Embora você possa não ter encontrado esse termo formalmente,

você sem dúvida encontrou esse conceito. Por exemplo,

considere a *curva em sino* , que é uma distribuição de probabilidade que indica que a maioria das medições cairá perto e será distribuída

igualmente em ambos os lados da média. Geralmente é chamado de *normal* ou

Distribuição gaussiana em estatísticas.

Escolhendo parâmetros para um modelo AR(p)

Para avaliar a adequação de um modelo AR para seus dados, comece traçando o processo e sua *função de autocorrelação parcial* (PACF). O PACF de um processo AR deve ser cortado para zero além da ordem *p* de um processo AR(*p*), dando uma indicação concreta e visual da ordem de um processo de AR visto empiricamente nos dados.

Por outro lado, um processo AR não terá uma função de autocorrelação informativa (ACF), embora tenha a forma

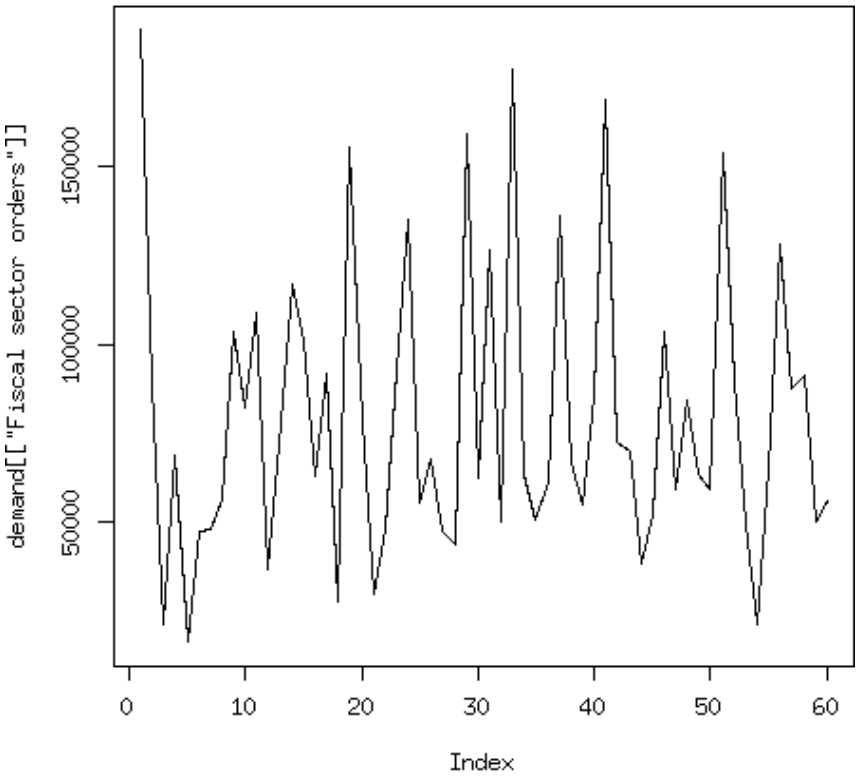
característica de um ACF: redução gradual exponencial com aumento do deslocamento de tempo.

Vamos dar uma olhada nisso com alguns dados reais. Usamos alguns dados de previsão de demanda

publicado no Repositório de Aprendizado de Máquina UCI.

Primeiro plotamos os dados em ordem cronológica (Figura 6-1). Como modelaremos isso como um processo AR, recorremos ao PACF para definir um ponto de corte na ordem do processo

(Figura 6-2).



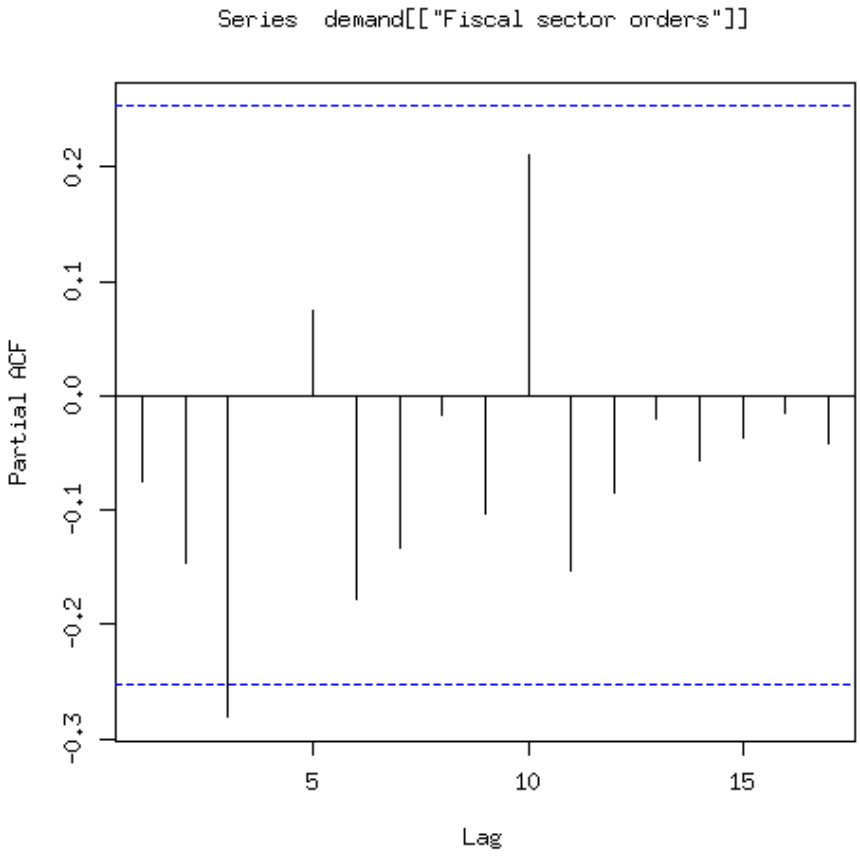


Figura 6-1. Número diário de ordens bancárias (2).

Figura 6-2. PACF da série temporal de pedidos não transformados ilustrada na Figura 6-1.

Podemos ver que o valor do PACF ultrapassa o limite de significância de 5% no atraso 3.

Isto é consistente com os resultados da função ar() disponível nas estatísticas do R. **172 | Capítulo 6: Modelos Estatísticos para Séries Temporais**

pacote. ar() escolhe automaticamente a ordem de um modelo autoregressivo se nenhum for especificado:

##R

> fit <- ar(demand[["Ordens bancárias (2)"]], método = "mle")

> ajuste

Chamar:

ar(x = demanda[["Ordens bancárias (2)"]], método = "mle")

Coeficientes:

1 2 3

-0,1360 -0,2014 -0,3175

Se olharmos a documentação da função ar(), podemos ver que a ordem selecionada é determinada (com os parâmetros padrão que deixamos intactos) com base no *critério de informação de Akaike* (AIC). É útil saber isso porque mostra que a seleção visual que fizemos ao examinar o PACF é consistente com a seleção que seria feita minimizando um critério de informação. Estas são duas formas diferentes de selecionar a ordem do modelo, mas neste caso são consistentes.

Critério de Informação Akaike

O AIC de um modelo é igual a AIC = 2 k – 2ln L onde k é o número de parâmetros do modelo e L é o valor de máxima verossimilhança para aquela função. Em geral, queremos diminuir a complexidade do modelo (ou seja, diminuir k) e, ao mesmo tempo, aumentar a probabilidade/adequação do ajuste do modelo (ou seja, L). Então vamos favorecer modelos com AIC menores

valores superiores àqueles com maiores valores de AIC.

Uma função de verossimilhança é uma medida da probabilidade de um determinado conjunto de parâmetros para uma função ser relativo a outros parâmetros dessa função, dados os dados. Então imagine que você está ajustando uma regressão linear de x sobre y dos seguintes dados:

xy

1 1

2 2

3 3

Se você ajustasse isso ao modelo $y = b \times x$, sua função de verossimilhança lhe diria que uma estimativa de $b = 1$ era muito mais provável do que uma estimativa de $b = 0$. Você pode pensar nas funções de verossimilhança como uma ferramenta para ajudar você identifica os parâmetros verdadeiros mais prováveis de um modelo, dado um conjunto de dados.

Observe que a função ar() também nos forneceu os coeficientes do modelo.

Podemos, no entanto, querer limitar os coeficientes. Por exemplo, olhando para o PACF, podemos nos perguntar se realmente queremos incluir um coeficiente para a defasagem – 1 termo **Métodos Estatísticos Desenvolvidos para Séries Temporais | 173**

ou se deveríamos atribuir a esse termo um coeficiente obrigatório de 0, dado que o seu valor PACF está bem abaixo do limiar utilizado para significância. Neste caso, podemos usar a função arima() também do pacote stats.

Aqui, demonstramos como chamar a função para ajustar um AR(3), definindo o parâmetro order como c(3, 0, 0), onde 3 se refere à ordem do componente AR (em exemplos posteriores especificaremos outros componentes para os parâmetros de média diferencial e móvel abordados nas próximas páginas deste capítulo):

##R

> est <- arima(x = demanda[["Ordens bancárias (2)"]],

> ordem = c(3, 0, 0))

> est

Chamar:

arima(x = demanda[["Ordens bancárias (2)"]], pedido = c(3, 0, 0))

Coeficientes:

ar1 ar2 ar3 interceptar

-0,1358 -0,2013 -0,3176 79075,350

se 0,1299 0,1289 0,1296 2981,125

sigma ^ 2 estimado como 1,414e + 09: log de probabilidade = -717,42,

aico = 1444,83

Para injetar conhecimento ou opinião prévia em nosso modelo, podemos restringir um coeficiente a 0. Por exemplo, se quisermos restringir o termo lag – 1 para permanecer 0 em nosso modelo, usamos a seguinte chamada:

##R

> est.1 <- arima(x = demanda[["Ordens bancárias (2)"]],

> ordem = c(3, 0, 0),

> fixo = c(0, **NA** , **NA** , **NA**))

>est.1

Chamar:

arima(x = demanda[["Ordens bancárias (2)"]],

ordem = c(3, 0, 0),

fixo = c(0, **NA** , **NA** , **NA**))

Coeficientes:

ar1 ar2 ar3 interceptar

0 -0,1831 -0,3031 79190,705

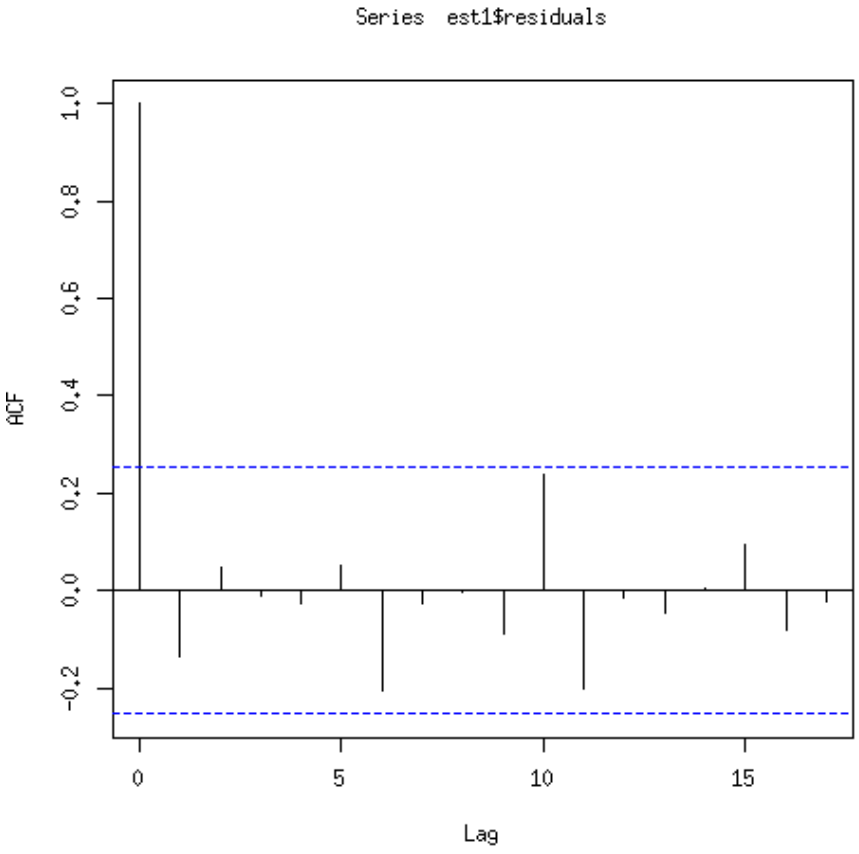
se 0 0,1289 0,1298 3345,253

sigma ^ 2 estimado como 1,44e + 09: log de probabilidade = -717,96,

aico = 1443,91

Definir um valor no vetor passado para o parâmetro fixo da função arima como 0

em vez de NA, restringirá esse valor para permanecer 0:



##R

> fixo <- c(0, **NA** , **NA** , **NA**)

Agora inspecionamos o desempenho de nosso modelo em nossos dados de treinamento para avaliar a qualidade do ajuste de nosso modelo a esse conjunto de dados. Podemos fazer isso de duas maneiras. Primeiro, traçamos o ACF

dos resíduos (ou seja, os erros) para ver se existe um padrão de autocorrelação que nosso modelo não cobre.

Traçar os resíduos é bastante simples graças à saída da função arima()

(veja a Figura 6-3):

##R

> acf(est.1$resíduos)

Figura 6-3. ACF dos resíduos do modelo AR(3) ajustamos forçando a defasagem – 1 parâmetro para 0.

Nenhum dos valores do ACF ultrapassa o limite de significância. É claro que não devemos ceder cegamente a um limiar de significância para avaliar ou rejeitar a significância, mas esta observação é um dado útil num modelo que já consideramos razoável por outras razões.

Não vemos aqui um padrão de autocorrelação entre os resíduos (ou seja, os termos de erro). Se tivéssemos observado tal padrão, provavelmente desejaríamos retornar ao nosso modelo original e considerar a inclusão de termos adicionais para aumentar a complexidade e dar conta da autocorrelação significativa dos resíduos.

Outro teste comumente realizado é o *teste Ljung-Box* , um teste geral de aleatoriedade de uma série temporal. Mais formalmente, apresenta as seguintes hipóteses nulas e alternativas:

• H0: Os dados não apresentam correlação serial.

• H1: Os dados apresentam correlação serial.

Este teste é comumente aplicado a modelos AR (e mais geralmente, ARIMA), e

mais especificamente aos resíduos do ajuste do modelo, e não ao modelo em si:

##R

> Box.test(est.1$residuais, lag = 10, type = "Ljung", fitdf = 3) Teste Box-Ljung

dados: est.1$resíduos

X-quadrado = 9,3261, df = 7, valor p = 0,2301

Aplicamos o teste Ljung-Box ao nosso modelo est.1 para avaliar sua qualidade de ajuste. Não podemos rejeitar a hipótese nula de que os dados não apresentam correlação serial. Isto é uma confirmação do que acabamos de descobrir ao traçar o ACF dos resíduos.

Previsão com um processo AR(p)

Nas seções a seguir ilustraremos como fazer previsões com processos de AR.

Primeiro exploramos o caso de um passo à frente e depois discutimos como a previsão de vários passos à frente difere do caso posterior. A boa notícia é que do ponto de vista da codificação não há muita diferença, embora a matemática subjacente seja mais elaborada neste último caso.

Prevendo um passo à frente. Primeiro consideramos o caso em que queremos prever um passo à frente com um modelo AR conhecido (ou estimado). Nesse caso, na verdade temos todas as informações de que precisamos.

Continuamos trabalhando com o modelo a partir dos dados de demanda, com o coeficiente defasagem – 1 restrito a 0 (ajustado como est.1 anteriormente).

Traçamos a previsão usando a função equipada() do pacote de previsão. Aqui está o código completo; é bem fácil de fazer:

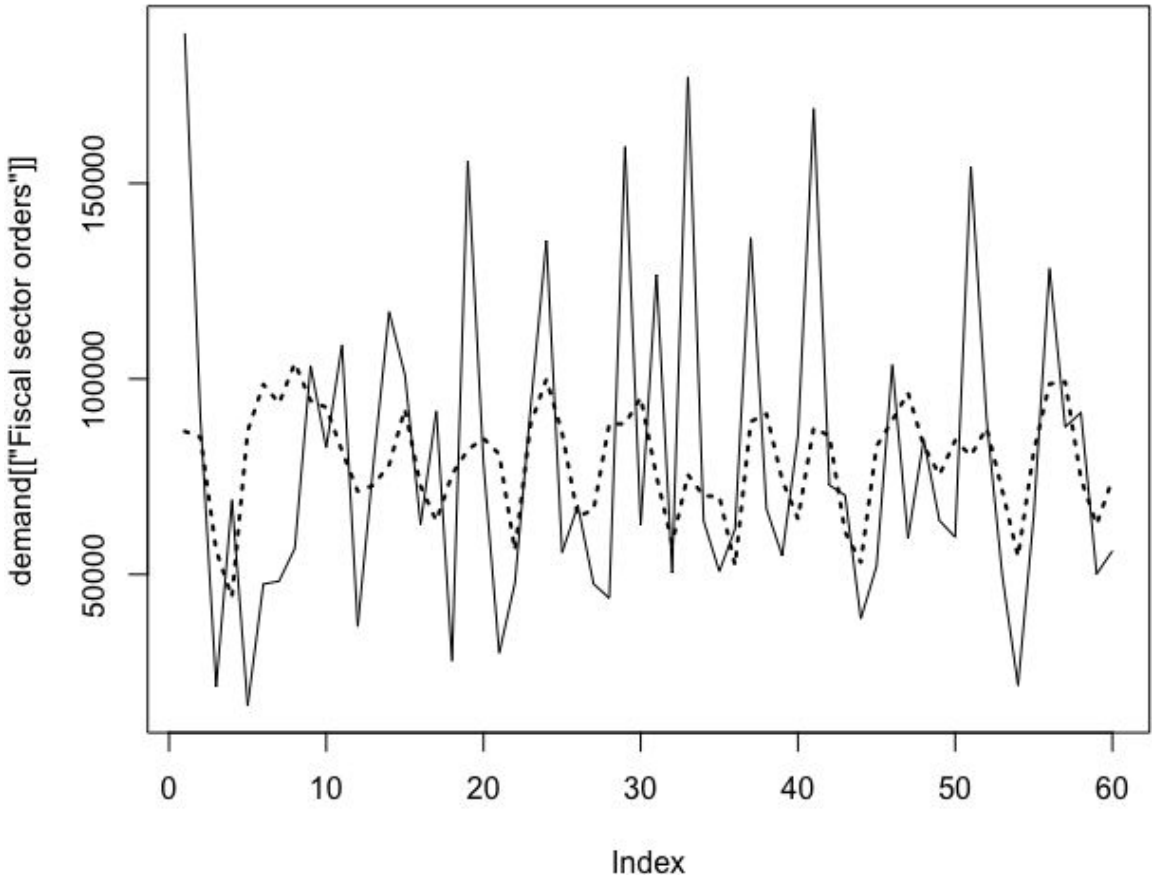
##R

> exigir (previsão)

> plot(demand[["Ordens bancárias (2)"]], type = 'l')

> linhas(fitted(est.1), col = 3, lwd = 2) *## use o pacote de previsão*





Isso resulta no gráfico mostrado na Figura 6-4.

Figura 6-4. Aqui vemos a série temporal original na linha sólida e a série temporal ajustada na linha tracejada.

Modelos AR(p) estão movendo funções de janela

Temos uma opção além de usar o pacote de previsão ajustado

funcionalidade para produzir uma previsão do nosso modelo AR(3): nós

poderia codificar manualmente uma previsão de forma mais explícita usando o pacote zoo –

função rollapply() de age, que discutimos anteriormente. Que

função para calcular funções de janela também pode calcular um AR

processo. Para fazer isso, pegaríamos os coeficientes do ajuste ar()

e aplique esses pesos a um vetor de entrada representando os valores

dos diferentes desfasamentos para produzir uma previsão em cada ponto. eu deixo isso

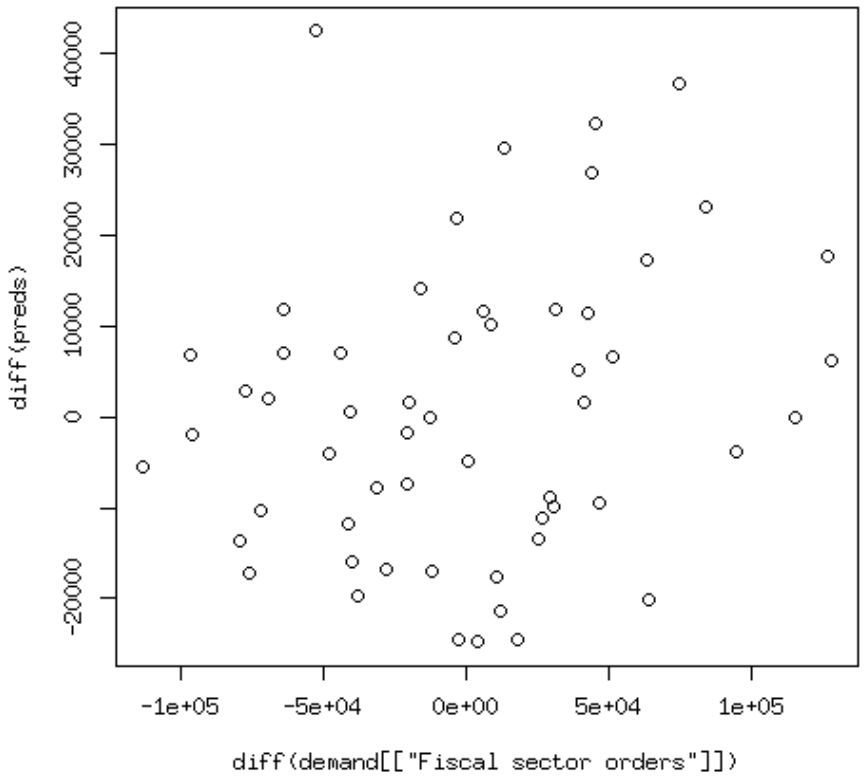
como um exercício para você.

Agora vamos pensar na qualidade da previsão. Se calcularmos a correlação entre o valor previsto e o valor real, obtemos 0,29. Isso não é ruim em alguns contextos, mas lembre-se de que, às vezes, diferenciar os dados removerá o que parecia ser um relacionamento forte e o substituirá por um que seja essencialmente aleatório.

Este será o caso particularmente se os dados não forem verdadeiramente estacionários quando os ajustamos, de modo que uma tendência não identificada se disfarça como um bom desempenho do modelo quando é

na verdade, uma característica dos dados que deveríamos ter abordado antes da modelagem.

Métodos Estatísticos Desenvolvidos para Séries Temporais | 177



Podemos diferenciar as séries e os valores previstos para ver se a mudança de um período de tempo para o próximo é bem prevista pelo modelo. Mesmo após a diferenciação, as nossas previsões e os dados mostram padrões semelhantes, sugerindo que o nosso modelo é um

significativo (veja a Figura 6-5).

Figura 6-5. As séries diferenciadas e as previsões diferenciadas estão fortemente correlacionadas, sugerindo que uma relação subjacente é de facto identificada pelo modelo.

Também podemos testar se prevemos os mesmos movimentos ao mesmo tempo, traçando as séries diferenciadas e observando essa correlação. Este gráfico revela alguma correlação, que podemos confirmar

calculando o valor da correlação. O modelo funciona até mesmo para prever a mudança de um intervalo de tempo para outro.

Olhando novamente para o gráfico original da previsão versus valores reais, vemos que a principal diferença entre a previsão e os dados é que a previsão é menos variável que os dados. Pode prever corretamente a direção do futuro, mas não a escala da mudança de um período de tempo para outro. Isto não é um problema em si, mas reflecte o facto de que as previsões são médias das distribuições previstas e, portanto, terão necessariamente uma variabilidade menor do que os dados amostrados.

Esta propriedade dos modelos estatísticos é por vezes esquecida na facilidade de visualizações rápidas que tendem a sugerir um futuro mais estável do que normalmente será o caso. Ao apresentar visualizações, lembre-se de lembrar ao público o que está sendo planejado. Neste caso, as nossas previsões sugerem um futuro muito mais tranquilo do que provavelmente será o caso.

Prevendo muitos passos para o futuro. Até agora fizemos uma previsão de um passo à frente. No entanto, podemos querer prever ainda mais no futuro. Vamos imaginar que **178 | Capítulo 6: Modelos Estatísticos para Séries Temporais**

queríamos produzir uma previsão dois passos à frente em vez de uma previsão um passo à frente.

O que faríamos é primeiro produzir a previsão um passo à frente e, em seguida, usá-la para fornecer o valor *de yt* necessário para prever *yt* +1.

Observe que em nosso modelo atual, a mudança de uma previsão de um passo à frente para uma previsão de dois passos à frente não exigiria realmente essa ginástica porque o *yt* – 1

value não é usado ao prever *yt* . Tudo o que precisamos saber para fazer uma previsão dois passos à frente é conhecido e não há necessidade de fazer estimativas. Na

verdade, deveríamos chegar à mesma série de valores que obtivemos para a previsão um passo à frente – não haverá novas fontes de erro ou variabilidade.

Se quisermos prever ainda mais, precisaremos gerar valores futuros previstos como dados de nossa previsão. Vamos prever *yt* +3. Isso envolverá um modelo com coeficientes dependendo de *yt* +1 e *yt* . Portanto, precisaremos prever esses dois valores — *yt* + 1 e *yt* — e então, por sua vez, usar esses valores estimados na previsão para *yt* +3. Como antes, podemos usar a função equipada() do pacote de previsão para fazer isso

– não é mais difícil em termos de código do que a previsão de um passo à frente. Como mencionado anteriormente, isso também pode ser feito com um método rollapply(), mas é mais trabalhoso e mais sujeito a erros.

Usamos a função equipada() com um parâmetro adicional agora, h para horizonte. Como lembrete, nosso objeto est.1 representa um processo AR(3) com o coeficiente lag – 1 (tempo menos 1) restrito a 0:

##R

> ajustado (estimado 1, h = 3)

Podemos usar a facilidade de prever muitos passos de tempo no futuro para gerar muitas previsões com vários passos à frente para diferentes horizontes. No próximo exemplo, podemos ver a variação das previsões feitas cada vez mais no futuro a partir do mesmo modelo subjacente. (Observe que na exibição, os arredondamentos e os delineamentos de vírgula modificam a saída original, de modo que é mais fácil ver o que está acontecendo com a variação da estimativa à medida que o horizonte futuro aumenta.)

Como você pode ver na Figura 6-6 , a variância da previsão diminui com o aumento do horizonte futuro. A razão para isto - que destaca uma limitação importante do modelo - é que quanto mais avançamos no tempo, menos os dados reais importam porque os coeficientes para os dados de entrada olham apenas para um conjunto anterior finito de

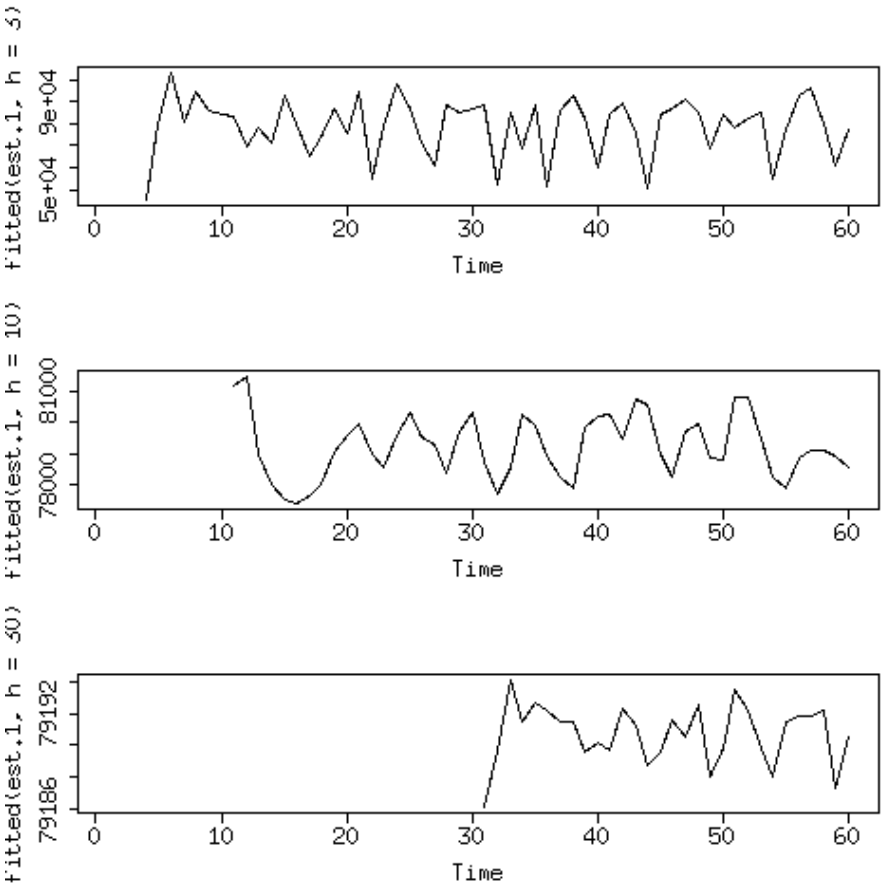
pontos no tempo (neste modelo , voltando apenas ao atraso – 3; ou seja, ao tempo – 3). Uma maneira de colocar isso é que as previsões mais distantes no tempo convergem para serem a previsão incondicional –

isto é, não condicionado a dados. A previsão futura aproxima-se do valor médio da série à medida que o horizonte temporal cresce e, portanto, a variância tanto do termo de erro como dos valores previstos diminui para 0 à medida que os valores previstos tendem para o valor médio incondicional:

##R

> var(ajustado(est.1, h = 3), na.rm = **TRUE**)

Métodos Estatísticos Desenvolvidos para Séries Temporais | 179



[1] 174.870.141

> var(ajustado(est.1, h = 5), na.rm = **TRUE**)

[1] 32.323.722

> var(ajustado(est.1, h = 10), na.rm = **TRUE**)

[1] 1.013.396

> var(ajustado(est.1, h = 20), na.rm = **TRUE**)

[1] 1.176

> var(ajustado(est.1, h = 30), na.rm = **TRUE**)

[1] 3,5

As previsões para um futuro suficientemente distante irão apenas prever a média do processo, o que faz sentido. Em algum momento num futuro distante, os nossos dados actuais não nos fornecem informação sensata específica e relevante para o futuro, e assim a nossa previsão reverte cada vez mais para as propriedades de base conhecidas sobre o processo, tais como a sua média.

Figura 6-6. Parcelas de previsões para o futuro. Os valores medidos pelo eixo y cada vez mais estreito à medida que prevemos mais no futuro e o modelo mais de perto parece fornecer uma previsão constante do valor médio do processo. Os horizontes aumento previsto de cima para baixo e são intervalos de tempo de 3, 10 e 30, respectivamente.

O ponto importante a lembrar disso é que os modelos AR (assim como MA, ARMA e ARIMA, que serão discutidos em breve) são os melhores para fazer previsões de curto prazo. Esses modelos perdem poder preditivo para grandes horizontes futuros.





Para os restantes modelos, iremos proceder a um tratamento semelhante ao que demos aos modelos autorregressivos, embora com menos detalhe global. Uma discussão completa está disponível em todos os livros padrão sobre análise de séries temporais.

Modelos de média móvel

Um modelo de média móvel (MA) baseia-se na imagem de um processo no qual o valor em cada ponto no tempo é uma função dos termos de “erro” do valor passado recente, cada um dos quais é independente dos outros. Revisaremos esse modelo na mesma série de etapas que usamos para estudar modelos AR.

Equivalência AR MA

Em muitos casos, um processo MA pode ser expresso como uma ordem infinita

Processo de AR. Da mesma forma, em muitos casos, um processo AR pode ser

expresso como um processo MA de ordem infinita. Para saber mais, veja

na invertibilidade de um processo MA, a teoria da representação de Wold

rem e a dualidade dos processos MA/AR em geral. A matemática

A matemática envolvida está muito além do escopo deste livro!

O modelo

Um modelo de média móvel pode ser expresso de forma semelhante a um modelo autoregressivo, exceto que os termos incluídos na equação linear referem-se a termos de erro presentes e passados, em vez de valores presentes e passados do próprio processo. Portanto, um modelo MA de ordem q é expresso como:

$yt = \mu + et + \theta 1 \times et\text{-}1 + \theta 2 \times et\text{-}2. . + \theta q \times et - q$ Não confunda o modelo MA com uma média móvel. Eles são

não é a mesma coisa. Depois de saber como ajustar uma média móvel

processo, você pode até comparar o ajuste de um modelo MA a um modelo móvel

média da série temporal subjacente. Deixo isso como um exercício para

você.

Os economistas falam sobre esses termos de erro como "choques" para o sistema, enquanto alguém com formação em engenharia elétrica poderia falar sobre isso como uma série de impulsos e o próprio modelo como um filtro de resposta ao impulso finito, o que significa que os efeitos de qualquer impulso específico permanecem apenas por um período finito de tempo. A redação não é importante, mas o conceito de muitos eventos independentes em diferentes momentos passados afeta os **métodos estatísticos desenvolvidos para séries temporais | 181**

o valor atual do processo, cada um fazendo uma contribuição individual, é a ideia principal.

O operador de retrocesso

O operador backshift, também conhecido como operador lag, opera em pontos da série temporal e os move para trás

um passo de tempo cada vez que é aplicado. Em geral:

$B\ k\ yt = yt - k$

O operador backshift ajuda a simplificar a expressão de modelos de séries temporais. Por exemplo, um modelo MA pode ser reescrito como:

$yt = \mu + (1 + \theta 1 \times B + \theta 2 \times B 2 + . . + \theta q \times B\ q\ )\ e$ os modelos MA são, por definição, fracamente estacionários, sem a necessidade de impor quaisquer restrições em seus parâmetros. Isso ocorre porque a média e a variância de um processo MA são finitas e invariantes com o tempo porque os termos de erro são considerados iid com média 0. Podemos ver isso como:

$E\ (\ yt = \mu + et + \theta 1 \times et\ -1 + \theta 2 \times et\ -2. + \theta q \times et\ -\ q\ )$

$= E\ (\ \mu\ ) + \theta 1 \times 0 + \theta 2 \times 0 + . . = \mu$

Para calcular a variância do processo, utilizamos o fato de que os termos *et* são iid e também a propriedade estatística geral de que a variância da soma de duas variáveis aleatórias é igual às suas variâncias individuais mais duas vezes sua covariância. Para variáveis iid, a covariância é 0. Isso produz a expressão:

Variar

2

2

2

2

$t\ ) = (1 + \theta 1 + \theta 2 + . . + \theta q\ ) \times \sigma e$

Portanto, tanto a média quanto a variância de um processo MA são constantes com o tempo, independentemente dos valores dos parâmetros.

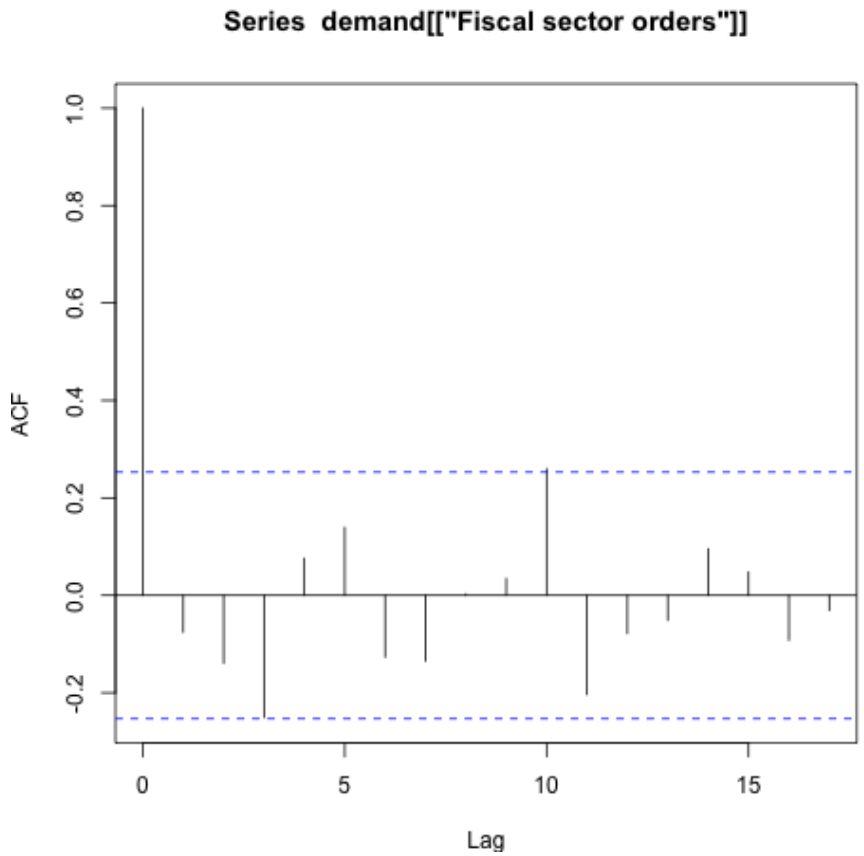
Selecionando parâmetros para um processo MA(q)

Ajustamos um modelo MA aos mesmos dados usados no modelo AR, e podemos usar o ACF

para determinar a ordem do processo MA (veja a Figura 6-7). Antes de continuar lendo, pense em como funciona um processo MA e veja se você consegue entender por que usamos o ACF

em vez do PACF para determinar a ordem do processo:





##R

```
> acf(demand[["Ordens bancárias (2)"]])
```

Figura 6-7. Usamos o ACF da série temporal de demanda para determinar a ordem do MA modelo.

Os padrões ACF e PACF diferem dos processos MA e AR

Ao contrário de um processo autorregressivo, que tem um processo de decaimento lento

ACF, a definição do processo MA garante um corte acentuado do

ACF para qualquer valor maior que q , a ordem do processo MA.

Isso ocorre porque um processo autorregressivo depende de

termos, e incorporam impulsos anteriores ao sistema,

enquanto um modelo MA, incorporando os impulsos diretamente

através do seu valor, possui um mecanismo para parar a propagação do impulso

impedimento de progredir indefinidamente.

Vemos valores significativos nas defasagens 3 e 9, então ajustamos um modelo MA com essas defasagens. Precisamos ter cuidado para não restringirmos acidentalmente os coeficientes errados a 0 no modelo, o que podemos confirmar imprimindo o display:

##R

```
> ma.est = arima(x = demanda[["Ordens bancárias (2)"]],

ordem = c(0, 0, 9),
```

fixo = c(0, 0, **NA** , rep(0, 5), **NA** , **NA**))

>ma.est

Chamar:

arima(x = demanda[["Ordens bancárias (2)"]], pedido = c(0, 0, 9),

Métodos Estatísticos Desenvolvidos para Séries Temporais | 183

fixo = c(0, 0, **NA** , rep(0, 5), **NA** , **NA**)) Coeficientes:

ma1 ma2 ma3 ma4 ma5 ma6 ma7 ma8 ma9 interceptar

0 0 -0,4725 0 0 0 0 0 -0,0120 79689,81

se 0 0 0,1459 0 0 0 0 0 0,1444 2674,60

sigma ^ 2 estimado como 1,4e + 09: log probabilidade = -717,31,

aico = 1442,61

Devemos também verificar nosso ajuste, como fizemos para o modelo AR, traçando o ACF dos resíduos do modelo e, como um segundo teste separado de desempenho do modelo, executando o teste Ljung-Box para verificar a aleatoriedade geral em qualquer ajuste aos resíduos. Observe que a entrada Box.test() exige que especifiquemos o número de graus de liberdade—

isto é, quantos parâmetros do modelo estavam livres para serem estimados, em vez de serem restritos a um valor específico. Neste caso, os parâmetros livres foram o intercepto e também os termos MA3 e MA9:

##R

> Box.test(ma.est$residuals, lag = 10, type = "Ljung", fitdf = 3) Teste Box-Ljung

dados: ma.est$resíduos

X-quadrado = 7,6516, df = 7, valor p = 0,3643

Não podemos rejeitar a hipótese nula de que não há correlação temporal entre os pontos residuais. Da mesma forma, um gráfico do ACF dos resíduos não sugere correlação temporal (isso fica como exercício para o leitor).

Previsão de um processo MA(q)

Podemos gerar uma previsão novamente usando as técnicas mostradas anteriormente para um processo AR, contando com o método fit() do pacote de previsão:

##R

> ajustado(ma.est, h=1)

Série temporal:

Início = 1

Fim = 60

Frequência = 1

[1] 90116,64 80626,91 74090,45 38321,61 74734,77 101153,20 65930,90

[8] 106351,80 104138,05 86938,99 102868,16 80502,02 81466,01 77619,15

[15] 100984,93 81463,10 61622,54 79660,81 88563,91 65370,99 104679,89

[22] 48047,39 73070,29 115034,16 80034,03 70052,29 70728,85 90437,86

[29] 80684,44 91533,59 101668,18 42273,27 93055,40 68187,65 75863,50

[36] 40195,15 82368,91 90605,60 69924,83 54032,55 90866,20 85839,41

[43] 64932,70 43030,64 85575,32 76561,14 82047,95 95683,35 66553,13

[50] 89532,20 85102,64 80937,97 93926,74 47468,84 75223,67 100887,60

[57] 92059,32 84459,85 67112,16 80917,23



Os modelos MA exibem forte reversão à média e assim as previsões convergem rapidamente para a média do processo. Isto faz sentido, visto que o processo é considerado uma função do ruído branco.

Se você fizer uma previsão além do intervalo do modelo estabelecido pela sua ordem, a previsão será necessariamente a média do processo por definição do processo. Considere um modelo MA(1):

$yt = \mu + \theta 1 \times et\text{-}1 + et$

Para prever um intervalo de tempo no futuro, nossa estimativa para $yt+1$ é $\mu + \theta 1 \times yt + et$. Se quisermos prever dois intervalos de tempo no futuro, nossa estimativa é:

$E(yt+2 = \mu + et+2 + \theta 1 \times ; et+1) = \mu + 0 + \theta 1 \times 0 = \mu$

Com um processo MA(1) não podemos oferecer uma previsão informada além de um passo à frente, e para um processo MA(q) em geral não podemos oferecer uma previsão mais informada além de q passos do que o valor médio emitido pelo processo. Por previsão *informada* ,

quero dizer aquela em que as nossas medições mais recentes têm impacto na previsão.

A notação tradicional é negativa

Observe que o modelo MA normalmente não é escrito como aqui. Tra-

tradicionalmente, o sinal na frente dos coeficientes teta é negativo. Esse

é por razões de como ele pode ser derivado, e uma maneira de pensar sobre

um modelo MA é como um modelo AR com restrições em seus parâmetros

termos. Esta formulação, com muita álgebra, leva a coeficiente negativo

especialistas em teta.

Podemos ver isso produzindo previsões com nosso modelo MA(9) que acabamos de ajustar e para o qual agora buscamos previsões 10 passos à frente:

##R

> ajustado(ma.est, h=10)

Série temporal:

Início = 1

Fim = 60

Frequência = 1

[1] **NA E E E E E E E**

[9] **NA NA** 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81

[17] 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81

[25] 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81

[33] 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81

[41] 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81

[49] 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81 79689,81

[57] 79689,81 79689,81 79689,81 79689,81

Quando tentamos prever 10 passos de tempo no futuro, prevemos a média de cada passo de tempo. Poderíamos ter feito isso sem um modelo estatístico sofisticado!

O bom senso é importante. Se você aplicar um modelo sem entender como ele funciona, poderá fazer coisas embaraçosas, como enviar ao seu chefe exatamente a mesma previsão todos os dias depois de ter perdido tempo anterior e recursos de computação produzindo um modelo inadequado para a pergunta que você está fazendo.

Modelos de média móvel integrados autorregressivos

Agora que examinamos os modelos AR e MA individualmente, olhamos para o

Modelo de média móvel integrada autoregressiva (ARIMA), que os combina, reconhecendo que a mesma série temporal pode ter dinâmicas de modelo AR e MA subjacentes. Isto por si só nos levaria a um modelo ARMA, mas estendemos ao

Modelo ARIMA, que leva em conta a diferenciação, uma forma de remover tendências e tornar uma série temporal

estacionária.

O que é diferencial?

Conforme discutido anteriormente neste livro, diferenciar é converter uma série temporal de valores em uma série temporal de mudanças de valores ao longo do tempo. Na maioria das vezes, isso é feito calculando diferenças entre pares de pontos adjacentes no tempo, de modo que o valor da série diferenciada no tempo t seja o valor no tempo t menos o valor no tempo t – 1. No entanto, a diferenciação também pode ser realizada em diferentes janelas de atraso, conforme for conveniente.

Os modelos ARIMA continuam a oferecer desempenho próximo do estado da arte, especialmente em casos de pequenos conjuntos de dados onde o aprendizado de máquina mais sofisticado ou modelos de aprendizado profundo não estão no seu melhor. No entanto, mesmo os modelos ARIMA apresentam o perigo de overfitting, apesar da sua relativa simplicidade.

O modelo

Você provavelmente está coçando a cabeça neste ponto se estiver prestando atenção, porque acabei de ajustar os mesmos dados a um processo de AR e MA sem comentar sobre isso. Esse é o tipo de hábito irritante que você às vezes pode encontrar em livros didáticos de análise de séries temporais. Alguns autores aceitarão essa preguiça de dados, enquanto outros a ignorarão brandamente. Não investigamos em profundidade se algum dos nossos modelos anteriores é um ajuste particularmente bom, mas parece claro, a partir do processo de ajuste que utilizamos, que existem argumentos defensáveis para descrever os dados com um modelo AR ou MA. Isto levanta a questão: não seria útil incorporar ambos os comportamentos no mesmo modelo?



A Tabela 6-1 pode ser uma maneira prática de examinar um processo de série temporal para ver se um

A descrição AR, MA ou ARMA do processo é a melhor.

Tabela 6-1. Determinar qual modelo descreve melhor nossa série temporal

Tipo de enredo

AR(p)

MA(q)

ARMA

Comportamento da ACF

Cai lentamente

Queda acentuada após atraso = q Sem corte acentuado

Comportamento do PACF Queda acentuada após lag = p Cai lentamente

Sem corte brusco

Isto nos leva a um modelo de média móvel autorregressiva (ARMA), que é aplicado no caso em que nem os termos AR nem MA por si só descrevem suficientemente a dinâmica empírica. Este é um caso provável quando os diagnósticos para estatísticas de ordem AR e MA (PACF e ACF, respectivamente) apontam para valores diferentes de zero, indicando um termo de uma determinada ordem para um termo AR ou MA. Estes podem ser combinados com um ARMA

modelo.

Teorema de Wold

O teorema de Wold nos diz que toda série temporal estacionária de covariância

pode ser escrito como a soma de duas séries temporais, uma determinística e outra

um estocástico. Com base neste teorema, também podemos dizer que uma estação

processo temporário pode ser razoavelmente aproximado por um ARMA

modelo, embora possa ser bastante difícil encontrar o

modelo apropriado.

Aqui mudamos para uma notação estatística mais tradicional, aplicando sinais negativos aos coeficientes do processo MA:

$yt = \phi 0 + \sum (\phi i \times ; rt - i) + et - \sum (\theta i \times et - i)$ **Um modelo ARMA não é necessariamente único**

Como pode haver fatores comuns entre as porções AR e MA da equação, é possível que um modelo ARMA(p , q) possa realmente ser reduzido a outro conjunto de parâmetros. Precisamos evitar esse tipo de situação degenerada. Em geral, você deve escolher um modelo ARMA parcimonioso. Discussões adicionais estão além do escopo deste livro, mas referências adicionais são fornecidas no final do capítulo.

Um método simples de identificar casos em que isso pode acontecer é usar a formulação do operador lag (discutido antes) do modelo ARMA e usar a boa e velha fatoração polinomial para procurar fatores comuns em seus modelos AR e MA que poderiam se dividir para produzir um modelo equivalente, mas mais simples. Aqui está um exemplo. Uma forma de expressar um modelo ARMA é colocar os componentes AR de um lado e os componentes MA do outro.

Suponha que temos o seguinte modelo ARMA:

yt = 0,5 × yt-1 + 0,24 × yt-2 + et + 0,6 × et-1 + 0,09 × et-2

Podemos reescrever isso de forma fatorada movendo os termos AR (os coeficientes *x*) para um lado e deixando os termos MA do outro lado:

yt – 0,5 × yt-1 – 0,24 × yt-2 = et + 0,6 × et-1 + 0,09 × et-2

Em seguida, reexpressamos isso usando o operador lag. Assim, por exemplo, *yt* –2 é reexpresso como *L* 2 *y* :

yt – 0,5 × L × yt – 0,24 × L2 × yt = et + 0,6 × L × et + 0,09 × L 2 × et Ou simplificando fatorando os fatores comuns de *yt* no lado esquerdo e *et* no lado direito, e então fatorando os polinômios, temos:

(1 + 0,3L) × (1 - 0,8L) × yt = (1 + 0,3L) × (1 + 0,3L) × et

Observe que, ao fatorar, devemos manter os valores operados pelo operador de atraso à direita dele, para que fatoremos *yt* à direita dos fatores polinomiais que envolvem o operador *L* backshift, em vez de à esquerda, como pode parecer mais natural.

Depois de termos feito esta fatoração, temos um fator comum no lado esquerdo e no lado direito, a saber (1 + 0,3 *L*), e podemos cancelar isso de ambos os lados, ficando com:

(1 – 0,8L) × yt = (1 + 0,3L) × et

Após a fatoração, a ordem do modelo geral foi reduzida e agora envolve apenas valores no tempo *t* – 1, em vez de também valores em *t* – 2. Isso deve servir como um aviso de que mesmo um modelo que não parece muito complicado, como como ARMA(2,2), às vezes pode mascarar o fato de que é na verdade equivalente a uma forma de modelo mais simples.

Também é bom ter isso em mente ao ajustar modelos computacionalmente diretamente. Em conjuntos de dados do mundo real, você pode acabar com algo em que os dois lados sejam numericamente bastante semelhantes, mas não exatos. Talvez no lado esquerdo você possa encontrar um fator como (1 – 0,29 L), enquanto no lado direito é (1 – 0,33 L). Nesses casos, vale a pena considerar que isso está próximo o suficiente de um fator comum para diminuir a complexidade do modelo.

A estacionariedade do processo ARMA se resume à estacionariedade de seu componente AR e é controlada pela mesma equação característica que controla se um modelo AR é estacionário.

De um modelo ARMA, é uma transição simples para um modelo ARIMA. A diferença entre um modelo ARMA e um modelo ARIMA é que o modelo ARIMA inclui

o termo *integrado* , que se refere a quantas vezes a série temporal modelada deve ser diferenciada para produzir estacionariedade.

Os modelos ARIMA são muito mais amplamente utilizados na prática, particularmente em pesquisas acadêmicas e problemas de previsão, do que os modelos AR, MA e ARMA. Uma rápida pesquisa acadêmica no Google revela que o ARIMA é aplicado a uma variedade de problemas de previsão, incluindo:

• Passageiros de viagens aéreas de entrada para Taiwan

• Demanda de energia na Turquia por tipo de combustível

• Vendas diárias em um mercado atacadista de vegetais na Índia

• Demanda de pronto-socorro no oeste dos Estados Unidos

É importante ressaltar que a ordem de diferenciação não deve ser muito grande. Em geral, o valor de cada parâmetro

de um modelo ARIMA(*p, d, q*) deve ser mantido o menor possível para evitar complexidade injustificada e ajuste excessivo aos dados da amostra. Como regra prática nada universal, você deve ser bastante cético em relação aos valores de *d* acima de 2 e aos valores de *p* e *q* acima de 5 ou mais. Além disso, você deve esperar que o termo *p* ou *q* domine e o outro seja relativamente pequeno. Estas são notas práticas coletadas de analistas e não verdades matemáticas concretas.

Selecionando parâmetros

O modelo ARIMA é especificado em termos dos parâmetros (*p, d, q*). Selecionamos os valores de *p* , *d* e *q* que são apropriados de acordo com os dados que temos.

Aqui estão alguns exemplos bem conhecidos da descrição do ARIMA na Wikipedia

modelos:

• ARIMA(0, 0, 0) é um modelo de ruído branco.

• ARIMA(0, 1, 0) é um passeio aleatório, e ARIMA(0, 1, 0) com uma constante diferente de zero é um passeio aleatório com desvio.

• ARIMA(0, 1, 1) é um modelo de suavização exponencial, e um ARIMA(0, 2, 2) é igual ao método linear de Holt, que estende a suavização exponencial a dados com uma tendência, para que possa ser usado para dados de previsão com uma tendência subjacente.

Escolhemos a ordem do nosso modelo com base em uma combinação de conhecimento de domínio, várias métricas de avaliação de ajuste (como o AIC) e conhecimento geral de como **métodos estatísticos desenvolvidos para séries temporais | 189**

o PACF e o ACF devem aparecer para um determinado processo subjacente (conforme descrito em

Tabela 6-1). A seguir, demonstraremos o ajuste de um modelo ARIMA a um processo usando um processo iterativo

manual baseado no PACF e ACF e também baseado em uma ferramenta automatizada de seleção de parâmetros por meio do auto.arima() do pacote de previsão.

função.

Ajustar manualmente um modelo. Existem heurísticas para escolha dos parâmetros de um modelo ARIMA, onde *a parcimônia é fundamental* . Um método popular e antigo é o método Box-Jenkins, que é um processo iterativo de várias etapas:

1. Use seus dados, visualizações e conhecimento subjacente para selecionar uma classe de modelo apropriada para seus dados.

2. Estime os parâmetros com base nos seus dados de treinamento.

3. Avalie o desempenho do seu modelo com base nos dados de treinamento e ajuste os parâmetros do modelo para resolver os pontos fracos que você vê no diagnóstico de desempenho.

Vamos trabalhar com um exemplo de ajuste de dados. Primeiro, precisamos de alguns dados. Neste caso, para fins de transparência e conhecimento da resposta certa, geramos nossos dados a partir de um processo ARMA:

##R

> exigir (previsão)

> set.seed(1017)

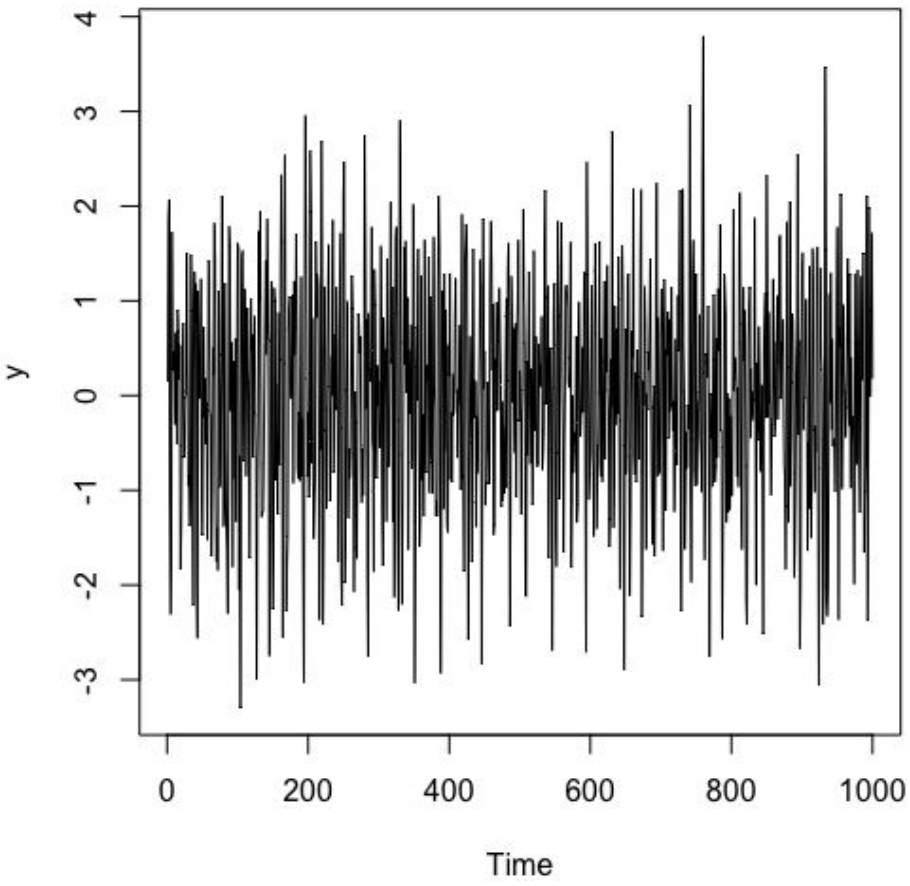
> *## ordem do modelo arima escondida propositalmente*

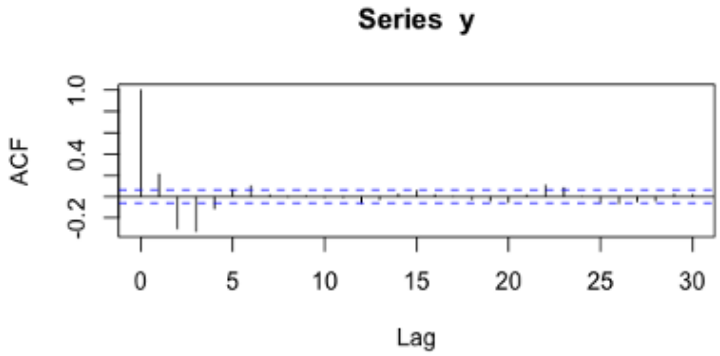
> y = arima.sim(n = 1000, lista(ar = *###, ma = ###))*

Tente não olhar ainda a ordem do modelo criado; vamos nos concentrar em tratar isso como um mistério. Primeiro, devemos representar graficamente a série temporal, como sempre fazemos, para ver se ela parece estacionária (

Figura 6-8). Em seguida, examinamos o ACF e o PACF de *sim*

(Figura 6-9) e compare com a Tabela 6-1.





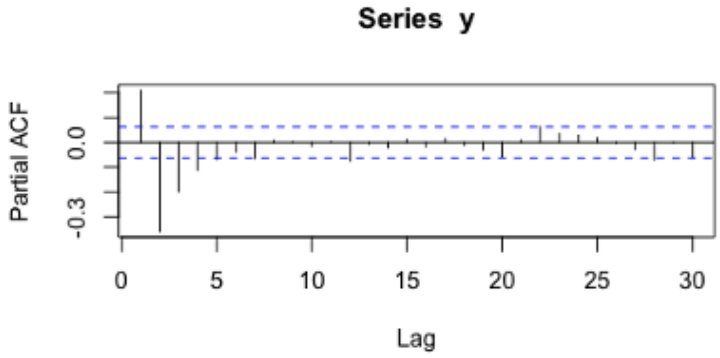
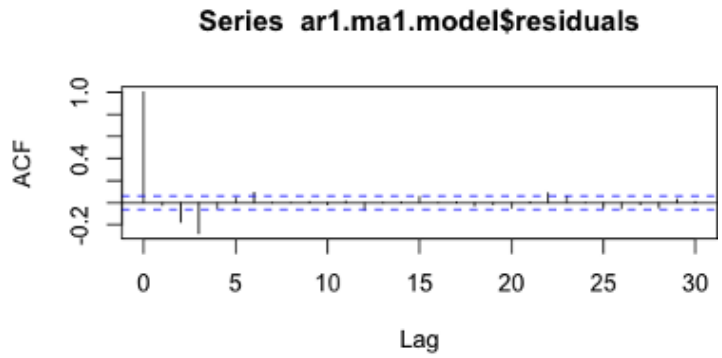
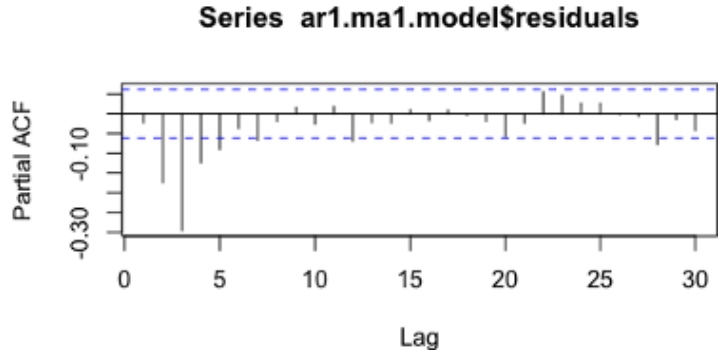


Figura 6-8. Gráfico de nossa série temporal.

Figura 6-9. ACF e PACF de nossa série temporal.





Podemos ver que nem o ACF nem o PACF parecem ter um corte acentuado, sugerindo (ver Tabela 6-1) que este é um processo ARMA. Começamos ajustando um relativamente

modelo ARIMA(1, 0, 1) simples, pois não vemos necessidade de diferenciação e nenhuma evidência

(Figura 6-10):

##R

> ar1.ma1.model = Arima(y, ordem = c(1, 0, 1))

> par(mfrow = c(2,1))

> acf(ar1.ma1.model$resíduos)

> pacf(ar1.ma1.model$resíduos)

Figura 6-10. ACF e PACF dos resíduos de um modelo ARIMA(1, 0, 1).

Os resíduos na Figura 6-10 mostram valores PACF particularmente grandes, sugerindo que

não descreveram completamente o comportamento autorregressivo. Por esse motivo, construímos o modelo adicionando um componente AR de ordem superior, testando um modelo ARIMA(2, 0, 1) no código a seguir e, em seguida, traçando o ACF e o PACF dos resíduos deste modelo mais

modelo complexo (Figura 6-11):

##R

```
> ar2.ma1.model = Arima(y, ordem = c(2, 0, 1))

> plot(y, tipo = 'l')

> linhas(ar2.ma1.model$fitted, col = 2)

> plot(y, ar2.ma1.model$fitted)

> par(mfrow = c(2,1))

> acf(ar2.ma1.model$resíduos)

> pacf(ar2.ma1.model$resíduos)
```

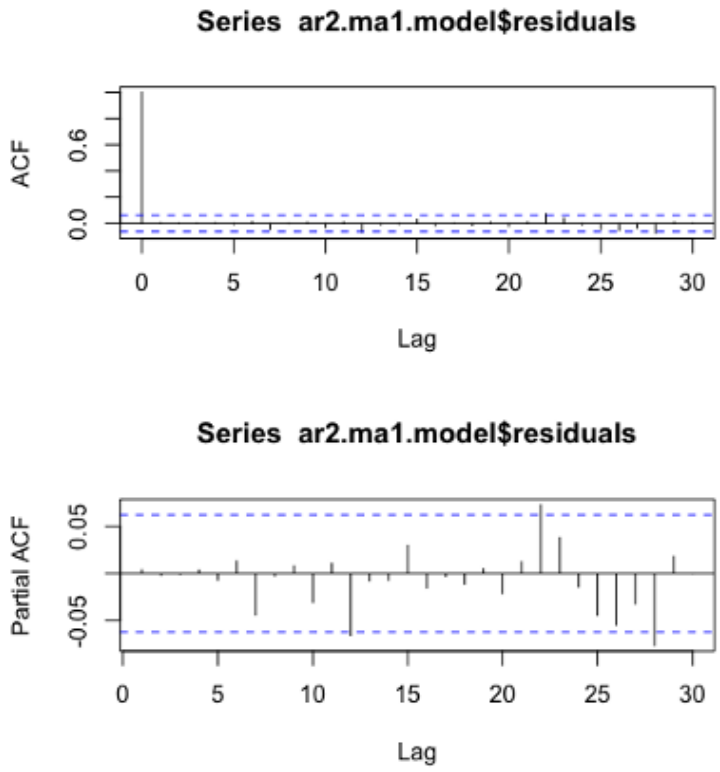


Figura 6-11. ACF e PACF dos resíduos de um modelo ARIMA(2, 0, 1).

Os resíduos na Figura 6-11 não mostram mais valores grandes nem para o ACF nem para o

PACF. Dado o nosso desejo de um modelo parcimonioso e os perigos de sobreajustar um modelo ARIMA, um analista sábio provavelmente pararia por aqui, não vendo qualquer comportamento adicional nos resíduos que precisasse ser ajustado através de um componente autorregressivo, de média móvel ou diferencial. Fica como exercício para o leitor considerar o ajuste de modelos mais complicados. Embora os gráficos não sejam mostrados aqui, tentei com o código a seguir ajustar modelos mais complexos. O que vi foi que eles não melhoraram substancialmente o ajuste do modelo aos dados, nem contribuíram para diminuir os valores de ACF ou PACF mais do que o modelo anterior. Você pode verificar isso por si mesmo:

##R

```
> ar2.ma2.model = Arima(y, ordem = c(2, 0, 2))
```

```
> plot(y, tipo = 'l')

> linhas(ar2.ma2.model$fitted, col = 2)

> plot(y, ar2.ma2.model$equipado)

> par(mfrow = c(2,1))

> acf(ar2.ma2.model$resíduos)

> pacf(ar2.ma2.model$resíduos)

>

> ar2.d1.ma2.model = Arima(y, ordem = c(2, 1, 2))

> plot(y, tipo = 'l')

> linhas (ar2.d1.ma2.model$fitted, col = 2)

> plot(y, ar2.d1.ma2.model$fitted)

> par(mfrow = c(2,1))

> acf(ar2.d1.ma2.model$resíduos)

> pacf(ar2.d1.ma2.model$resíduos)
```


Uma maneira rápida de comparar os modelos é mostrada aqui, onde observo quão bem as previsões de um modelo ajustado se correlacionam com os valores reais:

```
##R

> cor(y, ar1.ma1.model$equipado)

[1] 0,3018926

> cor(y, ar2.ma1.model$equipado)
```

[1] 0,4683598

> cor(y, ar2.ma2.model$equipado)

[1] 0,4684905

> cor (y, ar2.d1.ma2.model$equipado)

[1] 0,4688166

Vemos uma melhoria substancial na transição de um modelo ARIMA(1, 0, 1) para um modelo ARIMA(2, 0, 1) (os dois primeiros modelos), à medida que a correlação vai de 0,3 para 0,47.

Por outro lado, à medida que adicionamos mais complexidade, não vemos melhorias substanciais na correlação. Isto oferece suporte adicional para a conclusão que tiramos anteriormente de que o modelo ARIMA(2, 0, 1) parecia descrever bem o comportamento do modelo e que não havia necessidade de adicionar mais componentes AR ou MA para melhorar o ajuste.

Aliás, podemos ver o desempenho do ajuste abaixo, comparando os coeficientes de ajuste originais (mostrados aqui, embora obscurecidos antes) com os coeficientes de ajuste:

##R

coeficientes originais

> y = arima.sim(n = 1000, lista(ar = c(0,8, -0,4), ma = c(-0,7)))

> ar2.ma1.model$coef

ar1 ar2 ma1 interceptação

0,785028320 -0,462287054 -0,612708282 -0,005227573

Há uma boa correspondência entre os coeficientes ajustados e os coeficientes reais usados para simular os dados.

Ajustar um modelo ARIMA manualmente envolve mais coisas do que demonstramos aqui.

Ao longo de décadas, os profissionais desenvolveram boas regras práticas para identificar problemas, como quando existem demasiados termos de um determinado tipo, quando um modelo foi sobrediferenciado, quando o padrão nos resíduos aponta para determinados problemas, e assim por diante.

Um bom recurso inicial é um guia escrito por um professor da Penn State disponível

on-line .

Existem algumas críticas legítimas à montagem de um processo ARIMA “manualmente”. O ajuste manual pode ser um processo um tanto mal especificado e coloca muita pressão sobre o critério do analista, além de ser demorado e dependente do caminho quanto ao resultado final. É uma boa solução que funcionou bem durante décadas para produzir previsões reais utilizadas no mundo real, mas não é perfeita.

Usando ajuste de modelo automatizado. Hoje em dia podemos fugir de um processo de ajuste iterativo manual em favor de uma seleção automatizada de modelos em alguns casos. Nosso modelo **194 | Capítulo 6: Modelos Estatísticos para Séries Temporais**

a seleção pode ser conduzida de acordo com vários critérios de perda de informação, como o AIC

discutimos brevemente por meio da função auto.arima() no pacote de previsão:

##R

> est = auto.arima(demanda[["Ordens bancárias (2)"]],

stepwise = **FALSE** , *## isso fica mais lento*

mas nos dá uma pesquisa mais completa

máx.p = 3, máx.q = 9)

> est

Série: demanda[["Ordens bancárias (2)"]]

ARIMA(0,0,3) com média diferente de zero

Coeficientes:

ma1 ma2 ma3 significa

-0,0645 -0,1144 -0,4796 79914,783

se 0,1327 0,1150 0,1915 1897,407

sigma ^ 2 estimado como 1,467e + 09: log de probabilidade = -716,71

AIC=1443,42 AICc=1444,53 BIC=1453,89

Neste caso, colocamos uma linha com algum conhecimento prévio de nossa exploração anterior na configuração das entradas. Ou seja, especificamos a ordem máxima do AR

e MA que estávamos preparados para aceitar e, de fato, a seleção do modelo escolheu um modelo muito mais parcimonioso do que especificamos, incluindo nenhum termo AR. No entanto, este modelo ajusta-se bem e não deveríamos bisbilhotar demasiado os dados para tentar vencê-lo, a menos que tenhamos uma razão de princípio

para o fazer. Observe que, de acordo com os critérios da AIC, nosso modelo MA escolhido a dedo na seção anterior tem um desempenho apenas um pouco melhor do que este modelo, mas a diferença não parece significativa quando olhamos para os gráficos.

Deixado como exercício para o leitor é traçar o modelo em relação às previsões, bem como verificar os resíduos deste modelo selecionado automaticamente e confirmar se o modelo utilizado não parece deixar nenhum comportamento residual que precise ser resolvido pela adição de mais termos . O código não é diferente daquele que usamos nas seções anteriores de AR e MA. Isto também é verdade para o caso de fazer previsões e, portanto, é omitido aqui, visto que o modelo ARIMA que selecionamos não é muito diferente do modelo MA discutido e usado para uma previsão anteriormente.

Também podemos estudar rapidamente o desempenho de auto.arima() no modelo que geramos na seção anterior quando ajustamos o modelo manualmente:

##R

> auto.model = auto.arima(y)

> auto.model

Série: y

ARIMA(2,0,1) com média zero

Coeficientes:

ar1 ar2 ma1

0,7847 -0,4622 -0,6123

Métodos Estatísticos Desenvolvidos para Séries Temporais | 195

se 0,0487 0,0285 0,0522

sigma ^ 2 estimado como 1,019: log de probabilidade = -1427,21

AIC=2862,41 AICc=2862,45 BIC=2882,04

Nem usamos os parâmetros opcionais para sugerir ao auto.arima() onde ele deveria iniciar sua busca de modelo, e mesmo assim ele convergiu para a mesma solução que nós. Então, como podemos ver aqui, em alguns casos encontraremos a mesma solução com metodologias diferentes. Conduzimos nossa análise observando o ACF e o PACF dos resíduos de modelos mais simples para construir modelos mais complexos, enquanto auto.arima() é amplamente conduzido por uma pesquisa em grade para minimizar o AIC. É claro que, dado que geramos os dados originais a partir de um processo ARIMA, isto representa um caso mais simples do que muitos dados do mundo real. Neste último caso, nem sempre será verdade que os nossos ajustes manuais e a seleção automatizada do modelo cheguem à mesma conclusão.

Se você pretende tornar auto.arima() ou uma ferramenta de seleção de modelo automatizada semelhante uma parte importante de sua análise, é importante ler a documentação, experimentar dados sintéticos e também ler sobre as experiências de outros analistas com esta função. Existem alguns cenários conhecidos em que a função não terá o desempenho ingenuamente esperado e também existem soluções alternativas conhecidas. No geral, este é um excelente

solução, mas não perfeita.4 Além disso, para uma boa descrição de como auto.arima()

funciona, dê uma olhada no capítulo do livro online sobre este assunto escrito pelo autor do artigo, Professor Rob Hyndman.

Demonstramos duas maneiras distintas de estimar parâmetros: seguindo uma abordagem Box-Jenkins para ajustar o modelo ou usando um ajuste automatizado do pacote de previsão. Na verdade, os profissionais têm opiniões muito fortes sobre este assunto, com alguns defendendo veementemente apenas a favor da iteração manual e outros igualmente ferozmente a favor de

ferramentas de seleção automatizadas. Este continua sendo um debate contínuo na área. No longo prazo, à medida que o big data assume cada vez mais a análise de séries temporais, é provável que a exploração e o ajuste automático de modelos venham a dominar a análise de séries temporais de grandes conjuntos de dados.

Autoregressão vetorial

No mundo real, muitas vezes temos a sorte de ter várias séries temporais em paralelo que estão presumivelmente relacionadas entre si. Já examinamos como limpar e alinhar esses dados e agora podemos aprender como aproveitá-los ao máximo. Podemos fazer isso gerando um modelo AR(p) para o caso de múltiplas variáveis. A beleza de tal modelo é que ele prevê o fato de que as variáveis influenciam umas às outras e, por sua vez, são influenciadas - ou seja, não há y *privilegiado* enquanto todo o resto é designado . e o blog de Rob Hyndman.





como x . Em vez disso, o ajuste é simétrico em relação a todas as variáveis. Observe que a diferenciação pode ser aplicada como em outros modelos anteriores, se as séries não forem estacionárias.

Variáveis Exógenas e Endógenas

Em linguagem estatística, uma vez que empregamos um modelo onde as variáveis

influenciam umas às outras, chamamos essas variáveis *de endógenas* , ou seja

seus valores são explicados pelo que vemos no modelo. Por con-

Trast, variáveis *exógenas* são aquelas que não são explicadas dentro

o modelo - isto é, eles não podem ser explicados por suposições - e

por isso aceitamos os seus valores e não questionamos a dinâmica de como

eles vieram a existir.

Como cada série temporal prediz supostamente todas as outras e também a si mesma, teremos uma equação por variável. Digamos que temos três séries temporais: denotaremos o valor dessas séries temporais no tempo t como $y\,1,\,t$ e $y\,2,\,t$ e $y\,3,\,t$. Então podemos escrever as equações de autoregressão vetorial (VAR) de ordem 2 (fatorando dois intervalos de tempo) como:

$$y\,1,\,t = \phi\,01 + \phi\,11{,}1 \times y\,1,\,t\text{-}1 + \phi\,12{,}1 \times y\,2,\,t\text{-}1 + \phi\,13{,}1 \times y\,3,\,t\text{-}1 + \phi\,11{,}2 \times y\,1,\,t\text{-}2 + \phi\,12{,}2 \times y\,2,\,t\text{-}2 + \phi\,13{,}2 \times y\,3,\,t\text{-}2$$

$$y\,2,\,t = \phi\,02 + \phi\,21{,}1 \times y\,1,\,t\text{-}1 + \phi\,22{,}1 \times y\,2,\,t\text{-}1 + \phi\,23{,}1 \times y\,3,\,t\text{-}1 + \phi\,21{,}2 \times y\,1,\,t\text{-}2 + \phi\,22{,}2 \times y\,2,\,t\text{-}2 + \phi\,23{,}2 \times y\,3,\,t\text{-}2$$

$$y\,1,\,t = \phi\,03 + \phi\,31{,}1 \times y\,1,\,t\text{-}1 + \phi\,32{,}1 \times y\,2,\,t\text{-}1 + \phi\,33{,}1 \times y\,3,\,t\text{-}1 + \phi\,31{,}2 \times y\,1,\,t\text{-}2 + \phi\,32{,}2 \times y\,2,\,t\text{-}2 + \phi\,33{,}2 \times y\,3,\,t\text{-}2$$

Multiplicação da matriz

Como você deve ter notado se estiver familiarizado com álgebra linear,

expressando as relações mostradas nas três equações anteriores

é muito mais simples quando você usa notação de matriz. Em particular,

você pode escrever um VAR de maneira muito semelhante a um AR. No

forma de matriz, as três equações podem ser expressas como:

$$y = \phi 0 + \phi 1 \times yt\text{-}1 + \phi 2 \times yt\text{-}2$$

onde y e $\phi 0$ são matrizes 3 × 1, e as outras matrizes ϕ são matrizes 3 × 3.

Mesmo para um caso simples, você pode ver que o número de parâmetros no modelo cresce muito rapidamente. Por exemplo, se tivermos p lags e N variáveis, podemos ver que a equação preditora para cada variável é $1 + p \times N$ valores. Como temos N valores para prever, isso se traduz em $N + p \times N 2$ variáveis totais, o que significa que o número de variáveis cresce na proporção $O (N 2)$ do número de séries temporais estudadas. Portanto, não devemos lançar séries temporais gratuitamente simplesmente porque temos os dados, mas, em vez disso, reservar esse método para quando realmente esperamos um relacionamento.

Os modelos VAR são mais frequentemente usados em econometria. Às vezes, são criticados porque não têm nenhuma estrutura além da hipótese de que todos os valores influenciam uns aos outros. É precisamente por esta razão que a qualidade do ajuste do modelo pode ser

difícil de avaliar. No entanto, os modelos VAR ainda são úteis – por exemplo, para testar se uma variável causa outra variável. Eles também são úteis para situações em que uma série de variáveis precisam ser previstas e o analista não possui conhecimento de domínio para afirmar qualquer tipo específico de relacionamento. Às vezes, eles também podem ser úteis para determinar quanta variação em uma previsão de um valor é atribuível às suas “causas” subjacentes.

Aqui está uma demonstração rápida. Vejamos as informações subjacentes à procura de OIC e consideremos a utilização de uma segunda coluna para prever as ordens bancárias (2) em vez de apenas os seus próprios dados (observe que também iremos prever essa coluna devido à forma simétrica como as variáveis são tratadas). Vamos considerar o uso de ordens do setor de controle de tráfego. Parece que deveria ser bastante diferente, pelo que poderá fornecer uma fonte de informação bastante independente relativa às ordens passadas do próprio sector fiscal. Podemos também imaginar que cada coluna oferece informações subjacentes sobre o desempenho da economia e se a procura aumentará ou diminuirá no futuro.

Para determinar quais parâmetros usar, usamos o pacote vars, que vem com um método VARselect():

##R

> VARselect(demand[, 11:12, with = **FALSE**], lag.max=4,

+ tipo = "const")

$seleção

AIC(n) HQ(n) SC(n) FPE(n)

3 3 1 3

$critérios

1 2 3 4

AIC(n) 3.975854e+01 3.967373e+01 3.957496e+01 3.968281e+01

QG(n) 3.984267e+01 3.981395e+01 3.977126e+01 3.993521e+01

SC(n) 3.997554e+01 4.003540e+01 4.008130e+01 4.033382e+01

FPE(n) 1,849280e+17 1,700189e+17 1,542863e+17 1,723729e+17

Podemos ver que a função fornece uma variedade de critérios de informação para você escolher. Observe também que indicamos que queríamos ajustar um termo “const” para acomodar uma média diferente de zero. Poderíamos também optar por ajustar um termo de deriva ou ambos ou nenhum, mas o

A opção "const" parece ser a mais adequada para nossos dados. Aqui começaremos observando as três defasagens e veremos como isso acontece:

##R

> est.var <- VAR(demand[, 11:12, with = **FALSE**], p=3, type="const")

>est.var

> par(mfrow = c(2, 1))

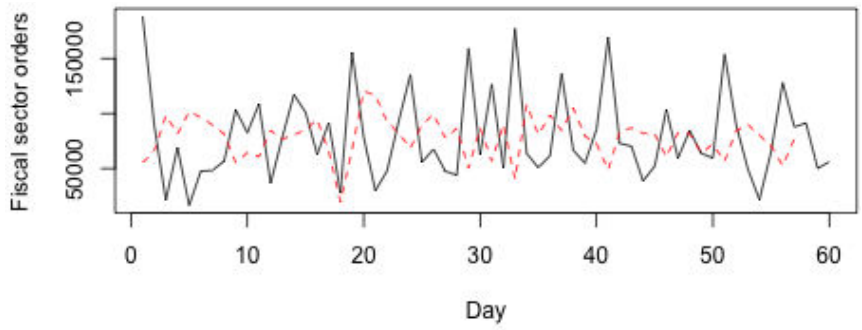
> plot(demand$`Ordens bancárias (2)`, type = "l")

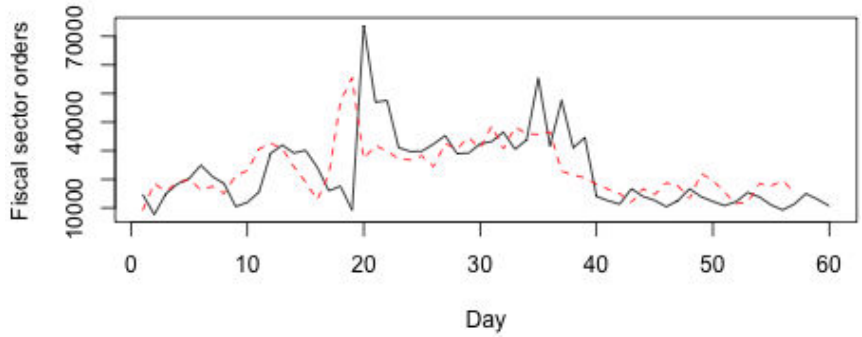
> linhas(equipadas(est.var)[, 1], col = 2)

> plot(demand$`Ordens bancárias (3)`,

> tipo = "l")

> linhas(equipadas(est.var)[, 2], col = 2)






```
> par(mfrow = c(2, 1))

> acf(demand$`Ordens bancárias (2)` - equipado(est.var)[,
1])

> acf(demand$`Ordens bancárias (3)` -

> equipado(est.var)[, 2])
```

Este código produz os gráficos mostrados nas Figuras 6-12 e 6-13.

Figura 6-12. Na parte superior, vemos os valores reais (sólidos) versus os previstos (tracejados) para Ordens bancárias (2), e na parte inferior vemos o mesmo para as Ordens bancárias (3).

Curiosamente, o gráfico superior parece mais uma previsão típica, onde a previsão é um pouco "lenta" na mudança em relação aos dados reais, enquanto no gráfico inferior vemos o a previsão na verdade previu as mudanças com um pouco de antecedência em relação ao momento em que aconteceriam. Esse sugere que as ordens bancárias (2)

“lideram” as ordens bancárias (3), o que significa que as ordens bancárias as ordens (2) são úteis para prever as ordens do controlador de tráfego, mas possivelmente não o inverso, ou pelo menos não a tal ponto.

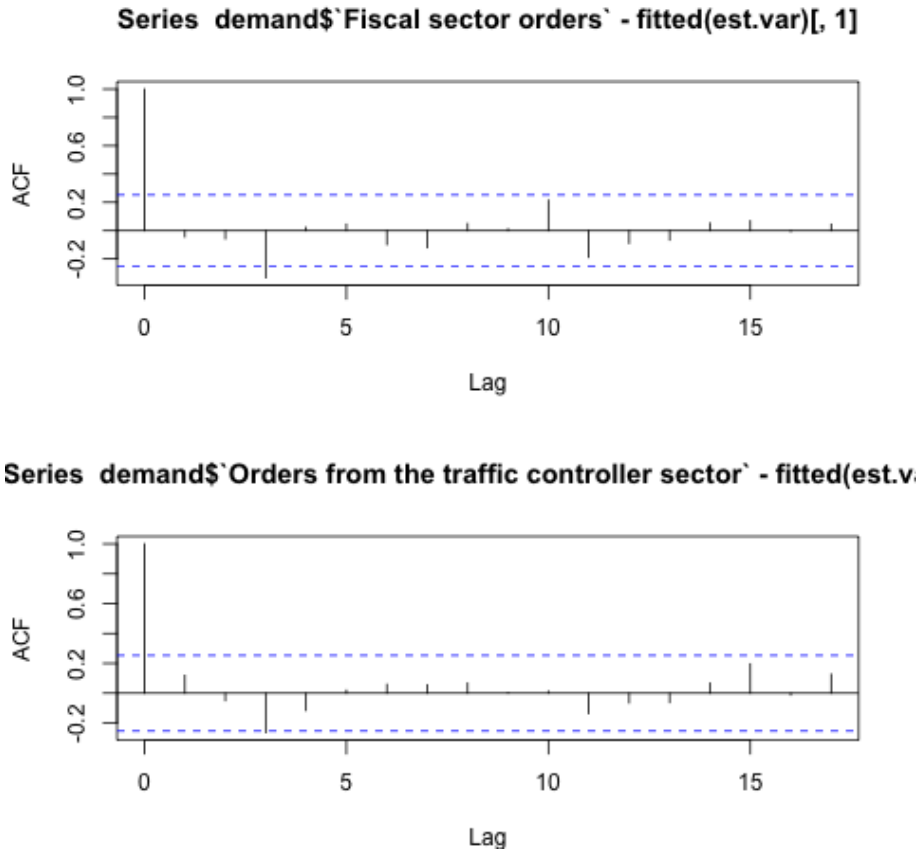


Figura 6-13. Plotamos a função de autocorrelação dos resíduos para cada série temporal.

Observe que para ambas as séries há alguma autocorrelação limítrofe significativa do erros no atraso 3 que podem não ser totalmente contabilizados no modelo.

O ACF não apoia tão claramente a ausência de autocorrelação nos resíduos quanto gostaríamos, portanto, neste caso, também podemos aplicar o *teste Portmanteau* para correlação serial por meio do método serial.test() do pacote vars. Este teste é semelhante aos testes de correlação serial que vimos no caso univariado:

##R

> serial.test(est.var, lags.pt = 8, type="PT.asymptotic")

Teste de Portmanteau (assintótico)

dados: Resíduos do objeto VAR est.var

Qui-quadrado = 20,463, df = 20, valor p = 0,4293

Como o valor *p* é tão alto, não podemos rejeitar a hipótese nula de que não há correlação serial nos resíduos. Isso nos dá mais evidências de que o modelo faz um trabalho aceitável.

Dado que para modelos univariados examinamos uma variedade de modelos, levando às versões ARMA e ARIMA, você pode estar se perguntando se existe um modelo VARIMA.

Sim, existe, mas não é muito utilizado dado que o VAR tem um desempenho relativamente bom e já é bastante complicado. Na prática, para casos de uso industrial e acadêmico, você verá predominantemente o uso de VAR em vez de VARIMA.

Outra classe de modelo relacionada é o modelo CVAR, que é a autorregressão vetorial cointegrada. Isto se refere ao caso em que as séries temporais individuais não são estacionárias, mas uma combinação linear das séries temporais é estacionária sem diferença.

Variações em modelos estatísticos

Existem muitos outros tipos de modelos estatísticos desenvolvidos para dados de séries temporais. Alguns deles expandem o modelo ARIMA, enquanto outros fazem suposições subjacentes sobre a dinâmica temporal diferentes daquelas usadas no modelo ARIMA. Nesta seção discutiremos brevemente alguns dos modelos estatísticos de séries temporais mais comumente usados e conhecidos.

ARIMA sazonal

Um modelo ARIMA sazonal (SARIMA) assume sazonalidade multiplicativa. Por esta razão, um modelo SARIMA pode ser expresso como ARIMA (p, d, q) × (P, D, Q) m . O modelo postula que o próprio comportamento sazonal pode ser pensado como um processo ARIMA, com m especificando o número de passos de tempo por ciclo sazonal. O que é importante nesta factorização é que o modelo reconheça que pontos adjacentes no tempo podem ter influência uns sobre os outros, quer dentro da mesma estação ou em estações diferentes, mas através dos métodos habituais de estarem temporalmente próximos.

Identificar um SARIMA é ainda mais complicado do que identificar um modelo ARIMA precisamente porque é necessário abordar os efeitos sazonais. Felizmente, auto.arima() no pacote de previsões pode lidar com isso da mesma forma que lida com uma tarefa de estimativa ARIMA padrão. Conforme discutido anteriormente, há bons motivos para optar pela seleção automatizada de parâmetros, a menos que você tenha um conhecimento sólido que sugira que você substitua o modelo selecionado determinado por métodos automatizados.

ARCH, GARCH e seus muitos irmãos

ARCH significa “Heteroscedasticidade Condicional Autoregressiva”. Este modelo é usado quase exclusivamente no setor financeiro. Muitas vezes é abordado em cursos de série temporal, por isso vale a pena mencioná-lo aqui. Esta classe de modelos baseia-se na observação de que os preços das ações não têm variância constante e que, de facto, a própria variância parece autoregressiva condicional às variâncias anteriores (por exemplo, os dias de alta volatilidade na bolsa de valores ocorrem em clusters). Nestes modelos, é a variância de um processo que é modelado como um processo autorregressivo e não o processo em si.

Modelos hierárquicos de série temporal

Séries temporais hierárquicas são bastante comuns no mundo real, embora não sejam apresentadas como tal. Você pode facilmente formular situações em que elas podem

• Demanda mensal total em dólares pelos produtos de uma empresa, que pode então ser dividida por número de SKU

• Dados semanais de pesquisas políticas para o eleitorado como um todo e, em seguida, os mesmos dados de pesquisas divididos por dados demográficos (sobrepostos ou não),

como mulher versus homem ou hispânico versus afro-americano

• Contagem total de turistas que chegam diariamente à UE versus contagens de turistas que chegam a cada país membro em particular

Uma maneira conveniente de lidar com séries temporais hierárquicas é por meio do pacote hts do R. Este pacote pode ser usado tanto para visualizar dados hierárquicos de séries temporais quanto para gerar previsões.

As previsões podem ser geradas com diversas metodologias diferentes que historicamente estão disponíveis com este pacote:

• Gerar as previsões de nível mais baixo (as mais individualizadas) e agregá-las para produzir as previsões de nível mais alto.

• Gerar as previsões de nível mais alto e depois gerar previsões de nível inferior com base nas proporções históricas de cada componente do agregado. Esta metodologia tende a ser menos precisa na realização de previsões de baixo nível, embora isto possa ser um pouco mitigado com uma variedade de técnicas para prever como as próprias proporções do agregado mudarão ao longo do tempo.

• É possível tentar obter o melhor de cada metodologia escolhendo uma abordagem “intermediária” onde é gerada uma previsão de nível médio (assumindo que você tenha várias camadas de hierarquia). Isso é então propagado para cima e para baixo em geral para fazer as outras previsões.

Em última análise, o padrão ouro do pacote hts é que, para todos os níveis da hierarquia, podem ser geradas previsões independentes de outros níveis da hierarquia. hts

em seguida, combina essas previsões para garantir consistência em um método conforme Hyndman et

al. .

Muitos dos modelos estatísticos que discutimos neste capítulo seriam, por sua vez, aplicados ao problema de previsão, qualquer que fosse o nível de série temporal estabelecido.

A parte hierárquica da análise tende a envolver esses modelos fundamentais.

Vantagens e desvantagens de

Métodos Estatísticos para Séries Temporais

Quando você está pensando em aplicar um dos modelos estatísticos descritos aqui a um problema de série temporal, pode ser bom começar com um inventário de vantagens e desvantagens. Eles são apresentados nesta seção.

Vantagens

• Estes modelos são simples e transparentes, pelo que podem ser claramente compreendidos em termos dos seus parâmetros.

• Devido às expressões matemáticas simples que definem estes modelos, é possível derivar as suas propriedades de interesse de uma forma estatística rigorosa.

• Você pode aplicar esses modelos a conjuntos de dados relativamente pequenos e ainda assim obter bons resultados.

• Esses modelos simples e modificações relacionadas apresentam desempenho extremamente bom, mesmo em comparação com modelos de aprendizado de máquina muito complicados. Assim, você obtém um bom desempenho sem o perigo de overfitting.

• Metodologias automatizadas bem desenvolvidas para escolher pedidos de seus modelos

e estimar seus parâmetros simplifica a geração dessas previsões.

Desvantagens

• Como esses modelos são bastante simples, eles nem sempre melhoram o desempenho quando recebem grandes conjuntos de dados. Se você estiver trabalhando com conjuntos de dados extremamente grandes, poderá se sair melhor com modelos complexos de aprendizado de máquina e metodologias de redes neurais.

• Estes modelos estatísticos colocam o foco em estimativas pontuais do valor médio de uma distribuição e não na distribuição. É verdade que você pode derivar variações amostrais e similares como uma proxy para a incerteza em suas previsões, mas seu modelo fundamental oferece apenas maneiras limitadas de expressar a incerteza relativa a todas as escolhas que você faz ao selecionar um modelo.

• Por definição, estes modelos não são construídos para lidar com dinâmicas não lineares e farão um mau trabalho ao descrever dados onde as relações não lineares são dominantes.

Vantagens e desvantagens de métodos estatísticos para séries temporais | 203

Mais recursos

• Textos clássicos:

Rob J. Hyndman e George Athanasopoulos, Previsão: Princípios e Prática , 2ª edição. (Melbourne: OTexts, 2018), https://perma.cc/9JNK-K6US.

Este livro prático e altamente acessível oferece uma excelente introdução em R, gratuitamente, na internet, de todos os fundamentos do pré-processamento de dados de séries temporais e do uso desses dados para fazer previsões. A ênfase está em deixar os leitores atualizados e competentes no uso de métodos práticos para séries temporais.

previsão.

Ruey Tsay, Análise de Séries Temporais Financeiras (Hoboken, NJ: John Wiley & Sons, 2001).

Este livro clássico apresenta uma variedade de modelos de séries temporais, incluindo um capítulo muito completo e acessível sobre o desenvolvimento de AR, MA e ARIMA

modelos, com aplicações a preços históricos de ações. Extensos exemplos de R também estão incluídos. Este é um livro intermediário em termos de acessibilidade,

presumindo alguma familiaridade com estatística e outras matemáticas, mas bastante

legível para qualquer pessoa com cálculo do ensino médio e estatística introdutória

curso sob seu cinto.

Robert H. Shumway, Análise de série temporal e suas aplicações (NY, NY: Springer-International, 2017).

Este é outro livro clássico escrito de uma forma um pouco mais teórica e

nível menos acessível. É melhor dar uma olhada no livro de Tsay primeiro, mas este livro inclui discussões adicionais sobre os processos matemáticos de modelos estatísticos de séries temporais. Inclui também uma maior variedade de análises de dados provenientes das ciências, bem como de fontes de dados económicos. Este livro é uma leitura mais desafiadora do que os outros livros listados, mas isso ocorre porque é

denso e informativo e não porque exija um nível muito mais alto de

proficiência matemática (não, embora possa parecer que sim se você ler muito rápido).

• Orientação heurística:

Robert Nau, "Resumo das regras para identificação de modelos ARIMA," notas do curso da Fuqua School of Business, Duke University, https://perma.cc/37BY-9RAZ.

Estas notas fornecem orientação detalhada sobre como escolher os três parâmetros

termos de um modelo ARIMA. Você notará imediatamente uma forte ênfase em

modelos parcimoniosos quando você lê este resumo.

Instituto Nacional de Padrões e Tecnologia (NIST), "Modelos Box-Jenkins", em

Manual eletrônico de métodos estatísticos do NIST/SEMATECH (Washington, DC: NIST,

Departamento de Comércio dos EUA, 2003), https://perma.cc/3XSC-Y7AG.

Esta seção do manual on-line do NIST oferece etapas concretas para implementar

mento do método Box-Jenkins, uma metodologia comumente usada para ARIMA

seleção de parâmetros. É também um bom exemplo dos recursos que o NIST produz para análise estatística de séries temporais de forma mais geral, como parte do manual mais amplo e meticulosamente compilado de melhores práticas para análise de séries temporais.

Rob J. Hyndman, "A confusão do modelo ARIMAX", Blog da Hyndsight, 4 de outubro de 2010, https://perma.cc/4W44-RQZB.

Esta postagem concisa no blog do famoso previsor Rob Hyndman descreve incor-

porando covariáveis em modelos ARIMA, uma metodologia alternativa ao VAR

para lidar com séries temporais multivariadas.

Richard Hardy, "Validação Cruzada: Regularização para Modelos ARIMA," pergunta publicado em Cross Validated, StackExchange, 13 de maio de 2015, https://perma.cc/G8NQ-RCCU.

No cenário de uma ordem elevada de termos autorregressivos ou de muitos inputs no caso do VAR, pode fazer sentido regularizar, e em muitos setores isso

leva a melhorias substanciais de desempenho. Este post sobre a Cruz Vali-

O site de perguntas e respostas datado fornece algumas discussões preliminares, bem como links para uma implementação computacional e pesquisas acadêmicas relacionadas.

CAPÍTULO 7

Modelos de espaço de estados para séries temporais

Os modelos de espaço de estados são semelhantes aos modelos estatísticos que examinamos no capítulo anterior, mas com uma motivação mais "real". Eles abordam preocupações que surgem em problemas de engenharia do mundo real, tais como como levar em consideração erros de medição ao fazer estimativas e como injetar conhecimento prévio ou crença nas estimativas.

Os modelos de espaço de estados postulam um mundo em que o verdadeiro estado não pode ser medido diretamente, mas apenas inferido a partir do que pode ser medido. Os modelos de espaço de estados também dependem da especificação da dinâmica de um sistema, tal como a forma como o verdadeiro estado do mundo evolui ao longo do tempo, tanto devido à dinâmica interna como às forças externas que são aplicadas a um sistema.

Embora você possa não ter visto modelos de espaço de estados antes em um contexto matemático, provavelmente já os usou em sua vida cotidiana. Por exemplo, imagine que você vê um motorista ziguezagueando no trânsito. Você tenta determinar para onde o motorista está indo e como pode se defender melhor. Se o motorista estiver embriagado, você consideraria chamar a polícia, ao passo que, se o motorista estivesse temporariamente distraído por um motivo que não se repetiria, você provavelmente cuidaria da sua vida. Nos próximos segundos ou minutos você atualizaria seu próprio modelo de espaço de estado desse driver antes de decidir o que fazer.

Um exemplo clássico de onde você usaria um modelo de espaço de estado é um foguete lançado ao espaço. Conhecemos as leis de Newton, por isso podemos escrever as regras para a dinâmica do sistema e como deve ser o movimento ao longo do tempo. Sabemos também que o nosso GPS ou sensores ou o que quer que utilizemos para rastrear a localização terá algum erro de medição que podemos quantificar e tentar levar em conta na incerteza sobre os nossos cálculos. Finalmente, sabemos que não podemos explicar todas as forças do mundo que actuam num determinado foguetão, pois existem muitas incógnitas no sistema, por isso queremos um processo que seja robusto a outras fontes desconhecidas de ruído, talvez o vento solar
.

ou vento terrestre ou ambos. Acontece que os avanços estatísticos e de engenharia nos últimos 50 anos revelaram-se bastante úteis para lidar com este tipo de situações.

Duas tendências históricas diferentes levaram ao desenvolvimento de modelos de espaço de estados e ao interesse nos tipos de problemas que eles abordam. Primeiro, por volta de meados do século XX, entramos na

era da automação mecânica. Havia foguetes e naves espaciais no céu, sistemas de navegação para submarinos e todo tipo de outras invenções automatizadas que exigiam a estimativa de um estado do sistema que não podia ser medido. À medida que os pesquisadores pensavam em como estimar o estado do sistema, eles começaram a desenvolver métodos de espaço de estados, principalmente para eliminar a ambiguidade dos erros de medição de outros tipos de incerteza no sistema. Isso levou aos primeiros usos de métodos de espaço de estados.

Também durante este período, a tecnologia de manutenção de registos e a computação associada também se desenvolveram. Isso levou à criação de conjuntos de dados muito maiores para séries temporais, incluindo conjuntos de dados de séries temporais muito mais longos ou mais detalhados. À medida que mais dados de séries temporais se tornassem disponíveis, métodos mais intensivos em dados poderiam ser desenvolvidos para eles, em conjunto com o novo pensamento sobre modelagem de espaço de estados.

Neste capítulo estudaremos estes métodos de espaço de estados comumente usados:

• O filtro de Kalman aplicado a um modelo linear gaussiano

• Modelos ocultos de Markov

• Séries temporais estruturais bayesianas

Em cada um destes casos, a utilização de tais modelos é bastante acessível e bem implementada. Para cada modelo, desenvolveremos alguma intuição matemática e discutiremos que tipo de dados são apropriados para uso com o método. Finalmente, veremos exemplos de código para cada método.

Em cada caso, distinguiremos entre o que observamos e o estado que produziu as nossas observações. Ao estimar o estado subjacente com base em observações, podemos dividir o nosso trabalho em diferentes fases ou categorias:

Filtragem

Usando a medição no tempo *t* para atualizar nossa estimativa do estado no tempo *t Previsão*

Usando a medição no tempo *t* – 1 para gerar uma previsão para o estado esperado no tempo *t* (permitindo-nos inferir também a medição esperada no tempo *t) Suavização*

Usando a medição durante um intervalo de tempo que inclui *t* , antes e depois dela, para estimar qual era o verdadeiro estado no momento *t*

A mecânica destas operações será muitas vezes semelhante, mas as distinções são importantes. A filtragem é uma forma de decidir como pesar as informações mais recentes. **208 | Capítulo 7: Modelos de Espaço de Estados para Séries Temporais**

contra informações passadas na atualização de nossa estimativa de estado. Previsão é a previsão do estado futuro sem qualquer informação sobre o futuro. Suavização é o uso de informações futuras e passadas para fazer a melhor estimativa do estado em um determinado momento.

Modelos de espaço de estado: vantagens e desvantagens

Os modelos de espaço de estados podem ser usados para aplicações determinísticas e estocásticas e podem ser aplicados tanto a dados amostrados continuamente quanto a dados amostrados discretamente. [1]

Isso por si só já lhe dá uma ideia de sua utilidade e tremenda flexibilidade. A flexibilidade dos modelos de espaço de estados é o que impulsiona as vantagens e desvantagens desta classe de modelos.

Existem muitos pontos fortes de um modelo de espaço de estados. Um modelo de espaço de estados permite modelar o que muitas vezes é mais interessante em uma série temporal: o processo dinâmico e os estados que produzem os dados ruidosos que estão sendo analisados, em vez de apenas os dados ruidosos em si. Com um modelo de espaço de estados, injetamos um modelo de *causalidade* no processo de modelagem para explicar o que está gerando

um processo em primeiro lugar. Isto é útil para casos em que temos teorias fortes ou conhecimentos fiáveis sobre como funciona um sistema, e onde queremos que o nosso modelo nos ajude a descobrir mais detalhes sobre a dinâmica geral com os quais já estamos familiarizados.

Um modelo de espaço de estados permite alterar coeficientes e parâmetros ao longo do tempo, o que significa que permite alterar o comportamento ao longo do tempo. Não impusemos uma condição de estacionariedade aos nossos dados ao usar modelos de espaço de estados. Isto é bastante diferente dos modelos que examinamos no Capítulo 6, nos quais um processo estável é assumido e modificado.

eledos com apenas um conjunto de coeficientes em vez de coeficientes variantes no tempo.

No entanto, existem também algumas desvantagens num modelo de espaço de estados e, por vezes, a força do modelo de espaço de estados é também a sua fraqueza:

• Como os modelos de espaço de estados são muito flexíveis, há muitos parâmetros a serem definidos e muitas formas que um modelo de espaço de estados pode assumir. Isto significa que as propriedades de um determinado modelo de espaço de estados muitas vezes não foram bem estudadas. Ao formular um modelo de espaço de estados adaptado aos seus dados de série temporal, raramente você terá livros de estatística ou artigos de pesquisa acadêmica nos quais outras pessoas também estudaram o modelo. Isso deixa você em um território menos certo no que diz respeito à compreensão do desempenho do seu modelo ou onde você pode ter cometido erros.

1 Neste livro analisamos dados de amostragem discreta, o que é mais comum em aplicações do mundo real.

• Os modelos de espaço de estados podem ser muito exigentes do ponto de vista computacional porque existem muitos parâmetros. Além disso, o número muito elevado de

parâmetros para alguns tipos de modelos de espaço de estados pode deixá-lo vulnerável ao overfitting, principalmente se você não tiver muitos dados.

O Filtro de Kalman

O filtro de Kalman é um método bem desenvolvido e amplamente utilizado para incorporar novas informações de uma série temporal e incorporá-las de maneira inteligente com informações previamente conhecidas para estimar um estado subjacente. Um dos primeiros usos do filtro Kalman foi na missão Apollo 11 – o filtro foi escolhido quando a NASA

os engenheiros perceberam que os elementos de computação integrados não permitiriam outras técnicas de estimativa de posição com mais uso intensivo de memória. Como você verá nesta seção, os benefícios do filtro de Kalman são que ele é relativamente fácil de calcular e não requer armazenamento de dados passados para fazer estimativas presentes ou previsões futuras.

Visão geral

A matemática do filtro de Kalman pode ser intimidante para um recém-chegado, não

porque é especialmente difícil, mas porque há um bom número de quantidades para acompanhar e é um processo iterativo e um tanto circular com muitas quantidades relacionadas. Por esta razão, não derivaremos as equações do filtro de Kalman aqui, mas em vez disso faremos uma visão geral de alto nível dessas equações para ter uma noção de como elas

trabalho.2

Começamos com um modelo linear gaussiano, postulando que nosso estado e nossas observações têm a seguinte dinâmica:

$$xt = F \times xt–1 + B \times ut + peso$$

$$yt = A \times xt + vt$$

Ou seja, o estado no tempo t é uma função do estado na etapa de tempo anterior ($F \times xt$ –1), um termo de força externa ($B \times ut$) e um termo estocástico (wt). Da mesma forma, a medição no tempo t é uma função do estado no tempo t e de um termo de erro estocástico, erro de medição.

Vamos imaginar que xt é a posição real de uma nave espacial, enquanto yt é a posição que medimos com um sensor. vt é o erro de medição em nosso dispositivo sensor (ou intervalo de 2. Eu também recomendaria a leitura de muitas explicações alternativas simples do filtro de Kalman em Matemática

StackExchange .

210 | Capítulo 7: Modelos de Espaço de Estados para Séries Temporais

dispositivos). A equação fundamental aplicável ao filtro de Kalman é esta equação que ilustra como atualizar nossa estimativa com novas informações para o tempo t : $x\hat{}t = Kt \times yt + (1 - Kt) \times x\hat{}t - 1$

O que vemos aqui é uma etapa de filtragem – isto é, uma decisão sobre como usar a medição no tempo t para atualizar nossa estimativa de estado no tempo t . Lembre-se, postulamos uma situação em que podemos observar apenas yt e fazer inferências sobre o estado, mas nunca podemos ter certeza do estado exato. Vemos acima que a quantidade Kt estabelece um equilíbrio em nossa estimativa entre informações antigas ($x\hat{}t$ –1) e informações novas (yt).

Para entrar na mecânica mais detalhada, precisamos definir alguns termos. Usamos Pt para denotar nossa estimativa da covariância do nosso estado (pode ser um escalar ou uma matriz, dependendo se o estado é univariado ou multivariado, sendo este último mais comum). P – t é a estimativa para t antes que nossa medida no tempo t seja levada em consideração.

Também usamos R para denotar a variância do erro de medição, que é a variância de vt , que novamente pode ser uma matriz escalar ou de covariância dependendo da dimensionalidade das medições. R é geralmente bem

definido para um sistema, pois descreve propriedades físicas bem conhecidas de um sensor ou dispositivo de medição específico. O valor apropriado para *wt* , *Q* , é menos bem definido e sujeito a ajustes durante o processo de modelagem.

Então começamos com um processo tal que temos valores conhecidos ou estimados para *x* e *P* no tempo 0. Então, avançando para tempos após o tempo 0, seguimos um processo iterativo de uma fase de previsão e atualização, com a fase de previsão vindo primeiro , seguido pela fase de atualização/filtragem e assim por diante:

Predição:

$x\hat{} - t = F \times x\hat{} t{-}1 + B \times ut$

$P{-}t = F \times Pt{-}1 \times FT + Q$

Filtragem:

$x\hat{} t = x\hat{} - t + Kt \times (yt - A \times x\hat{} -t)$

$Pt = (I - Kt \times A) \times Pt$

onde *Kt* , o ganho de Kalman, é:

$Kt = P - t \times AT \times (A \times Pt \times AT + R) - 1$

Você verá muitas visualizações desse processo recursivo. Alguns dividirão isso em muitas etapas, talvez até quatro ou cinco, mas a maneira mais simples de pensar sobre isso é que existem cálculos realizados para prever os valores no tempo *t* , sem uma medição para *yt* (a previsão), e então existem as etapas realizadas no tempo *t* , após a medição *yt* ser conhecida (a filtragem).

Para começar, você precisa dos seguintes valores:

• Estimativas para *R* e *Q* —suas matrizes de covariância para erro de medição (fácil de saber) e estocasticidade de

estado (geralmente estimada), respectivamente

• Estimativas ou valores conhecidos para o seu estado no momento 0, $x\hat{}\ 0$ (estimado com base em $y\ 0$)

• Conhecimento avançado de quais forças estão planejadas para serem aplicadas no momento t e como elas impactam o estado - ou seja, a matriz B e o valor ut

• Conhecimento da dinâmica do sistema que determina a transição de estado de um passo de tempo para outro, nomeadamente F

• Conhecimento de como a medição depende do estado, nomeadamente A

Há muitas maneiras de derivar as equações do filtro de Kalman, inclusive de uma perspectiva probabilística em valores esperados, um problema de minimização de mínimos quadrados ou como um problema de estimativa de máxima verossimilhança. Derivações do filtro de Kalman estão amplamente disponíveis, e o leitor interessado pode prosseguir neste tópico com algumas pesquisas na Internet.

Código para o Filtro Kalman

Consideramos um caso de uso clássico: tentar rastrear um objeto sujeito à mecânica de Newton com sensores propensos a erros. Geramos uma série temporal baseada nas leis do movimento de Newton, ou seja, que a posição de um objeto é função de sua velocidade e aceleração. Imaginamos fazer medições discretas mesmo que o movimento subjacente seja contínuo. Primeiro determinamos uma série de acelerações e depois assumimos que tanto a posição quanto a velocidade começam em 0. Embora não seja fisicamente realista, assumimos mudanças instantâneas de aceleração no início de cada intervalo de tempo e um valor de aceleração constante:

##R

foguete dará 100 passos de tempo

```
ts.comprimento <- 100

## a aceleração impulsionará o movimento

a <- rep(0,5, ts.comprimento)

## posição e velocidade começam em 0

x <- rep(0, ts.comprimento)

v <- rep(0, ts.comprimento)

for (ts em 2:ts.length) {
```



```
x[ts] <- v[ts - 1] * 2 + x[ts - 1] + 1/2 * a[ts-1] ^ 2

x[ts] <- x[ts] + rnorm(1, sd = 20) ## componente estocástico

v[ts] <- v[ts - 1] + 2 * a[ts-1]

}
```

Se você não conhece ou não se lembra das leis do movimento de Newton, você pode querer se familiarizar com elas, embora você também possa considerá-las pelo valor nominal para os propósitos atuais (o cálculo de x[ts] e v[ts]).

Um rápido exercício de plotagem nos mostra o movimento que criamos com a aceleração

estruturado conforme mostrado (veja a Figura 7-1):

```
##R

par(mfrow = c(3, 1))

plot(x, main = "Posição", type = 'l')
```

```
plot(v, main = "Velocidade", type = 'l')

plot(aceleração, main = "Aceleração", type = 'l')
```

Postulamos que essas variáveis representariam uma descrição completa do estado, mas que os únicos dados disponíveis para nós são a posição do objeto e que esses dados estão disponíveis apenas a partir de um sensor ruidoso. Este sensor é x no código a seguir, e traçamos como o

o valor medido está relacionado à posição real na Figura 7-2 :

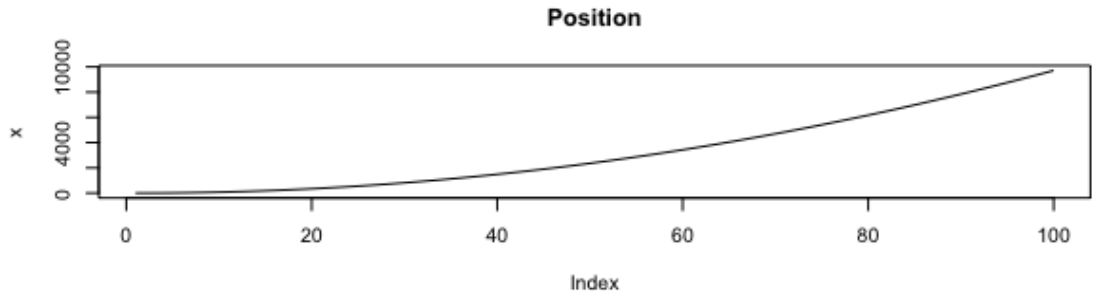
```
##R

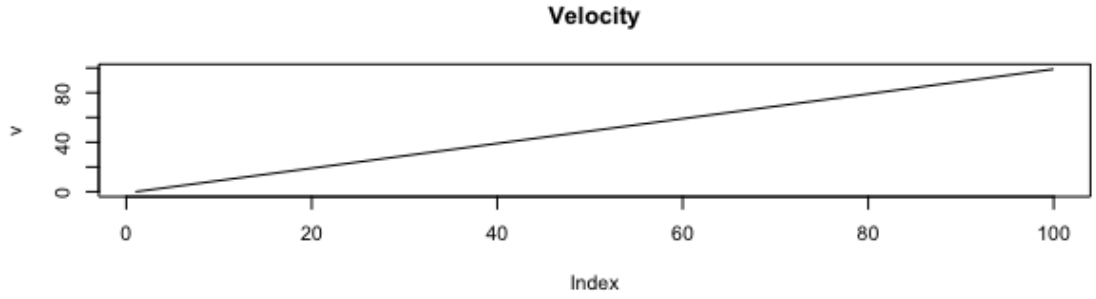
z <- x + rnorm(ts.comprimento, sd = 300)

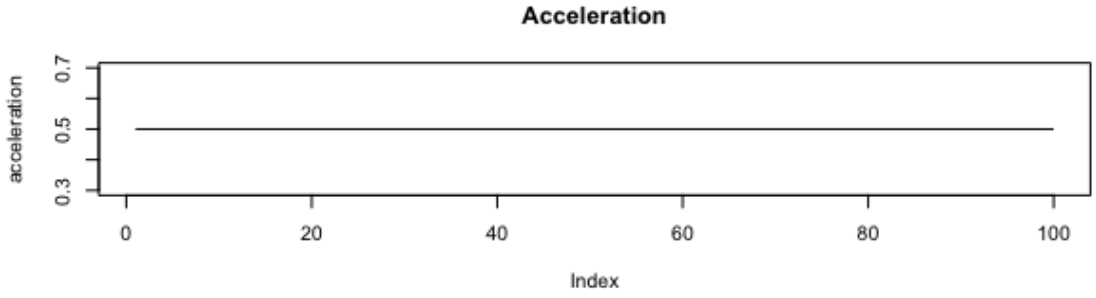
gráfico (x, ylim = intervalo (c (x, z)))

linhas (z)
```

Na Figura 7-2 vemos uma aceleração constante (gráfico inferior) conduzindo uma velocidade linearmente crescente (gráfico do meio) para produzir uma forma parabólica no deslocamento (gráfico superior). Se essas relações mecânicas não forem familiares, você pode considerá-las pelo valor nominal ou procurar uma rápida revisão da mecânica básica em um livro introdutório à física.


Position
x
0
4000
10000
0
20
40
60
80
100
Index


Velocity
v
0
40
80
0
20
40
60
80
100
Index


Acceleration
acceleration
0.3
0.5
0.7
0
20
40
60
80
100
Index

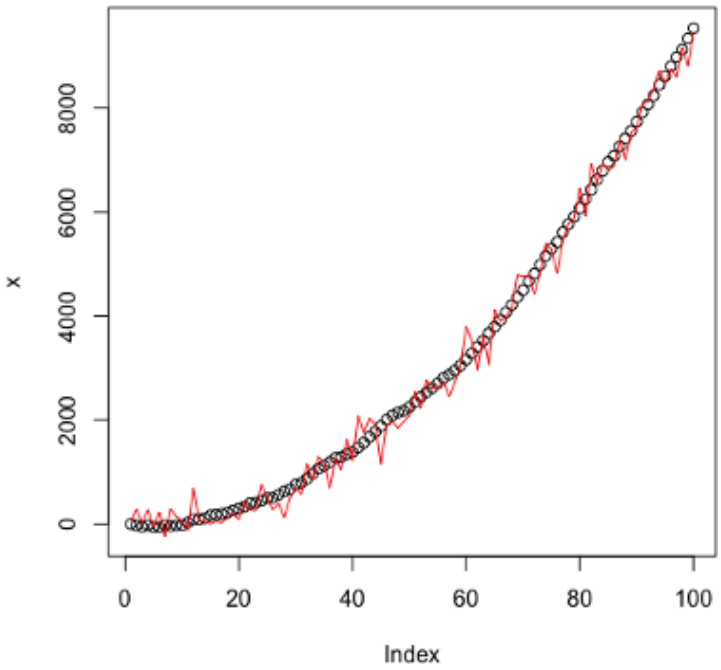


Figura 7-1. A posição, velocidade e aceleração do nosso foguete.

Figura 7-2. A posição verdadeira (pontos) versus nossa medição ruidosa (linha). Notar que a posição x não reflete uma parábola perfeita devido ao ruído que colocamos no estado equação de transição.

Agora aplicamos um filtro de Kalman. Primeiro, codificamos uma função geral que reflete nossa discussão e derivação anteriormente nesta seção:

##R

kalman.motion <- função (z, Q, R, A, H) {

dimState = dim(Q)[1]

xhatminus <- array(rep(0, ts.length * dimState),

```
c(ts.comprimento, dimState))
xhat <- array(rep(0, ts.length * dimState),
c(ts.comprimento, dimState))

Pmenos <- array(rep(0, ts.length * dimState * dimState),
c(ts.length, dimState, dimState))
P <- array(rep(0, ts.length * dimState * dimState),
c(ts.length, dimState, dimState))

K <- array(rep(0, ts.length * dimState),
c(ts.length, dimState)) # ganho de Kalman

# estimativas iniciais = começando em 0 para todas as métricas
xhat[1, ] <- rep(0, dimState)
P[1, , ] <-diag(dimState)

para (k em 2:ts.comprimento) {
# atualização de horário
xhatminus[k, ] <- Uma matriz %*%(xhat[k-1, ])
Pmenos[k, , ] <- A %*% P[k-1, , ] %*% t(A) + Q
```

```
K[k, ] <- Pmenos[k, , ] %*% H %*%

resolver (t(H) %*% Pmenos[k, , ] %*% H + R )

xhat[k, ] <- xhatminus[k, ] + K[k, ] %*%

(z[k]- t(H) %*% xhatminus[k, ])

P[k, , ] <- (diag(dimState)-K[k,] %*% t(H)) %*% Pminus[k, , ]

}


## retornamos a previsão e o valor suavizado

return(lista(xhat = xhat, xhatminus = xhatminus))

}
```

Aplicamos esta função, tornando mensurável apenas a posição do foguete (portanto, não a aceleração ou velocidade):

```
##R

## parâmetros de ruído

R <- 10^2 ## variação de medição - este valor deve ser definido

## de acordo com os limites físicos conhecidos da ferramenta de medição

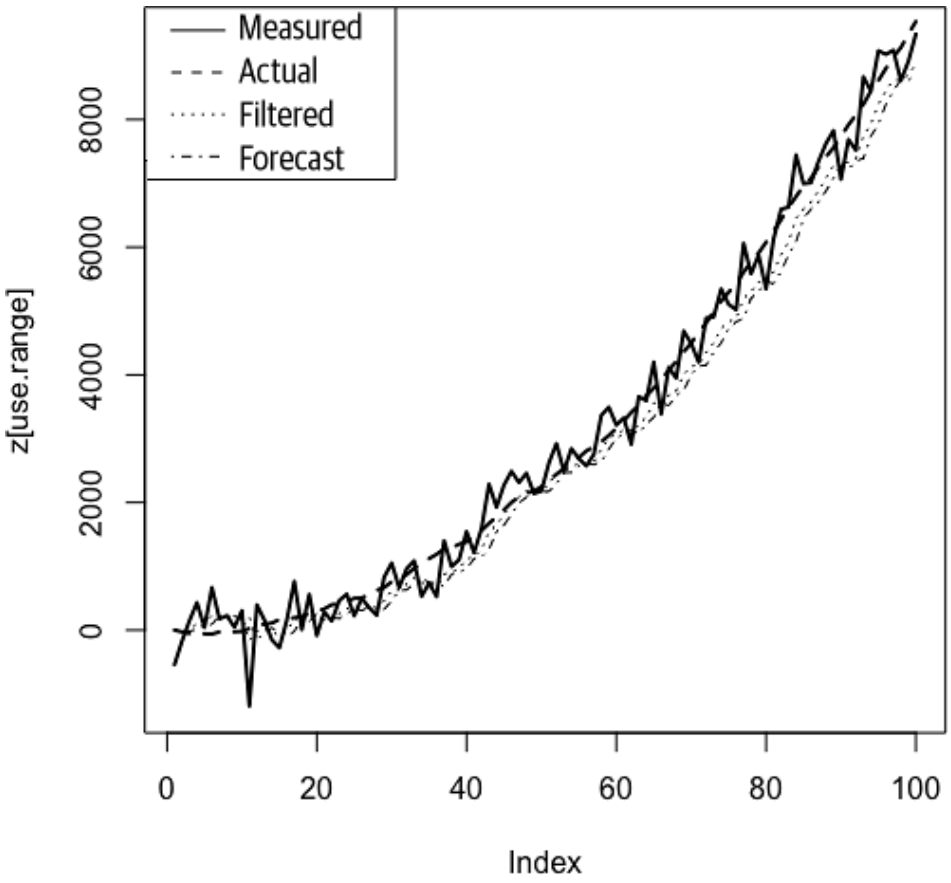
## definimos isso de forma consistente com o ruído que adicionamos a x

## para produzir x na geração de dados acima

Q <- 10 ## variação do processo - geralmente considerada como hiperparâmetro

## a ser ajustado para maximizar o desempenho
```

parâmetros dinâmicos



A <- array(1) *## x_t = A * x_t-1 (como o x anterior afeta o x posterior)*

H <- matriz(1) *## y_t = H * x_t (traduzindo estado em medição)*

executa os dados através do método de filtragem Kalman

xhat <- kalman.motion(z, diag(1) * Q, R, A, H)[[1]]

Traçamos a posição verdadeira, a posição medida e a posição estimada em

Figura 7-3 .

Figura 7-3. Muitas quantidades relacionadas: a posição medida, a posição real/verdadeira, a estimativa filtrada da posição (ou seja, a melhor estimativa para a posição no momento t incorporando a medição no momento t) e a previsão da posição (ou seja, a melhor estimativa estimativa para a posição no tempo t incorporando apenas a dinâmica conhecida do sistema mais medições até o tempo t – 1 e não incluindo o tempo t).

O filtro de Kalman elimina grande parte do ruído do erro de medição. O quanto isso acontecerá dependerá do nosso valor para R, o parâmetro de ruído de medição, que reflete quanto o filtro deve pesar o valor mais recente em relação aos valores anteriores. Como podemos ver, o filtro faz um trabalho satisfatório de previsão dos dados. Em particular, note que não existe um desfasamento entre os dados previstos e os dados reais, o que sugeriria que o método está apenas a prever o valor actual com base no último valor.

Aqui trabalhamos com um exemplo simples de filtro de Kalman. O filtro de Kalman é amplamente estudado porque é muito útil em uma variedade de aplicações, particularmente naquelas onde a dinâmica interna do sistema é muito bem compreendida. Isso o torna um

ferramenta ideal em casos como o exemplo do foguete simples, onde entendemos a dinâmica que impulsiona o sistema.

Observe que neste exemplo simples, todo o poder e utilidade do filtro de Kalman não são totalmente realizados. É particularmente útil quando temos vários tipos de medições, por exemplo, medindo quantidades diferentes ou medindo a mesma coisa simultaneamente com vários dispositivos. Além disso, existem muitas extensões para o filtro de Kalman que valem a pena estudar se esta for uma área promissora para o seu domínio de interesse.

Um dos grandes benefícios do filtro de Kalman, como ilustramos, é que ele é recursivo. Isto significa que não é necessário examinar todos os pontos de dados em cada iteração do processo. Pelo contrário, em cada passo de

tempo, toda a informação dos passos de tempo anteriores já está incorporada da melhor maneira possível nos poucos parâmetros estimados, nomeadamente o estado mais recente e a estimativa de covariância. A beleza deste método é que podemos atualizar de forma inteligente apenas usando essas “estatísticas resumidas” como medidas, e já sabemos como pesá-las de forma inteligente em relação aos dados mais recentes. Isso torna o filtro de Kalman muito útil para aplicações do mundo real onde o tempo ou os recursos computacionais são escassos. Em muitos casos, isto também coincide com a dinâmica dos sistemas reais, na medida em que os processos são relativamente Markovianos (sem memória, exceto o estado imediatamente anterior) e uma função de um estado subjacente que só pode ser medido com algum erro.

Existem muitas extensões úteis para o filtro de Kalman que não discutimos aqui. Um dos principais usos do filtro de Kalman é adaptá-lo à *suavização* , o que significa usar dados antes e depois do tempo t para fazer a melhor estimativa do estado verdadeiro no tempo t . A matemática e o código são semelhantes ao que já foi apresentado.

Também semelhante é o Filtro de Kalman Estendido (EKF), que adapta o filtro de Kalman a dados com dinâmica não linear. Isso também é relativamente simples de implementar e está amplamente disponível em uma variedade de pacotes R e Python.

O filtro de Kalman é $O(T)$ em relação ao comprimento da série temporal, mas $O(d2)$ em relação a d , a dimensão do estado. Por esse motivo, é importante não especificar demais o estado, quando uma especificação mais simplificada servirá igualmente bem. No entanto, é essa linearidade em relação ao comprimento da série temporal que torna o filtro de Kalman comumente usado em cenários reais de produção e muito mais popular do que outros filtros desenvolvidos para fins de modelagem em espaço de estados de séries temporais.

O Filtro de Kalman | 217

Modelos ocultos de Markov

Os modelos ocultos de Markov (HMMs) são uma forma particularmente útil e interessante de modelar uma série temporal porque é um caso raro de aprendizagem não supervisionada na análise de série temporal, o que significa que não há uma resposta correta rotulada para treinar. Um HMM é motivado por uma intuição semelhante à que usamos ao experimentar o filtro de Kalman anteriormente neste capítulo, ou seja, a ideia de que as variáveis que somos capazes de observar podem não ser as variáveis mais descritivas do sistema. Tal como acontece com o filtro de Kalman aplicado a um modelo linear gaussiano, postulamos a ideia de que o processo tem um estado, e as nossas observações fornecem informações sobre esse estado. E novamente, como antes, precisamos de ter alguma opinião sobre como as variáveis de estado influenciam o que podemos observar. No caso de um HMM, o que postulamos é que o processo é não linear caracterizado por saltos entre estados discretos.

Como funciona o modelo

Um HMM postula um sistema no qual existem estados que não são diretamente observáveis.

O sistema é um processo de Markov, o que significa que é “sem memória”, no sentido de que as probabilidades de eventos futuros podem ser totalmente calculadas, dado apenas o estado atual do sistema. Ou seja, conhecer o estado atual do sistema e os seus estados anteriores não é mais útil do que simplesmente conhecer o estado atual do sistema.

Os processos de Markov são frequentemente descritos em termos de matrizes. Por exemplo, suponha que houvesse um sistema que flutuasse entre o estado A e o estado B. Quando em qualquer um dos estados, o sistema tinha maior probabilidade estatisticamente de permanecer no mesmo estado do que de mudar para outro estado em qualquer intervalo de tempo distinto. Um desses sistemas seria descrito pela seguinte matriz:

A

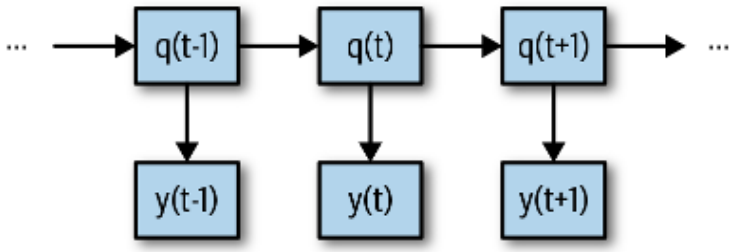
B

A 0,7 0,3

B 0,2 0,8

Vamos imaginar que nosso sistema esteja no estado A, ou seja, (1, 0). (O estado B seria (0, 1).) Nesse caso, a probabilidade de o sistema permanecer no estado A é 0,7, enquanto a probabilidade de o sistema virar é 0,3. Não precisamos saber em que estados o sistema se encontrava antes do seu momento mais recente. Isto é o que significa ser um processo de Markov.

Um modelo oculto de Markov representa o mesmo tipo de sistema, exceto que não somos capazes de inferir diretamente o estado do sistema a partir de nossas observações. Em vez disso, as nossas observações oferecem pistas sobre o estado do sistema (ver Figura 7-4).



Time series with four underlying states

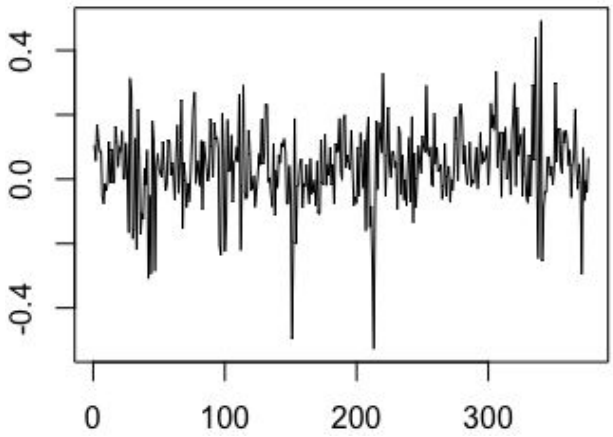


Figura 7-4. O processo para um HMM. Os estados reais do sistema em um determinado momento são representado por x(t), enquanto os dados observáveis em um determinado momento são representados pelo você(t). Apenas x(t) é relevante para pensar em y(t). Em outras palavras, x(t − 1) não oferece nenhuma informação adicional para prever y(t) se conhecermos x(t). Da mesma forma, apenas x(t) tem alguma influência para prever x(t + 1) e não há informações adicionais de saber x(t – 1).

Este é o aspecto Markov do sistema.

Observe que, para aplicações realistas, os estados geralmente produzem resultados que se sobrepõem, de modo que não fica 100% claro qual estado está produzindo resultados. Por exemplo, estamos

aplicaremos um HMM a dados semelhantes aos da Figura 7-5. Estes são dados simulados

de quatro estados, mas não é óbvio, a partir de uma simples inspeção visual da série temporal, quantos estados existem, quais são suas divisões ou onde o sistema faz a transição entre os estados.

Figura 7-5. Esta série temporal foi simulada com quatro estados, mas não está claro a partir de uma observação visual que existem quatro estados, nem está claro onde termina um estado e outro começa.

Alguns exemplos de casos de uso reais de HMMs são:

• Identificação de mudanças de regime nos mercados financeiros

• Classificação, previsão e correção de informações de sequenciamento de DNA

• Reconhecer os estágios do sono refletidos nos dados de ECG

Como ajustamos o modelo

Estamos a postular que existe um estado que não podemos medir diretamente e que não há forma de obter uma resposta comprovadamente correta em muitos dos conjuntos de dados onde poderíamos aplicar esta técnica. Então, como esse algoritmo pode identificar estados ocultos sem saber nada sobre eles a priori? A resposta é: iterativamente. Não existe uma solução mágica para derivar a sequência mais provável de estados ocultos para explicar as observações, mas é possível avançar em direção a uma estimativa uma vez que tenhamos especificado completamente o sistema.

Em um HMM, postulamos que o sistema é totalmente descrito com o seguinte

Informação:

• A *probabilidade* de transição de *x(t)* para *x* (*t*) + 1. Isso equivale a especificar uma matriz como a que acabamos de mencionar: fazer a transição entre os estados A e B. O tamanho da matriz dependeria do número de hipotetizado

estados.

• A *probabilidade de emissão* , ou a probabilidade de observação *y(t)* dado *x(t)* .

• O estado inicial do sistema.

Mais concretamente, aqui está uma listagem das variáveis necessárias para caracterizar e ajustar um processo HMM:

• $Q = q\,1, q\,2, \ldots qN$ os estados distintos do sistema

• $A = ai,j = a\,1,1, a\,1,2, \ldots aN,N$ a matriz de probabilidade de transição indicando a transição em qualquer intervalo de tempo dado da mudança do estado i para o estado j

• $O = o\,1, o\,2, \ldots oT$ uma sequência de observações amostradas deste processo em ordem, que é uma série temporal de observações

• *bi(ot)* indicando as probabilidades de emissão, ou seja, as probabilidades de ver um determinado valor de observação,

ou se o estado for *qi*

• $p = p\,1, p\,2, \ldots pN$, as distribuições de probabilidade iniciais, nomeadamente a probabilidade de o sistema iniciar no estado $q\,1, q\,2, \ldots qN$, respectivamente. No entanto, com dados reais geralmente nenhuma destas variáveis é conhecida. Tudo o que se sabe é a sequência real de observáveis $y\,1, y\,2, \ldots yt$.

Algoritmo Baum Welch

Ao estimar os parâmetros de um modelo oculto de Markov, usamos o *Baum Welch algoritmo* . Isso nos orienta em nossa complexa tarefa de estimar os valores de todos os parâmetros, conforme detalhados na seção anterior. Nossa tarefa é múltipla. Estamos buscando:

• Identificar as probabilidades de emissão distintas para cada possível estado oculto e identificar as probabilidades de transição de cada possível estado oculto para cada outro possível estado oculto. Para isso usamos o algoritmo Baum-Welch.

• Identifique o estado oculto mais provável para cada intervalo de tempo, considerando o histórico completo de observações. Para isso utilizamos o algoritmo de Viterbi (descrito em breve).

Essas são duas tarefas relacionadas e cada uma é difícil e computacionalmente desgastante.

Além do mais, eles são dependentes. No caso de duas tarefas inter-relacionadas, estimativa de parâmetros e maximização de verossimilhança, podemos usar o algoritmo *de maximização de expectativa* para iterar entre essas duas etapas até que uma solução aceitável seja encontrada.

Para aplicar o algoritmo de Baum Welch, o primeiro passo é especificar a função de verossimilhança, que é a probabilidade de observar uma determinada sequência dados os parâmetros hipotéticos. No nosso caso, os

parâmetros hipotéticos seriam os parâmetros matemáticos por estado postulado.

Por exemplo, se assumissemos que os estados produziam resultados gaussianos com médias e desvios padrão distintos dos valores observados que dependiam do estado, e se assumíssemos um modelo de dois estados, poderíamos descrever o modelo em termos de μ1, σ1, μ2 , e σ2, com μ u = i indicando a média do i- ésimo estado e σ i indicando o desvio padrão do i- ésimo estado. Estes poderiam descrever as probabilidades de emissão, e nós os denotamos coletivamente como θ. Também poderíamos postular a sequência de estados como x 1, x 2, … xt (vou resumir isso como Xt para abreviar), o que não fomos capazes de observar, mas imaginamos por enquanto que poderíamos.

Então, a função de verossimilhança descreveria a probabilidade de observar a sequência que observamos dados os parâmetros da probabilidade de emissão (ou seja, a probabilidade de uma determinada observação dado um estado específico) e a sequência de estados ocultos como a integral sobre todos os Xt *possíveis.* de p (y 1, y 2, … yt |μ1, σ1, μ2, σ2, …μ N , σ Nt) = p (y 1, y 2, …

yt |μ1, σ1, μ2, σ2,…μ N , σ N).

No entanto, este é um problema difícil por várias razões, incluindo o facto de a complexidade crescer exponencialmente com o número de passos de tempo, o que significa que uma pesquisa exaustiva na grelha não é realista. Então, simplificamos a tarefa aplicando o EM

algoritmo da seguinte forma:

1. Inicialize aleatoriamente as variáveis de probabilidade de emissão.

2. Calcule a probabilidade de cada Xt *possível* , assumindo os valores de probabilidade de emissão.

3. Utilize estes valores de Xt para gerar uma melhor estimativa das variáveis de probabilidade de emissão.

4. Repita as etapas 2 e 3 até atingir a convergência.

O que isto significa, mais informalmente, é que no caso de postularmos (aleatoriamente) duas distribuições, olharíamos então para cada intervalo de tempo e determinaríamos a probabilidade de que para cada intervalo de tempo um determinado estado estivesse ocupado (por exemplo, no intervalo de tempo t qual era a probabilidade do estado A? do estado B?) Depois de atribuirmos um estado putativo a cada intervalo de tempo, usaríamos esses rótulos para reestimar as probabilidades de emissão (definindo uma melhor média e desvio padrão para o estado). Depois repetiríamos o processo novamente, utilizando as variáveis de probabilidade de emissão recentemente atualizadas para melhorar a nossa estimativa da trajetória Xt .

Um ponto importante a lembrar sobre esses usos do algoritmo EM é que não há garantia de encontrar um conjunto de parâmetros globalmente ideal. Por esse motivo, vale a pena executar vários ajustes para ver qual é o consenso global em inúmeras inicializações. Também é importante lembrar que um ajuste exigirá um período de adaptação, cuja duração apropriada dependerá dos detalhes dos seus dados e modelo.

Algoritmo de Viterbi

Uma vez estimados os parâmetros de um processo HMM, como via Baum

Algoritmo de Welch, a próxima questão de interesse é qual a série mais provável de estados subjacentes dada a série temporal medida de valores observáveis.

Ao contrário do algoritmo Baum Welch, o algoritmo Viterbi garante a melhor solução para a pergunta que você está fazendo. Isso ocorre porque é um algoritmo de programação dinâmica projetado para explorar de forma completa e eficiente a gama de ajustes possíveis, salvando as soluções em partes de um caminho, de modo que, à medida que o caminho é alongado, não haja necessidade de recalcular todos os caminhos possíveis para todos os comprimentos de caminho (consulte

Figura 7-6).

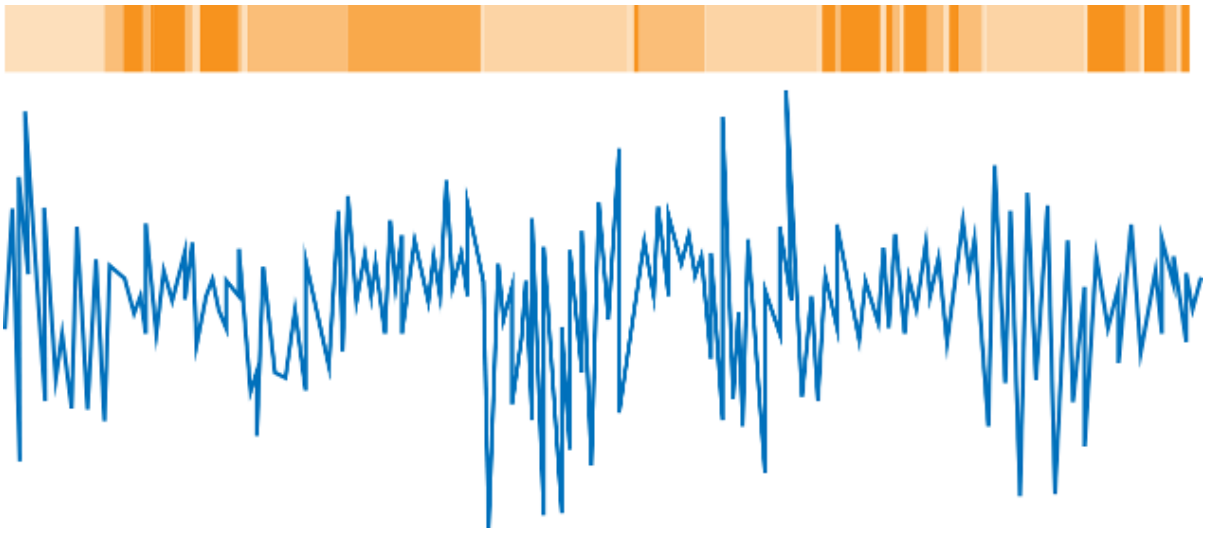


Figura 7-6. O algoritmo de Viterbi procura todos os caminhos possíveis que possam explicar uma determinada série temporal observada, onde um caminho indica qual estado foi ocupado em cada passo de tempo.

Programaçao dinamica

Os cientistas de dados nem sempre têm um curso de algoritmos padrão em seu kit de ferramentas, mas algoritmos básicos podem ser particularmente úteis para análise de séries temporais, onde inferência e recorrência são conceitos úteis para pensar sobre dados ordenados temporalmente.

A maneira mais fácil de explicar o que é programação dinâmica tende a ser com um exemplo, e o clássico é a sequência de Fibonacci. Imagine que lhe pedissem para calcular o oitavo número de Fibonacci. A maneira mais fácil de fazer isso seria se você já conhecesse o sexto e o sétimo números de Fibonacci. Para descobrir isso, você precisa do quarto e quinto números de Fibonacci e assim por diante. Como você sabe que as soluções para os problemas posteriores de Fibonacci são determinadas a partir dos anteriores, ao calcular os números de Fibonacci, você deve armazená-los em algum lugar para que possam ser usados

em problemas posteriores e mais complicados. Por esta razão, a programação dinâmica também é descrita como uma técnica de *memoização* .

As características de uma situação em que a programação dinâmica poderia ser aplicada são:

• Você pode calcular uma solução para um problema de tamanho N se souber a solução para um problema de tamanho N – 1. Por esse motivo, lembrar ou *memorizar* as soluções para problemas menores anteriores o ajudará a resolver problemas posteriores com mais eficiência.

• O problema tem uma ordem claramente definida para escalar de um problema menor para um problema maior.

• Você pode identificar um único caso base que seja diretamente (e facilmente) calculável.

Ajustando um HMM no código

Embora o processo de ajuste do HMM seja muito complicado, ele é implementado em vários pacotes em R. Aqui trabalharemos com o pacote depmixS4. Primeiro, precisamos formular uma série temporal apropriada. Fazemos isso com o seguinte código:

```
##R

## aviso que neste caso optamos por definir uma semente

## se você definir a mesma semente, nossos números deverão corresponder

definir.seed(123)

## definir parâmetros para a distribuição de cada um dos quatro

## estados de mercado que queremos representar
```

```
bull_mu <- 0,1

bull_sd <- 0,1

neutro_mu <- 0,02

neutro_sd <- 0,08

urso_mu <- -0,03

urso_sd <- 0,2

pânico_mu <- -0,1

pânico_sd <- 0,3

## colete esses parâmetros em vetores para facilitar a indexação

mus <- c(bull_mu, neutral_mu, bear_mu, panic_mu)

sds <- c(bull_sd, neutral_sd, bear_sd, panic_sd)

## definir algumas constantes descrevendo a série temporal que iremos gerar

NÚM.PERÍODOS <- 10

MENOR.PERÍODO <- 20

PERÍODO MAIS LONGO <- 40

## determinar estocasticamente uma série de contagens de dias, com

## contagem de cada dia indicando uma 'corrida' ou um estado do mercado

dias <- amostra (SMALLEST.PERIOD:LONGEST.PERIOD,
NUM.PERIODS,

substituir = VERDADEIRO )
```

para cada número de dias no vetor de dias

geramos uma série temporal para esse período de dias em um determinado

estado do mercado e adicione isso à nossa série temporal geral

retorna <- numérico()

true.mean <- numérico()

para (d em dias) {

idx = amostra(1:4, 1, prob = c(0,2, 0,6, 0,18, 0,02))

retorna <- c(retorna, rnorm(d, média = mus[idx], sd = sds[idx]))

verdadeiro.mean <- c(true.mean, rep(mus[idx], d))

}

No código anterior, usamos um exemplo inspirado no mercado de ações, com modos de alta, baixa, neutro e pânico. Um número aleatório de dias para um estado persistir é selecionado, como **224 | Capítulo 7: Modelos de Espaço de Estados para Séries Temporais**

são variáveis que descrevem a distribuição de probabilidade de emissão para cada estado (ou seja, as variáveis _mu e _sd, que indicam que tipo de valores esperamos ver medidos em determinado estado específico).

Podemos ter uma noção da aparência de nossa série temporal gerada e da frequência de cada estado, vendo quantos dias na amostra correspondem a cada true.mean, que é a variável que estamos usando para rastrear o estado:

##R

> tabela (true.mean)

verdadeiro.mean

-0,03 0,02 0,1

155 103 58

Catástrofe! Embora pretendíamos incluir quatro estados na série simulada, apenas três estão incluídos. Isto é provavelmente porque o quarto estado tinha uma probabilidade de inclusão muito baixa (0,02). Vemos que o estado menos provável nem sequer foi selecionado para inclusão na série. Nem sempre poderíamos saber isso para uma determinada série temporal – que nem todos os estados possíveis foram realmente incluídos em uma série temporal – o que destaca parte do motivo pelo qual é difícil ajustar HMMs e por que as coisas podem ser um tanto injustas com o algoritmo. No entanto, avançaremos com a análise especificando quatro grupos, para ver o que isso proporciona. 3

Ainda precisamos instalar um HMM. O HMM resultante fornecerá uma série temporal de probabilidades posteriores para cada estado para quantos estados indicarmos serem desejados. Consistente com a descrição anterior do algoritmo EM, nada mais do que o número de estados putativos precisa ser especificado. O restante será determinado gradualmente por meio de um processo iterativo de ida e volta.

Como costuma acontecer, com o pacote certo, a parte difícil da análise é, na verdade, bastante fácil em termos da quantidade de código que precisa ser escrita. Neste caso utilizamos o pacote depmixS4 4 em R. O modelo é ajustado em duas etapas. Primeiro, ele é especificado com a função depmix(), que indica a distribuição esperada, o número de estados e os dados de entrada a serem usados no ajuste. Em seguida, o modelo é ajustado por meio da função fit, que toma a especificação do modelo como entrada. Por fim, para gerar a distribuição posterior dos rótulos de estado dado o ajuste aos dados, utilizamos a função posterior().

Neste ponto, o modelo em si já foi ajustado, então esta é uma tarefa separada para rotular o 3 Observe que outro problema com os dados simulados é que não estabelecemos uma matriz de probabilidade de transição para controlar o

fluxo do estado oculto de um estado para outro. Essencialmente, partimos do pressuposto de que é muito mais provável que um estado permaneça como está por muitos dias consecutivos e depois salte para qualquer um dos outros estados com igual probabilidade. Deixamos de fora a especificação formal e o uso de uma matriz de transição simplesmente para manter o código mais simples.

4 O nome deste pacote reflete outro nome usado para HMM, *modelos de mistura dependente* .

dados agora que os parâmetros que descrevem as distribuições de estado e as probabilidades de transição foram estimados:

##R

exigir (depmixS4)

hmm.model <- depmix(retorna ~ 1, família = gaussian(),

nstates = 4, data=data.frame(retornos=retornos))

model.fit <- fit(hmm.model)

post_probs <- posterior(model.fit)

Aqui produzimos um hmm.model onde indicamos que o observável é o vetor de retornos. Indicamos também o número de estados (4) e que as probabilidades de emissão vêm em uma distribuição gaussiana através de nossa especificação para o parâmetro da família. Ajustamos o modelo por meio de uma função fit() e então calculamos as probabilidades posteriores com a função posterior(). As probabilidades posteriores nos dão a probabilidade de um determinado estado em um determinado momento para os parâmetros do modelo que determinamos com o processo de ajuste.

Agora podemos visualizar os estados junto com os valores medidos como segue (veja

Figura 7-7):

##R

plot(retorna, tipo = 'l', lwd = 3, col = 1,

yaxt = "n", xaxt = "n", xlab = "", ylab = "", ylim = c(-0,6, 0,6))

lapply(0:(comprimento(retorna) - 1, função (i) {

adicione um retângulo de fundo da cor apropriada

para indicar o estado durante um determinado intervalo de tempo

rect(i,-0,6,(i + 1),0,6,

col = rgb(0,0,0,0,0,0,alfa=(0,2 * post_probs$estado[i + 1])),

fronteira = **NA**)

}

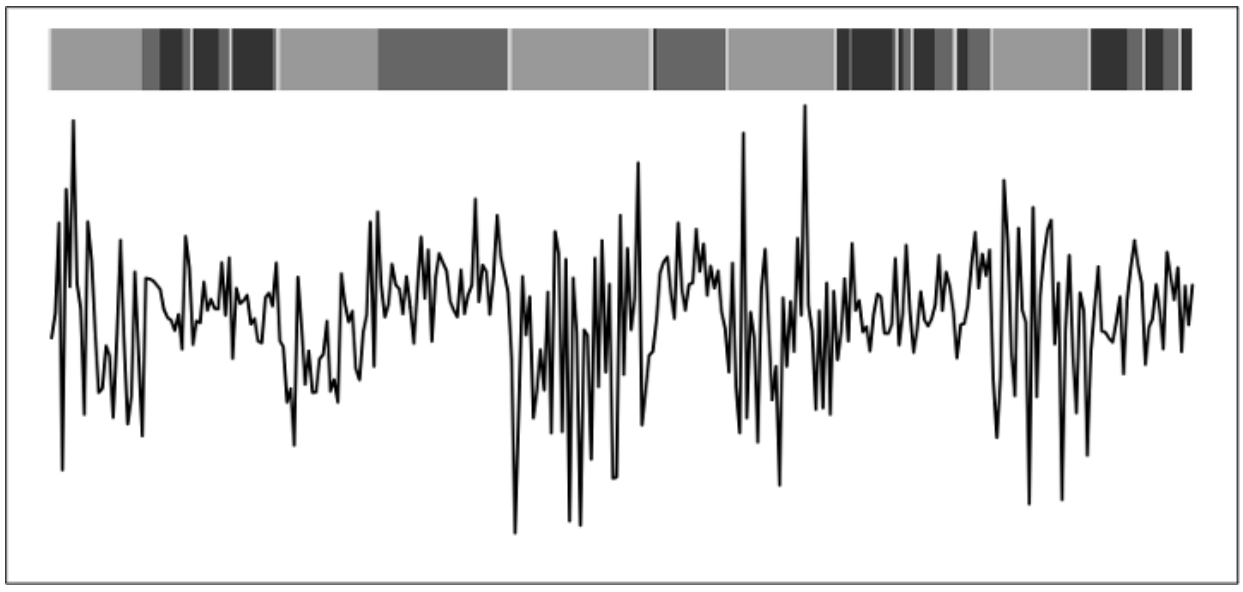


Figura 7-7. Neste gráfico, o fundo indica um estado distinto e a linha preta sólida representa os valores reais. Os objetos do tipo linha branca vertical são, na verdade, muito estreitos fatias que representam os momentos em que se estima que o processo esteja no que acontece ser o mais raro dos quatro estados.

Podemos acessar informações sobre os parâmetros distributivos hipotéticos que o modelo determinou por meio de seus atributos. Ao revisarmos isso, vamos ter em mente nossos parâmetros originais de geração de dados:

bull_mu <- 0,1

bull_sd <- 0,1

neutro_mu <- 0,02

neutro_sd <- 0,08

urso_mu <- -0,03

urso_sd <- 0,2

pânico_mu <- -0,1

pânico_sd <- 0,3

Se fôssemos combinar (lembrando que o regime de pânico não apareceu nos dados), pareceria que, dos estados realmente presentes nos dados, os seguintes grupos estavam aproximadamente correlacionados:

> attr(model.fit, "resposta")

[[1]]

[[1]][[1]] <- *coincidentemente tem média próxima ao regime de pânico*

mas esse regime não produziu realmente quaisquer dados

na amostra que foi ajustada. em vez disso, o algoritmo

atribuiu este quarto estado a valores mais negativos

Modelo do tipo gaussiano (identidade), fórmula: retorna ~ 1

Coeficientes:

(Interceptar)

-0,09190191

dp 0,03165587

[[2]]

[[2]][[1]] *<- poderia ser consistente com o regime de mercado em baixa*

Modelo do tipo gaussiano (identidade), fórmula: retorna ~ 1

Coeficientes:

(Interceptar)

-0,05140387

dp 0,2002024

[[3]]

[[3]][[1]] *<- poderia ser consistente com o regime de mercado em alta*

Modelo do tipo gaussiano (identidade), fórmula: retorna ~ 1

Coeficientes:

(Interceptar)

0,0853683

dp 0,07115133

[[4]]

[[4]][[1]] *<- poderia ser um tanto consistente com o regime de mercado neutro* Modelo do tipo gaussiano (identidade), fórmula: retornos ~ 1

Coeficientes:

(Interceptar)

-0,0006163519

dp 0,0496334

Uma possível razão pela qual o ajuste não se alinha melhor com nossos estados ocultos de linha de base é que não usamos uma matriz de transição adequada, enquanto o modelo usou uma para ajuste.

Nossas transições entre estados não foram propriamente Markovianas e isso poderia impactar o ajuste. Além disso, nos ajustamos a uma série temporal relativamente curta com poucas transições entre estados, enquanto os HMMs terão melhor desempenho em séries temporais mais longas, com mais oportunidades para observar/inferir transições de estado. Eu recomendaria criar dados sintéticos mais realistas para testar um HMM proposto quando você estiver experimentando essa técnica e se preparando para ajustá-la a dados reais. Lembre-se, na maioria dos casos de dados reais você está postulando um estado não observável, então você quer pensar sobre os limites do desempenho do modelo em um ambiente mais controlado (com dados sintéticos) antes de se preparar para projetos mais ambiciosos.

Os HMMs estão aptos a analisar muitos tipos de dados. HMMs têm sido usados para modelar

se um mercado financeiro está em fase de crescimento ou recessão, determinar em que estágio se encontra o dobramento de uma proteína dentro de uma célula e descrever o movimento humano (antes da chegada do

aprendizado profundo). Estes modelos continuam a ser úteis, mais frequentemente para compreender a dinâmica de um sistema do que para fazer previsões. Além disso, os HMMs fornecem mais do que apenas uma estimativa pontual ou previsão. Finalmente, podemos injetar conhecimentos ou crenças anteriores em nosso modelo, especificando, por exemplo, o número de estados usados para ajustar nosso HMM. Neste

Dessa forma, colhemos os benefícios dos métodos estatísticos, mas também temos parâmetros para parametrizar nosso conhecimento prévio sobre o sistema.

A matemática e a computação dos HMMs são bastante interessantes e acessíveis.

Você pode aprender muitas técnicas de programação acessíveis e algoritmos de otimização numérica examinando as maneiras mais comuns pelas quais os HMMs se ajustam aos dados.

Você também aprenderá sobre técnicas de programação dinâmica, que são úteis para um cientista de dados ou engenheiro de software.

Tal como acontece com um filtro de Kalman, os HMMs podem ser usados para uma variedade de tarefas. Na verdade, a variedade de problemas de inferência relacionados aos sistemas HMM é ainda mais complexa devido ao aumento da complexidade dos estados discretos, cada um com sua própria probabilidade de emissão. Algumas das tarefas de inferência que você pode enfrentar ao usar um HMM incluem:

• Determinar a descrição mais provável dos estados produzindo uma série de observações. Isto envolve estimar as probabilidades de emissão destes estados, bem como a matriz de transmissão que descreve a probabilidade de um estado conduzir a outro. Fizemo-lo, embora não tenhamos olhado explicitamente para a transição

probabilidades.

• Determinar a sequência mais provável de estados a partir de uma série de observações e de uma descrição dos estados e das suas probabilidades de emissão e transição. Fizemos isso também como parte da tarefa anterior. Isto é por vezes referido como a “explicação mais provável” e é normalmente calculado com o algoritmo de Viterbi.

• Filtragem e suavização. Nesta situação, a filtragem corresponderia a estimar o estado oculto do intervalo de tempo mais recente dada a observação mais recente. A suavização corresponderia à determinação da distribuição mais provável do estado oculto em um determinado intervalo de tempo, dadas as observações antes, durante e depois desse intervalo de tempo.

Série temporal estrutural bayesiana

As séries temporais estruturais bayesianas (BSTS) estão relacionadas ao modelo linear gaussiano que trabalhamos com a filtragem de Kalman anteriormente. A principal diferença é que as séries temporais estruturais bayesianas fornecem uma maneira de usar componentes preexistentes para construir modelos mais complexos que podem refletir fatos conhecidos ou hipóteses interessantes sobre um sistema. Podemos então projetar a estrutura, usar técnicas de ajuste robustas para estimar parâmetros para o modelo construído no caso de nossos dados e ver se o modelo faz um bom trabalho ao descrever e prever o comportamento de um sistema.

A matemática dos modelos BSTS é bastante complicada e computacionalmente desgastante em comparação com o modelo linear gaussiano que abordamos em nossa discussão sobre o filtro de Kalman. Por esse motivo, nos limitaremos a uma visão geral e depois aplicaremos o código.

Série temporal estrutural bayesiana | 229

Existem quatro etapas para ajustar um modelo BSTS, realizadas na seguinte ordem: 1. Um modelo estrutural é definido, incluindo a especificação de anteriores.

2. Um filtro de Kalman é aplicado para atualizar estimativas de estado com base nos dados observados.

3. O método spike-and-slab é usado para realizar a seleção de variáveis dentro do

modelo estrutural.5

4. A média do modelo bayesiano é aplicada para combinar resultados com a finalidade de gerar uma previsão.

No próximo exemplo, focaremos apenas nas etapas 1 e 2, nas quais definimos um modelo flexível a partir de componentes modulares preexistentes e, em seguida, usamos isso para ajustar nossos dados com um método bayesiano que atualiza a estimativa de parâmetros com o passar do tempo.

Código para bsts

Aqui usamos um pacote BSTS popular e poderoso do Google, bsts, e um pacote aberto

conjunto de dados de OpenEI.org.

Primeiro plotamos os dados para ter uma ideia do que estamos tentando modelar (Figura 7-8):

##R

elec = fread("elétrico.csv")

exigir (bsts)

n = nomes de colunas(elec)[9]

par(mfrow = c(2, 1))

plotar(elec[[n]][1:4000])

enredo(elec[[n]][1:96)

conforme discutido anteriormente no livro

escala temporal apropriada é a chave para entender seus dados

5 Para ler mais sobre o método spike-and-slab, comece com Wikipedia . A matemática é bastante complicada, por isso não discutiremos mais sobre ela aqui. O método spike-and-slab é mais útil em casos onde você tem muitas entradas e precisa de seleção de variáveis para simplificar seu modelo.

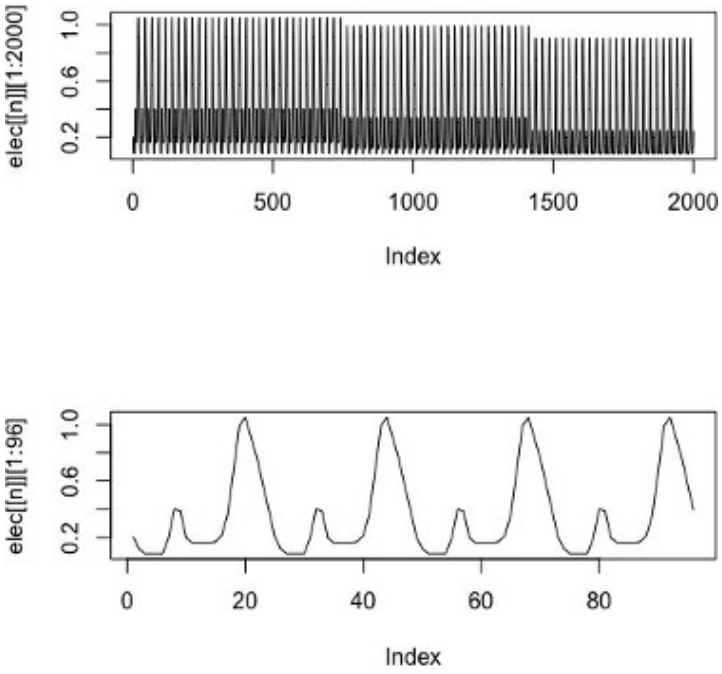


Figura 7-8. A série completa que ajustamos (duas mil medições horárias consecutivas) em o gráfico superior e um subconjunto mais curto e mais compreensível dos dados no gráfico inferior.

O enredo faz mais sentido quando olhamos para ele de uma forma que torna os padrões diários evidente.

Observar nossos dados nos dá uma noção de como modelar. Podemos ver que certamente existe um padrão diário, e pode até haver um padrão de dia da semana. Estes refletem o comportamento sazonal que descreveremos em nosso modelo. Também queremos permitir uma tendência nos dados, dado o comportamento não estacionário que vemos quando plotamos todos os dados no

painel superior da Figura 7-8 :

##R

```
ss <- AddLocalLinearTrend(lista(), elec[[n]])

ss <- AddSeasonal(ss, elec[[n]], nseasons = 24,
season.duration = 1 )

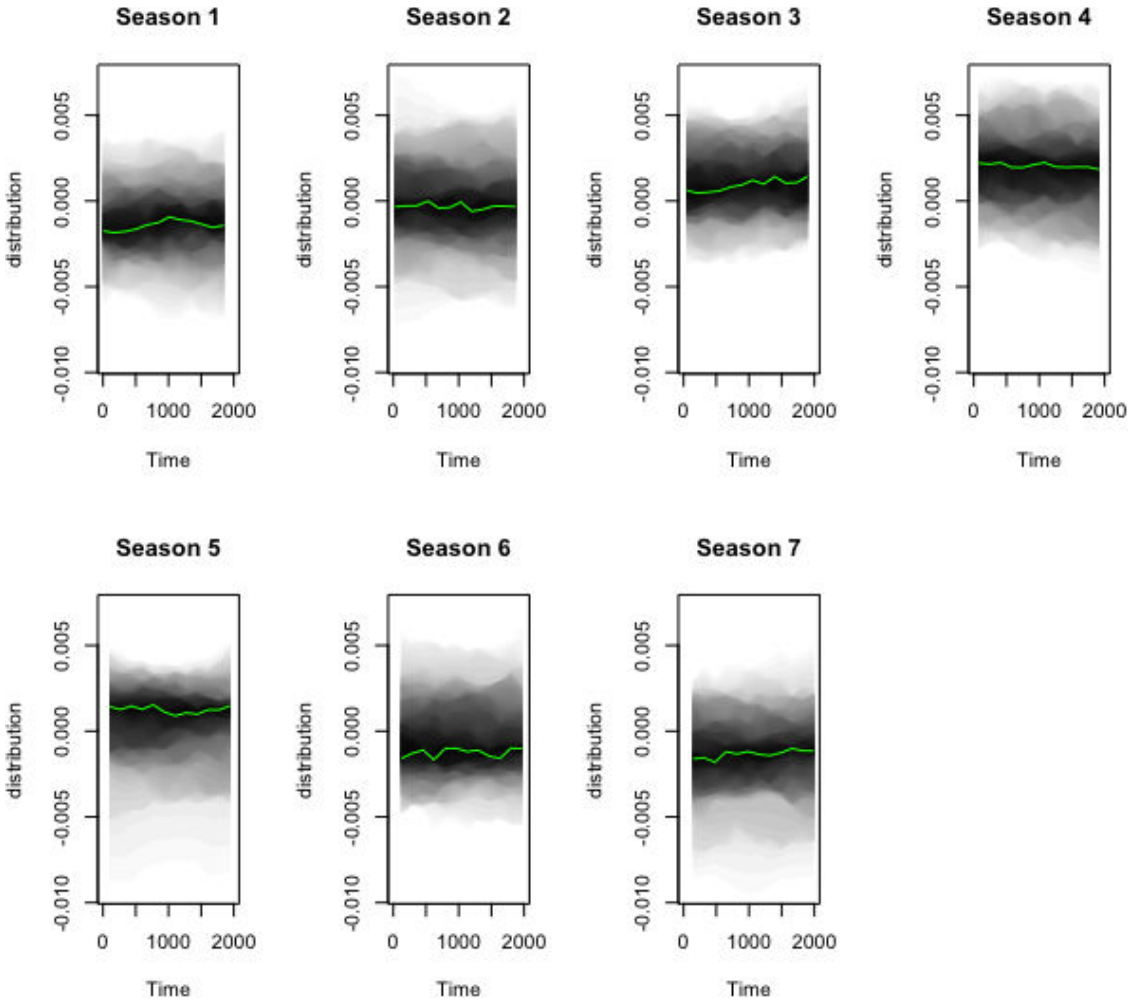
ss <- AddSeasonal(ss, elec[[n]], nseasons = 7,
season.duration = 24)
```

A tendência linear local neste modelo assume que tanto a média quanto a inclinação de uma tendência

nos dados seguem um passeio aleatório.6

A componente sazonal do modelo utiliza dois argumentos, um indicando o número de estações distintas e outro indicando a duração da estação. Na primeira componente sazonal acrescentamos, que reflete um ciclo diário, queremos uma estação para cada hora do dia, e cada estação dura apenas uma hora. Na segunda componente sazonal acrescentamos, que reflete um ciclo semanal, queremos uma estação para cada dia da semana e queremos que cada estação dure 24 horas.

Embora você possa estar se perguntando se realmente começaremos às 12h01 de segunda-feira (ou como quisermos definir a semana), ser consistente é mais importante do que 6 Mais informações na documentação.



se um rótulo sazonal do dia 1 corresponde exatamente a uma segunda-feira. No padrão recorrente que vemos aqui, parece que qualquer forma de dividir os dados em 24 horas seria provavelmente aceitável para a análise de sazonalidade.

A parte mais exigente do código, computacionalmente, é mostrada a seguir. A beleza do pacote bsts é que somos capazes de executar uma série de cálculos de Markov Chain Monte Carlo (MCMC) posteriores:

##R

modelo1 <- bsts(elec[[n]],

estado.especificação = ss,

nitro = 100)

gráfico(modelo1, xlim = c(1800, 1900)

Também podemos inspecionar os componentes sazonais. Por exemplo, inspecionamos o dia do

componente sazonal semanal da seguinte forma (veja as Figuras 7-9 e 7-10):

##R

plot(model1, "sazonal", nseasons = 7, season.duration = 24)

Figura 7-9. As estações dos dias da semana, que mostram que há diferença nos diferentes dias da semana. Também mostra que as distribuições dos parâmetros do dia da semana são estável ao longo do tempo para cada dia da semana.

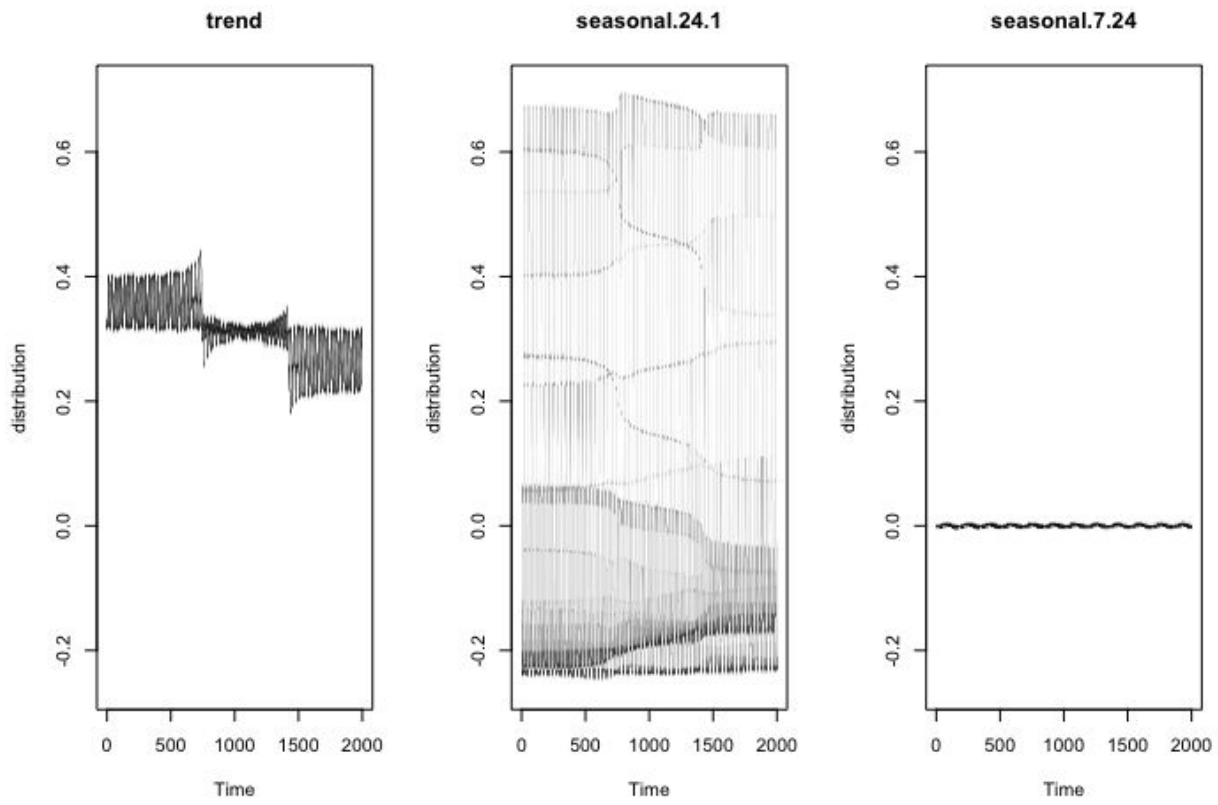


Figura 7-10. Distribuições de contribuições da tendência dos dados, bem como do componentes sazonais diários e do dia da semana. Se você somar esses três componentes, você obterá o valor da previsão.

A componente sazonal do dia da semana mostra grande estabilidade, enquanto a sazonalidade da hora do dia mostrada no gráfico central da Figura 7-10 tende a mostrar tendências ao longo do tempo, provavelmente relacionadas com a mudança das horas do dia. Na Figura 7-10 também vemos o ajuste do parâmetro para a nossa tendência linear local, que mostra um padrão decrescente na procura global de electricidade.

Finalmente, fazemos previsões, completadas com um gráfico completo da distribuição posterior das previsões para o futuro (ver Figura 7-11). Observe que temos a flexibilidade aumentada

até o final deste processo de modelagem para indicar quantos horizontes de tempo queremos prever adiante.

Lembre-se de que estes são dados de hora em hora, portanto, prever 24 horizontes de tempo futuros pode parecer bastante ambicioso, mas equivale a apenas um dia. Também indicamos que queremos ver os 72 períodos anteriores à previsão para contexto:

```
pred <- prever (modelo1, horizonte = 24, quantis = c (0,05, 0,95))

enredo(pred, enredo.original = 72)
```

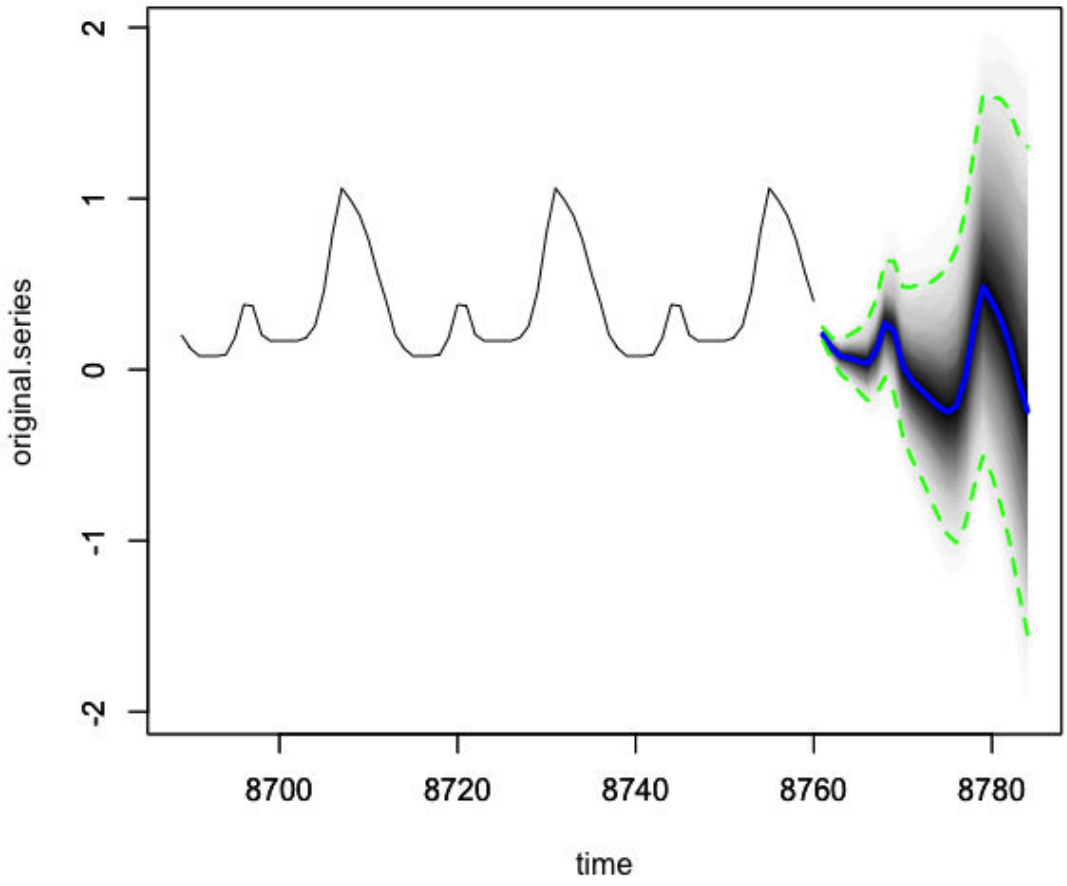



Figura 7-11. As últimas 72 horas em nossos dados combinadas com uma previsão para as próximas 24

horas, bem como limites de quantis de 5% e 95% nas previsões. Observe que a distribuição da previsão fica mais espalhada à medida que fazemos previsões no futuro.

Há uma grande quantidade de opcionalidades no pacote bsts e na modelagem bayesiana de séries temporais estruturais das quais ainda nem aproveitamos, a saber:

• Não especificamos antecedentes fora do padrão.

• Não utilizamos o método spike-and-slab para selecionar regressores.

• Não utilizamos a média do modelo Bayesiano.

Tudo isso é possível fazer com o pacote bsts e você pode encontrar facilmente orientação na documentação.

Apenas arranhamos a superfície em nosso exemplo do que pode ser feito com o BSTS.

Aqui estão algumas outras opções importantes disponíveis com BSTS:

• BSTS permite que você injete qualquer tipo de antecedente em sua modelagem. O modelo gaussiano linear padrão que abordamos ao discutir o filtro de Kalman é apenas uma maneira de injetar um anterior bastante básico, enquanto o BSTS oferece muitas opções (por exemplo, anteriores assimétricos).

• Os modelos BSTS permitem realizar seleção de variáveis.

• Os modelos BSTS podem ser combinados através da média do modelo bayesiano, o que pode ajudar a explicar a incerteza associada à seleção de um modelo em primeiro lugar.

Embora não tenhamos utilizado essas opções em nosso caso de modelagem atual, elas não são difíceis de incorporar ao usar o pacote bsts, e você pode encontrar muitos exemplos online.

Mais recursos

• Em filtros de Kalman e modelos lineares de espaço de estado gaussiano:

Greg Welch e Gary Bishop, "Uma introdução ao filtro de Kalman", técnico relatório, Universidade da Carolina do Norte em Chapel Hill, 1995, https://perma.cc/ZCU8-MXEF.

Esta visão geral introdutória do filtro de Kalman oferece uma derivação e

formulação matricial do filtro. A introdução também discute a Extensão

ded Kalman Filter, uma filtragem mais comum em cenários da vida real que pode ser usada para processos não lineares ou erros de medição não lineares.

RE Kalman, "Uma nova abordagem para filtragem linear e problemas de previsão",

Transações da ASME — Journal of Basic Engineering 82, Série D (1960): 35–

45, https://perma.cc/GNC4-YLEC.

Este artigo de pesquisa de 1960 é a introdução original do filtro de Kalman.

A matemática é bastante acessível com uma formação básica em estatística

e cálculo, e esta é uma leitura interessante para aprender as motivações originais do filtro de Kalman e o contexto intelectual de seu criador.

R. Labbe, "Kalman and Bayesian Filters in Python", repositório GitHub, https://

perma.cc/CMU5-Y94A.

Este repositório épico do GitHub inclui dezenas de exemplos de uso do Kalman

filtros e técnicas relacionadas de “filtragem” de forma mais geral. O repositório é semelhante a um livro didático, apresentando exemplos trabalhados, um livro em PDF relacionado e exercícios

com soluções.

Marie Auger-Méthé et al., “Pequenos segredos sujos dos modelos de espaço de estado: até simples

Modelos lineares gaussianos podem ter problemas de estimativa”, Relatórios Científicos 6, não.

26677 (2016), https://perma.cc/9D8V-Z7KJ.

Este artigo destaca um caso em que mesmo modelos gaussianos lineares simples, como aqueles que usamos em nossa discussão do filtro de Kalman, são facilmente propensos a especificações incorretas extremas, especialmente no caso de um erro de medição

que é grande em relação aos valores de uma série temporal. Os autores concentram-se neste problema do ponto de vista dos ecologistas, mas a preocupação geral permanece

válido para uma variedade de disciplinas baseadas em dados e oferece uma visão de contrapeso em comparação com as muitas vantagens enfatizadas para este método.

Mais recursos | 235

• Em modelos ocultos de Markov:

Andrew Moore, “Hidden Markov Models”, notas de aula, School of Computer Sci-

ence, Carnegie Mellon University, https://perma.cc/K3HP-28T8.

Estas notas de aula abrangentes oferecem uma visão geral dos HMMs, completa

com ilustrações de algoritmos de estimativa e exemplos de robótica de

como os HMMs são usados em aplicações reais.

Dan Klein, "Inteligência Artificial: Modelo Oculto de Markov", notas de aula, Univer-

cidade da Califórnia, Berkeley, https://perma.cc/V7U4-WPUA.

Este é outro conjunto de notas de referência acessíveis. Estes dão exemplos da utilidade dos HMMs para digitalizar a fala e também para desenvolver IA para reproduzir

jogos estratégicos.

user34790, "Qual é a diferença entre o Forward-Backward e o Viterbi

Algoritmos?" pergunta postada em Cross Validated, StackExchange, 6 de julho de 2012,

https://perma.cc/QNZ5-U3CN.

Esta postagem do StackExchange oferece uma discussão interessante e um esboço relacionado aos muitos algoritmos de estimativa implantados para casos de uso específicos de HMM.

Esta postagem ajudará você a ter uma ideia de como os HMMs podem ser usados para entender

dados de séries temporais, mesmo que você não esteja interessado nos detalhes do

algoritmos de modelagem relacionados.

• Nas séries temporais estruturais bayesianas:

Mark Steel, "Análise Bayesiana de Séries Temporais", em Macroeconometria e Série Temporal

Análise de ies, ed. Steven N. Durlauf e Lawrence E. Blume (Basingstoke, Reino Unido: Pal-grave Macmillan, 2010), 35–45, https://perma.cc/578D-XCVH.

Esta breve leitura oferece uma visão abrangente das diferentes técnicas associadas à análise Bayesiana de séries temporais, bem como comentários concisos sobre os pontos fortes e fracos de cada método.

Steven Scott e Hal Varian, "Prevendo o Presente com Estrutural Bayesiana

Série temporal", artigo não publicado, 28 de junho de 2013, https://perma.cc/4EJX-6WGA.

Este artigo do Google baseado em séries temporais econômicas apresenta um exemplo de

aplicar um problema de previsão de série temporal a dados sobre um determinado período de tempo que

fica disponível com atrasos diferentes. Em particular, os autores utilizam pesquisas atuais do Google para prever as taxas de desemprego, onde estas últimas taxas são

publicado apenas periodicamente, enquanto as contagens de pesquisa do Google foram contínuas

totalmente disponível. Este é um exemplo do que é frequentemente chamado de "nowcasting" para

indicam uma previsão que realmente está sendo feita sobre o presente devido a uma

atraso no relatório. O artigo usa uma combinação de serviços de tempo estruturais Bayesianos.

ies e técnicas de conjunto.

Jennifer Hoeting et al., "Média do modelo bayesiano: um tutorial", Statistical Science

14, não. 4 (1999): 382–401, https://perma.cc/BRP8-Y33X.

Este artigo fornece uma visão abrangente de como o modelo Bayesiano calcula

o envelhecimento funciona através de vários métodos diferentes. Como este artigo descreve, o objetivo da média do modelo bayesiano é levar em conta a incerteza no

processo de modelagem devido à seleção do modelo. Através de exemplos trabalhados, o

os autores fornecem uma maneira de estimar melhor a incerteza nas previsões. A

um resumo mais simples e relacionado é fornecido em uma visão geral do BMA , um pacote R para média do modelo bayesiano.

Mais recursos | 237

CAPÍTULO 8

Gerando e selecionando recursos

para uma série temporal

Nos dois capítulos anteriores, examinamos métodos de análise de séries temporais que dependem do uso de todos os pontos de dados em uma série temporal para ajustar um modelo. No entanto, em preparação para a discussão do próximo capítulo sobre a aplicação do aprendizado de máquina à análise de séries temporais, neste capítulo estudaremos a geração e seleção de recursos para séries temporais.

Se você não está familiarizado com o conceito de geração de recursos, não permanecerá assim por muito tempo. É um processo intuitivo e que permite um lado criativo na análise de dados.

A geração de recursos é o processo de encontrar uma maneira quantitativa de encapsular as características mais importantes dos dados de séries temporais em apenas alguns valores numéricos e rótulos categóricos. Você está compactando os dados brutos da série temporal em uma representação mais curta por meio de um conjunto de recursos para descrever essa série temporal (trabalharemos em um exemplo rápido em breve). Por exemplo, uma geração de recursos muito simples poderia descrever cada série temporal com seu valor médio e o número de etapas de tempo na série. Essa seria uma maneira de descrever essa série temporal sem passar passo a passo por todos os dados brutos.

O objetivo da geração de recursos é compactar o máximo possível de informações sobre a série temporal completa em algumas métricas ou, alternativamente, usar essas métricas para identificar as informações mais importantes sobre a série temporal e descartar o restante. Isso é importante para métodos de aprendizado de máquina, muitos dos quais foram desenvolvidos em dados não temporais, mas que podem ser aplicados de maneira proveitosa a problemas de séries temporais, desde que possamos digerir uma série temporal em uma entrada formatada adequadamente. Neste capítulo, focaremos particularmente em pacotes que nos permitem gerar automaticamente recursos de séries temporais comumente usados, de forma que não haja necessidade de reinventá-los ou codificá-los manualmente.

Depois de gerarmos alguns recursos supostamente úteis, devemos garantir que eles sejam realmente úteis. Embora seja improvável que você crie muitos recursos inúteis manualmente, **239**

esse é um problema que você encontrará ao usar código que gera automaticamente um grande número de recursos de uma série temporal para uso posterior em aprendizado de máquina.

Por esse motivo, devemos inspecionar as funcionalidades, uma vez geradas, para ver quais podem ser descartadas em análises subsequentes.

Os modelos tradicionais de aprendizado de máquina não foram originalmente desenvolvidos com séries temporais em

mente e, portanto, não se prestam automaticamente a aplicações analíticas de séries temporais. No entanto, uma forma de fazer esses modelos funcionarem com dados temporais é a geração de recursos. Por exemplo, ao descrever uma série temporal univariada não com uma série de números detalhando as saídas passo a passo de um processo, mas sim descrevendo-a com um conjunto de características, podemos acessar métodos projetados para dados transversais.

Neste capítulo, trabalharemos primeiro com um exemplo muito simples de geração de recursos para uma série temporal curta. Em seguida, revisaremos os pacotes de geração de recursos para séries temporais, tanto em R quanto em Python. Por fim, trabalharemos com um exemplo de geração e seleção automatizada de recursos. Depois de ler este capítulo, você terá todas as habilidades necessárias para pré-processar um conjunto de dados de série temporal para aplicações downstream de aprendizado de máquina no Capítulo 9 .

Exemplo introdutório

Imagine que as temperaturas da manhã, do meio-dia e da noite da semana passada foram as mostradas

na Tabela 8-1 .

Tabela 8-1. Temperaturas da última semana

Tempo

Temperatura (°F)

segunda-feira de manhã

35

Segunda-feira ao meio-dia

52

tarde de segunda

15

terça-feira de manhã

37

Terça-feira ao meio-dia

52

Terça-feira à noite

15

manhã de quarta-feira

37

Quarta-feira ao meio-dia

54

quarta à noite

16

manhã de quinta-feira

39

Quinta-feira ao meio-dia

51

quinta-feira à noite

12

sexta-feira de manhã

41

Sexta-feira ao meio-dia

55

sexta-feira à noite

20

sábado de manhã

43

Sábado ao meio-dia

58

Tempo

Temperatura (°F)

Sábado à noite

22

domingo de manhã

46

Domingo ao meio-dia

61

Domingo à noite

35

Você poderia representar graficamente esses dados e veria elementos de periodicidade (um ciclo diário) e também uma tendência geral de aumento das temperaturas. Mas não

podemos armazenar a imagem de um gráfico em um banco de dados, e a maioria dos métodos que aceitam uma imagem como entrada consome muitos dados e procuram reduzir a imagem em métricas resumidas. Portanto, devemos fazer nós mesmos as métricas resumidas. Em vez de descrever os 21 números da Tabela 8-1 como uma série temporal, poderíamos descrever a série com algumas palavras e números:

• Diário/periódico

• Tendência crescente; poderíamos tornar isso mais quantitativo calculando uma inclinação

• Valores médios para manhã, meio-dia e noite

Ao fazer isso, resumiríamos a série temporal de 21 pontos com 2 a 5 números – bastante compactação de dados sem perder muitos detalhes. Este é um caso simples de geração de recursos. Então, *a seleção de recursos* implicaria eliminar quaisquer recursos que não fossem descritivos o suficiente para justificar a inclusão. O que justifica a inclusão dependerá do uso posterior dos recursos.

Considerações gerais ao calcular recursos

Como acontece com qualquer aspecto da análise, quando você está computando recursos de séries temporais para um conjunto de dados de séries temporais, você vai querer pensar se sua análise faz sentido e se o esforço que você coloca na geração de recursos tem maior probabilidade de levar ao overfitting de um excesso de recursos do que levar a insights significativos.

A melhor abordagem é desenvolver um conjunto de recursos potencialmente úteis à medida que você explora e limpa séries temporais. Ao visualizar dados e pensar sobre o que distingue diferentes séries temporais no mesmo conjunto de dados ou diferentes períodos temporais na mesma série temporal, você desenvolverá ideias sobre quais tipos de medidas seriam úteis para rotular ou prever uma série temporal. Você também pode obter assistência útil de qualquer conhecimento prévio que tenha sobre um sistema

ou até mesmo de uma hipótese de trabalho que gostaria de testar com análise subsequente.

A seguir, discutiremos algumas preocupações distintas que você deve ter em mente ao gerar recursos de série temporal.



A natureza da série temporal

Ao decidir quais recursos de série temporal gerar, você precisa ter em mente os atributos subjacentes de sua série temporal, que você determinou durante a exploração e limpeza de dados.

Estacionaridade

A estacionariedade é uma consideração. Muitos recursos de séries temporais assumem estacionariedade e são inúteis, a menos que os dados subjacentes sejam estacionários ou pelo menos ergódicos. Por exemplo, usar a média de uma série temporal como característica só é prático quando a série temporal é estacionária, de modo que a ideia de média faz sentido. Este valor não é muito significativo quando temos uma série temporal não estacionária, pois o valor medido como média nesse caso é mais ou menos um acidente, resultado de demasiados processos emaranhados, como uma tendência ou um ciclo sazonal.

Séries temporais estacionárias versus ergódicas

Uma série temporal *ergódica* é aquela em que cada sub-série (razoavelmente grande)

amostra é igualmente representativa da série. Este é um rótulo mais fraco

do que a estacionariedade, o que exige que essas subamostras tenham iguais

média e variância. A ergodicidade exige que cada fatia do tempo seja

“igual” por conter informações sobre a série temporal, mas não

necessariamente iguais em suas medidas estatísticas (média e variação).

iance). Você pode encontrar uma discussão útil no StackExchange.

Duração da série temporal

Outra consideração para a geração de recursos é a duração da série temporal. Algumas características podem ser sensíveis para uma série temporal estacionária, mas tornam-se instáveis à medida que o comprimento da série temporal aumenta, como o valor mínimo e máximo da série.

Para o mesmo processo subjacente, uma série temporal mais longa provavelmente medirá valores máximos e mínimos mais extremos do que uma série temporal mais curta produzida pelo mesmo processo, simplesmente porque havia mais oportunidades para recolha de dados.

Conhecimento de Domínio

O conhecimento do domínio deve ser fundamental para a geração de recursos de séries temporais, onde você tiver a sorte de ter alguns insights. Alguns exemplos de como o conhecimento de domínio é aplicado para gerar características específicas de séries temporais são fornecidos posteriormente neste capítulo, mas por enquanto nos concentraremos no ponto mais geral.

Por exemplo, se você estiver trabalhando com uma série temporal de física, você deve quantificar recursos que façam sentido na escala de tempo do sistema que você está estudando, bem como certificar-se de que os recursos selecionados não sejam indevidamente influenciados pelas

características de, digamos, o erro de um sensor, e não os caracteres de um sistema subjacente.

Como outro exemplo, imagine que você está trabalhando com dados de um mercado financeiro específico. Para garantir a estabilidade financeira, este mercado impõe variações máximas de preços num determinado dia. Se o preço mudar muito, o mercado fecha. Você pode considerar, neste contexto, gerar um recurso indicando o preço máximo visto em um determinado dia.

Considerações Externas

A extensão dos seus recursos computacionais e recursos de armazenamento associados também é importante. Da mesma forma, sua motivação para gerar recursos é importante. Você está gerando recursos que serão armazenados para que você possa descartar volumosos dados brutos? Ou você está apenas computando os recursos para uma única análise e planejando manter apenas os dados brutos?

O propósito da geração de recursos de série temporal pode influenciar quantos recursos você decide computar e se você deve contemplar recursos particularmente exigentes em termos computacionais. Isso também pode depender do tamanho geral do conjunto de dados que você está analisando. Para um conjunto de dados pequeno, todas essas decisões serão de baixo risco, mas no caso de conjuntos de dados de séries temporais extremamente grandes, você pode correr o risco de embarcar em uma tarefa de geração de recursos que ficará pela metade, desperdiçando energia computacional e codificação.

Depois de considerar todos esses fatores, tente reunir uma lista de recursos e executá-los em um pequeno conjunto de dados para ter uma ideia de quão rápido ou lento eles são executados. Se o conjunto pequeno for muito lento, você deverá reduzir substancialmente sua série temporal antes de continuar sua análise. Da mesma forma, você pode considerar explorar a utilidade de recursos de tributação

computacional em um subconjunto de seus dados antes de realizar uma análise com o conjunto de dados completo.

Um catálogo de lugares para encontrar recursos de inspiração

A geração de recursos de série temporal é limitada apenas por seus dados, sua imaginação, sua habilidade de codificação e seu conhecimento de domínio. Contanto que você consiga pensar em uma maneira razoavelmente geral e bem definida de quantificar o comportamento de uma série temporal, poderá gerar um recurso. Alguns recursos de série temporal simples e frequentemente usados equivalem às mesmas funções estatísticas resumidas que você usará em outros aplicativos, como:

- Média e variância
- Máximo e mínimo
- Diferença entre o último e o primeiro valor

Você também identificará visualmente outros recursos que são mais desafiadores do ponto de vista computacional, mas que geralmente são úteis. Alguns exemplos incluem:

- Número de máximos e mínimos locais

Um catálogo de lugares para encontrar recursos de inspiração | 243

- Suavidade da série temporal
- Periodicidade e autocorrelação da série temporal

Nesses casos, você precisará fazer algumas definições de implementação, pois existem diferentes maneiras de identificar esses recursos comumente usados. Isso ajudará a manter disponível sua própria biblioteca pessoal de código de geração de recursos, mas você também pode querer examinar bibliotecas de geração de recursos para dados de série temporal, especialmente à medida que se interessa pelos recursos mais exigentes em termos computacionais.

Nesses casos, você deve procurar uma implementação excelente, de modo que o código seja confiável e eficiente.

Agora passaremos ao uso de bibliotecas de geração de recursos de séries temporais, prestando atenção especial à ampla gama de recursos dos quais você pode se beneficiar por meio da geração automática de recursos.

Bibliotecas de geração de recursos de série temporal de código aberto

Tem havido muitos esforços para automatizar a criação de recursos de séries temporais porque eles tendem a ser interessantes, descritivos e até mesmo preditivos em todos os domínios.

O módulo Python tsfresh

Um exemplo particularmente atraente de geração automática de recursos em Python é o módulo tsfresh, que implementa um conjunto grande e geral de recursos. Podemos conseguir um

sentido da amplitude dos recursos implementados, considerando algumas das categorias gerais de recursos que estão disponíveis. Esses incluem:

Estatísticas descritivas

Eles são impulsionados pelas metodologias estatísticas tradicionais de séries temporais que estudamos.

mencionado no Capítulo 6, incluindo:

- Um valor de teste Dickey-Fuller aumentado
- Um coeficiente AR(k)
- A autocorrelação para um atraso, k

Indicadores de não linearidade e complexidade inspirados na física

Esta categoria inclui:

• A função c3(), que é uma proxy para calcular o valor esperado de

L 2(X 2) × L (X) × X (L é o operador de atraso). Isso foi proposto como uma medida de *não linearidade* em uma série temporal.

• A função cid_ce(), que calcula a raiz quadrada da soma de 0

para n – 2 × defasagem de (*xi – xi + 1*)2. Isso foi proposto como uma medida da *complexidade* de uma série temporal.

• A função friedrich_coefficients() retorna coeficientes de um modelo ajustado para descrever movimentos não lineares complexos.

Contagens de compactação de histórico

Esta categoria compreende recursos como:

• A soma dos valores em uma série temporal que ocorre mais de uma vez

• O comprimento da subsequência consecutiva mais longa que está acima ou abaixo da média

• A ocorrência mais antiga dentro da série temporal do mínimo ou máximo

valor

Um módulo como o tsfresh pode ajudar você a economizar tempo e escolher implementações eficientes para seleção de recursos. Também pode educá-lo sobre maneiras de descrever dados que podem ser relevantes, mas que você talvez não tenha encontrado em sua própria pesquisa. Existem vários outros benefícios em usar um módulo, como

sempre há quando você combina sua análise com ferramentas de código aberto e bem avaliadas, incluindo:

• Não há necessidade de reinventar a roda quando você computa recursos padrão. Ao usar uma biblioteca compartilhada, você tem a garantia de alguma verificação de precisão por outros usuários, em vez de duvidar do seu próprio código e ter que verificá-lo.

• Uma biblioteca como esta fornece uma estrutura para calcular recursos, não apenas uma longa lista de recursos. Por exemplo, tsfresh possui uma classe de calculadora de recursos, que você pode usar para estender esta biblioteca para seus próprios propósitos, mas com os benefícios de uma estrutura sistemática.

• Essa biblioteca foi projetada para se conectar com consumidores posteriores de recursos, principalmente com o sklearn, para que seus recursos possam ser facilmente passados para modelos de aprendizado de máquina.

A biblioteca tsfresh tem um toque particularmente técnico, pois muitos dos recursos são derivados de ideias sobre a análise de dados experimentais científicos.

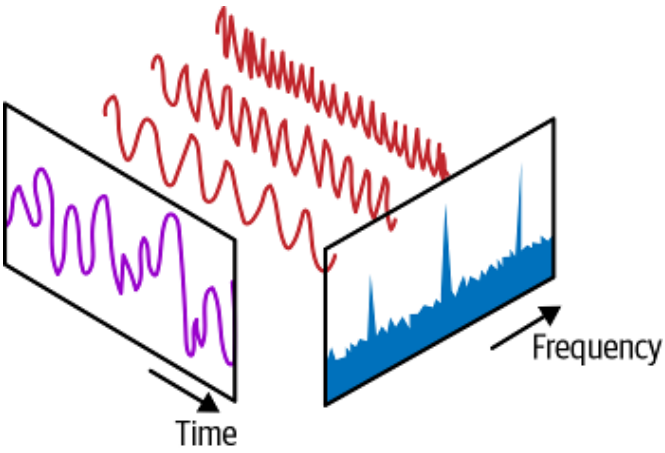
A plataforma de análise de série temporal de Césio

Um catálogo mais acessível, mas igualmente extenso, de recursos gerados é a lista

implementado na biblioteca Césio. A lista atual está disponível em sua documentação , e a seguir selecionamos alguns recursos interessantes para discussão e inspeção. O

categorias gerais são divididas no código-fonte , mas as detalhamos melhor aqui:

Um catálogo de lugares para encontrar recursos de inspiração | 245



• Recursos que descrevem a distribuição geral dos valores dos dados, independentemente de suas relações temporais. Esta categoria pode incluir um conjunto diversificado de recursos que, no entanto, são independentes do tempo:

— Quantos picos locais existem em um histograma dos dados?

— Qual porcentagem dos pontos de dados está dentro de uma janela fixa de valores próximos à mediana dos dados?

• Recursos que descrevem a distribuição do tempo dos dados:

— Recursos que consideram a distribuição do tempo entre as medições como

própria distribuição e calcular estatísticas semelhantes às que acabamos de descrever, agora sobre a distribuição de diferenças de tempo em vez de valores de dados

— Recursos que calculam a probabilidade de que a próxima observação ocorra dentro de *n* intervalos de tempo, dada a distribuição observada

• Características que descrevem medidas da periodicidade do comportamento dentro da série temporal. Muitas vezes essas características estão associadas ao *periodograma Lomb-Scargle* .

Periodogramas

Um periodograma de uma série temporal é uma estimativa de quanto diferentes frequências subjacentes contribuem para a série temporal. Um periodograma perfeito poderia traduzir uma série temporal do espaço de fase de valor de tempo, com o qual trabalhamos mais comumente, para um espaço de frequência de potência em que uma série temporal seria descrita em termos de quanto dela foi moldada por processos recorrentes em vários momentos. carimbos de data/hora. Se você não está satisfeito

miliar com esse conceito, a Figura 8-1 o mostra visualmente.

Figura 8-1. Os mesmos dados representados no domínio do tempo, onde aparecem semelhantes a uma soma de séries temporais senoidais. No domínio da frequência, vemos todo o carregamento de massa espectral nas três frequências que caracterizam os senos que foram somados para produzir o Series.

Existem muitas maneiras de calcular periodogramas. Uma definição e método computacional é a *transformada de Fourier* da função de autocorrelação. Uma maneira mais intuitiva de pensar sobre a criação de um periodograma é ajustar os mínimos quadrados dos dados originais a um conjunto de curvas senoidais de frequências variadas.

Este último é o que faz o muito alardeado método Lomb-Scargle. O método Lomb-Scargle constrói um periodograma para dados de séries temporais amostrados irregularmente, ao contrário de um periodograma tradicional, que assume uma série temporal amostrada regularmente. Lomb, e depois Scargle, desenvolveram técnicas para estudar séries temporais irregulares com este método e estabeleceram que as propriedades estatísticas do periodograma construído a partir de tais dados tinham as mesmas propriedades estatísticas de um periodograma tradicional amostrado a partir de dados de séries temporais regularmente amostrados. Os avanços de Lomb e Scargle permitiram o estudo de muitos problemas de ciências naturais para os quais séries temporais amostradas irregularmente são

inevitáveis, como dados climáticos, dados astrofísicos e dados geológicos.

Os recursos que acabamos de descrever podem ser calculados ao longo de toda a série temporal ou como funções de janela contínua ou expansível. Dado o que aprendemos nos capítulos anteriores sobre a mecânica de codificação de funções de janela de rolagem e expansão, certamente poderíamos implementar esses recursos nós mesmos e estamos em uma boa posição para entender a documentação e o significado do que esta biblioteca faz. Nesse caso, estaríamos aplicando funções de janela rolante para resumir os dados, em vez de limpá-los. As mesmas técnicas de análise de séries temporais são utilizadas em muitos casos diferentes, mas igualmente úteis.

A biblioteca de césio fornece funcionalidade suplementar além da geração de recursos. Por exemplo, inclui uma GUI baseada na web para realizar a geração de recursos e também se integra ao sklearn.

Se você experimentar essas bibliotecas em seus dados, notará que a geração de séries temporais consome muito tempo. Por esse motivo, você deve pensar cuidadosamente sobre quantos recursos você precisa gerar para seus dados e quando realmente faz sentido gerar recursos automaticamente, em vez de desenvolver cuidadosamente os seus próprios.

Muitos dos recursos gerados por essas bibliotecas são computacionalmente desgastantes e—

dada a extensão das listas de recursos, muitas vezes não abordam pontos de interesse para a pergunta que você está tentando responder. Com algum conhecimento do domínio, você pode até reconhecer que um determinado tipo de recurso é irrelevante, barulhento ou não preditivo.

Não calcule esses recursos inúteis desnecessariamente. Isso desacelerará toda a sua análise sem adicionar qualquer clareza. Bibliotecas de geração automática de recursos são úteis, mas devem ser usadas criteriosamente, e não indiscriminadamente.

Pacote tsfeatures do R

tsfeatures, desenvolvido por Rob Hyndman et al., é um pacote R conveniente para gerar uma variedade de recursos de série temporal comumente usados e úteis. A documentação inclui uma lista dos recursos, que incluem funções úteis como:

• acf_features() e pacf_features(), cada qual calcula um número de valores relacionados, dando uma noção geral de quão importante é a autocorrelação no comportamento de uma série. Para a função acf_features(), a documentação descreve os seguintes valores de retorno: “Um vetor de 6 valores: primeiro coeficiente de autocorrelação e soma do quadrado dos primeiros 10 coeficientes de autocorrelação da série original, série primeira diferenciada e série duplamente diferenciada. Para dados sazonais, o coeficiente de autocorrelação no primeiro desfasamento sazonal também é retornado.”

• lumpiness() e stable(), que são funções orientadas por janelas lado a lado, e

max_level_shift() e max_var_shift(), que são controlados por janelas rolantes

funções. Em cada um desses casos, diferenças e estatísticas de medição de variedade são aplicadas a valores medidos em janelas sobrepostas (rolantes) ou não sobrepostas (ladrilhadas) da série temporal.

• unitroot_kpss() e unitroot_pp() .

O pacote tsfeatures consolida e inclui de forma útil o trabalho de uma variedade de projetos acadêmicos relacionados ao estudo de recursos de séries temporais, bem como de outros esforços contínuos para melhorar o processo de criação de recursos de séries temporais que são úteis em vários domínios. Esses incluem:

• compengine() calcula os mesmos recursos de série temporal desenvolvidos pelo comp-

projeto engine.org, que foi considerado útil em uma ampla variedade de dados de séries temporais de muitos domínios. 1

• Vários recursos emprestados do pacote hctsa , que se destina a executar análises de séries temporais altamente comparativas no Matlab. Alguns desses recursos estão em: autocorr_features(), firstmin_ac(), pred_features() e trev_num().

Você também encontrará outros lendo a documentação.

A documentação do tsfeatures também inclui ilustrações úteis dos usos e resultados de cada uma das funções geradoras de recursos e uma extensa bibliografia 1 Os leitores interessados que desejam explorar os detalhes devem consultar o conjunto Catch22 de recursos de séries temporais,

que se mostraram úteis em uma ampla variedade de conjuntos de dados de séries temporais. Reduzir o conjunto de recursos de mais de 4.000 para apenas 22 reduziu o tempo de cálculo em um fator de 1.000, com uma redução na precisão em uma tarefa de classificação de apenas 7%. Também é bastante educativo ler sobre o pipeline que os pesquisadores usaram para selecionar recursos e garantir um conjunto de recursos relativamente independentes, mas ainda assim precisos, desde seu conjunto inicial.

links para trabalhos relacionados de estatística e aprendizado de máquina no conjunto de recursos de série temporal disponível no pacote.

Exemplos de recursos específicos de domínio

Outra fonte de inspiração pode vir de recursos específicos de domínio que foram desenvolvidos para uma variedade de dados de séries temporais. Freqüentemente, esses recursos

foram desenvolvidos ao longo de décadas, seja a partir de heurísticas que funcionam empiricamente, mesmo que não sejam bem compreendidas, ou a partir do conhecimento científico sobre como funciona a mecânica subjacente de um sistema.

A seguir, revisamos alguns recursos específicos de domínio para dois tipos de dados de série temporal: financeiros e de saúde.

Indicadores técnicos do mercado de ações

Os indicadores técnicos do mercado de ações são provavelmente os conjuntos de indicadores mais amplamente documentados e formalizados usados para uma aplicação de série temporal específica de domínio. Os economistas que estudam dados de séries temporais ao longo do último século têm uma extensa lista de recursos que normalmente usam para quantificar séries temporais nos mercados financeiros e fazer previsões. Mesmo que você não tenha interesse nos mercados financeiros, esta lista é inspiradora ao mostrar como uma lista de recursos específicos de um domínio também pode ser bastante extensa, descritiva e criativa.

Uma lista não exaustiva desses recursos está incluída aqui para que você possa ter uma ideia de quão complexos e altamente específicos para os mercados financeiros são esses indicadores. Dada a complexidade, não é surpresa que as pessoas possam passar toda a sua carreira a tentar compreender como estes “sinais” podem prever a ascensão e queda dos mercados financeiros.

Índice de Força Relativa (RSI)

Esta medida é igual a $100 – 100 / (1 + RS)$ onde RS é a razão entre o ganho médio durante um período de “alta” (aumento de preços) dividido pela perda média durante um período de “alta” (aumento de preços).

período de “baixa” (preços decrescentes) e o período de retrospectiva para esses períodos de alta e

períodos de inatividade é um parâmetro de entrada, para que você possa ter diferentes valores de RSI para diferentes períodos de lookback. Os traders desenvolveram regras práticas sobre quais valores de corte do RSI indicam que um ativo está subvalorizado ou sobrevalorizado em relação ao seu verdadeiro valor. O RSI é conhecido como um "indicador de impulso" porque se baseia em

medidas do movimento de um ativo.

Convergência/divergência de médias móveis (MACD)

Este indicador é composto por três séries temporais:

• A série temporal MACD é uma série temporal da diferença entre um período curto

média móvel exponencial de longo prazo do ativo ("rápida") e uma exposição de longo prazo

média móvel nencial do ativo ("slow").

Um catálogo de lugares para encontrar recursos de inspiração | 249

• A série temporal "média" é uma média móvel exponencial do MACD

série temporal.

• A série temporal de "divergência" é a diferença entre a série temporal MACD e a série temporal "média". Este é o valor normalmente utilizado para fazer previsões financeiras. As outras séries de entrada (MACD e média) são normalmente

aliado apenas preparado para criar a série "divergência".

Fluxo de dinheiro Chaikin (CMF)

Este indicador mede a direção da tendência dos gastos. Para calculá-lo:

• Calcule o multiplicador do fluxo de dinheiro: ((*Close* – *Low*) – (*High – Close*)) / (*High*

- *Baixo*).

• Calcule o volume do fluxo de dinheiro, que é o volume de negociação do dia multi-

aplicado pelo multiplicador do fluxo de dinheiro.

• Some esse volume de fluxo de dinheiro para um determinado período de dias e divida-o pelo volume para esse mesmo período de dias. Este indicador é um "oscilador" que varia entre –1 e 1. Indica "pressão de compra" ou "pressão de venda".

uma medida da direção do mercado.

Como você pode ver nesta pequena amostra dos muitos recursos técnicos que podem ser construídos a partir de séries temporais financeiras simples, há muitas maneiras de descrever uma série temporal. Os mercados financeiros são um domínio particularmente rico e muito estudado para geração de recursos. 2

Série temporal de saúde

A saúde é outra área em que os recursos de série temporal têm significados e até nomes específicos de domínio. Como discutimos no Capítulo 1 , os dados de saúde oferecem uma ampla gama de dados de séries temporais. Um exemplo são os dados do ECG (ver Figura 8-2). Ler EKGs é ambos

uma ciência e uma arte, e vários recursos são identificados manualmente pelos médicos e usados para ler uma série temporal. Se você fosse selecionar recursos para um estudo de aprendizado de máquina em dados de EKG, certamente gostaria de começar estudando esses recursos e conversando com um médico experiente para entender a finalidade dos recursos e o que eles indicam.

2 Para obter um extenso catálogo de recursos que podem ser usados em aprendizado de máquina para séries temporais de mercados financeiros, consulte a postagem do

blog Kaggle . Infelizmente, não parece que esse código tenha tido grande sucesso, mas é um excelente exemplo de preparação exaustiva de recursos potencialmente úteis com base na relevância de seu domínio.

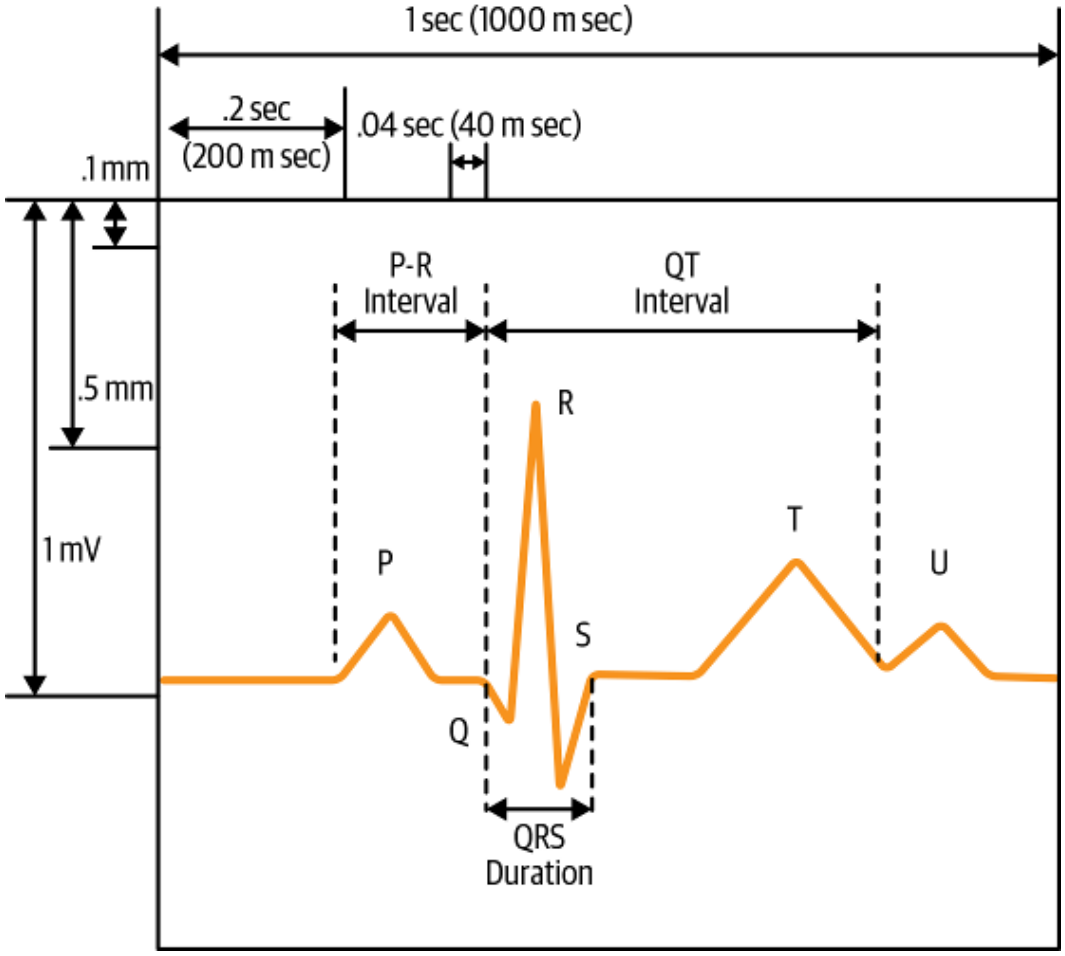


Figura 8-2. Exemplos de recursos de série temporal que os profissionais médicos usam para ler a hora do EKG dados da série.

Da mesma forma, se você estiver analisando dados de séries temporais de glicemia de alta resolução, também será útil compreender o tipo de padrões que tendem a afetar os dados diários, bem como como um profissional de saúde entenderia e rotularia esses dados (ver Figura 8-3).

As duas séries temporais ilustradas nas Figuras 8-2 e 8-3 fornecem bons exemplos de quando estaríamos interessados em encontrar máximos locais ou distâncias entre eles. Conseqüentemente, podemos facilmente imaginar recursos específicos das bibliotecas tsfresh ou césio que são relevantes para séries temporais de saúde, dado o conhecimento específico do domínio.

Um catálogo de lugares para encontrar recursos de inspiração | 251

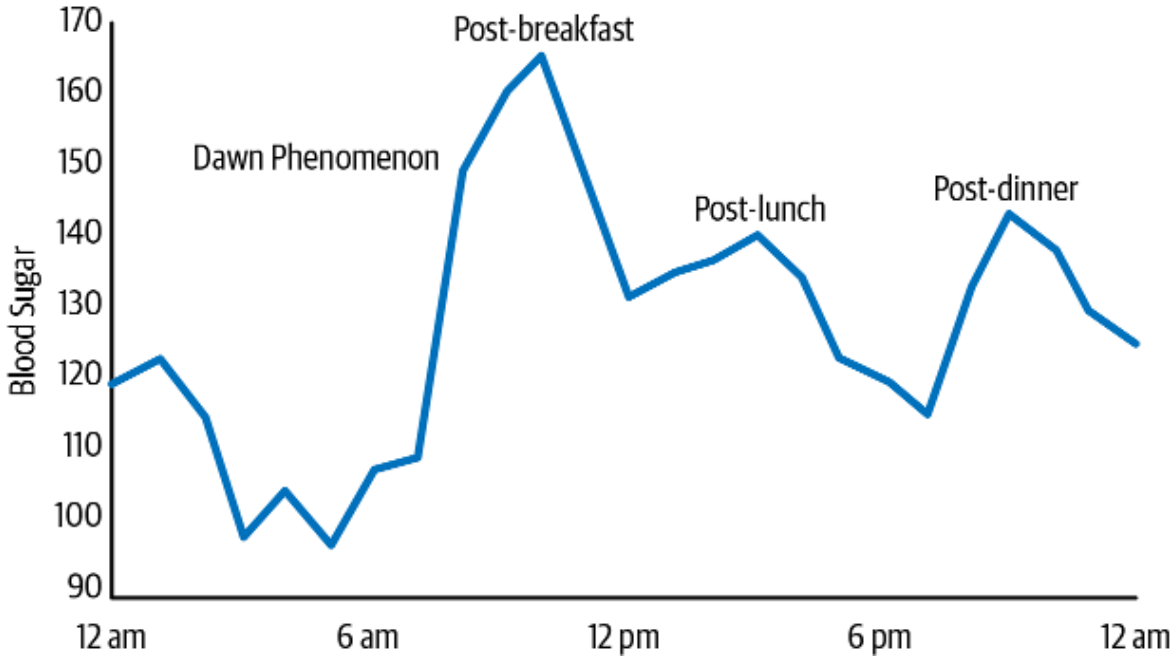


Figura 8-3. Em uma série temporal de níveis de glicose no sangue de um dia, podemos ver que há quatro características que a maioria dos profissionais de saúde identificaria em um dia normal, uma das quais é não está relacionado à comida, mas é conhecido como o "fenômeno do amanhecer". Isso é algo que nós gostaria de considerar a identificação em um ou vários recursos.

Como selecionar recursos depois de gerá-los

Suponha que você tenha gerado muitos recursos automaticamente para descrever seu grande conjunto de dados de série temporal. Talvez você não consiga visualizar todos os recursos propostos em uma primeira passagem pelos dados, portanto, pode ser útil complementar o uso da

geração automática de recursos com o uso da *seleção automática de recursos* . Um algoritmo útil de seleção de recursos é o algoritmo FRESH, que é implementado no pacote tsfresh descrito anteriormente. FRESH significa extração de recursos com base em testes de hipóteses escalonáveis.

O algoritmo FRESH é motivado pela quantidade cada vez maior de dados de séries temporais disponíveis, muitas vezes armazenados de maneira distribuída, o que facilita a paralelização da computação. O algoritmo avalia a importância de cada recurso de entrada em relação a uma variável alvo por meio do cálculo de um valor *p* para cada recurso. Uma vez calculados, os valores *p* por recurso são avaliados em conjunto por meio do procedimento Benjamini-Yekutieli, que determina quais recursos manter com base em parâmetros de entrada sobre taxas de erro aceitáveis e similares. O *procedimento Benjamini-Yekutieli* é um método para limitar o número de falsos positivos descobertos durante o teste de hipóteses usado para produzir os valores *p* na etapa inicial do algoritmo FRESH.

Implementar essas etapas por conta própria seria uma tarefa bastante difícil, mas podemos fazer isso em algumas linhas de código via tsfresh. Aqui seguimos o código utilizado em uma ilustração na documentação do módulo. Primeiro, baixamos dados de séries temporais relacionados a falhas de execução do robô:

Pitão

>> **de tsfresh.examples.robot_execution_failures importar** download_robot_execution_failures,

load_robot_execution_failures

>> download_robot_execution_failures()

>> série temporal, y = load_robot_execution_failures()

Em seguida, extraímos os recursos sem precisar especificá-los, pois o pacote calcula automaticamente todos os recursos. Nesse sentido, vai contra o conselho dado neste capítulo de ser extremamente inclusivo sem preocupação com recursos computacionais. Neste conjunto de dados de teste não há muitos pontos de dados, mas você provavelmente não desejaria implantá-los cegamente em seu conjunto de dados sem reduzi-lo a um número razoável de pontos:

Pitão

>> **de fresco importar** extract_features

>> extraídas_features = extract_features(série temporal,

coluna_id = "id",

coluna_sort = "tempo")

Embora tsfresh forneça uma maneira de especificar quais recursos você gostaria de calcular, neste exemplo simplesmente optamos por incluir todos os recursos. Você também pode definir manualmente os parâmetros para os recursos que levam os parâmetros em consideração para sua calibração.

cálculo em vez de usar os padrões. Tudo isso está descrito e ilustrado no documento

umentação.

Se você realizar uma extração completa, como fizemos com os dados de exemplo fornecidos por tsfresh, poderá ver que existem vários recursos calculados:

Pitão

>> extraído_features.columns

Index(['F_x__abs_energia', 'F_x__absolute_sum_of_changes',

'F_x__agg_autocorrelação__f_agg_"média"',

'F_x__agg_autocorrelação__f_agg_"mediana"',

'F_x__agg_autocorrelação__f_agg_"var"',

'F_x__agg_linear_trend__f_agg_"max"__chunk_len_10__attr_

"interceptar"',

'F_x__agg_linear_trend__f_agg_"max"__chunk_len_10__attr_

"rvalor"',

'F_x__agg_linear_trend__f_agg_"max"__chunk_len_10__attr_

"declive"',

'F_x__agg_linear_trend__f_agg_"max"__chunk_len_10__attr_

"stderr"',

'F_x__agg_linear_trend__f_agg_"max"__chunk_len_50__attr_

"interceptar"',

...

'T_z__time_reversal_asymmetry_statistic__lag_1',

'T_z__time_reversal_asymmetry_statistic__lag_2',

'T_z__time_reversal_asymmetry_statistic__lag_3',

'T_z__value_count__value_-inf', 'T_z__value_count__value_0',

'T_z__value_count__value_1', 'T_z__value_count__value_inf',



'T_z__value_count__value_nan', 'T_z__variância',

'T_z__variance_larger_than_standard_deviation'],

dtype='objeto', nome='variável', comprimento=4764)

Existem 4.764 colunas. Este é um número muito maior de recursos do que poderíamos ter calculado manualmente, mas também consome muito tempo para ser executado em um conjunto de dados realista.

Ao decidir como e quando implantar um conjunto tão grande de recursos, tente ser realista em relação ao seu poder computacional e à sua capacidade de analisar cuidadosamente os resultados.

Lembre-se de que, para dados de séries temporais, os valores discrepantes podem ser particularmente influentes de uma forma desagradável e inútil para análises subsequentes. Você desejará garantir que os recursos escolhidos sejam resistentes a valores discrepantes.

Embora o algoritmo FRESH seja útil para contabilizar a dependência entre os recursos, é um tanto difícil de raciocinar. Também podemos usar uma técnica de seleção de recursos mais tradicional e transparente, *a eliminação recursiva de recursos* (RFE). Podemos usar RFE para complementar nosso uso do algoritmo FRESH e melhorar nossa compreensão do grau de diferença entre os recursos selecionados pelo FRESH

algoritmo e aqueles não selecionados.

RFE descreve uma abordagem incremental para seleção de recursos

em que os recursos são gradualmente eliminados de um ambiente mais inclusivo

modelo, criando um modelo menos inclusivo até o número mínimo

número de recursos a serem incluídos, que é definido no início da seleção

procedimento de ção.

Esta técnica é conhecida como *seleção para trás* porque você inicia

com o modelo mais inclusivo e retroceder para um modelo mais simples

modelo. Em contraste, na *seleção direta* os recursos são incrementalmente

adicionado até o número máximo de recursos especificados, ou alguns

outro critério de parada é alcançado.

Podemos usar RFE tanto para seleção de recursos quanto como forma de classificar a importância dos recursos. Para executar um experimento, combinamos 10 recursos selecionados aleatoriamente da lista de recursos mantidos pelo algoritmo FRESH com 10 recursos selecionados aleatoriamente da lista de recursos rejeitados pelo algoritmo FRESH:

##R

>> x_idx =
random.sample(range(len(features_filtered.columns)), 10)

>> selX = recursos_filtered.iloc[:, x_idx].valores

>> recursos_não selecionados = lista (set
(recursos_extraídos.columns)

.diferença(set(features_filtered.columns)))

>> recursos_não selecionados = random.sample (recursos não selecionados, 10)

>> unsel_x_idx = [idx para (idx, val) em enumerar(

extraídas_features.columns) se val em unselected_features]

>> unselX = extraídas_features.iloc[:, unsel_x_idx].valores

>> misto_X = np.hstack([selX, unselX])

Com este conjunto de 20 recursos, podemos então realizar RFE para ter uma noção da importância classificada desses recursos para o conjunto de dados e para o modelo que usamos no RFE:

>> svc = SVC(kernel="linear", C=1)

>> rfe = RFE(estimador=svc, n_features_to_select=1, passo=1)

>> rfe.fit(misto_X, y)

>> rfe.ranking_

matriz([9, 12, 8, 1, 2, 3, 6, 4, 10, 11,

16, 5, 15, 14, 7, 13, 17, 18, 19, 20])

Aqui podemos ver as classificações relativas dos 20 recursos que inserimos no algoritmo RFE. Esperamos que os primeiros 10 recursos – aqueles selecionados, entre outros, pelo algoritmo FRESH – sejam classificados em uma posição mais elevada do que aqueles não selecionados pelo FRESH

algoritmo. Este é em grande parte, mas não exclusivamente, o caso. Por exemplo, podemos ver que na segunda metade da matriz, representando as classificações dos recursos não selecionados, temos na verdade o 5º e o 7º recursos mais importantes de todos os 20. No entanto, não esperaríamos

uma combinação perfeita, e os resultados são amplamente consistentes.

Podemos usar RFE nos recursos selecionados como uma forma de selecionar ainda mais nossos recursos. Também podemos usá-lo como uma verificação de integridade se tentarmos ajustar os parâmetros de entrada do algoritmo FRESH ou o número de recursos que estamos gerando como entrada para o FRESH

algoritmo em primeiro lugar.

Observe que o algoritmo FRESH em si é essencialmente livre de parâmetros, portanto, o número e a qualidade dos recursos que inserimos são a melhor maneira de afetar sua saída. O outro parâmetro que definimos para o algoritmo FRESH é o fdr_level, que é a porcentagem de recursos irrelevantes que esperamos após gerar os recursos. O padrão desse parâmetro é 0,05, mas você pode decidir definir esse valor muito mais alto para aprimorar a seletividade da filtragem de recursos, principalmente quando estiver gerando um grande número de recursos sem considerar se eles são apropriados para seu domínio de interesse.

Considerações finais

Neste capítulo, discutimos a motivação para a seleção de recursos, bem como um exemplo simples de como a geração de recursos pode funcionar para converter até mesmo uma série temporal curta em um conjunto de números mais compactado que é quase tão informativo quanto o original. Também vimos exemplos de dois módulos Python projetados para implementar a geração e seleção automatizada de recursos em dados de série temporal, que podem gerar facilmente milhares de recursos de uma série temporal. Como existe o perigo de que muitos dos recursos gerados dessa maneira não sejam especialmente úteis, também analisamos métodos para selecionar os recursos mais úteis para transmitir mais adiante em nosso pipeline analítico, para que a geração de recursos não produza recursos ruidosos ou pouco informativos.

A geração de recursos é útil para vários propósitos:

• Produzir dados posteriores sobre séries temporais em um formato propício para uso em algoritmos de aprendizado de máquina, muitos dos quais são projetados para aceitar conjuntos de recursos por ponto de dados em vez de uma série temporal.

• Resumir dados de séries temporais de uma forma que comprima as observações temporais na abreviação de alguns números e indicadores qualitativos. Isso pode ser útil não apenas para análise, mas também para armazenar dados de séries temporais em um formato mais sucinto e legível em casos em que não precisamos manter a série temporal completa.

• Fornecer um conjunto comum de métricas para descrever e identificar semelhanças entre dados que podem ter sido medidos sob muitas condições diferentes. Ao resumir os nossos dados de forma mais ampla, podemos tornar comparáveis dados que de outra forma não pareceriam fáceis de comparar.

No Capítulo 9 usaremos a geração de recursos para preparar a entrada de dados para uma série de

algoritmos de aprendizado de máquina que dependem da entrada de recursos de séries temporais em vez de dados brutos de séries temporais para fins de classificação e previsão.

Mais recursos

• Na análise de série temporal baseada em recursos:

Ben D. Fulcher, "Análise de série temporal baseada em recursos", eprint arXiv:1709.08055,

2017, https://perma.cc/6LZ6-S3NC.

Este artigo de revisão acessível foi escrito pela mesma pessoa que implementou

Mencionei o algoritmo tsfresh no Matlab. Oferece uma extensa taxonomia de

diferentes tipos de recursos que podem ser implementados em dados de série temporal. Isto

também inclui ilustrações úteis das categorias de recursos discutidas.

Há também ênfase no uso da geração de recursos como forma de exploração de dados para compreender a análise apropriada para uma determinada série temporal

conjunto de dados.

• Na seleção de recursos:

Maximilian Christ et al., "Extração de recursos de série temporal com base em escalabilidade

Testes de hipóteses (tsfresh – um pacote Python)," *Neurocomputação 307 (2018): 72–7, https://oreil.ly/YDBM8.*

Este artigo apresenta o pacote tsfresh Python e inclui FRESH

algoritmo, que foram discutidos neste capítulo. Informações sobre o com-

eficiência putacional de recursos específicos para calcular e amostrar padrões de uso

termos também são apresentados.

Maximilian Christ et al., "Extração de recursos de série temporal distribuída e paralela

for Industrial Big Data Applications", artigo apresentado no ACML Workshop em

Learning on Big Data (WLBD), Hamilton, Nova Zelândia, 16 de novembro de 2016, https://

arxiv.org/pdf/1610.07717.pdf.

Esta explicação mais técnica e teste do algoritmo FRESH pro-

fornece mais detalhes sobre como o algoritmo funciona e por que ele é apropriado

para aplicação industrial de séries temporais e uma extensa lista de informações básicas

lendo na seção de referências.

• Em recursos específicos de domínio:

Wikipedia, "Análise Técnica para Mercados Financeiros", https://perma.cc/8533-

XFSZ.

Este artigo da Wikipedia fornece um histórico de como a análise técnica foi desenvolvida

aberto para os mercados financeiros, bem como uma extensa lista de muitos

indicadores amplamente utilizados.

Amjed S. Al-Fahoum e Ausilah A. Al-Fraihat, "Métodos de recursos de sinal EEG

Extração usando análise linear em domínios de frequência e tempo-frequência,"

ISRN Neurociência 2014, não. 730218 (2014), https://perma.cc/465U-QT53.

Este artigo fornece um exemplo de teste de recursos de série temporal padrão no domínio dos sinais de EEG. Existe toda uma indústria acadêmica de geração de recursos para

séries temporais médicas, e este artigo fornece uma abordagem acessível

exemplo de como isso é feito.

Juan Bautista Cabral et al., "Das GORDURAS aos Pés: Melhorias Adicionais para um

Ferramenta de extração de características astronômicas baseada em aprendizado de máquina e astronomia

e Computação 25 (2018), https://perma.cc/8ZEM-Y892.

Este artigo discute um redesenho recente de um pacote Python projetado para a extração de recursos de dados de séries temporais astronômicas. Embora este documento presuma alguma familiaridade com astronomia e pacotes específicos, ainda é uma leitura útil para se ter uma ideia de como o software é projetado para extração de características e para compreender alguns desafios e mudanças específicos que

têm ocorrido em dados de séries temporais astronômicas em particular.

Alvin Rajkomar et al., "Informações suplementares para escalabilidade e precisão

Aprendizado profundo para registros eletrônicos de saúde", npj Digital Medicine 1, no. 18

(2018), https://perma.cc/2LKM-326C.

Esta informação suplementar refere-se a um interessante artigo publicado por

Google ilustrando um uso altamente bem-sucedido de aprendizagem profunda para a compreensão de registros eletrônicos de saúde. As informações complementares fornecem informações detalhadas

informações sobre como os registros de saúde foram transformados em dados quantitativos

recursos de entrada que podem ser acessados por uma rede neural. Se você não está

familiarizado com aprendizagem profunda, pode valer a pena retornar a este recurso apenas

depois de ler o Capítulo 10 . Além disso, seria uma boa ideia primeiro obter um

compreensão básica de incorporações para aprendizagem profunda.

Mais recursos | 257

• Em periodogramas:

"O Periodograma", notas de aula, Eberly College of Science, Estado da Pensilvânia

Universidade, https://perma.cc/5DRZ-VPR9.

Estas notas de aula do curso da Penn State sobre Análise Aplicada de Séries Temporais

sis oferecem uma introdução ao que é um periodograma, como interpretá-lo e como calculá-lo usando R.

Jacob T. VanderPlas, "Compreendendo o periodograma Lomb-Scargle", Suplemento do Astrophysical Journal Série 236, no. 1 (2018), https://arxiv.org/pdf/

1703.09824.pdf.

Este extenso artigo fornece uma compreensão intuitiva dos estimadores de periodicidade em geral e do método Lomb-Scargle em particular.

258 | Capítulo 8: Gerando e selecionando recursos para uma série temporal

CAPÍTULO 9

Aprendizado de máquina para séries temporais

Neste capítulo, veremos alguns exemplos de aplicação de métodos de aprendizado de máquina à análise de séries temporais. Esta é uma área relativamente nova de análise de séries temporais, mas que se mostrou promissora. Os métodos de aprendizado de máquina que estudaremos não foram originalmente desenvolvidos para dados específicos de séries temporais — ao contrário dos modelos estatísticos que estudamos nos dois capítulos anteriores — mas provaram ser úteis para isso.

Essa mudança para o aprendizado de máquina é uma mudança em relação ao nosso trabalho anterior sobre previsão nos capítulos anteriores deste livro. Até agora nos concentramos em modelos estatísticos para previsões de séries temporais. Ao desenvolver tais modelos, formulamos uma teoria subjacente sobre a dinâmica de uma série temporal e as estatísticas que descrevem o ruído e a incerteza no seu comportamento. Em seguida, usamos a dinâmica hipotética do processo para fazer previsões e também para estimar nosso grau de incerteza sobre as previsões. Com esses métodos, tanto a identificação do modelo quanto a estimativa dos parâmetros exigiram que pensássemos cuidadosamente sobre a melhor maneira de descrever a dinâmica dos nossos dados.

Passamos agora para metodologias nas quais não postulamos um processo subjacente ou quaisquer regras sobre esse processo subjacente. Em vez disso, nos concentramos na identificação de padrões que descrevem o comportamento do processo de maneira relevante para prever o resultado de interesse, como o rótulo de classificação apropriado para uma série temporal. Também consideraremos o aprendizado não supervisionado para séries temporais, na forma de agrupamento de séries temporais.

Cobrimos previsão e classificação com metodologias baseadas em árvores, bem como agrupamento como forma de classificação. No caso de metodologias baseadas em árvores, a formulação de características da nossa série temporal é um passo necessário no caminho da utilização da metodologia, uma vez que as árvores não são uma metodologia “consciente do tempo”, ao contrário, digamos, de um modelo ARIMA.

No caso de clustering e classificação baseada em distância, veremos que temos a opção de usar recursos ou de usar a série temporal original como entrada. Para usar o tempo
259

Se a própria série for usada como entrada, estudamos uma métrica de distância conhecida como *Dynamic Time Warping* , que pode ser aplicada diretamente a séries temporais, preservando todo o conjunto cronológico de informações em nossos dados, em vez de reduzi-lo a um conjunto necessariamente limitado de recursos.

Classificação de série temporal

Nesta seção, trabalharemos com um exemplo de conversão de dados brutos de séries temporais de eletroencefalograma (EEG) em recursos, que por sua vez podem ser usados para algoritmos de aprendizado de máquina. Em seguida, usamos métodos de árvore de decisão para classificar os dados do EEG depois de extrairmos os recursos da série temporal do EEG.

Selecionando e gerando recursos

No capítulo anterior, passamos por uma discussão geral sobre os propósitos da geração de recursos de séries temporais. Também trabalhamos em um breve exemplo de geração de recursos para um conjunto de dados de série temporal por meio de tsfresh. Agora iremos gerar recursos com outro pacote de recursos de série temporal que foi discutido: césio.

Um dos atributos muito úteis do pacote de césio é que ele vem com uma variedade de conjuntos de dados de séries temporais úteis, incluindo um conjunto de dados de EEG originalmente derivado de um

Artigo de pesquisa de 2001 . Neste artigo, você pode ler mais sobre os detalhes da preparação de dados.

Para nossos propósitos, basta saber que as cinco categorias de séries temporais de EEG presentes no conjunto de dados representam segmentos de igual comprimento cortados de

leituras de tempo contínuas de amostras de EEG do seguinte:

• Gravações de EEG de pessoas saudáveis com olhos abertos e fechados (duas categorias separadas)

• Registros de EEG de pacientes com epilepsia durante períodos sem convulsões de duas áreas do cérebro não relacionadas a convulsões (duas categorias separadas)

• Um registro intracraniano de EEG durante uma convulsão (uma categoria).

Baixamos esse conjunto de dados por meio de uma função de conveniência fornecida pelo césio:

```
## Pitão

>>> de césio importar conjuntos de dados

>>> eeg = conjuntos de dados.fetch_andrzejak()
```

Pode ser útil ver primeiro alguns exemplos dos dados que estamos analisando para ter uma ideia de como gostaríamos de classificar essas séries temporais:

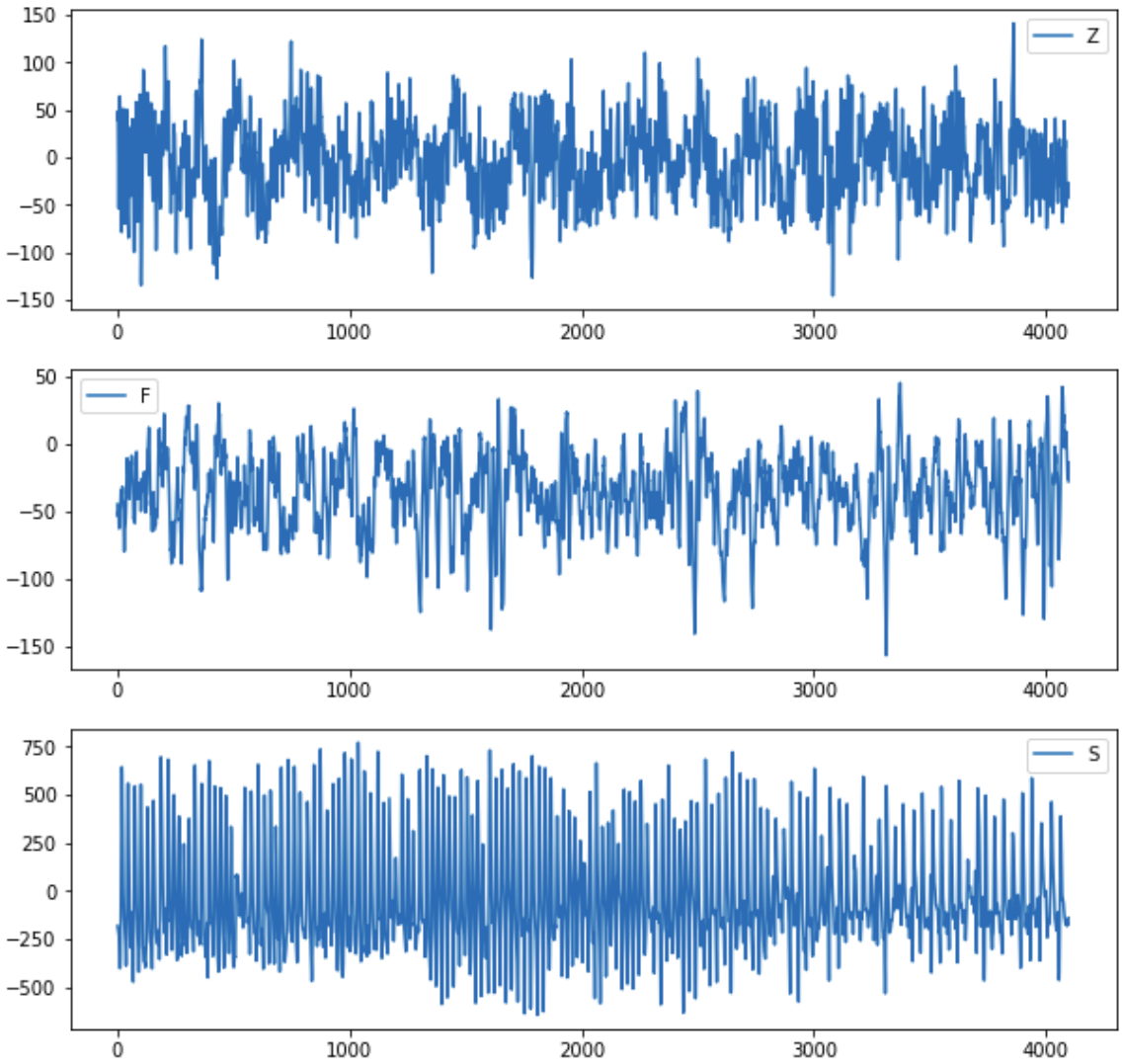
```
## Pitão

>>> plt.subplot(3, 1, 1)

>>> plt.plot(eeg["medições"][0])

>>> plt.legend(eeg['classes'][0])

>>> plt.subplot(3, 1, 2)
```




```
>>> plt.plot(eeg["medições"][300])
>>> plt.legend(eeg['classes'][300])
>>> plt.subplot(3, 1, 3)
>>> plt.plot(eeg["medições"][450])
>>> plt.legend(eeg['classes'][450])
```

Esses gráficos mostram algumas diferenças entre as classes de medição de EEG (ver

Figura 9-1). Não é surpreendente que os gráficos do EEG sejam marcadamente diferentes: eles são

medir a atividade em diferentes partes do cérebro durante diferentes atividades em indivíduos saudáveis e pacientes com epilepsia.

Figura 9-1. Traçamos três amostras selecionadas aleatoriamente do conjunto de dados de EEG. Estes são amostras independentes e não medições contemporâneas de diferentes partes do o mesmo cérebro. Cada uma é uma medição de série temporal independente feita de um paciente em um horário diferente.

Essas visualizações fornecem orientação para a geração de recursos. Por exemplo, classe Z

e G parecem ter dados menos distorcidos do que a classe S. Além disso, cada classe tem uma faixa de valores bem diferente, como podemos ver inspecionando o eixo y. Isto sugere que um recurso de amplitude pode ser útil. Além disso, não é apenas a amplitude geral, mas a distribuição geral dos pontos que parece caracteristicamente diferente nas três **Classificações de Séries Temporais | 261**

Aulas. Usaremos esses recursos e alguns outros em nossa análise e a seguir mostraremos o código para gerar tais recursos.

A visualização encontra o aprendizado de máquina

Esses dados de EEG seriam uma ótima oportunidade para fazer uso de um histograma 2D. Em particular, um histograma 2D formaria uma imagem composta útil de cada tipo de série temporal e poderia confirmar (ou negar) a provável representatividade das características que acabamos de discutir, como estabelecer se a amplitude ou a inclinação é uma diferença consistente entre as classes. .

Também seriam úteis os histogramas univariados, formando um histograma por classe. Você também pode considerar dar um passo adiante para calcular e traçar uma *estimativa de densidade do kernel* para cada classe.

Uma estimativa de densidade do kernel é como um histograma, mas pode ser moldada para ter propriedades como continuidade e suavidade, transformando efetivamente os dados brutos do histograma em uma estimativa suave, mas não paramétrica, de uma distribuição possível para a variável subjacente. A Wikipedia oferece uma boa visão geral da estimativa de densidade do kernel, que pode ser

realizado por vários meios, como via scikit-learn em Python ou a função den sity() no pacote R stats.

Aqui, geramos os recursos com césio:

Pitão

>>> **de césio importar** featurerize.featurize_time_series **como** pés

>>> features_to_use = ["amplitude",

>>> "porcentagem_além_1_std",

>>> "percent_close_to_median",

>>> "inclinar",

>>> "inclinação_máx"]

>>> fset_césio = ft(vezes = eeg["vezes"],

>>> valores = eeg["medições"],

>>> erros = Nenhum,

>>> recursos_para_usar = recursos_para_usar,

>>> agendador = Nenhum)

Isso produz nossos recursos conforme mostrado na Figura 9-2 , uma captura de tela de um notebook Jupyter.

```
fset_cesium.head()
```

feature	amplitude	percent_beyond_1_std	percent_close_to_median	skew	max_slope
channel	0	0	0	0	0
0	143.5	0.327313	0.505004	0.032805	11107.796610
1	211.5	0.290212	0.640469	-0.092715	20653.559322
2	165.0	0.302660	0.515987	-0.004100	13537.627119
3	171.5	0.300952	0.541128	0.063678	17008.813559
4	170.0	0.305101	0.566268	0.142753	13016.949153

Figura 9-2. Valores numéricos de recursos que geramos para as primeiras amostras em nosso conjunto de dados.

Observe que muitos desses valores não são normalizados, então isso seria algo que gostaríamos de ter em mente se estivéssemos usando uma técnica que assumisse entradas normalizadas.

Devemos também confirmar que entendemos o que as nossas características indicam e que a nossa compreensão corresponde ao que o césio calcula. Como ilustração da verificação de erros e afirmação de bom senso, podemos verificar o percent_beyond_1_std para uma amostra de série temporal:

Pitão

>>> np.std(eeg_small["medições"][0])

40.411

>>> np.mean(eeg_small["medições"][0])

-4.132

>>> amostra_ts = eeg_small["medições"][0]

>>> sz = len(amostra_ts)

```
>>> ll = -4,13 - 40,4

>>> ul = -4,13 + 40,4

>>> quals = [i para i no intervalo (sz) se sample_ts[i] < ll
ou sample_ts[i] > ul ]

>>> len(quals)/len(ser)
```

0.327 *## isso confere com o recurso gerado na Figura 9-2*





Os recursos devem ser ergódicos

Ao escolher recursos a serem gerados para uma série temporal, certifique-se de selecionar

características que são *ergódicas* , o que significa que os valores medidos serão

cada um converge para um valor estável à medida que mais dados do mesmo processo

é coletado. Um exemplo em que este não é o caso é um

caminhada, para a qual a medição da média do processo é

sem sentido e não será ergódico. A média de um passeio aleatório

não convergirá para um valor específico.

Nos dados de EEG que traçamos, diferentes subamostras de qualquer

as séries temporais são claramente comparáveis e a própria série é fracamente estacionária.

temporário, então os recursos que geramos fazem sentido.

Você deve ser capaz de verificar qualquer um dos recursos que está usando. Esta é uma simples questão de análise responsável. Você não deve fornecer ao seu algoritmo informações que não possa compreender, explicar e verificar.

Não abuse das bibliotecas de geração de recursos. Não é difícil escrever

seu próprio código para gerar recursos. Se você trabalha em um domínio onde

um determinado conjunto de recursos é gerado frequentemente e no mesmo com-

binações repetidamente, você deve escrever seu próprio código, mesmo que

inicialmente usei um pacote.

Você será capaz de otimizar seu código de maneira que os autores de um

o pacote exploratório geral não pode. Por exemplo, se você tiver vários

recursos gerais que dependem do cálculo da média de uma série de tempo

ies, você pode criar código para calcular essa média apenas uma vez, em vez

mais de uma vez por recurso separado calculado.

Métodos de árvore de decisão

Os métodos baseados em árvores refletem a forma como os humanos tomam decisões: um passo de cada vez e de uma forma altamente não linear. Eles refletem a forma como tomamos decisões complicadas: um passo de cada vez, pensando em como uma variável deve afetar a nossa decisão, e depois outra, como um fluxograma.

Presumo que você já tenha trabalhado com árvores de decisão ou possa intuir rapidamente o que são árvores de decisão. Se você precisar de mais suporte, faça uma pausa aqui e confira alguns back-

leitura terrestre .

A Figura 9-3 mostra um exemplo simples de árvore de decisão que pode ser usada para fazer uma estimativa do peso corporal de alguém.

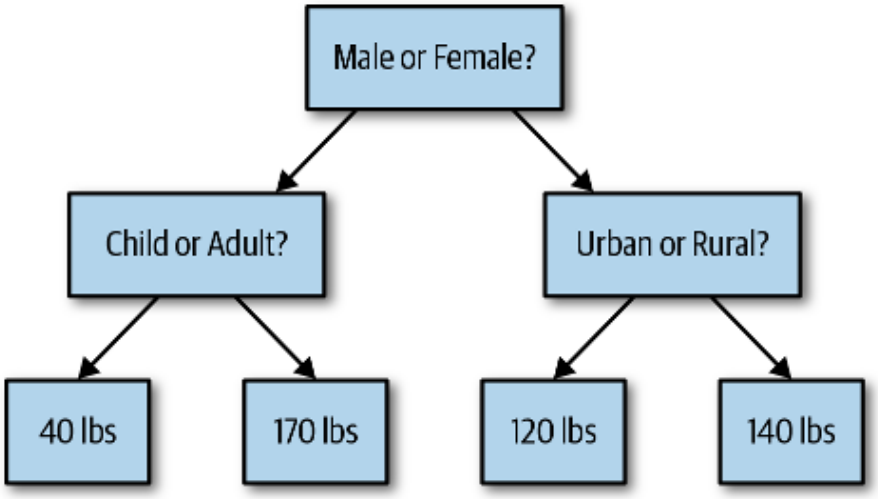


Figura 9-3. Nesta árvore de regressão simples, uma série de ramificações lógicas são usadas para chegar com um peso previsto para um ser humano. Este é um modelo bruto, mas ilustra a abordagem não linear e variável que mesmo um simples modelo de árvore pode aplicar a uma regressão problema.

Existem exemplos abundantes de humanos se comportando como uma árvore de decisão ao analisar dados de séries temporais. Por exemplo, um negociante discricionário do mercado de ações pode usar indicadores técnicos, mas provavelmente os usará de forma hierárquica em série, assim como uma árvore – primeiro perguntando, por exemplo, em qual direção o momentum está tendendo de acordo com um indicador técnico antes de perguntar como a volatilidade está evoluindo ao longo do tempo, com a resposta a esta segunda pergunta interagindo com a primeira de uma forma não linear em forma de árvore. Provavelmente eles têm algo como uma estrutura de árvore de decisão em seu cérebro que usam para fazer previsões sobre como o mercado se moverá.

Da mesma forma, quando um profissional médico lê um EEG ou ECG, não é incomum procurar primeiro a presença de uma característica antes de considerar outra, trabalhando sequencialmente através de uma série de fatores. Se uma característica estiver presente e outra não, isso levará a um diagnóstico diferente do caso inverso e, portanto, a uma previsão diferente em relação ao prognóstico do paciente.

Usaremos os recursos que geramos a partir dos dados de EEG como entradas para dois métodos diferentes de árvore de decisão, florestas aleatórias e árvores com gradiente aumentado, cada um dos quais pode ser usado para tarefas de classificação. Nossa tarefa será classificar os dados de EEG que discutimos apenas com base nas características que geramos a partir dos dados brutos.

Floresta aleatória

Uma floresta aleatória é um modelo no qual usamos não uma árvore de decisão, mas muitas. Nossa classificação ou regressão é o resultado da média dos resultados dessas árvores. As florestas aleatórias olham para a “sabedoria da multidão”, onde a multidão é composta por muitos modelos simples, nenhum dos quais pode ser particularmente bom, mas todos juntos muitas vezes superam uma árvore de decisão altamente refinada, mas única.

A ideia de reunir uma coleção de modelos para produzir uma previsão, em vez de apenas se esforçar para encontrar o “melhor” modelo, foi articulada já em 1969 no *The Combination of Forecasts* , um artigo de pesquisa de dois veneráveis estatísticos, JM Bates e CWJ Granger. Esse artigo mostrou que a combinação de duas previsões separadas de dados de passageiros de companhias aéreas poderia levar a um modelo com erro quadrático médio mais baixo, um resultado surpreendente e, na época, não intuitivo. A geração mais jovem de analistas, que muitas vezes trabalhou na análise de dados através da aprendizagem automática em vez de estatística, considera esta ideia intuitiva em vez de perturbadora, e a floresta aleatória tornou-se um burro de carga para todos os tipos de problemas de previsão.

Uma floresta aleatória é construída de acordo com parâmetros que especificam o número de árvores a serem treinadas, bem como a profundidade máxima permitida dessas árvores. Então, para cada árvore individual, uma amostra aleatória dos dados e de seus recursos é usada para treinar essa árvore. As árvores são geralmente parametrizadas para serem bastante simples, de modo que o overfitting possa ser evitado e o modelo possa calcular a média de muitos modelos gerais, nenhum dos quais é especialmente bom, mas todos são suficientemente gerais para evitar “armadilhas” nos dados.

Conforme mencionado anteriormente, inseriremos os recursos calculados para cada amostra de série temporal no modelo como nossos resultados de treinamento. Em teoria, poderíamos pensar em maneiras de inserir nossos dados brutos de série temporal, em vez dos recursos computados, mas há vários problemas com isso:

• Seria complicado lidar com séries temporais de duração desigual.

• Um número tão grande de entradas (igual ou próximo ao número de passos de tempo) resultaria em modelos e treinamento computacionalmente dispendiosos.

• Supondo que nenhum intervalo de tempo específico seja muito importante (uma vez que qualquer intervalo de tempo estaria correlacionado a um recurso), haveria muito ruído e

muito pouco sinal para treinar a partir da perspectiva da árvore, que seria vista a cada vez passo como entrada.

Portanto, florestas aleatórias não são uma boa ferramenta para trabalhar com dados de séries temporais em sua forma bruta, mas podem ser úteis para trabalhar com eles depois de compactados em recursos de resumo. Aqui estão alguns motivos específicos:

• Do ponto de vista da eficiência/recursos computacionais, é maravilhoso pensar que podemos destilar séries temporais extremamente longas em um punhado de características e encontrar um modelo com precisão razoável.

• É útil que uma floresta aleatória reduza o risco de overfitting. Como discutimos anteriormente, o overfitting é particularmente um problema para a análise de séries temporais devido às sinergias infelizes entre o overfitting e o lookahead. Ter uma metodologia deliberadamente estúpida/simples combate algumas dessas preocupações.

• As florestas aleatórias podem ser particularmente adequadas para dados de séries temporais para os quais não temos um modelo funcional ou hipótese sobre a mecânica subjacente do processo.

Como regra geral, os analistas tiveram mais sucesso ao implantar florestas aleatórias para casos de classificação de séries temporais do que para casos de previsão de séries temporais. Em contraste, o próximo método que discutiremos, árvores com gradiente aumentado, tem sido amplamente bem-sucedido em ambas as tarefas.

Árvores com gradiente aumentado

Boosting é outra maneira de construir um conjunto de preditores. O Boosting cria modelos sequencialmente com a ideia de que os modelos posteriores devem corrigir os erros dos modelos anteriores e que o desajuste dos dados pelos modelos anteriores deve ser mais fortemente ponderado pelos modelos posteriores.

As árvores com gradiente aumentado tornaram-se a metodologia de impulso preferida e particularmente bem-sucedida para séries temporais, como evidenciado por seu sucesso em algumas competições de ciência de dados nos últimos anos.

O XGBoost funciona construindo árvores sequencialmente, com cada árvore buscando prever os resíduos da combinação das árvores anteriores. Assim, por exemplo, a primeira árvore construída pelo XGBoost tentará corresponder os dados diretamente (uma categoria ou um valor numérico). A segunda árvore tentará prever o valor verdadeiro menos o valor previsto. A terceira árvore tentará prever o valor verdadeiro menos o valor previsto da primeira árvore menos a previsão da segunda árvore dos resíduos da primeira árvore.

No entanto, o XGBoost não constrói simplesmente modelos infinitamente, tentando minimizar os resíduos dos resíduos previstos dos resíduos previstos ad infinitum. O algoritmo XGBoost minimiza uma função de perda que também inclui um termo de penalidade para a complexidade do modelo, e esse termo de penalidade limita o número de árvores que serão produzidas. Também é possível limitar diretamente o número de árvores produzidas.

Nos últimos anos, muitos relataram um sucesso muito maior com o XGBoost do que com métodos de aprendizado de máquina para séries temporais, como em competições Kaggle ou em conferências de aprendizado de máquina do setor.



Bagging (ou, mais formalmente, *agregação de bootstrap*) refere-se a uma técnica para modelos de treinamento em que treinamento gerado aleatoriamente

conjuntos são criados para cada modelo diferente em um conjunto. Aleatório

as florestas geralmente usam uma metodologia de ensacamento no treinamento de modelos.

Boosting, como observado, refere-se a uma técnica para treinar modelos em

em que um conjunto é composto por modelos treinados sequencialmente,

cada um dos quais se concentra em corrigir o erro cometido por seus antecessores.

ai. O reforço está no centro de como os modelos de árvore com aumento de gradiente são

treinado.

Exemplo de código

No caso de florestas aleatórias e XGBoost, pode ser mais fácil codificar um modelo de aprendizado de máquina do que entender como esse modelo funciona. Neste exemplo, treinaremos uma floresta aleatória e um modelo de árvore com gradiente aumentado para classificar nosso EEG

dados com base nos recursos que geramos.

Usamos o sklearn para dividir nossos dados em conjuntos de dados de treinamento e teste:

Pitão

>>> **de sklearn.model_selection importar**
train_test_split

>>> X_train, X_test, y_train, y_test = train_test_split(

fset_cesium.values, eeg["classes"], random_state=21)

Começamos com um classificador florestal aleatório. Aqui vemos a facilidade com que podemos criar um modelo para classificar nossos dados de EEG:

Pitão

>>> **de sklearn.ensemble importar** RandomForestClassifier

>>> rf_clf = RandomForestClassifier(n_estimadores = 10,

>>> profundidade_max = 3,

>>> estado_aleatório = 21)

>>> rf_clf.fit(X_train, y_train)

Podemos então determinar a precisão fora da amostra de nossos dados com uma chamada de método no objeto Classificador:

Pitão

>>> rf_clf.score(X_test, y_test)

0,616

Em apenas algumas linhas de código, temos um modelo que pode funcionar melhor do que eu seria capaz de fazer como classificador humano (um modelo sem formação médica). Lembre-se também de que, graças à seleção de recursos, este modelo vê apenas estatísticas resumidas, em vez de um EEG inteiro.

O código do classificador XGBoost é igualmente simples e sucinto:

Pitão

>>> **importar xgboost como xgb**

>>> xgb_clf = xgb.XGBClassifier(n_estimadores = 10,

```
>>> profundidade_max = 3,

>>> estado_aleatório = 21)

>>> xgb_clf.fit(X_train, y_train)

>>> xgb_clf.score(X_test, y_test)
```

0,648

Podemos ver que o modelo classificador XGBoost se sai um pouco melhor do que o modelo de floresta aleatória. Ele também treina um pouco mais rápido, como podemos ver neste rápido experimento para calcular o tempo de treinamento de cada modelo:

Pitão

```
>>> início = hora.time()

>>> xgb_clf.fit(X_train, y_train)

>>> fim = hora.time()

>>> fim - início
```

0,0189

Floresta Aleatória

```
>>> início = hora.time()

>>> rf_clf.fit(X_train, y_train)

>>> fim = hora.time()

>>> fim - início
```

0,027

Essa velocidade de execução é uma melhoria substancial, com a floresta aleatória demorando 50% mais tempo que o XGBoost. Embora este não seja um teste definitivo, aponta para uma vantagem do XGBoost, especialmente se você estiver lidando com grandes conjuntos de dados. Você gostaria de ter certeza de que essa vantagem aumentasse ao usar conjuntos de dados maiores com mais exemplos e mais recursos.

Esta não é a maneira de fazer testes de desempenho!

Há muito o que criticar sobre o código anterior. Para começar, existem maneiras mais compactas de escrever um método para testar o desempenho. Além disso, você deseja calcular a média do tempo de treinamento e executá-lo com dados realistas em condições de treinamento realistas, não com um exemplo de brinquedo em um notebook Jupyter.

Mais importante ainda, só porque um algoritmo comumente disponível é executado lentamente não significa que o algoritmo seja intrinsecamente lento. Às vezes, os algoritmos em módulos são escritos mais para transparência do código do que para desempenho, e isso pode significar que eles são executados mais lentamente do que o necessário. Outras vezes, os algoritmos são implementados para o caso mais geral possível, enquanto você pode descobrir que suas necessidades de dados são altamente especializadas, o que significa que você pode modificar implementações padrão de algoritmos para atender aos seus propósitos.

Discutiremos considerações de desempenho específicas para séries temporais no Capítulo 12 .

Poderíamos perguntar se há algo em nosso conjunto específico de hiperparâmetros que deu vantagem ao XGBoost sobre uma floresta aleatória. Por exemplo, e se usarmos árvores menos complexas definindo uma profundidade menor? Ou e se permitirmos que existam menos árvores de decisão no modelo? É muito fácil testar essas possibilidades e novamente vemos que o XGBoost tende a manter sua vantagem.

Por exemplo, se permitirmos o mesmo número de árvores de decisão no conjunto, mas diminuirmos a complexidade diminuindo a profundidade das árvores, veremos que o modelo com gradiente reforçado mantém uma precisão maior do que o modelo de floresta aleatória:

Pitão

>>> ## Teste o mesmo número de árvores (10) mas com menos complexidade

>>> ## (profundidade_max = 2)

>>>

>>> ## XGBoost

>>> xgb_clf = xgb.XGBClassifier(n_estimadores = 10,

profundidade_máx = 2,

estado_aleatório = 21)

>>> xgb_clf.fit(X_train, y_train)

>>> xgb_clf.score(X_test, y_test)

0,616

>>> ## Floresta Aleatória

>>> rf_clf = RandomForestClassifier(n_estimadores = 10,

profundidade_máx = 2,

estado_aleatório = 21)

>>> rf_clf.fit(X_train, y_train)

>>> rf_clf.score(X_test, y_test)

0,544

Isso é verdade mesmo quando reduzimos ainda mais a complexidade da árvore:

>>> ## Teste o mesmo número de árvores (10) mas com menos complexidade

>>> ## (profundidade_max = 1)

>>> ## XGBoost

>>> xgb_clf = xgb.XGBClassifier(n_estimadores = 10,

profundidade_máx = 1,

estado_aleatório = 21)

>>> xgb_clf.fit(X_train, y_train)

>>> xgb_clf.score(X_test, y_test)

0,632

>>> ## Floresta Aleatória

>>> rf_clf = RandomForestClassifier(n_estimadores = 10,

profundidade_máx = 1,

estado_aleatório = 21)

>>> rf_clf.fit(X_train, y_train)

>>> rf_clf.score(X_test, y_test)

0,376



Existem várias razões possíveis para explicar o desempenho e a suposta vantagem das árvores com gradiente aumentado sobre as florestas aleatórias. Uma consideração importante é que não sabemos com certeza se todas as características que selecionamos para a nossa classificação são particularmente úteis. Isso destaca um exemplo de quando o boost (árvores com gradiente aumentado) pode ser preferível ao ensacamento (floresta aleatória). É mais provável que o Boosting ignore recursos inúteis porque sempre fará uso do conjunto completo de recursos e privilegiará os relevantes, enquanto algumas árvores resultantes do ensacamento serão forçadas a usar recursos menos significativos.

Isso também sugere a utilidade do boost quando combinado com as bibliotecas de geração de recursos sobrecarregadas que discutimos no capítulo anterior. Se você adotar a abordagem de gerar centenas de recursos de séries temporais – muito mais do que você pode inspecionar razoavelmente – o aumento pode ser uma proteção contra resultados verdadeiramente desastrosos.

Árvores com gradiente aumentado são particularmente úteis para grandes conjuntos de dados,

incluindo grandes conjuntos de dados de séries temporais. A implementação que você

escolher provavelmente afetará ligeiramente sua precisão e seu treinamento

velocidade. Além do XGBoost você também deve considerar o LightGBM

e CatBoost. Estes últimos pacotes foram por vezes relatados

para funcionar um pouco mais rápido que o XGBoost, embora às vezes em um

ligeira diminuição na precisão do teste fora da amostra.

Classificação versus regressão

Nos exemplos anteriores, consideramos florestas aleatórias e metodologias de árvores com gradiente aumentado para classificação de séries temporais. Essas mesmas metodologias também podem ser usadas para previsões de séries temporais.

Muitos estatísticos argumentam que o aprendizado de máquina tem sido menos bem-sucedido — ou não mais bem-sucedido — do que a análise estatística tradicional de séries temporais no domínio da previsão. No entanto, nos últimos anos, as árvores de previsão com gradiente aumentado decolaram e, de fato, muitas vezes superam os modelos estatísticos tradicionais quando recebem conjuntos de dados suficientemente grandes, tanto em competições de previsão quanto em aplicações industriais. Nesses casos, no entanto, é necessário muito tempo para ajustar os modelos

parâmetros, bem como para preparar recursos de séries temporais.

Um dos pontos fortes dos modelos de árvore com gradiente aumentado é que eles se aproximam do “piloto automático” em sua capacidade de eliminar recursos irrelevantes ou barulhentos e focar nos mais importantes. Esta tendência por si só, contudo, não será suficiente para obter um desempenho de última geração de um modelo. Mesmo para um método aparentemente automático, como árvores com gradiente aumentado, os resultados só podem ser tão bons quanto as entradas. A maneira mais importante de melhorar seu modelo ainda será fornecer recursos de entrada de alta qualidade e bem testados.

Existem muitas opções para melhorar o modelo atual. Poderíamos aprender com isso usando a opção do XGBoost para produzir métricas de importância de recursos. Isso poderia nos ajudar a identificar características de recursos úteis e inúteis, e poderíamos então expandir o conjunto de dados adicionando recursos semelhantes àqueles que já são considerados úteis. Também poderíamos fazer uma pesquisa na grade de hiperparâmetros para ajustar a parametrização do nosso modelo.

Finalmente, poderíamos examinar a série temporal bruta de nossos dados mal rotulados para ver se há características dos dados mal rotulados que podem não ser representados por nosso conjunto atual de recursos. Poderíamos considerar adicionar recursos que pudessem descrever melhor os dados mal rotulados, aumentando ainda mais nossas entradas.

Agrupamento

A ideia geral do agrupamento é que os pontos de dados semelhantes entre si constituam grupos significativos para fins de análise. Essa ideia é tão verdadeira para dados de séries temporais quanto para outros tipos de dados.

Assim como em nossa discussão anterior, presumo que você tenha alguma familiaridade com os conceitos relevantes de aprendizado de máquina em um contexto que não seja de série temporal. Se você não está familiarizado

com técnicas de agrupamento, recomendo que você busque alguns breves antecedentes

lendo sobre clustering antes de continuar esta seção.

O agrupamento de séries temporais pode ser usado tanto para classificação quanto para previsão. No caso de classificação, podemos utilizar algoritmos de agrupamento para identificar o número desejado de clusters durante a fase de treinamento. Podemos então usar esses clusters para estabelecer tipos de séries temporais e reconhecer quando novas amostras pertencem a um determinado grupo.

No caso de previsão, a aplicação pode ser clustering puro ou pode ser inspirada em clustering na forma de uso de métricas de distância relevantes (mais sobre métricas de distância em breve). Existem algumas opções para gerar previsões em um horizonte h no futuro com base em clustering e técnicas relacionadas. Lembre-se que neste caso não teremos observado completamente uma série temporal, mas apenas seus primeiros N passos, a partir dos quais queremos prever seu valor no passo de tempo $N + h$. Nesse caso, existem algumas opções.

Uma opção é usar a associação de classe para gerar uma previsão com base no comportamento típico dessa classe. Para fazer isso, primeiro determine a qual cluster uma amostra de série temporal pertence com base em seus primeiros N passos de tempo e, em seguida, infira o provável comportamento futuro com base na associação ao cluster. Especificamente, observe como os valores das séries temporais neste cluster tendem a mudar seus valores entre o intervalo de tempo N e o intervalo de tempo $N + h$. Observe que você gostaria de realizar o clustering original para todas as séries temporais com base em seus primeiros N

etapas, em vez de todas as partes da série temporal, para evitar antecipação.

Outra opção é prever o comportamento futuro de uma série temporal amostral com base no comportamento de seu vizinho (ou vizinhos) mais próximo no espaço amostral. Neste cenário, com base nas métricas dos primeiros N passos de tempo, encontre a amostra de série temporal mais próxima **. Capítulo 9: Aprendizado de Máquina para Séries Temporais**

vizinho(s), para os quais a trajetória completa é conhecida. Em seguida, calcule a média do comportamento $N + h$ desses vizinhos mais próximos e esta será sua previsão para a amostra atual.

Tanto no caso da classificação como da previsão, a consideração mais importante é como avaliar a similaridade entre as séries temporais. O agrupamento pode ser feito com uma variedade de métricas de distância, e muita pesquisa tem sido dedicada a pensar sobre como medir a distância em problemas de alta dimensão. Por exemplo, qual é a “distância”

entre dois candidatos a emprego? Qual é a “distância” entre duas amostras de sangue?

Estes desafios já estão presentes nos dados transversais e persistem nos dados de séries temporais.

Temos duas grandes classes de opções de métricas de distância ao aplicar técnicas de agrupamento a dados de séries temporais:

Distância baseada em recursos

Gere recursos para a série temporal e trate-os como as coordenadas para calcular os dados. Isto não resolve completamente o problema de escolha de uma métrica de distância, mas reduz o problema ao mesmo problema métrico de distância apresentado por qualquer conjunto de dados transversais.

Distância baseada nos dados brutos da série temporal

Encontre uma maneira de determinar o quão “próximas” são as diferentes séries temporais, de preferência de uma forma que possa lidar com diferentes escalas temporais, um número diferente de medições e outras disparidades prováveis entre amostras de séries temporais.

Aplicaremos essas duas métricas de distância a um conjunto de dados de série temporal em que cada amostra representa a projeção de uma palavra escrita à mão de uma imagem 2D para uma imagem 1D.

série temporal.

Gerando recursos a partir dos dados

Já discutimos formas de gerar e selecionar recursos. Aqui consideramos como avaliar a distância entre conjuntos de dados de séries temporais com base na similaridade de suas características.

Num cenário ideal, já teríamos eliminado séries temporais sem importância ou desinteressantes, talvez usando uma árvore para avaliar a importância do recurso. Não queremos incluir tais características num cálculo de distância, uma vez que podem indicar falsamente dissimilaridades entre duas séries temporais, quando na verdade simplesmente não indicam uma semelhança relevante relativamente às classes na nossa tarefa de classificação ou ao resultado na nossa tarefa de previsão.

Tal como fizemos com o conjunto de dados EEG, iniciamos a nossa análise dando uma olhada em alguns exemplos de classes e observando quais diferenças óbvias existem ao longo do tempo e na estrutura da série temporal.

Agrupamento | 273

Nossos dados são um subconjunto do conjunto de dados FiftyWords disponível na UEA e UCR

Repositório de Classificação de Séries Temporais. Este conjunto de dados foi divulgado pelos autores de um

Artigo de 2003 sobre agrupamento de palavras manuscritas em documentos históricos. Nesse artigo, os autores desenvolveram "perfis de palavras" como forma de mapear a imagem 2D de uma palavra manuscrita em uma curva 1D, consistindo no mesmo número de medidas.

independentemente do comprimento da palavra. O conjunto de dados do repositório não é idêntico ao do artigo, mas os mesmos princípios se aplicam. O objetivo do artigo original era desenvolver um método de marcar todas as palavras semelhantes ou idênticas em um documento com um único rótulo, para que os humanos pudessem voltar e rotular essas palavras digitalmente (um feito que hoje em dia poderia muito bem ser realizado diretamente por um sistema neural). rede, com 20 anos de aprimoramento tecnológico para ajudar na tarefa).

Portanto, neste exemplo, vemos exemplos de *perfis de projeção* das palavras, onde "projeção"

refere-se ao fato de que eles converteram uma imagem de um espaço 2D para um espaço 1D, e este último espaço, uma vez que a ordem é importante, é passível de análise de séries temporais. Observe que o eixo "tempo" não é realmente o tempo, mas sim progressões da esquerda para a direita das palavras escritas. No entanto, o conceito é o mesmo – dados ordenados e igualmente espaçados – por isso, para simplificar, utilizarei as palavras *tempo* e *temporal* na análise, embora isto não seja estritamente verdadeiro. Para nosso caso de uso, não há distinção.

Na Figura 9-4 vemos exemplos de algumas palavras distintas. 1

Quando examinamos estes dados, e particularmente quando consideramos os padrões óbvios ao olho humano nos gráficos, tal como acontece com o EEG, podemos formular um ponto de partida de algumas características a considerar para a nossa análise – por exemplo, as alturas e localizações dos os picos, bem como suas características, como a intensidade com que sobem e a forma que existe no topo do pico.

Muitos dos recursos que estou descrevendo começam a soar mais como recursos de reconhecimento de imagem do que como uma série temporal, e essa é uma perspectiva útil para a geração de recursos. Afinal, os dados visuais geralmente são dados que podemos processar facilmente e sobre os quais temos mais intuição do que dados de séries temporais. Pode ser útil pensar em termos de imagens ao pensar em recursos. É também uma perspectiva que ilustra por que gerar recursos pode ser surpreendentemente difícil. Em alguns casos, pode ser óbvio, ao examinar uma série temporal, como distinguir duas classes, mas podemos descobrir que como escrever o código não é tão óbvio. Ou podemos descobrir que podemos escrever o código, mas é extremamente desgastante.

1 Note-se que a informação sobre o conteúdo real de cada "palavra" não está disponível e não era especialmente relevante quando o conjunto de dados original foi reunido. A ideia era reconhecer que todas as palavras de uma etiqueta eram iguais, de modo a reduzir o esforço humano necessário para etiquetar documentos.

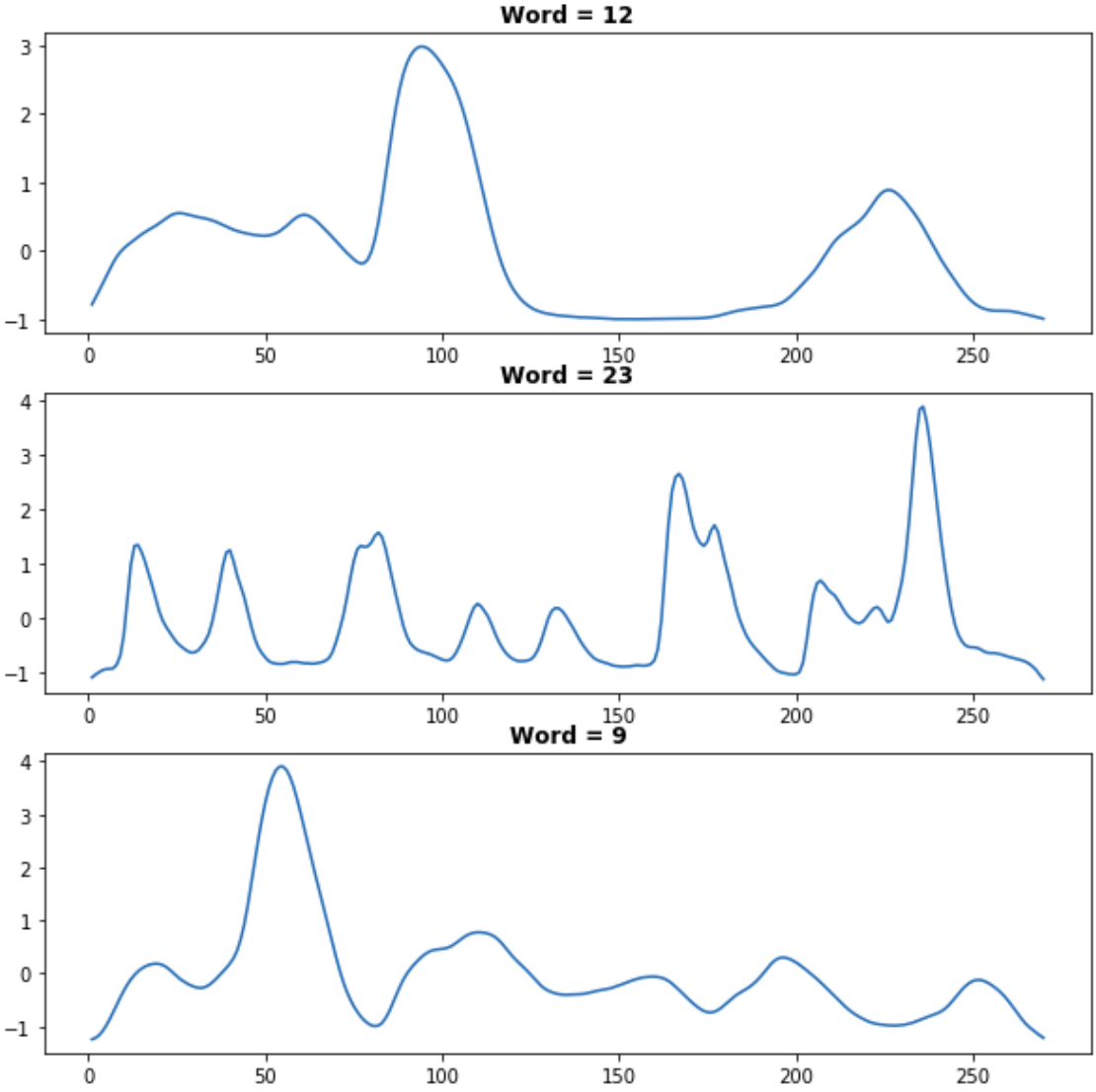
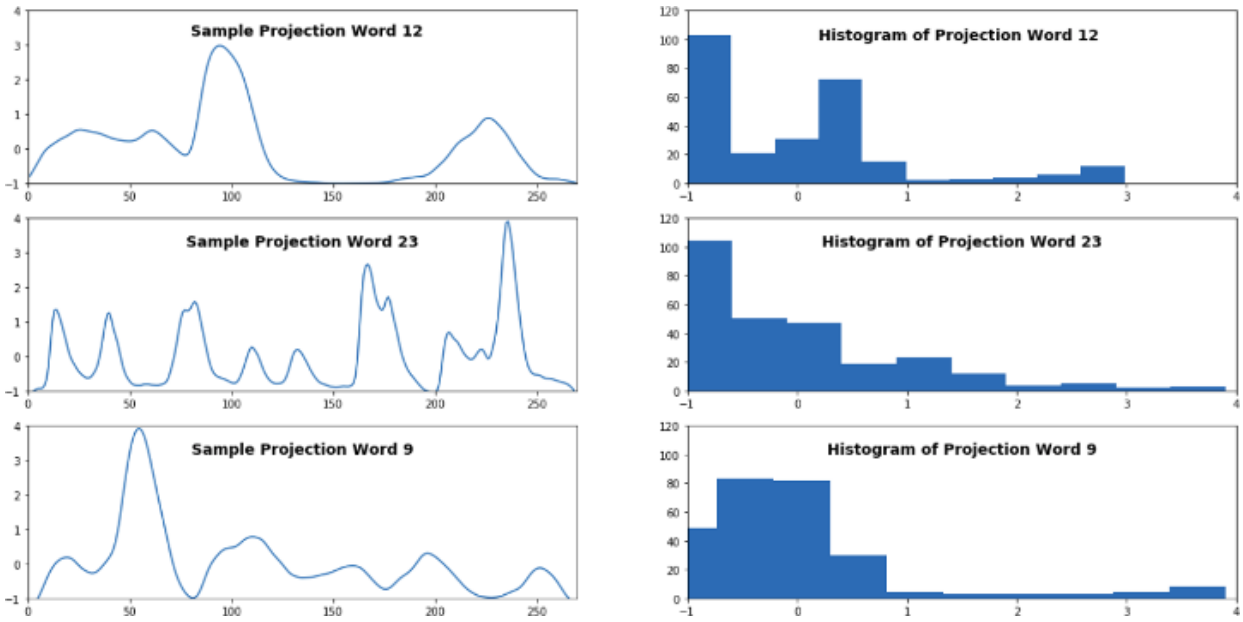


Figura 9-4. Os perfis de projeção de três palavras diferentes (12, 23 e 9) são bastante diferentes entre si. Também já vemos algumas características que poderiam distinguir estes palavras umas das outras: a localização temporal (eixo x) do maior pico ou mesmo de o segundo maior pico, quantos picos locais existem, a faixa geral de valores, e a convexidade média das curvas.

Na última década, o aprendizado profundo emergiu como o melhor desempenho para classificação de imagens e, com dados suficientes, poderíamos treinar um classificador de aprendizado profundo no

imagens dessas parcelas (mais sobre isso no Capítulo 10). Por enquanto, será útil pensarmos em maneiras de contornar a dificuldade de programação. Por exemplo, seria difícil gerar recursos para localizar cada pico, pois a localização de picos é programaticamente exigente e uma espécie de forma de arte.

Também podemos usar um histograma 1D, seja de todos os exemplos de classe ou de um exemplo individual. Isso pode sugerir maneiras computacionalmente menos exigentes de identificar picos ou encontrar outros valores proxy que serão mapeados nas formas gerais que vemos nas séries temporais. Aqui plotamos os mesmos membros individuais da classe plotados antes, agora acompanhados por seus histogramas 1D (veja a Figura 9-5):



Pitão

```
>>> plt.subplot(3, 2, 1)

>>> plt.plot(palavras.iloc[1, 1:-1])

>>> plt.title("Palavra = " + str(palavras.palavra[1]),
fontweight = 'bold')
```

```
>>> plt.subplot(3, 2, 2)

>>> plt.hist(palavras.iloc[1, 1:-1], 10)

>>> plt.subplot(3, 2, 3)

>>> plt.plot(palavras.iloc[3, 1:-1])

>>> plt.title("Palavra = " + str(palavras.palavra[3]),
fontweight = 'bold')

>>> plt.subplot(3, 2, 4)

>>> plt.hist(palavras.iloc[3, 1:-1], 10)

>>> plt.subplot(3, 2, 5)

>>> plt.plot(palavras.iloc[5, 1:-1])

>>> plt.title("Palavra = " + str(palavras.palavra[11]), peso
da fonte = 'bold')

>>> plt.subplot(3, 2, 6)

>>> plt.hist(palavras.iloc[5, 1:-1], 10)
```

Figura 9-5. Outra forma de medir classes para debater recursos úteis. Em particular, o histograma de exemplos de classes individuais indica que os atributos do histograma, como o número de picos locais, a assimetria e a curtose, seriam proxies úteis e possivelmente bons para alguns dos atributos das curvas de séries temporais são óbvios ao olho humano, mas não são fáceis de identificar com o código.

Também queremos ter certeza de que os exemplos que estamos vendo não são discrepantes em comparação com outros exemplos dessas palavras. Por esta razão, construímos a história 2D

tograma para duas palavras para ter uma ideia da variação individual (veja a Figura 9-6):

```
## Pitão
```

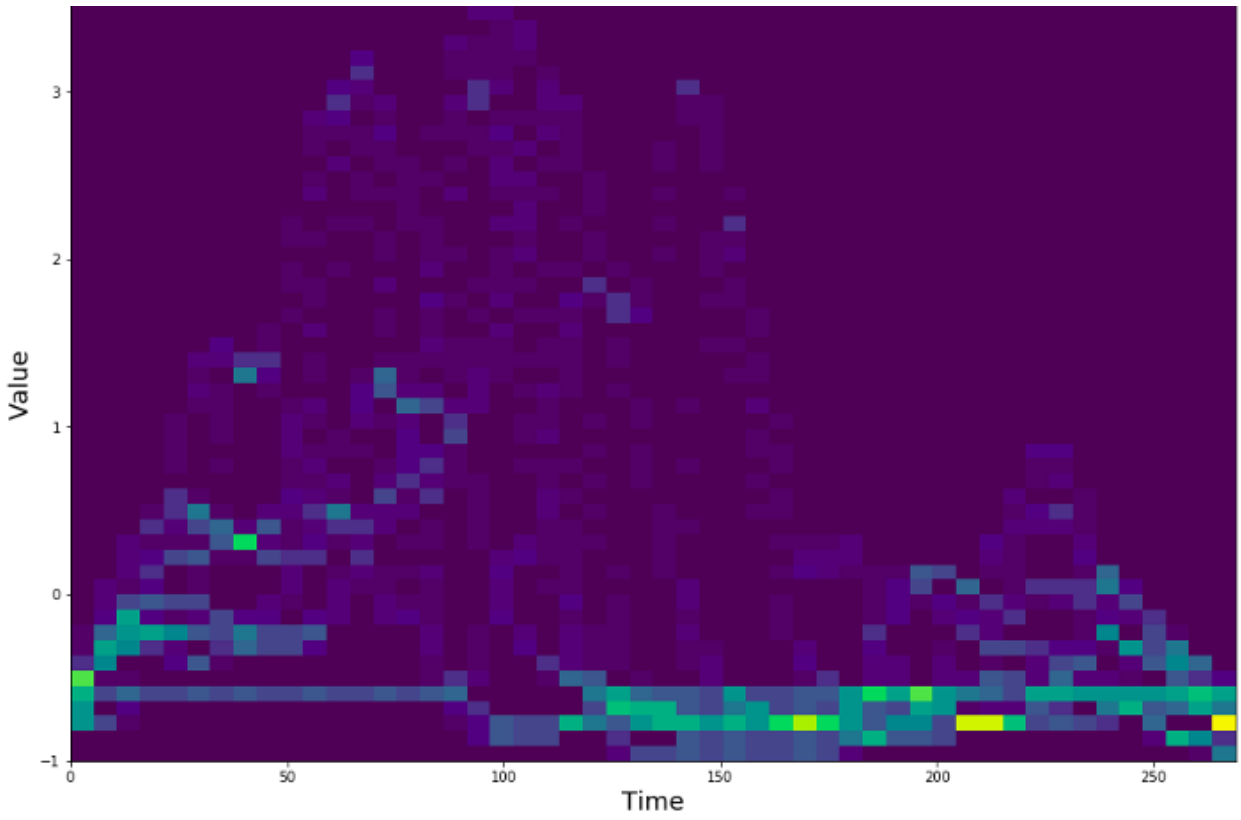
>>> x = np.array([])

>>> y = np.array([])

>>>

>>> w = 12

>>> palavras_selecionadas = palavras[palavras.palavra == w]



>>> palavras_selecionadas.forma

>>>

>>> **para** idx, linha **em** selected_words.iterrows():

>>> y = np.hstack([y, linha[1:271]])

```
>>> x = np.hstack([x, np.array(intervalo(270))])

>>>

>>> fig, machado = plt.subplots()
```

Figura 9-6. Um histograma 2D das projeções de palavras 1D para palavra = 12. O eixo y é o valor em um determinado intervalo de tempo, e o eixo x representa os 270 intervalos de tempo para cada intervalo de tempo amostra de série/projeção de palavra.

A Figura 9-6 mostra o histograma 2D para todas as palavras = 12 membros do conjunto de dados.

Embora a curva individual na Figura 9-5 sugerisse que nos concentrássemos em encontrar os dois

grandes picos que pareciam dominar a série temporal, aqui vemos que o que a maioria dos membros desta classe tem em comum é provavelmente a extensão plana entre esses picos, que parece ir dos intervalos de tempo 120 a 200, com base na intensidade dos pontos dessa região.

Também podemos usar este histograma 2D para estabelecer um ponto de corte para o pico máximo desta classe, que parece variar em localização entre os intervalos de tempo 50 e 150. Podemos até querer codificar uma característica que seja tão específica como “é o valor máximo alcançado entre os pontos 50 e 150?”

Traçamos outro histograma 2D pelo mesmo motivo, desta vez escolhendo a classe de palavras 23, que tem muitas pequenas saliências no exemplo que traçamos na Figura 9-5, uma expressão difícil de entender.

recurso de quantização (veja a Figura 9-7).

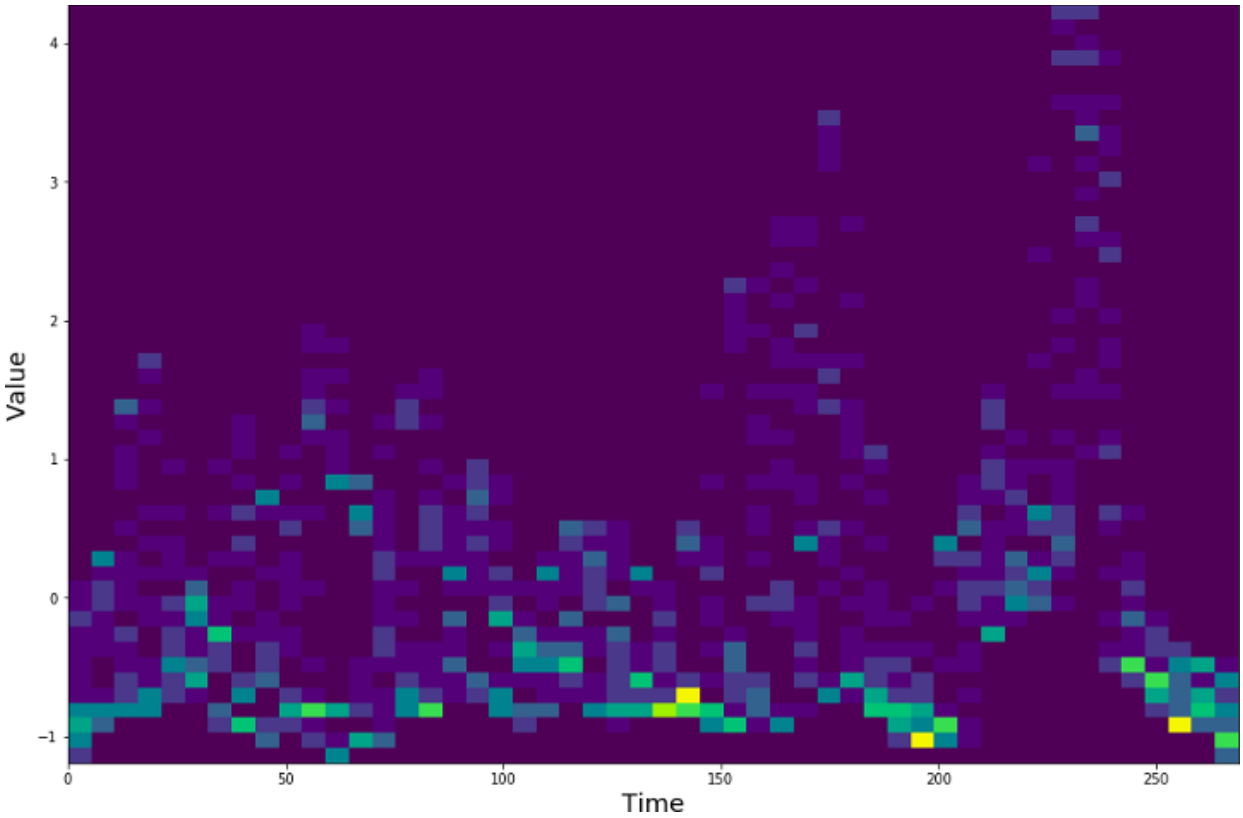


Figura 9-7. Um histograma 2D das projeções de palavras 1D para palavra = 23. O eixo y é o valor em um determinado intervalo de tempo, e o eixo x representa os 270 intervalos de tempo para cada intervalo de tempo amostra de série/projeção de palavra.

Não é surpreendente ver um histograma particularmente “manchado” para a classe de palavras 23 em

Figura 9-7 , dado que mesmo o exemplo que traçamos na Figura 9-5 mostrou tantas características que esperaríamos ver muitas manchas no histograma 2D se as características não correspondessem exatamente entre as amostras. No entanto, também vemos que o valor máximo do ponto nesta classe ocorre em um intervalo de intervalos de tempo não sobrepostos aqui, em comparação com a classe de palavras 12. Para esta classe, o valor máximo vem depois de 150, o que faz sentido, visto que os dois maiores picos vimos no exemplo da classe de palavras 23 que estavam nesta faixa. O histograma 2D tende a comprovar que o pico anterior não é tão alto quanto o pico posterior, sugerindo outras maneiras de quantizar a forma desta série temporal para distingui-la de outras classes.

Os histogramas 2D são úteis para nos informar a variabilidade de um recurso dentro de uma classe individual, para que não dependamos indevidamente de um único exemplo de classe ao pensar em como formar nossos recursos.

Neste caso, optamos por gerar um conjunto de características derivadas da forma da projeção da palavra e um conjunto adicional de características derivadas da forma do histograma da projeção da palavra (projetando um resumo 1D em um resumo 1D diferente, a partir do qual geramos características). Isto é uma resposta às grandes "manchas" que vemos nos histogramas 2D, indicando que existem picos, mas que a sua localização não é **278 | Capítulo 9: Aprendizado de Máquina para Séries Temporais**

particularmente estável. Usar um histograma para gerar uma segunda forma característica para cada projeção de palavra pode ser mais confiável e característico do que a própria projeção de palavra. Os histogramas caracterizam quais tipos de valores aparecem na série sem caracterizar sua localização na série, o que é importante para nós, visto que os picos nas projeções não possuem localizações temporais especialmente estáveis dentro da série temporal.

Primeiro, geramos recursos para as séries temporais, que têm 270 intervalos de tempo cada. Nesse caso, abreviamos o nome da função usada para gerar recursos para legibilidade do código:

de importação de césio featurize.featurize_time as ft

Pitão

>>> palavra_vals = palavras.iloc[:, 1:271]

>>> vezes = []

>>> palavra_valores = []

>>> **para** idx, linha **em** word_vals.iterrows():

>>> word_values.append(linha.valores)

>>> times.append(np.array([i **para** i **no** intervalo(row.values.shape[0])]))

>>>

>>> recursos_to_use = ['amplitude',

>>> 'porcentagem_além_1_std',

>>> 'percent_close_to_median']

>>> palavras_em destaque = ft(vezes = vezes,

>>> valores = palavra_valores,

>>> erros = Nenhum,

>>> recursos_para_usar = recursos_para_usar,

>>> agendador = Nenhum)

Em seguida, geramos histogramas e os usamos como outra série temporal para gerar

comi recursos:2

Pitão

>>> *## crie alguns recursos derivados do histograma*

>>> vezes = []

>>> valores_hist = []

>>> **para** idx, linha **em** words_features.iterrows():

>>> hist_values.append(np.histogram(linha.valores,

>>> caixas = 10,

>>> intervalo=(-2,5, 5,0))[0] + 0,0001)

>>> *## 0s causam problemas posteriores*

>>> times.append(np.array([i **para** i **no** intervalo(9)]))

>>>

>>> features_to_use = ["amplitude",

>>> "percent_close_to_median",

2 Tal como as próprias palavras projecções, para as quais o eixo x não é realmente o tempo, mas outro eixo ordenado e uniformemente espaçado que pode muito bem ser o tempo, o mesmo se aplica aos histogramas. Podemos pensar no eixo x como o tempo para fins de nossa análise, como a geração de recursos.

>>> "inclinar"

>>>]

>>>

>>> destaque_hists = ft(vezes = vezes,

>>> valores = hist_valores,

>>> erros = Nenhum,

>>> recursos_para_usar = recursos_para_usar,

>>> agendador = Nenhum)

Garantimos que todos os histogramas usem o mesmo número de compartimentos e o mesmo intervalo de valores como base para os compartimentos, por meio dos parâmetros que passamos para np.histogram(). Isso garante que todos os histogramas sejam diretamente comparáveis, tendo o mesmo intervalo de valores bin, que será o eixo “temporal” quando esses histogramas forem executados na geração de recursos de série temporal. Se não impusermos

esta consistência, as características geradas não seriam necessariamente significativas na comparação de um histograma com

outro.

Finalmente, combinamos essas duas fontes de recursos:

Pitão

>>> recursos = pd.concat([featurized_words.reset_index(drop=True),

>>> destaque_hists],

>>> eixo=1)

Métricas de distância com reconhecimento temporal

Ao executar a análise de cluster, temos que escolher uma métrica de distância. Com recursos de série temporal, como acabamos de fazer, podemos aplicar a eles uma variedade de métricas de distância padrão, como é feito na análise de agrupamento padrão em dados transversais. Se você não estiver familiarizado com o processo de escolha de uma métrica de distância nesses casos, recomendo um

pequeno desvio para fazer alguma leitura de fundo.

Nesta seção focaremos no problema de medir a similaridade entre séries temporais definindo uma métrica de distância entre elas. Um dos exemplos mais conhecidos de tais métricas é o Dynamic Time Warping (DTW). O DTW está apto a agrupar uma série temporal cuja característica mais saliente é a sua forma geral, como é o caso dos nossos dados de projeção de palavras.

O nome da técnica é inspirado na metodologia, que se baseia em

“warping” para alinhar séries temporais ao longo de seu eixo temporal para comparar suas formas.

Uma imagem é muito mais valiosa do que palavras para transmitir o conceito de distorção dinâmica do tempo, então dê uma olhada na Figura 9-8 . O eixo temporal (x) é distorcido – isto é, expandido ou contraído conforme conveniente – para encontrar o melhor alinhamento de pontos entre as duas curvas representadas (ou seja, duas séries temporais), a fim de comparar sua forma.

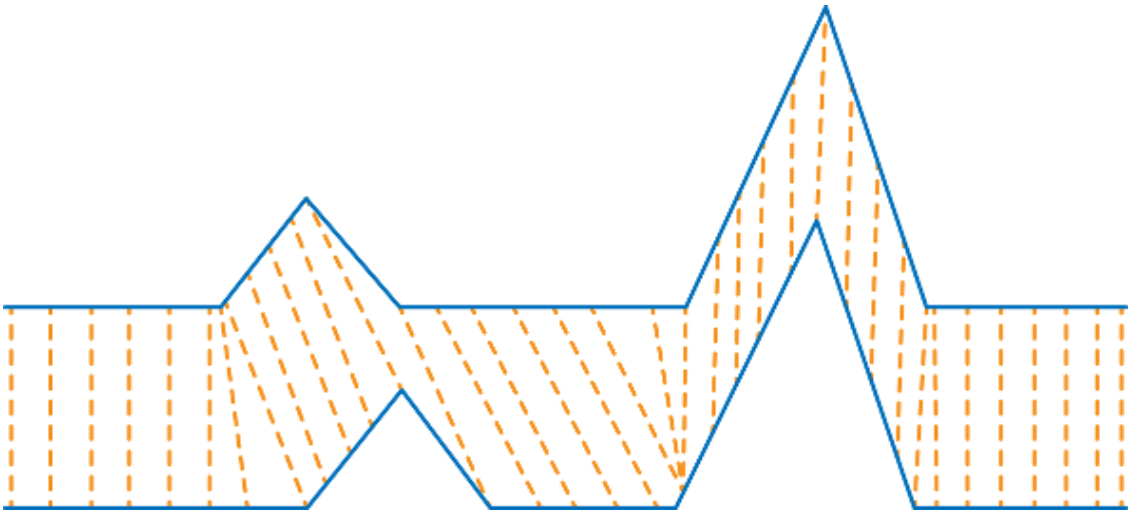


Figura 9-8. Como funciona a distorção dinâmica do tempo. Cada ponto em uma série temporal é mapeado para um ponto na série temporal oposta, mas não há exigência de que deva haver um mapeamento um-para-um de pontos. Isto tem algumas implicações: (1) As séries temporais não precisam ser da mesma duração ou mesmo da mesma escala de tempo. O que importa é a forma. (2) Tempo nem sempre avança durante o processo de adaptação e pode não se mover no mesmo ritmo para cada série temporal. Por movimento no tempo, quero dizer progredir ao longo da curva em

a direção do eixo x. Fonte: Wikipédia.

Observe que os valores de tempo reais no eixo do tempo de uma curva em comparação com outra não são relevantes na forma padrão deste algoritmo. Poderíamos estar comparando uma série temporal medida em nanossegundos com outra medida em milênios (embora isso provavelmente

não seja um exercício sensato). O objetivo do algoritmo é semelhante a comparar a “forma” visual desse algoritmo, em vez de pensar em quanto tempo está passando. Na verdade, “tempo” é realmente entendido apenas no sentido mais geral de um conjunto ordenado e uniformemente espaçado de pontos ao longo do eixo x, e não no tempo propriamente dito.

As regras do DTW são as seguintes:

• Cada ponto em uma série temporal deve corresponder a pelo menos um ponto da outra série temporal.

• O primeiro e o último índice de cada série temporal devem ser comparados com os seus homólogos nas outras séries temporais.

• O mapeamento dos pontos deve ser tal que o tempo avance e não retroceda.

Não há como voltar no tempo combinando um ponto em uma série temporal com um ponto em outra série temporal que já foi passado no eixo do tempo. No entanto, o tempo não precisa avançar constantemente. Por exemplo, dois intervalos de tempo consecutivos na série original poderiam ser distorcidos ao serem condensados no

mesmo lugar no eixo x durante o ajuste, conforme ilustrado na Figura 9-8 no primeiro

“dobra” na curva superior/linha sólida.

Existem muitas maneiras de ajustar o alinhamento temporal para seguir essas regras, mas a correspondência selecionada é aquela que minimiza a distância entre as curvas. Esta distância, ou função de custo, é frequentemente medida como a soma das diferenças absolutas entre os pontos correspondentes, onde a diferença absoluta é a diferença entre os valores dos pontos.

Agora que temos uma ideia de como o DTW funciona intuitivamente, podemos dar uma olhada no código:

Pitão

>>> **def** distDTW(ts1, ts2):

>>> *## esta é a configuração*

>>>DTW={}

>>> **para** i **no** intervalo(len(ts1)):

>>> DTW[(i, -1)] = np.inf

>>> **para** i **no** intervalo(len(ts2)):

>>> DTW[(-1, i)] = np.inf

>>> DTW[(-1, -1)] = 0

>>>

>>> *## é aqui que calculamos o valor ideal*

>>> *## um passo de cada vez*

>>> **para** i **no** intervalo(len(ts1)):

>>> **para** j **no** intervalo(len(ts2)):

>>> dist = (ts1[i] - ts2[j])**2

>>> DTW[(i, j)] = dist + min(DTW[(i-1, j)],

>>> DTW[(i, j-1)],

>>> DTW[(i-1, j-1)])

>>> *## este é um exemplo de programação dinâmica*

>>>

>>> ## assim que encontrarmos o caminho completo, retornamos

>>> ## a distância associada

>>> **retornar** sqrt(DTW[len(ts1)-1, len(ts2)-1])

Conforme indicado nos comentários, a solução para este problema é um exemplo de programação dinâmica, e a distância DTW é um problema clássico de programação dinâmica. Nós

podemos dar um passo de cada vez no caminho desde o início até o fim de cada série temporal, e sabemos que podemos desenvolver a solução um passo de cada vez e consultar nosso conhecimento anterior para tomar decisões posteriores.

Existem diversas implementações de DTW, com uma variedade de ideias para tornar mais eficiente a busca pela solução ótima, ou quase ótima. Estes devem ser explorados, especialmente se você estiver trabalhando com um conjunto de dados maior.

Existem também outras maneiras de medir distâncias entre séries temporais. Aqui estão alguns: *Distância Fréchet*

Esta é a distância máxima entre duas curvas durante um tipo de distorção temporal.

travessia das curvas que busca sempre minimizar a distância entre duas curvas. Essa métrica de distância é frequentemente explicada pela analogia de um cachorro e seu companheiro humano caminhando nas duas curvas com uma coleira entre eles. Eles



cada um precisa percorrer uma curva separada do começo ao fim, e eles podem ir em velocidades diferentes e variar suas velocidades ao longo da curva, desde que sempre se movam na mesma direção. A distância de Fréchet é o menor comprimento de guia necessário para que eles completem a tarefa seguindo a trajetória ideal (assumindo que consigam encontrá-la!).

Correlação de Pearson

A correlação entre duas séries temporais pode ser uma forma de medir a distância entre elas. Ao contrário de outras métricas de distância, você minimizará a distância entre as séries temporais maximizando a métrica da correlação. A correlação é relativamente fácil de calcular. No entanto, este método requer que a série temporal tenha o mesmo número de pontos de dados ou que um deles seja reduzido para

corresponder ao menor número de pontos de dados do outro. A complexidade temporal da correlação computacional é $O(n)$, o que a torna uma métrica particularmente eficiente do ponto de vista dos recursos computacionais.

Subsequência comum mais longa

Esta medida de distância é apropriada para séries temporais que representam uma sequência de valores categóricos ou integrais. Nesses casos, para considerar a similaridade de duas séries temporais, podemos determinar o comprimento da maior subsequência comum, ou seja, o comprimento mais longo de valores consecutivos que são exatamente idênticos, embora não seja necessário que sua localização exata na série temporal corresponda. Tal como acontece com o DTW, isto significa que estamos mais preocupados em encontrar uma forma de semelhança

em vez de onde no tempo a forma comum está ocorrendo. Observe também que, assim como o DTW, mas diferentemente da correlação de Pearson, isso não exige que as séries temporais tenham o mesmo comprimento. Uma medida relacionada é *a distância de edição* , por meio da qual encontramos o número de alterações que precisaríamos fazer em uma série temporal para torná-la

īdêntica a outra e usamos esse valor pāra dēfinir uma métrica de distância.

Distância versus similaridade

A literatura sobre medição de distâncias entre séries temporais também utiliza

o termo similaridade para descrever essas métricas. Na maioria dos casos, você pode

tratar estes termos de forma intercambiável, nomeadamente como uma forma de estabelecer

quais séries temporais são mais ou menos parecidas entre si. Dito isto, alguns

as métricas serão distâncias adequadas, como a distância de Fréchet,

que pode ser calculado com unidades adequadas (como “pés” ou “kg/

dólar” ou qualquer métrica que uma série temporal esteja medindo). Outras medidas

ures são livres de unidades, como uma correlação.

Às vezes, um pouco de criatividade pode ajudar muito a encontrar uma solução simples, mas adequada, por isso é sempre uma boa ideia considerar exatamente quais são as suas necessidades e definir

o mais especificamente possível. Considere uma postagem Stack Overflow buscando uma distância

métrica para uma aplicação específica, ou seja, classificar séries temporais para combiná-las com um dos três centróides de uma análise de agrupamento anterior. As três aulas foram:

• Uma linha plana.

• Um pico no início da série temporal e, caso contrário, uma linha plana.

• Um pico no final da série temporal e, caso contrário, uma linha plana. O usuário descobriu que várias métricas de distância padrão, incluindo distância euclidiana e DTW, não funcionaram. Neste caso, o DTW foi muito generoso e classificou qualquer série temporal com um pico como igualmente próxima da série temporal com um pico no final e um pico no início (portanto, o DTW não é uma panacéia, apesar de ser computacionalmente desgastante!).

Neste caso, um comentador inteligente sugeriu uma transformação que faria com que as métricas de distância funcionassem melhor, nomeadamente para comparar a série temporal cumulativa somada em vez da série temporal original. Após esta transformação, ambas as distâncias euclidiana e DTW deram a ordenação correta de modo que uma série temporal com pico no início mostrasse a menor distância ao protótipo daquela classe, em vez de uma distância igual ao protótipo com pico no início da série temporal e o protótipo com pico no final da série temporal. Isto deve nos lembrar de análises anteriores que estudamos, nas quais a transformação de uma série temporal pode tornar o ARIMA apropriado mesmo que os dados brutos não atendam às condições necessárias.

Evite medidas de distância euclidiana para séries temporais

Você deve ter notado que a distância euclidiana não foi mencionada em nossa discussão sobre medidas de similaridade de séries temporais. As distâncias euclidianas tendem a ter um desempenho insatisfatório na avaliação da similaridade entre séries temporais de uma forma que nos interessa. Para ver por quê, vamos comparar duas curvas senoidais e uma linha plana. Se as curvas senoidais tiverem o mesmo período, mas tiverem uma fase diferente (deslocamento ao longo do eixo x/tempo) ou se as curvas senoidais tiverem amplitudes suficientemente diferentes, a linha reta terá uma distância euclidiana menor à curva senoidal do que a outra. curva senoidal. Geralmente este não é o resultado desejado, mas mostra algumas vantagens

oferecidas por algumas das métricas de distância anteriores:

• Capacidade de compensar fase - isto é, deslocamento ao longo do eixo do tempo que não é realmente importante para comparação

• Capacidade de reconhecer semelhanças de formas em vez de semelhanças de magnitudes Este é um exercício fácil de codificar, então experimente.

Se você decidir que uma medida de distância euclidiana é apropriada, considere usar uma distância com transformada de Fourier em vez de uma distância euclidiana. Neste caso, você pode usar a redução de dimensão tendo um teto de frequência, que é apropriado para a maioria dos dados de séries temporais, onde frequências mais altas tendem a ser menos importantes para o conjunto geral **. Capítulo 9: Aprendizado de Máquina para Séries Temporais**

forma e dinâmica de uma série temporal. Outra opção é usar a técnica de Aproximação Agregada Simbólica (SAX) (desenvolvida pela primeira vez em 2007) para reduzir a dimensionalidade dos dados da série temporal antes de calcular um limite inferior para a distância euclidiana.

Infelizmente, não existe um “piloto automático” para escolher uma métrica de distância. Você precisará usar seu bom senso para encontrar um equilíbrio entre:

• Minimizar o uso de recursos computacionais.

• Escolher uma métrica que enfatize as características de uma série temporal mais relevante para o seu objetivo final.

• Certificar-se de que sua métrica de distância reflete as suposições e pontos fortes/

pontos fracos dos métodos analíticos com os quais você está combinando. Por exemplo, k -

significa que o agrupamento não usa distâncias entre pares, mas minimiza as variações, de modo que apenas distâncias do tipo euclidiano fazem sentido para esta técnica.

Código de agrupamento

Agora que discutimos como gerar recursos para análise de agrupamento e como medir a distância diretamente entre séries temporais como uma métrica de distância para agrupamento, realizaremos o agrupamento com nossos recursos selecionados e com nosso DTW pareado.

matriz de distância para comparar os resultados.

Cluster hierárquico de recursos normalizados

Calculamos recursos para nossas palavras como séries temporais tanto para a série temporal da gravação original quanto para o histograma da série temporal. Esses recursos podem ocorrer em escalas bastante diferentes, portanto, se quisermos aplicar uma única métrica de distância a eles, nós os normalizamos como é o procedimento operacional padrão para clustering baseado em recursos.

geralmente:

Pitão

>>> **de aprender** pré-processamento **de importação**

>>> feature_values = preprocessing.scale(features.values)

Selecionamos um algoritmo de agrupamento hierárquico e realizamos um ajuste para 50 clusters, uma vez que estamos buscando combinar esses clusters com as 50 palavras em nosso conjunto de dados:

Pitão

>>> **de sklearn.cluster importar**
AgglomerativeClustering

>>> feature_clustering =
AgglomerativeClustering(n_clusters = 50,

>>> ligação = 'ala')

```
>>> feature_clustering.fit(feature_values)

>>> palavras['feature_labels'] =
feature_clustering.fit_predict(p)
```


Então, queremos ver se os clusters (cujos rótulos são arbitrários em relação aos rótulos das palavras originais) mostram correspondências úteis com os rótulos das palavras:

```
## Pitão

>>> de sklearn.metrics.cluster importar
homogeneidade_score

>>> homogeneidade_score(palavras.palavra,
palavras.feature_labels)

0,508
```

Temos sorte de trabalhar com dados rotulados; caso contrário, poderíamos tirar conclusões incorretas com base nos clusters que formamos. Neste caso, menos de metade dos agrupamentos estão fortemente relacionados com uma única palavra. Se voltarmos e pensarmos em como melhorar esse resultado, temos uma série de opções:

• Usamos apenas seis recursos. Este não é um grande número de recursos, então poderíamos adicionar mais.

• Poderíamos procurar características que seriam relativamente não correlacionadas, o que não fizemos aqui.

• Ainda faltam recursos obviamente úteis. Observamos algumas características na exploração visual dos dados que não incluímos, como o número e a localização de picos distintos. Provavelmente deveríamos retrabalhar esta análise para incluir essa informação ou alguma representação melhor para ela.

• Deveríamos explorar o uso de outras métricas de distância, talvez algumas que dêem maior peso a certas

características do que outras, privilegiando as características que o olho humano considera úteis.

Clustering hierárquico com a matriz de distância DTW

Já concluímos a parte difícil do agrupamento direto com base no agrupamento de séries temporais, calculando a matriz de distância aos pares via DTW. Isso é computacionalmente desgastante, por isso temos o cuidado de salvar os resultados caso queiramos revisitar a análise:

```
## Pitão

>>> p = distâncias_pares(X, métrica = distDTW)

>>> ## isso leva algum tempo para calcular, então vale a pena economizar para reutilizar

>>> com open("pairwise_word_distances.npy", "wb") como f: np.save(f, p)
```

Agora que os temos, podemos usar um algoritmo de agrupamento hierárquico:

```
## Pitão

>>> de sklearn.cluster importar AgglomerativeClustering

>>> dtw_clustering = AgglomerativeClustering(linkage = 'média',

>>> n_clusters = 50,

>>> afinidade = 'pré-computado')

>>> palavras['dtw_labels'] = dtw_clustering.fit_predict(p)
```


E finalmente, como antes, comparamos a correspondência entre os clusters ajustados e os rótulos conhecidos:

Pitão

>>> **de sklearn.metrics.cluster importar** homogeneidade_score,

>>> completude_pontuação

>>> homogeneidade_score(palavras.palavras,palavras.dtw_labels)

0,828

>>> pontuação_completude(palavras.palavras, palavras.dtw_labels)

0,923

Vemos que esse clustering baseado em DTW tem um desempenho substancialmente melhor do que nosso clustering baseado em recursos. No entanto, se você executar o código de cálculo de distância do DTW em seu próprio computador – especialmente se for um laptop padrão – você verá quanto tempo o DTW leva para calcular em comparação com os recursos que escolhemos. É provável que possamos melhorar nosso clustering baseado em recursos, enquanto o clustering de distância DTW, agora que foi calculado, não deixa nenhum meio direto para melhoria. Nossas alternativas para melhorar isso seriam:

• Inclui recursos, bem como a distância DTW. Isso é complicado tanto do ponto de vista da codificação quanto do ponto de vista conceitual de decidir como combinar os recursos com a distância DTW.

• Experimente outras métricas de distância. Conforme discutido anteriormente, a métrica de distância apropriada dependerá dos seus dados, dos seus objetivos e da sua análise posterior. Precisaríamos definir nosso objetivo para esta análise de palavras de forma mais restrita e geométrica, e então poderíamos pensar se o DTW é realmente a melhor métrica para o que queremos alcançar.

Mais recursos

• Nas medidas de distância e similaridade de séries temporais:

Meinard Müller, "Deformação Dinâmica do Tempo", em Recuperação de Informação para Música e Motion (Berlim: Springer, 2007), 69–84, https://perma.cc/R24Q-UR84.

Este capítulo do livro de Müller oferece uma ampla visão geral do tempo dinâmico

empenamento, incluindo uma discussão de aproximações comuns feitas para reduzir

a complexidade computacional da computação DTW.

Stéphane Pelletier, "Calculando a distância de Fréchet entre duas poligonais

Curvas", (notas de aula, Geometria Computacional, Universidade McGill, 2002),

https://perma.cc/5QER-Z89V.

Este conjunto de notas de aula da Universidade McGill oferece uma explicação visual e algorítmica intuitiva do que é a distância de Fréchet e como ela pode ser

calculado.

Pjotr Roelofsen, "Clustering de séries temporais," dissertação de mestrado, Business Analytics, Vrije Universidade de Amsterdã, 2018, https://perma.cc/K8HJ-7FFE.

Esta dissertação de mestrado sobre agrupamento de séries temporais começa com uma discussão útil e muito completa das principais técnicas para cálculo de distâncias.

entre séries temporais, incluindo informações sobre o computador computacional

complexidade do cálculo da distância e ilustrações úteis que auxiliam na construção da intuição.

Joan Serrà e Josep Ll. Arcos, "Uma avaliação empírica de medidas de similaridade

para classificação de séries temporais", Knowledge-Based Systems 67 (2014): 305–14,

https://perma.cc/G2J4-TNMX.

Este artigo oferece uma análise empírica da precisão dos testes fora da amostra para modelos de classificação construídos com sete medidas diferentes de similaridade de séries temporais: distância euclidiana, coeficientes de Fourier, modelos AR, DTW, edit dis-

distância de edição distorcida no tempo e dissimilaridade de custos mínimos de salto.

Os autores testaram essas medidas em 45 conjuntos de dados disponíveis publicamente no repositório de séries temporais UCR.

• Sobre aprendizado de máquina para séries temporais:

Keogh Eamonn, "Introdução à mineração de dados de séries temporais", tutorial de apresentação de slides,

e, https://perma.cc/ZM9L-NW7J.

Esta série de slides oferece uma visão geral do pré-processamento de dados de séries temporais para fins de aprendizado de máquina, medindo a distância entre séries temporais e

identificar "motivos" que podem ser usados para análise e comparação.

Spyros Makridakis, Evangelos Spiliotis e Vassilios Assimakopoulos, "O M4

Competição: Resultados, Conclusões, Conclusão e Caminho a Seguir,” Internacional Jornal de Previsão 34, não. 4 (2018): 802–8, https://perma.cc/42HZ-YVUU.

Este artigo resume os resultados da competição M4 em 2018, que com-

comparou uma variedade de técnicas de previsão de séries temporais, incluindo muitos conjuntos

técnicas disponíveis, em um conjunto selecionado aleatoriamente de 100.000 séries temporais, incluindo dados coletados em diversas frequências (anual, horária, etc.). Nesta visão geral dos resultados da competição, os autores indicam que algumas abordagens “híbridas”, baseadas fortemente em estatísticas, mas também com alguns componentes de aprendizagem automática, conquistaram o primeiro e o segundo lugar na competição. Resultados como

estes apontam para a importância de compreender e implementar tanto as estatísticas

abordagens de cálculo e aprendizado de máquina para previsão.

CAPÍTULO 10

Aprendizado profundo para séries temporais

O aprendizado profundo para séries temporais é um empreendimento relativamente novo, mas promissor.

Como o aprendizado profundo é uma técnica altamente flexível, pode ser vantajoso para análise de séries temporais. O mais promissor é que oferece a possibilidade de modelar comportamentos temporais altamente complexos e não lineares sem ter que adivinhar formas funcionais – o que poderia potencialmente ser uma virada de jogo para técnicas de previsão não estatísticas.

Se você não está familiarizado com aprendizado profundo, aqui está um resumo de um parágrafo (entraremos em mais detalhes posteriormente). O aprendizado profundo descreve um ramo do aprendizado de máquina no qual é construído um “gráfico” que conecta nós de entrada a uma estrutura complicada de nós e arestas. Ao passar de um nó para outro através de uma aresta, um valor é multiplicado pelo peso dessa aresta e então, geralmente, passado por algum tipo de função de ativação não linear. É esta função de ativação não linear que torna a aprendizagem profunda tão interessante: permite-nos ajustar dados não lineares altamente complexos, algo que não tinha sido feito com muito sucesso anteriormente.

O aprendizado profundo se consolidou principalmente nos últimos 10 anos, à medida que melhorias no hardware disponível comercialmente foram combinadas com grandes quantidades de dados para permitir esse tipo de ajuste de modelo pesado. Os modelos de aprendizagem profunda podem ter milhões de parâmetros, então uma maneira de entendê-los é imaginar qualquer gráfico que você possa imaginar, com todos os tipos de multiplicações de matrizes e transformações não lineares, e então imaginar lançar um otimizador inteligente que otimize seu modelo um pequeno grupo de dados de cada vez, ajustando continuamente os pesos deste grande modelo para que produzam resultados cada vez mais bons. Em poucas palavras, isso é um aprendizado profundo.

O aprendizado profundo ainda não forneceu os resultados surpreendentes de previsão que oferece para outras áreas, como processamento de imagens e processamento de linguagem natural. No entanto, há boas razões para estar optimista de que a aprendizagem profunda acabará por melhorar o **289**

arte de prever, ao mesmo tempo que diminui a natureza frágil e altamente uniforme das suposições e requisitos técnicos comuns aos modelos de previsão tradicionais.

Muitas das dores de cabeça do pré-processamento de dados para atender às suposições de um modelo desaparecem quando modelos de aprendizado profundo são usados:

• Não há exigência de estacionariedade.

• Não há necessidade de desenvolver a arte e a habilidade de escolher parâmetros, como avaliar a sazonalidade e a ordem de um modelo ARIMA sazonal.

• Não há necessidade de desenvolver uma hipótese sobre a dinâmica subjacente de um sistema, o que é útil na modelagem em espaço de estados.

Essas vantagens devem parecer familiares após o Capítulo 9, onde discutimos muitas das mesmas vantagens com relação à aplicação do aprendizado de máquina a séries temporais. O aprendizado profundo é ainda mais flexível por vários motivos:

• Muitos algoritmos de aprendizado de máquina tendem a ser bastante frágeis em termos de dimensionalidade e tipos de dados de entrada necessários para que um algoritmo de treinamento funcione.

Em contraste, a aprendizagem profunda é altamente flexível quanto ao modelo e à natureza das entradas.

• Dados heterogêneos podem ser desafiadores com muitas máquinas comumente aplicadas

técnicas de aprendizagem, embora seja bastante comumente usado para aprendizagem profunda

modelos.

• Modelos de aprendizado de máquina raramente são desenvolvidos para problemas de séries temporais, enquanto o aprendizado profundo oferece muita flexibilidade para desenvolver arquiteturas específicas para dados temporais.

No entanto, o aprendizado profundo não é uma solução mágica. Embora não haja nenhum requisito de estacionariedade para o aprendizado profundo aplicado a séries temporais, na prática, o aprendizado profundo não faz um bom trabalho ao ajustar os dados a uma tendência, a menos que as arquiteturas padrão sejam modificadas para se ajustarem à tendência. Portanto, ainda precisamos pré-processar nossos dados ou nossa técnica.

Além disso, o aprendizado profundo funciona melhor com entradas numéricas em canais diferentes, todas dimensionadas para valores semelhantes entre –1 e 1. Isso significa que você precisará pré-processar seus dados, mesmo que isso não seja teoricamente necessário. Além disso, você precisa pré-processá-lo de uma forma que evite antecipar, o que não é algo que a comunidade de aprendizagem profunda como um todo tenha gasto muito tempo aperfeiçoando.

Finalmente, técnicas de otimização de aprendizagem profunda e modelagem para redes neurais orientadas no tempo (cuja maior classe são as redes neurais recorrentes, ou RNNs) não estão tão bem desenvolvidas quanto aquelas para processamento de imagens (cuja maior classe são as redes neurais convolucionais, ou CNN). Isso significa que você encontrará menos orientações sobre os melhores



práticas e regras práticas para selecionar e treinar uma arquitetura do que você faria para tarefas não temporais.

Além dessas dificuldades de aplicar o aprendizado profundo a séries temporais, você descobrirá que as recompensas por fazer isso são variadas. Para começar, o desempenho do aprendizado profundo para séries temporais não supera consistentemente os métodos mais tradicionais de previsão e classificação de séries temporais. Na verdade, a previsão é uma área propícia para melhorias decorrentes da aprendizagem profunda, mas no momento em que este livro foi escrito essas melhorias ainda não se materializaram.

No entanto, há motivos para esperar benefícios imediatos e de longo prazo ao adicionar aprendizado profundo ao seu kit de ferramentas de análise de série temporal. Primeiro, grandes empresas de tecnologia começaram a lançar aprendizagem profunda para serviços de séries temporais com arquiteturas personalizadas que desenvolveram

internamente, muitas vezes com tarefas de modelagem específicas do setor em mente. Você pode usar esses serviços ou combiná-los com suas próprias análises para obter um bom desempenho.

Em segundo lugar, você pode ter um conjunto de dados que funciona incrivelmente bem com aprendizado profundo para séries temporais. Em geral, quanto mais forte for a relação sinal-ruído, melhor será o seu desempenho. Mais de um programador relativamente novo me disse que obteve um sucesso surpreendente com algum aprendizado profundo e simples.

Por exemplo, uma faculdade descobriu que um simples LSTM (mais sobre isso mais tarde) funcionou tão bem quanto os orientadores sobrecarregados ao prever quais alunos provavelmente falhariam ou desistiriam em breve, para que a faculdade pudesse alcançar esses alunos e fornecer mais recursos . Embora o melhor resultado fosse ter mais orientadores, é reconfortante saber que um simples LSTM aplicado aos dados heterogêneos da série temporal das notas e do registro de frequência de um aluno poderia ajudar a mudar os alunos vulneráveis, sinalizando-os para divulgação e maior apoio. No futuro, acredito que podemos esperar mais usos inovadores e assistenciais do aprendizado profundo para séries temporais.

Lembre-se de que todo modelo tem suposições. Aprendizado de máquina

modelos, incluindo redes neurais, invariavelmente têm suposições

incorporados, tanto na arquitetura como nas metodologias de formação.

Até mesmo o fato de a maioria das redes neurais funcionar melhor com entradas

escalado para [–1, 1] sugere que há fortes suposições incorporadas

o modelo, mesmo que estes ainda não tenham sido bem identificados.

Pode até ser que as previsões da rede neural não tenham atingido o seu

desempenho ideal ainda, e que uma melhor compreensão da teoria

fundamentos teóricos e requisitos para previsão neural

redes levarão a melhorias de desempenho.

Se você é novo no aprendizado profundo, este capítulo não fornecerá todo o equipamento conceitual e de programação necessário para começar com competência. No entanto, isso lhe dará um ponto de partida e, a partir daqui, existem muitos bons tutoriais, livros, **Deep Learning for Time Series | 291**

e até cursos online que você pode usar para aprender mais a fundo. A boa notícia sobre a aprendizagem profunda é que não é necessário envolver matemática para se ter uma ideia geral de como ela funciona e como é programada. Além disso, as APIs estão disponíveis em vários níveis de especificidade. Isso significa que um iniciante pode usar APIs de alto nível para experimentar algumas técnicas introdutórias e, francamente, até mesmo especialistas usarão essas APIs de alto nível para economizar tempo. Mais tarde, quando alguma inovação arquitetônica específica for necessária, e à medida que seu entendimento se desenvolver, você poderá usar APIs de nível inferior, onde resta mais para você decidir e especificar concretamente.

Este capítulo fornecerá uma breve revisão dos conceitos que inspiraram e apoiaram o aprendizado profundo como uma atividade matemática e de ciência da computação, bem como exemplos concretos do código que você pode usar para aplicar um modelo de aprendizado profundo aos dados.

Conceitos de aprendizagem profunda

O aprendizado profundo tem raízes em diversas áreas. Houve uma inspiração biológica, com cientistas da computação e analistas quantitativos se perguntando se a maneira de construir máquinas inteligentes seria, em última análise, imitar cérebros, com suas redes de neurônios

disparando em resposta a determinados gatilhos. Houve inspiração matemática na forma do teorema da aproximação universal sendo comprovado para várias ativações, com muitas dessas provas originadas no final dos anos 1980 e início dos anos 1990. Finalmente, houve o crescimento das capacidades e da disponibilidade da computação, juntamente com um campo florescente de aprendizagem automática, cujo sucesso mostrou que, com dados e parâmetros suficientes, sistemas complicados podiam ser modelados e previstos. O aprendizado profundo levou todas essas ideias ainda mais longe, criando redes descritas por milhões de parâmetros, treinadas em grandes conjuntos de dados e com base teórica de que uma rede neural deveria ser capaz de representar uma função arbitrária e não linear com um alto grau de precisão.

A Figura 10-1 mostra uma rede neural simples, um *perceptron multinível* (ou rede totalmente conectada).

Na Figura 10-1 , podemos ver como as entradas multicanais são fornecidas ao modelo na forma de um vetor de dimensão d. Os nós representam os valores de entrada, enquanto as arestas representam os multiplicadores. Todas as arestas que entram em um nó representam um valor anterior multiplicado pelo valor da aresta que ele atravessou. Esses valores de todas as entradas em um único nó são somados e então, geralmente, passados através de uma função de ativação não linear, criando uma não linearidade.

Podemos ver que a entrada consiste em três canais, ou um vetor de comprimento 3. Existem quatro unidades ocultas. Multiplicamos cada uma das três entradas por um peso diferente para cada uma das quatro unidades ocultas a que se destina, o que significa que precisamos de $3 \times 4 = 12$

pesos para descrever completamente o problema. Além disso, como iremos então somar os resultados destas várias multiplicações, a multiplicação de matrizes não é apenas *análoga* ao que estamos a fazer, mas *exatamente* ao que estamos a fazer.

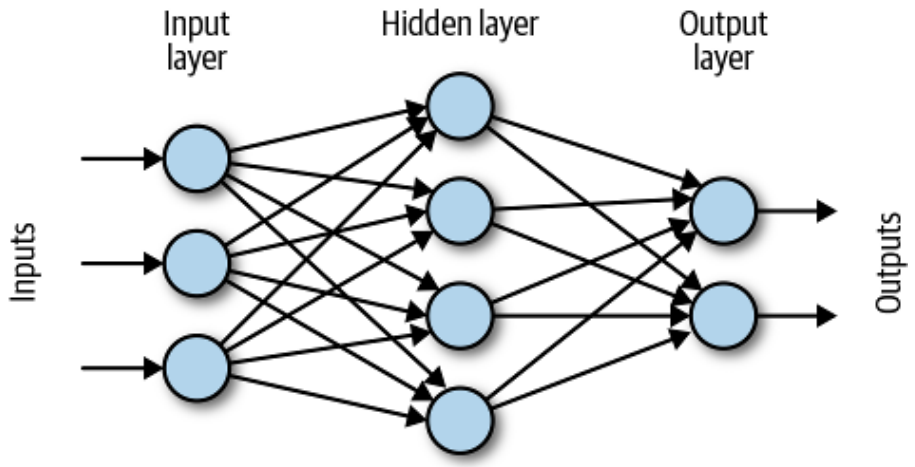


Figura 10-1. Uma rede feed forward simples.

Se quiséssemos escrever as etapas do que estamos fazendo, elas seriam mais ou menos assim:

1. O vetor de entrada X possui três elementos. Os pesos da camada 1 são designados por W1, um 4 × 3

matriz, de modo que calculamos os valores da camada oculta como W1 × X1. Isso resulta em uma matriz 4 × 1, mas na verdade não é a saída da camada oculta:

W1 × X1

2. Precisamos aplicar uma não linearidade, o que podemos fazer com várias “funções de ativação”, como tan hiperbólica (tanh) ou a função sigmóide (σ). Normalmente também aplicaremos uma tendência, B1, dentro da função de ativação, de modo que a saída da camada oculta seja realmente:

H = uma(W1 × X1 + B1)

3. Na rede neural representada na Figura 10-1, temos duas saídas para prever.

Por esta razão, precisamos converter a saída do estado oculto quadridimensional em duas saídas. Tradicionalmente, a última camada não inclui uma função de ativação não linear, a menos que contemos a aplicação de um softmax no

caso de um problema de categorização. Vamos supor que estamos apenas tentando prever dois números, não duas probabilidades ou categorias, então simplesmente aplicamos uma última "camada densa" para combinar os quatro resultados da camada oculta por resultado final. Esta camada densa combinará quatro entradas em duas saídas, então precisamos de uma matriz 2 × 4, W2:

Y = W2 × H

Esperamos que isso tenha lhe dado uma ideia de como esses modelos funcionam. A ideia geral é ter muitos parâmetros e oportunidades para não linearidades.

É uma espécie de arte escolher o número e a forma certos de parâmetros, os hiperparâmetros de treinamento corretos e, claro, um problema razoavelmente acessível. A mágica está em aprender como treinar esses parâmetros, inicializá-los de maneira inteligente desde o início e garantir que o modelo siga na direção certa em direção a uma solução razoavelmente boa. Esta é uma classe de modelos não convexos e a intenção não é encontrar o ótimo global. Em vez disso, pensa-se que, desde que você encontre maneiras inteligentes de regularizar o modelo, encontrará um ótimo local "suficientemente bom" para atender às suas necessidades.

Programando uma rede neural

Compreender como as redes neurais funcionam em princípio pode parecer um pouco mais fácil do que entender como as estruturas de programação associadas aplicadas a esse problema realmente funcionam. Contudo, como discutiremos aqui, existem alguns temas amplos que essas estruturas tendem a ter em comum.

Estilos de programação simbólica e imperativa

Você terá alguma margem de manobra em sua programação de aprendizado profundo para usar um estilo de programação simbólico ou imperativo. Em um estilo *simbólico* de programação, você declara todos os

relacionamentos antecipadamente, sem que eles sejam computados no momento da declaração. Em contraste, num estilo *imperativo* de programação, a computação ocorre quando é codificada, linha por linha, sem esperar para dar conta do que virá depois. Existem prós e contras em cada estilo, e não há uma divisão rígida entre eles. Aqui estão alguns pontos que você deve ter em mente quando estiver pensando sobre o que prefere, supondo que tenha escolha:

• A programação simbólica tende a ser mais eficiente porque você deixa espaço para a estrutura otimizar os cálculos em vez de executá-los quando e

como eles são ordenados.

• A programação imperativa tende a ser mais fácil de depurar e entender.

• TensorFlow e MXnet tendem a um estilo de programação simbólico, enquanto o Torch tem um sabor mais imperativo.

Dados, símbolos, operações, camadas e gráficos

As estruturas de aprendizado profundo geralmente se concentram em alguma noção de gráfico e na construção desse gráfico. A ideia é que qualquer arquitectura deve ser descritível em termos dos seus componentes individuais e da sua relação entre si. Além disso, a ideia de

variáveis divorciadas dos valores reais também é muito importante. Então você pode ter um símbolo, A, e um símbolo, B, e então um terceiro símbolo, C, que é o resultado da multiplicação da matriz A e B:

```
# pseudo-código

símbolo A;

símbolo B;

símbolo C = matmul(A, B);
```

A distinção entre símbolos e dados é essencial para qualquer estrutura devido ao relacionamento entre o símbolo e os dados. O símbolo é usado para aprender uma relação mais geral; os dados podem ter ruído. Existe apenas um símbolo A, mesmo que possamos ter milhões ou até bilhões de valores para alimentar A, cada um emparelhado com os respectivos B e C.

Estruturas populares

Tal como acontece com muitos outros domínios tecnológicos, o panorama das ofertas no mundo da aprendizagem profunda está em constante mudança. E, assim como acontece no mundo dos negócios, muitas vezes ocorrem fusões e aquisições indiscriminadas, como acontece com a fusão do TensorFlow e Theano, e da MXNet e Caffe. Nenhuma estrutura se estabeleceu particularmente como líder em aprendizado profundo de séries temporais, embora o MXNet possa ser a melhor aposta para o futuro próximo, dado o interesse da própria Amazon em previsões e seu recém-revelado aprendizado profundo para serviços de previsão de séries temporais. Como alguém que frequentemente precisa escrever iteradores de dados personalizados para projetos de análise de série temporal em aprendizado profundo, descobri que isso é feito mais facilmente no MXNet.

Para alguém novo na área, no momento em que este artigo foi escrito, as bibliotecas mais proeminentes e amplamente utilizadas, e seus respectivos pais gigantes da tecnologia, são:

- TensorFlow (Google)
- MXNet (Amazonas)
- Tocha (Facebook)

No entanto, muitas vezes há empresas académicas e puramente de código aberto que oferecem inovações que podem assumir um domínio, ou pelo menos dar uma contribuição substancial a uma estrutura existente. O domínio vai e vem rapidamente, embora, à medida que os gigantes da tecnologia comecem a implementar hardware

que corresponda ao seu software, os líderes de mercado possam ganhar mais resistência do que se viu nos últimos anos.

Ao darmos um passo para trás e pensarmos sobre o que fazemos com os dados, essas são as operações. Podemos adicionar ou multiplicar símbolos e podemos considerá-los como operações. Também podemos realizar operações univariadas, como alterar a forma de uma **rede neural | 295**

símbolo (talvez convertendo o símbolo A de uma matriz 2 × 4 para uma matriz 8 × 1) ou passando valores por meio de uma função de ativação, como calcular tanh(A).

Dando um passo ainda mais atrás, podemos pensar em camadas como aquelas unidades tradicionais de processamento que associamos a arquiteturas comuns, como uma camada totalmente conectada, que exploramos na seção anterior. Levando em consideração a função de ativação e polarização, bem como a operação da matriz central, podemos dizer:

camada L = tanh (A × B + polarização)

Em muitas estruturas, esta camada pode ser expressa como uma ou duas camadas, em vez de várias operações, dependendo se uma combinação de operações é popular o suficiente para garantir a sua própria designação como uma unidade de uma camada.

Finalmente, podemos relacionar múltiplas camadas entre si, com uma passando para a próxima: camada L1 = tanh(A × B + polarização1)

camada L2 = tanh (L1 × D + polarização2)

Todo o conglomerado desses símbolos, operações e camadas resulta em um gráfico. Um gráfico não precisa ser totalmente conectado – isto é, não será necessariamente o caso de todos os símbolos terem dependências uns dos outros. Um gráfico é usado precisamente para classificar quais símbolos dependem de quais outros símbolos e como. Isso é essencial para que, ao calcular gradientes para fins

de descida de gradiente, possamos ajustar nossos pesos a cada iteração para obter uma solução melhor. O bom é que a maioria dos pacotes modernos de aprendizagem profunda fazem tudo isso por nós. Não precisamos especificar o que depende do que e como o gradiente muda com cada camada adicionada – está tudo integrado.

Além do mais, como mencionei antes, podemos usar APIs de alto nível de forma que não precisemos explicar multiplicações de matrizes como fiz no exemplo anterior. Em vez disso, se quisermos usar uma camada totalmente conectada, que é o equivalente a uma multiplicação de matrizes, seguida por uma adição de matrizes e, em seguida, algum tipo de função de ativação elemento a elemento, não precisamos escrever toda a matemática . Por exemplo, no mxnet, podemos fazer isso com o seguinte código simples:

Pitão

>>> **importar mxnet como MX**

>>> fc1 = mx.gluon.nn.Dense(120, ativação='relu')

Isso nos dará uma camada totalmente conectada que converte entradas em uma dimensão de saída de 120. Além disso, esta linha representa tudo o que acabei de mencionar: uma multiplicação de matriz, seguida por uma adição de matriz, seguida por uma função de ativação elemento a elemento.

Você pode verificar isso na documentação ou experimentando amostras de saídas e entradas quando conhece os pesos. É realmente impressionante o quanto a API não exige que você faça. Você não é obrigado a especificar a forma de entrada (embora esta deva ser uma constante depois de construir seu gráfico para que possa ser deduzido). Você não é obrigado a especificar um tipo de dados – o padrão será o valor muito sensato e mais comumente usado de float32 (float64 é um exagero para aprendizado profundo). Se seus dados vierem em uma

forma não 1D, por exemplo, esta camada também será automaticamente

“Achate” seus dados para que fiquem no formato apropriado para serem inseridos em uma camada totalmente conectada/densa. Isso é útil para iniciantes, mas depois que você desenvolver um nível mínimo de competência, é bom revisar a documentação para entender o quanto está sendo decidido e cuidado para você, mesmo em um modelo simples de aprendizado profundo.

Este código simples, é claro, não requer tudo o que você precisa, mesmo para uma tarefa curta de aprendizado profundo. Você precisará descrever suas entradas, suas metas e como deseja medir a perda. Você também precisa situar sua camada dentro de um modelo. Fazemos isso no seguinte código:

Pitão

>>> *## crie uma rede em vez de uma camada independente*

>>> **de mx.gluon importar** nn

>>> líquido = nn.Sequencial()

>>> net.add(nn.Dense(120, ativação='relu'),

>>> nn.Dense(1))

>>> net.initialize(init=init.Xavier())

>>>

>>> *## definir a perda que queremos*

>>> L2Loss = gluon.loss.L2Loss()

>>>

>>> treinador = gluon.Train(net.collect_params(), 'sgd',

```
>>> {'taxa_de_aprendizagem': 0,01})
```

Finalmente, supondo que também configuramos nossos dados conforme necessário, podemos passar por uma época de treinamento (ou seja, uma passagem por todos os dados) da seguinte maneira:

```
## Pitão

>>> para dados, destino em train_data:

>>> ## calcule o gradiente

>>> com autograd.record():

>>> saída = rede(dados)

>>> perda = L2Loss(saída, dados)

>>> perda.backward()

>>> ## aplique o gradiente para atualizar os parâmetros

>>> trainer.step(batch_size)
```

Também é fácil na maioria dos pacotes, como no mxnet, salvar seu modelo e os parâmetros que o acompanham para uso posterior em produção ou treinamento adicional:

```
## Pitão

>>> net.save_parameters('model.params')
```


Nos exemplos a seguir, usaremos a API Module do mxnet para oferecer outra opção de criação de gráficos e modelos de treinamento.

Como um praticante de aprendizagem profunda, você eventualmente desejará se familiarizar com todos os principais pacotes e suas diversas APIs (geralmente os pacotes têm, pelo menos, uma API de nível muito alto e uma

API de baixo nível) para que você possa facilmente para ler o código de exemplo.

O conhecimento prático de todos os principais módulos de aprendizagem profunda é necessário para acompanhar as mais recentes práticas da indústria e pesquisas acadêmicas, porque essas informações são mais facilmente aprendidas por meio de código-fonte aberto.

Diferenciação Automática

A magia de um modelo de aprendizagem profunda – além de seu desempenho às vezes impressionante – é que ele pode até aprender. Os modelos padrão podem ter milhares ou até milhões de parâmetros, e esses parâmetros devem ser ajustados ao problema em questão.

Geralmente, isso envolve aproveitar técnicas sofisticadas de diferenciação simbólica disponibilizadas por estruturas de aprendizagem profunda. Nas últimas décadas, a maioria dos matemáticos, físicos e outros que precisavam otimizar funções dependiam de otimização e diferenciação numérica, o que significa que um gradiente era calculado empiricamente (digamos, via $f(x + \delta) - f(x) / \delta$). A vantagem disso é que não havia necessidade de ensinar aos computadores como realizar a diferenciação explicitamente, mas a desvantagem é que era muito lento e não havia como aproveitar o fato de que as relações matemáticas são conhecidas ou mesmo projetadas.

A diferenciação automática utiliza o fato de que, ao calcular as relações entre variáveis em diferentes camadas, sabemos exatamente como as variáveis estão matematicamente relacionadas e podemos diferenciar para obter uma expressão direta para o valor de um gradiente, em vez de realizar cálculos numéricos intensivos. Mais concretamente, a diferenciação automática tira partido do facto de que, particularmente num computador, mais do que na matemática em geral, qualquer sequência de operações matemáticas é limitada a algumas operações simples para as quais as regras de diferenciação são conhecidas e a regra da cadeia pode ser aplicado. A diferenciação automática faz isso para criar expressões computacionalmente tratáveis para gradientes. Uma vez disponíveis esses gradientes, é

possível ajustar os pesos ao longo do tempo e, assim, melhorar e treinar o modelo.

Construindo um pipeline de treinamento

Ao longo desta seção, modelaremos o mesmo conjunto de dados: uma medição do uso horário de energia elétrica em vários locais durante vários anos. Iremos pré-processar estes dados para podermos observar a mudança no consumo de eletricidade de hora em hora, uma tarefa mais difícil do que prever os valores globais porque estamos a olhar para a parte mais imprevisível da série temporal.

Inspecionando nosso conjunto de dados

Usamos um repositório aberto de dados de medições elétricas horárias fornecido em uma base de código que demonstra uma nova arquitetura de rede neural (que discutiremos mais adiante neste post). 1 Para ter uma ideia da aparência desses dados, nós os lemos em R e fazemos alguns gráficos rápidos:

##R

> elec = fread("eletricidade.txt")

> eletrico

V1 V2 V3 V4 V5 V6 V7 V8 V9 V10 V11 V12 V13 V14 V15 V16 V17 V18

1: 14 69 234 415 215 1056 29 840 226 265 179 148 112 171 229 1001 49 162

2: 18 92 312 556 292 1363 29 1102 271 340 235 192 143 213 301 1223 64 216

3: 21 96 312 560 272 1240 29 1025 270 300 221 171 132 185 261 1172 61 197

4: 20 92 312 443 213 845 24 833 179 211 170 149 116 151 209 813 40 173

5: 22 91 312 346 190 647 16 733 186 179 142 170 99 136 148 688 29 144

Já a partir de uma rápida inspeção, podemos ver quantas linhas e colunas de dados temos, bem como que não há carimbos de data/hora. Fomos informados de forma independente que esses carimbos de data e hora são de hora em hora, mas não sabemos quando exatamente essas medições ocorreram:

##R

> ncol(elec)

[1] 321

> agora(elec)

[1] 26304

Também podemos representar graficamente algumas amostras aleatórias dos dados para ter uma noção de sua aparência (veja a Figura 10-2). Como se trata de dados horários, sabemos que se plotarmos 24 pontos de dados teremos dados para um dia inteiro:

##R

> elec[125:148, plot(V4, tipo = 'l', col = 1, ylim = c(0, 1000))]

> elec[125:148, linhas(V14, tipo = 'l', col = 2)]

> elec[125:148, linhas(V114, tipo = 'l', col = 3)]

Também fazemos um gráfico semanal (veja a Figura 10-3):

##R

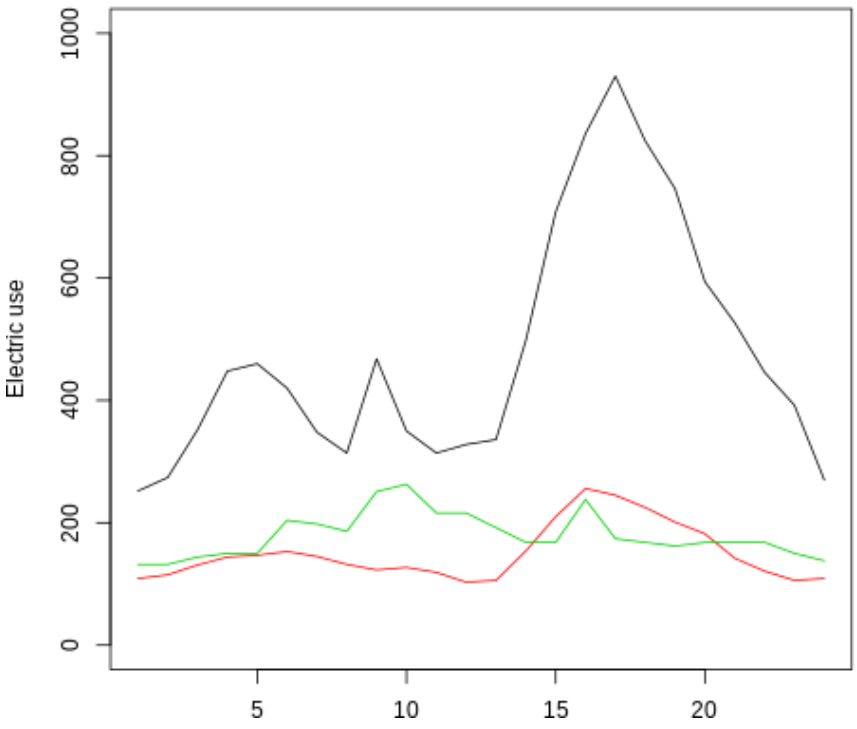
> elec[1:168, plot(V4, tipo = 'l', col = 1, ylim = c(0, 1000))]

> elec[1:168, linhas(V14, tipo = 'l', col = 2)]

> elec[1:168, linhas(V114, tipo = 'l', col = 3)]

Lembre-se de que não sabemos qual é a hora local de nenhum índice específico, embora conheçamos a relação entre eles. No entanto, de acordo com as nossas discussões no Capítulo 2, provavelmente poderíamos adivinhar que partes do dia representam

horários humanos padrão baseados nos padrões de uso elétrico. Provavelmente também poderíamos 1 O download do arquivo vem de provedores de dados no GitHub.



identificar o fim de semana. Não fazemos isso aqui, mas pode ser uma boa ideia incluir modelos relativos à hora do dia e ao dia da semana para uma análise e modelagem mais aprofundada deste conjunto de dados.

Figura 10-2. 24 horas amostradas do conjunto de dados para 3 locais diferentes dos 321

locais disponíveis no conjunto de dados. Embora não tenhamos noção de quais horários locais a que esses índices correspondem, vemos um padrão diário coerente.

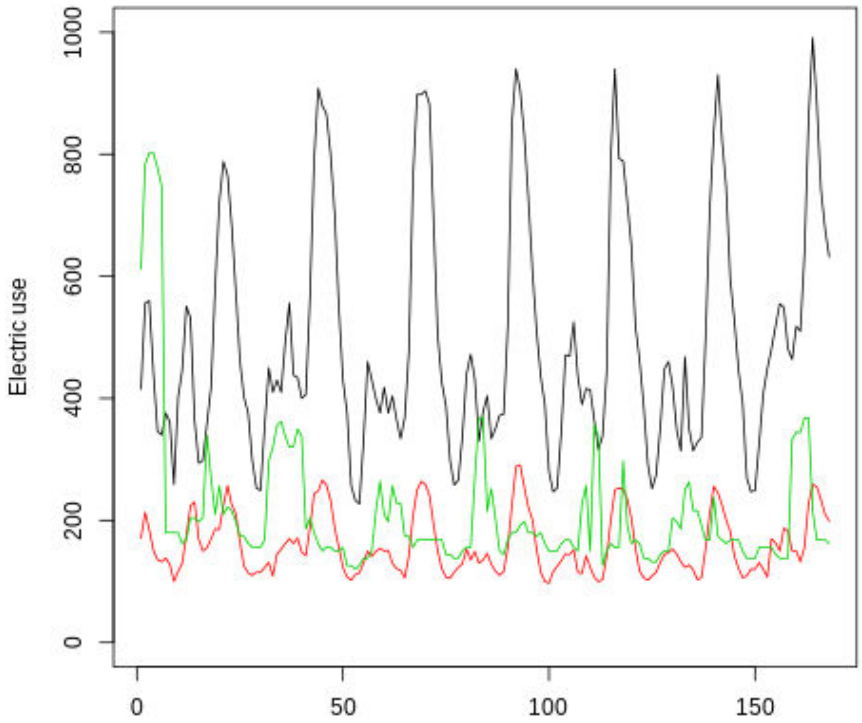


Figura 10-3. Um ciclo completo de sete dias dos dados amostrados para os mesmos três locais que retratado no enredo diário. Nosso sentido de um padrão diário é confirmado com esta visão mais ampla observe os dados, indicando que um grande pico por dia junto com algumas características menores parecem consistentes no comportamento.

Embora pudéssemos prever os valores absolutos dos dados como nossa técnica de previsão, isso já foi feito tanto em artigos acadêmicos quanto em postagens de blogs. Em vez disso, vamos prever a diferença dos dados. Prever diferenças em vez de valores totais de uma série temporal tende a ser mais desafiador porque os dados são mais ruidosos e

podemos ver isso mesmo em gráficos análogos ao que fizemos anteriormente (veja a Figura 10-4):2

##R

> elec.diff[1:168, plot(V4, tipo = 'l', col = 1, ylim = c(-350, 350))]

> elec.diff[1:168, linhas(V14, tipo = 'l', col = 2)]

> elec.diff[1:168, linhas(V114, tipo = 'l', col = 3)]

2 Não incluo o código diferencial, pois abordamos isso no Capítulo 11 e é questão de algumas linhas.

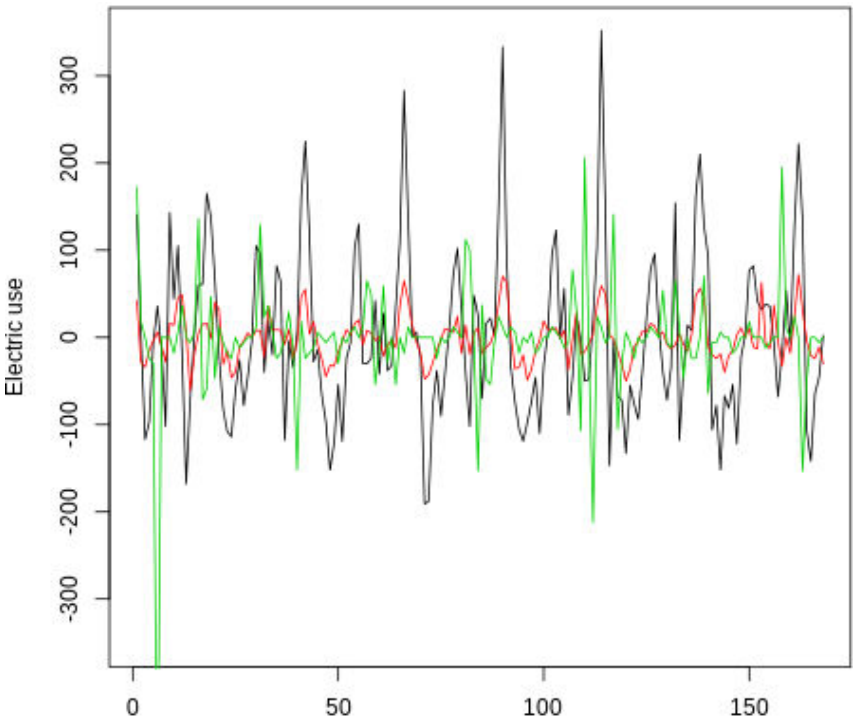


Figura 10-4. Uma amostra de uma semana das séries temporais de potência diferenciada dos mesmos três locais, representando a mudança de hora em hora de uso elétrico. Embora esta série ainda exiba um padrão, como fez a série original, os componentes imprevisíveis da série são tornam-se mais aparentes porque representam uma parcela maior do valor do diferencial série do que na série original.

Se fôssemos executar um modelo estatístico tradicional, um modelo de espaço de estados ou mesmo um modelo de

aprendizado de máquina, precisaríamos fazer uma boa quantidade de análises neste ponto para observar as correlações entre os diferentes locais consumidores de eletricidade no conjunto de dados .

Gostaríamos de ver se houve desvio nos dados ao longo do tempo e avaliar a estacionariedade.

Você também deve fazer essas coisas para aprendizado profundo, para poder avaliar modelos apropriados para um conjunto de dados e também estabelecer expectativas sobre o desempenho do seu modelo. No entanto, a beleza da aprendizagem profunda é que podemos avançar mesmo com dados um pouco mais confusos e mesmo sem passar em nenhum teste estatístico específico para a qualidade dos nossos dados. Em um ambiente de produção, gastaríamos mais tempo na exploração de dados, mas para os propósitos deste capítulo passaremos às nossas opções de modelagem.

Etapas de um pipeline de treinamento

Em geral, quando modelamos com uma rede neural, existem algumas etapas comuns que nossos scripts sempre terão. Muitas vezes, esses scripts podem ser mais difíceis de escrever do que quando ajustamos modelos estatísticos ou tradicionais de aprendizado de máquina, porque nossos conjuntos de dados tendem a ser maiores. Também ajustamos esses modelos profundos em lotes para usar iteradores em vez de matrizes do conjunto de dados completo.

Um pipeline de dados incluirá as seguintes etapas:

• Torne nosso código facilmente configurável importando uma lista predefinida de parâmetros para valores de treinamento padrão, o que é particularmente conveniente, pois de outra forma haveria muitos valores para definir.

• Coloque os dados na memória e pré-processe-os.

• Moldar os dados no formato esperado adequadamente.

• Crie iteradores apropriados para o módulo de aprendizagem profunda que você está usando.

• Construa um gráfico que use esses iteradores para saber qual formato de dados esperar; isso inclui a construção de todo o modelo.

• Defina parâmetros de treinamento, como seu otimizador, sua taxa de aprendizado e por quantas épocas você treinará.

• Estabeleça algum sistema de manutenção de registros tanto para seus pesos quanto para seus resultados de época em época.

Tornando nosso código facilmente configurável

O código a seguir mostra como realizamos essas tarefas. Começamos com uma lista de importações padrão:

Pitão

>>> **de matemática** piso **de importação**

>>>

>>> *## para arquivamento*

>>> **importar sistema operacional**

>>> **importar argparse**

>>>

>>> *## módulo de aprendizagem profunda*

>>> **importar mxnet como MX**

>>>

>>> *## processamento de dados*

>>> **importar entorpecido como np**

>>> **importar pandas como PD**

>>>

>>> *## relatórios personalizados*

>>> **importar desempenho**

Em seguida, usamos uma combinação de variáveis codificadas e parâmetros ajustáveis. Até certo ponto, isso é uma questão de experiência e de suas prioridades durante o treinamento. Não espere que as variáveis façam muito sentido ainda, pois muitos desses parâmetros se aplicam aos componentes da rede neural que discutiremos mais adiante neste capítulo. O principal é observar a ajustabilidade dos parâmetros:

Pitão

>>> *## alguns hiperparâmetros que não ajustaremos por meio de entradas de linha de comando*

>>> DATA_SEGMENTS = { 'tr': 0,6, 'va': 0,2, 'tst': 0,2}

>>> THRESHOLD_EPOCHS = 5

>>> COR_THRESHOLD = 0,0005

>>>

>>> *## configurar analisador*

>>> analisador = argparse.ArgumentParser()

>>>

>>> *## MODELAÇÃO DE DADOS*

>>> parser.add_argument('--win', type=int, default=24*7)

>>> parser.add_argument('--h', type=int, default=3)

```
>>>

>>> ## ESPECIFICAÇÕES DO MODELO

>>> parser.add_argument('--model', type=str,
default='rnn_model')

>>> ## componentes cnn

>>> parser.add_argument('--sz-filt', type=str, default=8)

>>> parser.add_argument('--n-filt', type=int, default=10)

>>> ## componentes rnn

>>> parser.add_argument('--rnn-units', type=int,
default=10)

>>>

>>> ## DETALHES DO TREINAMENTO

>>> parser.add_argument('--batch-n', type=int,
default=1024)

>>> parser.add_argument('--lr', type=float,
padrão=0,0001)

>>> parser.add_argument('--drop', type=float, padrão=0.2)

>>> parser.add_argument('--n-epochs', type=int,
default=30)

>>>

>>> ## TRABALHO DE ARQUIVO

>>> parser.add_argument('--data-dir', type=str,
default='../data')

>>> parser.add_argument('--save-dir', type=str,
default=None)
```

E crucial ter muitos parâmetros ajustáveis porque o treinamento de um modelo de aprendizado profundo sempre envolve pesquisas de hiperparâmetros para melhorar seu modelo a partir da linha de base. Hiperparâmetros comumente ajustados afetam todos os aspectos do treinamento, desde a preparação de dados (quanto tempo atrás no tempo olhar) até a especificação do modelo (quão complexo e que tipo de modelo construir) e treinamento (quanto tempo treinar e qual o tamanho da taxa de aprendizado). usar).

Existem categorias distintas de parâmetros no código anterior. Primeiro, a modelagem dos dados está relacionada à forma como queremos obter nossa entrada bruta, neste caso um CSV cheio de séries temporais paralelas diferenciadas de uso de eletricidade em 321 estações. Para moldar nossos dados, precisamos de dois parâmetros. A variável da janela é o quão longe no tempo permitiremos que nossos modelos olhem ao tentar fazer previsões. A variável horizonte é a distância

para frente queremos prever. Observe que eles não são específicos para unidades, como “5

minutos”, mas sim em intervalos de tempo, consistentes com nossas práticas nos capítulos anteriores.

Como outros modelos estatísticos e de aprendizado de máquina, as redes neurais se preocupam com nossa representação computacional e não têm a menor noção de 5 minutos versus 5 éons quando analisamos os dados.

A penúltima seção, os detalhes do treinamento, geralmente será a mais importante para a otimização de hiperparâmetros e a mais comumente ajustada. É crucial, desde o início, experimentar a taxa de aprendizagem e certificar-se de que você não escolheu algo totalmente errado. Uma boa regra é começar com 0,001 e depois ajustar para cima e para baixo em ordens de magnitude. Não é tão importante ter apenas a taxa de aprendizagem correta, mas é importante ter a ordem de grandeza correta.

As especificações do modelo nos permitem especificar uma variedade de modelos (como RNN versus CNN) e detalhes

arquitetônicos sobre esses modelos. Em geral, desejaremos ajustar hiperparâmetros.

Para os exemplos atuais, usamos os seguintes hiperparâmetros inseridos em nosso script na linha de comando:

--drop=0,2 --win=96 --batch-n=128 --lr=0,001 --n-épocas=25

--data-dir=/data/elec --save-dir=/archive/resultados

--model=model_will_vary

Preparando nossos dados de entrada

Uma vez que tenhamos parâmetros configuráveis, existe uma maneira de fornecer informações ao nosso script, como onde encontrar um arquivo, quanto tempo queremos prever e quanto tempo atrás queremos incluir em nossas entradas para um determinado horizonte. Esses parâmetros configuráveis são importantes mesmo na etapa preliminar de leitura dos dados e sua modelagem adequada. Também precisamos organizar a infraestrutura para trabalhar com os dados, porque as redes neurais são treinadas por meio de variações de gradiente descendente estocástico, o que significa que o treinamento ocorre em pequenos lotes de dados por vez, com uma época significando que todos os dados foram usados para treinamento (embora não todos ao mesmo tempo).

A seguir, discutiremos o processo de alto nível de fornecimento de dados para treinamento por meio de iteradores e os detalhes de baixo nível de modelagem dos dados que vão para os iteradores.

Moldando os dados de entrada. Na seção anterior, vimos como formar iteradores a partir de arrays NumPy e consideramos o fornecimento desses arrays NumPy garantido. Agora discutiremos como os dados são moldados, primeiro conceitualmente e depois com um exemplo de código. Discutiremos dois formatos de dados, NC e NTC.

Começamos nossa discussão sobre diferentes formatos de dados de entrada com um exemplo prático que não tem nada a ver com codificação. Vamos imaginar que temos dados de séries temporais multivariadas, com colunas A, B e C.

Construindo um pipeline de treinamento | 305

Tempo AB

C

t – 3 0 –1 –2

t – 2 3 –2 –3

t – 1 4 –2 –4

t

8 –3 –9

Queremos construir um modelo que preveja um passo à frente e queremos usar os dados dos dois pontos anteriores no tempo para fazer a previsão. Queremos usar os dados de A, B e C para prever A, B e C. Chamaremos nossas entradas de X e nossas saídas de Y.

Procuramos prever Y no tempo *t* . No tempo *t* , estes são os valores reais de Y = [A, B, C]: *t* 8 –3 –9

Especificamos que teríamos os dois pontos de tempo anteriores disponíveis para todas as variáveis fazerem a previsão. Isso equivale ao seguinte:

Em 1

UMA, *t* – 2

B, *t* – 1

B, *t* – 2

C, *t* – 1

C, *t* – 2

4

3

–2

–2

–4

–3

Da mesma forma, para prever Y no tempo *t* – 1, temos os seguintes dados como nossa meta: *t* – 1 4 –2 –4

e esperamos ter os seguintes valores disponíveis para fazer a previsão: **A, *t* – 2**

Às 3

B, *t* – 2

B, *t* – 3

C, *t* – 2

C, *t* – 3

3

0

–2

–1

–3

–2

Se quiséssemos armazenar as entradas para ambos os pontos de dados em um formato de dados, seria assim:

Tempo A, tempo – 1 A, tempo – 2 B, tempo – 1 B, tempo – 2 C, tempo – 1 C, tempo – 2

t – 1 3

0

–2

–1

–3

–2

t

4

3

–2

–2

–4

–3

Isso é chamado de formato de dados NC, onde *N* indica amostras individuais e *C* indica canais, que é outra forma de descrever informações multivariadas. Usaremos esse formato de dados para treinar uma rede neural totalmente conectada, e é a primeira opção no método que discutiremos eventualmente, que pega os dados de entrada no formato CSV e os converte em arrays NumPy com formato e dimensionalidade adequados.

Por outro lado, podemos moldar os dados de forma diferente, de forma a criar um eixo específico para o tempo. Isso normalmente é feito colocando os dados no formato NTC, que designa o número de amostras × tempo × canais. Nesse caso, a amostra é cada linha dos dados originais – ou seja, cada fatia do tempo para a qual queremos fazer uma previsão (e para a qual temos dados disponíveis para fazê-lo). A dimensão de tempo é o quão longe no tempo iremos olhar, que neste exemplo são dois intervalos de tempo (e que é especificado por --win em nosso exemplo de script para este capítulo).

No formato NTC, os dados de entrada que formatamos antes seriam mais ou menos assim para prever o horizonte t – 1:

Tempo

A

B

C

t -1

0, 3

–1, –2

–2, –3

Ou se quiséssemos representar os dados de entrada para ambas as amostras que produzimos anteriormente, poderíamos fazê-lo de forma compacta da seguinte forma:

Tempo

A

B

C

t -1

0, 3

–1, –2

–2, –3

t

3, 4

–2, –2

–3, –4

Isso poderia ser unido aos rótulos que produzimos para Y da seguinte forma:

Tempo

A

B

C

t -1

4

–2

–4

t

8

–3

–9

Nenhuma destas representações é mais precisa que a outra, mas um aspecto conveniente da representação NTC é que o tempo tem significado com um eixo temporal explícito.

A razão pela qual formamos dois formatos de entradas é que alguns modelos preferem um formato e outros outro. Usaremos o formato NTC para entrada em redes neurais convolucionais e recorrentes, que discutiremos mais adiante neste capítulo.

NTC versus TNC

Há algumas coisas a serem observadas sobre o formato de armazenamento de dados NTC mostrado acima. Uma é que é altamente repetitivo. Você pode evitar isso se a ordem dos seus dados não for importante, nomeadamente usando o formato TNC em vez do formato NTC. Dentro do TNC, você pode usar seu eixo “lote” para percorrer diferentes partes dos dados em paralelo. Assim, por exemplo, treinaríamos três lotes de uma vez na seguinte sequência de números:

56 **29** 56 94 10 92 52 32 19 59 88 94 6 57 73 59 95 79 97 38 65 51 27

18 **77** 39 4 19 60 38 73 51 65 4 96 96 6 12 62 59 21 49 65 37 64 69

36 **32** 48 97 33 44 63 99 10 56 75 20 61 53 71 48 41 2 58 18 4 10 17

66 **64** 53 24 36 23 33 38 1 17 59 11 36 43 61 96 55 21 45 44 53 26 55

99 22 10 26 25 82 54 82

Se você os preparasse no formato NTC, com apenas um canal seus dados ficariam assim:

Tempo

Valores

t

56, 29, 56, 94

$t + 1$

29, 56, 94, 10

$t + 2$

56, 94, 10, 92

Há muita repetição, mas a boa notícia é que o algoritmo treinará na ordem cronológica real dos dados, já que todos os lotes estão intimamente relacionados no tempo. Em contraste, se elegermos o formato TNC (N sendo o número de amostras por lote), e se supusermos um tamanho de lote de 4 (pequeno irrealisticamente), poderíamos imaginar nossos dados assim:

Tempo Membro do lote 1 Membro do lote 2 Membro do lote 3 Membro do lote 4

t

29

77

32

64

$t + 1$

56

39

48

53

$t + 2$

94

4

97

24

Neste caso podemos ver que não há repetição de dados na preparação dos nossos inputs. Se queremos treinar com dados sequenciados em ordem cronológica ou não, dependerá do conjunto de dados. Aqueles que trabalharam com processamento de linguagem natural (PNL) em aprendizagem profunda reconhecerão esse formato, que é bastante bem-sucedido para várias tarefas de PNL, mas nem sempre tão útil em dados numéricos de séries temporais.

Construindo iteradores. Em termos gerais, para fornecer dados a uma rotina de treinamento, precisamos fornecer iteradores. Os iteradores não são exclusivos do aprendizado profundo ou do Python, mas refletem a ideia geral de um objeto que percorre algum tipo de coleção, acompanhando onde ele está e indicando quando ele trabalhou em uma coleção inteira.

Formar um iterador é simples no caso de dados de treinamento provenientes de um array NumPy. Se X e Y são arrays NumPy, vemos que formar iteradores é simples:

Pitão

>>> ##############################

>>> ## PREPARAÇÃO DE DADOS ##

>>> ##############################

>>>

```
>>> def prepare_iters(data_dir, win, h, modelo, batch_n):
>>> X, Y = dados_preparados(data_dir, win, h, modelo)
>>>
>>> n_tr = int(Y.shape[0] * DATA_SEGMENTS['tr'])
>>> n_va = int(Y.shape[0] * DATA_SEGMENTS['va'])
>>>
>>> X_tr, X_válido, X_teste = X[ : n_tr],
>>> X[n_tr : n_tr + n_va],
>>> X[n_tr + n_va : ]
>>> Y_tr, Y_válido, Y_teste = Y[ : n_tr],
>>> Y[n_tr : n_tr + n_va],
>>> Y[n_tr + n_va : ]
>>>
>>> iter_tr = mx.io.NDArrayIter(dados = X_tr,
>>> rótulo = Y_tr,
>>> tamanho_lote = lote_n)
>>> iter_val = mx.io.NDArrayIter(dados = X_valid,
>>> rótulo = Y_válido,
>>> tamanho_lote = lote_n)
>>> iter_test = mx.io.NDArrayIter(dados = X_test,
```

>>> rótulo = Y_teste,

>>> tamanho_lote = lote_n)

>>>

>>> **retorno** (iter_tr, iter_val, iter_test)

Aqui temos um método para preparar os iteradores para o conjunto de dados, e esses iteradores agrupam matrizes numpy que são recebidas por um método chamado preparado_data() (mais sobre isso em instantes). Uma vez disponíveis, as matrizes são divididas em fontes de dados de treinamento, validação e teste, sendo os dados de treinamento os mais antigos, sendo a validação usada como uma forma de ajustar hiperparâmetros com feedback fora da amostra.

e dados de teste mantidos até o final para testes verdadeiros.3

Observe que o inicializador de um iterador usa a entrada (dados), o valor alvo (rótulo) e o parâmetro batch_size, que reflete quantos exemplos serão usados por iteração para calcular gradientes e atualizar os pesos do modelo.

Moldando os dados no código

Agora que conhecemos os dois formatos de dados que queremos criar, podemos observar o código que os molda:

Pitão

>>> **def** dados_preparados(data_dir, win, h, nome_modelo):

3 Como discutimos anteriormente neste livro, o padrão ouro é treinar e implementar modelos em vários segmentos de tempo, mas evitamos essa complicação para essa base de código. Em uma base de código de produção, você gostaria de ter validação e testes contínuos para otimizar seu modelo e obter uma noção melhor do desempenho real; a técnica aqui introduz um atraso adicional entre os dados de teste e treinamento e também significa que os dados de teste

usados para julgar o modelo refletem, em última análise, apenas um período de tempo de todo o histórico.


```
>>> df = pd.read_csv(os.path.join(data_dir,
'eletricidade.diff.txt'),

>>> set=',', cabeçalho=0)

>>> x = df.as_matrix()

>>> ## normalizar dados. observe que isso cria uma
antecipação, pois

>>> ## normalizamos com base nos valores medidos no
conjunto de dados

>>> ## então, em um pipeline menos básico,
calcularíamos isso como

>>> ## rolando estatísticas para evitar o problema de
antecipação

>>> x = (x - np.mean(x, eixo = 0)) / (np.std(x, eixo = 0))

>>>

>>> if model_name == 'fc_model': ## formato de dados
NC

>>> ## fornece lookbacks da primeira e da segunda etapa
em uma entrada simples

>>> X = np.hstack([x[1:-1], x[:-h]])

>>> Y = x[h:]

>>> retornar (X, Y)

>>> else : ## Formato de dados TNC

>>> # pré-alocar X e Y
```

>>> *# X shape = num exemplos * time win * num canais (NTC)*

>>> X = np.zeros((x.shape[0] - vitória - h, vitória, x.shape[1]))

>>> Y = np.zeros((x.forma[0] - vitória - h, x.forma[1]))

>>>

>>> **para** i **no** intervalo(win, x.shape[0] - h):

>>> *## o valor alvo/rótulo está h passos à frente*

>>> Y[i-vitória] = x[i + h - 1 , :]

>>> *## os dados de entrada são as etapas de vitória anteriores*

>>> X[i-vitória] = x[(i - vitória) : i , :]

>>>

>>> **retornar** (X, Y)

Depois de ler nossos dados de um arquivo de texto, padronizamos cada coluna. Observe que cada coluna é padronizada por si só, e não em todo o conjunto de dados de maneira homogênea. Isso ocorre porque vimos, mesmo em nossa breve exploração de dados, que diferentes

sítios elétricos tinham valores muito diferentes (veja os gráficos nas Figuras 10-2 e 10-4):

Pitão

>>> x = (x - np.mean(x, eixo = 0)) / (np.std(x, eixo = 0))

Formato de dados NC. Produzir o formato de dados NC é bastante simples:

Pitão

>>> **if** model_name == 'fc_model': *## formato de dados NC*

>>> ## fornece lookbacks da primeira e da segunda etapa em uma entrada simples

>>> X = np.hstack([x[1:-h], x[0:-(h+1)]])

>>> Y = x[(h+1):]

Para gerar as entradas *X que representam o tempo t – h* , para fazer uma previsão em *t* , pegamos *x* e removemos as últimas *h* linhas (uma vez que os dados de entrada exigiriam valores de rótulo posteriores aos dados mais recentes que temos). Em seguida, deslocamos esses dados de volta ao longo do eixo do tempo para produzir mais valores defasados, e precisamos ter certeza de que as matrizes NumPy que representam diferentes defasagens têm o mesmo formato para que possam ser empilhadas juntas. Isto é o que leva à formulação anterior. Vale a pena trabalhar nisso no seu **310 | Capítulo 10: Aprendizado profundo para séries temporais**

próprio computador e comprovando que funciona. Você também pode pensar em como generalizar a expressão para uma retrospectiva arbitrariamente longa.

Verificamos nosso trabalho definindo um ponto de interrupção Pdb e verificando se os valores em X e Y correspondem às suas contrapartes esperadas em x:

Pitão

(Pdb) X[0, 1:10] == x[1, 1:10]

array([Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro])

(Pdb) X[0, 322:331] == x[0, 1:10]

array([Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro])

(Pdb) Y[0, 1:10] == x[4, 1:10]

array([Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro])

A primeira metade das colunas em X representa os últimos pontos no tempo que disponibilizamos para previsão, e o rótulo/alvo da previsão está três passos à frente disso.

É por isso que X[0, 1:10] deve corresponder a x[1, 1:10] e Y[0, 1:10] deve corresponder

x[4, 1:10], porque deve estar três passos à frente (nossa entrada definiu o horizonte como 3).

Pode ser confuso que o tempo e as amostras (índices de pontos de dados) geralmente tenham o mesmo rótulo, mas são coisas separadas. Há o tempo à frente que estamos prevendo, há o momento em que estamos tirando o instantâneo dos insumos para fazer uma previsão e há o tempo atrás que olhamos para coletar dados para fazer a previsão. Esses valores estão necessariamente inter-relacionados, mas é uma boa ideia manter os conceitos separados.

Formato de dados NTC. Produzir o formato NTC também não é tão ruim:

Pitão

>>> # pré-alocar X e Y

*>>> # X shape = num exemplos * time win * num canais (NTC)*

>>> X = np.zeros((x.shape[0] - vitória - h, vitória, x.shape[1]))

>>> Y = np.zeros((x.forma[0] - vitória - h, x.forma[1]))

>>>

>>> **para** i **no** intervalo(win, x.shape[0] - h):

>>> *## o valor alvo/rótulo está h passos à frente*

>>> Y[i-vitória] = x[i + h - 1 , :]

>>> *## os dados de entrada são as etapas de vitória anteriores*

>>> X[i-vitória] = x[(i - vitória) : i , :]

Para qualquer exemplo (a dimensão N, ou seja, a primeira dimensão), pegamos as últimas linhas vencedoras dos dados de entrada em todas as colunas. É assim que criamos três dimensões. A primeira dimensão é essencialmente o índice de pontos de dados, e os valores fornecidos nesse ponto de dados equivalem a dados 2D, nomeadamente tempo × canais (aqui, locais de eletricidade).

Como antes, definimos um ponto de interrupção Pdb para testar este código. Observe também que confirmamos nosso próprio entendimento do que fizemos para testar o código. Freqüentemente, o código para testar um formato de dados é mais esclarecedor do que o código real porque usamos números concretos para fazer verificações pontuais:

Pitão

(Pdb) Y[0, 1:10] == x[98, 1:10]

array([Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro])

(Pdb) X.forma

(26204, 96, 321)

(Pdb) X[0, :, 1] == x[0:96, 1]

array([Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,

Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro, Verdadeiro,
Verdadeiro])

Vemos que o primeiro ponto de dados que preparamos em X e Y (ou seja, a primeira linha) corresponde às linhas 0:96 (porque definimos nosso lookback de janela configurável como 96

intervalos de tempo em nossas entradas do analisador), e o horizonte direto 3 passos de tempo à frente corresponde à linha 98 (porque x termina em 95; lembre-se, a indexação em uma fatia exclui o último número na fatia, então x representa todas as linhas de 0 a 95 inclusive ou 0 a 96

exclusivo).

O código de processamento de dados é propenso a erros, confuso e lento. No entanto, você descobrirá que quanto mais vezes escrever e trabalhar nele, mais sentido fará. No entanto, é bom testar seu código de processamento de dados minuciosamente e mantê-lo em algum lugar seguro para evitar ter que resolver os problemas de amostra sempre que precisar moldar seus dados. Também é aconselhável manter esse código em um sistema de controle de versão e ter alguma maneira de rastrear qual versão do seu código foi usada no treinamento de um modelo específico.

Definir parâmetros de treinamento e estabelecer um sistema de manutenção de registros

Discutiremos os detalhes de vários modelos nas próximas seções, portanto, por enquanto, ignoraremos a parte do código relacionada à construção de gráficos e iremos diretamente para a logística de treinamento e manutenção de registros.

É assim que implementamos o treinamento nos exemplos simples em que nos concentraremos:

Pitão

>>> **def** trem(símbolo, iter_train, valid_iter, iter_test,

>>> nomes_dados, nomes_rótulos,

>>> save_dir):

>>> *## salvar informações/resultados do treinamento*

>>> **se não** os.path.exists(args.save_dir):

>>> os.makedirs(args.save_dir)

>>> printFile = open(os.path.join(args.save_dir, 'log.txt'), 'w')

>>> **def** print_to_file(msg):

>>> **imprimir** (mensagem)

>>> **imprimir** (msg, arquivo = printFile, flush = True)

>>> *## cabeçalho de resultados de arquivamento*

>>> print_to_file('Epoch Training Cor Validation Cor')

>>>

>>> *## armazenando valores da época anterior para definir um limite de melhoria*

>>> *## termina mais cedo se o progresso for lento*

>>> buf = RingBuffer(THRESHOLD_EPOCHS)

>>> val_antigo = Nenhum

>>>

>>> *## padrão mxnet*

>>> *## o padrão é 1 GPU, cujo índice é 0*

>>> desenvolvedores = [mx.gpu(0)]

>>> módulo = mx.mod.Module(símbolo,

>>> nomes_dados=nomes_dados,

>>> label_names=label_names,

>>> contexto=desenvolvedores)

>>> module.bind(data_shapes=iter_train.provide_data,

```
>>> label_shapes=iter_train.provide_label)
>>> module.init_params(mx.initializer.Uniform(0.1))
>>> module.init_optimizer(optimizer='adam',
>>> optimizador_params={'taxa_de_aprendizagem':
>>> args.lr})
>>>
>>> ## treinamento
>>> para época no intervalo (args.n_epochs):
>>> iter_train.reset()
>>> iter_val.reset()
>>> para lote em iter_train:
>>> # calcular previsões
>>> module.forward(lote, is_train=True)
>>> # calcular gradientes
>>> módulo.backward()
>>> # atualizar parâmetros
>>> módulo.update()
>>>
>>> ## resultados do treinamento
>>> train_pred = module.predict(iter_train).asnumpy()
>>> train_label = iter_train.label[0][1].asnumpy()
```

```
>>> train_perf = perf.write_eval(train_pred, train_label,
>>> save_dir, 'trem', época)
>>>
>>> ## resultados de validação
>>> val_pred = module.predict(iter_val).asnumpy()
>>> val_label = iter_val.label[0][1].asnumpy()
>>> val_perf = perf.write_eval(val_pred, val_label,
>>> save_dir, 'válido', época)
>>>
>>> print_to_file('%d %f %f ' %
>>> (época, train_perf['COR'], val_perf['COR']))
>>>
>>> # se ainda não temos medidas de melhoria, pule
```



```
>>> se época > 0:
>>> buf.append(val_perf['COR'] - old_val)
>>> # se tivermos medidas de melhoria, verifique-as
>>> se época > 2:
>>> vals = buf.get()
>>> vals = [v para v em vals se v!= 0]
>>> if sum([v < COR_THRESHOLD para v em vals]) ==
len(vals):
```

```
>>> print_to_file('SAIR ANTECIPADO')

>>> pausa

>>> val_antigo = val_perf['COR']

>>>

>>> ## testes

>>> test_pred = module.predict(iter_test).asnumpy()

>>> test_label = iter_test.label[0][1].asnumpy()

>>> test_perf = perf.write_eval(test_pred, test_label,

>>> save_dir, 'tst', época)

>>> print_to_file('DESEMPENHO DE TESTE')

>>> imprimir_para_arquivo(teste_perf)
```

Por que usar um buffer de anel?

Um *buffer de anel* é um local para colocar valores de forma que o primeiro a entrar seja o primeiro a sair quando você recupera um valor. Também tem a propriedade de crescer apenas até um valor predefinido. Embora o desempenho e o tamanho da memória não sejam um problema aqui, o uso dessa classe nos ajuda a evitar padrões.

Python não possui um buffer de anel integrado, mas você pode encontrar exemplos simples online.

Eles são úteis sempre que você deseja acompanhar um pequeno número de valores na ordem em que foram recebidos.

O código anterior realiza um conjunto diversificado de tarefas, todas rotineiras. Primeiro, configuramos valores para rastrear o histórico das pontuações de precisão da validação como uma forma de garantir que o treinamento esteja apresentando melhorias. Se um modelo não estiver

treinando com rapidez suficiente, não queremos continuar girando nossas GPUs, desperdiçando tempo e eletricidade.

O padrão MXNet usa a API Module (em oposição à API Gluon que vimos anteriormente neste capítulo):

```
## Pitão

>>> ## padrão mxnet

>>> ## o padrão é 1 GPU cujo índice é 0

>>> desenvolvedores = [mx.gpu(0)]

>>> módulo = mx.mod.Module(símbolo,

>>> nomes_dados=nomes_dados,

>>> label_names=label_names,

>>> contexto=desenvolvedores)

>>> module.bind(data_shapes=iter_train.provide_data,

>>> label_shapes=iter_train.provide_label)

>>> module.init_params(mx.initializer.Uniform(0.1))
```



```
>>> module.init_optimizer(optimizer='adam',

>>> optimizador_params={'taxa_de_aprendizagem':

>>> args.lr})
```

Essas quatro linhas de código realizam o seguinte:

1. Configure o componente bruto de uma rede neural como um gráfico computacional.

2. Configure os formatos de dados para que a rede saiba o que esperar e possa otimizar.

3. Inicialize todos os pesos no gráfico com valores aleatórios (isto é uma arte, não um conjunto puramente aleatório de números extraídos de infinitas possibilidades).

4. Inicialize um otimizador, que pode ter vários sabores e para o qual definimos explicitamente a taxa de aprendizado inicial dependendo de nossos parâmetros de entrada.

Em seguida, usamos nosso iterador de dados de treinamento para incrementar nosso caminho através dos dados à medida que treinamos:

Pitão

>>> **para** época **no** intervalo (args.n_epochs):

>>> iter_train.reset()

>>> iter_val.reset()

>>> **para** lote **em** iter_train:

>>> module.forward(batch, is_train=True) *# calcula previsões*

>>> module.backward() *# calcula gradientes*

>>> module.update() *# parâmetros de atualização* Em seguida, medimos os resultados previstos para os conjuntos de treinamento e validação (no mesmo loop for externo de antes):

Pitão

>>> *## resultados do treinamento*

>>> train_pred = module.predict(iter_train).asnumpy()

>>> train_label = iter_train.label[0][1].asnumpy()

```
>>> train_perf = avaliar_e_escrever(train_pred, train_label,
>>> save_dir, 'trem', época)
>>>
>>> ## resultados de validação
>>> val_pred = module.predict(iter_val).asnumpy()
>>> val_label = iter_val.label[0][1].asnumpy()
>>> val_perf = avaliar_e_escrever(val_pred, val_label,
>>> save_dir, 'válido', época)
```

O loop termina com alguma lógica para parada antecipada:

```
## Pitão
>>> se época > 0:
>>> buf.append(val_perf['COR'] - old_val)
>>> se época > 2:
>>> vals = buf.get()
>>> vals = [v para v em vals se v!= 0]
>>> if sum([v < COR_THRESHOLD para v em vals]) ==
len(vals):
>>> print_to_file('SAIR ANTECIPADO')
```



```
>>> pausa
>>> val_antigo = val_perf['COR']
```

Esse código desajeitado faz alguns registros e lógica simples, registrando cada valor de correlação sucessivo entre valores previstos e reais. Se a correlação de época para época não tiver melhorado suficientemente durante épocas suficientes (ou mesmo piorado), o treinamento será interrompido.

Métricas de avaliação

Nossa função, avaliar_e_escrever, registra as correlações por época e o valor bruto tanto para o valor alvo quanto para o valor estimado. Fazemos o mesmo para os testes no final de todo o treinamento:

Pitão

>>> **def** avaliar_e_escrever(pred, rótulo, save_dir, modo, época):

>>> **se não** os.path.exists(save_dir):

>>> os.makedirs(save_dir)

>>>

>>> pred_df = pd.DataFrame(pred)

>>> rótulo_df = pd.DataFrame(rótulo)

>>> pred_df.to_csv(os.path.join(save_dir, '%s_pred%d.csv'

>>> % (modo, época)))

>>> label_df.to_csv(os.path.join(save_dir, '%s_label%d.csv'

>>> % (modo, época)))

>>>

>>> **return** { 'COR': COR(rótulo,pred) }

Isso, por sua vez, faz uso de uma função de correlação que definimos da seguinte forma:

Pitão

>>> **def** COR(rótulo, pred):

>>> rótulo_demeaned = rótulo - rótulo.mean(0)

>>> label_sumsquares =
np.sum(np.square(label_demeaned), 0)

>>>

>>> pred_demeaned = pred - pred.mean(0)

>>> pred_sumsquares =
np.sum(np.square(pred_demeaned), 0)

>>>

>>> cor_coef = np.diagonal(np.dot(label_demeaned.T,
pred_demeaned)) /

>>> np.sqrt(label_sumsquares * pred_sumsquares)

>>>

>>> **retornar** np.nanmean(cor_coef)

Ocasionalmente neste conjunto de dados há casos de variância zero, que podem criar um NAN em uma coluna, então optamos por usar np.nanmean() em vez de np.mean().

Observe que uma funcionalidade básica que não incluímos aqui é salvar os pesos do modelo, verificando durante todo o processo de treinamento. Se estivéssemos treinando para produção e precisássemos recarregar o modelo e implantá-lo, gostaríamos de usar Module.save_checkpoint (para salvar os pesos) e Module.load (para carregar um

modelo de volta à memória, a partir do qual você pode continuar o treinamento ou implantar

seu modelo para produção). Há muito o que aprender para começar com um pipeline de aprendizado profundo adequado, mas aqui vamos mantê-lo básico.

Juntar as peças

Colocamos nossos componentes de pipeline juntos no corpo do nosso escopo __main__:

Pitão

>>> **se __nome__** == '__principal__':

>>> *# analisa argumentos da linha de comando*

>>> args =parser.parse_args()

>>>

>>> *# cria iteradores de dados*

>>> iter_train, iter_val, iter_test = prepare_iters(

>>> args.data_dir, args.win, args.h,

>>> args.model, args.batch_n)

>>>

>>> *## preparar símbolos*

>>> input_feature_shape = iter_train.provide_data[0][1]

>>>

>>> X = mx.sym.Variable(iter_train.provide_data[0].nome)

>>> Y = mx.sym.Variable(iter_train.provide_label[0].nome)

>>>

>>> *# configurar modelo*

```
>>> modelo_dict = {
>>> 'fc_model': fc_model,
>>> 'rnn_model': rnn_model,
>>> 'cnn_model': cnn_model,
>>> 'simple_lstnet_model': simple_lstnet_model
>>> }
>>> modelo = model_dict[args.model]
>>>
>>> símbolo, data_names, label_names = model(iter_train,
>>> input_feature_shape,
>>> X, Y,
>>> args.win, args.sz_filt,
>>> args.n_filt, args.drop)
>>>
>>> ## trem
>>> treinar(símbolo, iter_train, iter_val, iter_test,
>>> nomes_dados, nomes_rótulos, args.save_dir)
```

Aqui utilizamos a infraestrutura que acabamos de montar. Primeiro, analisamos nossos argumentos de linha de comando. Em seguida, criamos iteradores com as entradas configuráveis, incluindo nosso horizonte de previsão, nossa janela de lookback, nosso tamanho de lote e o nome do modelo que queremos construir. Criamos símbolos MXNet e também registramos a forma de entrada, e estes são repassados à medida que criamos o modelo. Por fim, passamos informações sobre o modelo, juntamente com

nossos iteradores e nosso diretório de salvamento, para a função de treinamento, que faz a parte interessante: treinar um modelo e gerar seu

métricas de desempenho.

Construindo um pipeline de treinamento | 317

Portanto, neste caso, vemos um pipeline de treinamento mínimo, mas totalmente funcional, apresentando ingestão e remodelagem de dados, construção de modelo, treinamento de modelo e registro de valores importantes para avaliações de modelo.

A propósito, nosso método print_to_file() é apenas um wrapper útil para print():

Pitão

def imprimir_para_arquivo(mensagem):

print (msg, arquivo = printFile, flush = True)esfa

imprimir_para_arquivo(args)

Você desejará criar um registro do seu modelo enquanto ele é treinado. É possível que os pesos do seu modelo preferido não sejam aqueles em que você concluiu a corrida de treinamento, mas algum tempo antes. Ter um registro de como o treinamento progrediu ajudará você a ajustar os hiperparâmetros relacionados à estrutura do modelo e ao treinamento, variando de quantos parâmetros ter (para evitar underfitting ou overfitting) até qual taxa de aprendizado treinar (para evitar fazer ajustes excessivamente grandes ou pequenos). durante o treinamento).

Agora temos um pipeline mínimo viável completo, exceto que estamos perdendo nossos modelos.

Agora veremos alguns tipos básicos de modelos que podem ser aplicados a dados de séries temporais e treinaremos cada um para ver seu desempenho relativo.

Redes Feed Forward

Este livro é bastante incomum ao apresentar redes feed forward no contexto da análise de séries temporais. A maioria dos problemas modernos de análise de séries temporais são realizados com estruturas de rede recorrentes ou, menos comumente, estruturas de rede convolucionais.

No entanto, começamos com a rede neural feed forward, pois é a estrutura de rede mais simples e histórica. Existem vários motivos pelos quais este é um bom lugar para começar:

• As redes feed forward são altamente paralelizáveis, o que significa que têm bastante desempenho. Se você puder encontrar um modelo de feed forward razoavelmente bom para o seu propósito,

poses, você pode calculá-lo muito rapidamente.

• Redes feed forward são bons testes para verificar se realmente existem dinâmicas complexas de eixo de tempo em sua sequência. Nem todas as séries temporais são realmente séries temporais no sentido de terem dinâmica de eixo temporal, onde os valores anteriores têm uma relação específica com os valores posteriores. Pode ser bom ajustar uma rede neural feed forward como uma linha de base, além de um modelo linear mais simples.

• Componentes de rede feed forward são frequentemente integrados em redes maiores e mais

arquiteturas complexas de aprendizagem profunda de séries temporais. Portanto, você precisa saber como funcionam mesmo que não formem um modelo completo em seu trabalho.

Um exemplo simples

Uma rede feed forward é o tipo mais simples de rede neural, então começamos aplicando uma arquitetura feed forward a um exemplo de série temporal. Não há nada na estrutura de uma rede neural feed forward padrão que indique uma relação temporal, mas a ideia é que o algoritmo possa, ainda assim, aprender algo sobre como as entradas passadas predizem as entradas futuras. Vimos um exemplo de rede feed forward em

Figura 10-1. Uma rede feed forward é uma série de camadas totalmente conectadas, o que significa que

a entrada de cada camada se conecta a todos os nós do gráfico.

Começamos com este exemplo simples, que utiliza o primeiro tipo de formatação de dados que vimos anteriormente. Ou seja, nossa entrada, X, será simplesmente dados bidimensionais de N × C (isto é, amostras × canais), onde os componentes de tempo foram achatados em canais.

Isso corresponde a esta formatação de dados:

A, *t* – 2 A, *t* – 3 B , *t* – 2 B, t – 3 C , *t* – 2 C, *t* – 3

3

0

–2

–1

–3

–2

Como lembrete, no código que representa esta ramificação dos dados:

Pitão

>>> **se** nome_modelo == 'fc_model':

>>> *## fornece lookbacks da primeira e da segunda etapa em uma entrada simples*

>>> X = np.hstack([x[1:-1], x[:-2]])

>>> Y = x[2:]

>>> **retornar** (X, Y)

Em seguida, construímos um modelo totalmente conectado com a API Module do MXNet:

Pitão

>>> **def** fc_model(iter_train, janela, filter_size, num_filter, dropout):

>>> X = mx.sym.Variable(iter_train.provide_data[0].nome)

>>> Y = mx.sym.Variable(iter_train.provide_label[0].nome)

>>>

>>> saída = mx.sym.FullyConnected(dados=X, num_hidden = 20)

>>> saída = mx.sym.Activation(saída, act_type = 'relu')

>>> saída = mx.sym.FullyConnected(dados=saída, num_hidden = 10)

>>> saída = mx.sym.Activation(saída, act_type = 'relu')

>>> saída = mx.sym.FullyConnected(dados = saída, num_hidden = 321)

>>>

>>> loss_grad = mx.sym.LinearRegressionOutput(dados = saída,

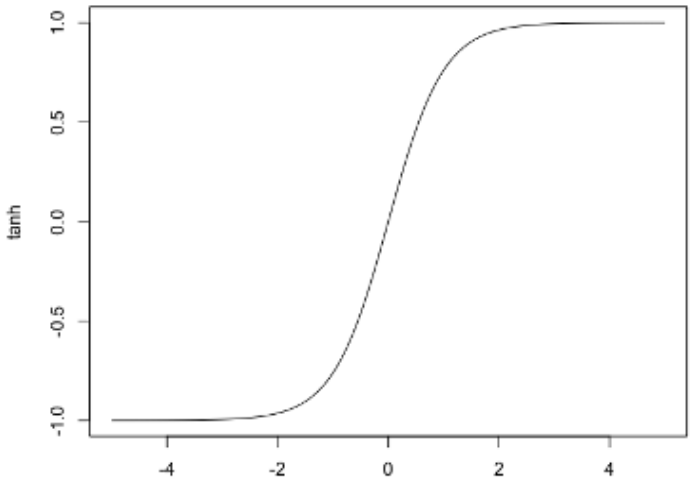
>>> rótulo = Y)

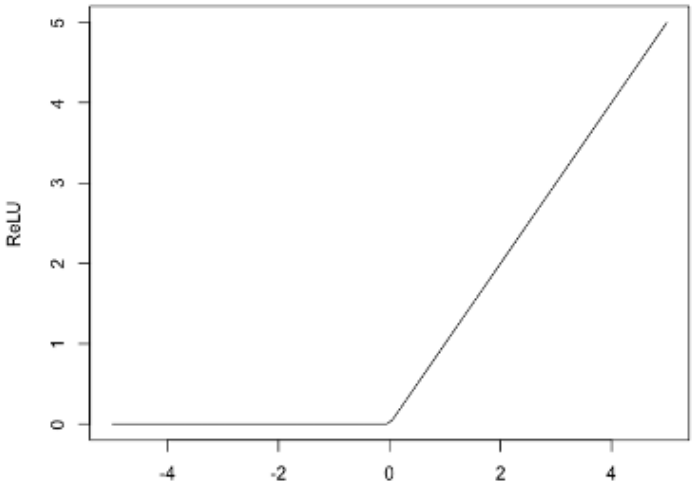
>>> **retornar** loss_grad, [v.name **para** v **em** iter_train.provide_data],

>>> [v.nome **para** v **em** iter_train.provide_label]

Aqui construímos uma rede totalmente conectada de três camadas. A primeira camada possui 20 unidades ocultas e a segunda camada possui 10 unidades ocultas. Após as duas primeiras camadas, há uma

camada de “ativação”. Isto é o que torna o modelo não linear, e sem isso seria **Feed Forward Networks | 319**





seria simplesmente uma série de multiplicações de matrizes que se resumiriam a um modelo linear.

Uma camada totalmente conectada é efetivamente:

$Y = XWT + b$

W é um conjunto de pesos correspondentes às dimensões de correção para uma multiplicação de matrizes, com a saída resultando em um vetor de dimensões iguais às unidades ocultas. Isso é então adicionado com um viés. Os pesos em W e b são treináveis. No entanto, treinar um desses conjuntos de pesos, ou mesmo uma série de tais pesos, não seria muito interessante – seria uma forma indireta de executar uma regressão linear. Porém, é a ativação aplicada após esse conjunto de operações matriciais que gera o interesse. A camada de ativação aplica várias funções não lineares,

como tanh e ReLU, mostrados nas Figuras 10-5 e 10-6, respectivamente.

Figura 10-5. A função tanh exibe comportamento não linear em valores pequenos antes tornando-se funcionalmente constante, com derivada zero, em valores mais elevados.

Figura 10-6. A função ReLU é fácil de calcular e ao mesmo tempo introduz um Não-linearidade.

No código anterior, você notará que usamos a função ReLU como nossa função de ativação. Muitas vezes faz sentido testar uma variedade de funções de ativação (ReLU, tanh, sigmoid) nos estágios iniciais de construção do modelo e ajuste de hiperparâmetros.

Agora vamos treinar com este modelo e ver seu desempenho:

Epoch Training Cor Validação Cor

0 0,578666 0,557897

1 0,628315 0,604025

2 0,645306 0,620324

3 0,654522 0,629658

4 0,663114 0,636299

5 0,670672 0,640713

6 0,677172 0,644602

7 0,682624 0,648478

8 0,688570 0,653288

9 0,694247 0,657925

10 0,699431 0,663191

11 0,703147 0,666099

12 0,706557 0,668977

13 0,708794 0,670228

14 0,711115 0,672429

15 0,712701 0,673287

16 0,714385 0,674821

17 0,715376 0,674976

18 0,716477 0,675744

19 0,717273 0,676195

20 0,717911 0,676139

21 0,718690 0,676634

22 0,719405 0,677273

23 0,719947 0,677286

24 0,720647 0,677451

TESTE DE DESEMPENHO

{'COR': 0,66301745}

Este é um bom desempenho? É difícil saber no vazio. Ao avançar para um modelo de aprendizado profundo, você deverá colocá-lo em produção somente se ele puder superar os modelos substancialmente mais simples que examinamos nos capítulos anteriores. Um modelo de aprendizagem profunda levará mais tempo para calcular uma previsão e geralmente tem uma sobrecarga maior, por isso deve ser usado somente quando puder justificar os custos adicionais. Por esse motivo, quando você aborda um problema de série temporal com ferramentas de aprendizado profundo, geralmente deseja ter modelos anteriores adequados para servir como um padrão objetivo a ser superado.

De qualquer forma, vemos que mesmo um modelo sem qualquer consciência temporal pode treinar e produzir previsões razoavelmente correlacionadas com os valores medidos.

Usando um mecanismo de atenção para avançar

Redes mais conscientes do tempo

Embora as redes feed forward não tenham feito grandes avanços em problemas de séries temporais, ainda há pesquisas sendo feitas para imaginar variações arquiteturais que poderiam melhorar seu desempenho em dados sequenciais. Uma ideia é *a atenção* , que descreve mecanismos adicionados a uma rede neural que permitem aprender em que parte de uma sequência focar e que parte de uma sequência recebida pode estar relacionada aos resultados desejáveis.

A ideia de atenção é que uma arquitetura de rede neural deve fornecer um mecanismo para um modelo aprender quais informações são importantes e quando. Isso está feito

por meio *de pesos de atenção* , que são ajustados para cada intervalo de tempo para que o modelo aprenda como combinar informações de diferentes intervalos de tempo. Esses pesos de atenção multiplicam o que de outra forma seria a saída ou o estado oculto de um modelo, convertendo assim esse estado oculto em um *vetor de contexto* , assim chamado porque o estado oculto agora está melhor contextualizado para explicar e se relacionar com todas as informações contidas em a série temporal ao longo do tempo, incluindo esperançosamente os padrões temporais.

A função Softmax

Você pode não estar familiarizado com a função softmax; aqui está a forma funcional: $softmax(yi) = exp(yi) / \sum j\ exp(yj)$, que serve a dois propósitos:

• Os resultados somarão agora 1, como faria um conjunto completo de probabilidades. Esta é frequentemente a interpretação dos resultados de um softmax.

• Tendo em mente a forma geral da função exponencial, os insumos maiores (yi) tendem a ficar ainda maiores, enquanto os insumos menores tendem a ficar relativamente menores. Isso acentua ativações/pesos fortes e diminui a ênfase dos sinais mais fracos.

A função softmax também é conhecida como função *exponencial normalizada* , com o nome claramente relacionado à forma funcional que vemos aqui.

A atenção começou a ser usada em redes neurais recorrentes (falaremos mais sobre isso posteriormente neste capítulo), mas seu uso e implementação são, na verdade, mais fáceis de entender em um contexto mais amplo.

variante proposta para uma arquitetura feed forward em um artigo de pesquisa. O papel pró-

apresenta uma maneira de aplicar redes neurais feed forward a dados sequenciais, de modo que a reação da rede a cada etapa da sequência possa ser usada como uma entrada para a saída final dessa rede. Essa abordagem também permite sequências de comprimento variável, assim como séries temporais de comprimento variável são comuns em problemas do mundo real.

A Figura 10-7 mostra um exemplo de arquitetura de como uma rede neural feed forward pode ser usada para uma tarefa que envolve o processamento de dados sequenciais.

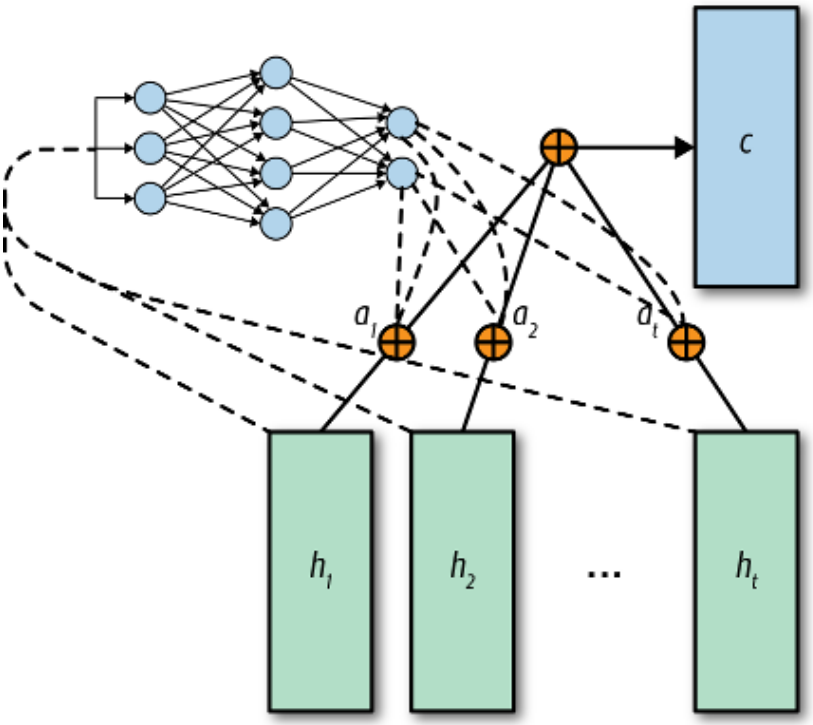


Figura 10-7. Um "mecanismo de atenção feed forward".

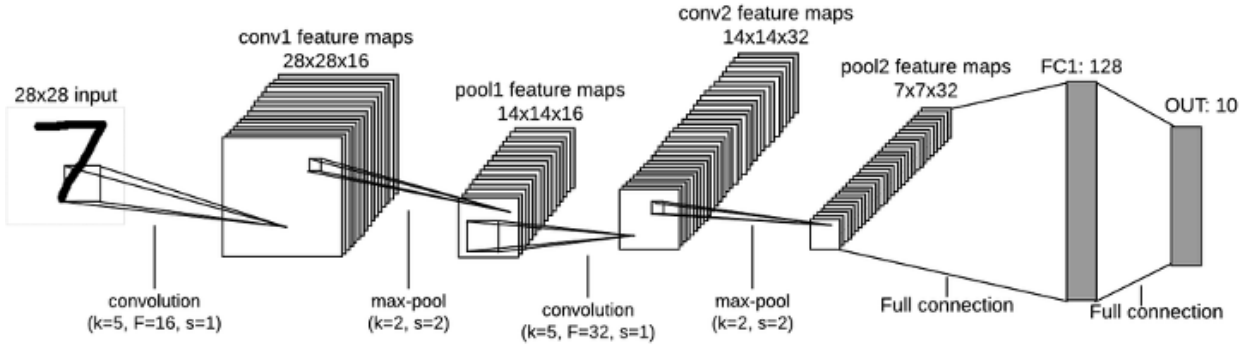
Uma rede feed forward é aplicada às entradas em cada intervalo de tempo individualmente, gerando um "estado oculto" para cada intervalo de tempo *h1* ... *hT* . Isso efetivamente cria uma série de estados ocultos. Uma segunda rede neural, desenvolvida para aprender *a* (*ht*), está representada no canto, e este é o mecanismo de atenção. O mecanismo de atenção permite que a rede

aprenda quanto pesar cada entrada, onde cada entrada representa um estado proveniente de um ponto diferente no tempo. Isso permite que o mecanismo determine quais etapas de tempo terão maior ou menor peso na soma final das entradas.

Em seguida, os estados ocultos de diferentes tempos são combinados com seus coeficientes de atenção antes do processamento final para produzir qualquer alvo/rótulo procurado. Os pesquisadores que projetaram essa rede descobriram que ela teve um desempenho respeitável em uma variedade de tarefas que geralmente exigem uma rede neural recorrente para atender aos requisitos de lembrar entradas anteriores e combiná-las com as posteriores.

Este é um ótimo exemplo de como não existem arquiteturas simples em modelos de aprendizagem profunda porque existem muitas possibilidades de adaptar até mesmo um modelo básico a uma questão complexa de série temporal.

Conforme observado, as redes neurais feed forward não são as principais redes para problemas de séries temporais. No entanto, são um ponto de partida útil para estabelecer o desempenho de um modelo relativamente simples. Curiosamente, se não incluirmos funções de ativação, podemos usá-las para codificar modelos AR e VAR usando a estrutura MXNet. As vezes, isso pode ser útil por si só. Mais importante ainda, existem variações arquitetônicas de modelos totalmente conectados que podem ser bastante precisos para determinados conjuntos de dados de séries temporais.



CNN

Se você já conhece o aprendizado profundo, provavelmente está bastante familiarizado com redes neurais convolucionais (CNNs). A maior parte da funcionalidade semelhante à humana, de cair o queixo e recorde, atribuível aos computadores nos últimos anos foi graças a arquiteturas convolucionais extremamente sofisticadas e complexas. Apesar de tudo isso, a ideia de uma convolução é bastante intuitiva e é muito anterior ao aprendizado profundo.

As convoluções têm sido usadas há muito tempo em formas mais transparentes de controle humano.

estudos de processamento e reconhecimento de imagens, começando com algo tão simples como um desfoque gaussiano. Se você não está familiarizado com os conceitos de kernels de processamento de imagem

e não tem ideia de como você pode codificar um, Stack Overflow fornece uma boa explicação de como isso pode ser feito com uma API de alto nível ou manualmente via NumPy.

Convolução significa aplicar um kernel (uma matriz) a uma matriz maior, deslizando-o pela matriz maior, formando uma nova. Cada elemento da nova matriz é a soma da multiplicação elemento a elemento do kernel e uma subseção da matriz maior. Este kernel é aplicado repetidamente enquanto desliza através de uma matriz/imagem. Isso é feito com um número pré-especificado de kernels para que diferentes recursos possam surgir. Um esquema de como

isso funciona em muitas camadas é mostrado na Figura 10-8.

Figura 10-8. Uma rede convolucional. Muitas janelas bidimensionais de especificações o tamanho do kernel desliza pela imagem original, produzindo muitos mapas de recursos a partir do pesos treináveis aplicados à imagem. Freqüentemente, eles são agrupados e pós-processados com uma função de ativação. O processo é repetido diversas vezes durante vários camadas para reduzir muitos recursos

em um intervalo menor de valores, levando para, por exemplo, classificações de classificação.

Por muitas razões, a convolução tradicional não combina bem com séries temporais. Uma característica principal das convoluções é que todos os espaços são tratados igualmente. Isto faz sentido para imagens, mas não para séries temporais, onde alguns pontos no tempo estão necessariamente mais próximos do que outros.

As redes convolucionais são geralmente estruturadas para serem invariantes em escala, de modo que, digamos, um cavalo possa ser identificado em uma imagem, seja ela uma porção maior ou menor da imagem. No entanto, em séries temporais, novamente, provavelmente desejaremos preservar a escala e os recursos escalonados. Uma oscilação sazonal anual não deve ser interpretada da mesma forma ou

“disparado” pelo mesmo seletor de recursos que uma oscilação diária, embora em alguns contextos isso possa ser útil.

É esta natureza dupla das convoluções – que os seus pontos fortes são os seus pontos fracos de uma perspectiva de série temporal – que faz com que sejam utilizadas mais frequentemente como um componente de uma rede para análise de séries temporais, em vez de toda a rede. Além disso, fez com que fossem estudados com mais frequência para fins de classificação do que de previsão, embora ambos os usos sejam encontrados em campo.

Alguns usos de redes convolucionais para aplicações de séries temporais incluem:

• Estabelecer uma “impressão digital” para o histórico de navegação de um usuário da Internet, o que ajuda a detectar atividades de navegação anômalas

• Identificação de padrões anômalos de batimentos cardíacos a partir de dados de EKG

• Gerar previsões de tráfego com base em gravações anteriores de vários locais em uma cidade grande

As convoluções não são tão interessantes por si só para serem aplicadas a uma série temporal univariada.

Séries temporais multicanal podem ser mais interessantes porque então podemos desenvolver uma série 2D

(ou mesmo 3D) imagem onde o tempo é apenas um eixo.

Há outras inovações arquitetônicas que valem a pena mencionar, pois discutiremos dois exemplos de convoluções para séries temporais na seção seguinte.

Um modelo convolucional simples

Podemos ajustar um modelo convolucional aos nossos dados, em vez de um modelo totalmente conectado, trocando por um modelo diferente. Neste caso (e para os demais exemplos), pegamos os dados no formato NTC, que, como lembrete, tem a seguinte aparência:

Tempo A

B

C

***t* – 1 0, 3 –1, –2 –2, –3**

t

3, 4

–2, –2

–3, –4

Isto é N × T × C. No entanto, os dados esperados por uma camada convolucional são

batch_size, canal, altura × largura:

Pitão

>>> **def** cnn_model(iter_train, input_feature_shape, X, Y,

>>> ganhar, sz_filt, n_filter, drop):

>>> conv_input = mx.sym.reshape(dados=X, forma=(0, 1, vitória, -1))

>>> *## Convolução espera entrada 4d (N x canal x altura x largura)*

>>> *## no nosso caso canal = 1 (semelhante a uma imagem em preto e branco*

>>> *## altura = tempo e largura = canais cortam locais elétricos*

>>>

>>> cnn_output = mx.sym.Convolution(dados=conv_input,

>>> kernel=(sz_filt,

>>> input_feature_shape[2]),

>>> num_filtro=n_filtro)

>>> cnn_output = mx.sym.Activation(data=cnn_output, act_type='relu')

>>> cnn_output = mx.sym.reshape(mx.sym.transpose(dados=cnn_output,

>>> eixos=(0, 2, 1, 3)),

>>> forma=(0, 0, 0))

>>> cnn_output = mx.sym.Dropout(cnn_output, p=drop)

>>>

>>> saída = mx.sym.FullyConnected(dados=cnn_output,

>>> num_hidden=input_feature_shape[2])

>>> loss_grad = mx.sym.LinearRegressionOutput(dados=saída, rótulo=Y)

>>> **retorno** (perda_grad,

>>> [v.nome **para** v **em** iter_train.provide_data],

>>> [v.name **for** v **in** iter_train.provide_label]) Observe que, novamente, isso não inclui qualquer consciência temporal explícita. A diferença é que agora o tempo é disposto ao longo de um único eixo, dando-lhe alguma ordem.

Essa consciência temporal melhora o desempenho? Possivelmente não:

0 0,330701 0,292515

1 0,389125 0,349906

2 0,443271 0,388266

3 0,491140 0,442201

4 0,478684 0,410715

5 0,612608 0,564204

6 0,581578 0,543928

7 0,633367 0,596467

8 0,662014 0,586691

9 0,699139 0,600454

10 0,692562 0,623640

11 0,717497 0,650300

12 0,710350 0,644042

13 0,715771 0,651708

14 0,717952 0,651409

15 0,712251 0,655117

16 0,708909 0,645550

17 0,696493 0,650402

18 0,695321 0,634691

19 0,672669 0,620604

20 0,662301 0,597580

21 0,680593 0,631812

22 0,670143 0,623459

23 0,684297 0,633189

24 0,660073 0,604098

TESTE DE DESEMPENHO

{'COR': 0,5561901}

Pode ser difícil determinar por que um modelo não melhora outro. Na verdade, muitas vezes até mesmo o criador de um modelo pode estar errado sobre por que o modelo funciona tão bem. Existem poucas provas analíticas, e dado o quanto pode depender dos contornos de um dado

definido, isso também pode adicionar confusão. Será que a CNN não reflecte melhor o facto de que a maior parte da informação importante está nos pontos mais próximos no tempo? Ou reflete uma diferença no número de parâmetros? Ou talvez reflita uma falha na escolha de bons hiperparâmetros. Seria importante, em um caso de uso real,

entender se o desempenho parecia razoável dada a estrutura do modelo, a estrutura dos dados e o número geral de parâmetros disponíveis.

Também vemos no código uma falha em nossa lógica de parada antecipada. Parece que foi muito brando. Nesse caso, revisei o problema e percebi que as mudanças na correlação poderiam ser semelhantes às seguintes ao longo de uma série de épocas:

[-0,023024142, 0,03423196, -0,008353353, 0,009730637, -0,029091835]

Isto significa que a mudança na correlação pode ser terrível – até mesmo negativa – muitas vezes, desde que haja melhoria, mesmo que pequena, de vez em quando.

Esta leniência acaba por ser uma má decisão, por isso seria uma boa ideia voltar ao nosso pipeline e ter uma condição de paragem antecipada mais rigorosa. Este é um exemplo de troca de ideias que você encontrará ao se ajustar aos modelos de aprendizagem profunda. É útil fazer vários testes enquanto você constrói seu pipeline para ter uma ideia de quais parâmetros funcionam para seu conjunto de dados e modelos de interesse, tendo em mente que eles variam substancialmente de um conjunto de dados para outro.

Modelos Convolucionais Alternativos

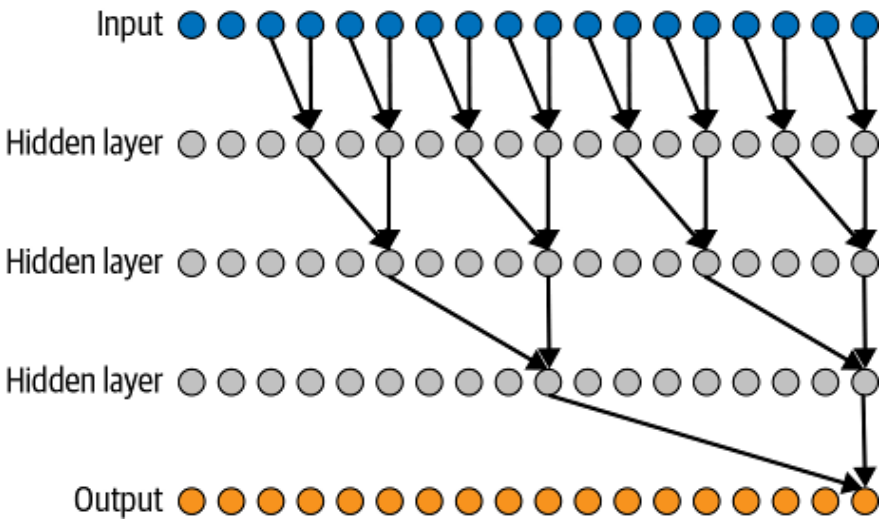
O modelo convolucional simples que acabamos de ver teve um desempenho surpreendentemente bom, embora não incluísse nenhuma modificação para ter consciência do tempo. Agora discutimos duas abordagens da pesquisa e da indústria para usar arquiteturas convolucionais em problemas de séries temporais.

Por que isso é atraente? Existem vários motivos. Primeiro, as arquiteturas convolucionais são métodos testados e comprovados, com uma série de melhores práticas que são bem conhecidas pelos profissionais. Isso torna os modelos convolucionais atraentes porque são uma quantidade conhecida. Além disso, os modelos convolucionais têm poucos parâmetros, uma vez que os mesmos filtros são repetidos continuamente, o que significa que não há muitos pesos para treinar. E, finalmente, grandes porções de modelos convolucionais podem ser computadas em paralelo, o que significa que podem ser muito rápidos para fins de inferência.

Convoluções causais

As convoluções causais são melhor compreendidas com uma imagem, pois esta pode expressar intuitivamente como as convoluções são modificadas para produzir causalidade e uma sensação de tempo. Figura 10-9

mostra um exemplo de uma *convolução causal dilatada* . A parte da causalidade refere-se ao fato de que apenas os pontos anteriores no tempo entram em qualquer filtro convolucional. É por isso que a imagem não é simétrica: os pontos anteriores fluem para as circunvoluções usadas posteriormente, mas não vice-versa.



A parte de dilatação refere-se ao fato de que pontos são ignorados na disposição dos filtros convolucionais, de forma que qualquer ponto vai apenas para um filtro convolucional em cada nível de estratificação. Isso promove a dispersão do modelo e reduz convoluções redundantes ou sobrepostas, permitindo que o modelo olhe mais para trás no tempo, mantendo os cálculos gerais razoavelmente contidos.

Figura 10-9. Este gráfico mostra um componente importante do WaveNet arquitetura.

Aqui vemos como as redes neurais convolucionais têm modificações arquitetônicas apropriado para séries temporais.

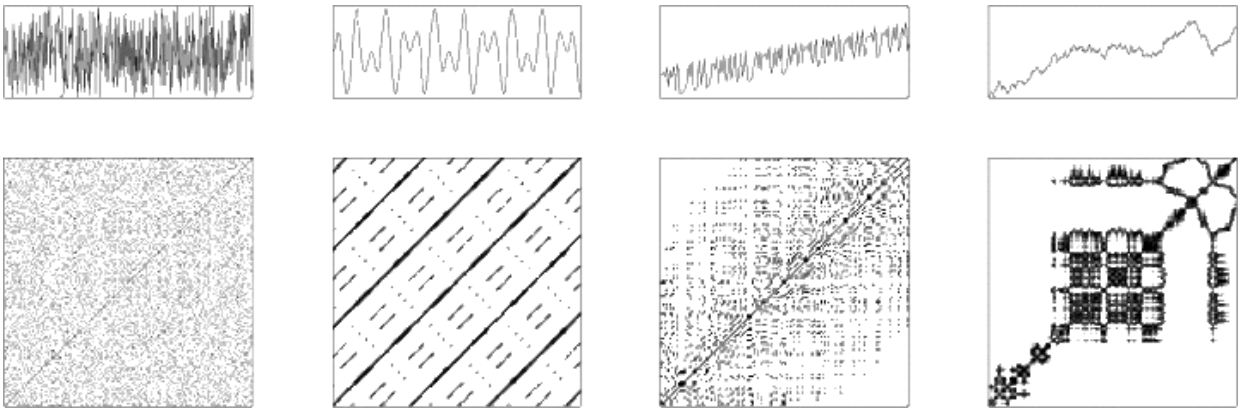
Este exemplo de convolução causal dilatada introduz a noção de causalidade temporal ao permitir apenas dados de pontos temporais anteriores. Ou seja, as circunvoluções nesta imagem não são oportunidades iguais; eles permitem que os dados fluam apenas do passado para o futuro e não o contrário. Cada ponto de dados na entrada original tem um impacto no ponto final. Observe o que dilatação significa aqui, que é que camadas cada vez mais “profundas” na convolução saltam entre números crescentes de pontos de dados nas camadas anteriores. Da forma como a dilatação aqui foi configurada, cada ponto de dados na entrada original é incluído na entrada final, mas apenas uma vez. Isso não é necessário para a dilatação, mas apenas o que foi usado neste caso. A dilatação também pode ser usada para pular completamente os pontos no tempo.

Embora as convoluções causais pareçam complicadas e teóricas, elas são surpreendentemente fáceis de executar. Simplesmente adicione preenchimento no lado esquerdo da matriz - ou seja, adicione substitutos de intervalo de tempo anteriores que sejam zero, para que não contribuam para o valor - e então defina o preenchimento como “válido” para que a convolução apenas executado contra os limites reais da matriz sem incluir células vazias imaginárias. Dado que as convoluções funcionam como a soma dos produtos elemento a elemento, adicionando os zeros ao lado esquerdo de

A Figura 10-9 significaria que poderíamos executar uma convolução padrão e que as células zero extras não

alterariam o resultado final.

As convoluções causais tiveram grande sucesso, principalmente como parte do modelo usado para as tecnologias de conversão de texto em fala e de reconhecimento de fala do Google.



Convertendo uma série temporal em imagens

Sabe-se que os modelos convolucionais funcionam muito bem na análise de imagens, portanto, uma boa ideia ao tentar torná-los relevantes para séries temporais é encontrar uma maneira de transformar uma série temporal em uma imagem. Há muitas maneiras de fazer isso, uma das quais é interessante porque pode transformar até mesmo uma série temporal univariada em uma imagem: construir um gráfico de recorrência

(veja a Figura 10-10).

Figura 10-10. Uma visualização bonita e perspicaz de quatro tipos de séries temporais, de da esquerda para a direita: (1) ruído branco, (2) séries harmônicas/sazonais com duas frequências, (3) dados caóticos com tendência e (4) um processo autorregressivo. Fonte: Wikipedia, fornecida

por Norbert Marwan em 2006.

Um gráfico de recorrência é uma forma de descrever, no espaço de estado de fase, quando uma série temporal revisita aproximadamente a mesma fase e estado em que se encontrava em um momento anterior. Isso é definido por meio da função de recorrência binária. A recorrência é definida como $R(i, j) = 1$ se $f(i) - f(j)$ for suficientemente pequena; 0 caso contrário. Isso resulta em uma imagem binária em preto e branco, como as que vemos representadas na Figura 10-10 . Observe que i e j referem-se aos valores de tempo, e o eixo do tempo não é limitado ou de alguma forma restrito.

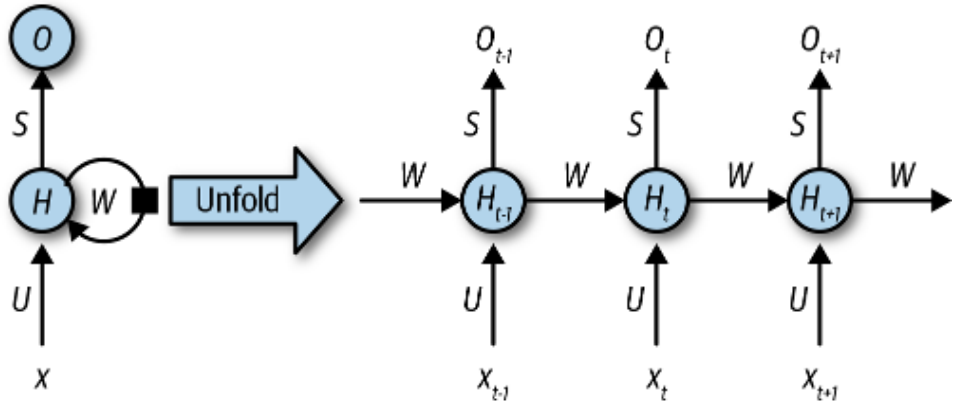
Embora seja relativamente fácil codificar seu próprio gráfico de recorrência, o gráfico de recorrência funciona

A funcionalidade também está disponível em pacotes como pyts e o código-fonte da plotagem é fácil de encontrar e entender.

Podemos imaginar estender essa ideia para além de uma série temporal univariada, tratando diferentes variáveis de uma série temporal multivariada como diferentes “canais” de uma imagem. Este é apenas um exemplo de como as arquiteturas e técnicas de aprendizagem profunda são bastante versáteis e abertas à variação.

RNNs

Redes neurais recorrentes (RNNs) são uma ampla classe de redes em que os mesmos parâmetros são aplicados repetidamente, mesmo quando as entradas mudam com o passar do tempo. Isso se parece muito com a rede neural feed forward que abordamos anteriormente e, ainda assim,



As RNNs constituem – no todo ou em parte – muitos dos modelos de maior sucesso na indústria académica.

demia e indústria para tarefas baseadas em sequência, linguagem, previsão e classificação de séries temporais. As diferenças importantes entre uma RNN e uma rede feed forward são:

• Uma RNN vê os passos de tempo, um de cada vez, em ordem.

• Uma RNN possui um estado que preserva de um passo de tempo para outro, e é esse estado, bem como seus parâmetros estáticos, que determinam as atualizações de sua resposta para cada nova informação em cada passo.

• Uma RNN possui, portanto, parâmetros que a ajudam a “atualizar” seu estado, incluindo um estado oculto, de intervalo de tempo para intervalo de tempo.

Muitas vezes, quando as RNNs são introduzidas, elas são apresentadas através do paradigma de “desenrolar”

porque uma arquitetura recorrente é baseada em células. Isto descreve a maneira como os mesmos parâmetros são usados repetidas vezes, de modo que o número de parâmetros seja bastante pequeno

mesmo para sequências de tempo muito longas (veja a Figura 10-11).

Figura 10-11. Como uma arquitetura de rede neural recorrente é desenrolada uma vez para cada intervalo de tempo quando é aplicado aos dados.

A maneira mais fácil de entender como funciona um RNN, entretanto, pode ser apenas ver um exemplo. Minha célula RNN favorita é a Gated Recurrent Unit (GRU). Às vezes pode ser mais intimidante ver as expressões em forma matemática do que em código, então aqui forneço uma implementação de uma GRU em Python com a ajuda de NumPy. Como você pode ver, existem duas funções de ativação usadas: sigmóide e tanh.

Fora isso, tudo o que fazemos é realizar a multiplicação e adição de matrizes, bem como a multiplicação de matrizes elemento a elemento (o *produto Hadamard*):

Pitão

>>> ## este código foi projetado para funcionar com pesos exportados

>>> ## do TensorFlow

>>> ## https://www.tensorflow.org/api_docs/python/tf/contrib/cudnn_rnn/CudnnGRU

>>> ## mas pode ser facilmente reaproveitado para acomodar outros pesos

>>> **def** calc_gru(X, pesos, num_inputs, num_features):

>>> Nós = pesos[:(3*num_features*num_inputs)]

>>> Nós = np.reshape(Nós, [3, num_features, num_inputs])

>>>

```
>>> Ws = pesos[(3*num_features*num_inputs):
(3*num_features*num_features +

3*num_features*num_inputs)]

>>> Ws = np.reshape(Ws, [3, num_features,
num_features])

>>>

>>> Bs = pesos[(-6 * num_features) :]

>>> Bs = np.reshape(Bs, [6, num_features])

>>> s = np.zeros([129, num_features])

>>> h = np.zeros([129, num_features])

>>>

>>> para t no intervalo (X.shape[0]):

>>> z = sigmóide(np.matmul(Us[0, :, :], X[t, :]) +

>>> np.matmul(Ws[0, :, :], s[t, :]) + Bs[0, :] + Bs[3, :])

>>> r = sigmóide(np.matmul(Us[1, :, :], X[t, :]) +

>>> np.matmul(Ws[1, :, :], s[t, :]) + Bs[1, :] + Bs[4, :])

>>> h[t+1, :] = np.tanh(np.matmul(Us[2, :, :], X[t, :]) +

>>> Bs[2, :] +

>>> r*(np.matmul(Ws[2, :, :], s[t, :]) + Bs[5, :]))

>>> s[t+1, :] = (1 - z)*h[t + 1, :] + z*s[t, :]

>>>

>>> retornar h, s
```

Uma GRU é atualmente uma das células RNN mais utilizadas. É uma versão mais simples da célula Long Short-Term Memory (LSTM), que opera de maneira semelhante. As diferenças entre uma GRU e um LSTM são as seguintes:

• Uma GRU possui duas “portas”, enquanto um LSTM possui três. Essas portas são usadas para determinar quanta informação nova é permitida, quanta informação antiga é permitida.

preservado, e assim por diante. Como um LSTM possui mais portas, ele possui mais parâmetros.

• Um LSTM tende a ter melhor desempenho, mas um GRU é mais rápido de treinar (devido ao número de parâmetros). No entanto, existem resultados publicados onde um GRU supera um LSTM. É particularmente provável que uma GRU supere um LSTM para

tarefas não linguísticas.

Como você pode ver, a diferença é mais uma questão do grau de complexidade apropriado para seus recursos de treinamento e do que você está tentando compreender e prever.

É importante estar familiarizado com a implementação matricial do GRU e LSTM

para que você tenha uma noção de como eles funcionam. Depois de fazer isso, você também poderá desenvolver uma intuição, quando eles não treinarem em um conjunto de dados específico, sobre o motivo pelo qual não o fazem. Pode haver algo na dinâmica que não seja facilmente reconhecido pelo formato mostrado aqui.

Observe que tanto GRUs quanto LSTMs ajudam a resolver o problema que foi encontrado pela primeira vez quando RNNs foram usados, ou seja, gradientes de explosão e desaparecimento. Devido à aplicação recorrente dos mesmos parâmetros, muitas vezes acontecia que os gradientes iam rapidamente para zero (não útil) ou para o infinito (também não útil), o que significa que

a retropropagação foi difícil ou mesmo impossível à medida que a rede recorrente foi desenrolada. Esse problema foi resolvido com GRU e LSTM porque eles tendem a manter as entradas e saídas da célula em faixas de valores tratáveis. Isso se deve tanto à forma da função de ativação que eles usam quanto à maneira como a porta de atualização pode aprender a passar informações ou não, fazendo com que valores de gradiente razoáveis sejam muito mais prováveis do que em uma célula RNN vanilla, que não tem noção de um portão.

Embora GRUs e LSTMs sejam razoavelmente fáceis de fazer DIY, como acabamos de demonstrar, na realidade você não gostaria de fazer isso. A razão mais importante é que muitas das operações de multiplicação de matrizes podem ser fundidas. A implementação mais eficiente e acessível para cuidar disso e explorar o hardware é o cuDNN da NVIDIA, que combina as operações de multiplicação de matrizes necessárias para células GRU e LSTM. Usar a interface cuDNN em vez de outra implementação é substancialmente mais rápido e até foi citado por alguns vencedores do concurso Kaggle como a diferença entre vencer e nem chegar perto, porque ajuda muito a acelerar o treinamento. Todas as principais estruturas de aprendizagem profunda oferecem acesso a esta implementação, embora em alguns casos (como tf.con do TensorFlow

trib.cudnn_rnn), você precisa usar a interface especialmente designada. Em outros casos, como MXNet, você usará cuDNN como padrão, desde que não faça nada sofisticado com células desenroladas personalizadas.

Continuando nosso exemplo elétrico

Também podemos trabalhar com o exemplo da previsão elétrica com um RNN. Novamente, começamos com o formato de dados TNC como entrada. Este é o formato esperado para uma RNN, então nem precisamos alterá-lo:

Pitão

>>> **def** rnn_model(iter_train, janela, filter_size, num_filter,

>>> desistência):

```
>>> input_feature_shape = iter_train.provide_data[0][1]

>>> X = mx.sym.Variable(iter_train.provide_data[0].nome)

>>> Y = mx.sym.Variable(iter_train.provide_label[0].nome)

>>>

>>> rnn_cells = mx.rnn.SequentialRNNCell()

>>>
rnn_cells.add(mx.rnn.GRUCell(num_hidden=args.rnn_units)
)

>>> rnn_cells.add(mx.rnn.DropoutCell(dropout))

>>> saídas, _ = rnn_cells.unroll(comprimento=janela,
entradas=X,

>>> merge_outputs=Falso)

>>>

>>> saída = mx.sym.FullyConnected(dados=saídas[-1],

>>> num_hidden=

input_feature_shape[2])

>>> loss_grad = mx.sym.LinearRegressionOutput(dados =
saída,

>>> rótulo = Y)

>>>

>>> retornar loss_grad, [v.name para v em
iter_train.provide_data],

>>> [v.nome para v em iter_train.provide_label]
```

O desempenho deste modelo é bastante decepcionante, visto que foi projetado para lidar com dados temporais:

Epoch Training Cor Validação Cor

0 0,072042 0,069731

1 0,182215 0,172532

2 0,297282 0,286091

3 0,371913 0,362091

4 0,409293 0,400009

5 0,433166 0,422921

6 0,449039 0,438942

7 0,453482 0,443348

8 0,451456 0,444014

9 0,454096 0,448437

10 0,457957 0,452124

11 0,457557 0,452186

12 0,463094 0,455822

13 0,469880 0,461116

14 0,474144 0,464173

15 0,474631 0,464381

16 0,475872 0,466868

17 0,476915 0,468521

18 0,484525 0,477189

19 0,487937 0,483717

20 0,487227 0,485799

21 0,479950 0,478439

22 0,460862 0,455787

23 0,430904 0,427170

24 0,385353 0,387026

TESTE DE DESEMPENHO

{'COR': 0,36212805}

Seria necessária muito mais investigação para ver por que o desempenho é desanimador.

Não fornecemos parâmetros suficientes para o RNN? Faria sentido adicionar

algumas arquiteturas de modelo relacionadas que são comumente usadas, como atenção? (Isso se aplica a RNNs, bem como a redes feed forward, como foi discutido anteriormente.) O modelo atingiu seu desempenho máximo um pouco mais cedo no processo de treinamento do que os modelos feed forward ou convolucionais. Isto é uma falta de parâmetros suficientes para descrever o conjunto de dados, ou que um RNN é especialmente adaptado para estes dados ou qualquer outra coisa. Você precisaria fazer experimentos e ajustes adicionais para entender o motivo.

A inovação do codificador automático

Ocasionalmente, você pode se deparar com um conjunto de dados onde um modelo muito simples já faz um trabalho espetacular para a missão que você espera cumprir. 4 Mas, por vezes, pensar fora da caixa também pode ser frutífero. Por exemplo, descobriu-se logo no início que uma inovação simples com um RNN também poderia melhorar

substancialmente o desempenho na modelagem do tipo sequência. Embora este modelo tenha sido originalmente desenvolvido para aprendizagem de línguas e tradução automática, muitas vezes provou ser bastante bem-sucedido em tarefas mais numéricas, como previsão de carga elétrica ou preços de ações.

Conhecido como *autoencoder* ou, alternativamente, como modelo *seq2seq* , esse é um modelo muito usado e que você deve incluir como parte de seu kit de ferramentas de aprendizado profundo de série temporal (veja a Figura 10.12). Iremos implantá-lo em vários capítulos de exemplo

posteriormente neste livro, onde analisamos dados de séries temporais do mundo real.

O autoencoder consiste em duas camadas recorrentes, mas não no sentido tradicional, onde cada camada processa cada entrada sucessivamente. Em vez disso, a primeira camada é concluída. Seu estado oculto é então passado para a segunda camada, que utiliza esse estado oculto e suas próprias saídas como novas entradas para a próxima etapa. Este modelo foi formulado particularmente com a ideia de tradução automática, em que produzir uma previsão em cada intervalo de tempo não era significativo, visto que em diferentes idiomas, a ordem de palavras e conceitos pode ser drasticamente diferente e dizer a mesma coisa. A ideia era que, uma vez que a primeira camada tivesse processado totalmente a série temporal, seu estado oculto pudesse passar a ter uma espécie de funcionalidade de resumo. Este resumo seria então injetado no novo modelo, que gradualmente desenrolaria o significado na nova linguagem, combinando este resumo com o seu próprio resultado em cada passo de tempo, para que soubesse o que tinha dito.

Conforme mencionado, apesar das origens deste modelo no processamento de linguagem natural, ele também é útil para tarefas de séries temporais mais tradicionais, como previsão de séries temporais univariadas ou multivariadas. Por exemplo, em uma competição Kaggle para prever o tráfego da web em postagens da Wikipedia, o vencedor do primeiro lugar, depois de tentar muitos hiperparâmetros e componentes arquitetônicos, acabou optando por um modelo de autoencoder. Parece que a vantagem do vencedor provavelmente estava no treinamento inteligente e na

pesquisa de hiperparâmetros, bem como em um bom estudo de geração e seleção de recursos úteis.

4 Há rumores de que em algum momento o Google Translate foi alimentado por um LSTM de sete camadas bastante simples. No entanto, a amplitude do seu conjunto de dados sem dúvida ajudou, assim como o treinamento inteligente e cuidadoso. Nem todos os modelos vanilla são iguais!

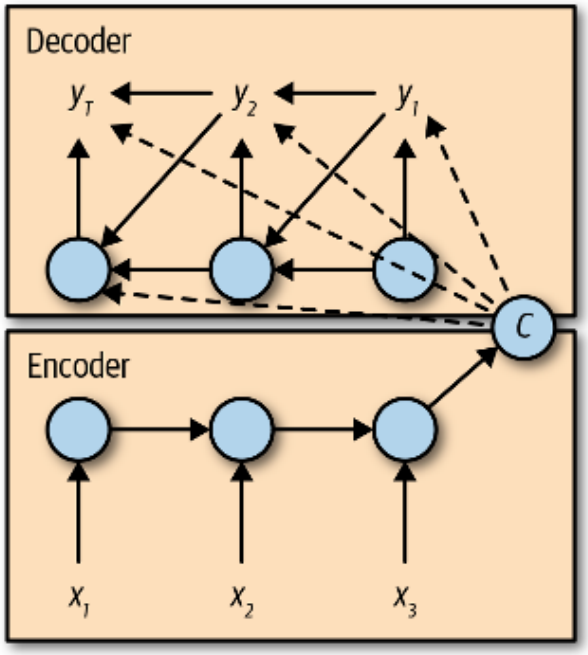


Figura 10-12. Um autoencoder, também conhecido como modelo seq2seq, é muito popular para processamento e modelagem de linguagem, mas também mostrou sucesso significativo no tempo análise de série.

Arquiteturas de Combinação

Muitas vezes, aplicações bem-sucedidas de aprendizagem profunda na indústria e em concursos para previsão de séries temporais usarão algum tipo de arquitetura nova, seja uma nova abordagem na aplicação de células LSTM tradicionais ou na combinação de diferentes componentes. Um exemplo disso é uma arquitetura de rede neural de

2018 proposta por pesquisadores da Carnegie Mellon University. Os pesquisadores procuraram explorar os pontos fortes das arquiteturas convolucional e recorrente, mas também acrescentaram outras inovações.

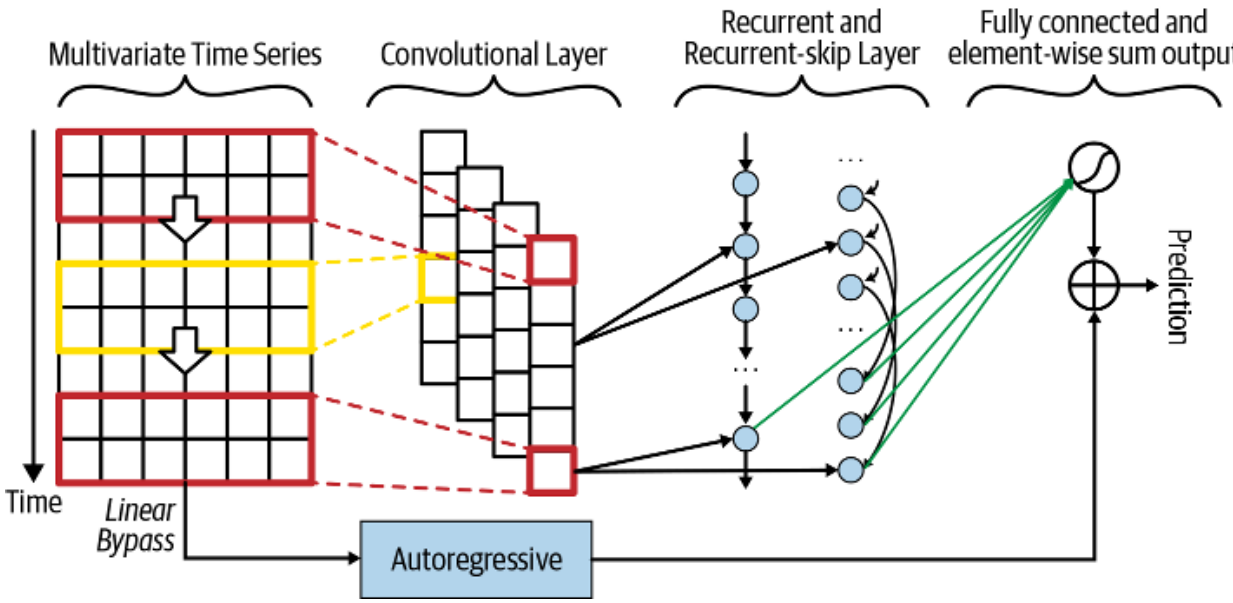
Eles desenvolveram uma camada de “pular recorrente” por meio da qual o modelo recorrente poderia ser ajustado para prestar atenção às periodicidades (como anual, semanal, diária, dependendo da natureza do conjunto de dados) presentes nos dados (e essa periodicidade poderia ser explorada como um hiperparâmetro).

Eles reconheceram que muitas séries temporais apresentam tendências que não são bem modeladas por modelos não lineares de aprendizagem profunda. Isto significava que um modelo de aprendizagem profunda por si só não mostraria as mudanças substanciais de escala ao longo do tempo que alguns conjuntos de dados de séries temporais demonstram. Os investigadores adaptaram-se utilizando um modelo linear tradicional, nomeadamente o modelo do autor.

modelo egressivo que discutimos no Capítulo 6.

O modelo final, o *LSTNet modificado* , foi uma soma das saídas do modelo AR e um modelo construído com uma camada recorrente tradicional e uma camada recorrente de salto em paralelo. As entradas alimentadas em cada camada recorrente foram as saídas das camadas convolucionais

que envolveu ao longo do eixo do tempo e do canal (veja a Figura 10-13).



Os pesquisadores descobriram que em três dos quatro conjuntos de dados que testaram, eles superaram os resultados publicados de última geração, abrangendo uma ampla gama de áreas temáticas. Fracassaram apenas no caso das taxas de câmbio, um domínio das finanças que é notoriamente difícil de prever, com elevados rácios ruído/sinal e um mercado altamente eficiente em que qualquer sinal se dissipa rapidamente à medida que o mercado se torna eficiente e os investidores tentam encontrar uma vantagem.

Figura 10-13. Na arquitetura LSTNet modificada, podemos ver que existe um componente autoregressivo (parte inferior da imagem) em paralelo a uma arquitetura de rede neural. O arquitetura de rede neural coloca um elemento convolucional e um elemento recorrente em ordem consecutiva, operando nas mesmas entradas uma após a outra.

A inspiração para a próxima elaboração vem do artigo dos pesquisadores e trabalharemos com o código5 modificado do catálogo de exemplos do pacote MXNet

descrito detalhadamente pelo autor do exemplo, Oliver Pringle, em uma postagem no blog.

Aqui, conforme observado, aplicamos código baseado em modificações no repositório MXNet, simplificando-o

removendo as conexões sazonais/pular e também usando apenas um tamanho de filtro convolucional. Aplicamos uma camada convolucional, assim como fizemos no exemplo cnn_model anteriormente:

Pitão

>>> ## deve ser 4d ou 5d para usar a funcionalidade de preenchimento

>>> conv_input = mx.sym.reshape(dados=X, forma=(0, 1, vitória, -1))

>>>

>>> *## elemento convolucional*

>>> *## adicionamos preenchimento no final do tempo ganho*

>>> cnn_output = mx.sym.pad(dados=conv_input,

5 Code também está disponível no repositório GitHub pessoal de Oliver Pringle.

>>> modo="constante",

>>> valor_constante=0,

>>> pad_width=(0, 0,

>>> 0, 0,

>>> 0, sz_filt - 1,

>>> 0, 0))

>>> cnn_output = mx.sym.Convolution(dados=cnn_output,

>>> kernel=(sz_filt,

```
>>> input_feature_shape[2]),

>>> num_filtro=n_filtro)

>>> cnn_output = mx.sym.Activation(data=cnn_output,
act_type='relu')

>>> cnn_output =
mx.sym.reshape(mx.sym.transpose(dados=cnn_output,

>>> eixos=(0, 2, 1, 3)),

>>> forma=(0, 0, 0))

>>> cnn_output = mx.sym.Dropout(cnn_output, p=drop)
```

Aplicamos então um RNN não às entradas originais, mas ao componente convolucional, como segue:

```
## Pitão

>>> ## elemento recorrente

>>> stacked_rnn_cells = mx.rnn.SequentialRNNCell()

>>>
stacked_rnn_cells.add(mx.rnn.GRUCell(num_hidden=args.r
nn_units))

>>> saídas, _ = stacked_rnn_cells.unroll(length=win,

>>> entradas=cnn_output,

>>> merge_outputs=Falso)

>>>

>>> rnn_output = saídas[-1]

>>> n_outputs = input_feature_shape[2]

>>> cnn_rnn_model =
mx.sym.FullyConnected(dados=rnn_output,
```

>>> num_hidden=n_outputs)

Finalmente, paralelamente a esta combinação CNN/RNN, treinamos um modelo AR, com um modelo AR por localização de estação elétrica (ou seja, 321 modelos AR distintos, um por coluna/

local variável/elétrico), conforme mostrado aqui. Isso usa todos os pontos no tempo para cada estação, com o modelo sendo especificado uma estação por vez:

Pitão

>>> *## elemento ar*

>>> ar_outputs = []

>>> **para** i **na** lista(range(input_feature_shape[2])):

>>> ar_series = mx.sym.slice_axis(dados=X,

>>> eixo=2,

>>> começo = eu,

>>> fim=i+1)

>>> fc_ar = mx.sym.FullyConnected(dados=ar_series, num_hidden=1)

>>> ar_outputs.append(fc_ar)

>>> ar_model = mx.sym.concat(*ar_outputs, dim=1)

O código completo é o seguinte:

Pitão

>>> **def** simple_lstnet_model(iter_train, input_feature_shape, X, Y,

```
>>> ganhar, sz_filt, n_filter, drop):

>>> ## deve ser 4d ou 5d para usar a funcionalidade de preenchimento

>>> conv_input = mx.sym.reshape(dados=X, forma=(0, 1, vitória, -1))

>>>

>>> ## elemento convolucional

>>> ## adicionamos preenchimento no final do tempo ganho

>>> cnn_output = mx.sym.pad(dados=conv_input,

>>> modo="constante",

>>> valor_constante=0,

>>> pad_width=(0, 0,

>>> 0, 0,

>>> 0, sz_filt - 1,

>>> 0, 0))

>>> cnn_output = mx.sym.Convolution(dados = cnn_output,

>>> kernel=(sz_filt,

>>> input_feature_shape[2]),

>>> num_filtro = n_filtro)

>>> cnn_output = mx.sym.Activation(dados = cnn_output,

>>> act_type = 'relu')
```

```
>>> cnn_output =
mx.sym.reshape(mx.sym.transpose(dados = cnn_output,

>>> eixos = (0, 2, 1, 3)),

>>> forma=(0, 0, 0))

>>> cnn_output = mx.sym.Dropout(cnn_output, p = soltar)

>>>

>>> ## elemento recorrente

>>> stacked_rnn_cells = mx.rnn.SequentialRNNCell()

>>> stacked_rnn_cells.add(mx.rnn.GRUCell(num_hidden =
args.rnn_units))

>>> saídas, _ = stacked_rnn_cells.unroll(length = win,

>>> entradas = cnn_output,

>>> merge_outputs = Falso)

>>> rnn_output = saídas[-1]

>>> n_outputs = input_feature_shape[2]

>>> cnn_rnn_model =
mx.sym.FullyConnected(dados=rnn_output,

>>> num_hidden=n_outputs)

>>> ## elemento ar

>>> ar_outputs = []

>>> para i na lista(range(input_feature_shape[2])):

>>> ar_series = mx.sym.slice_axis(dados=X,

>>> eixo=2,
```

```
>>> começo = eu,

>>> fim=i+1)

>>> fc_ar = mx.sym.FullyConnected(dados = ar_series,

num_oculto = 1)

>>> ar_outputs.append(fc_ar)

>>> ar_model = mx.sym.concat(*ar_outputs, dim=1)

>>>

>>> saída = cnn_rnn_model + ar_model

>>> loss_grad =
mx.sym.LinearRegressionOutput(dados=saída, rótulo=Y)

>>> retorno (perda_grad,
```



```
>>> [v.nome para v em iter_train.provide_data],

>>> [v.name for v in iter_train.provide_label]) Observe que
o desempenho é surpreendentemente melhor neste modelo
do que qualquer outro: Epoch Training Cor Validation Cor

0 0,256770 0,234937

1 0,434099 0,407904

2 0,533922 0,506611

3 0,591801 0,564167

4 0,630204 0,602560

5 0,657628 0,629978
```

6 0,678421 0,650730

7 0,694862 0,667147

8 0,708346 0,680659

9 0,719600 0,691968

10 0,729215 0,701734

11 0,737400 0,709933

12 0,744532 0,717168

13 0,750767 0,723566

14 0,756166 0,729052

15 0,760954 0,733959

16 0,765159 0,738307

17 0,768900 0,742223

18 0,772208 0,745687

19 0,775171 0,748792

20 0,777806 0,751554

21 0,780167 0,754034

22 0,782299 0,756265

23 0,784197 0,758194

24 0,785910 0,760000

TESTE DE DESEMPENHO

{'COR': 0,7622162}

Obviamente, esse modelo se sai um pouco melhor do que os outros modelos, portanto, há algo especial e útil nessa arquitetura que usa imagens convolucionais como dados sequenciais nos quais a camada recorrente é treinada. A tradicional ferramenta estatística de

um modelo AR também adiciona um pouco de funcionalidade.6 Este modelo é de longe o

melhor, e esta é uma boa lição para a pequena quantidade de exploração de dados e treinamento que fizemos. Vale a pena experimentar vários modelos, e você nem precisa gastar muito tempo treinando todos os modelos para descobrir um líder claro.

6 Tente treinar sem o componente AR se você não acredita em mim – não é difícil modificar o código para remover esse componente.

Resumindo

Um aspecto interessante do nosso treinamento neste capítulo é que o desempenho dos modelos não ocorreu da maneira que esperávamos. A rede feed forward simples superou substancialmente alguns modelos que parecem mais conceitualmente

complicado. No entanto, isto não resolve necessariamente a questão de quais modelos são melhores ou piores para o nosso conjunto de dados por uma série de razões:

• Não verificamos a quantidade de parâmetros utilizados em cada modelo. É possível que diferentes gêneros de modelos tenham desempenhos drasticamente diferentes com diferentes números de parâmetros. Poderíamos brincar com a complexidade do modelo, como número de camadas convolucionais/recorrentes ou número de filtros/unidades ocultas.

• Não ajustamos hiperparâmetros. Às vezes, obter os hiperparâmetros corretos pode fazer uma enorme diferença no desempenho de um modelo.

• Não exploramos nossos dados o suficiente para ter uma ideia prévia de qual modelo devemos

esperaria ter um desempenho melhor ou pior, dadas as correlações ao longo do tempo e entre diferentes colunas/locais elétricos.

Como desenvolver suas habilidades

Se você começar a navegar em periódicos acadêmicos, conferências e postagens do arXiv procurando especificamente recursos de aprendizagem profunda para séries temporais, muitas vezes ficará frustrado com algumas convenções na literatura:

• Grande parte da pesquisa de séries temporais se vê primeiro como um domínio específico e depois como série temporal. Isso significa que você realmente deveria procurar domínios relacionados, como economia, saúde e estudos climáticos.

• Os pesquisadores de aprendizagem profunda muitas vezes pensam em "aprendizagem profunda" ou "redes neurais" como redes convolucionais. Portanto, se você está realmente tentando se aprofundar especificamente em "redes neurais recorrentes", procure essas palavras no título. A maioria dos artigos sem essas palavras (ou similares) no título não terá RNNs no corpo da pesquisa.

• Infelizmente, ninguém ainda escreveu um modelo de aprendizagem profunda para ajudá-lo a encontrar artigos de aprendizagem profunda, mas esse seria um projeto que valeria a pena.

Mais recursos

• Documentos históricos:

Sepp Hochreiter e Jürgen Schmidhuber, "Long Short-Term Memory", Neural Computação 9, não. 8 (1997):1735–80, https://perma.cc/AHR3-FU5H.

Este trabalho seminal de 1997 introduziu a célula de memória de longo e curto prazo

(LSTM) e também propôs vários benchmarks experimentais que permanecem em

uso hoje para estudar o desempenho da rede neural na análise de sequência.

Peter G. Zhang, Eddy Patuwo e Michael Hu, "Previsão com Neu-

Redes rais: O Estado da Arte", International Journal of Forecasting 14, no. 1

(1998): 35–62, https://perma.cc/Z32G-4ZQ3.

Este artigo fornece uma visão geral do estado da arte em séries temporais e aprendizado profundo em 1998.

• Em RNNs:

Aurélien Geron, "Processando sequências usando RNNs e CNNs", em Hands-On

Aprendizado de máquina com Scikit-Learn, Keras e TensorFlow, 2ª edição (Sebastopol: O'Reilly Media, Inc., 2019).

Aurélien Geron oferece um conjunto excepcionalmente extenso de exemplos sobre como

aplique aprendizado profundo a dados de séries temporais neste livro popular. Se você está com-

confortável com essas ferramentas, este notebook Jupyter oferece um excelente exemplo de muitos tipos diferentes de modelos aplicados a dados sequenciais, incluindo alguns

exercícios com soluções.

Valentin Flunkert, David Salinas e Jan Gasthaus, "DeepAR: Análise Probabilística

lançando com redes recorrentes autorregressivas”, artigo não publicado, 2017, https://

perma.cc/MT7N-A2L6.

Este artigo inovador ilustrou o modelo da Amazon que foi desenvolvido

para ajustar séries temporais para seus dados de varejo que ocorrem em uma grande variedade de escalas e com tendências. Uma inovação particular do trabalho foi a capacidade de fazer

previsões probabilísticas em vez das estimativas pontuais usuais que tendem a ser o resultado de análises de aprendizagem profunda.

Lingxue Zhu e Nikolay Laptev, “Previsão profunda e confiável para séries temporais

at Uber”, artigo apresentado na Conferência Internacional IEEE sobre Dados de 2017

Oficinas de Mineração (ICDMW), Nova Orleans, LA, https://perma.cc/PV8R-PHV4.

Este artigo ilustra outro exemplo de probabilidade e estatisticamente

modificações inspiradas em RNNs típicos. Neste caso, a Uber propôs um romance

Modelo profundo bayesiano que forneceu uma estimativa pontual e uma incerteza

estimativa que poderia ser implantada na produção com razoavelmente rápido

desempenho.

Zhengping Che et al., "Redes Neurais Recorrentes para Séries Temporais Multivariadas

com valores ausentes", Scientific Reports 8, no. 6085 (2018), https://perma.cc/

4YM4-SFNX.

Este artigo fornece um forte exemplo de trabalho de última geração em medicina

série temporal. Ele demonstra o uso de uma GRU, combinada com novas tecnologias arquitetônicas.

turas para contabilizar os dados faltantes e transformar a falta em um informativo

atributo. Os autores apresentam redes neurais que superam todas as métricas clínicas existentes atualmente implantadas para fazer previsões sobre a saúde do paciente e

estatísticas de internação hospitalar. Este é um ótimo exemplo de como uma modificação intuitiva e fácil de entender em uma estrutura RNN simples e amplamente utilizada (o

GRU) pode levar a resultados inovadores em um bom conjunto de dados.

• Nas CNNs:

Aäron van den Oord e Sander Dieleman, "WaveNet: Um modelo generativo para

Áudio bruto DeepMind," Blog DeepMind, 8 de setembro de 2016, https://perma.cc/

G37Y-WFCM.

Este blog fornece uma descrição extremamente bem feita e acessível de um

arquitetura CNN inovadora que foi usada para aprimorar tanto a conversão de texto em

tecnologias de fala e fala para texto em vários idiomas e com

alto-falantes diferentes. A nova arquitetura levou a um aumento significativo no desempenho e foi posteriormente implantada em outras tarefas de IA relacionadas à sequência,

particularmente no que diz respeito à previsão de séries temporais.

• Sobre a aplicação da aprendizagem profunda:

Vera Rimmer et al., "Impressão digital automatizada de sites por meio de aprendizado profundo",

artigo apresentado no NDSS 2018, San Diego, CA, https://perma.cc/YR2G-UJUW.

Este artigo ilustra uma maneira pela qual o aprendizado profundo pode ser usado para descobrir

informações privadas sobre o conteúdo de navegação de um usuário na Internet através da Web

impressão digital do site. Em particular, os autores destacaram uma maneira pela qual várias arquiteturas de redes neurais poderiam ser usadas para formular websites.

Ataques de impressão digital para violar as proteções de privacidade do usuário.

CPMP, "Solução de segundo lugar para a competição Kaggle Web Traffic Forecasting

ção", blog Kaggle, 2017, https://perma.cc/UUR4-VNEU.

Este blog, escrito antes da conclusão da competição, descreve o

pensamento do vencedor do segundo lugar ao projetar um aprendizado de máquina misto/profundo

solução de aprendizagem para previsão. Há também alguns comentários retrospectivos

tary disponível em uma postagem de blog relacionada. Este é um ótimo exemplo de uma mistura de

pacotes modernos e um estilo de codificação relevante e prático. O repositório GitHub

para a solução de primeiro lugar também está disponível juntamente com uma discussão de seus novos

arquitetura baseada em rede ral.

CAPÍTULO 11

Erro de medição

Nos capítulos anteriores utilizamos uma variedade de medidas para comparar modelos ou julgar quão bem um modelo executou sua tarefa. Neste capítulo examinamos as melhores práticas para avaliar a precisão das previsões, com ênfase nas preocupações específicas dos dados de séries temporais.

Para aqueles que são novos na previsão de séries temporais, é mais importante compreender que a validação cruzada padrão geralmente não é aconselhável. Você não pode selecionar conjuntos de dados de treinamento, validação e teste selecionando amostras de dados selecionadas aleatoriamente para cada uma dessas categorias de maneira independente do tempo.

Mas é ainda mais complicado do que isso. Você precisa pensar em como diferentes amostras de dados se relacionam entre si no tempo, mesmo que pareçam independentes. Por exemplo, suponha que você esteja trabalhando em uma tarefa de classificação de série

temporal, de modo que tenha muitas amostras de séries temporais separadas, onde cada uma é seu próprio ponto de dados. Pode ser tentador pensar que, neste caso, você pode escolher aleatoriamente séries temporais para cada treinamento, validação e teste, mas seria errado seguir essa ideia. O problema com essa abordagem é que ela não refletirá como você usaria seu modelo, ou seja, seu modelo será treinado em dados anteriores e testado em dados posteriores.

Você não quer que informações futuras vazem para o seu modelo porque não é assim que isso acontecerá em uma situação de modelagem do mundo real. Isso, por sua vez, significa que o erro de previsão medido em seu modelo será menor durante o teste do que na produção, porque nos testes você terá validado cruzado seu modelo de uma forma que produzirá informações futuras (ou seja, feedback sobre qual modelo usar).

Aqui está um cenário concreto que demonstra como isso pode acontecer. Imagine que você está treinando um monitor de qualidade do ar para as principais cidades do oeste dos Estados Unidos. Em seu conjunto de treinamento, você inclui todos os dados de 2017 e 2018 de São Francisco, Salt Lake City, Denver e San Diego. Em seu conjunto de testes, você inclui o mesmo intervalo de datas para Las Vegas, Los Angeles, Oakland e Phoenix. Você descobre que seu modelo de qualidade do ar **343**

vai particularmente bem em Las Vegas e Los Angeles, mas também extremamente bem em 2018

geral. Ótimo!

Em seguida, você tenta replicar seu processo de treinamento de modelo em dados de décadas anteriores e descobre que ele não testa tão bem quanto treina em outras execuções de treinamento/teste. Você então se lembra dos incêndios florestais recordes no sul da Califórnia em 2018 e percebe que eles foram “incorporados” ao teste/treinamento original porque seu conjunto de treinamento lhe deu uma janela para o futuro. Esta é uma razão importante para evitar a validação cruzada padrão.

Pode haver momentos em que não seja um problema que as informações do futuro se propaguem para trás na escolha de um modelo. Por exemplo, se você está apenas procurando entender a dinâmica de uma série temporal testando quão bem você pode predizê-la, você não está procurando prever, por si só, mas sim testar o melhor ajuste possível de um determinado modelo aos dados. . Neste caso, incluir dados do futuro é útil para compreender a dinâmica, embora seja aconselhável ter cuidado com o sobreajuste. Portanto, é indiscutível que, mesmo neste caso, a manutenção de um conjunto de testes válido – que exige que não se deixe vazar informação do futuro – ainda justificaria preocupações sobre séries temporais e validação cruzada.

Concluído este comentário geral, voltamo-nos para um exemplo da mecânica concreta de divisão de dados para treinar, validar e testar um modelo. Em seguida, falamos de forma mais geral sobre como determinar quando uma previsão é boa o suficiente, ou tão boa quanto razoavelmente possível. Também analisamos como estimar a incerteza da nossa previsão quando utilizamos técnicas que não produzem diretamente uma medida de incerteza ou de erro como parte do resultado. Concluímos com uma lista de dicas que podem ser úteis para revisar de tempos em tempos quando você estiver construindo um modelo de série temporal ou se preparando para colocá-lo em produção. Isso pode ajudá-lo a evitar algum constrangimento!

O básico: como testar previsões

O elemento mais importante da geração de uma previsão é certificar-se de que você a está construindo apenas com dados que você possa acessar com antecedência suficiente para que esses dados sejam usados na geração da previsão. Por esse motivo, você precisa pensar não apenas em quando os eventos acontecem, mas também em quando os dados estarão disponíveis para você. [1]

Embora isso pareça bastante simples, lembre-se de que o pré-processamento comum, como

suavização exponencial, pode inadvertidamente levar a vazamentos do período de treinamento para o período de teste. Você mesmo pode testar isso em um exemplo,

ajustando uma regressão linear primeiro a uma série temporal autoregressiva e depois a uma série temporal suavizada exponencialmente. 1 Aliás, modelar com ou sem dados disponíveis em determinados momentos para mostrar a importância da entrega oportuna de dados pode motivar seus engenheiros de dados e gerentes a priorizar entradas de dados específicas se você puder demonstrar que isso faria diferença na precisão do seu modelo e/ou nos resultados financeiros da sua organização.

séries temporais autorregressivas. Você verá que quanto mais suavizar a série temporal e quanto mais longa for a meia-vida de suavização, "melhores" serão suas previsões. Isso ocorre porque, na verdade, você está tendo que fazer cada vez menos previsões, pois cada vez mais seu valor é composto por uma média exponencial de valores passados. Esta é uma previsão bastante perigosa e tão insidiosa que ainda aparece em trabalhos acadêmicos!

Devido a estes perigos e a outras formas difíceis de identificar de alimentar o futuro no passado, ou vice-versa, o padrão-ouro para qualquer modelo deve ser o backtesting com períodos de treino, validação e testes que avançam.

No backtesting, um modelo é desenvolvido para um conjunto de datas, ou um intervalo de datas, mas depois é testado extensivamente em dados históricos, de preferência representando toda a gama de condições e variabilidade possíveis. Também importante é que os profissionais precisam ter razões de princípio para testar um modelo específico e devem tentar evitar testar muitos modelos. Como a maioria dos analistas de dados sabe, quanto mais modelos você testar, maior será a probabilidade de você se ajustar demais aos seus dados – ou seja, escolher um modelo que tenha selecionado detalhes excessivamente específicos sobre o conjunto de dados atual, em vez de generalizar de maneira robusta. Infelizmente para os profissionais de séries temporais, esse é um ato de equilíbrio complicado e que pode levar a resultados embaraçosos quando você coloca modelos em produção.

Concretamente, como implementamos alguma forma de backtesting? Fazemos isso de uma forma que preserva uma

estrutura semelhante à validação cruzada, embora temporalmente consciente. O paradigma comum, supondo que você tenha dados que representem o tempo passando sequencialmente

“em ordem alfabética” é o seguinte:

Treine com [A]

teste com [B]

Treine com [AB]

teste com [C]

Treine com [ABC]

teste com [D]

Treine com [ABCD]

teste com [E]

Treine com [ABCDE] teste com [F]

A Figura 11-1 ilustra essa estrutura de teste.

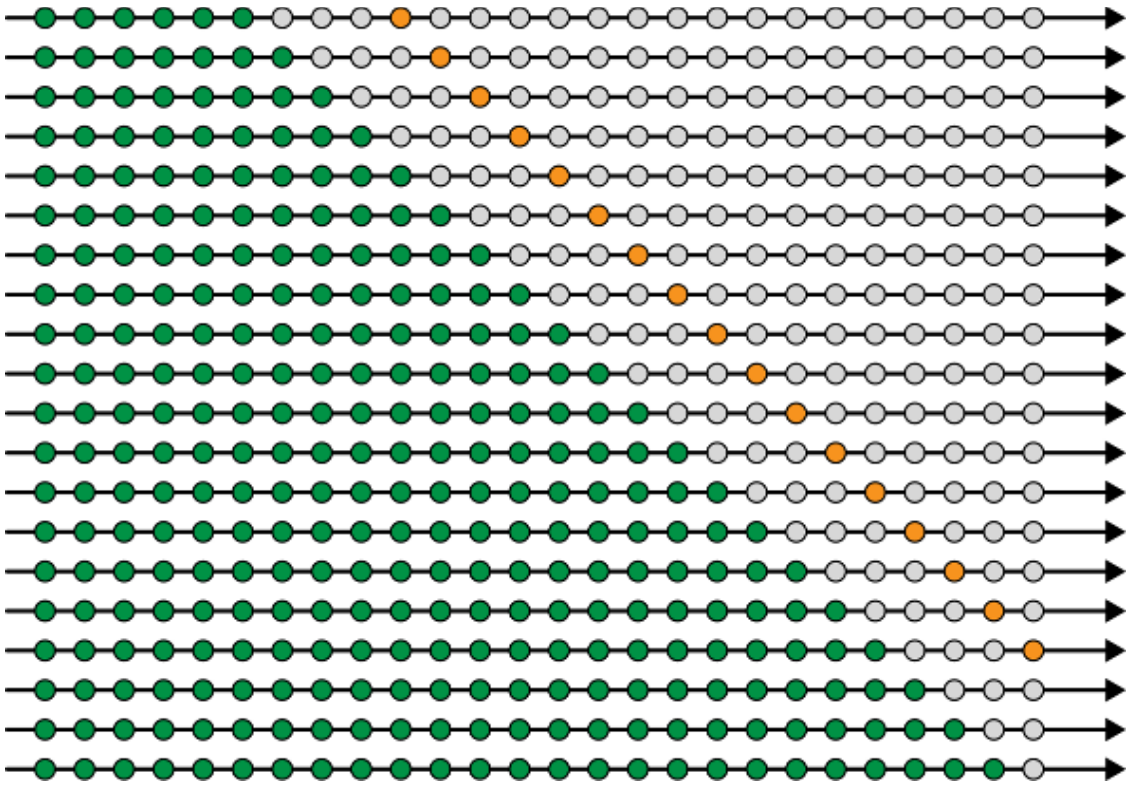


Figura 11-1. O padrão ouro para avaliar o desempenho de um modelo de série temporal, ou seja, avançar suas janelas de treinamento, validação e teste.

Também é possível mover a janela de dados de treinamento em vez de expandi-la. Nesse caso, seu treinamento poderia ser mais ou menos assim:

Treine com [AB] teste com [C]

Treine com [BC] teste com [D]

Treine com [CD] teste com [E]

Treine com [DE] teste com [F]

O método escolhido depende em parte se você acha que o comportamento da sua série está evoluindo ao longo do tempo. Se você acha que sim, será melhor servido por uma janela móvel para que todos os períodos de teste sejam testados com um modelo treinado para os dados mais relevantes. Outra consideração poderia ser que você deseja evitar o ajuste excessivo, onde o uso de uma janela de

expansão disciplinará melhor seu modelo do que uma janela de comprimento fixo.

Como esse tipo de divisão contínua é uma necessidade comum de treinamento, tanto R quanto Python oferecem maneiras fáceis de gerá-la:

• Em Python, uma maneira fácil de gerar divisões de dados é com sklearn.model_selec tion.TimeSeriesSplit.

• Em R, o tsCV do pacote de previsão avançará um modelo no tempo usando esse esquema de backtesting e reportará os erros.



Existem outros pacotes em R e Python que farão o mesmo. Você também pode escrever suas próprias funções para dividir seus dados se tiver ideias específicas sobre como implementar esse teste de modelo para um projeto específico. Talvez você queira pular determinados períodos de tempo porque eles exibiram dinâmicas anômalas ou talvez queira avaliar mais o desempenho de determinados períodos de tempo.

Por exemplo, suponha que você trabalhe com dados financeiros. Dependendo dos seus objetivos, pode valer a pena excluir dados de tempos extraordinários, como a crise financeira de 2008. Ou, se trabalhar com dados de retalho, poderá querer ponderar mais fortemente o desempenho do modelo para a época de compras de Natal, mesmo que sacrifique alguns precisão nas previsões de temporadas de menor volume.

Considerações específicas do modelo para backtesting

Considere a dinâmica do modelo que você está treinando ao estruturar seu backtesting, principalmente no que diz

respeito ao que significa treinar um modelo com um intervalo temporal específico de dados.

Com modelos estatísticos tradicionais, como ARIMA, todos os pontos de dados são considerados igualmente ao selecionar os parâmetros do modelo, portanto, mais dados podem tornar o modelo menos preciso se você achar que os parâmetros do modelo devem variar ao longo do tempo. Isto também é

verdadeiro para modelos de aprendizado de máquina para os quais todos os dados de treinamento são considerados igualmente.2

Os métodos estocásticos em lote, por outro lado, podem resultar em pesos e estimativas que evoluem ao longo do tempo. Portanto, modelos de redes neurais treinados com métodos estocásticos típicos de descida de gradiente levarão, até certo ponto, a natureza temporal dos dados em consideração se você treinar os dados em ordem cronológica. Os ajustes de gradiente mais recentes no peso refletirão os dados mais recentes. Na maioria dos casos, os modelos de redes neurais de séries temporais são treinados nos dados em ordem cronológica, pois isso tende a produzir melhores resultados do que os modelos treinados nos dados em ordem aleatória.

Não faça buracos nos seus dados. A própria dificuldade e beleza de

dados de série temporal é que os pontos de dados são autocorrelacionados. Por esta

razão, não podemos selecionar aleatoriamente pontos dentro do nosso ser-

para validação ou teste porque isso destruirá alguns dos

autocorrelação em nossos dados. Como resultado, o reconhecimento do nosso modelo de

um componente autorregressivo de nossos dados seria prejudicado, um

resultado mais indesejável.

2 Observe que você pode escrever sua própria função de ponderação personalizada para sua função de perda como uma forma de ponderar mais os dados mais recentes, mas isso requer habilidade de programação acima da média e conhecimento sobre otimizações numéricas.



Os modelos de espaço de estados também oferecem oportunidades para o ajuste se adaptar ao modo ao longo do tempo. Isto favorece uma janela de treinamento mais longa porque uma janela de tempo longa não impedirá que a estimativa posterior evolua ao longo do tempo.

Capture um instantâneo do seu modelo ao longo do tempo

Porque você testará seus modelos avançando-os no tempo

depois de ajustá-los aos dados anteriores, você precisa de uma maneira fácil de salvar um

modelo com um carimbo de data/hora para que você saiba o primeiro momento

que você poderia ter usado apropriadamente esse modelo. Isso vai ajudar

você evita testar inadvertidamente um modelo em seus próprios dados de treinamento. Isto

também lhe dará a oportunidade de aplicar vários modelos diferentes

desde diferentes períodos de tempo até dados de teste. Esta pode ser uma forma de

ver se importa quão recentemente um modelo foi treinado com

relação à precisão desse modelo nos dados de teste, o que pode, em última análise,

ajudá-lo a escolher um ritmo para a frequência com que seus modelos de série temporal

precisa ser reajustado no código de produção.

Quando sua previsão é boa o suficiente?

Quando sua previsão é boa o suficiente depende de seus objetivos gerais, da qualidade mínima com a qual você pode "se safar" para o que precisa fazer e dos limites e da natureza de seus dados. Se os seus dados em si tiverem uma relação ruído-sinal muito alta, você deverá ter expectativas limitadas para o seu modelo.

Lembre-se de que um modelo de série temporal não é perfeito. Mas você deve esperar se sair tão bem ou um pouco melhor que os métodos alternativos, como resolver um sistema de equações diferenciais sobre mudanças climáticas, pedir uma dica a um corretor de ações bem informado ou consultar um livro médico que mostre como classificar traços de EEG. À medida que você avalia

desempenho, tenha em mente os limites conhecidos dos especialistas do domínio para a previsão, conforme indicam essas medidas - por enquanto, o limite superior para o desempenho em muitos problemas de previsão.

Há momentos em que você sabe que seu modelo ainda não é bom o suficiente e que pode fazer melhor. Aqui estão algumas ações que você pode realizar para identificar essas oportunidades: *Trace os resultados do seu modelo para o conjunto de dados de teste*

A distribuição produzida pelo modelo deve corresponder à distribuição dos valores que se pretende prever, assumindo

que não existe nenhuma mudança de regime esperada ou tendência subjacente. Por exemplo, se você está tentando prever os preços das ações e sabe que os preços das ações sobem e descem com a mesma frequência, você tem uma

modelo inadequado se o modelo sempre prevê um aumento de preços. Às vezes, as distribuições estarão obviamente erradas, enquanto outras vezes você pode aplicar uma estatística de teste para comparar a saída do seu modelo com seus alvos reais.

Trace os resíduos do seu modelo ao longo do tempo

Se os resíduos não forem homogêneos ao longo do tempo, seu modelo está subespecificado.

O comportamento temporal dos resíduos pode indicar parâmetros adicionais necessários em seu modelo para descrever o comportamento temporal.

Teste seu modelo em um modelo nulo simples com reconhecimento temporal

Um modelo nulo comum é que cada previsão para o tempo t deve ser o valor no tempo t – 1. Se o seu modelo não estiver superando um modelo tão simples, não há qualquer justificativa para isso. Se um modelo simples e ingênuo para todos os fins pode superar o modelo que você criou, é um problema fundamental com seu modelo, função de perda ou dados

pré-processamento em vez de uma questão de pesquisa em grade de hiperparâmetros. Alternativamente, pode ser um sinal de dados que apresenta muito ruído em relação ao sinal, o que também sugere

sugere que seu modelo é inútil para os fins pretendidos.3

Estude como seu modelo lida com outliers

Em muitos setores, os valores discrepantes são simplesmente o que são chamados: valores discrepantes.

Provavelmente não há como prever esses eventos, 4 o que significa que o melhor que seu modelo pode fazer é ignorar esses valores discrepantes em vez de ajustá-los. Na verdade, seu modelo prevendo muito bem valores discrepantes pode até ser um sinal de overfitting ou seleção inadequada da função de perda. Isso depende do modelo escolhido e das funções de perda empregadas, mas para a maioria das aplicações você desejará um modelo cujas previsões não sejam tão extremas quanto os valores extremos em seu conjunto de dados. É claro que este conselho não se aplica quando o custo de eventos discrepantes é alto e quando a tarefa de previsão consiste principalmente em alertar sobre eventos discrepantes quando possível.

Realizar análise de sensibilidade temporal

Os comportamentos qualitativamente semelhantes em séries temporais relacionadas estão produzindo resultados relacionados em seu modelo? Usando o seu conhecimento da dinâmica subjacente do seu sistema, certifique-se de que isso seja verdade e que o seu modelo reconheça e trate padrões temporais semelhantes da mesma maneira. Por exemplo, se uma série temporal mostrar uma tendência ascendente com um desvio de 3 unidades por dia e outra mostrar uma tendência ascendente com um desvio de 2,9 unidades por dia, você provavelmente desejará ter certeza de que as previsões feitas para essas séries serão eram semelhantes. Além disso, você gostaria de ter certeza de que a classificação das previsões em comparação com os dados de entrada era sensata (um desvio maior deve resultar em um valor de previsão maior). Se este não for o caso, seu modelo pode estar superajustado.

Esta não é uma lista completa das muitas maneiras de testar seu modelo de série temporal, mas pode ser um ponto de partida que você desenvolverá à medida que ganhar experiência em um domínio específico.

3 Um modelo nulo simples pode ser surpreendentemente difícil de superar.

4 Em alguns casos, pode não haver nada a aprender para decisões futuras.

Estimando a incerteza em seu modelo com uma simulação

Uma vantagem da análise estatística tradicional de séries temporais é que tais análises vêm com fórmulas analíticas bem definidas para a incerteza em uma estimativa. Contudo, mesmo nesses casos – e certamente no caso de métodos não estatísticos – pode ser útil compreender a incerteza associada a um modelo de previsão através de métodos computacionais. Uma maneira muito intuitiva e acessível de fazer isso é uma simulação simples.

Suponha que conduzimos uma análise do que acreditamos ser um processo AR(1). Como lembrete, um processo AR(1) pode ser expresso como:

$yt = \phi \times yt\text{-}1 + et$

Após um ajuste do modelo, queremos estudar quão variável poderia ser nossa estimativa do coeficiente ϕ . Nesse caso, uma forma de estudar isto é executar uma série de simulações de Monte Carlo. Isso pode ser feito facilmente em R, desde que nos lembremos do que

aprendi sobre processos de AR no Capítulo 6:

##R

```
> exigir (previsão)

>

> phi <- 0,7

> passos_tempo <- 24

> N <- 1000

> sigma_error <- 1

>
```

```
> sd_series <- sigma_error^2 / (1 - phi^2)

> inicia <- rnorm(N, sd = sqrt(sd_series))

> estimativas <- numérico(N)

> res <- numérico(time_steps)

>

> para (i em 1:N) {

> err = rnorm(time_steps, sd = sigma_error)

> res[1] <- inicia[i] + erra[1]

>

> for (t em 2:time_steps) {

> res[t] <- phi * tail(res, 1) + err[t]

> }

> estimativas <- c(estimativas, arima(res, c(1, 0,
0))$coef[1])

> }

>

>hist(estimativas,

> main = "Phi estimado para AR(1) quando ts é AR(1)",

> quebras = 50)
```

Isto leva ao histograma mostrado na Figura 11-2 para o ϕ *estimado* .

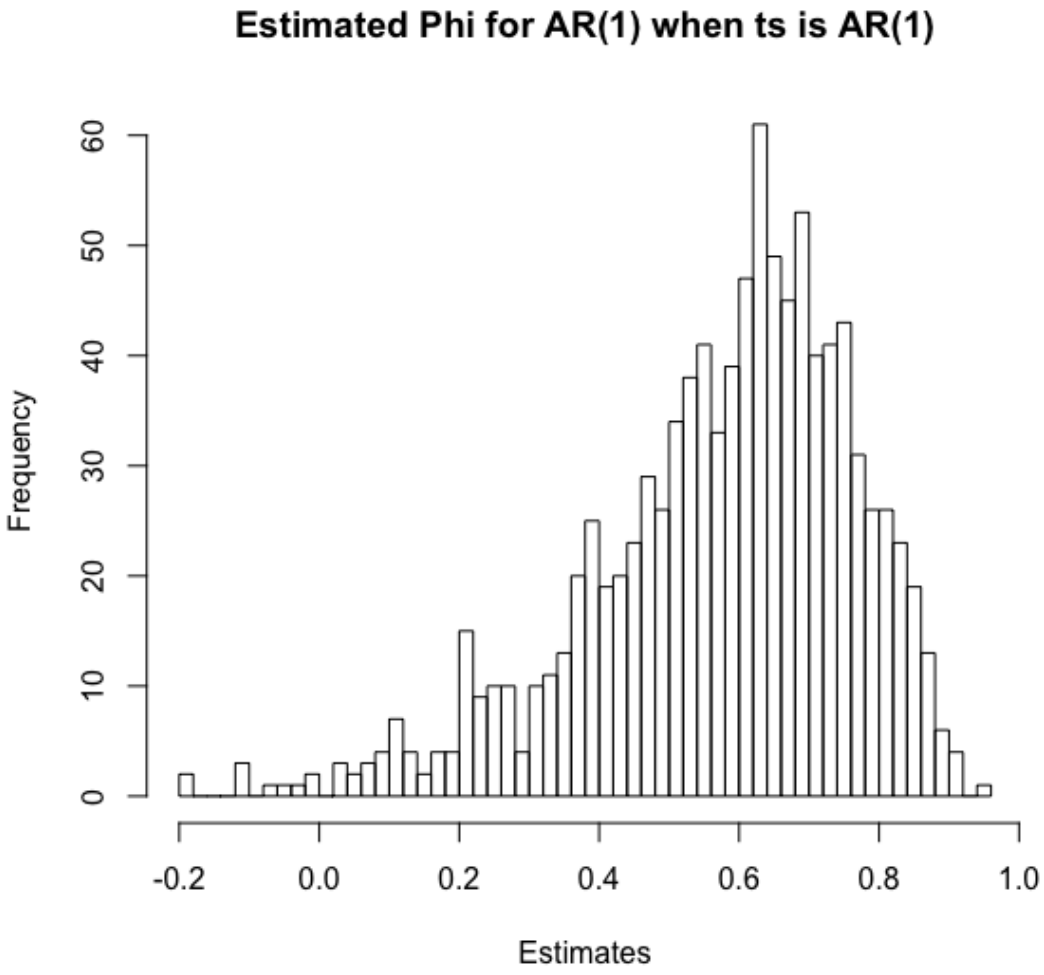


Figura 11-2. A distribuição de estimativas para ϕ.

Também podemos ter uma noção do intervalo das estimativas e dos quantis com a função sum mary() aplicada às estimativas:

##R

> resumo(estimativas1)

Min. 1º Qu. Mediana Média 3º Qu. Máx.

-0,3436 0,4909 0,6224 0,5919 0,7204 0,9331

Também podemos usar o bootstrapping para fazer perguntas mais complicadas. Suponha que queiramos saber quais custos numéricos podemos pagar por simplificar demais nosso modelo em comparação com a verdade básica.

Imagine que o processo estudado é um processo AR(2), embora o tenhamos diagnosticado como um processo AR(1). Para descobrir como isso afeta nossa estimativa, podemos modificar o código R anterior da seguinte forma:

##R

> ## agora vamos supor que temos um processo AR(2) verdadeiro

> ## porque isso é mais complicado, mudamos para arima.sim

> phi_1 <- 0,7

>phi_2 <- -0,2

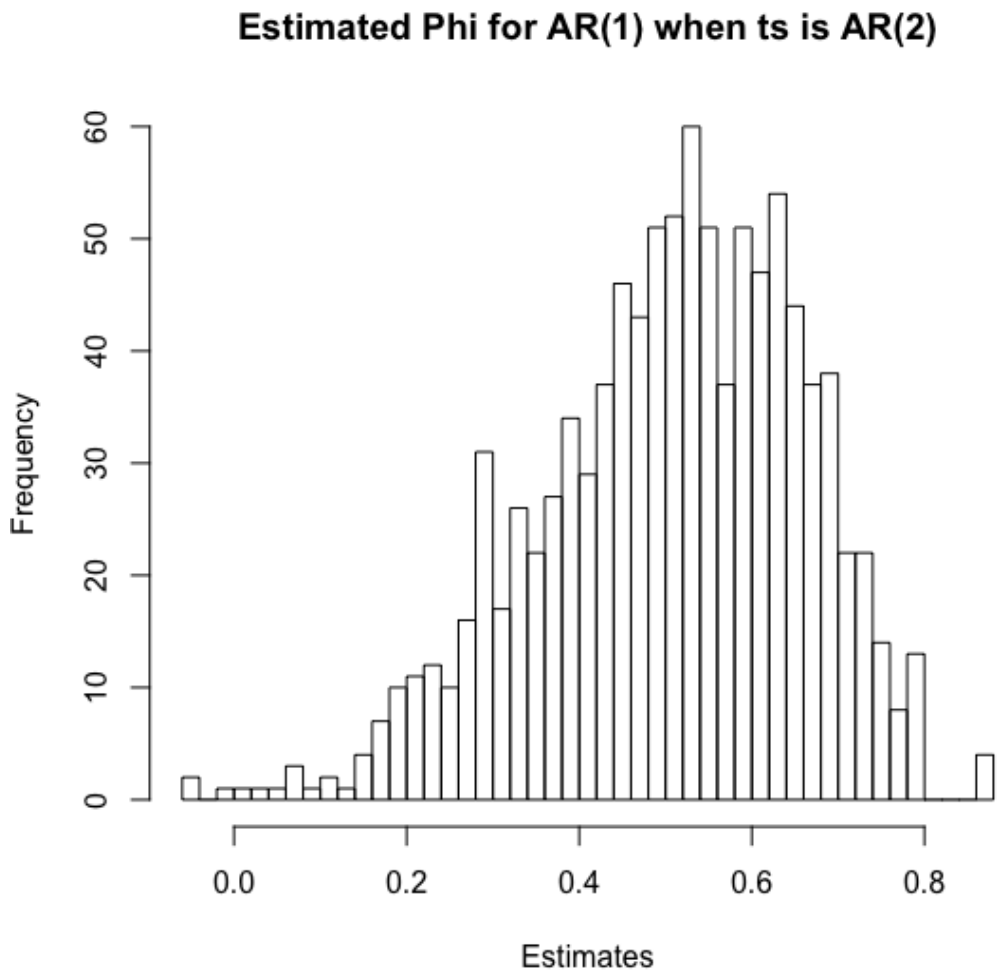
>

> estimativas <- numérico(N)

> para (i em 1:N) {

> res <- arima.sim(lista(ordem = c(2,0,0),

> ar = c(phi_1, phi_2)),



> n = passos_tempo)

> estimativas[i] <- arima(res, c(1, 0, 0))$coef[1]

> }

>

>hist(estimativas,

> main = "Phi estimado para AR(1) quando ts é AR(2)",

> quebras = 50)

Vemos a distribuição resultante na Figura 11-3 . Como podemos ver, a distribuição não é tão suave e bem definida para este modelo mal especificado como era para o modelo especificado adequadamente da Figura 11-2 .

Figura 11-3. A distribuição estimada do coeficiente de defasagem 1 para um modelo AR(1) ajustado a um processo para o qual a verdadeira descrição é um processo AR(2).

Você pode perceber que essas distribuições não são tão diferentes, e você está certo.

Confirmamos isso com as estatísticas resumidas:5

##R

> resumo (estimativas)

Min. 1º Qu. Mediana Média 3º Qu. Máx.

-0,5252 0,4766 0,6143 0,5846 0,7215 0,9468

5 Se o desajuste é um problema dependerá dos coeficientes específicos. Aqui o impacto da especificação incorreta do nosso modelo não é tão ruim, mas em outros casos pode ser catastrófico.

Podemos ver que o intervalo de estimativas é mais amplo quando o modelo é especificado incorretamente e que a estimativa para o termo de primeira ordem é ligeiramente pior do que era quando o modelo foi especificado corretamente, mas o desvio não é muito grande.

Isso pode resolver preocupações de que subestimar a ordem do modelo afetará nossa estimativa de ϕ . Podemos executar uma variedade de cenários de simulação para resolver possíveis problemas e compreender a gama de prováveis estimativas erradas, dadas algumas possibilidades imaginadas.

Prevendo vários passos à frente

Embora a previsão de um passo à frente seja o que abordamos na maioria dos capítulos anteriores, você também pode querer prever vários passos de tempo à frente. Isso acontece, entre outros motivos, quando os dados da série temporal que você possui têm uma resolução

temporal maior do que os valores da série temporal que você gostaria de prever. Por exemplo, você pode ter preços diários de ações disponíveis, mas gostaria de prever os preços mensais das ações para poder executar uma estratégia de longo prazo para suas economias para a aposentadoria. Como alternativa, você pode ter leituras de atividade elétrica cerebral feitas a cada minuto, mas gostaria de prever uma convulsão com pelo menos cinco minutos de antecedência para dar aos seus usuários/pacientes o máximo de aviso possível. Nesses casos, você tem inúmeras opções para gerar previsões com vários passos à frente.

Ajustar-se diretamente ao horizonte de interesse

Isso é tão simples quanto definir seu valor y (alvo) para refletir o horizonte de previsão de interesse. Portanto, se seus dados consistissem em indicadores minuto a minuto, mas você quisesse um horizonte futuro de cinco minutos para sua previsão, você cortaria as entradas do modelo no momento t e as treinaria para um rótulo gerado com dados até o momento $t + 5$ Então você ajustaria esses dados como o que você está tentando prever, seja por meio de uma regressão linear simples ou de um modelo de aprendizado de máquina ou até mesmo de uma rede de aprendizado profundo. Isso efetivamente funciona como:

$$modelo\,(\,X\,) = Y$$

onde você pode escolher Y para ter o horizonte de tempo que desejar. Portanto, cada um deles seria um cenário legítimo, dependendo do seu horizonte futuro de interesse (sejam 10 etapas ou 3 etapas, conforme mostrado aqui):

- $o\ modelo\ 1(\,Xt\,)$ é adequado para $Yt + 10$
- $o\ modelo\ 2(\,Xt\,)$ é adequado para $Yt + 3$

Abordagem recursiva para horizontes temporais distantes

Usando uma abordagem recursiva para ajustar uma variedade de horizontes, você constrói um modelo, mas se prepara para alimentá-lo com sua própria saída como uma entrada para prever horizontes mais distantes. Essa ideia deve parecer familiar porque demonstramos como fazer previsões multipasso à frente com modelagem ARIMA usando essa mesma estratégia. Suponha que desenvolvemos um modelo para se ajustar um passo à frente, treinando no $modelo\ (\ Xt\ ) = Yt + 1$. Se quiséssemos ajustar-nos a horizontes três passos à frente, faríamos o seguinte:

- $modelo\ (\ Xt\ ) \rightarrow estimativa Yt + 1$

- $modelo\ (\ Xt$ com estimativa de $Yt + 1) \rightarrow$ estimativa de $Yt + 2$

- $modelo\ (\ Xt$ com estimativa de $Yt + 1$ e estimativa de $Yt + 2) \rightarrow$ estimativa de $Yt + 3$

O erro esperado para a nossa estimativa de $Yt + 3$ seria necessariamente maior do que o da nossa estimativa para $Yt + 1$. Quanto maior? Isso pode ser complicado de pensar.

Uma ótima opção para ter uma ideia seria executar uma simulação, conforme discutido anteriormente neste capítulo.

Aprendizagem multitarefa aplicada a séries temporais

A aprendizagem multitarefa é um conceito geral de aprendizagem profunda que pode ser aplicado com um significado particular para análise de séries temporais. De forma mais geral, a aprendizagem multitarefa descreve a ideia de que um modelo pode ser construído para servir vários propósitos ao mesmo tempo ou para aprender a generalizar, tentando prever vários alvos diferentes, mas relacionados, ao mesmo tempo. Alguns pensam nisso como um meio de regularização, encorajando o modelo a ser mais geral, ensinando-lhe tarefas relacionadas.

No contexto de série temporal, você pode aplicar o aprendizado multitarefa configurando metas para diferentes horizontes temporais em um contexto de previsão. Nesse caso, o ajuste do seu modelo poderia ser assim:

- $modelo\,(\,Xt\,) = (\,Yt+1,\,Yt+10,\,Yt+100)$

- $modelo\,(\,Xt\,) = (\,Yt+1,\,Yt+2,\,Yt+3)$

Ao treinar esse modelo, você também poderia pensar em como ver a função de perda: você gostaria de ponderar todas as previsões igualmente ou gostaria de privilegiar determinados horizontes de previsão em detrimento de outros?

Se você estivesse procurando fazer uma previsão muito distante, poderia usar horizontes multitarefa para ensinar seu modelo, incluindo horizontes de curto prazo que poderiam apontar para características salientes, úteis para o horizonte de tempo mais longo, mas difíceis de descobrir diretamente a partir do sinal baixo. dados de ruído de previsões para um futuro distante. Outro cenário para modelagem multitarefa seria ajustar várias janelas de tempo no futuro, todas na mesma estação, mas talvez em pontos diferentes no tempo (por exemplo, primavera ao longo de vários anos ou segundas-feiras ao longo de vários **anos).**

diversas semanas). Essa seria uma maneira de ajustar a sazonalidade e a tendência ao mesmo tempo.

Dicas de validação de modelo

Estas são as questões mais importantes a serem consideradas ao pensar se você testou seu modelo adequadamente em comparação com a produção esperada

implementações:

Olhe para frente

A antecipação é enfatizada ao longo deste livro porque é muito difícil de evitar e é potencialmente catastrófica e embaraçosa quando um modelo é colocado em produção. Sempre que possível, você deseja evitar o perigo de lançar modelos que de repente param de ser validados na produção. Esta é a marca registrada de não estar ciente da antecipação em seu sistema. Não deixe isso acontecer com você!

Mudanças Estruturais

Identificar mudanças estruturais é em parte uma questão de julgamento, em parte uma questão de assunto e em parte uma questão de qualidade dos seus dados. A dinâmica subjacente a uma série temporal pode mudar ao longo do tempo e pode mudar o suficiente para que um modelo adequado a uma parte de uma série temporal possa não ser adequado a outra. Esta é uma das razões pelas quais a análise exploratória é importante – para garantir que você não está treinando o mesmo modelo em mudanças estruturais onde fazê-lo não é justificável nem sensato.

Mais recursos

Christoph Bergmeir, Rob J. Hyndman e Bonsoo Koo, "Uma nota sobre a validade de

Validação cruzada para avaliar a previsão de séries temporais autoregressivas", Computa-

Estatística Nacional e Análise de Dados 120 (2018): 70–83, https://perma.cc/YS3J-6DMD.

Os autores deste artigo (incluindo o estimável e altamente prolífico Rob Hyndman) discutem casos em que a validação cruzada produzirá bons resultados mesmo para análise de séries temporais. Este é um artigo útil principalmente porque oferece insights sobre por que os erros surgem da validação cruzada e também que tipos de modelos você pode usar com um detalhamento de validação cruzada padrão, em vez de

retrofit.

Angelo Canty, "tsboot: Bootstrapping de séries temporais", nd, https://perma.cc/MQ77-

U5HL.

Esta descrição da função tsboot() incluída no pacote de inicialização é uma leitura educacional, pois implementa diversas variações amplamente aceitas na amostragem de blocos como uma forma de inicialização. Explorar as opções

disponíveis nesta função é uma boa maneira de se familiarizar com as várias maneiras de inicializar séries temporais, e a familiaridade com esta função, por sua vez, permitirá que você faça

estimativas convenientes e confiáveis de estatísticas de teste e avalie a incerteza de sua estimativa, mesmo no caso de pequenos conjuntos de dados.

Hans R. Kunsch, "O Jackknife e o Bootstrap para Observação Estacionária Geral

ções", Annals of Statistics 17, no. 3 (1989): 1217–41, https://perma.cc/5B2T-XPBC.

Este artigo clássico e amplamente citado demonstra uma abordagem estatística para estender os métodos jackknife (menos comumente usados) e bootstrapping ao longo do tempo.

dados de séries, eliminando a suposição de tais métodos de que os pontos de dados são distribuídos de forma independente. Este artigo introduz a ideia de pensar e amostrar séries temporais com unidades de blocos de pontos em vez de pontos individuais.

Christian Kleiber, "Mudanças Estruturais em Séries Temporais (Econômicas)," em Complexidade e Sinergética, ed. Stefan Müller et al. (Cham, Suíça: Springer International, 2018), https://perma.cc/7U8N-T4RC.

Este capítulo resume uma variedade de métodos canônicos para identificar mudanças estruturais em séries temporais e foi escrito a partir de uma perspectiva de séries temporais empresariais e econômicas, embora as lições se apliquem de forma mais ampla. É particularmente útil porque fornece uma lista dos pacotes R que podem ser usados para aplicar esses métodos de identificação de mudanças estruturais amplamente aceitos e porque oferece visualizações úteis de como os métodos se comportam de maneira um pouco diferente ao analisar a mesma série temporal.

Robert Stambaugh, "Regressões Preditivas", Jornal de Economia Financeira 54 (1999): 375–421, https://perma.cc/YD7U-RXBM.

Este artigo amplamente citado demonstra como realizar uma estimativa posterior bayesiana de coeficientes de regressão e como as estimativas de quantidades de amostras finitas podem diferir substancialmente dadas as diferentes suposições sobre a dinâmica subjacente dos modelos de séries temporais financeiras. Este é um excelente exemplo, embora específico do domínio, de quanto as suposições iniciais podem afetar as estimativas de

incerteza em suas estimativas, parâmetros.

CAPÍTULO 12

Considerações de desempenho na montagem e

Servindo modelos de série temporal

Na literatura sobre aprendizado de máquina e análises estatísticas, o foco principal tende a ser o desempenho dos modelos em termos de precisão. Embora a precisão geralmente deva ser a principal preocupação ao avaliar um modelo, às vezes as considerações de desempenho computacional são extremamente importantes diante de grandes conjuntos de dados ou modelos amplamente implantados para atender a grandes populações de aplicativos clientes.

Os conjuntos de dados de séries temporais ficam tão grandes que as análises não podem ser feitas — ou não podem ser feitas adequadamente — porque são muito intensivas em suas demandas pelos recursos computacionais disponíveis. Nesses casos, muitas organizações tratam as suas opções da seguinte forma:

• Aumentar os recursos computacionais (caros e muitas vezes um desperdício, tanto económica como ambientalmente).

• Fazer o projeto mal (sem ajuste suficiente de hiperparâmetros, dados insuficientes, etc.).

• Não faça o projeto.1

Nenhuma dessas opções é satisfatória, principalmente quando você está apenas começando com um novo conjunto de dados ou uma nova técnica analítica. Pode ser frustrante não saber se suas falhas são resultado de dados inadequados, de um problema excessivamente difícil ou de falta de recursos. Esperamos encontrar algumas soluções alternativas para expandir suas opções no caso de análises muito exigentes ou conjuntos de dados muito grandes.

Este capítulo foi elaborado para orientá-lo em algumas considerações sobre como diminuir os recursos de computação necessários para treinar ou inferir usando um modelo específico. Para o 1 Sim, isso acontece o tempo todo no mundo real.

na maioria das vezes, essas perguntas são específicas para um determinado conjunto de dados, os recursos que você tem disponíveis e suas metas de precisão e velocidade. Você verá essa realidade refletida nas preocupações detalhadas neste capítulo, mas a esperança é que elas cubram parcialmente os problemas que você enfrenta e possam fornecer inspiração para futuros brainstormings. Estas são considerações que devem ser feitas quando você tiver concluído suas primeiras rodadas de análise e modelagem e não devem ser uma prioridade quando você anexar um problema pela primeira vez. No entanto, quando chegar a hora de colocar algo em produção ou estender um pequeno projeto de pesquisa para um projeto maior, você deve rever essas preocupações com frequência.

Trabalhando com ferramentas desenvolvidas para casos de uso mais gerais

Um desafio dos dados de séries temporais é que a maioria das ferramentas, especialmente aquelas para aprendizado de máquina, são construídas para um caso de uso mais geral, com a maioria dos exemplos ilustrativos mostrando

usos de dados transversais. Existem algumas maneiras pelas quais esses métodos de aprendizado de máquina não conseguem ser tão eficientes quanto possível com dados de séries temporais. As soluções para seus problemas individuais variam, mas as ideias gerais são as mesmas. Nesta seção discuto problemas comuns e possíveis soluções.

Modelos construídos para dados transversais não “compartilham” dados

Entre amostras

Em muitos casos em que você alimenta amostras discretas de dados de séries temporais em um algoritmo, geralmente com modelos de aprendizado de máquina, você notará que grandes partes dos dados que você está alimentando entre as amostras se sobrepõem. Por exemplo, suponha que você tenha os seguintes dados sobre vendas mensais:

Mês

Widgets vendidos

Janeiro de 2014

11.221

Fevereiro de 2014

9.880

Março 2014

14.423

Abril de 2014

16.720

Maio de 2014

17.347

Junho de 2014

22.020

Julho de 2014

21.340

Agosto de 2014

25.973

Setembro de 2014

11.210

Outubro de 2014

11.583

Novembro de 2014

12.014

Dezembro de 2014

11.400

Janeiro de 2015

11.539

Fevereiro de 2015

10.240

Você está tentando fazer previsões mapeando cada “forma” para uma curva vizinha mais próxima. Você prepara muitas formas a partir desses dados. Apenas alguns desses pontos

de dados estão listados aqui, já que você pode querer usar curvas de seis meses como as “formas” de interesse (observe que não estamos fazendo nenhum pré-processamento de dados para normalizar ou criar acréscimos).

recursos adicionais de interesse, como média móvel ou curvas suavizadas):2

11221, 9880, 14423, 16720, 17347, 22020

9880, 14423, 16720, 17347, 22020, 21340

14423, 16720, 17347, 22020, 21340, 25973

Curiosamente, tudo o que conseguimos fazer com esta preparação de inputs foi aumentar o nosso conjunto de dados seis vezes, sem incluir qualquer informação adicional. Isso é uma catástrofe do ponto de vista do desempenho, mas muitas vezes é necessário para entradas em diversos módulos de aprendizado de máquina.

Se este for um problema que você enfrenta, existem algumas soluções que você deve considerar.

Não use dados sobrepostos

Considere produzir apenas um “ponto de dados” para que cada mês individual ocupe apenas uma curva. Se você fizer isso, os dados anteriores poderão ser semelhantes à tabela a seguir:

11221, 9880, 14423, 16720, 17347, 22020

21340, 25973, 11210, 11583, 12014, 11400

Observe que isso seria particularmente simples porque equivale a uma simples remodelagem de array, em vez de uma repetição personalizada de dados.

Empregue um paradigma semelhante ao gerador para iterar pelo conjunto de dados

Empregar um paradigma semelhante ao gerador para iterar pelo conjunto de dados, reamostrando a partir da mesma estrutura de dados conforme apropriado, é particularmente fácil de codificar em Python, mas também pode ser feito com R e outras linguagens. Se imaginarmos que os dados originais são armazenados em um array NumPy 1D, isso poderia ser parecido com o código a seguir (observe que isso teria que ser emparelhado com uma estrutura de dados de aprendizado de máquina ou algoritmo que aceita geradores):

Pitão

>>> **def** array_to_ts(arr):

>>> idx = 0

2 Extraímos muitas amostras de séries temporais de uma grande série temporal no caso dos modelos de aprendizado de máquina e de aprendizado profundo discutidos nos capítulos anteriores.

Trabalhando com ferramentas desenvolvidas para casos de uso mais gerais | 359

>>> **enquanto** idx + 6 <= arr.shape[0]:

>>> **rendimento** arr[idx:(idx+6)]

Observe que projetar código de modelagem de dados que não exploda desnecessariamente um conjunto de dados é desejável tanto do ponto de vista de treinamento quanto de produção. No treinamento isso permitirá que você coloque mais exemplos de treinamento na memória, e na produção você poderá fazer múltiplas previsões com menos recursos de treinamento no caso de previsões (ou classificações) em dados sobrepostos. Se você fizer previsões frequentes para o mesmo caso de uso, provavelmente trabalhará com dados sobrepostos, portanto, esse problema e suas soluções serão bastante relevantes.

Modelos que não pré-calculam criam atrasos desnecessários entre eles

Medindo dados e fazendo uma previsão

Normalmente, os modelos de aprendizado de máquina não se preparam nem levam em consideração a possibilidade de pré-computar parte de um resultado antes de obter todos os dados. No entanto, para séries temporais este é um cenário muito comum.

Se você estiver servindo seu modelo em um aplicativo urgente, como previsões médicas, estimativas de localização de veículos ou previsão de preços de ações, poderá descobrir que o atraso no cálculo de uma previsão somente depois que todos os dados estiverem disponíveis é muito grande.

Nesse caso, você deve considerar se o modelo escolhido pode realmente ser parcialmente pré-computado antecipadamente. Alguns exemplos de onde isso é possível são os seguintes:

• Se você estiver usando uma rede neural recorrente que usa vários canais de informação em 100 intervalos de tempo diferentes, você pode pré-calcular/desenrolar a rede neural para os primeiros 99 intervalos de tempo. Então, quando o último ponto de dados finalmente chegar, você só precisará fazer 1 conjunto final de multiplicações de matrizes (e outros cálculos de função de ativação) em vez de 100. Em teoria, isso poderia acelerar seu

tempo de resposta 100 vezes maior.

• Se você estiver usando um modelo AR(5), poderá pré-calcular todos os termos, exceto o mais recente, na soma que constitui a moda. Como lembrete, um processo AR(5)

se parece com a equação a seguir. Se você está prestes a gerar uma previsão, significa que você já conhece os valores de yt – 4, yt –3, yt –2 e yt –1, o que significa que você pode ter tudo, exceto $ph\ i\ 0\ Âyt$ pronto para vá antes de *saber* :

$$yt\ +1 = ph\ i\ 4 \times yt\ -4 + ph\ i\ 3 \times yt\ -3 + ph\ i\ 2 \times yt\ -2 + ph\ i\ 1 \times yt\ -1 + ph\ i\ 0 \times yt$$

• Se você estiver usando um modelo de agrupamento para encontrar vizinhos mais próximos resumindo características de uma série temporal (média, desvio padrão, máximo, mínimo, etc.), você pode calcular essas características com uma série temporal com um ponto de dados a menos e executar seu

modelo com essa série temporal para identificar vários vizinhos mais próximos. Você pode então atualizar esses recursos assim que o valor final chegar e executar novamente a análise apenas com os vizinhos mais próximos encontrados na primeira rodada de análise. Na verdade, isso exigirá mais recursos de computação em geral, mas resultará em um intervalo de tempo menor entre a medição final e a entrega da previsão.

Em muitos casos, seu modelo pode não ser tão lento quanto o atraso da rede ou outros fatores, portanto, a pré-computação é uma técnica válida apenas onde o tempo de feedback é extremamente importante e onde você está confiante de que a computação do modelo está contribuindo substancialmente para o tempo entre o recebimento de um aplicativo. todas as informações necessárias e gerando uma previsão útil.

Formatos de armazenamento de dados: vantagens e desvantagens

Uma área negligenciada de gargalos de desempenho para modelos de séries temporais de treinamento e produção é o método de armazenamento de dados. Alguns erros comuns são:

• *Armazenar dados em um formato de dados baseado em linhas, mesmo que a série temporal seja formada por percorrendo uma coluna* . Isso resulta em dados onde os pontos adjacentes ao tempo não são adjacentes à memória.

• *Armazenar dados brutos e executar análises desses dados* . Dados pré-processados e com amostragem reduzida são preferíveis na maior medida possível para um determinado modelo.

A seguir, discutiremos esses fatores de armazenamento de dados para manter o treinamento e a inferência do modelo o mais rápido possível.

Armazene seus dados em formato binário

É tentador armazenar dados em um arquivo de texto separado por vírgula, como um arquivo CSV. Muitas vezes é assim que os dados são fornecidos, por isso a inércia leva-nos a fazer esta escolha. Esses formatos de arquivo também são legíveis por humanos, o que facilita a verificação dos dados no arquivo em relação às saídas do pipeline. Finalmente, esses dados geralmente são fáceis de enviar para diferentes plataformas. 3

No entanto, não é fácil para o seu computador ler arquivos de texto . Se você estiver trabalhando com conjuntos de dados tão grandes que não consiga colocar todos os seus dados na memória durante o treinamento, você estará lidando com E/S e processamento relacionado, todos vinculados ao formato de arquivo escolhido. Ao armazenar dados em formato binário, você pode reduzir substancialmente a lentidão relacionada à E/S de diversas maneiras:

3 Embora existam problemas de Unicode relacionados a diferentes plataformas e dispositivos, você não está claro só porque usa um formato de arquivo baseado em texto.

Formatos de armazenamento de dados: vantagens e desvantagens | 361

• Como os dados estão em formato binário, seu pacote de processamento de dados já

“entende” isso. Não há necessidade de ler um CSV e transformá-lo em um quadro de dados.

Você já terá um quadro de dados ao inserir seus dados.

• Como os dados estão em formato binário, eles podem ser muito mais compactados do que um CSV

ou outro arquivo baseado em texto. Isso significa que a própria E/S será mais curta porque há menos memória física para ler em um arquivo e recriar seu conteúdo.

Os formatos de armazenamento binário são facilmente acessíveis em R e Python. Em R, use save() e load() para data.table. Em Python, use decapagem e observe que tanto Pandas (pd.DataFrame.load() e pd.DataFrame.save()) quanto NumPy (np.load() e

np.save()) inclui wrappers em torno da decapagem que você pode usar para seus objetos específicos.

Pré-processe seus dados de uma forma que permita "deslizar" sobre eles

Esta recomendação está relacionada a "Modelos construídos para dados transversais não

"Compartilhar" dados entre amostras" na página 358. Nesse caso, você também deve pensar em

como você pré-processa seus dados e certifica-se de que a maneira como você faz isso é consistente com o uso de uma janela móvel sobre esses dados para gerar várias amostras de teste.

Por exemplo, considere a normalização ou médias móveis como etapas de pré-processamento. Se você planeja fazer isso para cada janela de tempo, isso pode levar a uma maior precisão do modelo (embora, na minha experiência, esses ganhos sejam frequentemente marginais). No entanto, existem várias desvantagens:

• Você precisa de mais recursos computacionais para calcular esses recursos de pré-processamento

muitas e muitas vezes em dados sobrepostos - apenas para acabar com resultados muito semelhantes

números.

• Você precisa armazenar dados sobrepostos com pré-processamento ligeiramente diferente muitas e muitas

vezes.

• Você não pode aproveitar ao máximo deslizando uma janela sobre seus dados.

Modificando sua análise para se adequar ao desempenho

Considerações

Muitos de nós somos culpados de nos sentirmos demasiado confortáveis com um determinado conjunto de ferramentas analíticas e com o conjunto de software e regras práticas que o acompanham sobre como ajustar um modelo. Também tendemos a avaliar as necessidades de precisão uma vez e não reavaliá-las quando determinamos o custo computacional de vários desempenhos possíveis do modelo.

362 | Capítulo 12: Considerações de desempenho no ajuste e veiculação de modelos de série temporal

Os dados de séries temporais, frequentemente usados para fazer previsões rápidas, são particularmente propensos a necessitar de modelos que possam ser ajustados e produzidos rapidamente. Os modelos precisam ser ajustados rapidamente para que possam ser atualizados com a chegada de novos dados, e precisam ter um desempenho rápido para que os consumidores das previsões dos modelos tenham o máximo de tempo possível para agir de acordo com essas previsões. Por esse motivo, às vezes você pode querer modificar suas expectativas – e a análise que as acompanha – para tornar processos de análise e previsão mais rápidos e computacionalmente simplificados.

Usar todos os seus dados não é necessariamente melhor

Um fator importante ao pensar em como agilizar sua análise é entender que nem todos os dados de uma série temporal são igualmente importantes. Dados mais distantes são menos importantes. Os dados durante períodos “excepcionais” são menos importantes para a construção de um modelo para períodos normais.

Há muitas maneiras de considerar diminuir a quantidade de dados usados para treinar um modelo. Embora muitas dessas opções tenham sido discutidas anteriormente neste livro, é bom revisá-las, principalmente no que diz respeito ao desempenho:

Redução da resolução

Muitas vezes você pode usar dados menos frequentes para cobrir a mesma janela de lookback ao fazer uma previsão. Esta é uma forma de reduzir o tamanho dos seus dados por um fator multiplicativo. Observe que, dependendo da técnica analítica usada, você também terá opções mais criativas, como reduzir a resolução em taxas diferentes, dependendo de quão longe os dados vão.

Treinamento apenas com dados recentes

Embora o aprendizado de máquina adore dados, existem muitos modelos de séries temporais onde

técnicas estatísticas ou mesmo de aprendizado profundo funcionarão melhor apenas focando em dados recentes, em vez de treinar em todos os dados. Isso o ajudará a reduzir os dados de entrada, subtraindo dados que são apenas marginalmente informativos para o seu modelo.

Reduza a janela de lookback usada para fazer sua previsão

Em muitos modelos de série temporal, o desempenho do modelo continuará a melhorar, mesmo que ligeiramente, à medida que você olha cada vez mais para o passado. Você deve tomar algumas decisões sobre quanta precisão é realmente necessária em relação ao desempenho. Pode ser que você esteja carregando muito mais dados na memória por amostra do que o realmente necessário para um desempenho aceitável.

Modelos complicados nem sempre funcionam melhor o suficiente

Pode ser interessante e divertido experimentar o que há de melhor e mais recente quando se trata de escolher um modelo analítico. No entanto, a verdadeira questão é se um

modelo mais sofisticado “paga” por quaisquer recursos computacionais adicionais necessários.

Modificando sua análise para atender às considerações de desempenho | 363

Quase todos os avanços computacionais em aprendizado de máquina dos últimos anos resultaram do uso de mais poder computacional em um problema. Com problemas como

reconhecimento de imagem, onde há definitivamente uma resposta certa e 100% de precisão, isso certamente pode fazer sentido.

Por outro lado, com problemas como previsões de séries temporais, onde pode haver limites físicos ou matemáticos para a precisão de uma previsão, você deve certificar-se de que a escolha de um modelo mais complexo não seja simplesmente uma atualização automática sem uma análise de custo-benefício. . Pense se os ganhos de precisão justificam o atraso adicional que o modelo pode produzir no cálculo de uma previsão, ou o tempo de treinamento adicional que será necessário, ou os recursos computacionais adicionais que serão gerados. Pode ser que um método que exija menos recursos e com uma precisão ligeiramente inferior seja um “negócio” muito melhor do que um modelo sofisticado que mal melhora a versão simples.

Se você é o analista de dados, essa compensação entre complexidade/precisão e tempo de atraso/

recursos do computador é algo que você deve analisar, pensando nisso como mais um hiperparâmetro a ser ajustado. É sua função apontar essas compensações em vez de presumir que um engenheiro de dados cuidará disso. As pessoas a montante ou a jusante no pipeline de dados não podem substituir o seu julgamento na seleção do modelo, portanto, você deve reconhecer o lado da engenharia da ciência de dados enquanto avalia os prós e os contras.

Uma breve menção a ferramentas alternativas de alto desempenho

Se você explorou completamente as opções anteriores, também pode considerar alterar sua base de código subjacente, mais especificamente, afastando-se de linguagens de script mais lentas, como Python e R. Há várias maneiras de fazer isso:

• Aposte tudo em C++ e Java. Mesmo que você não tenha olhado para essas linguagens, apenas aprender o básico às vezes pode acelerar as partes lentas do seu pipeline

o suficiente para transformar tarefas impossíveis em administráveis. C++, em particular, evoluiu tremendamente em termos de usabilidade e bibliotecas padrão aplicáveis ao processamento de dados. A sintaxe STL e C++ 17 agora oferece muitas opções bastante

comparável ao Python para operar em conjuntos de dados em uma variedade de estruturas de dados.

Mesmo que você odiasse C++ e Java anos atrás, você deveria revisitá-los.4

• Em Python, você pode usar vários módulos diferentes onde você pode escrever código Python que é compilado em código C ou C++, acelerando o tempo de execução. Isso pode ser especialmente útil para códigos muito repetitivos com muitos loops for, que são lentos em Python e podem ter muito mais desempenho em C ou C++ sem o

necessidade de um design inteligente - simplesmente implementar o mesmo código de maneira mais rápida

4 Há uma curva de aprendizado acentuada, mas depois de descobrir a infraestrutura básica de compilação, isso pode ser um grande bônus para sua organização.

a linguagem pode resolver o problema. Numba e Cython são módulos Python acessíveis que podem ajudá-lo a acelerar trechos lentos de código Python dessa maneira.

• Da mesma forma, em R, você pode usar Rcpp para funcionalidades semelhantes.

Mais recursos

• Em modelos com desempenho igual:

Anthony Bagnall et al., "The Great Time Series Classification Bake Off: An Experi-

Avaliação mental de algoritmos recentemente propostos", *Mineração de dados e conhecimento Descoberta 31, não. 3 (2017): 606–60, https://perma.cc/T76B-M635.*

Este artigo executa uma ampla gama de experimentos para avaliar o desempenho de metodologias modernas de classificação de séries temporais, comparando seu desempenho em uma ampla variedade de conjuntos de dados disponíveis publicamente. O computacional

a complexidade dos conjuntos de dados acaba variando um pouco mais do que o desempenho real dos métodos testados. Como destacam os autores, continua a ser uma arte e uma área de investigação determinar, sem tentar todos, qual o método que funcionará melhor num determinado conjunto de dados. Do ponto de vista dos recursos computacionais, a lição a aprender aqui é que a complexidade computacional

deve levar em consideração significativamente nas decisões metodológicas. A menos que você tenha um caso de uso muito atraente para um algoritmo complicado e que consome muitos recursos, escolha algo mais simples.

• Na construção de modelos mais simples:

Yoon Kim e Alexander M. Rush, "Destilação de conhecimento em nível de sequência", em

Anais da Conferência de 2016 sobre Métodos Empíricos em Linguagem Natural Processamento, ed. Jian Su, Kevin Duh e Xavier Carreras (Austin, TX: Associação para Lingüística Computacional, 2016), 1317–27, https://perma.cc/V4U6-EJNU.

Este artigo aplica o conceito geral de “destilação” ao aprendizado sequencial, aplicado a uma tarefa de tradução automática. O conceito de destilação é um

amplamente útil. A ideia é que primeiro um modelo complexo seja projetado e

treinado nos dados originais e, em seguida, um modelo mais simples é treinado nas saídas do modelo complexo. Os resultados do modelo complexo, em vez de

os dados em si, reduz o ruído e simplifica o problema de aprendizagem, tornando mais fácil para um modelo mais simples aprender aproximadamente o mesmo relacionamento por

cortando o barulho. Embora tal técnica não diminua o tempo de treinamento, ela deverá produzir um modelo que execute mais rapidamente e exija menos recursos

quando colocado em produção.

Mais recursos | 365

CAPÍTULO 13

Aplicações de saúde

Neste capítulo, examinaremos a análise de séries temporais no contexto da saúde com dois estudos de caso: previsão e previsão da gripe e previsão da glicemia. > Ambas são aplicações de saúde importantes para problemas de saúde comuns. Além disso, em ambos os casos, estes não são problemas resolvidos, mas sim tópicos de investigação em curso na academia e na indústria da saúde.

Prevendo a gripe

Prever a taxa de gripe semanalmente em uma determinada área geográfica é um problema antigo e contínuo. Especialistas em doenças infecciosas e profissionais de segurança global

concordam que as doenças infecciosas representam um risco significativo para o bem-estar humano. Isso é

particularmente o caso da gripe, que atinge os vulneráveis em todo o mundo, causando centenas de mortes todos os anos, principalmente entre os muito jovens e os muito idosos. É crucial, tanto do ponto de vista da saúde como da segurança nacional, desenvolver modelos precisos de como a gripe irá evoluir numa determinada estação. Os modelos de previsão da gripe são úteis tanto para prever especificamente o vírus como também para ajudar os investigadores a explorar teorias gerais sobre como as doenças infecciosas se propagam geograficamente.

Um estudo de caso de gripe em uma área metropolitana

Veremos um conjunto de dados de relatórios semanais de gripe para diversas regiões administrativas da França durante os anos de 2004 a 2013. Preveremos a taxa de gripe para Île de

França, região metropolitana de Paris. Os dados podem ser baixados do Kaggle 1 e também estão disponíveis no repositório de código deste livro.

1 Embora este não seja um conjunto de dados público, é possível acessar os dados registrando-se no concurso.

Exploração de dados e alguma limpeza

Começamos nos familiarizando com os dados brutos, examinando-os primeiro em forma tabular:

##R

```
> gripe = fread("train.csv")

> gripe[, flu.rate := as.numeric(TauxGrippe)]

> cabeça(gripe)
```

Id semana região_código região_nome TauxGrippe gripe.rate

1: 3235 201352 42 ALSÁCIA 7 7

2: 3236 201352 72 AQUITÂNIA 0 0

3: 3237 201352 83 AUVERGNE 88 88

4: 3238 201352 25 BAIXA-NORMANDIA 15 15

5: 3239 201352 26 BORGONHA 0 0

6: 3240 201352 53 BRETAGNE 67 67

Também fazemos algumas verificações básicas de qualidade, como procurar NA em nossas variáveis de interesse. Podemos não saber de onde vêm esses valores de NA, mas precisamos considerá-los:

##R

> nrow(gripe[is.na(flu.rate)]) / nrow(gripe)

[1] 0,01393243

> único(gripe[is.na(flu.rate)]$region_name)

[1] "HAUTE-NORMANDIE" "NORD-PAS-DE-CALAIS" "PICARDIE"

[4] "LIMUSIN" "FRANCHE-COMTE" "CENTRO"

[7] "AUVERGNE" "BASSE-NORMANDIE" "BOURGOGNE"

[10] "CHAMPAGNE-ARDENNE" "LANGUEDOC-ROUSSILLON" "PAYS-DE-LA-LOIRE"

[13] "CORSE"

A taxa global de pontos de dados de NA não é muito elevada. Além disso, a nossa região de interesse, Île-de-

France, não está incluída na lista de regiões com valores NA.

Realizamos uma limpeza de dados, separando a parte da semana e do ano da coluna de carimbo de data/hora (que atualmente está em formato de caractere, não em formato numérico ou de carimbo de data/hora):

##R

> gripe[, ano := as.numeric(substr(semana, 1, 4))]

> gripe[, sem := as.numeric(substr(semana, 5, 6))]

> *## nota de estilo, não é ótimo, temos colunas de 2 semanas*

Adicionamos uma coluna de classe Date para que possamos ter eixos de plotagem melhores para o tempo do que se tratássemos os dados como sem registro de data e hora:

##R

> gripe[, data:= as.Date(paste0(as.character(flu$week), "1"), "%Y%U%u")]

Esta linha de código é um pouco complicada. Para converter as combinações mês-semana em datas, adicionamos um componente que indica o dia. Este é o propósito de paste0(), que marca cada data como o primeiro dia da semana, colando um “1” em uma string que **368 | Capítulo 13: Aplicações de Saúde**

já designa o ano e a semana do ano (das 52 semanas do ano – falaremos mais sobre isso em breve). 2 Observe o %U e %u na string de formato: eles têm a ver com a marcação do horário de acordo com a semana do ano e o dia da semana, um formato de carimbo de data/hora um tanto incomum. 3

Em seguida, subdividimos os dados relacionados especificamente a Paris e classificamos os dados por data:4

##R

vamos nos concentrar em Paris

> paris.flu = gripe[region_name == "ILE-DE-FRANCE"]

> paris.flu = paris.flu[ordem(data, decrescente = **FALSE**)]

> paris.flu[, .(semana, data, gripe.taxa)]

data da semana gripe.rate

1: 200401 2004-01-05 66

2: 200402 2004-01-12 74

3: 200403 2004-01-19 88

4: 200404 2004-01-26 26

5: 200405 2004-02-02 17

518: 201350 16/12/2013 13

519: 201351 23/12/2013 49

520: 201352 30/12/2013 24

521: 200953 < **NA** > 145

522: 200453 < **NA** > 56

Se você estiver prestando atenção, a contagem de linhas deverá surpreendê-lo. Se um ano tem 52 semanas e temos dados de 10 anos, por que temos 522 linhas? Teríamos esperado 52 semanas × 10 anos = 520 linhas. Além disso, por que existem duas datas de NA?

Você verá uma explicação se voltar aos dados originais. Parece haver uma 53ª semana tanto para 2004 quanto para 2009. A cada poucos anos há um ano que tem 53

semanas em vez de 52 – não é um erro, mas sim uma parte do calendário gregoriano

sistema .

Em seguida, verificamos se os dados cobrem um intervalo de datas completo e regularmente amostrado, certificando-nos primeiro de que cada ano tenha o mesmo número de pontos de dados: 5

2 O dia correto da semana (com as opções de 1 a 7) para começar depende da data em que a taxa de gripe foi calculada, mas não sabemos isso pelos dados fornecidos. Escolhemos o primeiro dia pela simplicidade.

O último dia seria igualmente sensato nesta situação e não é importante para a análise, desde que sejamos consistentes.

3 A string de formatação apropriada para semana do ano e dia da semana pode depender do seu sistema operacional. Minha solução funcionou em um macOS, enquanto usei uma formatação ligeiramente diferente no Linux.

4 Se estivéssemos trabalhando em um grande conjunto de dados, deveríamos fazer isso como um primeiro passo para evitar operações computacionalmente onerosas, como a conversão de strings em objetos Date, para dados irrelevantes.

5 Esta não é a única verificação necessária para garantir que os dados sejam regularmente amostrados e completos. Deixo o resto para você.

Prevendo a gripe | 369

##R

> paris.flu[, .N, ano]

ansiar

1: 2004 53

2: 2005 52

...

9: 2012 52

10: 2013 52

> paris.flu[, .N, sem]

semana N

1: 1 10

2: 2 10

3: 3 10

...

51: 51 10

52: 52 10

53: 53 2

semana N

Podemos ver que os dados estão conforme o esperado; isto é, cada ano (além dos dois que acabamos de discutir) tem 52 semanas e cada rótulo de semana do ano tem 10 pontos de dados, um para cada ano (exceto a semana 53).

Agora que consideramos os carimbos de data/hora dos dados, inspecionamos os valores reais da série temporal (até agora consideramos apenas a indexação de tempo). Existe uma tendência?

Sazonalidade? Vamos descobrir (veja a Figura 13-1):

##R

> paris.flu[, plot(data, gripe.taxa,

```
> tipo = "l", xlab = "Data",

> ylab = "Taxa de gripe")]
```

Fica claro, a partir de um gráfico simples, que existe uma sazonalidade substancial (algo que você provavelmente já experimentou em sua própria comunidade). Este gráfico sugere uma forte componente sazonal, mas não sugere que haja um desvio temporal independente da sazonalidade.

O comportamento sazonal complica a 53ª semana. Se quisermos ajustar um modelo sazonal, precisamos de definir a sazonalidade em termos de semanas do ano, e não podemos ter tamanhos sazonais variáveis (é isso que distingue uma estação de um ciclo, como discutido no Capítulo 3). Embora possamos imaginar algumas soluções criativas para o problema da 53ª semana, tomaremos a simples opção de suprimir estes dados:

```
##R

> paris.flu <- paris.flu[semana!= 53]
```

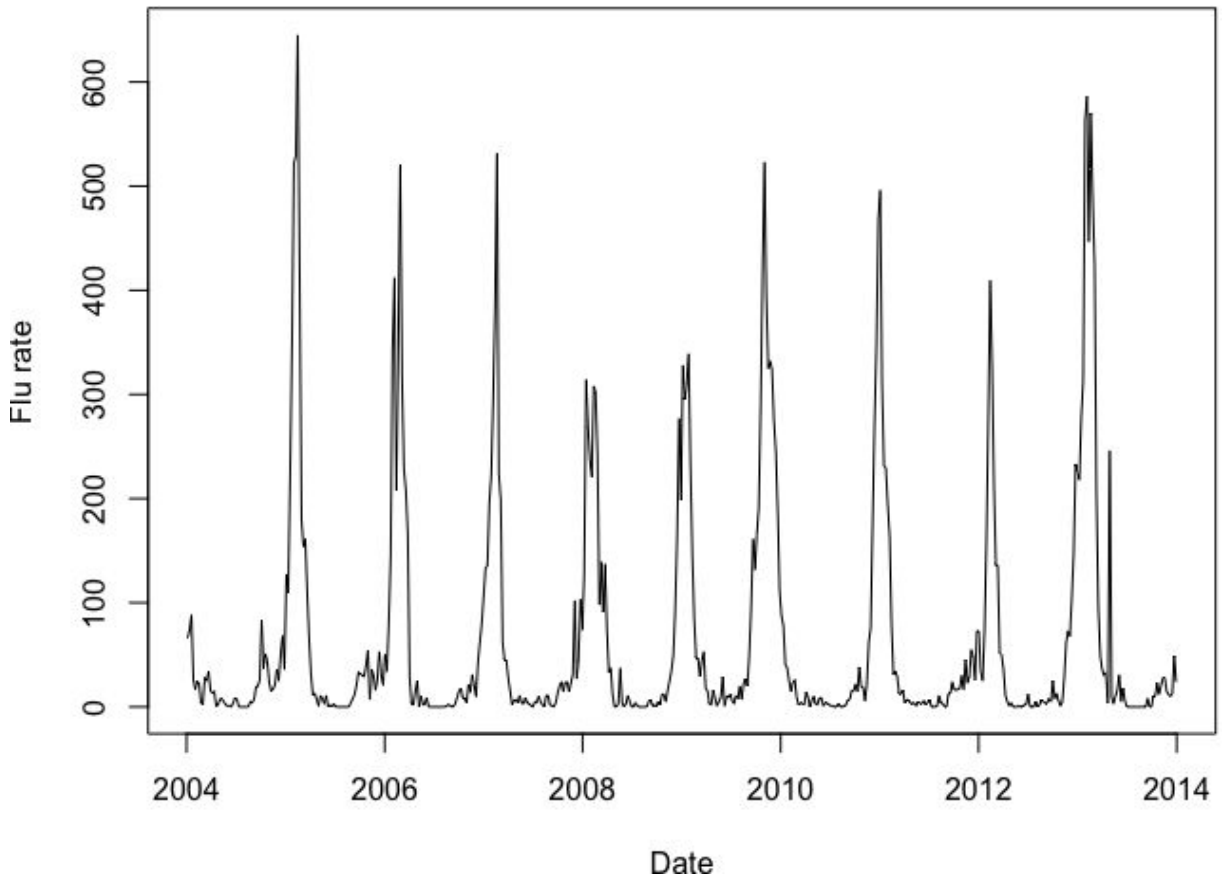


Figura 13-1. Ao traçar a série temporal da taxa de gripe, podemos ver a sazonalidade do taxa de gripe em Paris.

Se excluir um ponto de dados é um problema significativo depende do conjunto de dados e da pergunta que estamos fazendo. Deixo como exercício para o leitor explorar outras possibilidades de ajuste dos dados, mantendo os dados da 53ª semana. Existem várias opções para fazer isso. Uma delas é mesclar os dados da 53ª semana com os dados da 52ª semana, calculando a média das duas semanas. Outra é usar um modelo que possa levar em conta o comportamento cíclico sem ter que ficar preso a um ciclo exatamente com a mesma duração a cada ano. Uma terceira opção é que um modelo de aprendizado de máquina possa acomodar isso com alguma rotulagem criativa dos dados para indicar a sazonalidade ao modelo como um recurso de entrada.

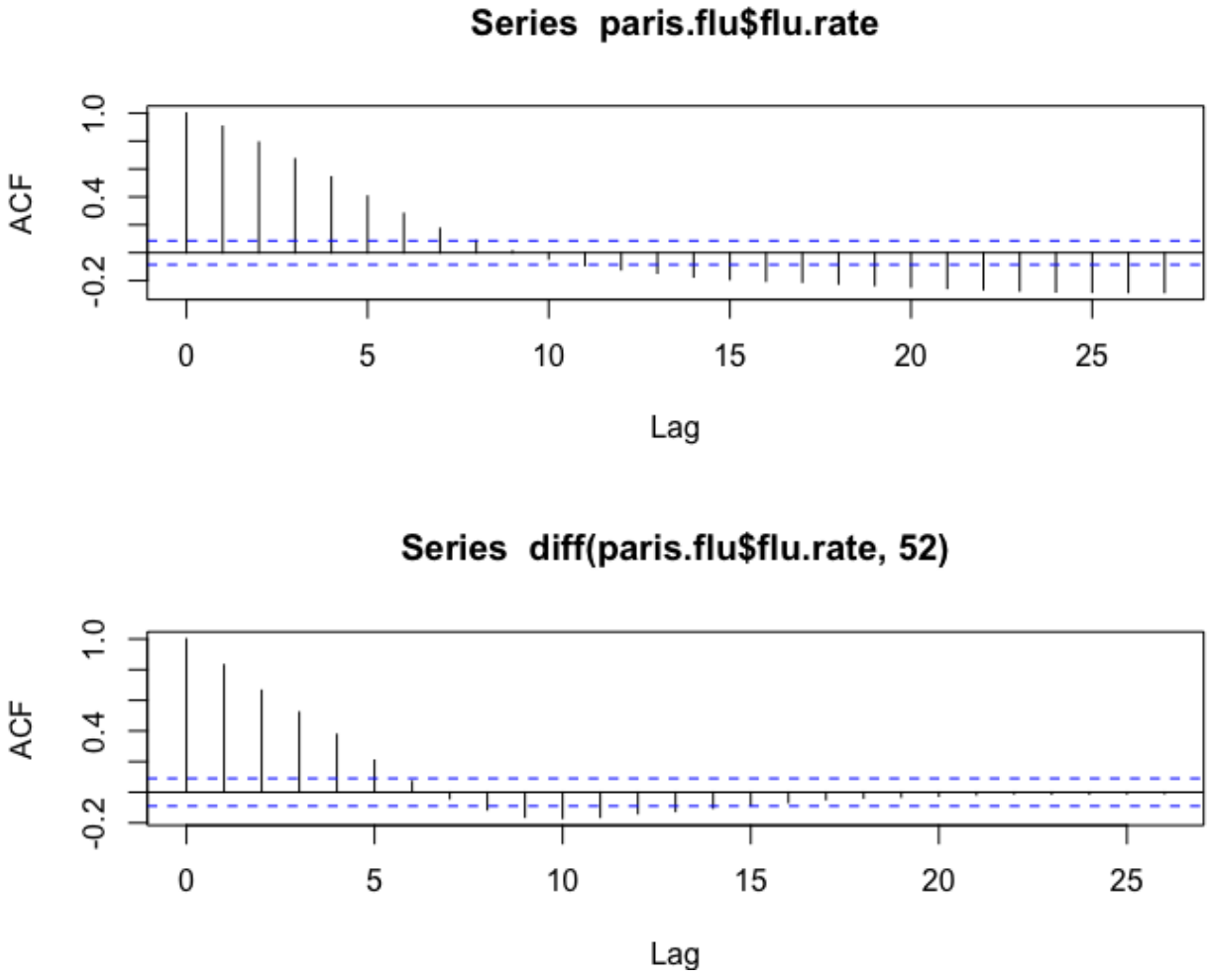
Ajustando um modelo ARIMA sazonal

Primeiro consideramos ajustar um modelo ARIMA sazonal aos dados devido à forte sazonalidade. Neste caso a

periodicidade dos dados é 52, uma vez que os dados são amostrados semanalmente. Queremos escolher um modelo bastante parcimonioso – isto é, sem muitos parâmetros – porque com 520 pontos de dados a nossa série temporal não é particularmente longa.

Esta série temporal é um bom exemplo de como podemos errar se confiarmos demais no piloto automático. Por exemplo, podemos primeiro considerar se devemos diferenciar os dados e, assim, podemos considerar o gráfico da autocorrelação da taxa de gripe e da autocorrelação.

ção das séries temporais diferenciadas da taxa de gripe, cada uma mostrada na Figura 13-2 : **Previsão da gripe | 371**



##R

```
> acf(paris.flu$flu.rate, )

> acf(diff(paris.flu$flu.rate, 52))
```

Figura 13-2. Gráfico da função de autocorrelação para a taxa de gripe de Paris e a taxa de gripe de Paris diferenciada. Observamos apenas uma faixa limitada de valores de atraso.

Se fôssemos descuidados, poderíamos muito bem nos parabenizar por termos resolvido o problema da estacionariedade para esta série temporal, diferenciando uma vez. Mas isso não faz sentido algum. São dados semanais e observamos uma forte sazonalidade. Por que não estamos vendo isso em nosso gráfico de autocorrelação? Pegamos os parâmetros padrão da função acf(), e isso não nos leva longe o suficiente no espaço de atraso para ver os efeitos sazonais, que começamos no atraso 52 (um ano). Vamos refazer o acf() com uma janela adequada (Figura 13-3):

##R

```
> acf(paris.flu$flu.rate, lag.max = 104)

> acf(diff(paris.flu$flu.rate, 52), lag.max = 104)
```

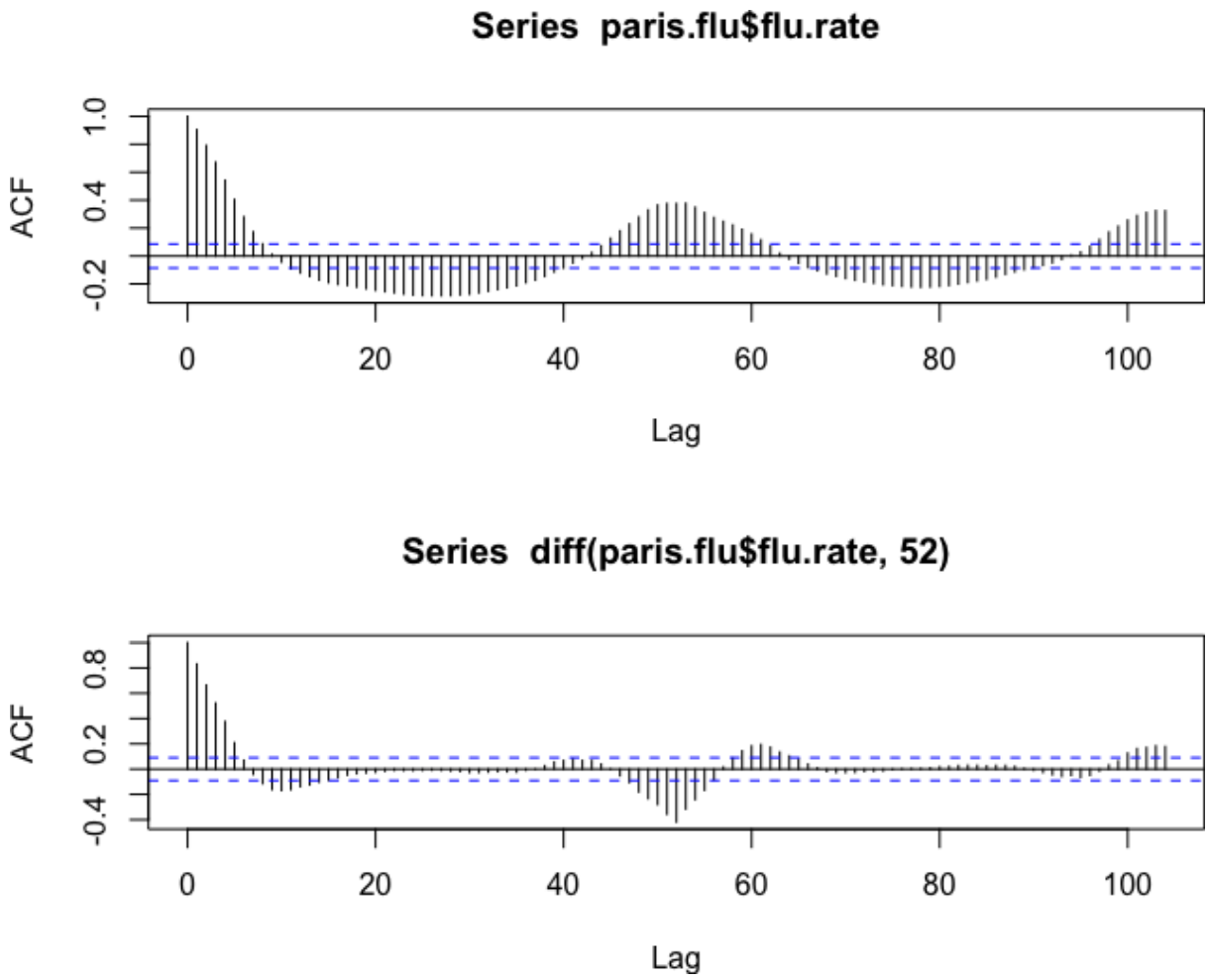


Figura 13-3. Gráfico da função de autocorrelação para a taxa de gripe de Paris e a taxa de gripe de Paris diferenciada. Vemos agora uma gama mais extensa de valores de atraso.

Isto nos dá uma imagem mais realista da autocorrelação de nossa série. Como podemos ver, existem autocorrelações substanciais em vários desfasamentos, e isto faz sentido (pelo menos dada a minha experiência) viver num clima de quatro estações. As taxas de gripe terão uma forte correlação com as semanas vizinhas – ou seja, próximas do momento da medição.

As taxas de gripe também terão uma forte correlação, dada a sazonalidade, com períodos desfasados em cerca de 52 ou em torno de 104, porque isto indica a sazonalidade anual. Mas as taxas de gripe também têm uma relação bastante forte com períodos desfasados por valores intermédios,

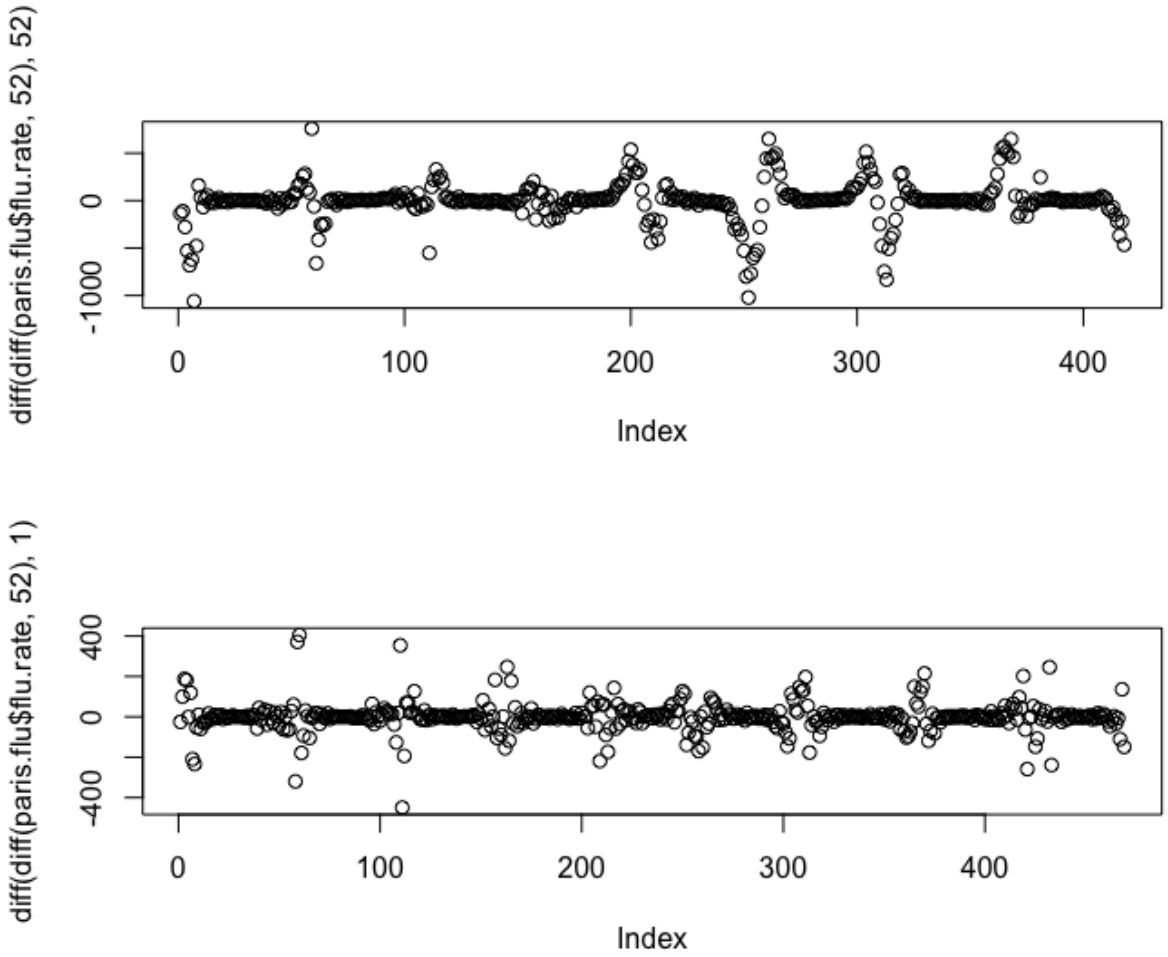
como meio ano, como meio ano (26 semanas), porque esses desfasamentos também se relacionam com diferenças sazonais e variações climáticas previsíveis. Por exemplo, sabemos que em meio ano o valor da gripe provavelmente terá mudado bastante. Se antes estava alto, agora deverá estar baixo e vice-versa, novamente devido à sazonalidade. Tudo isso é

representado no gráfico superior da Figura 13-3 .

Examinamos então as séries diferenciadas conforme ilustrado no gráfico inferior em

Figura 13-3. Agora vemos que uma quantidade substancial da autocorrelação de séries temporais

sido diminuído. No entanto, permanece bastante autocorrelação num intervalo de valores, não apenas às 52 ou 104 semanas (um ou dois anos), mas também em valores intermédios.



Embora possamos ficar tentados a continuar diferenciando, precisamos lembrar que os dados do mundo real nunca se ajustarão perfeitamente a um modelo SARIMA. Em vez disso, procuramos a maneira mais razoável de modelar os dados. Podemos considerar a diferenciação novamente sazonalmente ou adotar uma tática diferente e diferenciar no tempo linear. Representamos cada uma dessas possibilidades aqui (veja a Figura 13-4):

##R

```
> plot(diff(diff(paris.flu$flu.rate, 52), 52))

> plot(diff(diff(paris.flu$flu.rate, 52), 1))
```

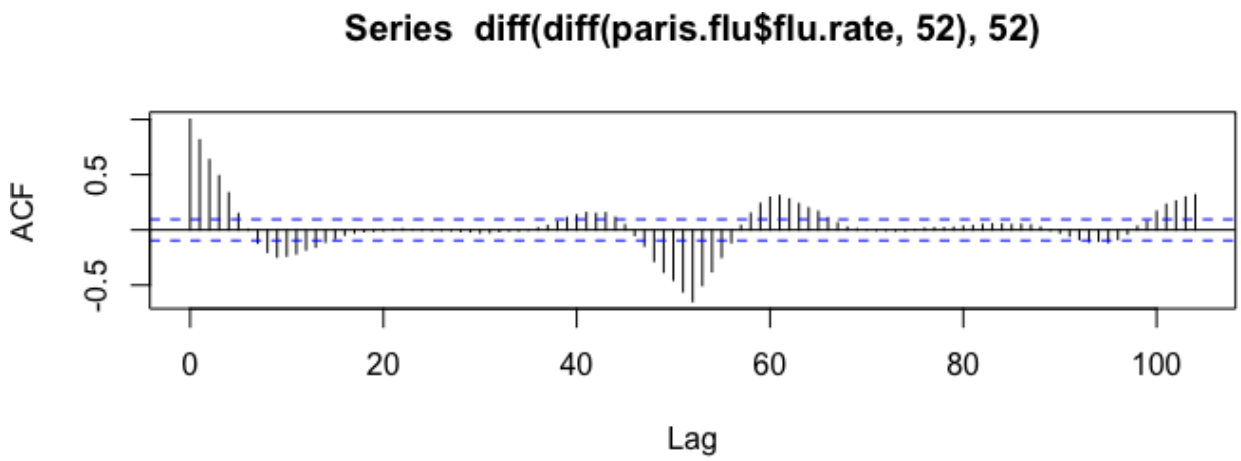
Figura 13-4. Trama de duas versões de diferenciação de nossa série para ter uma noção sazonal comportamento nos

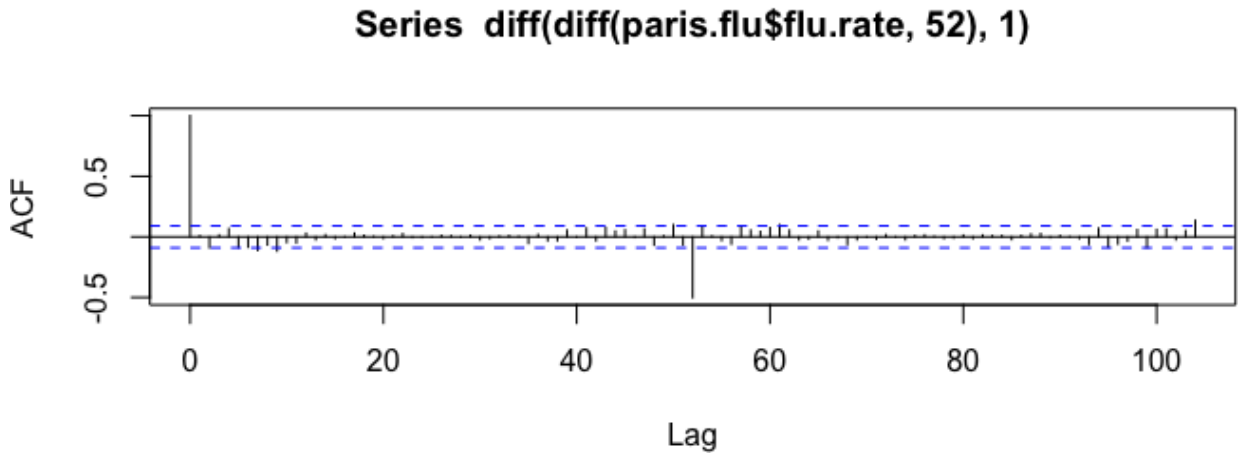
dados.

Embora nenhum dos resultados seja ideal, o último, uma primeira diferenciação padrão de uma diferenciação sazonal, é mais satisfatório.

A decisão de um ajuste ou escolha de um parâmetro é tanto uma questão de julgamento quanto de aplicação de testes. Aqui optamos por dar peso à sazonalidade que observámos claramente, mas também por não complicar demasiado o modelo ou torná-lo indevidamente opaco. Portanto, em um modelo SARIMA $(p , d , q) (P , D , Q)$, ajustaremos $d = 1$ e $D = 1$. Em seguida, escolhemos nossos parâmetros AR e MA para nossos parâmetros ARIMA padrão, p e q . Fazemos isso com stan-

visualizações padrão, usando o seguinte código (veja a Figura 13-5):





##R

```
> par(mfrow = c(2, 1))

> acf (diff(diff(paris.flu$flu.rate, 52), 1), lag.max = 104)

> pacf(diff(diff(paris.flu$flu.rate, 52), 1), lag.max = 104)
```

Figura 13-5. Gráfico da função de autocorrelação parcial das séries diferenciadas que selecionei.

Temos um conjunto de dados limitado e optamos por um modelo mais simples. O modelo PACF sugere que um modelo AR(2) poderia ser apropriado, portanto modelaremos nossos dados parcimoniosamente como um SARIMA (2, 1, 0), (0, 1, 0).

Estamos interessados em compreender como este modelo funcionaria se o ajustássemos continuamente a novos dados à medida que se tornassem disponíveis, da mesma forma que a maioria dos modelos construídos para sistemas do mundo real funcionarão. Ou seja, se estivéssemos modelando a gripe há vários anos, modelando cada semana apenas com os dados que estavam disponíveis até aquele momento, como seria o desempenho do nosso modelo? Respondemos a isso avançando o ajuste e a avaliação do modelo, como segue:

##R

> *## arima fit*

> *## vamos estimar 2 semanas à frente*

> primeiro.fit.tamanho <- 104

> h <- 2

> n <- nrow(paris.flu) - h - first.fit.size

>

> *## obtenha dimensões padrão para ajustes que produziremos*

> *## e informações relacionadas, como coefs*

> first.fit <- arima(paris.flu$flu.rate[1:first.fit.size], pedido = c(2, 1, 0),

> sazonal = lista(ordem = c(0,1,0), período = 52))

> primeiro.pedido <- arimaorder(primeiro.fit)

>

> *## pré-alocar espaço para armazenar nossas previsões e coeficientes*

```
> fit.preds <- array(0, dim = c(n, h))

> fit.coefs <- array(0, dim = c(n, length(first.fit$coef)))

>

> ## após o ajuste inicial, rolamos o ajuste para frente

> ## uma semana de cada vez, cada vez reajustando o
modelo

> ## e salvando os novos coefs e a nova previsão

> ## cuidado! esse loop demora um pouco para ser
executado

> for (i in (first.fit.size + 1):(nrow(paris.flu) - h)) {

> ## prever uma janela cada vez maior

> dados.to.fit = paris.flu[1:i]

> fit = arima(data.to.fit$flu.rate, pedido =
primeiro.pedido[1:3],

> sazonal = primeiro.pedido[4:6])

> fit.preds[i - first.fit.size, ] <- previsão(ajuste, h = 2)$mean

> fit.coefs[i - first.fit.size, ] <- fit$coef

> }
```

Em seguida, plotamos esses resultados contínuos (veja a Figura 13-6):

```
##R

> ylim <- range(paris.flu$flu.rate[300:400],

> fit.preds[, h][(300-h):(400-h)])
```

```
> par(mfrow = c(1, 1))

> plot(paris.flu$data[300:400], paris.flu$flu.rate[300:400],

> ylim = ylim, cex = 0,8,

> main = "Gripe real e prevista com SARIMA (2, 1, 0), (0, 1, 0)",

> xlab = "Data", ylab = "Taxa de gripe")

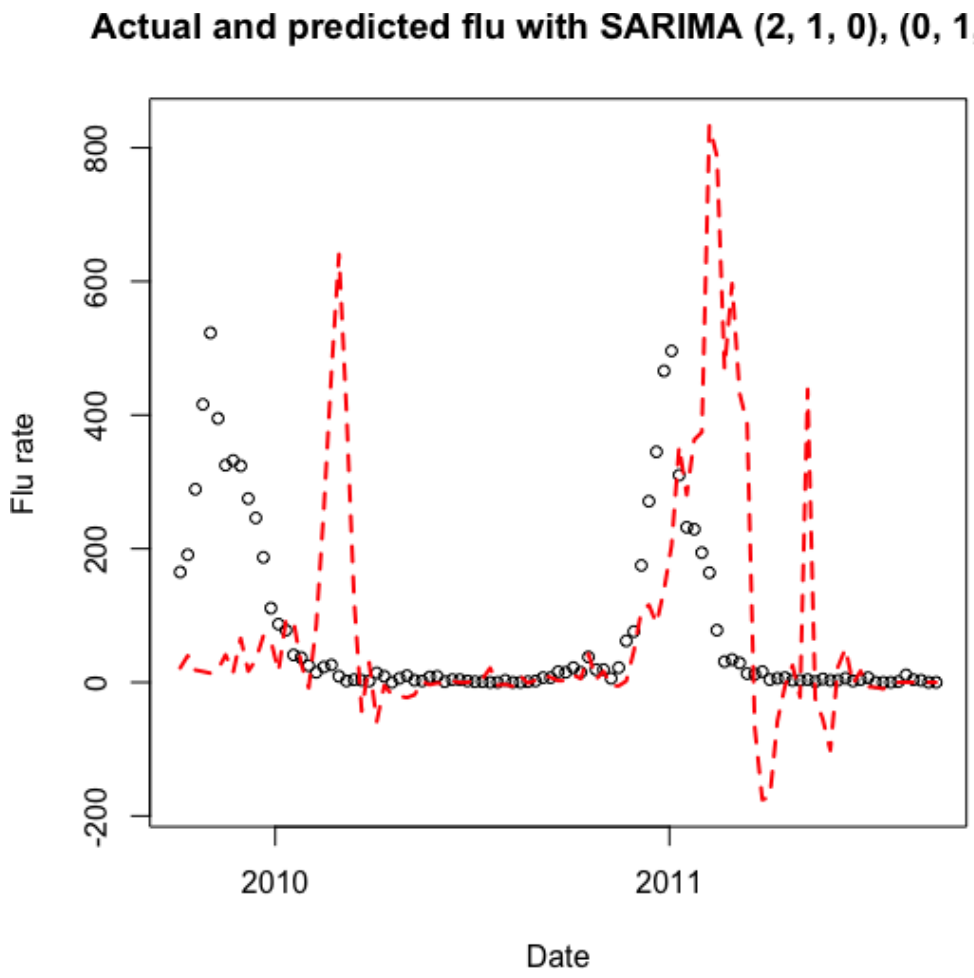
> linhas(paris.flu$date[300:400], fit.preds[, h][(300-h):(400-h)],

> col = 2, tipo = "l",

> lty = 2, lwd = 2)
```

O gráfico mostra uma previsão útil, mas também destaca algumas limitações deste modelo. Não é suficientemente realista, como demonstra o facto de, por vezes, prever taxas de gripe negativas. O modelo pode fazê-lo porque não há nada intrínseco a um modelo ARIMA para impor restrições, tais como as taxas de gripe terem de ser não negativas. Aplicar esses tipos de restrições físicas é algo que devemos fazer com as transformações de dados antes de ajustar um modelo.

Além disso, o modelo parece mais sensível a pontos discrepantes do que gostaríamos. O início de 2013 é um excelente exemplo de onde várias vezes o modelo superestima drasticamente a taxa de gripe. Quando se trata de afectação de recursos cruciais para combater doenças, este não é um modelo aceitável.



Finalmente, este modelo produz valores mais extremos nos picos do que os jamais medidos num determinado ano. Isto poderia levar a uma sobrealocação de recursos relativamente às necessidades reais, o que não é um bom resultado, especialmente quando os recursos são altamente limitados. Esta é uma preocupação sobre o modelo que está tão enraizada nas restrições de recursos do mundo real como na pura análise de dados.

Figura 13-6. As taxas de gripe (pontos) combinadas com nossas previsões SARIMA (linha pontilhada).

As previsões deste modelo simples poderiam auxiliar no planejamento da saúde pública.

Agora que consideramos o ajuste de um modelo ARIMA básico a este problema, daremos uma olhada em outras possibilidades de modelagem.

Um modelo ARIMA alternativo: regressores harmônicos exógenos em vez de sazonalidade Dado o desempenho do modelo SARIMA discutido na seção anterior, há uma série de modificações que podemos fazer. Aqui consideramos duas modificações, cada uma das quais é independente da outra e pode ser aplicada separadamente. [6]

Primeiro, gostaríamos de incluir restrições em nosso modelo para evitar a previsão de valores negativos. Uma maneira de fazer isso é uma transformação logarítmica dos dados, de modo que prevejamos o logaritmo do valor da série temporal em vez do valor em si.

6 Normalmente, você deve considerá-los separadamente, mas aqui eles são combinados para serem sucintos.

Então, quando quisermos ver a série "verdadeira" que representa os números realmente medidos, usaremos uma transformação exponencial para reverter a transformação logarítmica e obter as previsões em suas unidades reais.

Em segundo lugar, gostaríamos de encontrar uma forma mais transparente de lidar com a sazonalidade dos dados. Embora tenhamos descrito nosso modelo ARIMA sazonal como simples, na verdade um modelo que lida com uma recorrência sazonal de 52 não é nada simples. Os modelos ARIMA sazonais funcionam melhor em ciclos sazonais mais curtos e menos bem em ciclos sazonais mais longos (52

é um longo ciclo sazonal).

Aqui usaremos *regressão harmônica dinâmica* . Nesta abordagem, encontramos uma série de Fourier que descreve a periodicidade em nossos dados e, em seguida, usamos esta série como uma série exógena.

regressor nous que se ajusta aos termos ARIMA.7 Como podemos extrapolar a série de Fourier para a frente no tempo (devido à sua natureza puramente periódica), também podemos pré-calcular os valores que esperamos para o futuro ao gerar previsões.

A força deste modelo é que os graus de liberdade do modelo podem ser usados para explicar o comportamento subjacente, além do comportamento sazonal, em vez de dedicar muito poder explicativo ao comportamento sazonal.

Existem algumas desvantagens na regressão harmônica dinâmica. Primeiro, assumimos que o comportamento é bastante regular e ocorre exatamente no mesmo intervalo. Em segundo lugar, assumimos que o comportamento sazonal não está a mudar; isto é, o período e a amplitude do comportamento sazonal não mudam. Estas limitações são semelhantes às do modelo SARIMA, embora o modelo SARIMA mostre mais flexibilidade na forma como a amplitude da sazonalidade afecta os dados ao longo do tempo.

Aqui, demonstramos como realizar uma regressão harmônica dinâmica com código R semelhante ao que empregamos antes:

##R

> *## pré-alocar vetores para conter coefs e ajustes*

> fit.preds <- array(0, dim = c(n, h))

> fit.coefs <- array(0, dim = c(n, 100))

>

> *## regressores exógenos*

> *## que são componentes da série de Fourier ajustados aos dados*

> flu.ts <- ts(log(paris.flu$flu.rate + 1) + 0,0001,

> frequência = 52)

> *## adicione pequenos deslocamentos porque valores pequenos/0*

> *## causa problemas numéricos no ajuste*

```
> exog.regressores <- fourier(flu.ts, K = 2)

> exog.colnames <- colnames(exog.regressores)
```

7 Uma série de Fourier é a prática de descrever uma função como uma série de senos e cossenos. Mencionamos anteriormente a série de Fourier de passagem. Se você não está familiarizado com a série de Fourier, reserve alguns minutos para ler o histórico. Existem maneiras bem estabelecidas de "ajustar" uma série de Fourier a qualquer série temporal e estão amplamente disponíveis em pacotes R e Python. Ritchie Vink tem um breve tutorial que gosto muito.


```
>

> ## ajuste o modelo novamente a cada semana com

> ## expandindo janela de dados de treinamento

> for (i in (first.fit.size + 1):(nrow(paris.flu) - h)) {

> data.to.fit <- ts(flu.ts[1:i], frequência = 52)

> exogs.for.fit <- exog.regressors[1:i,]

> exogs.for.predict <- exog.regressors[(i + 1):(i + h),]

>

> ajustar <- auto.arima(data.to.fit,

> xreg = exogs.for.fit,

> sazonal = FALSO )

>

> fit.preds[i - first.fit.size, ] <- previsão(ajuste, h = h,

> xreg = exogs.for.predict)$mean
```

> fit.coefs[i - first.fit.size, 1:length(fit$coef)] = fit$coef

> }

Aqui fizemos alguns ajustes no código da seção anterior. Primeiro, usamos um objeto ts.8 Com um objeto ts, indicamos explicitamente a sazonalidade da série temporal (52 semanas), ao criar o objeto ts.

Também transformamos os dados em log neste ponto para garantir previsões positivas do nosso valor final de interesse, a taxa de gripe:

##R

> flu.ts = ts(log(paris.flu$flu.rate + 1) + 0,0001, *## adicionar épsilon*

> frequência = 52)

Também adicionamos um pequeno deslocamento numérico (+ 0,0001) porque o ajuste numérico não funciona bem com valores zero estritos ou mesmo com valores muito pequenos. Um dos nossos dois ajustes já foi realizado apenas com esta linha de código (ou seja, a transformação logarítmica para impor uma condição física de valores não negativos).

Em seguida, geramos os regressores harmônicos exógenos (ou seja, a aproximação de Fourier) que estamos usando no lugar dos parâmetros sazonais no SARIMA. Fazemos isso através da função fourier() do pacote de previsão:

##R

> exog.regressores <- fourier(flu.ts, K = 2)

> exog.colnames <- colnames(exog.regressores)

Geramos primeiro a série harmônica que a acompanha para toda a nossa série temporal e, em seguida, subdefinimos-a conforme apropriado para nosso ajuste de rolagem de janela de expansão posteriormente no loop.

O hiperparâmetro K indica quantos pares seno/cosseno separados incluiremos em nosso ajuste, onde cada um representa uma nova frequência usada para ajustar o seno/cosseno. Em geral, K será maior para períodos sazonais maiores e menor para períodos menores. 8 Estes foram brevemente discutidos no Capítulo 3.

uns. Num exemplo mais extenso, poderíamos considerar como usar um critério de informação para ajustar K, mas para este exemplo usamos K = 2 como um modelo razoável.

Finalmente, tudo o que resta fazer é gerar novos ajustes que levem em conta as componentes exógenas de Fourier que acabamos de ajustar. Fazemos o ajuste da seguinte maneira, onde o parâmetro xreg considera a série de Fourier ajustada como regressores adicionais, que são ajustados junto com os parâmetros ARIMA padrão:

```
##R

> ajustar <- auto.arima(data.to.fit,

> xreg = exogs.for.fit,

> sazonal = FALSO )
```

Definimos o parâmetro sazonal como FALSE para garantir que não teremos parâmetros sazonais redundantes, dada a nossa decisão de usar a regressão harmônica dinâmica neste caso.

Também precisamos incluir regressores quando geramos uma previsão, o que significa que precisamos indicar quais serão os regressores no momento em que a previsão é direcionada:

```
##R

> fit.preds[i - first.fit.size, ] <- previsão(ajuste, h = 2h,

> xreg = exogs.for.predict)$mean
```

Traçamos o desempenho deste modelo da seguinte forma (veja a Figura 13-7):

##R

> ylim = intervalo(paris.flu$flu.rate)

> plot(paris.flu$data[300:400], paris.flu$flu.rate[300:400],

> ylim = ylim, cex = 0,8,

> main = "Gripe real e prevista com ARIMA +

> regressores harmônicos",

> xlab = "Data", ylab = "Taxa de gripe")

> linhas(paris.flu$date[300:400], exp(fit.preds[, h][(300-h):(400-h)]),

> col = 2, tipo = 'l',

> lty = 2, lwd = 2)

Do lado positivo, nossa série prevista não apresenta mais valores negativos. No entanto, existem muitos aspectos decepcionantes no desempenho do modelo. Obviamente, muitas das previsões estão bastante erradas. Os picos têm a magnitude errada e ocorrem na hora errada.

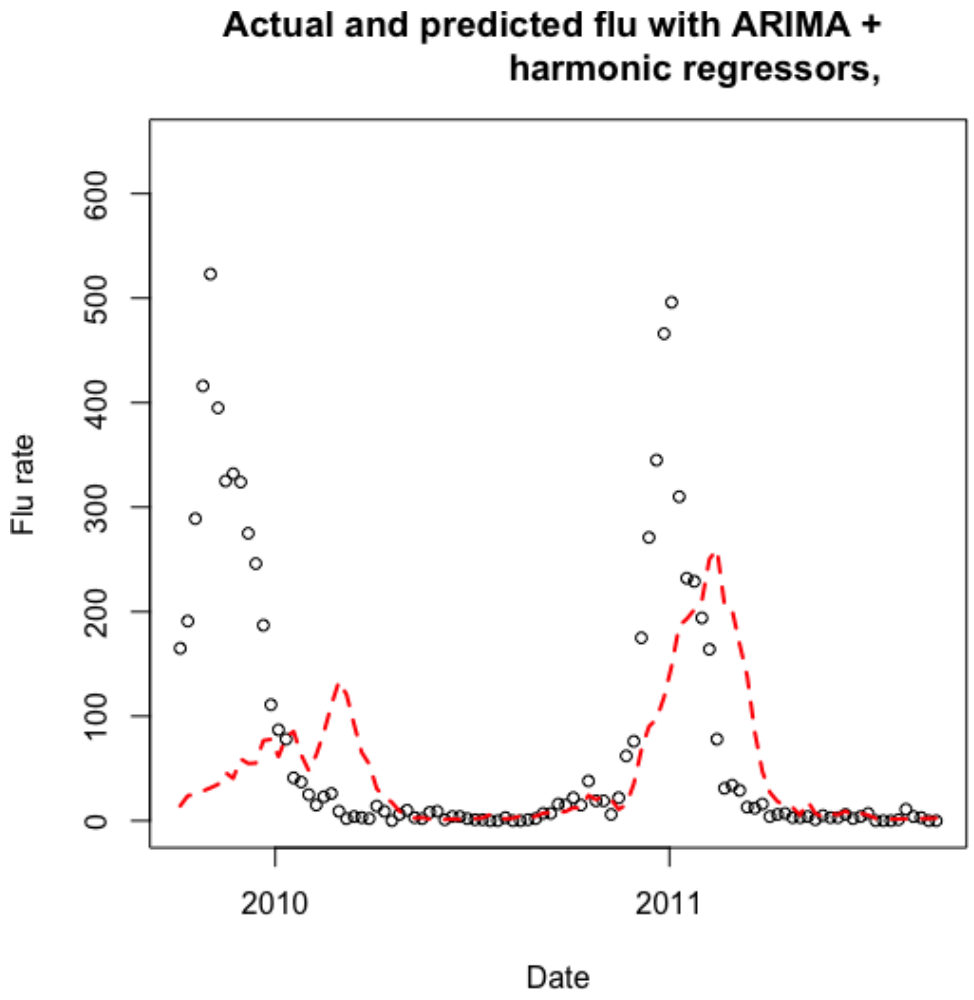


Figura 13-7. Gráfico das taxas reais de gripe (pontos) em comparação com nossa dinâmica ARIMA + previsões do modelo de regressão harmônica (linha tracejada).

Uma explicação para os problemas é que a sazonalidade regular não é uma boa descrição

da sazonalidade da gripe.9 De acordo com os Centros de Controle e Prevenção de Doenças

Convenção (CDC) , nos Estados Unidos, a gripe pode atingir o pico em cada temporada de inverno, no início de dezembro ou no final de março. Podemos ver isso nos dados de teste. Considere o seguinte

código e o gráfico na Figura 13-8, que identifica os picos na faixa de teste do

dados:

##R

> plot(teste$flu.rate)

> qual(teste$flu.rate > 400)

Erro em which(res$flu.rate > 400): objeto 'res' não encontrado

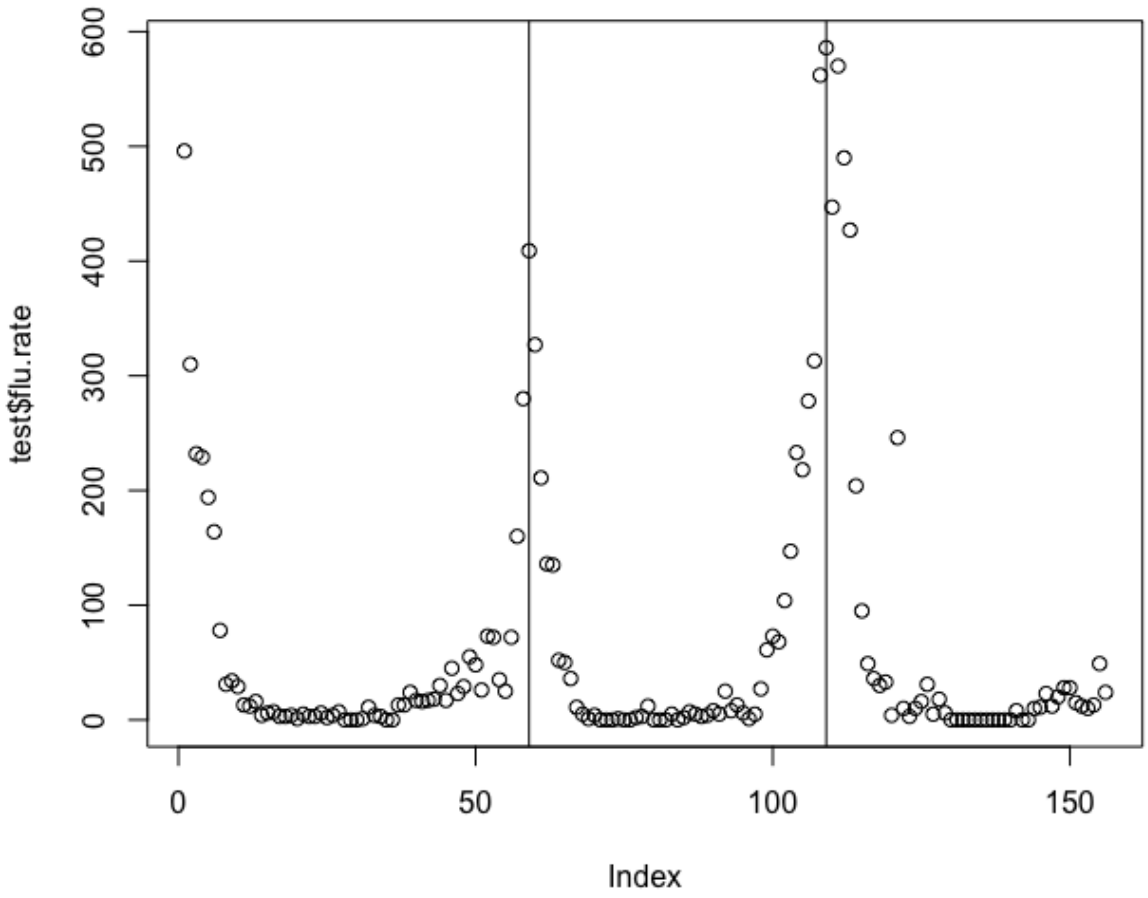
> qual(teste$flu.rate > 400)

[1] 1 59 108 109 110 111 112 113

> abline(v = 59)

> which.max(teste$flu.rate)

9 Para testar se esta explicação é boa para a situação, você poderia usar uma simulação para gerar alguns dados sintéticos para testar conjuntos de dados alternativos com este modelo e ver se o comportamento que é cíclico e não sazonal, como os dados da gripe parecem sugerir , causa esse tipo de falha do modelo de regressor harmônico exógeno.



[1] 109

> abline(v = 109)

Figura 13-8. Gráfico apenas dos valores de teste para a gripe e seus aparentes locais de pico.

Os picos ocorrem nos índices 59 e 109, ou seja, com 50 semanas (e não 52 semanas) de intervalo um do outro. Além disso, o pico que ocorreu no índice 59 ocorreu pelo menos 59 semanas após o pico anterior, e possivelmente mais, uma vez que não temos a extensão total do pico em torno do índice

= 0 em nosso gráfico.

Com distâncias amostrais pico a pico de mais de 59 em um caso e 50 em outro, podemos ver uma grande variabilidade de ano para ano. A nossa regressão harmónica dinâmica não

levaria isto em consideração e impõe um modelo de sazonalidade mais rígido do que o modelo SARIMA, porque o comportamento sazonal deste último pode mudar ao longo do tempo. Esta incompatibilidade entre os pressupostos e os dados pode explicar em grande parte o mau desempenho deste modelo, de tal forma que a utilização de um modelo mau chamou realmente a nossa atenção para uma característica importante dos nossos dados que não tínhamos notado anteriormente.

Apesar do fraco desempenho, este modelo alternativo provou ser útil por alguns motivos.

Ele nos mostrou o valor de incluir as limitações físicas do sistema em nosso pré-processamento de dados, neste caso por meio de uma transformação de log. Também nos mostrou o valor de experimentar diversas classes de modelos com uma teoria subjacente por trás da seleção do modelo em cada ponto. Escolhemos este modelo para simplificar a nossa sazonalidade, mas **382 | Capítulo 13: Aplicações de Saúde**

em vez disso, o que descobrimos é que, provavelmente, nem o SARIMA nem o modelo de regressão harmónica dinâmica da sazonalidade são muito bons para este sistema.

O que é o estado da arte na previsão da gripe?

Os modelos que acabámos de explorar eram relativamente simples, mas razoavelmente eficazes na elaboração de previsões ou previsões imediatas das taxas de gripe na região metropolitana de Paris para horizontes de previsão curtos. Estes modelos são um bom começo tanto para conhecer o seu conjunto de dados como para reconhecer as limitações do que poderia ser preditivo num sistema ruidoso mas complicado, como o da gripe numa determinada região geográfica.

É interessante comparar as nossas abordagens simples com algumas abordagens de ponta actualmente implementadas por agências governamentais ou publicadas recentemente por académicos que procuram discutir o estado da arte. A seguir, forneço uma breve visão geral de como as previsões da gripe funcionam atualmente e quais novas abordagens os

pesquisadores desenvolveram nos últimos anos para tentar melhorar os modelos tradicionais.

Pesquisa em previsões de gripe

O CDC incentiva ativamente os investigadores a trabalhar na previsão da gripe, patrocinando até um pacote R para disponibilizar os seus dados prontamente. Há mais de cinco anos, o

O CDC também patrocinou competições de previsão da gripe, embora tenha sido apenas para a temporada de gripe 2017-2018 que realmente incorporou uma previsão da gripe nas suas comunicações e boletins oficiais. O grupo de pesquisa Delphi da Carnegie Mellon venceu a maioria das competições até agora, e o grupo prevê a gripe com o que descreve como três esforços distintos:

• Uma abordagem empírica de Bayes que aplica dados da temporada de gripe passada com uma série de manipulações para formar dados anteriores para a temporada atual com base no total

"formas" das séries temporais de temporadas de gripe anteriores

• Uma plataforma de crowdsourcing onde qualquer pessoa pode enviar uma previsão de

taxas de gripe

• Um método de previsão a curto prazo que utiliza *fusão de sensores* , agregando dados de muitas fontes, como contagens de acesso à Wikipédia e consultas relevantes no Twitter, para produzir previsões de gripe geograficamente localizadas

Este é o conjunto de diversas abordagens utilizadas em apenas um grupo de pesquisa acadêmica! Se olharmos mais longe, para a academia, vemos ainda mais diversidade:

• Usar redes convolucionais profundas (CNNs) para classificar imagens do Instagram e usar os resultados dessas CNNS junto com recursos relacionados a texto do

Twitter como entradas em uma variedade de modelos de aprendizado de máquina, incluindo árvores XGBoost. Um desses artigos teve a vantagem de focar em uma pequena comunidade linguística (Fin-

oradores finlandeses), o que permitiu a utilização das principais plataformas de redes sociais de uma forma que ainda poderia ser regionalmente específica.

• Identificação de usuários confiáveis em agrupamentos maiores de mídia social. Um artigo se concentra em melhorar as previsões da gripe, encontrando os usuários mais bem posicionados e mais confiáveis nas plataformas de mídia social.

• Acessar registros eletrônicos de saúde para ter uma visão mais completa e complementar

fonte de dados, além dos dados disponíveis publicamente. Um artigo mostrou que ganhos extremamente grandes na precisão das previsões em diversas escalas de tempo poderiam ser obtidos através da integração de registros eletrônicos de saúde em fluxos de dados de previsão. Infelizmente, isto é difícil de organizar e sugere que as capacidades precisas de previsão da gripe irão para os detentores de dados ricos e não para os investigadores mais criativos (embora, claro, por vezes estes possam ser a mesma coisa).

Como podemos ver pela diversidade de abordagens a este problema, existem muitos caminhos para fazer uma previsão da gripe, e nenhum é o vencedor definitivo ainda. Esta continua a ser uma área de investigação e desenvolvimento activos, embora algumas das melhores estratégias tenham sido aplicadas a utilizações governamentais e à informação pública.

Nossa discussão aqui é apenas a ponta do iceberg. Há muita biologia, sociologia, medicina e política econômica que determinam o curso de uma temporada de gripe, e há uma variedade de modelos que são menos orientados para a análise de séries temporais e mais orientados para outros

aspectos do comportamento da gripe. . As séries temporais trazem uma perspectiva rica para um tópico muito complicado.

Previsão dos níveis de glicose no sangue

Outra área ativa de pesquisa de aprendizado de máquina para dados de séries temporais em aplicações de saúde é a previsão dos níveis de glicose no sangue para pacientes individuais. Os próprios pacientes com diabetes fazem tais previsões o tempo todo, especialmente se tiverem um nível de doença tal que se injectem com insulina em bolus, que é usada especificamente às refeições. Nesses casos, os diabéticos precisam estimar como os alimentos que estão prestes a ingerir afetarão o açúcar no sangue e ajustar a dose de acordo.

Da mesma forma, os pacientes com diabetes devem programar as refeições e a medicação para otimizar o nível de açúcar no sangue, que é melhor mantido dentro de uma faixa específica, nem muito alta nem muito baixa.

Além de precisar levar em conta as atividades que alteram o açúcar no sangue, como alimentação e exercícios, os diabéticos também precisam levar em conta os efeitos específicos do horário do dia. Por exemplo, o fenómeno do amanhecer é um aumento do açúcar no sangue que ocorre em todos os seres humanos, mas pode ser problemático para aqueles com diabetes. Por outro lado, para pessoas com diabetes tipo 1, os níveis baixos de açúcar no sangue durante as horas de sono podem ser eventos potencialmente fatais resultantes da incapacidade de fazer uma previsão precisa.

Aqui olhamos para um pequeno conjunto de dados: os dados do monitor contínuo de glicose (CGM) autorreferidos para um indivíduo em vários segmentos de tempo não contíguos. Esses dados foram autopublicados na internet e modificados para preservar a privacidade do paciente.

Existem outras opções para obter conjuntos de dados sobre diabetes. Além de grandes organizações de saúde e algumas

startups com grandes quantidades de dados de CGM, existem numerosos conjuntos de dados autopublicados disponíveis à medida que os indivíduos gerenciam cada vez mais seus

diabetes através de esforços DIY, como o projeto Night Scout. Existem também vários diabéticos

testes conjuntos de dados CGM abertos para fins de pesquisa.

Nesta seção, exploraremos a confusão de um conjunto de dados do mundo real e tentaremos prever esses dados.

Limpeza e exploração de dados

Os dados são armazenados em vários arquivos disponíveis no repositório GitHub deste livro. Nós primeiro

carregue os arquivos e combine-os em um data.table:

##R

> arquivos <- list.files(full.names = **TRUE**)

> arquivos <- grep("entradas", arquivos, valor = **TRUE**)

> dt <- dados.tabela()

> para (f em arquivos) {

> dt <- rbindlist(lista(dt, fread(f)))

> }

>

> *## remova as colunas na*

> dt <- dt[!is.na(sgv)]

Strings de informações de data estão disponíveis, mas não temos uma coluna de classe de carimbo de data/hora

adequada, então criamos uma com as informações que temos, que inclui um fuso horário, bem como uma string de data:

##R

> dt[, carimbo de data/hora:= as.POSIXct(data)]

> *## isso funciona para mim porque meu computador está no horário EST*

> *## mas isso pode não ser verdade para você*

>

> *## maneira correta*

> dt[, carimbo de data/hora := force_tz(as.POSIXct(data), "EST")]

>

> *## ordenar cronologicamente*

> dt = dt[ordem(timestamp, decrescente = **FALSE**)]

Em seguida, inspecionamos o pós-processamento dos dados:

##R

> head(dt[, .(data, sgv)])

data sgv

1: 18/02/2015 06:30:09 162

2: 18/02/2015 06:30:09 162

3: 18/02/2015 06:30:09 162

4: 18/02/2015 06:30:09 162

5: 18/02/2015 06:35:09 154

6: 18/02/2015 06:35:09 154

Existem muitas entradas de dados duplicadas, então precisamos limpá-las por dois motivos:

• Não há nenhuma razão a priori para que certos pontos de dados sejam privilegiados ou mais ponderados do que outros, mas a duplicação produzirá esse efeito.

• Se formos formar uma feição baseada em janelas de séries temporais, elas não farão sentido se houver pontos no tempo duplicados.

Primeiro veremos se podemos resolver este problema removendo todas as linhas perfeitamente duplicadas:

##R

> dt <- dt[!duplicado(dt)]

No entanto, não devemos assumir que isto resolve o problema, por isso verificamos se existem pontos de dados não idênticos para o mesmo carimbo de data/hora:

##R

> agora(dt)

[1] 24861

> comprimento(único(dt$timestamp))

[1] 23273

> ## ainda temos dados duplicados em termos de carimbos de data/hora

> ## vamos excluí-los

Como ainda temos pontos de tempo duplicados, fazemos algumas análises de dados para ver o que isso significa. Identificamos o carimbo de data/hora com o maior número de linhas duplicadas e examinamos estes:

##R

> ## podemos identificar um exemplo usando

> ## dt[, .N, carimbo de data/hora][pedido(N)]

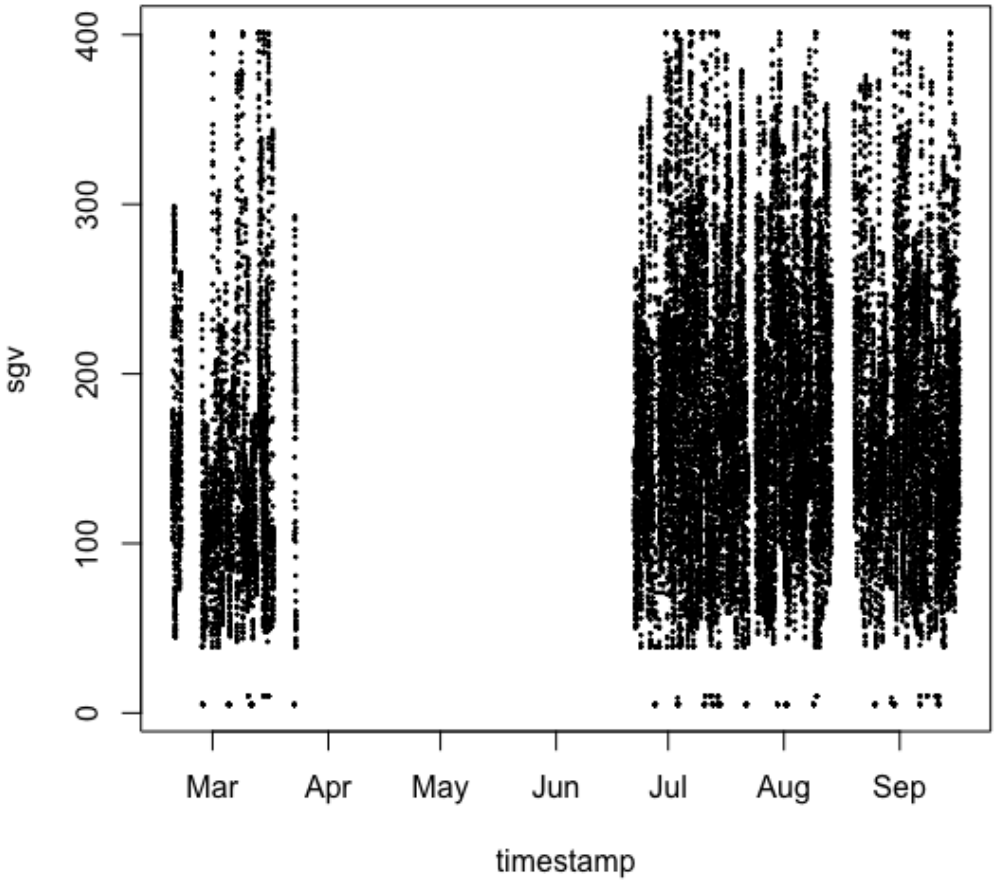
> ## então observe os dados mais repetidos

> dt[data == "10/03/2015 06:27:19"]

data do dispositivo dateString tipo de direção sgv

1: dexcom 10/03/2015 06:27:19 Ter, 10 de março 06:27:18 EDT 2015 66 Plano sgv 2: dexcom 10/03/2015 06:27:19 Ter, 10 de março 06:27:18 EDT 2015 70 Plano sgv 3: dexcom 10/03/2015 06:27:19 Ter 10 de março 06:27:18 EDT 2015 66 Plano sgv 4: dexcom 10/03/2015 06:27:19 Ter 10 de março 06:27:18 EDT 2015 70 Flat sgv 5: dexcom 2015-03-10 06:27:19 Ter 10 de março 06:27:18 EDT 2015 66 Flat sgv 6: dexcom 2015-03-10 06:27:19 Ter 10 de março 06:27: 18 EDT 2015 70 Flat sgv 7: dexcom 2015-03-10 06:27:19 Ter 10 de março 06:27:18 EDT 2015 66 Flat sgv 8: dexcom 2015-03-10 06:27:19 Ter 10 de março 06: 27:18 EDT 2015 70 Flat sgv

mais inspeções sugerem que isso não é muito importante



Olhando para estes valores, podemos ver que existem valores distintos de glicemia

relatados, mas eles não são muito diferentes,10 então suprimimos os carimbos de data e hora duplicados

mesmo que não sejam duplicatas exatas:11

##R

> dt <- único(dt, by=c("timestamp"))

Agora que temos carimbos de data e hora adequados e valores únicos por carimbo de data e hora, estamos em uma situação

posição para traçar nossos dados e ter uma noção de sua extensão e comportamento (veja a Figura 13-9).

Figura 13-9. O domínio de tempo integral e a faixa de valores dos dados disponíveis neste ingênuo gráfico da série temporal. Infelizmente, os dados estão tão dispersos no tempo e desarticulados que este gráfico não nos dá uma boa compreensão do comportamento do sistema.

Ampliamos o período que vemos no início da série temporal, março de 2015 (ver

Figura 13-10):

##R

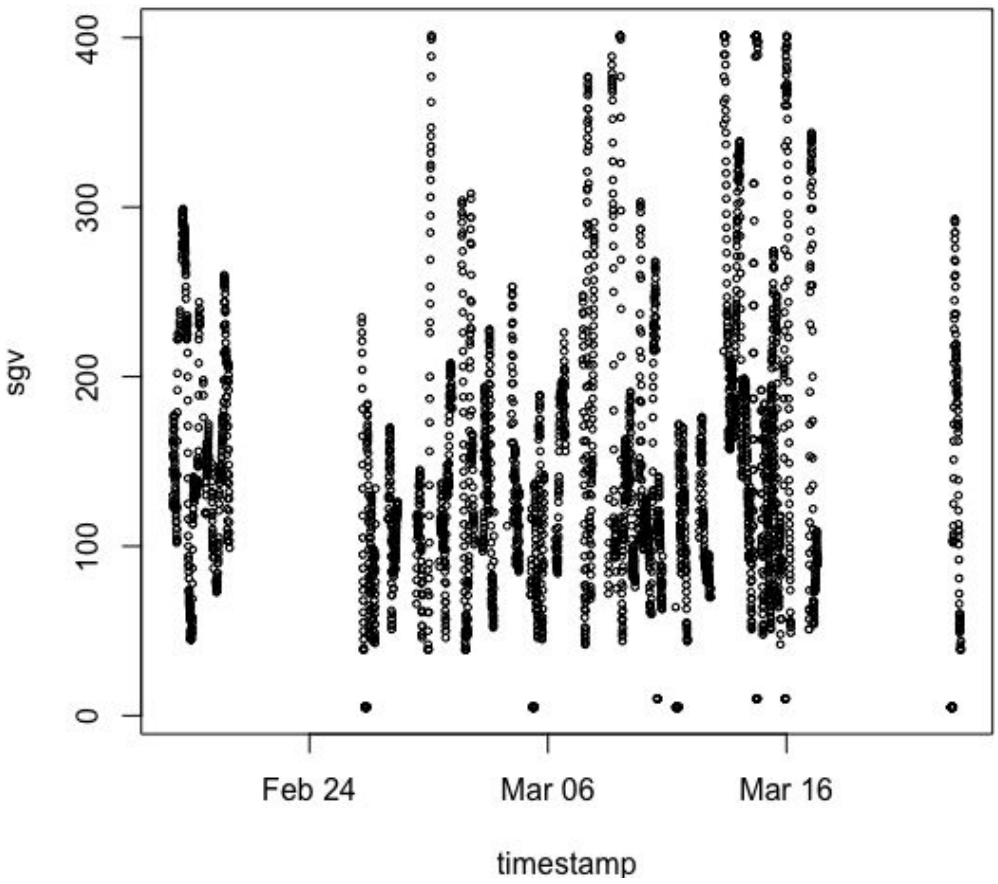
> ## vamos dar uma olhada em algumas janelas de tempo mais curtas nos dados

> data inicial <- as.Date("01/01/2015")

> stop.date <- as.Date("01/04/2015")

10 O erro aceite nos medidores de glicemia tende a ser cerca de 15 no sistema de unidades de medida dos EUA, que estes números utilizam.

11 Existem outras opções além da supressão. Você pode explorá-los por conta própria.



> dt[entre(timestamp, data de início, data de término),

> plot(carimbo de data e hora, sgv, cex = 0,5)]

Figura 13-10. Concentrar-se em um segmento específico da série temporal é mais útil, mas ainda assim muito comprimido para que possamos ter uma noção de qualquer tipo de dinâmica de série temporal. Deveríamos amplie ainda mais o eixo do tempo.

Mesmo no gráfico apenas das datas de março, ainda não temos uma noção do comportamento da série temporal. Existe sazonalidade? Deriva? Um padrão diário? Nós não sabemos, então nós

trace uma janela de tempo ainda mais curta (veja a Figura 13-11):

##R

```
> ## também observe um único dia para ver como pode ser

> ## se tivéssemos mais tempo provavelmente deveríamos
fazer histogramas 2D de dias

> ## e veja como isso funciona

> par(mfrow = c(2, 1))

> início.data = as.Date("04/07/2015")

> stop.date = as.Date("05/07/2015")

> dt[entre(timestamp, data de início, data de término),

> plot(carimbo de data e hora, sgv, cex = 0,5)]

>

> início.data = as.Date("05/07/2015")

> stop.date = as.Date("06/07/2015")

> dt[entre(timestamp, data de início, data de término),

> plot(carimbo de data e hora, sgv, cex = 0,5)]

> ## se tivéssemos mais dias provavelmente poderíamos
fazer alguns "day shape"

> ## análise, mas aqui parece que não temos dados
suficientes
```

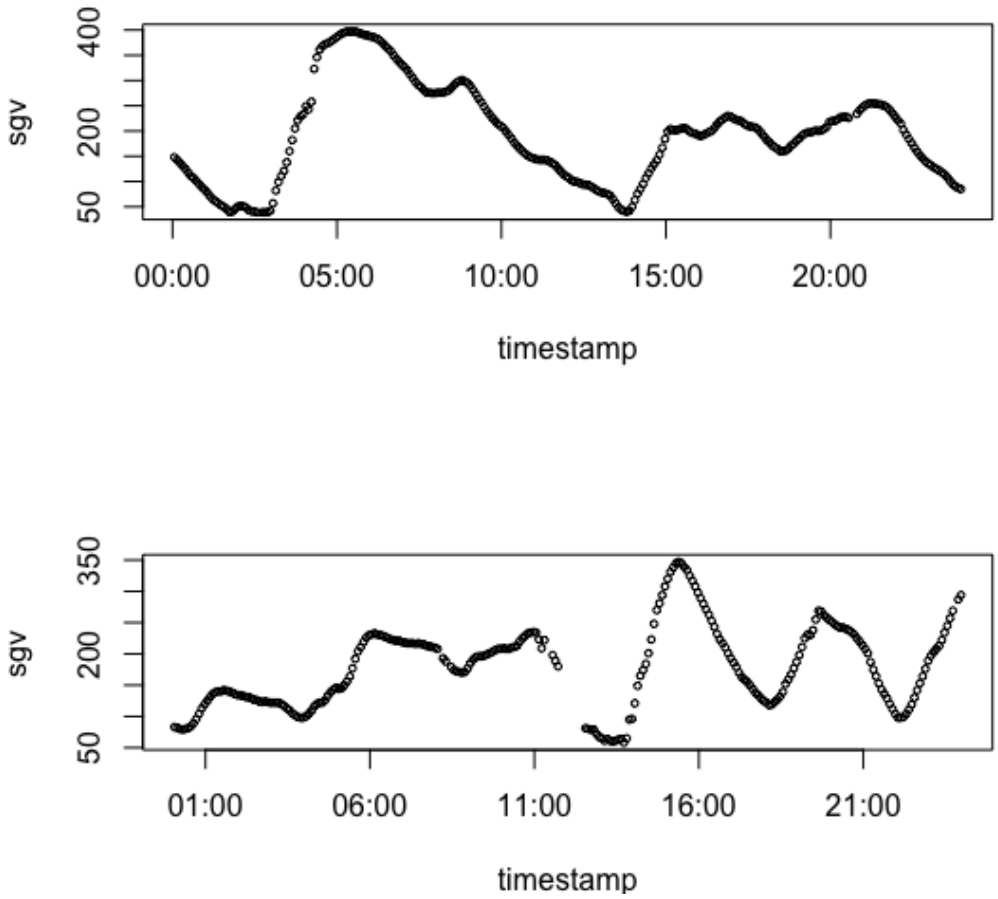


Figura 13-11. Gráfico dos dados de dois dias específicos de julho. Podemos ver algo de padrão diário emerge. Finalmente, estamos olhando para os dados em uma escala temporal onde o o olho humano pode entender o que está acontecendo.

Ao fazer mais desses gráficos exploratórios (deixados como exercício para o leitor), você deverá desenvolver uma intuição da escala dos dados, da dinâmica qualitativa e do que pode ser importante na descrição de seu comportamento. Algumas técnicas exploratórias adicionais que você deve aplicar a este conjunto de dados incluem:

• Histogramas 2D de duração do dia para procurar padrões intradiários e exercícios de agrupamento relacionados. Existem diferentes tipos de dias?

• Agrupar estatísticas sobre horas do dia ou estações do ano para procurar diferenças sistemáticas ao longo do tempo.

• Suavização dos dados para procurar tendências de longo prazo e particularmente para comparar relações interdiárias em vez de relações intradiárias. Seria valioso desenvolver previsões de glicemia de longo alcance, o que só pode ser feito olhando além dos padrões intradiários mais óbvios.

Prevendo os níveis de glicose no sangue | 389

Gerando Recursos

Munidos de algum conhecimento prévio – bem como de observações de nossos

breve exploração do conjunto de dados – podemos gerar recursos para nossos dados que seriam úteis para prever a glicemia.

Começamos caracterizando o próprio tempo. Por exemplo, notamos que o dia parece ter uma estrutura, o que sugere que a hora do dia deve ser relevante para fazer previsões. Da mesma forma, se você representar graficamente a série temporal de glicemia para meses diferentes

dos dados, alguns meses apresentam maior variação do que outros.12 Geramos alguns

recursos relacionados ao tempo aqui (veja a Figura 13-12):

##R

> ## deveríamos considerar incluir um recurso para o mês do ano.

> ## talvez haja efeitos sazonais

> dt[, mês := strftime(timestamp, "%m")]

>

> dt[, local.hora := as.numeric(strftime(timestamp, "%H"))]

> ## notamos que algumas horas do dia estão muito sobrerepresentadas

> ## esta pessoa não usa o dispositivo constantemente ou talvez

> ## o dispositivo tem alguma funcionalidade de horário do dia ou manutenção de registros

> ## questões. já que é um pico estranho à meia-noite, isso parece

> ## sugere o dispositivo em vez de um usuário (que coloca o dispositivo ao mesmo tempo

> ## liga e desliga à meia-noite?)

>

> hist(dt$local.hora)

Também consideramos as qualidades dos valores das séries temporais e não apenas os carimbos de data e hora da coleta de dados. Por exemplo, podemos ver que os valores médios de glicose no sangue variam com

a hora do dia (Figura 13-13):

##R

> ## os valores de glicemia tendem a depender da hora do dia

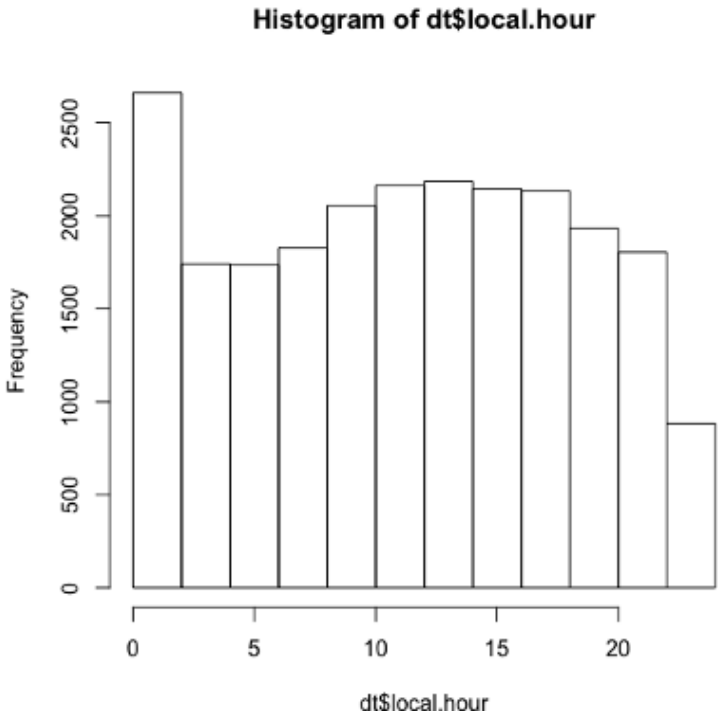
> plot(dt[, média(sgv), local.hora][order(local.hora)], tipo = 'l',

> ylab = "Glicemia média por hora local",

> xlab = "Hora local")

12 Todas estas observações devem ser encaradas com um elevado grau de cepticismo. Este conjunto de dados consiste apenas em dados de um único usuário durante partes de um

único ano e os dados não são contínuos. Isso significa que não podemos tirar conclusões sobre a sazonalidade, mesmo que sejamos tentados a especular. Ainda assim, para efeitos de ajuste aos dados limitados que temos, consideraremos estas componentes temporais.



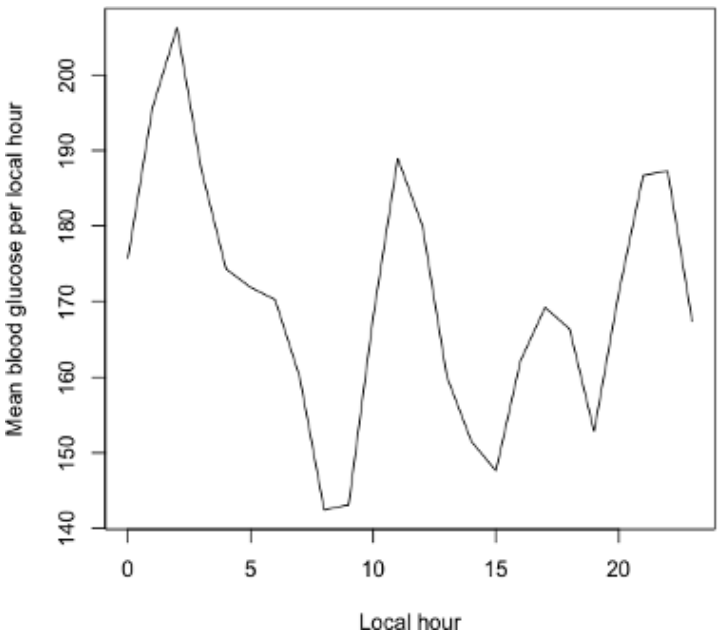


Figura 13-12. A partir do histograma da hora local podemos ver que os horários em que o CGM está a recolha e a comunicação de dados não são aleatórias. A forte concentração por volta da meia-noite sugere adicionalmente uma irregularidade nos relatórios ou no funcionamento do dispositivo que é provavelmente não devido ao comportamento do usuário (o usuário provavelmente não está manipulando seu dispositivo em aquela hora do dia).

Figura 13-13. A glicemia média varia substancialmente de acordo com a hora do dia.

Existem outros dados interessantes sobre os valores de glicemia. A coluna denominada direção no conjunto de dados extrai informações do dispositivo CGM, aplica o software proprietário do fabricante e apresenta um rótulo de tendência direcional para os dados. Podemos usar isso em vez de calcular nossas próprias estatísticas de tendências e, por isso, tentamos **prever os níveis de glicose no sangue | 391**

entenda um pouco, perguntando, por exemplo, se existem tendências diferentes para horários diferentes.

Definimos uma função que nos dará o *enésimo* rótulo mais popular para um determinado vetor e usamos isso primeiro para encontrar o rótulo mais popular para cada hora local do dia:

##R

> nth.pos = função(x, pos) {

> nomes(classificar(-tabela(x)))[pos]

> *## este trecho de código graças ao grupo de usuários r*

> }

> dt[, nth.pos(direção, 1), local.hora][pedido(local.hora)]

local.hora V1

1: 0 plano

2: 1 plano

3: 2 planos

...

21: 20 plano

22: 21 plano

23: 22 plano

24: 23 plano

local.hora V1

O rótulo de direção mais popular é “plano” para todas as horas do dia, o que é tranquilizador porque a dinâmica do sistema seria questionável se a tendência pudesse ser fortemente prevista simplesmente pela hora do dia.

No entanto, a segunda etiqueta direcional mais comum por hora do dia varia com a hora do dia, como podemos ver aqui:

##R

> dt[, nth.pos(direção, 2), local.hora][pedido(local.hora)]

local.hora V1

1: 0 Quarenta e cinco para cima

2:1 FortyFiveUp

3:2 Quarenta e cinco para cima

4:3 Quarenta e cinco para baixo

5:4 Quarenta e cinco para baixo

6:5 Quarenta e cinco para cima

7:6 Quarenta e cinco para baixo

8:7 Quarenta e cinco para baixo

9:8 Quarenta e cinco para baixo

10:9 Quarenta e cinco para cima

11h10 Quarenta e cinco para cima

12:11 Quarenta e cinco para baixo

13:12 Quarenta e cinco para baixo

14:13 Quarenta e cinco para baixo

15:14 Quarenta e cinco para baixo

16:15 Quarenta e cinco para baixo

17:16 Quarenta e cinco para cima

18:17 Quarenta e cinco para cima

19h18 Quarenta e cinco para baixo

20:19 Quarenta e cinco para cima

21:20 Quarenta e cinco para cima

22:21 Quarenta e cinco para cima

23: 22 Quarenta e cinco para cima

24: 23 Quarenta e cinco para baixo

local.hora V1

É interessante cruzar esses rótulos com os gráficos de dados intradiários que incluímos na seção anterior. Os

resultados coincidem? (A resposta fica como exercício para o leitor.)

A seguir abordamos um rótulo numérico que pode ser preditivo, nomeadamente a variância das medidas de glicemia na janela de tempo mais recente. Calculamos o desvio padrão para uma curta janela de lookback.

Para fazer isso, devemos levar em conta a natureza não contígua dos dados. Não queremos calcular desvios padrão de pontos de dados que não são adjacentes no tempo, mas apenas adjacentes na tabela de dados. Calcularemos os deltas de tempo entre o início e o fim da janela de destino para garantir que estejam na escala de tempo correta, mas precisamos ter cuidado ao fazer isso. Aqui está um exemplo de como podemos facilmente dar errado:

##R

> ## PRIMEIRO não faça isso da maneira errada

> ## Nota: cuidado ao calcular diferenças de fuso horário ingenuamente!

> as.numeric(dt$timestamp[10] - dt$timestamp[1])

[1] 40

> as.numeric(dt$timestamp[1000] - dt$timestamp[1])

[1] 11.69274

> dt$timestamp[1000] - dt$timestamp[1]

Diferença horária de 11,69274 dias

É possível que as diferenças de horário sejam retornadas em unidades diferentes, o que tornaria os resultados do código anterior absurdos. A maneira correta de calcular um delta de tempo, bem como a maneira como usamos esse delta de tempo para determinar a validade de um cálculo de desvio padrão para uma determinada linha, é a seguinte:

##R

> dt[, delta.t := as.numeric(difftime(timestamp, shift(timestamp, 6),

> unidades = 'minutos'))]

> dt[, válido.sd := !is.na(delta.t) & delta.t < 31]

> dt[, .N, válido.sd]

válido.sd N

1: **FALSO** 1838

2: **VERDADEIRO** 21435

Depois de rotularmos as linhas para as quais um cálculo de desvio padrão é apropriado, executamos os cálculos. Calculamos em todas as linhas e planejamos substituir os valores inválidos por uma média de coluna como uma forma simples de valor ausente

imputação:

##R

> dt[, sd.window := 0]

> para (i em 7:nrow(dt)) {

> dt[i,]$sd.window = sd(dt[(i-6):i]$sgv)

> }

> ## imputaremos os dados faltantes para os casos SD inválidos

> ## preenchendo a média da população (alerta LOOKAHEAD, mas estamos atentos)

> imputado.val = média(dt[valid.sd == **TRUE**]$sd.window)

> dt[valid.sd == **FALSE** , sd.window:= imputado.val]

A seguir, configuramos uma coluna para estabelecer o verdadeiro valor que devemos almejar para nossa previsão.

Optamos por prever os valores de glicose no sangue com 30 minutos de antecedência, o que seria tempo suficiente para avisar alguém com diabetes se fosse previsto um nível de açúcar no sangue perigosamente alto ou baixo. Colocamos o valor da previsão alvo em uma coluna chamada alvo. Também criamos outra coluna, pred.valid, que indica se os pontos de dados anteriores ao momento em que queremos fazer uma previsão estão suficientemente completos (ou seja, amostrados regularmente nos 30 minutos anteriores em incrementos de 5 minutos):

##R

> ## agora também precisamos preencher nosso valor y

> ## o alvo real

> ## isso também exigirá que verifiquemos a validade como quando

> ## computação sd devido à sessão de dados contínua e não temporal

> ## na mesma tabela de dados. vamos tentar prever 30 minutos à frente

> ## (previsões mais curtas são mais fáceis)

>

> *## mudamos em 6 porque estamos amostrando a cada 5 minutos*

> dt[, pred.delta.t := as.numeric(difftime(shift(timestamp, 6,

> tipo = "lead"),

> carimbo de data/hora,

> unidades = 'minutos'))]

> dt[, pred.valid := !is.na(pred.delta.t) & pred.delta.t < 31]

>

> dt[, alvo:= 0]

> para (i em 1:nrow(dt)) {

> dt[i,]$alvo = dt[i + 6]$sgv

> }

Verificamos nosso trabalho para ver se ele produz resultados sensatos:

##R

> *## agora devemos verificar nosso trabalho*

> eu = 300

> dt[i + (-12:10), .(timestamp, sgv, alvo, pred.valid)]

carimbo de data e hora alvo sgv pred.valid

1: 19/02/2015 16:15:05 146 158 **VERDADEIRO**

2: 19/02/2015 16:20:05 150 158 **VERDADEIRO**

3: 19/02/2015 16:25:05 154 151 **FALSO**

4: 19/02/2015 16:30:05 157 146 **FALSO**

5: 19/02/2015 16:35:05 160 144 **FALSO**

6: 19/02/2015 16:40:05 161 143 **FALSO**

7: 19/02/2015 16:45:05 158 144 **FALSO**

8: 19/02/2015 16:50:05 158 145 **FALSO**

9: 19/02/2015 17:00:05 151 149 **VERDADEIRO**

10: 19/02/2015 17:05:05 146 153 **VERDADEIRO**

11: 19/02/2015 17:10:05 144 154 **VERDADEIRO**

12: 19/02/2015 17:15:05 143 155 **VERDADEIRO**

13: 19/02/2015 17:20:05 144 157 **VERDADEIRO**

14: 19/02/2015 17:25:05 145 158 **VERDADEIRO**

15: 19/02/2015 17:30:05 149 159 **VERDADEIRO**

16: 19/02/2015 17:35:05 153 161 **VERDADEIRO**

17: 19/02/2015 17:40:05 154 164 **VERDADEIRO**

18: 19/02/2015 17:45:05 155 166 **VERDADEIRO**

19: 19/02/2015 17:50:05 157 168 **VERDADEIRO**

20: 19/02/2015 17:55:05 158 170 **VERDADEIRO**

21: 19/02/2015 18:00:04 159 172 **VERDADEIRO**

22: 19/02/2015 18:05:04 161 153 **FALSO**

23: 19/02/2015 18:10:04 164 149 **FALSO**

carimbo de data e hora alvo sgv pred.valid

Dê uma olhada cuidadosa nesta saída. Algo nele deveria fazer você questionar se fomos muito duros na avaliação da "validade" para calcular o desvio padrão de uma janela recente nos dados de glicose no sangue. Como um exercício independente, veja se você consegue identificar esses pontos e pense em como você poderia retrabalhar a rotulagem pred.valid para ser mais corretamente inclusiva.

Agora temos uma ampla gama de recursos e um valor alvo para usar no treinamento de qualquer modelo que usaremos para gerar previsões, mas ainda não terminamos de gerar recursos. Deveríamos simplificar algumas das características temporais que já geramos para diminuir as complicações do nosso modelo. Por exemplo, em vez de a hora local ser uma entrada de 23 binários (um para cada hora do dia menos um), deveríamos diminuir o número de categorias de "hora". Fazemos isso assim:

##R

> ## Vamos dividir o dia em trimestres em vez de

> ## em segmentos de 24 horas. Fazemos isso com base em uma noção

> ## de um dia 'típico'

> dt[, dia.q.1 := entre(local.hora, 5, 10,99)]

> dt[, dia.q.2 := entre(local.hora, 11, 16,99)]

> dt[, dia.q.3 := entre(local.hora, 17, 22,99)]

> dt[, dia.q.4 := !dia.q.1 & !dia.q.2 & !dia.q.3]

Também simplificamos os dados do mês para um conjunto mais simples de categorias:

##R

> *## vamos ter um rótulo "inverno/não inverno" em vez de*

> *## um rótulo de mês. esta decisão baseia-se em parte*

> *## propagação temporal de nossos dados*

> dt[, é.inverno := as.numeric(mês) <4]

Finalmente, para usar a coluna de direção, precisamos codificar esse valor em um ponto, assim como fizemos nos diferentes horários do dia. Também limpamos alguns recursos rotulados de forma inconsistente ("NOT COMPUTABLE" versus "NOT_COMPUTABLE"):



##R

> *## também precisamos codificar um recurso de direção*

> *## e limpe um pouco esses dados*

> dt[direção == "NÃO COMPUTÁVEL", direção := "NÃO_COMPUTABLE"]

> dir.nomes = personagem()

> for (nm em único(dt$direção)) {

> novo.col = paste0("dir_", nm)

> dir.nomes = c(dir.nomes, nova.col)

> dt[, eval(parse(text = paste0(new.col, " :=

(direção == '", nm, "')")))]

> }

Agora temos recursos relevantes codificados a quente e outros simplificados, então estamos prontos para direcionar esses recursos em um modelo.

Ajustando um modelo

Finalmente chegamos à parte divertida da análise de séries temporais: fazer uma previsão.

Quanto tempo você deve gastar modelando?

Como você viu com os dois modelos do mundo real deste capítulo,

os dados do mundo real são confusos e cada vez que são limpos, precisam ser

feito isso com conhecimento de domínio e bom senso. Não há

modelo geral. No entanto, você deve sempre proceder com cuidado

e sem pressa para ajustar um modelo. Queremos apenas ajustar modelos

quando estivermos confiantes de que não estamos colocando lixo neles!

Nossa primeira tarefa é criar conjuntos de dados de treinamento e teste: 13

##R

> ## precisamos configurar dados de treinamento e teste

> ## para testes, não queremos que todos os testes venham

> ## no final do período de teste, já que levantamos algumas hipóteses de comportamento

```
> ## pode ser sazonal, vamos fazer os dados de teste

> ## os dados finais de ambas as "temporadas"

> inverno.data <- dt[is.winter == TRUE ]

> train.row.cutoff <- round(nrow(winter.data) * .9)

> train.winter <- inverno.data[1:train.row.cutoff]

> test.winter <- inverno.data[(train.row.cutoff + 1):
nrow(winter.data)]

>

> spring.data <- dt[is.winter == FALSE ]

> train.row.cutoff <- round(nrow(spring.data) * .9)
```

13 Na prática, e com um conjunto de dados maior, você também desejaria reservar um conjunto de testes de validação, que seria um subconjunto especial de seus dados de treinamento semelhante a um teste, sem poluir seus dados de teste reais. Assim como os dados de teste geralmente devem ser os dados temporalmente mais recentes (para evitar que informações futuras vazem para trás, por exemplo, antecipadas), o conjunto de dados de validação em tais casos deve vir no final do período de treinamento. Considere configurar isso sozinho para explorar os hiperparâmetros do modelo.


```
> train.spring <- spring.data[1:train.row.cutoff]

> test.spring <- spring.data[(train.row.cutoff + 1):
nrow(spring.data)]

>

> train.data <- rbindlist(list(train.winter, train.spring))

> test.data <- rbindlist(list(test.winter, test.spring))
```

>

> *## agora inclui apenas as colunas que deveríamos usar*

> *## valores categóricos: valid.sd, day.q.1, day.q.2, day.q.3, is.winter*

> *## mais todos os nomes de colunas 'dir_'*

> col.names <- c(dir.names, "sgv", "sd.window", "valid.sd",

> "dia.q.1", "dia.q.2", "dia.q.3", "é.inverno")

>

> train.X <- train.data[, col.names, with = **FALSE**]

> trem.Y <- trem.data$target

>

> teste.X <- teste.data[, col.names, com = **FALSE**]

> teste.Y <- teste.data$target

Consistente com muitos trabalhos de ponta, escolhemos árvores com gradiente aumentado XGBoost como nosso modelo. Em publicações recentes, estes atingiram ou excederam os requisitos do estado da arte.

recursos para previsão de glicose em alguns casos de uso:14

##R

> modelo <- xgboost(dados = as.matrix(train.X), rótulo = train.Y,

> profundidade máxima = 2,

> eta = 1,

>nthread = 2,

> n rodadas = 250,

> objetivo = "reg:linear")

> y.pred <- prever(modelo, as.matriz(teste.X))

Podemos então inspecionar os resultados desta previsão (ver Figura 13-14). Também olhamos

nossa previsão para um dia específico (veja a Figura 13-15):

##R

> *## agora vamos ver um dia específico*

> test.data[data < as.Date("2015/03/17")]

> par(mfrow = c(1, 1))

> eu <- 1

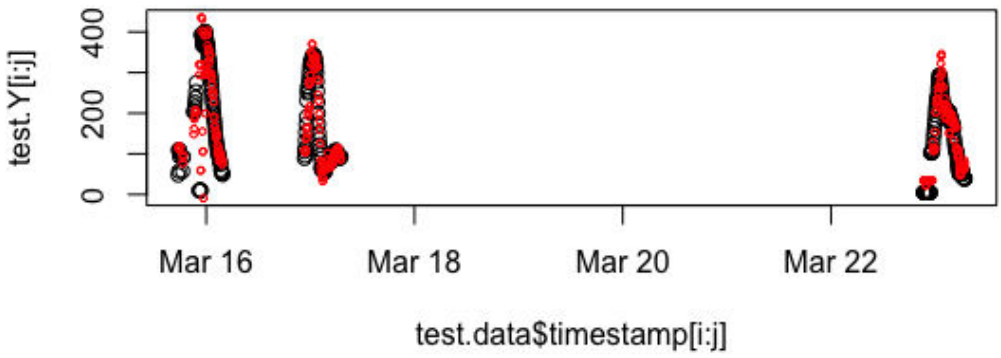
> j <- 102

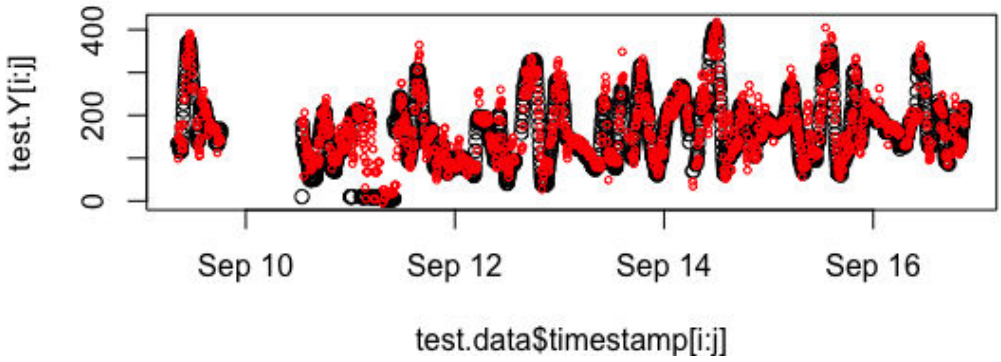
> ylim <- intervalo(teste.Y[i:j], y.pred[i:j])

> plot(teste.dados$timestamp[i:j], teste.Y[i:j], ylim = ylim)

> pontos (test.data$timestamp[i:j], y.pred[i:j], cex = 0,5, col = 2)

14 Não ajustamos os hiperparâmetros aqui (o que exigiria um conjunto de dados de validação), mas isso é algo que você deve fazer com o XGBoost depois de definir um pipeline de modelagem geral.


test.Y[i:j]
0
200
400
Mar 16
Mar 18
Mar 20
Mar 22
test.data$timestamp[i:j]


test.Y[i:j]
0
200
400
Sep 10
Sep 12
Sep 14
Sep 16
test.data$timestamp[i:j]

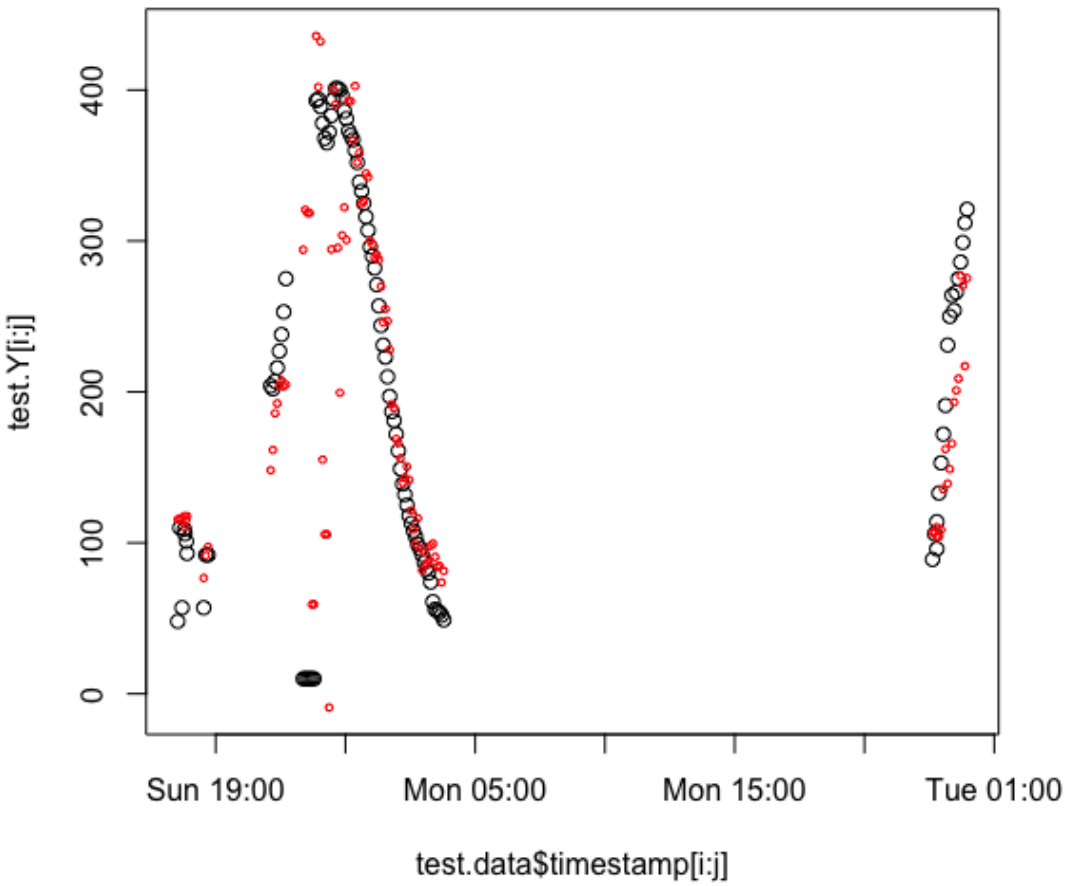


Figura 13-14. Nesta grande escala, as nossas previsões parecem boas, mas é difícil julgar diminuído até agora. Não é assim que alguém que usasse a previsão experimentaria isso.

Figura 13-15. Examinar dias individuais de cada vez oferece uma perspectiva melhor sobre quão bem nosso algoritmo de previsão funciona.

As previsões parecem razoáveis, mas este gráfico sugere que também pode haver problemas com os dados subjacentes. A bolha por volta da meia-noite entre domingo e segunda-feira parece irrealisticamente baixa, especialmente porque os pontos vizinhos não são baixos. É mais provável que o dispositivo tenha funcionado mal do que alguém tenha experimentado um nível tão drástico, mas de curta duração, de glicose no sangue. Poderíamos considerar mais limpeza de dados ou talvez rótulos

adicionais para indicar que pontos específicos parecem suspeitos. 15 É provavelmente um sinal de sobreajuste o facto de o nosso modelo prever estes pontos de dados de níveis baixos de glicose no sangue prováveis e inválidos. Queremos que nosso modelo preveja os valores reais de glicose no sangue e não o mau funcionamento do dispositivo.

Neste ponto, devemos considerar os objetivos do nosso algoritmo. Se quisermos apenas mostrar que podemos fazer um trabalho razoavelmente bom na previsão da glicemia com meia hora de antecedência – sem sequer o benefício de saber o que alguém está a comer ou quando está a fazer exercício – já podemos dizer "missão cumprida". Isso já é bastante – e é impressionante que possamos fazer previsões razoáveis sem ter informações relevantes, como informações sobre refeições e exercícios.

No entanto, o nosso objectivo principal é uma previsão que alerte as pessoas sobre o perigo – isto é, quando o nível de açúcar no sangue ficará demasiado baixo ou demasiado alto. Vamos nos concentrar nesses pontos de dados enquanto consideramos como nossas previsões se comparam aos valores medidos.

É importante ressaltar que plotamos o valor alto e baixo da glicose no sangue separadamente. Se os plotássemos juntos, veríamos uma correlação aparentemente alta, criando uma correlação artificialmente

distribuído em forma de haltere de pontos altos e baixos (veja a Figura 13-16):

##R

```
> par(mfrow = c(2, 1))

> alto.idx = qual(teste.Y > 300)

> plot(teste.Y[high.idx], y.pred[high.idx], xlim = c(200, 400),

> lim = c(200, 400))

> cor(teste.Y[alto.idx], y.pred[alto.idx])
```

[1] 0,3304997

>

> low.idx = which((teste.Y < 60 & teste.Y > 10))

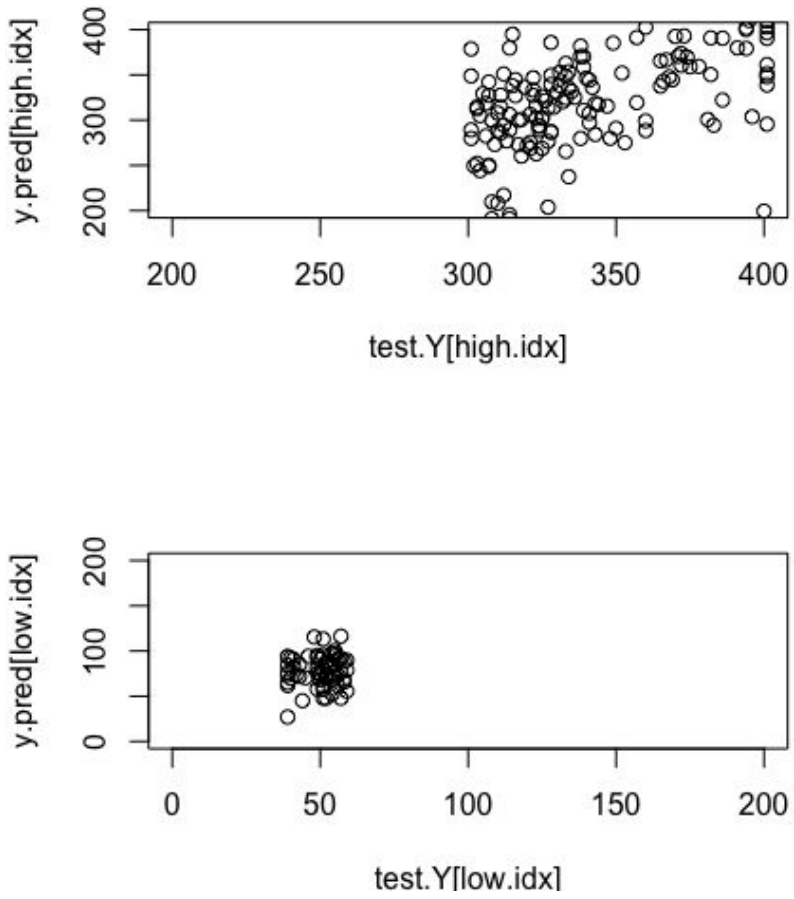
> plot(teste.Y[low.idx], y.pred[low.idx], xlim = c(0, 200),

> lim = c(0, 200))

> cor(teste.Y[low.idx], y.pred[low.idx])

[1] 0,08747175

Esses gráficos colocam o desempenho do nosso modelo sob uma luz mais dura. Particularmente preocupante é que o modelo não funciona muito bem com níveis baixos de açúcar no sangue. Isso pode ocorrer porque as ocorrências perigosamente baixas de açúcar no sangue são incidentes raros, de modo que o modelo não tem muita oportunidade de treinar sobre elas. Deveríamos considerar o aumento de dados ou 15 Uma ideia seria aplicar a detecção de valores discrepantes de maneira consciente de séries temporais para observar que esses pontos não parecem corresponder à tendência geral para aquela janela de tempo. Também poderíamos usar física, química e biologia básicas em alguns casos para dizer que certas quedas simplesmente não são possíveis.



reamostragem para encontrar maneiras de tornar esse fenômeno mais saliente para nosso modelo. Também podemos modificar nossa função de perda para pesar mais os exemplos já em nossos dados.

Figura 13-16. Gráfico das previsões e valores reais entre si individualmente em cada extremidade do extremo. No topo, plotamos os valores uns contra os outros para valores altos valores medidos de glicose no sangue e na parte inferior para níveis baixos de glicose no sangue medidos valores.

Vimos aqui um caso em que as previsões de um modelo inicial parecem respeitáveis, mas podem não cumprir o propósito mais essencial do modelo. Neste caso, podemos prever tendências gerais de glicemia, mas deveríamos trabalhar mais para prever níveis elevados e baixos de açúcar no sangue, com especial ênfase nos níveis baixos de açúcar no sangue que ameaçam a vida.

Neste caso, tal como no caso da previsão da gripe, vimos que o bom senso é essencial para modelar séries temporais. Não podemos limpar dados cegamente ou preparar recursos sem pensar no que eles significam no contexto. Da mesma forma, quanto mais sabemos sobre o domínio das séries temporais, como a compreensão da importância da hora do dia para os níveis de açúcar no sangue em humanos, mais bem equipados estaremos para limpar dados, preparar características e examinar as nossas previsões para sucessos importantes ou casos de falha.

Mais recursos

Vasileios Lampos et al., "Avanços na previsão de taxas de doenças semelhantes à gripe usando

Registros de consulta de pesquisa," Relatórios Científicos 5, não. 12760 (2015), https://perma.cc/NQ6B-RUXF.

Este artigo da Nature Communications de 2015 é altamente acessível e um bom

introdução à previsão da gripe, bem como uma peça histórica que mostra como a previsão e a previsão foram revolucionadas pelo big data e pelas mídias sociais em tempo real e pelas entradas de dados da Internet. Os autores trabalham tanto com um modelo linear tradicional quanto com um modelo não linear inovador para comparar diferentes abordagens e

mostram a utilidade de modelos não lineares em um sistema tão complexo como as taxas sazonais de doenças infecciosas.

David Farrow, "Modelando o passado, o presente e o futuro da gripe", tese de doutorado, Departamento de Biologia Computacional, Escola de Ciência da Computação, Carnegie Mellon Universidade, 2016, https://perma.cc/96CZ-5SX2.

Esta tese detalha um pouco da teoria e prática da "fusão de sensores" com o propósito de integrar fontes de mídia social de diversas granularidades geográficas e temporais para

produzir uma das principais previsões de gripe do grupo Delphi. Ele fornece uma visão geral extremamente aprofundada da previsão da gripe, desde a virologia até a dinâmica populacional e experimentos de ciência de dados sobre como o processamento de dados afeta as previsões.

Rob J. Hyndman, "Regressão Dinâmica", notas de aula, sd, https://perma.cc/5TPY-

PYZS.

Estas notas de aula oferecem exemplos práticos de como usar a regressão dinâmica para complementar os modelos tradicionais de previsão estatística com uma série de modelos alternativos para sazonalidade quando não se espera que o SARIMA seja um bom ajuste - geralmente

porque a periodicidade é muito complexa ou os períodos são muito longos em relação à quantidade de dados ou recursos computacionais disponíveis.

Mais recursos | 401



CAPÍTULO 14

Aplicações Financeiras

Os mercados financeiros são os avôs de todos os dados de séries temporais. Se você pagar por dados comerciais proprietários em uma bolsa de alta tecnologia, poderá receber inundações de dados do tamanho de terabytes que podem levar dias para serem processados, mesmo com computação de alto desempenho e processamento embaraçosamente paralelo.

Os traders de alta frequência estão entre os membros mais novos e infames da comunidade financeira e negociam com base em informações e insights resultantes da análise de

séries temporais no nível de microssegundos. Por outro lado, as empresas comerciais tradicionais

— olhando para séries temporais de longo prazo ao longo de horas, dias ou mesmo meses — continuam a ter sucesso nos mercados, mostrando que a análise de séries temporais para dados financeiros pode ser conduzida de uma infinidade de maneiras bem-sucedidas e em escalas de tempo que abrangem muitas ordens de grandeza, de milissegundos a meses.

Embaraçosamente paralelo descreve tarefas de processamento de dados onde o

os resultados do processamento de um segmento de dados não dependem de forma alguma

nos valores de outro segmento de dados. Nesses casos é constrangedor

extremamente fácil converter tarefas de análise de dados para execução em paralelo, em vez

do que em sequência para aproveitar as vantagens de multicore ou multimáquina

opções de computação.

Considere, por exemplo, a tarefa de calcular a média diária de

retornos minuto a minuto de uma determinada ação. Cada dia pode ser

tratados separadamente e em paralelo. Em contraste, computando a exposição

A média móvel ponderada da volatilidade diária não é embaraçosa.

extremamente paralelo porque o valor para um determinado dia depende de

`

os valores dos dias anteriores. As vezes, tarefas que não são

embaraçosamente paralelo ainda pode ser executado parcialmente em paralelo, mas

depende dos detalhes.

403

Aqui, trabalharemos com um exemplo clássico de análise de série temporal para diversão e lucro: prever os retornos das ações de amanhã para o S&P 500.

Obtenção e exploração de dados financeiros

Pode ser extremamente difícil obter dados financeiros se você estiver buscando um determinado produto ou resolução temporal. Nesses casos, geralmente você precisa comprar dados.

Mas os preços históricos das ações estão amplamente disponíveis em uma variedade de serviços, incluindo:

• Yahoo Finanças. Embora o Yahoo tenha descontinuado o serviço de sua API de dados históricos, 1

dados históricos diários estão disponíveis para download .

• Empresas mais recentes, como AlphaVantage e Quandl, oferecem uma combinação de informações históricas e de preços em tempo real para dados de ações.

Limitamos nossa análise aos dados diários de preços de ações disponíveis gratuitamente no Yahoo. Baixamos dados para o S&P 500 abrangendo datas de 1990 a 2019. No código a seguir, vemos quais colunas estão disponíveis no conjunto de dados baixado e traçamos o gráfico diário

preço de fechamento para começar a explorar nossos dados (veja a Figura 14-1):

Pitão

```
>>> df = pd.read_csv("sp500.csv")

>>> df.head()

>>> df.tail()

>>> df.index = df.Data

>>> df.Close.plot()
```

Podemos ver que os valores são notavelmente diferentes no início e no final do período coberto pelos arquivos CSV. A mudança nos valores é ainda mais aparente quando traçamos a série temporal completa para o preço de fechamento (Figura 14-2) do que quando olhamos para amostras do quadro de dados.

Uma olhada na Figura 14-2 mostra que a série temporal não é estacionária. Também vemos isso

pode haver diferentes “regimes”. Por razões claramente visíveis no gráfico, os analistas financeiros estão interessados em desenvolver modelos que identifiquem mudanças de regime nos preços das acções. Parece haver regimes diferentes, mesmo que não tenhamos uma definição exacta de onde

um termina e outro começa.2

1 Aliás, isso criou bastante código que não funcionava nas comunidades R e Python.

2 Note-se que o S&P 500 também é complicado porque é um composto de muitas ações diferentes como inputs, com ponderações ajustadas periodicamente e com um fator que é inteiramente proprietário também utilizado para dividir a média ponderada das ações. Por esta razão, um forte conhecimento do domínio e uma compreensão de como diferentes ações tomadas pelas empresas podem afetar os preços das suas ações e as suas ponderações no S&P 500 também seriam importantes para compreender melhor o comportamento de longo prazo que vemos aqui.

```
In [9]: df.head()
```

Out[9]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	1990-02-13	330.079987	331.609985	327.920013	331.019989	331.019989	144490000
1	1990-02-14	331.019989	333.200012	330.640015	332.010010	332.010010	138530000
2	1990-02-15	332.010010	335.209991	331.609985	334.890015	334.890015	174620000
3	1990-02-16	334.890015	335.640015	332.420013	332.720001	332.720001	166840000
4	1990-02-20	332.720001	332.720001	326.260010	327.989990	327.989990	147300000

```
In [75]: df.tail()
```

Out[75]:

	Date	Open	High	Low	Close	Adj Close	Volume
7301	2019-02-06	2735.050049	2738.080078	2724.149902	2731.610107	2731.610107	3472690000
7302	2019-02-07	2717.530029	2719.320068	2687.260010	2706.050049	2706.050049	4099490000
7303	2019-02-08	2692.360107	2708.070068	2681.830078	2707.879883	2707.879883	3622330000
7304	2019-02-11	2712.399902	2718.050049	2703.790039	2709.800049	2709.800049	3361970000
7305	2019-02-12	2722.610107	2748.189941	2722.610107	2744.729980	2744.729980	3827770000

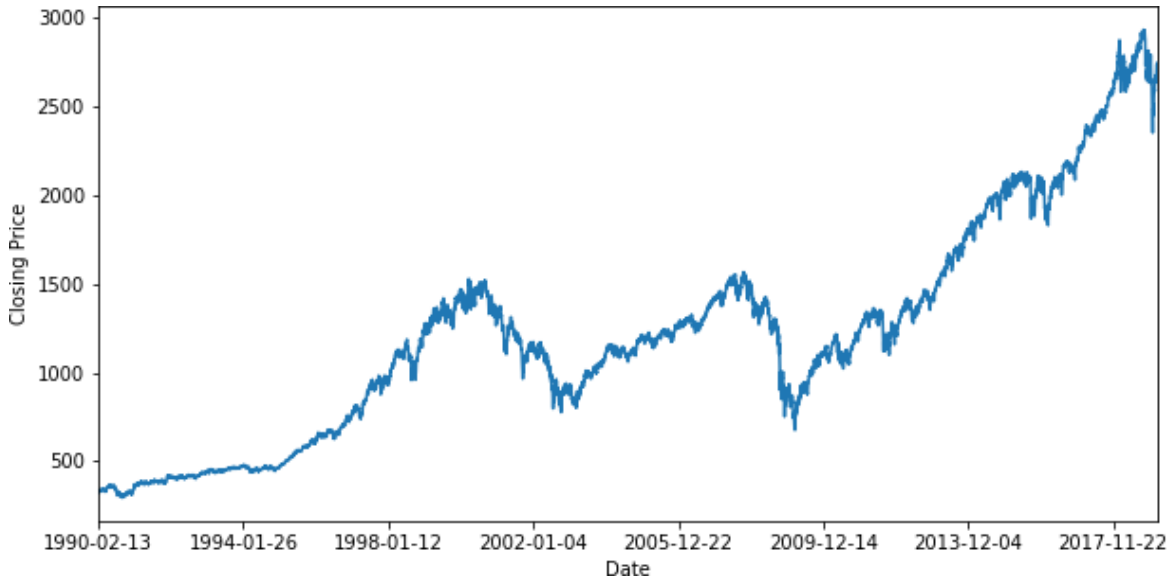


Figura 14-1. Dados brutos do início e do final do arquivo CSV. Note-se que os valores mudam visivelmente entre 1990 e 2019 – o que não surpreende ninguém familiarizado com a história dos mercados financeiros dos EUA.

Figura 14-2. O preço de fechamento diário do S&P 500 não é uma série temporal estacionária.

O potencial para pontos de mudança e diferentes regimes sugere que poderia ser uma boa ideia dividir o conjunto de dados em diferentes subconjuntos de dados para serem modelados separadamente. No entanto, queremos manter todos os dados juntos porque os dados diários não produzem muitos pontos de dados em algumas décadas; precisamos manter todos os dados que pudermos. Consideramos **a obtenção e exploração de dados financeiros | 405**

se podemos justificar a manutenção de todos estes dados juntos se estamos interessados apenas nas previsões para o dia seguinte.

Podemos considerar se a normalização dos dados pode tornar comparáveis os dados de diferentes períodos de tempo. Vamos dar uma olhada nos preços de fechamento escalonados de uma semana para três

diferentes décadas dentro da série temporal (Figura 14-3):

```
## Pitão

>>> ## escolha três semanas (segunda a sexta) de anos diferentes

>>> ## escalar preços de fechamento para cada dia pelo preço médio de fechamento da semana

>>> ## 1990

>>> vals = df['1990-05-07':'1990-05-11'].Close.values

>>> valor_médio = np.mean(vals)

>>> plt.plot([1, 2, 3, 4, 5], vals/valor_médio)

>>> plt.xticks([1, 2, 3, 4, 5],
```

```
>>> rótulos = ['Segunda-feira', 'Terça-feira', 'Quarta-feira',
'Quinta-feira', 'Sexta-feira'])

>>>

>>> ## 2000

>>> vals = df['2000-05-08':'2000-05-12'].Close.values

>>> valor_médio = np.mean(vals)

>>> plt.plot([1, 2, 3, 4, 5], vals/valor_médio)

>>> plt.xticks([1, 2, 3, 4, 5],

>>> rótulos = ['Segunda-feira', 'Terça-feira', 'Quarta-feira',
'Quinta-feira', 'Sexta-feira'])

>>>

>>> ## 2018

>>> vals = df['2018-05-07':'2018-05-11'].Close.values

>>> valor_médio = np.mean(vals)

>>> plt.plot([1, 2, 3, 4, 5], vals/valor_médio)

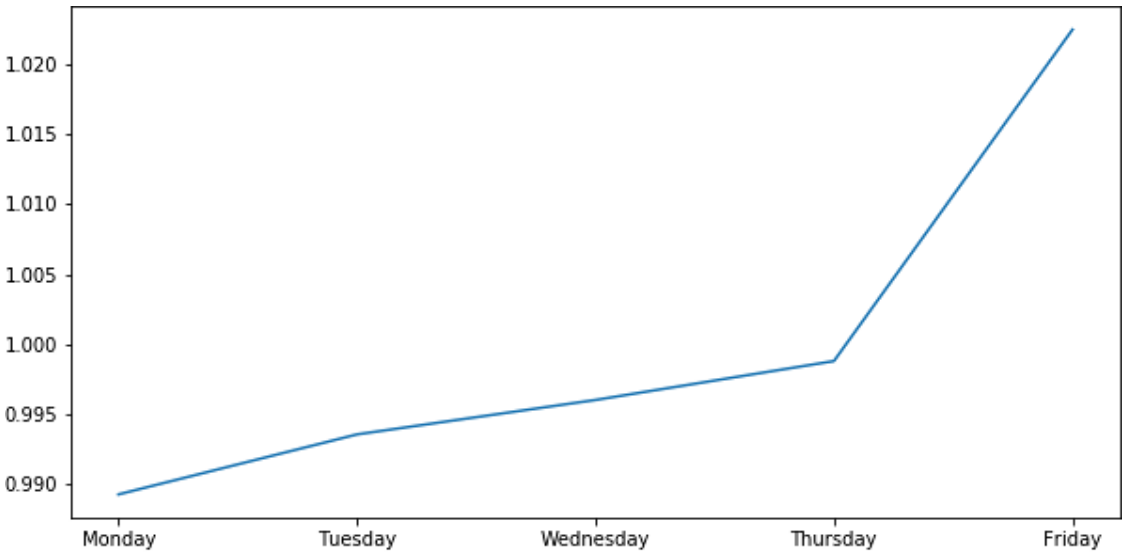
>>> plt.xticks([1, 2, 3, 4, 5],

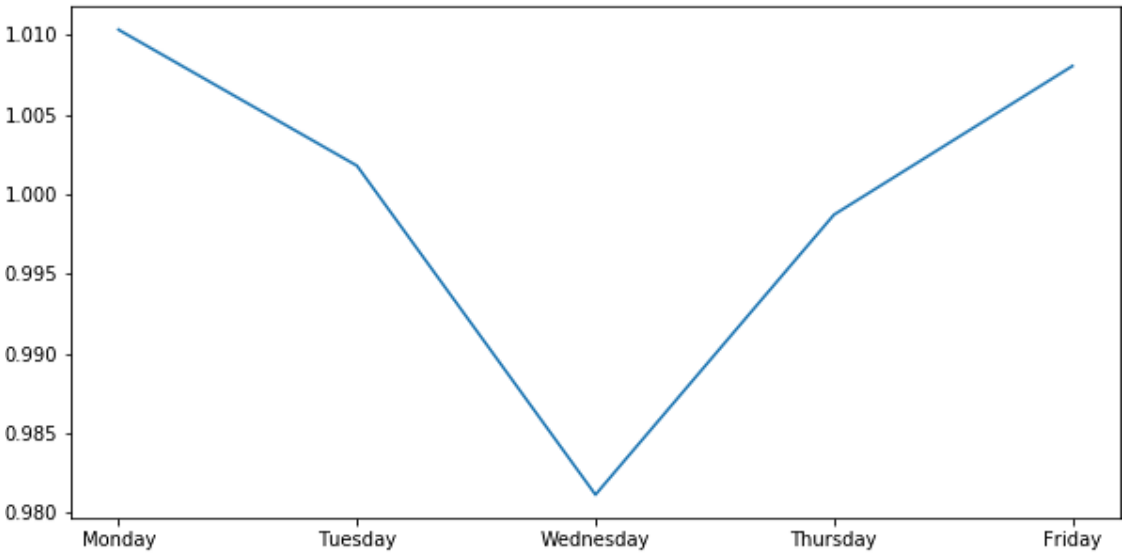
>>> rótulos = ['segunda-feira', 'terça-feira', 'quarta-feira',
'quinta-feira', 'sexta-feira']) Traçamos os preços de
fechamento ao longo de três semanas em três décadas
diferentes, cada um dimensionado pela média daquela
semana de acordo com o código anterior . As mudanças
percentuais relativas de um dia para o outro ao longo de
uma semana parecem praticamente as mesmas ao longo das
décadas.

Essas parcelas são promissoras. Embora os valores médios
e a variância dos preços de fecho tenham mudado
substancialmente ao longo do tempo, os gráficos sugerem
um comportamento semelhante ao longo do tempo, uma vez
```

normalizados os dados para o seu valor médio para uma determinada década.

Diante disso, consideraremos a seguir se podemos encontrar uma maneira de tornar todos os dados durante todo o período semelhantes o suficiente para treinar de forma significativa com um modelo. Queremos saber se existe uma maneira de transformar os dados que seja financeiramente significativo, mas que também torne os dados comparáveis ao longo de todo o período de tempo.


1.020
1.015
1.010
1.005
1.000
0.995
0.990
Monday
Tuesday
Wednesday
Thursday
Friday


1.010
1.005
1.000
0.995
0.990
0.985
0.980
Monday
Tuesday
Wednesday
Thursday
Friday

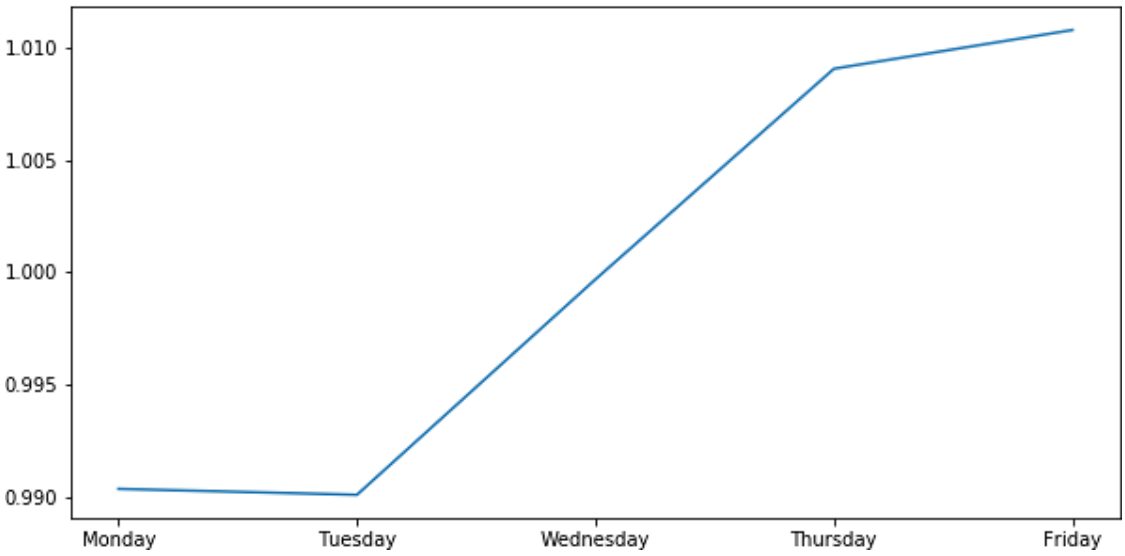

1.010
1.005
1.000
0.995
0.990
Monday
Tuesday
Wednesday
Thursday
Friday

Figura 14-3. Preços médios de fechamento em escala para uma semana em maio de 1990, 2000 e 2018.

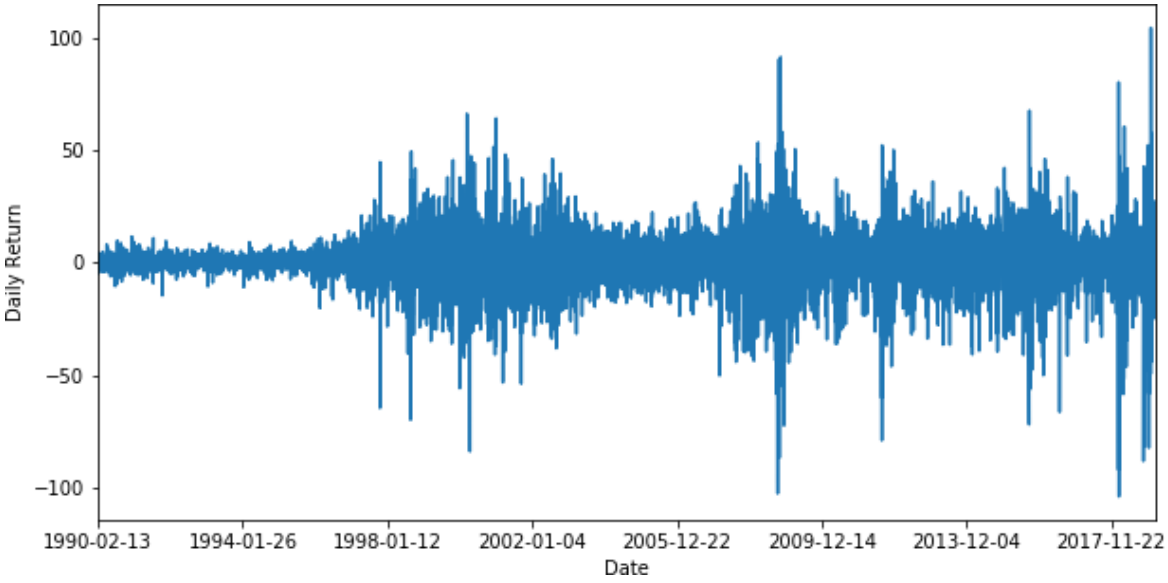
Calculamos o retorno diário, ou seja, a variação do preço do início ao fim do

[cada dia de negociação (ver Figura 14-4):]

Pitão

>>> df['Retornar'] = df.Fechar - df.Open

>>> df.Return.plot()



[Como podemos ver na Figura 14-4, isto por si só não é suficiente para tornar os dados comparáveis.]

Também teremos que encontrar uma maneira de normalizar os dados sem antecipação, para que os valores que usamos para entradas e saídas em nosso modelo sejam mais uniformes ao longo do período de interesse. Veremos como fazer isso na próxima seção.

Figura 14-4. Os retornos diários mostram uma média próxima de zero ao longo do tempo, mas a variação dos retornos diários retorna alterações marcadamente em

diferentes períodos de tempo. E um comportamento como este que tem modelos inspirados como o GARCH, que foi brevemente discutido no Capítulo 6.

Os mercados financeiros como um passeio aleatório

O passeio aleatório é um bom modelo inicial para séries temporais financeiras; na verdade, os preços das ações são frequentemente citados como o exemplo por excelência de um passeio aleatório que ocorre naturalmente.

Há dois insights importantes que devemos ter em mente ao analisarmos nossos dados.

Primeiro, muitas mudanças de regime podem ser apenas um efeito dos contornos naturais de um passeio aleatório. Se você não acredita em mim, considere alguns dos exemplos a seguir, que foram

gerado rapidamente em R (veja a Figura 14-5):

##R

> ## usou cada uma dessas sementes para produzir as parcelas

> ## set.seed(1)

> ## set.seed(100)

> ## set.seed(30)

> N <- 10.000

> x <- cumsum(amostra(c(-1, 1), N, **TRUE**))

> enredo (x)

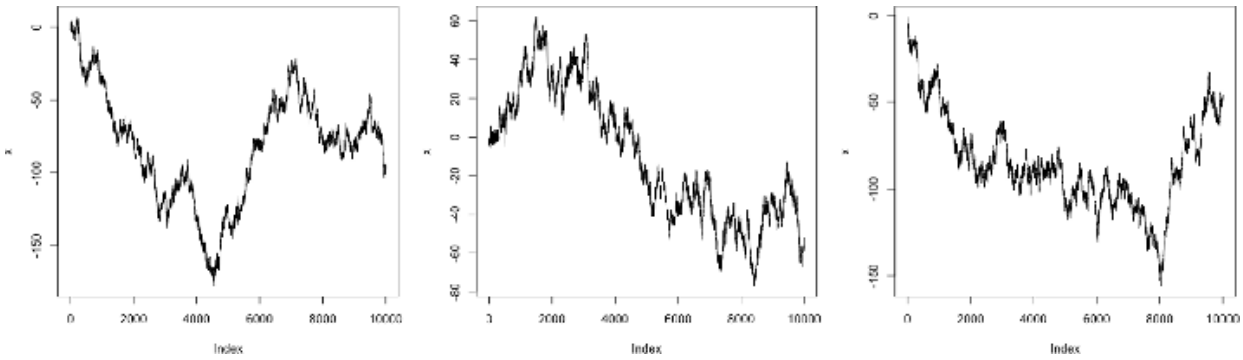


Figura 14-5. O resultado de uma variedade de passeios aleatórios, todos produzidos com diferentes sementes do código simples anterior.

Em segundo lugar, como os passeios aleatórios são um ajuste natural para dados de ações, o nosso modelo nulo ou mais básico, contra o qual podemos começar a comparar modelos mais complicados, seria aquele em que prevemos o preço de amanhã como sendo o preço de hoje, consistente. com a nossa compreensão do mercado de ações como um passeio aleatório. Nesse caso, esperaríamos uma correlação muito alta:

##R

> cor(x, shift(x), use = "complete.obs")

[1] 0,9996601

Por outro lado, se diferissemos as séries temporais, de modo que o valor em cada intervalo de tempo representasse a mudança no valor da série temporal de um intervalo de tempo para outro, a correlação desapareceria porque qualquer correlação teria sido devida à tendência. em vez da capacidade preditiva real de um modelo tão simples:

##R

> cor(diff(x), shift(diff(x)), use = "complete.obs")

[1] -0,005288012

Portanto, em finanças, o fato de você produzir um modelo com correlação alta ou baixa pode ter muito a ver com o

tipo de dados que você está modelando. Se você modelar os retornos (o que você deveria fazer) em vez dos preços das ações (um erro comum até mesmo na indústria), você terá correlações que parecem baixas, mas são muito mais propensas a serem preditivas no mundo real, em vez de modelos que prometem o mundo. mas não entregue na produção. É por isso que trabalhamos para modelar os retornos em vez do preço das ações no próximo exemplo.



Pré-processamento de dados financeiros para aprendizado profundo

Nosso pré-processamento de dados será feito em três etapas:

1. Formaremos novas quantidades de interesse economicamente significativas a partir dos insumos brutos.

2. Calcularemos uma média móvel ponderada exponencialmente e a variância das quantidades de interesse para que possamos escalá-las sem antecipação.

3. Empacotaremos nossos resultados em um formato apropriado para o modelo recorrente de aprendizagem profunda que usaremos para ajustar os dados.

A série temporal financeira é uma disciplina própria

As séries temporais financeiras são uma disciplina completa com milhares de estudos.

demics tentando diligentemente entender como os mercados financeiros

trabalho, tanto para o lucro como para uma regulamentação inteligente. Existem numerosos

modelos estatísticos que foram desenvolvidos para tratar alguns dos

aspectos complicados dos dados financeiros que abordamos aqui, como

como o modelo GARCH. Se você deseja aplicar estatísticas e

aprendizado de máquina para modelagem de séries temporais financeiras, você deve

estudar a história das finanças quantitativas e as principais classes de

modelos comumente implantados.

Adicionando Quantidades de Interesse aos Nossos Valores Brutos

Já calculamos o retorno diário na seção anterior. Outra quantidade de interesse que podemos gerar a partir dos insumos brutos é a *volatilidade diária* , que é a diferença entre os preços mais altos e mais baixos registrados durante o dia de negociação. Isso pode ser

facilmente calculado com base nos dados brutos (veja a Figura 14-6):

Pitão

```
>>> df['Volatilidade Diária'] = df.Alto - df.Low

>>> df.DailyVolatility.plot()
```

Assim como os preços de retorno diários são uma série temporal não estacionária, o mesmo ocorre com a série temporal de volatilidade diária. Isto confirma ainda mais que precisamos encontrar uma maneira de dimensioná-los de forma adequada. Também queremos fazê-lo sem introduzir uma previsão.

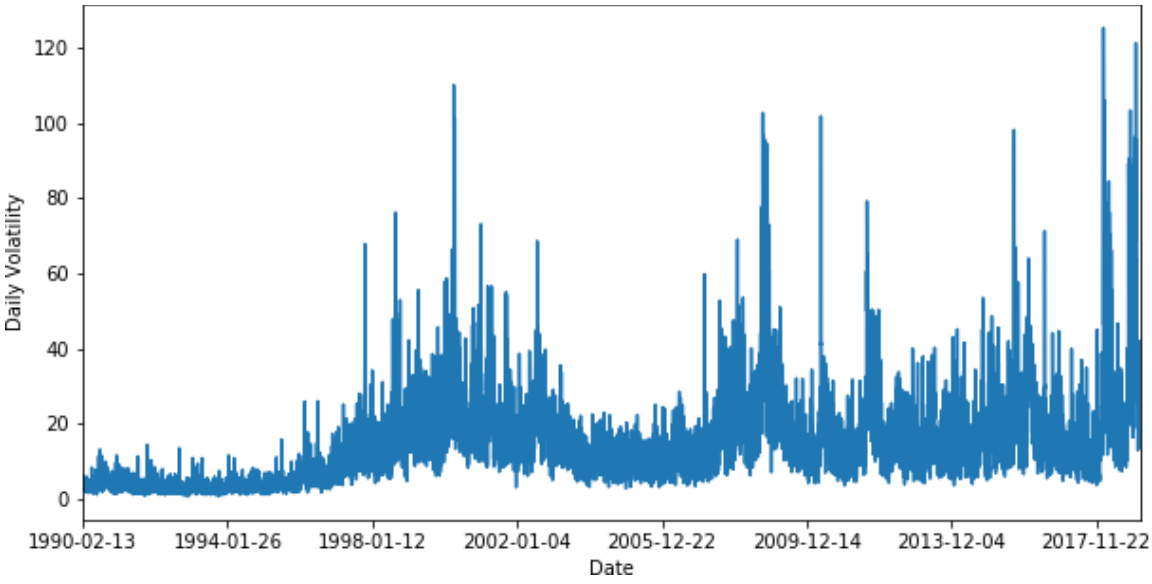


Figura 14-6. A volatilidade diária é uma série temporal de valor positivo (por definição) que mostra graus de variação visivelmente diferentes em diferentes pontos da série temporal do S&P 500.

Dimensionando quantidades de interesse sem previsão

Preveremos o retorno diário com um dia de antecedência. Algumas quantidades de interesse que podem ser úteis incluem:

• Retornos diários anteriores

• Volatilidade diária anterior

• Volume diário anterior

Escalaremos cada uma dessas quantidades subtraindo a média móvel exponencialmente ponderada e depois dividindo pelo desvio padrão exponencialmente ponderado.

Nossa exploração de dados semanal anterior mostrou que, com o pré-processamento apropriado, as diversas quantidades de interesse podem ser transformadas em séries temporais estacionárias.

Primeiro, calculamos a média móvel exponencialmente ponderada de cada coluna no quadro de dados e traçamos a

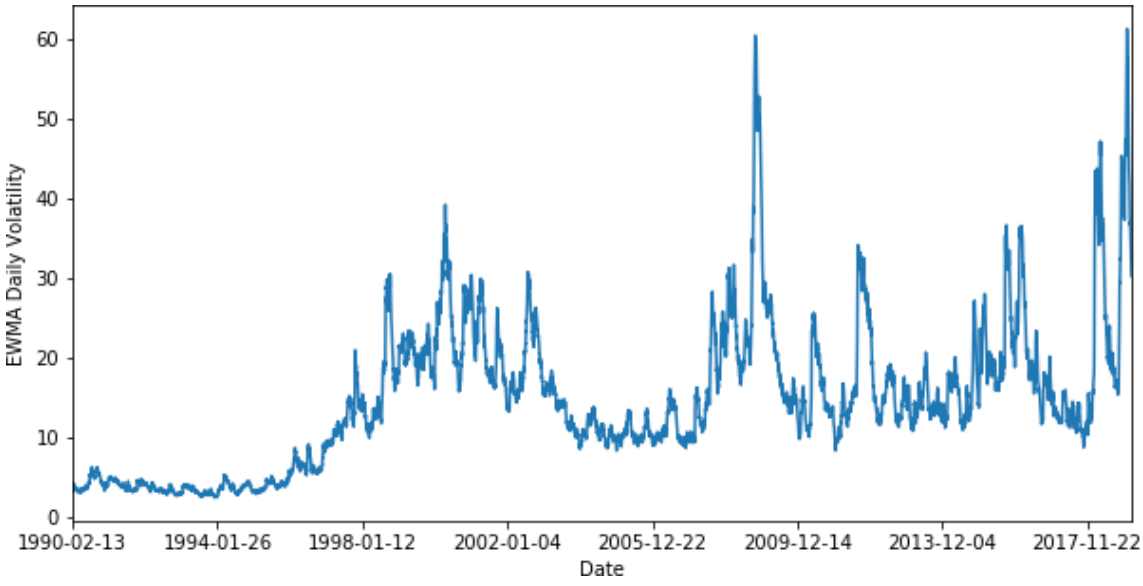
média móvel exponencialmente ponderada da volatilidade diária (veja

Figura 14-8). Compare isso com o gráfico da Figura 14-7 . Este gráfico é muito mais suave devido à média. Observe que há um parâmetro aqui que você deve considerar efetivamente como parte do seu modelo, como um hiperparâmetro, mesmo que seja usado na etapa de pré-processamento de dados: a meia-vida da suavização exponencial. O comportamento do seu modelo certamente dependerá deste parâmetro:

Pitão

>>> ewdf = df.ewm(meia-vida = 10).mean()

>>> ewdf.DailyVolatility.plot()



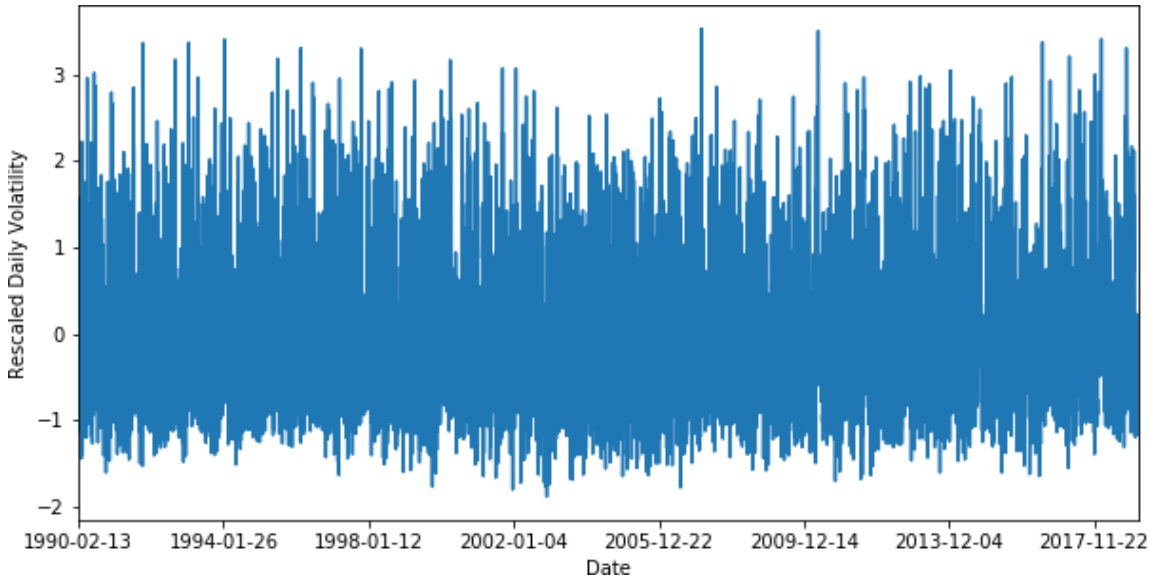


Figura 14-7. O gráfico da média móvel exponencialmente ponderada da volatilidade diária é muito mais suave do que o gráfico dos valores brutos, mas ainda mostra um tempo não estacionário Series.

Podemos então usar esse valor, bem como a variância móvel ponderada exponencialmente calculada aqui, para dimensionar valores de interesse de uma forma que resulte em séries com mais

comportamento consistente ao longo do tempo (veja a Figura 14-8):

Pitão

>>> ## calcula a variação móvel ponderada exponencialmente

>>> vewdf = df.ewm(meia-vida = 10).var()

>>>

>>> ## escala degradando e normalizando

>>> escalado = df.DailyVolatility - ewdf.DailyVolatility

>>> escalado = escalado / vewdf.DailyVolatility**0,5

```
>>> escalado.plot()
```

Figura 14-8. Transformando os dados com média e variância ponderadas exponencialmente resulta em uma série temporal muito mais uniforme com valores comparáveis ao longo do tempo período que vai de 1990 a 2019.

Transformamos todas as três entradas brutas de interesse em uma versão em escala da seguinte forma:

```
## Pitão

>>> df['ScaledVolatility'] = ((df.DailyVolatility -

>>> ewdf.DailyVolatility)

>>> / vewdf.DailyVolatility**0.5 )

>>> df['ScaledReturn'] = ((df.Return - ewdf.Return)

>>> /vewdf.Return**0.5 )

>>> df['ScaledVolume'] = ((df.Volume - ewdf.Volume)

>>> / vewdf.Volume**0.5 )
```

Finalmente, descartamos os [resultados de NA provenientes da suavização exponencial:3]

```
## Pitão

>>> df = df.dropna()
```

Você não precisa usar dados diários

Esperamos que você tenha notado que todo o processamento de dados ocorreu diariamente

[escala de tempo. Você deve se lembrar dos Capítulos 2 e 3] que aumentar e diminuir a amostragem de dados é uma forma de alterar a escala de tempo de sua análise. Da

mesma forma, você pode querer usar entradas de diversas escalas de tempo em seu modelo. Por exemplo, eu poderia considerar calcular médias móveis e variâncias ponderadas exponencialmente em uma variedade de meias-vidas e usar cada uma delas como uma entrada diferente. Da mesma forma, eu poderia calcular quantidades de interesse em diferentes escalas de tempo, como retornos semanais ou volatilidade mensal. Tudo isso é algo a ser explorado ao modelar uma série temporal, embora o espaço seja limitado aqui.

É aconselhável seguir uma escala de tempo em suas entradas semelhante à escala de tempo que você tem em mente para suas metas, ou seja, até que ponto no futuro você deseja prever. Quanto mais no futuro você deseja prever, mais para trás no tempo você deve olhar e mais você deve suavizar sua série com uma meia-vida mais longa para que possa reconhecer tendências de longo prazo. Isso o ajudará a fazer previsões de longo prazo.

Formatando nossos dados para uma rede neural

Nossos dados estão armazenados em um quadro de dados Pandas no momento, e nossas entradas planejadas são armazenadas junto com muitas entradas brutas que não temos intenção de usar. Além disso, para uma rede neural, moldaremos nossos dados no formato TNC, que, você deve se lembrar, significa tempo × número de amostras × canais. Por esta razão, precisamos fazer mais pré-processamento, mesmo após o trabalho de reescalonamento que já discutimos.

3 Em vez de eliminá-los, também poderíamos definir o valor da coluna suavizada exponencialmente para o único valor conhecido naquele momento. Em ambos os casos, isso não é importante.

Pré-processamento de dados financeiros para aprendizagem profunda | 413

Primeiro, dividimos nossos dados em componentes de treinamento e teste: 4

Pitão

>>> ## divida nossos dados em componentes de treinamento e teste

>>> trem_df = df[:7000]

>>> teste_df = df[7000:]

>>>

>>> ## construir nossas variáveis de pipeline a partir de dados de treinamento

>>> ## pegando apenas valores de interesse de quadros de dados maiores

>>> horizonte = 10

>>> X = train_df[:(7000 - horizonte)][["ScaledVolatility", "ScaledReturn",

"ScaledVolume"]].valores

>>> Y = train_df[horizon:]["ScaledReturn"].valores

Observe que há um certo problema com o que estamos prevendo com base em como configuramos Y aqui. Pense nisso por um minuto antes de continuar lendo.

Diluindo a tarefa de previsão

O que fizemos com os nossos dados diluiu a tarefa de previsão. Ou seja, pegamos um valor futuro e o “diluímos” com informações do passado. Embora esta seja uma forma de empurrar o passado para o futuro, aparentemente o oposto de antecipar, é um problema relacionado. A informação move-se ao longo do eixo do tempo de uma forma que podemos esquecer, o que por sua vez pode fazer-nos compreender mal o desempenho do nosso modelo. Deveríamos

acompanhe o que fizemos no pré-processamento para ter em mente ao avaliar nosso modelo.

O problema da previsão de valores suavizados é galopante na literatura económica e financeira – e é um problema que muitas vezes passa despercebido. Embora não seja errado fazer isto, por vezes os autores exageram os seus resultados e dizem que superaram um benchmark quando na verdade se propuseram uma tarefa mais fácil de prever um valor suavizado em vez do valor medido.

O problema com esta configuração é que o valor Y é o retorno escalonado e não apenas o retorno. Isto é melhor para o treino porque os valores estão dentro do intervalo apropriado, mas também significa que o Y que estamos a prever não é o retorno real que nos interessa, mas sim aquele retorno ajustado por uma média móvel. Estamos prevendo o quanto o nosso retorno difere da média móvel exponencialmente ponderada, em vez de apenas prever o retorno.

Isto não é errado por si só, mas significa que estamos a tornar a nossa tarefa mais fácil do que a verdadeira tarefa de previsão. Devemos estar cientes disso para que quando nosso treinamento parecer bet-4. A advertência usual de que deveríamos realmente ter um conjunto de validação separado para evitar vazamento de informações do conjunto de teste se aplica aqui, mas estou tentando manter o código simples.

Além do desempenho real do nosso modelo, saberemos que é em parte porque no treinamento nos concentramos em uma tarefa híbrida, ao passo que, em última análise, ganhar dinheiro com esse modelo dependeria apenas da verdadeira previsão.

Nós nos concentramos nos dados de treinamento e agora precisamos colocar X no formato esperado por uma arquitetura de rede neural recorrente, ou seja, TNC. Fazemos isso com uma série de operações NumPy.

Originalmente, X é bidimensional, pois vem de um quadro de dados do Pandas. Queremos adicionar uma terceira dimensão, o eixo 1 (empurrando a segunda dimensão para o eixo 2, onde os eixos são numerados de 0 para cima):

Pitão

>>> X = np.expand_dims(X, eixo = 1)

Nosso eixo temporal já é o eixo 0 porque o quadro de dados foi classificado temporalmente. O último eixo, agora eixo 2, já é o “eixo do canal” porque cada uma de nossas entradas ocupa uma coluna dessa dimensão.

Tentaremos um modelo que verá 10 intervalos de tempo – ou seja, 10 dias de dados olhando para trás. Portanto, precisamos cortar o eixo 0 para ter comprimento 10. Cortamos ao longo do eixo 0 a cada 10

linhas, e reformamos a lista resultante de submatrizes de modo que o número de amostras

(ou seja, comprimento da lista resultante) torna-se a dimensão do segundo eixo:5

Pitão

>>> X = np.split(X, X.shape[0]/10, eixo = 0)

>>> X = np.concatenate(X, eixo = 1)

>>> X.forma

(10, 699, 3)

Dado o formato TNC, temos séries temporais de comprimento 10, com três entradas paralelas. Destes temos 699 exemplos. O tamanho do lote determinará quantos lotes constituem uma época, onde uma época corresponde a um ciclo de nossos dados.

Não temos muitos dados para treinar, dados os poucos exemplos que parecemos ter. Como passamos de 30 anos de dados para poucos dados? A resposta é que cada ponto de dados foi, até agora, incluído apenas em uma série temporal de amostra. Porém, cada ponto de dados poderia estar em 10 séries temporais diferentes, ocupando uma posição diferente em cada uma.

Isso pode não ser imediatamente óbvio, então vejamos um exemplo simples. Suponha que temos a seguinte série temporal:

1, 3, 5, 11, 3, 2, 22, 11, 5, 7, 9

5 Para usuários de R: lembre-se de que Python conta a partir de 0, então o segundo eixo é o eixo 1, não o eixo 2.

Queremos treinar uma rede neural com esta série temporal, desta vez assumindo uma janela de tempo de comprimento 3. Se usarmos a preparação de dados que acabamos de realizar, nossos exemplos de série temporal seriam:

• 1, 3, 5

• 11, 3, 2

• 22, 11, 5

• 7, 9, _

No entanto, não há razão para privilegiar o início dos nossos dados como tendo de alguma forma definir o início e o fim de cada série temporal de amostra. As janelas são arbitrárias.

Algumas séries temporais igualmente válidas, cortadas do todo em uma janela, são:

• 3, 5, 11

• 2, 22, 11

• 5, 7, 9

Portanto, se precisássemos de mais dados, estaríamos bem servidos para gerar amostras de séries temporais ao deslizarmos uma janela sobre todo o conjunto de dados. Isso produziria mais amostras de séries temporais individuais do

que fizemos com nosso método de dividir os dados em amostras de séries temporais não sobrepostas. Tenha isso em mente ao preparar seus próprios conjuntos de dados. Abaixo, você verá que usamos este método de janela deslizante para pré-processar nossos dados.

Construindo e treinando um RNN

Conforme mencionado na introdução deste capítulo, as séries temporais financeiras são notoriamente difíceis de modelar e compreender. Embora a indústria financeira continue a ser um dos pilares da economia ocidental, os especialistas concordam que é muito difícil fazer previsões. Por esta razão, procuramos uma técnica que seja adequada para um sistema complicado com dinâmica potencialmente não linear, nomeadamente uma rede neural de aprendizagem profunda.

No entanto, devido à nossa falta de dados, escolhemos uma arquitetura de rede neural recorrente simples (LSTM) e um regime de treinamento descrito por estes parâmetros:

```
## Pitão

>>> ## parâmetros de arquitetura

>>> NUM_HIDDEN = 4

>>> NUM_LAYERS = 2

>>>

>>> ## parâmetros de formatação de dados

>>> BATCH_SIZE = 64

>>> WINDOW_SIZE = 20

>>>

>>> ## parâmetros de treinamento
```

>>> TAXA_APRENDIZAGEM = 1e-2

>>> ÉPOCAS = 30

Em contraste com o Capítulo 10 , usamos o pacote TensorFlow em vez do MXNet, para que você possa ver outro exemplo de estrutura de aprendizado profundo amplamente utilizada. No TensorFlow, definimos variáveis para todas as quantidades que usaremos em nossa rede, mesmo com alteração de valores que representam entradas. Para as entradas usamos espaços reservados, que são uma forma de informar ao gráfico qual formato esperar:

Pitão

>>> Xinp = tf.placeholder(dtype = tf.float32,

>>> forma = [WINDOW_SIZE, Nenhum, 3])

>>> Yinp = tf.placeholder(dtype = tf.float32, shape = [Nenhum])

Em seguida, construímos nossa rede e implementamos etapas de cálculo e otimização de perdas:

Pitão

>>> **com** tf.variable_scope("scope1", reuse=tf.AUTO_REUSE):

>>> células = [tf.nn.rnn_cell.LSTMCell(num_units=NUM_HIDDEN)

>>> **para** n **no** intervalo (NUM_LAYERS)]

>>> stacked_rnn_cell = tf.nn.rnn_cell.MultiRNNCell(células)

>>> rnn_output, estados = tf.nn.dynamic_rnn(stacked_rnn_cell,

>>> Xinp,

```
>>> dtype=tf.float32)

>>> W = tf.get_variable("W_fc", [NUM_HIDDEN, 1],

>>> inicializador =

>>> tf.random_uniform_initializer(-.2, .2))

>>>

>>> ## observe que não temos preconceito porque
esperamos retorno médio zero

>>> saída = tf.squeeze(tf.matmul(rnn_output[-1, :, :], W))

>>>

>>> perda = tf.nn.l2_loss(saída - Yinp)

>>> opt =
tf.train.GradientDescentOptimizer(LEARNING_RATE)

>>> train_step = opt.minimize(perda)
```

Temos uma maneira bastante complicada de alimentar os dados devido ao que discutimos anteriormente, ou seja, que cada ponto de dados deve estar em múltiplas séries temporais, dependendo de qual deslocamento usamos. Aqui tratamos o mesmo problema de formatação de dados que discutimos mais detalhadamente no Capítulo 10:

```
## Pitão

>>> ## para cada época

>>> y_hat_dict = {}

>>> Y_dict = {}

>>>

>>> in_sample_Y_dict = {}
```

```
>>> in_sample_y_hat_dict = {}

>>>

>>> para ep no intervalo (EPOCHS):

>>> época_training_loss = 0,0

>>> para i no intervalo (WINDOW_SIZE):

>>> X = train_df[:(7000 - WINDOW_SIZE)]
[["ScaledVolatility", Construindo e treinando um RNN | 417

>>> "ScaledReturn",

>>> "ScaledVolume"]].valores

>>> Y = train_df[WINDOW_SIZE:]["ScaledReturn"].valores

>>>

>>> ## torná-lo divisível pelo tamanho da janela

>>> num_to_unpack = math.floor(X.shape[0] /
WINDOW_SIZE)

>>> start_idx = X.shape[0] - num_to_unpack *
WINDOW_SIZE

>>> X = X[start_idx:]

>>> Y = Y[start_idx:]

>>>

>>> X = X[i:-(WINDOW_SIZE-i)]

>>> Y = Y[i:-(WINDOW_SIZE-i)]

>>>
```

```
>>> X = np.expand_dims(X, eixo = 1)

>>> X = np.split(X, X.shape[0]/WINDOW_SIZE, eixo = 0)

>>> X = np.concatenate(X, eixo = 1)

>>> S = Y[::WINDOW_SIZE]

>>> ## TREINAMENTO

>>> ## agora faça um lote e execute uma sessão

>>> para j no intervalo(math.ceil(Y.shape[0] /
BATCH_SIZE)):

>>> ll = BATCH_SIZE * j

>>> ul = BATCH_SIZE * (j + 1)

>>>

>>> se ul > X.shape[1]:

>>> ul = X.forma[1] - 1

>>> ll = X.forma[1]- BATCH_SIZE

>>>

>>> perda_treinamento, _, y_hat = sess.run([perda,
train_step,

>>> saída],

>>> feed_dict = {

>>> Xinp: X[:, ll:ul, :],

>>> Yinp: Y[ll:ul]

>>> })
```

```
>>> época_training_loss += training_loss
>>>
>>> in_sample_Y_dict[ep] = Y[ll:ul]
>>> ## observe que isso só nos renderá a última parte do
>>> ## dados treinados em
>>> in_sample_y_hat_dict[ep] = y_hat
>>>
>>> ## TESTE
>>> X = test_df[:(test_df.shape[0] - WINDOW_SIZE)]
>>> [["ScaledVolatility", "ScaledReturn",
>>> "ScaledVolume"]].valores
>>> Y = test_df[WINDOW_SIZE:]["ScaledReturn"].valores
>>> num_to_unpack = math.floor(X.shape[0] /
WINDOW_SIZE)
>>> start_idx = X.shape[0] - num_to_unpack *
WINDOW_SIZE
>>> ## é melhor jogar fora no início do que no final do
treino
>>> ## período em que deve ser excluído
>>> X = X[start_idx:]
>>> Y = Y[start_idx:]
>>>
```

```
>>> X = np.expand_dims(X, eixo = 1)

>>> X = np.split(X, X.shape[0]/WINDOW_SIZE, eixo = 0)

>>> X = np.concatenate(X, eixo = 1)

>>> S = Y[::WINDOW_SIZE]

>>> perda_testando, y_hat = sess.run([perda, saída],

>>> feed_dict = {Xinp: X, Yinp: Y})

>>> ## nb isso não é ótimo. deveríamos realmente ter uma
validação

>>> ## perda além dos testes

>>>

>>> print ("Época: %d Perda de treinamento: %0.2f

>>> Testando perda %0.2f:" %

>>> (ep, época_training_loss, testes_loss))

>>> Y_dict[ep] = Y

>>> y_hat_dict[ep] = y_hat
```

Aqui vemos nossas métricas de treinamento e teste:

Época: 0 Perda de treinamento: 2.670,27 Perda de teste 526,937:

Época: 1 Perda de treinamento: 2.669,72 Perda de teste 526,908:

Época: 2 Perda de treinamento: 2.669,53 Perda de teste 526,889:

Época: 3 Perda de treinamento: 2.669,42 Perda de teste 526,874:

Epoca: 4 Perda de treinamento: 2.669,34 Perda de teste 526,862:

Época: 5 Perda de treinamento: 2.669,27 Perda de teste 526,853:

Época: 6 Perda de treinamento: 2.669,21 Perda de teste 526,845:

Época: 7 Perda de treinamento: 2.669,15 Perda de teste 526,839:

Época: 8 Perda de treinamento: 2.669,09 Perda de teste 526.834:

Época: 9 Perda de treinamento: 2.669,03 Perda de teste 526.829:

Época: 10 Perda de treinamento: 2.668,97 Perda de teste 526,824:

Época: 11 Perda de treinamento: 2.668,92 Perda de teste 526,819:

Época: 12 Perda de treinamento: 2.668,86 Perda de teste 526,814:

Época: 13 Perda de treinamento: 2.668,80 Perda de teste 526,808:

Época: 14 Perda de treinamento: 2.668,73 Perda de teste 526.802:

Época: 15 Perda de treinamento: 2.668,66 Perda de teste 526,797:

Época: 16 Perda de treinamento: 2.668,58 Perda de teste 526.792:

Época: 17 Perda de treinamento: 2.668,49 Perda de teste 526,788:

Epoca: 18 Perda de treinamento: 2.668,39 Perda de teste 526,786:

Época: 19 Perda de treinamento: 2.668,28 Perda de teste 526,784:

Época: 20 Perda de treinamento: 2.668,17 Perda de teste 526,783:

Época: 21 Perda de treinamento: 2.668,04 Perda de teste 526.781:

Época: 22 Perda de treinamento: 2.667,91 Perda de teste 526,778:

Época: 23 Perda de treinamento: 2.667,77 Perda de teste 526.773:

Época: 24 Perda de treinamento: 2.667,62 Perda de teste 526,768:

Época: 25 Perda de treinamento: 2.667,47 Perda de teste 526.762:

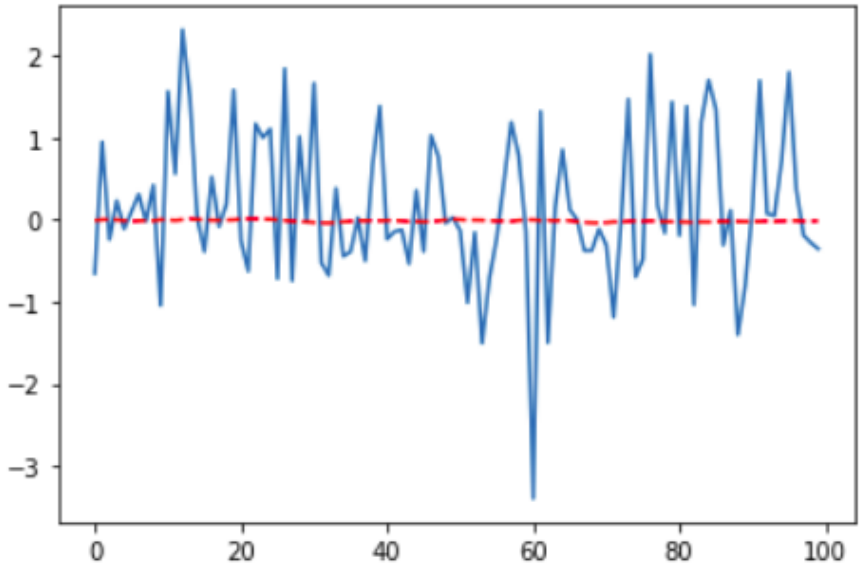
Época: 26 Perda de treinamento: 2.667,31 Perda de teste 526,755:

Época: 27 Perda de treinamento: 2.667,15 Perda de teste 526,748:

Época: 28 Perda de treinamento: 2.666,98 Perda de teste 526,741:

Época: 29 Perda de treinamento: 2.666,80 Perda de teste 526,734:

A métrica de erro que escolhemos não nos dá uma ideia de quão bem nossos dados gerais correspondem aos resultados, portanto, a plotagem é útil. Representamos graficamente o desempenho fora da amostra (importante) e o desempenho dentro da amostra (menos importante), conforme mostrado na Figura 14-9 :



Pitão

```
>>> plt.plot(test_y_dict[MAX_EPOCH])

>>> plt.plot(test_y_hat_dict[MAX_EPOCH], 'r--')

>>> plt.show()
```

Figura 14-9. Os valores de retorno reais para uma subseção do período de teste (linha sólida) são plotado em relação às previsões da rede neural (linha tracejada). A escala do a previsão é tão diferente dos dados reais que é difícil avaliar o modelo.

Podemos ver que nossos valores previstos para os retornos não tendem a ser iguais aos retornos reais. Verificamos a correlação de Pearson a seguir:

Pitão

```
>>> pearsonr(test_y_dict[MAX_EPOCH],
test_y_hat_dict[MAX_EPOCH])

(0,035957868817734 19, 0,2010510706894 9668)
```

Esses números podem parecer sombrios se você ainda não tiver trabalhado com séries temporais financeiras. Nesta

indústria, nosso modelo pode ser útil apesar do enredo e apesar do *p* -

valor. Nas finanças, uma correlação positiva é estimulante e algo que pode ser melhorado gradativamente. Na verdade, muitos projetos de pesquisa, ao começarem, não conseguem necessariamente chegar a correlações tão “altas”.

Podemos ter uma ideia melhor se as previsões vão pelo menos na mesma direção aumentando os retornos previstos em uma ordem de magnitude e plotando novamente

(Figura 14-10):

Pitão

```
>>> plt.plot(test_y_dict[MAX_EPOCH][:100])

>>> plt.plot(test_y_hat_dict[MAX_EPOCH][:100] * 50, 'r--')

>>> plt.show()
```

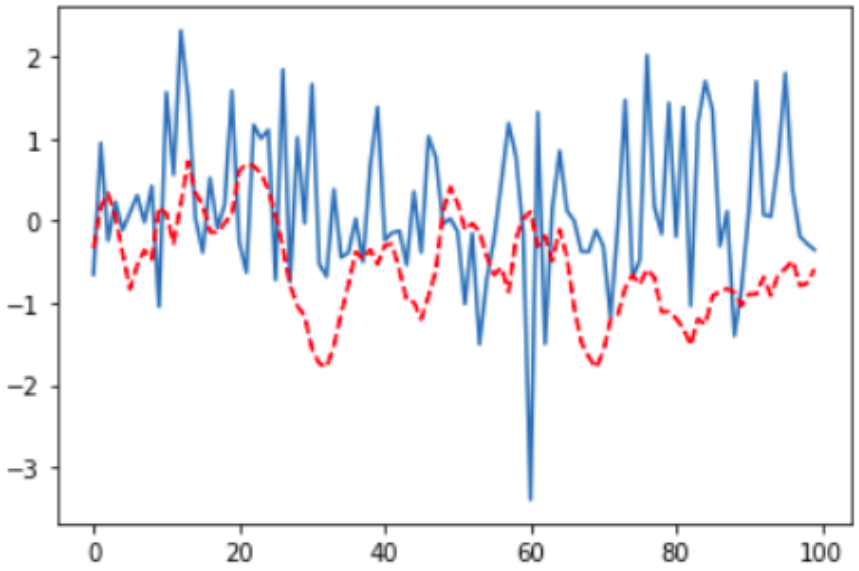



Figura 14-10. Uma melhor noção de como as previsões do modelo (linha tracejada) se comparam às os dados reais (linha sólida). No entanto, com uma correlação tão baixa, é

mais provável que pensar que vemos um padrão do que realmente ver um. Por esta razão, métricas quantitativas nos servirá melhor do que avaliações visuais para dados financeiros ruidosos.

Se você leu blogs sobre o uso de aprendizado profundo para séries temporais financeiras, pode muito bem achar o desempenho aqui decepcionante. Por exemplo, você provavelmente já viu postagens de blog em que alguém aplica um LSTM multicamadas simples a alguns dados diários de ações e produz previsões que parecem quase idênticas aos dados reais do mercado de ações, mesmo fora da amostra. Há duas razões importantes pelas quais esses resultados podem parecer bons, mas na verdade não são impressionantes:

• Pré-processar o código para dimensioná-lo com uma solução de dimensionamento pronta para uso, como sklearn.preprocessing.MinMaxScaler. Isto não é ideal porque inclui um

antecipação usando os valores de todos os tempos para dimensionar os dados.

• Prever preços em vez de retornos. Esta é uma tarefa muito mais fácil – para começar, uma excelente previsão do preço no dia $T + 1$ é o preço no dia T. Assim, é fácil construir um modelo que pareça prever preços razoavelmente bem e produza gráficos impressionantes. Infelizmente, esses modelos não podem ser usados para ganhar dinheiro.

Tentamos um exemplo mais realista da indústria, o que significa que o desafio é maior e os enredos não serão tão satisfatórios.

Considerações de desempenho

Usamos um LSTM tf.nn.rnn_cell.LSTMCell padrão aqui por vários motivos, principalmente porque é um burro de carga geral. Também o usamos porque este livro usa código que

não requer o uso de GPU para que você possa executar esses scripts mesmo em um laptop padrão.

Se você tiver uma GPU, você deve usar a API para cuDNN da NVIDIA, que é um

implementação personalizada de hardware disponível para as duas principais elaborações em um RNN padrão: o LSTM e o GRU. Supondo que sua GPU seja da NVIDIA, que

a maioria é, isso será substancialmente mais rápido para o treinamento do que as implementações padrão. Isso está disponível no TensorFlow e também em outros pacotes convencionais de aprendizado profundo, e o uso é bastante semelhante ao que codificamos aqui, exceto por empregar uma célula RNN diferente.

Este conselho pode mudar à medida que as TPUs do Google forem implementadas no mercado e à medida que outras soluções de hardware surgirem para aprendizado profundo.

Escusado será dizer que não concluímos uma análise completa do desempenho do modelo.

Isso nos daria uma ideia de como construir nosso próximo modelo, o que poderíamos ter esquecido e se um modelo de aprendizagem profunda pode justificar sua complexidade adicional em relação a um modelo linear. Há muitos caminhos a seguir para melhorar o desempenho do modelo.

Há muitas maneiras de melhorar este modelo, que você deve considerar como

extensões deste código:

• Adicione mais entradas de dados brutos gerando recursos adicionais com base nessas entradas. Não usamos todas as colunas de entrada brutas e existem outras maneiras de reexpressar essas quantidades que podem ser úteis. Você pode considerar variáveis categóricas, como “A máxima ou a mínima do dia coincidiu com a abertura ou o fechamento?” (existem várias condições binárias agrupadas naquela

pergunta).

• Integrar séries temporais paralelas para outras ações. Isso adicionará mais informações e dados para treinar.

• Use dados em diversas escalas de tempo diferentes. Um artigo amplamente citado que faz isso

xinga uma arquitetura chamada ClockworkRNN.

• Aumente seus dados usando exemplos de séries temporais existentes e adicionando jitter.

Isso ajudará com o fato de que este conjunto de dados não oferece muitos dados.

• Permita que sua arquitetura de rede cresça se você tiver aumentado o número de entradas ou a quantidade de dados. Uma arquitetura mais complicada nem sempre é

maneira de melhorar o desempenho, mas pode ser apropriado se você perceber que sua rede está no limite.

• Tente treinar os dados em ordem cronológica, em vez da nossa abordagem de percorrer os dados várias vezes por época. As vezes isso pode ser bastante útil (mas depende do conjunto de dados). Dado que vimos o comportamento das séries temporais mudar ao longo do tempo, talvez seja melhor encerrar o treinamento com os últimos dados por último, para que o comportamento alterado seja refletido nos pesos.

• Considere diferentes funções de perda. Aqui usamos uma norma L2, que tende a punir muito mais diferenças maiores do que diferenças pequenas. Dado o domínio, contudo, podemos querer avaliar o sucesso de forma diferente. Talvez queiramos apenas acertar o sinal previsto do retorno diário e não nos preocupar tanto com sua magnitude. Neste caso poderíamos considerar definir as metas como variáveis categóricas: positivas, negativas, zero. No caso de dados categóricos, geralmente queremos usar uma medida de perda de entropia cruzada. No entanto, dado que estes não são dados puramente categóricos, uma vez que são classificados (ou seja, zero está mais próximo do negativo

do que positivo está do negativo), podemos querer usar uma função de perda personalizada para refletir isso.

• Considere construir um conjunto de redes neurais simples em vez de uma única. Mantenha cada rede individual pequena. Os conjuntos são particularmente úteis para dados de baixa relação sinal-ruído, como dados financeiros.

• Determine por que a escala das previsões é tão diferente da escala dos valores reais. Considere começar avaliando se a função de perda que usamos é problemática, dada a forte preponderância de retornos diários de valor zero.

Como você pode ver, existem inúmeras maneiras de melhorar o desempenho de uma rede ou ajustar sua funcionalidade. Como você fará isso dependerá do conjunto de dados. É sempre útil usar visualizações do desempenho da sua rede, conhecimento do domínio e uma meta firmemente definida (que neste caso é provavelmente "ganhar dinheiro") para orientar como você refina seu modelo. Caso contrário, você pode facilmente se perder no grande número de opções que tem.

Mais recursos

Joumana Ghosn e Yoshua Bengio, "Aprendizado multitarefa para seleção de ações", Cambridge: MIT Press, 1996, https://perma.cc/GR7A-5PQ5.

Este artigo de 1997 fornece um exemplo muito antigo de aplicação de uma rede neural ao problema dos mercados financeiros. Neste caso, o autor utilizou o que hoje seria considerado uma rede bastante simples com dados mínimos, mas mesmo assim descobriu que a sua rede poderia aprender a escolher ações de forma lucrativa. Curiosamente, este também é um dos primeiros exemplos de aprendizagem multitarefa.

Lawrence Takeuchi e Yu-Ying Lee, "Aplicando Aprendizado Profundo para Melhorar o Momen-

tum Estratégias de Negociação em Ações”, 2013, https://perma.cc/GJZ5-4V6Z.

Neste artigo, os autores interpretam sua rede neural através das lentes de

“treinamento momentâneo”, uma forma tradicional de prever quantitativamente os mercados financeiros em um mundo pré-aprendizado de máquina. Este artigo é interessante por discutir como as decisões de treinamento foram tomadas e o desempenho do modelo avaliado.

“Alguém está ganhando dinheiro usando aprendizado profundo nas negociações?” Quora, https://perma.cc/

Z8C9-V8FX.

Nesta pergunta e resposta, vemos uma variedade de opiniões sobre até que ponto o aprendizado profundo tem sido bem-sucedido em aplicações financeiras. Conforme descrito em algumas das respostas, qualquer PI lucrativa seria provavelmente fortemente protegida por acordos de não divulgação e pelo incentivo ao lucro, de modo que nesta indústria pode ser muito difícil avaliar qual é o desempenho de última geração. As respostas também apontam para uma vasta gama de potenciais aplicações financeiras – prever retornos é apenas um problema entre muitos.

CAPÍTULO 15

Séries temporais para o governo

A análise de séries temporais é bastante relevante e importante para aplicações governamentais por uma série de razões. Em primeiro lugar, os governos, grandes e pequenos, são os guardiões de alguns dos dados de séries cronológicas mais importantes do mundo, incluindo o relatório de emprego dos EUA, dados sobre a temperatura dos oceanos (ou seja, dados sobre o aquecimento global) e estatísticas sobre a criminalidade local. Em segundo lugar, os governos, por definição, fornecem alguns dos serviços mais essenciais de que todos dependemos e, por isso,

precisam de ser analistas da procura razoavelmente competentes se não quiserem gastar excessivamente ou faltar pessoal para esses serviços. Assim, todos os aspectos das séries temporais são relevantes para fins governamentais: armazenamento, limpeza, exploração e previsão.

Como mencionei no Capítulo 2, ao discutir séries temporais "encontradas", uma percentagem muito elevada de todos os dados governamentais pode parecer muito com dados de séries temporais com alguma reestruturação. Geralmente, a maioria dos conjuntos de dados governamentais são o resultado de uma recolha contínua de dados e não de uma única fatia de tempo. No entanto, os conjuntos de dados governamentais podem ser assustadores por uma série de razões:

• Manutenção de registros inconsistente (devido a restrições organizacionais ou mudanças de forças políticas ao longo do tempo)

• Práticas de dados opacas ou confusas

• Enormes conjuntos de dados com conteúdo de informação relativamente baixo

No entanto, pode ser bastante interessante examinar os conjuntos de dados governamentais, tanto por interesse intelectual como para muitos fins práticos. Neste capítulo, exploramos um conjunto de dados governamentais que consiste em todas as reclamações feitas na cidade de Nova York desde 2010.

até o presente para uma linha direta municipal que pode ser contatada discando 311. Porque o

o conjunto de dados é continuamente atualizado, os dados vistos no livro provavelmente serão diferentes daqueles que você vê ao baixá-lo; você terá ainda mais informações do que eu quando

preparando este capítulo. No entanto, os resultados devem ser bastante semelhantes. Neste capítulo abordaremos alguns tópicos:

• Fontes interessantes de dados governamentais, incluindo aquele que analisaremos

• Lidar com arquivos extremamente grandes de dados de texto simples

• Análise estatística on-line/contínua de grandes conjuntos de dados e outras opções para analisar dados sem manter tudo na memória

Obtenção de dados governamentais

Os conjuntos de dados governamentais na categoria “dados encontrados” podem ser um pesadelo do ponto de vista da consistência dos dados. Esses conjuntos de dados, embora tenham um carimbo de data/hora, geralmente são divulgados para uma iniciativa de dados abertos, e não para uma finalidade específica de série temporal.

Muitas vezes há pouca ou nenhuma informação disponível sobre as convenções de carimbo de data/hora dos dados ou outras convenções de gravação. Pode ser difícil confirmar se as práticas de registo subjacentes eram consistentes. 1

No entanto, se você é aventureiro ou deseja ser o primeiro a identificar características temporais interessantes nos comportamentos humanos relacionados às atividades governamentais, você tem sorte de viver na era do governo aberto e dos dados abertos. Muitos governos, a todos os níveis, envidaram mais esforços nos últimos anos para tornar os seus dados de séries cronológicas transparentes ao público. Aqui estão apenas alguns exemplos de onde você pode obter dados governamentais abertos com um componente de série temporal:

• Dados hospitalares mensais dos Serviços Nacionais de Saúde do Reino Unido.

Este conjunto de dados é surpreendentemente ciente de séries temporais: inclui uma guia chamada

“Série temporal MAR” e descreve as convenções de gravação e como elas foram

evoluiu ao longo do tempo.

• O portal de dados abertos da Jamaica também inclui uma apreciação e reconhecimento de dados de séries temporais, como o seu conjunto de dados com data e hora de casos de Chikungunya para 2014 e

o relatório de dados associado , que inclui uma animação (ou seja, uma visualização de série temporal) e uma curva epidêmica.

• O portal de dados abertos de Singapura apresenta extensos conjuntos de dados e divulga a natureza de séries temporais de alguns desses dados, incluindo dois gráficos de séries temporais em seu principal

página, conforme mostrado na Figura 15-1.

1 Observe que os dados de séries temporais muito procurados, como o relatório de empregos dos EUA, são muito conscientes das séries temporais e são meticulosamente limpos e formatados. No entanto, esses dados já são bastante escolhidos e é improvável que forneçam novas aplicações de séries temporais ao candidato a pesquisador ou empreendedor.

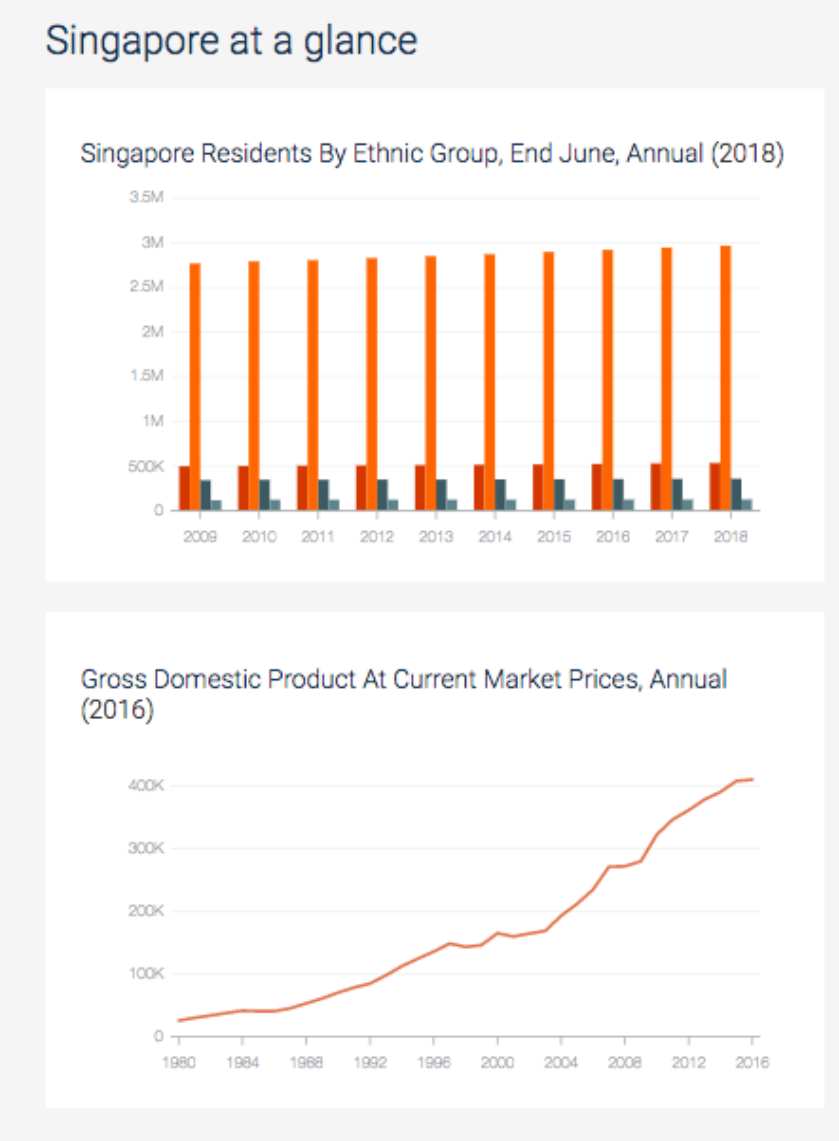


Figura 15-1. Dois dos quatro gráficos na página principal do site de dados abertos de Cingapura (conforme acessado na primavera de 2019) são visualizações de séries temporais usadas para mostrar informações importantes sobre o país.

Como você pode notar, todos os meus exemplos são de áreas de língua inglesa, mas é claro que elas não têm o monopólio do movimento de dados abertos no governo. Por exemplo, a cidade de Paris , a nação da Sérvia e o Grupo do Banco Africano de Desenvolvimento

todos administram sites de dados abertos. 2

Nos exemplos deste capítulo, extraímos nossas informações do portal de dados abertos da cidade de Nova York, selecionado porque Nova York é um lugar grande e interessante que por acaso é minha casa. Mergulharemos no conjunto de dados da linha direta 311 na próxima seção.

2 É claro que o seu acesso pode depender das suas capacidades linguísticas (ou das de um colega prestativo).

Explorando grandes dados de séries temporais

Quando os dados forem suficientemente grandes, você não conseguirá colocá-los na memória. O tamanho dos dados necessário antes de atingir esse limite dependerá do hardware que você está usando. 3

Eventualmente, você precisará entender como iterar seus dados, um pedaço gerenciável de cada vez. Para aqueles familiarizados com aprendizado profundo, provavelmente você já fez isso, principalmente se trabalha com processamento de imagens. Em estruturas de aprendizado profundo, muitas vezes há iteradores Python que percorrem um conjunto de dados com esse conjunto de dados armazenado em diretórios especificados, cada um com muitos arquivos. 4

Quando baixei o conjunto de dados 311, ele tinha mais de 3 gigabytes no formato CSV. Não havia como abrir isso no meu computador, então minha primeira ideia foi usar opções padrão do sistema operacional Unix, como head.

Infelizmente, o que foi impresso já era tão grande que era impossível de gerenciar em uma interface de linha de comando Unix, pelo menos para alguém inexperiente em ferramentas Unix:

```
## linha de comando do sistema operacional linux

$ cabeça 311.csv

Chave exclusiva, Data de criação, Data de fechamento,
Agência, Nome da agência, Tipo de reclamação, Descrito
```

27863591,17/04/2014 12h00,28/04/2014 12h00,DOHMH,Departamento de Cura 27863592 , 17/04/2014 12:00:00, 22/04/2014 12:00:00, DOHMH, Departamento de Cura 27863595,04/17/2014 10:23:00, 0417/2014 12:00 :00 PM, DSNY, Queens East 12, Derel 27863602,04/17/2014 17:01:00 PM, 17/04/2014 17:01:00 PM, DSNY,BCC - Queens East,D

27863603,04/17/2014 12:00:00, 23/04/2014 12:00:00 AM, HPD, Departamento de Housin 27863604,04/17/2014 12:00:00, 22/04/2014 12:00:00 AM, HPD, Departamento de Housin 27863605,04/17/2014 12:00:00 AM, 21/04/2014 12:00:00 AM, HPD, Departamento de Housin Embora o conteúdo seja pesado, este A visualização foi suficiente para mostrar que havia vários carimbos de data e hora, bem como outras informações interessantes e ordenadas, como coordenadas geográficas. É evidente que os dados são muito vastos, por isso precisamos de ser capazes de manipular estes dados.

informações para obter as colunas que desejamos.5

Mesmo se você for novo no Linux, poderá aprender facilmente sobre ferramentas simples de linha de comando que podem fornecer informações úteis. Podemos obter uma contagem de linhas do arquivo CSV para ter uma ideia da escala que estamos observando, ou seja, quantos pontos de dados temos.

Esta é uma linha única:

3 Se a sua organização sempre aumenta a escala para resolver o problema de falta de RAM, você está fazendo isso errado.

4 Para se inspirar, leia a documentação do TensorFlow sobre conjuntos de dados e classes associadas .

5 Observe que alguém especialista em sistemas Unix poderia facilmente usar o awk para manipular um CSV de forma bastante eficaz na linha de comando ou com um script de shell simples. Freqüentemente, essas ferramentas são excelentes opções para big data porque são extremamente eficientes e bem implementadas, ao

contrário, infelizmente, de muitas ferramentas de análise de dados comumente usadas em R e Python. Se você tiver problemas com suas ferramentas de processamento de dados favoritas, muitas vezes pode ser uma boa ideia aprender algumas ferramentas de linha de comando Unix para complementar, especialmente no caso de big data.

linha de comando do sistema operacional linux

$ wc -l 311.csv

19811967 311.csv

Podemos ver que Nova York recebeu cerca de 20 milhões de reclamações para o 311 desde 2010.

São mais de duas reclamações por morador.

Munidos desse conhecimento, usamos data.table do R sabendo que sua função fread() permite a leitura parcial de arquivos (vista na documentação quando você lê sobre os parâmetros nrows e skip6) e que data.table é extremamente eficiente ao lidar com grandes dados conjuntos. Podemos usar isso para obter informações iniciais, como no código a seguir:

##R

> df = fread("311.csv", pular = 0, nrows = 10)

> nomes de colunas (df)

[1] "Chave Exclusiva" "Data de Criação"

[3] "Data de Fechamento" "Agência"

[5] "Nome da agência" "Tipo de reclamação"

[7] "Descritor" "Tipo de localização"

[9] "Código postal do incidente" "Endereço do incidente"

[11] "Nome da rua" "Rua transversal 1"

[13] "Rua Cruzada 2" "Rua Cruzada 1"

[15] "Interseção Rua 2" "Tipo de endereço"

[17] "Cidade" "Marco"

[19] "Tipo de instalação" "Status"

[21] "Data de Vencimento" "Descrição da Resolução"

[23] "Data de atualização da ação de resolução" "Conselho comunitário"

[25] "BBL" "Bairro"

[27] "Coordenada X (Plano de Estado)" "Coordenada Y (Plano de Estado)"

[29] "Tipo de canal de dados aberto" "Nome da instalação do parque"

[31] "Park Borough" "Tipo de veículo"

[33] "Taxi Company Borough" "Local de retirada do táxi"

[35] "Nome da rodovia da ponte" "Direção da rodovia da ponte"

[37] "Rampa Rodoviária" "Segmento Rodoviário da Ponte"

[39] "Latitude" "Longitude"

[41] "Localização"

Só de ler as primeiras 10 linhas já podemos ver os nomes das colunas. Apesar de todos os pontos fortes que listei para abordagens NoSQL para dados de série temporal, pode ser bom, em um grande conjunto de dados, saber os nomes das colunas desde o início. É claro que existem soluções alternativas para isso com dados NoSQL, mas a maioria

exige algum esforço por parte do usuário, em vez de acontecer automaticamente.

Várias colunas sugerem informações úteis:

"Data de criação"

"Data Fechada"

6 Observe que o Pandas do Python oferece funcionalidade semelhante .

"Data de vencimento"

"Data de atualização da ação de resolução"

Provavelmente serão do tipo caractere antes da conversão, mas depois de fazermos uma conversão para um tipo POSIXct, podemos ver como são os intervalos de tempo entre essas datas:

##R

> df$DataCriada = df[,DataCriada := as.POSIXct(DataCriada,

> format = "%m/%d/%Y %I:%M:%S %p") Na string de formatação, precisamos usar %I para a hora, pois ela é expressa apenas no formato 01–12, e um %p porque os carimbos de data/hora incluem uma designação AM/PM.

Para ter uma noção de como essas datas tendem a ser espaçadas, especialmente no que diz respeito a quando uma reclamação é criada e quando uma reclamação é encerrada, vamos carregar mais linhas e examinar a distribuição do que chamarei de *tempo de vida da reclamação* (ou seja, o intervalo entre criação e fechamento):

##R

> resumo(as.numeric(df$ClosedDate - df$CreatedDate,

> unidades = "dias"))

Min. 1º Qu. Mediana Média 3º Qu. Máx. **NA**

-75,958 1,000 4,631 13,128 12,994 469,737 113

Como podemos ver, é uma ampla distribuição. Mais surpreendente do que os tempos de espera de um ano para o encerramento de algumas reclamações é o facto de algumas reclamações terem tempos negativos, até mesmo extremamente negativos, entre a sua criação e a data de encerramento. Se o tempo negativo fosse em torno de –365 dias (um ano), poderíamos imaginar um problema de entrada de dados, mas isso parece menos provável com números como –75 dias. Este é um problema que precisaremos investigar.

Podemos identificar outro problema considerando um intervalo de datas de criação:

##R

> intervalo(df$DataCriada)

[1] "13/03/2014 12:56:24 EDT" "06/02/2019 23:35:00 EST"

Dado o tamanho deste arquivo CSV e o fato de que ele deve ser continuamente atualizado, é surpreendente que as primeiras linhas não sejam da data de 2010, que deveria marcar os primeiros conjuntos de dados. Esperávamos que o CSV fosse continuamente anexado. Mais surpreendente ainda é o facto de as datas de 2019 estarem nas primeiras linhas e de as datas de 2014 e 2019 estarem ambas nas mesmas primeiras 10.000 linhas. Isso sugere que não podemos determinar facilmente a ordem das datas dos dados no arquivo. Podemos visualizar a distribuição de datas de uma linha para a próxima realizando um gráfico de linha do índice de linha versus data, como fazemos na Figura 15-2.

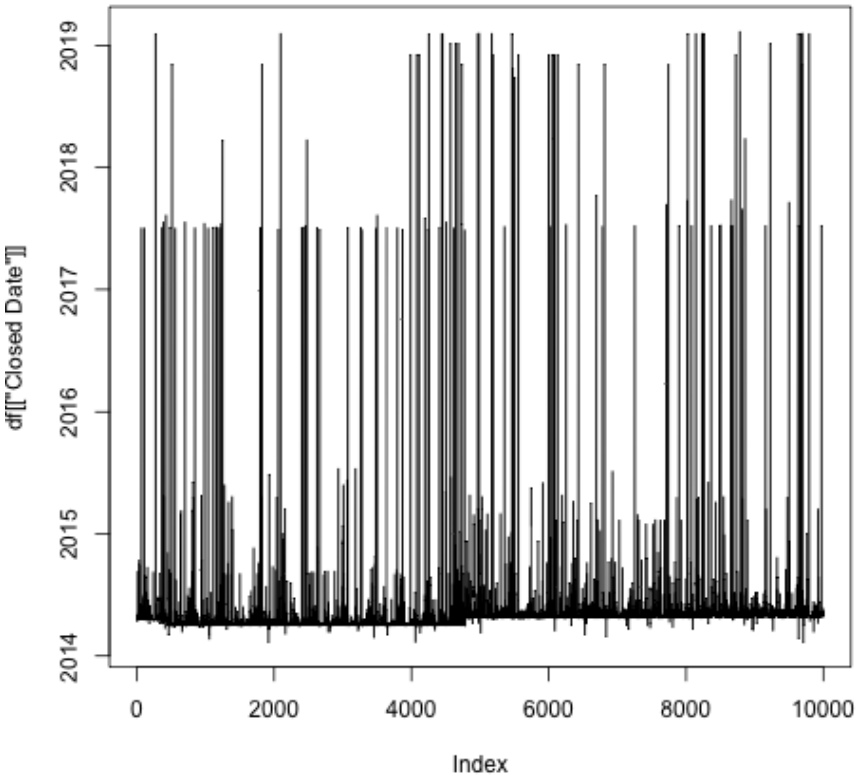


Figura 15-2. Embora a maioria das datas nas primeiras 10.000 linhas de código pareçam cair 2014, eles também avançam até 2019 – e com frequência!

Não há como evitar o problema de agir. Se quisermos compreender como o comportamento muda ao longo do tempo, teremos de confrontar os dados não ordenados. Mas temos algumas opções.

Faça um upsample e agregue os dados à medida que os iteramos

Uma opção é aumentar a resolução dos dados à medida que os iteramos para construir uma série temporal condensada com estatísticas agregadas. Poderíamos escolher uma resolução e contagens de agregação desde o início de nossa análise e depois calculá-las à medida que iteramos.

Então poderíamos classificar nossos resultados no final de nossa análise. Isso seria algo como ter, digamos, um dicionário/lista de todas as datas de 2010 até o presente e depois adicionar a data apropriada para cada linha. Este método produziria uma lista/dicionário com um número

relativamente pequeno de entradas e poderíamos então classificá-la no final com base na data.

A vantagem disso é que seria relativamente simples de codificar e seria uma forma de combinar limpeza, exploração e análise de dados em uma etapa exploratória.

A desvantagem é que os dados detalhados ainda seriam perdidos no pântano do arquivo não classificado, de modo que, se houvesse um período específico de interesse, teríamos que pesquisar todo o arquivo não classificado para encontrar todas as entradas relevantes.

Como já fizemos exemplos de upsampling no Capítulo 2, deixo isso como uma

exercício para o leitor.

Classifique os dados

Outra opção é classificar os dados. Esta é uma tarefa difícil, dado o grande tamanho do arquivo e as datas relativamente desordenadas do que podemos observar. Mesmo para os dados de 2014 não parece que as datas estejam em ordem. Por esta razão, não temos indicações de que possamos confiar em qualquer fatia dos dados, por isso devemos pensar nisso como uma pilha de dados em ordem aleatória.

Classificar o arquivo completo consumiria muita memória, mas há dois motivos pelos quais isso pode valer a pena. Uma delas é que só precisamos classificar uma vez e então podemos salvar nossos resultados para qualquer análise subsequente que desejarmos fazer. A segunda é que podemos então preservar todo o nível de detalhe para que, se identificarmos períodos de tempo específicos de interesse na nossa análise, possamos examinar os dados em todos os seus detalhes para compreender o que está a acontecer.

Concretamente, pensando em como poderíamos conseguir isso, temos algumas opções:

• O Linux possui uma ferramenta de linha de comando para classificação.

• A maioria dos bancos de dados pode classificar dados, então poderíamos transferir os dados para um banco de dados e deixar que o banco de dados cuide disso. 7

• Poderíamos criar nosso próprio algoritmo de classificação e implementá-lo. Precisaríamos formular algo que não consumisse enormes quantidades de memória. As probabilidades são fortemente contra o nosso esforço de combinar o que está disponível nas opções de classificação pré-embaladas.

Optamos por usar a ferramenta de linha de comando do Linux. Embora possa levar algum tempo para acertar, desenvolveremos uma nova habilidade para a vida, além de obter acesso a uma classificação correta e bem implementada para esse arquivo grande.

Começamos criando primeiro um pequeno arquivo de teste para usar:

linha de comando do linux

$ head -n 1000 311.csv | cauda -n 999 > teste.csv

Observe que isso envolve os comandos head (ou seja, imprimir o início) e tail (ou seja, imprimir o final). head incluirá a primeira linha do arquivo, que para este arquivo fornece os nomes das colunas. Se incluirmos isso e classificarmos junto com os valores, isso não preservará os nomes das colunas como a linha superior do arquivo, portanto, cortamos antes de classificar.

Se você estiver trabalhando em um sistema operacional baseado em Linux, poderá aplicar o comando sort da seguinte maneira:

7 Teríamos que escolher um que oferecesse essa funcionalidade – nem todos os bancos de dados oferecem, e especialmente nem todos os bancos de dados de séries temporais, que tendem a presumir que os dados chegam ao banco de dados em ordem cronológica.

linha de comando do linux

$ sort --field-separator=',' --key=2,3 test.csv > testsorted.csv

Nesse caso, identificamos o separador de campos e indicamos que queremos ordenar pela segunda e terceira colunas - ou seja, pela data de criação e data de fechamento (que só sabemos pela inspeção prévia do arquivo). Nós enviamos isso para um novo arquivo, pois não seria muito útil ter a saída padronizada.

Agora podemos inspecionar o arquivo classificado em R, mas infelizmente descobriremos que isso também não retorna um arquivo classificado. Volte algumas páginas até onde executamos o comando head no arquivo CSV e você verá por quê. Classificamos de acordo com uma coluna de data (que seria processada como uma string, não com reconhecimento de data). No entanto, a formatação atual das datas começa com o mês, portanto terminaremos com uma coluna de datas por mês, em vez de ordenadas por tempo geral, como vemos quando revisamos a coluna "classificada".

CSV resultante do nosso comando anterior:

1: 02/02/2019 10:00:27 06/02/2019 07:44:37

2: 27/03/2014 07:38:15 01/04/2014 12:00:00

3: 27/03/2014 11:07:31 28/03/2014 13:09:00

4: 28/03/2014 06:35:13 28/03/2014 22:47:00

5: 28/03/2014 08:31:38 28/03/2014 10:37:00

995: 03/07/2017 12:40:04 03/07/2017 15:40:02

996: 04/07/2017 01:30:35 04/07/2017 02:50:22

997: 03/09/2014 15:32:57 04/09/2014 16:30:30

998: 05/09/2014 11:17:53 08/09/2014 15:37:25

999: 06/11/2018 19:15:28 06/11/2018 20:17:52

Como podemos ver, a classificação faz sentido como uma classificação por string, em vez de uma classificação por data. Na verdade, isso aponta para uma das virtudes do uso da formatação de data ISO adequada, que é que a classificação ainda será correta ao classificar como uma string, ao contrário do formato anterior.

Este é um exemplo de um problema muito comum com dados de séries temporais "encontrados": o formato de carimbo de data/hora disponível pode não ser o mais adequado para a análise de séries temporais.

Revisitamos a interface NYC Open Data para ver se há uma solução alternativa para essa formatação (veja a Figura 15-3).

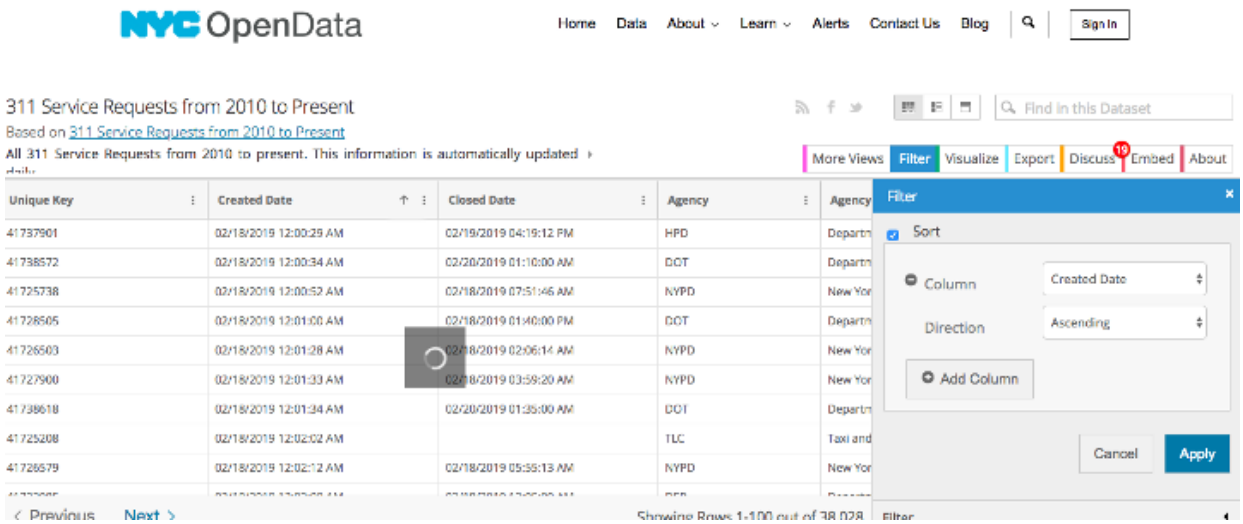


Figura 15-3. O portal NYC Open Data oferece classificação por meio de uma interface web, que parece estar disponível para qualquer coluna neste grande conjunto de dados. Este

é um impressionante grátis recurso quando você pensa sobre o poder computacional necessário para esse tipo.

Uma olhada na visualização tabular dos dados na página da web parece concordar com nosso CSV, pois os dados não parecem estar classificados em ordem de tempo. No entanto, vemos que existe uma opção de classificação, então aplicamos isso ao conjunto de dados, que de fato atualiza os dados após uma espera compreensível enquanto o grande conjunto de dados é armazenado. Esta seria uma ótima solução, exceto que, infelizmente, o CSV resultante baixado ainda está fora de serviço.

Há uma série de outras soluções que poderíamos explorar neste momento. Poderíamos ver se o uso da API do Open Data em vez da interface da web fornece datas mais digeríveis ou uma forma de garantir a classificação. Poderíamos trabalhar com mais Linux

ferramentas de linha de comando, como o awk, para extrair as diferentes partes do carimbo de data/hora em colunas diferentes ou em uma única coluna reorganizada com formatação ISO.

Em vez disso, adotamos uma abordagem simplificada para ver se nossas ferramentas disponíveis podem lidar com esse CSV se lermos apenas determinadas colunas. A primeira questão que me interessa sobre este conjunto de dados é como o intervalo entre a criação de uma reclamação 311 e o encerramento dessa reclamação pode ter variado ao longo do tempo. Nesse caso, suponho que preciso de apenas duas colunas: CriadoDate e ClosedDate. Verei se apenas ler essas duas colunas, que são uma pequena parte de todas as colunas em termos de contagem e contagem de caracteres (porque algumas colunas são bastante longas), é possível em meu laptop leve. (Eu também poderia explorar a maneira preguiçosa de resolver o problema, que seria uma atualização de hardware, temporária ou permanentemente.)

Portanto, agora podemos ler todas as linhas dos dados, e nossa análise subsequente será no conjunto de dados completo, em vez de apenas nas primeiras 1.000 linhas:

##R

```
> ## leia apenas as colunas de interesse
```



```
> df = fread("311.tsv", select = c("Data de criação", "Data
de fechamento"))

>

> ## use o paradigma 'set' recomendado do data.table para
definir nomes de colunas

> setnames(df, gsub(" ", "", colnames(df)))

>

> ## eliminar linhas com um campo de data em branco

> df = df[nchar(DataCriada) > 1 & nchar(DataFechada) >
1]

>

> ## converte colunas de data de string para POSIXct

> fmt.str = "%m/%d/%Y %I:%M:%S %p"

> df[, Data de criação: = as.POSIXct (Data de criação,
formato = fmt.str)]

> df[, ClosedDate := as.POSIXct(ClosedDate, formato =
fmt.str)]

>

> ## ordem em ordem cronológica de CriadoDate

> setorder(df,DataCriada)

>

> ## calcule o número de dias entre a criação e o
fechamento
```

> *## uma reclamação 311*

> df[, LagTime := as.numeric(difftime(ClosedDate, CriadoDate,

> unidades = "dias"))]

Isso foi feito em um laptop leve fabricado em 2015, então é provável que você também possa fazer isso com tudo o que usa no trabalho ou em casa. Aqueles que são novos em “big data” podem se surpreender com o fato de 19 milhões de linhas não serem grande coisa, mas geralmente é o caso. Na verdade, precisamos apenas de uma pequena fatia dos dados para abordar uma questão relevante de série temporal.

Quando olhamos para a coluna LagTime, notamos alguns números surpreendentemente incorretos

—numeração em dezenas de milhares de dias, ou dias negativos. Eliminamos esses números e limitamos os dados, impulsionados em parte por uma distribuição de uma amostra aleatória de pontos de dados:

##R

> resumo(df$LagTime)

Min. 1º Qu. Mediana Média 3º Qu. Máx. **NA**

-42943,4 0,1 2,0 2,0 7,9 368961,1 609835

> nrow(df[LagTime < 0]) / nrow(df)

[1] 0,01362189

> nrow(df[LagTime > 1000]) / nrow(df)

[1] 0,0009169934

> df = df[LagTime <1000]

> df = df[LagTime > 0]

```
> df.new = df[seq(1, nrow(df), 2), ]

> write.csv(df.new[order(ClosedDate)], "abridged.df.csv")
```

Descartamos dados com tempos de defasagem negativos, pois não temos documentação ou conhecimento de domínio para saber o que são. Também rejeitamos dados que consideramos irrealistas **Explorando grandes dados de séries temporais | 435**



ou valores extremos inúteis, descartando dados em que o tempo de atraso para encerrar a reclamação 311 foi superior a 1.000 dias. 8

Tenha cuidado ao descartar dados. Neste exercício, descartamos

cerca de 1,3% dos dados devido a tempos de atraso na resolução de um problema 311

reclamação que não fazia sentido, seja porque demoraram tanto a

justificam uma explicação que não conseguiríamos obter ou porque eles

foram negativos, implicando um erro de entrada de dados ou algum outro problema

além do que provavelmente poderíamos descobrir apenas com os dados.

Dado que a nossa questão diz respeito à distribuição global do

dados, é improvável que uma porção tão pequena de pontos afete

nossa análise sobre uma questão distributiva. No entanto, na realidade

mundo, você gostaria de investigar esses pontos de dados e as possíveis

possíveis efeitos posteriores em sua tarefa de análise. Este não é o momento

conselhos específicos da série, mas apenas uma questão de prática geral.

Agora que somos capazes de manter todos os dados de interesse na memória ao mesmo tempo, poderíamos fazer perguntas sobre a série de forma holística. Contudo, a questão de interesse é se e como a distribuição do tempo de atraso pode ter mudado ao longo do tempo. Poderíamos fazer isso com uma janela deslizante ou rolante ao longo da série temporal, mas isso é computacionalmente desgastante; teríamos que fazer muitos cálculos relacionados repetidamente enquanto deslizamos uma janela sobre os dados. Também queremos explorar a possibilidade de fazer isso de uma forma que também funcione em uma transmissão ao vivo, já que poderíamos imaginar a continuação deste projeto nos dados atuais à medida que eles chegam. Seria melhor não ter que armazenar esse arquivo de dados de vários gigabytes indefinidamente.

Análise estatística on-line de dados de séries temporais

Usaremos uma ferramenta online bastante simples de estimativa de quantil chamada

Algoritmo P-quadrado , com uma ligeira modificação para torná-lo consciente do tempo. O algoritmo original presumia que havia uma distribuição estável a partir da qual os quantis estavam sendo inferidos, mas queremos levar em conta o caso de distribuições que mudam ao longo do tempo. Tal como acontece com uma média móvel ponderada exponencialmente, adicionamos esta consciência de tempo ponderando menos as observações anteriores, e fazemos

isso da mesma maneira, introduzindo um fator para reduzir os pesos das medições anteriores cada vez que um novo

a medição está disponível (veja a Figura 15-4).

8 Embora 1.000 dias possam parecer surpreendentemente elevados, na verdade, eu pessoalmente fiz várias 311 reclamações que não foram resolvidas no mesmo número de dias. Há anos que espero que Nova York substitua a árvore em frente à minha casa depois que a antiga morreu - toda vez que ligo para o 311, sou informado de que eles estão trabalhando nisso.

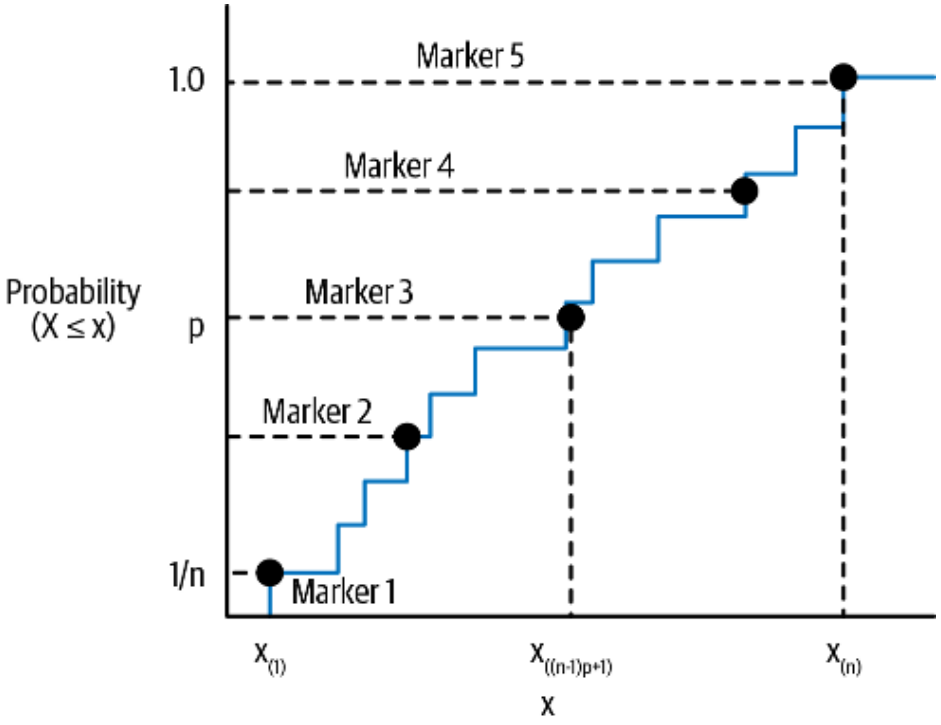


Figura 15-4. A estrutura computacional de nossa estimativa de quantil online usando o Algoritmo P-quadrado. Mantemos uma série de marcadores indicando onde pensamos que o são os quantis e quais são as contagens cumulativas para todos os pontos de dados menores ou iguais para cada um desses quantis.

O algoritmo requer um pouco de manutenção de registros, tornando-o mais fácil de implementar em uma linguagem de

programação mais orientada a objetos, por isso mudaremos de R para Python. Observe, entretanto, que usaremos os dados pré-processados do R, pois o pacote data.table tem desempenho substancialmente melhor com esse tipo de big data do que as ferramentas disponíveis em Python.

A versão do algoritmo P-quadrado que implementamos forma um histograma dos valores.

Portanto, à medida que criamos um objeto PQuantile, alocamos um número predefinido de compartimentos para nossas contagens de histograma, posições de compartimento e registros contínuos de observações:

```
## Pitão

>>> ## importações e um utilitário lambda

>>> importar bissectar

>>> importar matemática

>>>

>>> sinal = lambda x: (1, -1)[x < 0]

>>>

>>> ## início da definição da classe

>>> aula PQuantil :

>>> def __init__ (self, b, fator_desconto):

>>> ## inicialização

>>> self.num_obs = 0 ## autoexplicativo

>>> ## contagens por quantil

>>> self.n = [i para i no intervalo(self.b+1)]

>>> self.q = [] ## os valores quantis estimados
```

>>>


```
>>> ## b é o número de quantis,

>>> ## incluindo o 0 e o 100

>>> ## (valores mínimo e máximo) quantil

>>> eu.b = b

>>> ## o fator de desconto define como ajustamos as contagens anteriores

>>> ## quando novos dados estiverem disponíveis

>>> self.fator_desconto = fator_desconto
```

Existem dois parâmetros configuráveis: o número de quantis uniformemente espaçados para estimar e o fator de desconto para observações antigas.

Os outros membros da classe incluem um total acumulado do número de observações (isso estará sujeito ao desconto de tempo pelo fator de desconto de tempo configurável), os quantis estimados e a contagem contínua de observações menores ou iguais a um determinado valor de quantil.

Existe apenas uma função pública, cujo papel é aceitar a próxima observação. Quando chega uma nova observação, o que acontece é resultado de quão cedo estamos na série.

Para os primeiros valores de self.b, as entradas são aceitas e necessariamente compõem a estimativa do quantil. self.q é classificado de modo que seus valores reflitam os quantis.

Então, por exemplo, imagine que você insira ab = 5 para o número de valores quantis desejados e depois insira a sequência 2, 8, 1, 4, 3. No final desta sequência, self.q seria igual a [1, 2, 3, 4, 8]. self.n, as contagens de valores menores ou iguais a cada um dos quantis, seriam iguais a

[1, 2, 3, 4, 5], o valor para o qual já foi inicializado em __init__:

Pitão

>>> **def** next_obs(self, x):

>>> **se** self.num_obs <(self.b + 1):

>>> self.q.append(x)

>>> self.q.sort()

>>> self.num_obs = self.num_obs + 1

>>> **mais** :

>>> self.next_obs2(x)

>>> self.next_obs = self.next_obs2

As coisas ficam interessantes quando você tem mais do que valores self.b. Neste ponto, o código começa a tomar decisões sobre como combinar valores para estimar quantis sem manter todos os pontos de dados armazenados para análises repetidas. Neste caso, o algoritmo P-quadrado faz isso com o que chamamos de self.next_obs2:

Pitão

>>> **def** next_obs2(self, x):

>>> *## descontando o número de observações*

>>> **se** self.num_obs > self.b * 10:

>>> obs_corrigidos = max(self.fator_desconto * self.num_obs,

>>> eu.b)

>>> self.num_obs = corrigido_obs + 1

```
>>> self.n = [math.ceil(nn * self.discount_factor)
```



```
>>> para nn em si mesmo.n]
>>>
>>> para i no intervalo(len(self.n) - 1):
>>> se self.n[i + 1] - self.n[i] == 0:
>>> eu.n[i+1] = eu.n[i + 1] + 1
>>> elif self.n[i + 1] <self.n[1]:
>>> ## na prática isso não parece acontecer
>>> self.n[i + 1] = self.n[i] - self.n[1 + 1] + 1
>>> mais :
>>> self.num_obs = self.num_obs + 1
>>>
>>> k = bisect.bisect_left(self.q, x)
>>> se k for 0:
>>> self.q[0] = x
>>> elif k é self.b+1 :
>>> eu.q[-1] = x
>>> k = self.b
>>> se k for não 0:
>>> k = k - 1
```

```
>>>

>>> self.n[(k+1):(self.b+1)] = [self.n[i] + 1

>>> para i no intervalo((k+1),

>>> (self.b+1))]

>>> para i no intervalo(1, self.b):

>>> np = (i)*(self.num_obs - 1)/(self.b)

>>> d = np - self.n[i]

>>> se (d >= 1 e (self.n[i+1] - self.n[i]) > 1):

>>> self._update_val(i, d)

>>> elif (d <= -1 e (self.n[i-1] - self.n[i]) < -1):

>>> self._update_val(i, d)
```

Idealmente, o *i* -ésimo valor do quantil deve ser espaçado uniformemente de modo que exatamente *i/b* × observações totais sejam menores que ele. Se este não for o caso, o marcador é movido uma posição para a esquerda ou para a direita, e o seu valor quantil associado é modificado com uma fórmula derivada da presunção de uma forma localmente parabólica do histograma. Essa fórmula dita os critérios indicados anteriormente para dimensionar a variável d para determinar se um determinado valor e contagem de quantil devem ser ajustados.

Se o valor tiver que ser ajustado, há outra decisão a tomar sobre se o ajuste parabólico ou linear é apropriado. Isto é implementado a seguir

código. Para mais detalhes, veja a derivação no artigo original. Esse papel é ótimo

por causa da matemática acessível usada na técnica e também porque fornece instruções muito claras sobre como implementar o método e depois testar sua implementação:

Pitão

>>> *## atualização geral*

>>> *## como você pode ver, tanto self.q quanto self.n estão atualizados*

>>> *## à medida que uma posição quantílica é deslocada*

>>> **def** _update_val(self, i, d):

>>> d = sinal(d)

>>> qp = self._adjust_parabolic(i, d)

>>> **se** self.q[i] < qp < self.q[i+1]:

>>> self.q[i] = qp

>>> **mais** :

>>> self.q[i] = self._adjust_linear(i, d)

>>> eu.n[i] = eu.n[i] + d

>>>

>>> *## este é o método de atualização principal*

>>> **def** _adjust_parabolic(self, i, d):

>>> novo_val = self.q[i]

>>> m1 = d/(self.n[i+1] - self.n[i-1])

>>> s1 = (self.n[i] - self.n[i-1] + d) *

>>> (self.q[i+1] - self.q[i]) /

```
>>> (self.n[i+1] - self.n[i])
>>> s2 = (self.n[i+1] - self.n[i] - d) *
>>>
>>> ## este é o ajuste linear de backup quando parabólico
>>> ## condições não foram atendidas
>>> def _adjust_linear(self, i, d):
>>> novo_val = self.q[i]
>>> novo_val = novo_val + d * (self.q[i + d] - self.q[i]) /
>>> (self.n[i+d] - self.n[i])
>>> retornar novo_val
```

Para uma perspectiva da simplicidade deste método, todo o código da classe está listado aqui em um só lugar:

```
## Pitão
>>> aula PQuantil :
>>> ## INICIALIZAÇÃO
>>> def __init__ (self, b, fator_desconto):
>>> self.num_obs = 0
>>> eu.b = b
>>> self.fator_desconto = fator_desconto
>>> self.n = [i para i no intervalo(self.b+1)]
>>> eu.q = []
>>>
```

```
>>> ## ENTRADA DE DADOS
>>> def next_obs(self, x):
>>> se self.num_obs <(self.b + 1):
>>> self.q.append(x)
>>> self.q.sort()
>>> self.num_obs = self.num_obs + 1
>>> mais :
>>> self.next_obs2(x)
>>> self.next_obs = self.next_obs2
>>>
>>> def next_obs2(self, x):
>>> ## descontando o número de observações
>>> se self.num_obs > self.b * 10:
>>> obs_corrigidos = max(self.fator_desconto
>>> *self.num_obs,
>>> eu.b)
```

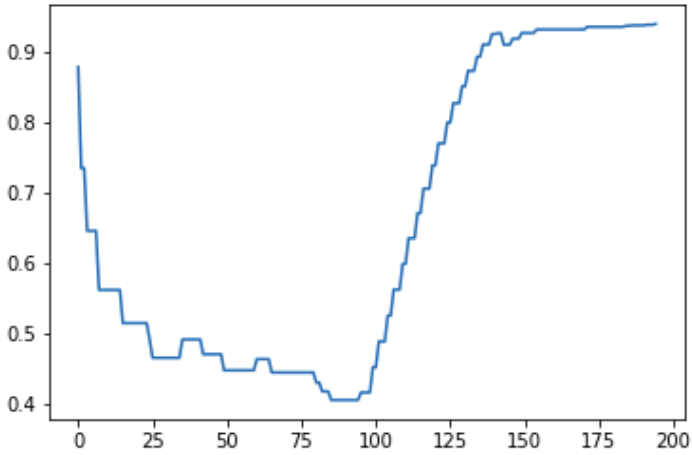


```
>>> self.num_obs = corrigido_obs + 1
>>> self.n = [math.ceil(nn * self.discount_factor)
>>> para nn em si mesmo.n]
>>>
```

```
>>> para i no intervalo(len(self.n) - 1):
>>> se self.n[i + 1] - self.n[i] == 0:
>>> eu.n[i+1] = eu.n[i + 1] + 1
>>> elif self.n[i + 1] <self.n[1]:
>>> ## na prática isso não parece acontecer
>>> self.n[i + 1] = self.n[i] - self.n[1 + 1] + 1
>>> mais :
>>> self.num_obs = self.num_obs + 1
>>>
>>> k = bisect.bisect_left(self.q, x)
>>> se k for 0:
>>> self.q[0] = x
>>> elif k é self.b+1 :
>>> eu.q[-1] = x
>>> k = self.b
>>> se k for não 0:
>>> k = k - 1
>>>
>>> self.n[(k+1):(self.b+1)] = [self.n[i] + 1
>>> para i no intervalo((k+1),
>>> (self.b+1))]
```

```
>>> para i no intervalo(1, self.b):
>>> np = (i)*(self.num_obs - 1)/(self.b)
>>> d = np - self.n[i]
>>> se (d >= 1 e (self.n[i+1] - self.n[i]) > 1):
>>> self._update_val(i, d)
>>> elif (d <= -1 e (self.n[i-1] - self.n[i]) < -1):
>>> self._update_val(i, d)
>>>
>>> ## AJUSTES DO HISTOGRAMA
>>> def _update_val(self, i, d):
>>> d = sinal (d)
>>> qp = self._adjust_parabolic(i, d)
>>> se self.q[i] < qp < self.q[i+1]:
>>> self.q[i] = qp
>>> mais :
>>> self.q[i] = self._adjust_linear(i, d)
>>> eu.n[i] = eu.n[i] + d
>>>
>>> def _adjust_parabolic(self, i, d):
>>> novo_val = self.q[i]
>>> m1 = d/(self.n[i+1] - self.n[i-1])
```

```
>>> s1 = (self.n[i] - self.n[i-1] + d) *
>>> (self.q[i+1] - self.q[i]) /
>>> (self.n[i+1] - self.n[i])
>>> s2 = (self.n[i+1] - self.n[i] - d) *
>>> (self.q[i] - self.q[i-1]) /
>>> (self.n[i] - self.n[i-1])
```


```
>>> novo_val = novo_val + m1 * (s1 + s2)
>>> retornar novo_val
>>>
>>> def _adjust_linear(self, i, d):
>>> novo_val = self.q[i]
>>> novo_val = novo_val + d * (self.q[i + d] - self.q[i]) /
>>> (self.n[i+d] - self.n[i])
```

```
>>> retornar novo_val
```

Agora que temos esse método orientado ao tempo, devemos nos convencer de que ele funciona razoavelmente bem com um exemplo de brinquedo. Primeiro tentamos amostrar pontos de dados de uma distribuição e depois mudar abruptamente para outra. Em cada caso, estamos amostrando para o percentil 40, embora com base na configuração de 10 pontos do histograma, estamos mantendo um histograma que indica os percentis 0, 10, 20,...90, 100. Isso é útil porque significa que podemos ter uma descrição bastante detalhada da distribuição variável. Para este exemplo de brinquedo, focamos apenas no 40º percentil (qt.q[4]), o que resulta no gráfico da Figura 15-5:

```
## Pitão

>>> qt = PQuantil(10, 0,3 )

>>> qt_ests = []

>>>

>>> para _ no intervalo (100):

>>> b.next_obs(uniforme())

>>> se len(bq) > 10:

>>> qt_ests.append(qt.q[4])

>>> para _ no intervalo (100):

>>> b.next_obs(uniforme(baixo = 0,9))

>>> qt_ests.append(qt.q[4])

>>>

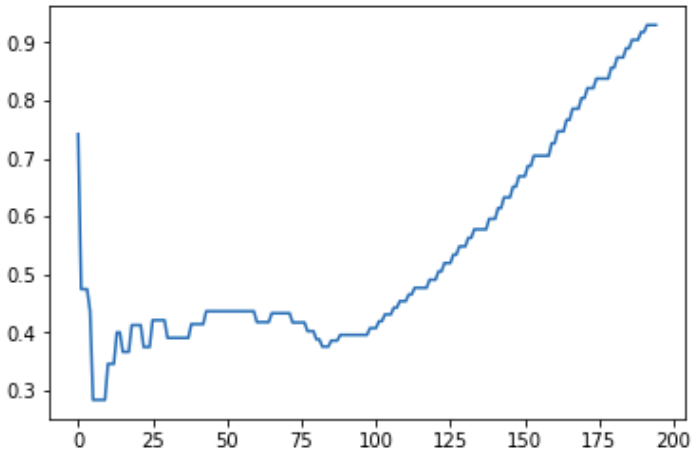
>>> plt.plot(qt_ests)
```

Figura 15-5. Quando descontamos pesadamente as medições mais antigas (multiplicando por um valor menor

fator de desconto), vemos mais rapidamente que a distribuição subjacente mudou.

Em contraste, vemos uma adoção mais lenta do quantil variável no caso de um número maior.

fator de desconto (ver Figura 15-6):



Pitão

```
>>> qt = PQuantil(10, 0,8 )
>>> qt_ests = []
>>>
>>> para _ no intervalo (100):
>>> b.next_obs(uniforme())
>>> se len(bq) > 10:
>>> qt_ests.append(qt.q[4])
>>> para _ no intervalo (100):
>>> b.next_obs(uniforme(baixo = 0,9))
```

```
>>> qt_ests.append(qt.q[4])

>>>

>>> plt.plot(qt_ests)
```

Figura 15-6. Quando descontamos menos as medidas mais antigas (multiplicando por um fator de desconto maior), nossa estimativa de quantil demora mais para reconhecer que a distribuição subjacente mudou.

Agora aplicamos esse quantil contínuo a um subconjunto de nossos dados (veja a Figura 15-7). Nós fazemos

Não fiz todo o conjunto de dados, não por causa do desafio computacional, mas porque era muito cansativo para meu laptop de trabalho representar graficamente todos os quantis registrados!

Pitão

```
>>> importar entorpecido

>>> nrows = 1000000

>>> qt_est1 = np.zeros(nrows)

>>> qt_est2 = np.zeros(nrows)

>>> qt_est3 = np.zeros(nrows)

>>> qt_est4 = np.zeros(nrows)

>>> qt_est5 = np.zeros(nrows)

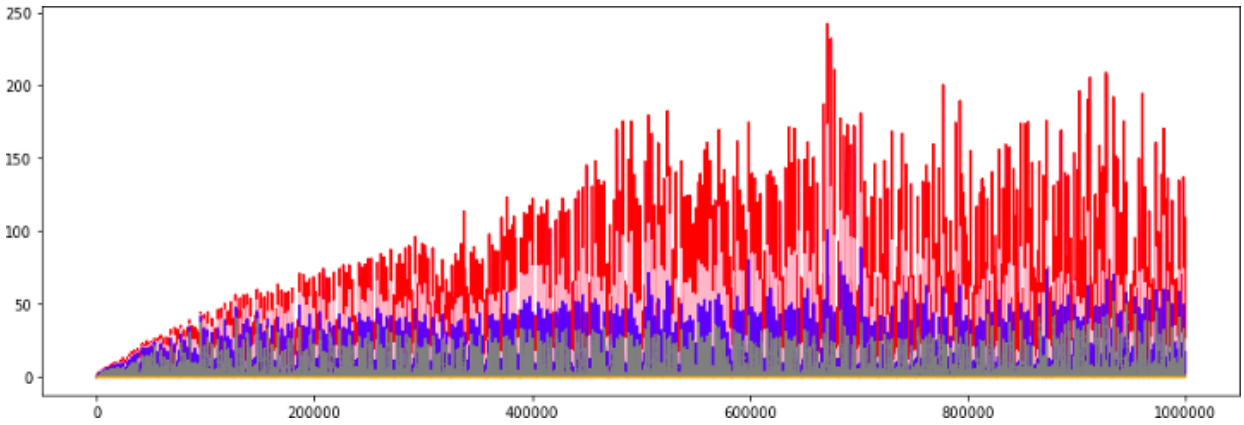
>>> qt_est6 = np.zeros(nrows)

>>> qt_est7 = np.zeros(nrows)

>>> qt_est8 = np.zeros(nrows)

>>> qt_est9 = np.zeros(nrows)
```

```
>>> para idx, val em enumerate(df.LagTime[:nrows]):
>>> qt.next_obs(val)
>>> se len(qt.q) > 10:
>>> qt_est1[idx] = qt.q[1]
```


```
>>> qt_est2[idx] = qt.q[2]
>>> qt_est3[idx] = qt.q[3]
>>> qt_est4[idx] = qt.q[4]
>>> qt_est5[idx] = qt.q[5]
>>> qt_est6[idx] = qt.q[6]
>>> qt_est7[idx] = qt.q[7]
>>> qt_est8[idx] = qt.q[8]
>>> qt_est9[idx] = qt.q[9]
>>>
>>> plot(qt_est9, color = 'vermelho')
```

```
>>> plt.plot(qt_est7, color = 'rosa')

>>> plt.plot(qt_est5, cor = 'azul')

>>> plt.plot(qt_est3, color = 'cinza')

>>> plt.plot(qt_est2, color = 'laranja'
```

Figura 15-7. Os valores dos percentis 90, 70, 50, 30 e 20 ao longo do tempo para o primeiras 100.000 linhas no conjunto de dados quando classificado por data de fechamento.

A Figura 15-7 mostra os valores dos percentis 90, 70, 50, 30 e 20 plotados sobre

hora para as primeiras 100.000 linhas no conjunto de dados quando classificado por data de fechamento. Como foram classificadas por data de encerramento, é provável que muitas reclamações 311 resolvidas rapidamente tenham sido antecipadas, o que explica as estimativas de quantis muito menores na primeira parte do conjunto de dados. 9

Então a distribuição mudou? Visualmente parece que sim, por alguns motivos. Uma delas é a censura à esquerda, descrita brevemente, que reflete como classificamos e selecionamos os dados. O fato de ordenarmos os dados pela coluna ClosedData, combinado com o fato de que esse conjunto de dados não parece ter lookback infinito (ou seja, presumivelmente 311 reclamações feitas antes de uma determinada data não chegaram a este sistema), faz com que o os tempos de atraso que vemos nas datas iniciais parecem mais curtos. Em outras palavras, esse aparente percentil 9.20 é tão pequeno que não pode realmente ser visto quando plotado na escala dos outros quantis aqui apresentados. Ela aparece como uma linha horizontal sólida na base das outras distribuições, mas você pode vê-la melhor mudando para escala logarítmica ou plotando-a sozinha.

a mudança ao longo do tempo é simplesmente um artefato de nossos dados incompletos (e um conjunto de dados

subjacente incompleto) combinado com nossa escolha de classificação.

Por outro lado, podemos ver características que sugerem mudanças nas distribuições ao longo do tempo. Parece haver altos e baixos nas estimativas da curva quantílica, e podemos até considerar se pode haver comportamento periódico em nossas curvas, pois pode haver momentos previsíveis em que os valores quantílicos sobem e descem devido a fatores organizacionais exógenos (talvez um impulso para encerrar reclamações antes do final do mês, ou certos ciclos de financiamento que aumentam o número de trabalhadores disponíveis para encerrar reclamações em determinadas épocas do ano).

Dado que agora temos resultados preliminares, a melhor opção seria estabelecer algumas datas significativas (ou seja, onde vemos picos ou comportamento periódico?) e tentar cruzá-las com quaisquer factos institucionais que possamos estabelecer sobre o ritmo de trabalho. . Deveríamos também realizar simulações para avaliar como pensamos que a censura à esquerda deveria afetar as estimativas iniciais dos quantis em diferentes cenários. Desta forma, poderíamos desenvolver uma melhor compreensão qualitativa e quantitativa dos aspectos desconhecidos do nosso sistema, e esta informação seria extremamente útil para fazer uma determinação final sobre se a distribuição dos tempos de resolução está a evoluir ao longo do tempo e, em caso afirmativo, como.

Digamos que tomamos essas medidas – e depois? Precisaríamos procurar uma metodologia para comparar distribuições para diferenças de similaridade onde apenas os quantis da distribuição, e não todos os pontos da amostra, estivessem disponíveis. Uma maneira de fazer isso seria executar simulações/inicializar todo o processo. Isso levaria a uma resposta onde poderíamos articular e controlar totalmente as suposições incluídas no modelo, codificando nossas simulações. Na verdade, muitas abordagens estatísticas para fazer tais comparações também se concentram em métodos bootstrap.

Censura à esquerda, censura à direita e censura de intervalo

Para análise de séries temporais e atividades relacionadas, como análise de sobrevivência, devemos estar cientes de como nossa seleção, classificação ou apresentação de dados pode afetar nossa análise, como determinamos que era o caso aqui, em que os períodos de tempo anteriores parecem ter uma influência muito maior. distribuição mais estreita do que os períodos posteriores. Provavelmente, isso se deve ao fato de termos classificado os dados por data de fechamento, uma forma de *censura à esquerda* , o que significa que o

“evento de interesse” já ocorreu antes do início do “estudo”. Nesse caso, nosso evento de interesse são as datas de início e término das reclamações, e estamos limitados pela data de início do conjunto de dados. Em contraste, o conceito relacionado de *censura à direita* significa que o evento de interesse ocorreria após o término de um evento. Por fim, *a censura intervalar* refere-se a situações em que temos apenas um registro inexato, como um intervalo, de quando ocorreu um evento, o que necessariamente também afetará nossa análise.

Perguntas restantes

Nossa visualização sugere novas consultas. Uma delas diz respeito à possibilidade de comportamento cíclico ou sazonal. Parece haver solavancos periódicos em todos os quantis estimados.

Poderíamos considerar uma investigação mais aprofundada sobre isso e temos várias opções para fazer isso:

• Poderíamos tentar ajustar harmônicos (senos e cossenos) a essas curvas quantílicas e ver se emergia uma periodicidade comum.

• Poderíamos modelar o próprio quantil como um processo ARIMA ou SARIMA e observar

para evidência de sazonalidade. Isto também implicaria etapas preliminares, como explorar o ACF e o PACF das

curvas que modelaríamos como séries temporais.

• Poderíamos pedir mais informações à agência que administra o serviço 311 e ver se ela reconhece algum comportamento periódico induzido pela sua estrutura organizacional e procedimentos operacionais.

Além do comportamento periódico, também podemos ver um salto nos valores estimados do quantil logo abaixo do local índice = 70.000. Dado que todos os quantis saltaram, parece improvável que isso se deva a apenas um ou a um punhado de valores discrepantes. Existem algumas maneiras de investigar isso:

• Volte aos dados brutos deste período e veja quais características podem sugerir uma explicação. Houve um aumento nas 311 reclamações? Ou um aumento repentino de um tipo específico de reclamação que tende a levar mais tempo para ser resolvido?

• Alternativamente, poderíamos revisitar os dados brutos para recuar a data aproximada deste salto nos quantis e fazer referência cruzada com as notícias locais, de preferência com a ajuda de alguém que possa nos indicar a direção certa. A assistência de alguém da agência ou de alguém com experiência no governo municipal pode ser muito valiosa. Também é possível, no entanto, que esta data corresponda a um grande evento em Nova Iorque que explicaria o salto, como a supertempestade Sandy em 2012.

Melhorias adicionais

Poderíamos tornar este algoritmo ainda mais consciente do tempo. Nossa modificação no algoritmo P-quadrado desconta as observações anteriores, mas assume que todas as observações estão espaçadas uniformemente. Isso se manifesta no fato de que a entrada da próxima observação não apresenta carimbo de data/hora, sendo sempre aplicado o mesmo fator de desconto. Poderíamos criar um algoritmo mais flexível usando a mudança no carimbo de data/hora das informações antigas em comparação com as novas informações, de modo que o desconto dependesse da mudança no tempo desde a medição da última atualização. Também seria mais preciso para nosso conjunto de dados 311. Isto é deixado como exercício para o leitor, mas

alterando algumas linhas de código. Dica: o fator de desconto deve passar a ser uma função do tempo.

Poderíamos também procurar outras formas de estimar quantis ao longo do tempo, seja com uma medição online ou de janela. Porque a importância dos dados online está crescendo—

e especialmente para big data online – há uma variedade de pesquisas emergentes sobre este tema. As abordagens estatística e de aprendizado de máquina lidaram com isso nos últimos anos, e há um bom número de artigos acadêmicos acessíveis aos cientistas de dados praticantes.

Mais recursos

Ted Dunning e Otmar Ertal, "Computando quantis extremamente precisos usando t-

Digests", artigo de pesquisa, 2019, https://perma.cc/Z2A6-H76H.

O algoritmo f-digest para cálculo extremamente eficiente e flexível de quantis de dados de séries temporais online está rapidamente ganhando força como uma técnica líder para lidar com estimativas de quantis de séries temporais online, mesmo para o caso de distribuições não estacionárias. As implementações estão disponíveis em diversas linguagens, incluindo Python e variações de C++ e Go de alto desempenho. Esta abordagem

é particularmente útil porque não há necessidade de decidir antecipadamente quais quantis lhe interessam - em vez disso, toda a distribuição é modelada como um conjunto de clusters a partir dos quais você pode inferir qualquer quantil que desejar.

Dana Draghicescu, Serge Guillas e Wei Biao Wu, "Estimativa de curva quantílica e

Visualização para séries temporais não estacionárias,” Jornal de Estatística Computacional e Gráfica 18, não. 1 (2009): 1–20, https://perma.cc/Z7T5-PSCB.

Este artigo ilustra vários métodos não paramétricos para modelar distribuições não estacionárias na estimativa de quantis de séries temporais. É útil porque abrange dados do mundo real, simulação e distribuições não padronizadas (como distribuições não gaussianas). Há também um código de amostra (incomum para um artigo de revista acadêmica de estatística), embora atrás de um acesso pago.

András A. Benczúr, Levente Kocsis e Róbert Pálovics, “Aprendizado de Máquina Online em

Fluxos de Big Data”, artigo de pesquisa, 2018, https://perma.cc/9TTY-VQL3.

Esta referência discute abordagens técnicas para uma variedade de tarefas comuns de aprendizado de máquina relacionadas a séries temporais. Particularmente interessante é a discussão do processamento de dados online para muitos tipos de aprendizado de máquina e dicas técnicas sobre paralelização de tarefas online.

Sanjay Dasgupta, “Algoritmos Online e Streaming para Clustering,” notas de aula, Ciência da Computação e Engenharia, Universidade da Califórnia em San Diego, primavera de 2008, https://perma.cc/V3XL-GPK2.

Embora não sejam específicas para dados de séries temporais, estas notas de aula fornecem uma visão geral do agrupamento não supervisionado para dados online. Essas notas são suficientes para você começar a reunir soluções potenciais para uma aplicação específica de série temporal.

Ruofeng Wen et al., “A Multi-Horizon Quantile Recurrent Forecaster”, artigo de pesquisa,

Novembro de 2017, https://perma.cc/22AE-N7F3.

Este artigo de pesquisa da Amazon fornece um exemplo de uso de informações quantílicas

informação a partir de dados como uma forma de treinar redes neurais recorrentes de forma eficaz para previsão de séries temporais. Os pesquisadores demonstram o treinamento eficaz de uma rede neural que pode produzir avaliações probabilísticas em vez de estimativas pontuais.

Este artigo é uma boa ilustração de outro caso de uso potencial para informações quantílicas, um recurso subutilizado na análise de séries temporais.

CAPÍTULO 16

Pacotes de série temporal

Nos últimos anos, tem havido uma série de pacotes e documentos lançados por grandes empresas de tecnologia relacionados à forma como lidam com o enorme número de séries temporais que coletam como organizações digitais com enormes bases de clientes, registros sofisticados, análises de negócios de ponta. e inúmeras necessidades de previsão e processamento de dados. Neste capítulo discutiremos algumas das principais áreas de pesquisa e desenvolvimento relacionadas a esses conjuntos de dados de séries temporais em constante expansão, especificamente: previsão em escala e detecção de anomalias.

Previsão em escala

Para muitas grandes empresas de tecnologia, lidar com séries temporais é um problema cada vez mais importante e que surgiu naturalmente dentro das suas organizações. Com o tempo, várias destas empresas responderam desenvolvendo pacotes de séries temporais inteligentes e automatizados especificamente direcionados para “previsões em escala”, porque eram necessárias muitas previsões numa ampla variedade de domínios. Veja como dois cientistas de dados do Google que desenvolveram o

pacote de previsão automatizada da empresa descreveram as circunstâncias que motivaram

avaliaram seu produto em uma postagem no blog de 2017 (ênfase adicionada):

A demanda por previsão de séries temporais no Google cresceu rapidamente junto com a empresa durante sua primeira década. *Várias necessidades comerciais e de engenharia levaram a uma infinidade de abordagens de previsão, a maioria dependente do apoio direto dos analistas.* O volume e a variedade das abordagens e, em alguns casos, a sua inconsistência, exigiram uma tentativa de unificar, automatizar e alargar os métodos de previsão e de distribuir os resultados através de ferramentas que pudessem ser implementadas de forma fiável em toda a empresa. Isto é, para uma tentativa de desenvolver métodos e ferramentas que facilitariam previsões precisas de séries temporais em grande escala no Google.

Há tantos dados relevantes e tanto para prever que seria extremamente caro e organizacionalmente desafiador integrar e empregar analistas suficientes **449**

para gerar todas as previsões de interesse organizacional. Em vez disso, estes pacotes passam para uma filosofia "suficientemente boa"; isto é, uma previsão razoavelmente boa é muito melhor do que não ter nenhuma previsão enquanto se espera pela previsão perfeita e habilmente elaborada por um especialista em séries temporais com conhecimento do domínio. A seguir, discutiremos duas previsões automatizadas

frameworks com mais detalhes, do Google e do Facebook.

Previsão industrial interna do Google

O Google divulgou algumas informações sobre sua ferramenta interna de previsão automatizada, que surgiu de um esforço liderado por vários cientistas de dados na divisão de infraestrutura de busca da empresa. A tarefa era escrever uma abordagem unificada para fazer previsões automatizadas em toda a organização. Como as tarefas eram automatizadas, isso significava que os resultados

tinham que ser seguros – eles não podiam sair muito dos trilhos e tinham que vir com alguma estimativa de incerteza na previsão. Além disso, como a equipa procurou uma solução amplamente aplicável, a metodologia teve de abordar problemas comuns em conjuntos de dados de séries temporais relacionados com humanos, tais como sazonalidade, dados em falta, feriados e comportamentos que evoluem ao longo do tempo.

A solução implementada no Google compreende três etapas interessantes relacionadas ao que abordamos nos capítulos anteriores:

1. Limpeza e suavização de dados automatizada e extensa

2. Agregação temporal e desagregação geográfica/conceitual dos dados 3. Combinação de previsões e geração de incerteza baseada em simulação

estimativas

Discutiremos cada etapa por vez.

Limpeza e suavização de dados automatizada e extensa

Na postagem não oficial do blog de ciência de dados do Google mencionada anteriormente, dois líderes do projeto interno indicaram que a limpeza e suavização de dados resolveram uma série de problemas:

Efeitos de dados imperfeitos

• Dados ausentes

• Detecção de valores discrepantes

• Mudanças de nível (como aquelas causadas por lançamentos de produtos ou mudanças repentinas, mas permanentes, de comportamento)

• Transformando dados

Dados imperfeitos são um fato da vida. Dados ausentes podem ocorrer devido a falhas técnicas.

Os valores discrepantes podem ter causas semelhantes ou ser “valores reais”, mas não valem a pena incluí- **los 450 | Capítulo 16: Pacotes de Séries Temporais**

em uma previsão se não for provável que se repitam. Mudanças de nível (*regime mudanças*) podem ocorrer por uma infinidade de razões, incluindo que o comportamento da linha de base está mudando drasticamente (mundo em evolução), o que está sendo medido está mudando drasticamente (produto em evolução) ou o que está sendo registrado está mudando drasticamente (exploração madeireira em evolução). Finalmente, os dados podem vir em distribuições que estão longe da

normalidade ou estacionariedade presumida em muitos modelos de séries temporais. O Google

abordagem aborda cada uma dessas imperfeições de dados por sua vez com automação

metodologias para detectar e “corrigir” esses problemas.

Efeitos relacionados ao calendário

• Sazonalidade anual

• Sazonalidade semanal (efeito do dia da semana)

• Feriados

O tratamento dos efeitos relacionados ao calendário foi particularmente complicado para uma organização como o Google, com operações e usuários em todo o mundo. A sazonalidade anual seria muito diferente em diferentes partes do mundo, particularmente em hemisférios opostos com os seus próprios padrões climáticos e também em culturas com diferentes calendários subjacentes. Como apontou a postagem do blog, às vezes o mesmo feriado pode ocorrer mais de uma vez em um ano civil (gregoriano). Da mesma forma, as “estações” sob as quais um determinado grupo de pessoas opera podem mudar dentro do ano do

calendário gregoriano, o que muitas vezes pode ocorrer porque o calendário islâmico tem uma periodicidade diferente do calendário gregoriano.

Agregação temporal e desagregação geográfica/conceitual

A equipe do Google descobriu que os dados semanais funcionavam bem para a maioria das previsões de interesse. Assim, depois de limpar os dados na etapa anterior, eles os agregaram em incrementos semanais para fins de fazer previsões. Nesse sentido, realizaram agregação temporal.

No entanto, a equipe também descobriu que era útil desagregar os dados, às vezes geograficamente, às vezes por categoria (como tipo de dispositivo) e às vezes por uma combinação de fatores (como região geográfica por tipo de dispositivo). A equipe descobriu que, em tais casos, era mais eficaz fazer previsões para as subséries desagregadas e depois reconciliá-las para produzir uma previsão global se tanto a subsérie quanto os séries globais

eram de interesse. Discutimos várias maneiras de ajustar séries temporais hierárquicas no Capítulo

ter 6 , e este método reflete a metodologia de começar com previsões de nível inferior e propagá-las para cima.

Combinando previsões e geração de estimativas de incerteza com base em simulação, o Google usa abordagens de conjunto, combinando vários resultados de modelos de previsão diferentes para gerar a previsão final. Isso é útil por vários motivos. Previsão do Google **em escala | 451**

acredita que isso gera um estilo de previsão baseado na sabedoria dos especialistas, aproveitando os benefícios de muitos modelos de previsão bem justificados e com bom desempenho (como suavização exponencial, ARIMA e outros). Além disso, um conjunto de previsões gera uma distribuição de previsões, fornecendo uma base para determinar se alguma delas é tão diferente da “multidão” a ponto de ser irracional. Por fim, o agrupamento fornece uma forma de quantificar a incerteza em relação à previsão, o

que o Google também faz ao simular a propagação direta dos erros ao longo do tempo.

Em última análise, como admite a equipa de ciência de dados, a abordagem da Google beneficia de abordagens em grande escala e altamente paralelas, adequadas aos enormes recursos computacionais disponíveis na empresa. Ele também se beneficia do estabelecimento de tarefas paralelas (como simular o erro propagado muitas vezes no tempo) e processos automatizados com espaço para entrada de um analista, mas sem necessidade disso. Desta forma, previsões razoáveis podem ser geradas em escala e para uma variedade de conjuntos de dados.

Embora seu trabalho possa não ter as mesmas vantagens ou precisar do alto nível de automação exigido no Google, há muitas ideias que você pode incorporar em seu fluxo de trabalho a partir do modelo deles, mesmo como cientista de dados solo. Esses incluem:

• Construir uma estrutura ou "pipeline" para limpar seus dados como uma versão básica "boa o suficiente" de como você gostaria de preparar cada conjunto de dados de série temporal

antes de modelar.

• Construir um conjunto de modelos de previsão respeitados como seu kit de ferramentas ideal.

Pacote Profeta de Código Aberto do Facebook

O Facebook abriu o código-fonte de seu pacote automatizado de previsão de séries temporais, Prophet, mais ou menos na mesma época em que o Google divulgou informações sobre seu pacote interno. Em

sua própria postagem no blog sobre o Profeta, o Facebook destacou alguns dos mesmos problemas

enfatizado na abordagem do Google, particularmente:

• Sazonalidades em "escala humana" e feriados com espaçamento irregular

• Mudanças de nível

• Dados ausentes e valores discrepantes

Além disso, o pacote do Facebook trouxe forças, como:

• Capacidade para lidar com conjuntos de dados em vários níveis de granularidade, como dados minuto a minuto ou por hora, além de dados diários

• Tendências que indicam um crescimento não linear, como atingir um ponto de saturação. O Facebook observou que os resultados com o seu pacote eram muitas vezes tão bons como os produzidos pelos analistas. Tal como o Google, o Facebook indicou ter encontrado muitos casos

de tarefas altamente paralelas enquanto desenvolve seu pipeline de ajuste de série temporal. O Facebook disse que o Prophet se tornou um previsor tão confiável que suas previsões foram usadas não apenas internamente, mas em produtos voltados para o exterior. Além disso, o Facebook disse que desenvolveu um padrão de trabalho de "analista no circuito", de modo que o processo automatizado pudesse ser supervisionado e corrigido quando necessário, levando, em última análise, a um produto que poderia ajudar ou substituir a análise humana, dependendo do nível de recursos dedicados a uma tarefa.

A abordagem do Facebook é bem diferente da do Google; inclui três componentes simples:

• Efeitos sazonais anuais e semanais

• Listas personalizadas de feriados

• Uma curva de tendência linear ou logística por partes

Esses componentes são usados para formar um *modelo de regressão aditivo* . Este é um modelo de regressão não paramétrico, o que significa que não são feitas suposições sobre a forma da função de regressão subjacente e nenhuma linearidade é imposta. O modelo é mais flexível e interpretável do que uma regressão linear, mas apresenta

uma compensação de maior variância (pense na compensação de viés-variância geralmente conhecida em aprendizado de máquina e estatística) e mais problemas de ajuste excessivo. Este modelo faz sentido para a tarefa geral que o Facebook procura, na medida em que comportamentos não lineares complexos devem ser modelados de uma forma automatizada que evite metodologias excessivamente exigentes.

Existem muitas vantagens no Profeta, incluindo:

• API simples

• Código aberto e em desenvolvimento ativo

• APIs completas e iguais em Python e R, que ajudam equipes multilíngues de ciência de dados

Profeta é fácil de usar. Encerramos esta seção com um exemplo de trecho de código retirado do guia de início rápido para que você possa ver o código mínimo necessário para implementar uma previsão automatizada. Combinamos nosso uso do Prophet com o uso de visualizações de página, um pacote R que recupera facilmente dados de séries temporais relacionados às visualizações de páginas da Wikipédia. Primeiro, baixamos alguns dados de série temporal:

##R

> biblioteca (visualizações de página)

> df_wiki = article_pageviews(project = "en.wikipedia",

> artigo = "Facebook",

> início = as.Data('01/11/2015'),

> fim = as.Data("02/11/2018"),

> tipo_usuário = c("usuário"),

> plataforma = c("web móvel"))

> nomes de colunas(df_wiki)

[1] "projeto" "idioma" "artigo" "acesso" "agente" "granularidade"

[7] "data" "visualizações"

Agora que temos alguns dados com resolução temporal diária e ao longo de alguns anos, podemos tentar prever esses dados com o Prophet em apenas alguns passos simples (veja a Figura 16-1):

##R

> ## subdividimos os dados de acordo com o que precisamos

> ## e forneça os nomes de colunas esperados

> df = df_wiki[, c("data", "visualizações")]

> nomes de colunas(df) = c("ds", "y")

> ## também transformamos os dados em log porque os dados

> ## tem mudanças tão extremas em valores que grandes

> ## valores diminuem a variação normal entre dias

> df$y = log(df$y)

> ## criamos um quadro de dados 'futuro' que

> ## inclui as datas futuras que gostaríamos de prever

> ## preveremos 365 dias além de nossos dados

> m = profeta (df)

> futuro <- make_future_dataframe(m, períodos = 365)

> cauda (futuro)

ds

1458 28/10/2019

1459 29/10/2019

1460 30/10/2019

1461 31/10/2019

1462 01-11-2019

1463 02-11-2019

> *## geramos a previsão para as datas de interesse*

> previsão <- prever (m, futuro)

> tail(previsão[c('ds', 'yhat', 'yhat_lower', 'yhat_upper')])

ds yhat yhat_lower yhat_upper

1458 28/10/2019 9,119005 8,318483 9,959014

1459 29/10/2019 9,090555 8,283542 9,982579

1460 30/10/2019 9,064916 8,251723 9,908362

1461 31/10/2019 9,066713 8,254401 9,923814

1462 01/11/2019 9,015019 8,166530 9,883218

1463 02/11/2019 9,008619 8,195123 9,862962

> *## agora temos previsões para o nosso valor de interesse*

> *## finalmente traçamos para ver como o pacote se saiu qualitativamente*

> plot(dfds, dfy, col = 1, type = 'l', xlim = range(previsão$ds),

> main = "Visualizações reais e previstas de páginas da Wikipédia do 'Facebook'")

> pontos(previsão$ds, previsão$yhat, tipo = 'l', col = 2)



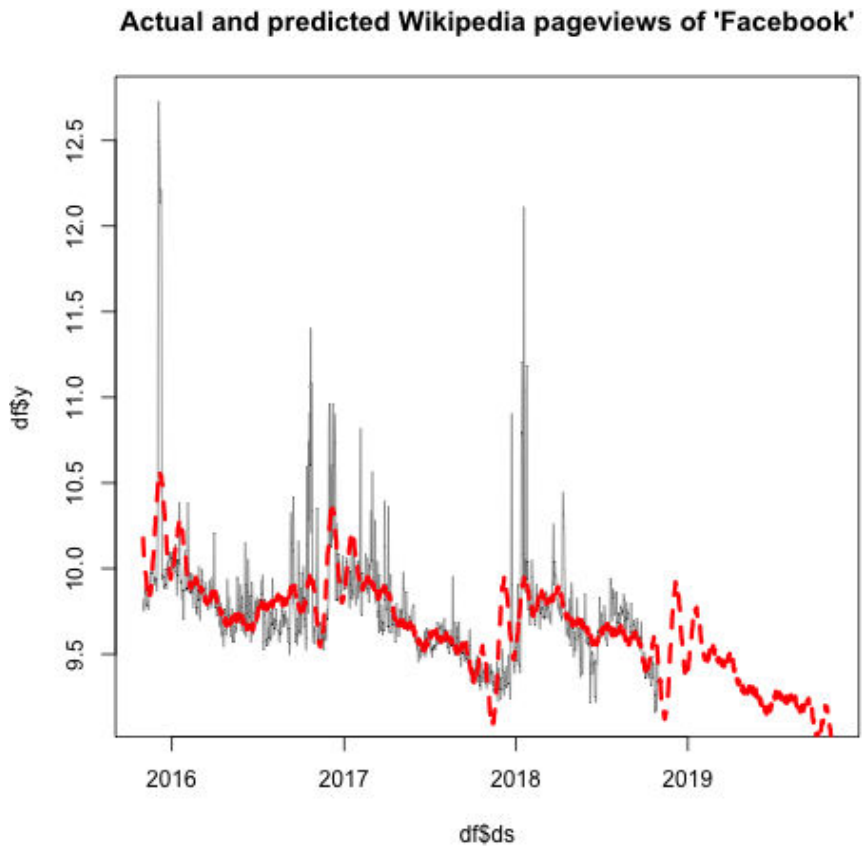


Figura 16-1. Gráfico dos dados de contagem de páginas da Wikipedia (linha sólida fina) e as previsões do Profeta para

esses dados (linha tracejada grossa).

O Prophet também oferece a opção de representar graficamente os componentes (tendência e sazonal) que formam a previsão (veja a Figura 16-2):

##R

> profeta_plot_components(m, previsão)

Restrinja o uso do profeta aos dados diários. Pela sua própria descrição,

o pacote Profeta foi desenvolvido e funciona melhor para

dados. Infelizmente, esta estreita especialização significa que o

mesmas técnicas podem não ser confiáveis para dados em momentos diferentes.

escalas. Por esta razão, você deve abordar este pacote e seus

técnicas associadas com cautela quando seus dados não são diários.

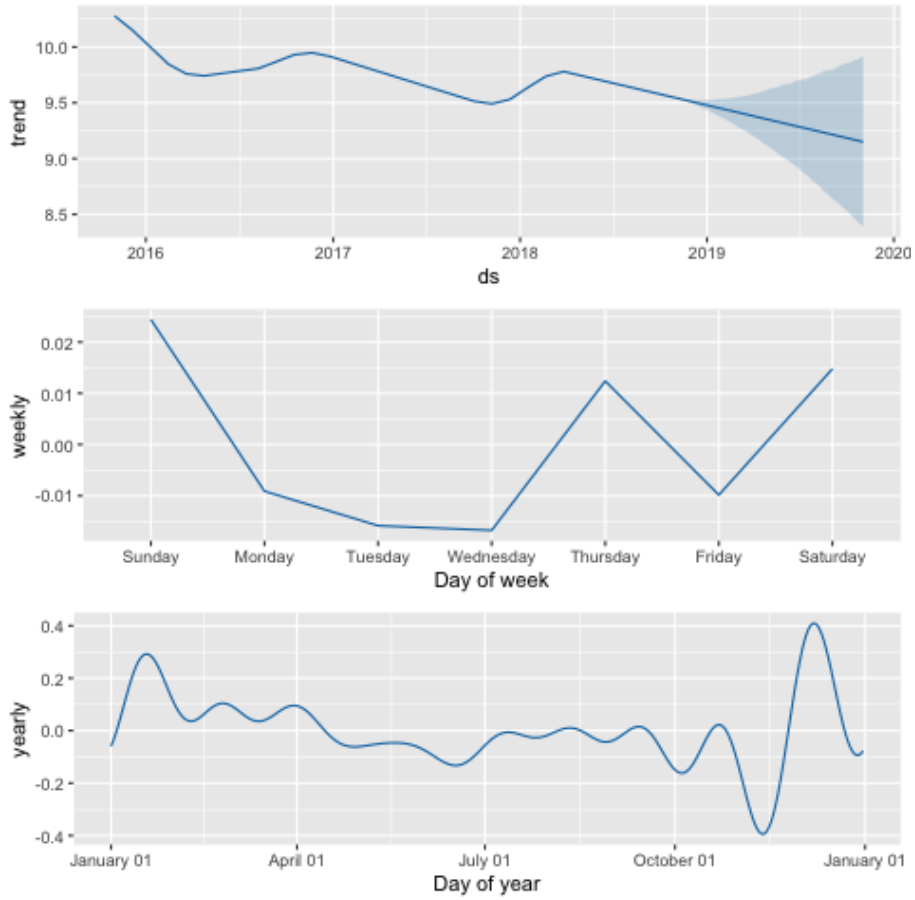


Figura 16-2. A previsão foi desagregada em componentes de tendência, semanais e anuais.

A partir disso a previsão é formada por uma soma dos componentes. Observe que diferente componentes são formados de maneira diferente. Os dados de tendência têm uma forma linear subjacente, enquanto os dados anuais são curvos devido ao ajuste subjacente da série de Fourier (leia mais no Postagem no blog do Facebook mencionada anteriormente).

Com o tempo, pacotes de código aberto de séries temporais mais automatizados e produtos de caixa preta estão se tornando disponíveis. Estes podem ser um bom ponto de entrada para uma organização que é novata em previsões e que procura fazer uma previsão razoável. No entanto, num futuro próximo, estes pacotes provavelmente não proporcionarão a melhor previsão disponível para todas as séries temporais e todas as organizações. Quando você puder incorporar conhecimento de domínio e restrições

organizacionais relevantes em seu modelo de série temporal, terá um resultado melhor e, por enquanto, isso continua sendo tarefa de um analista humano até que pacotes de previsão mais gerais sejam construídos.

Detecção de anomalia

A detecção de anomalias é outra área em que as empresas de tecnologia estão fazendo esforços significativos e compartilhando-os com a comunidade de código aberto. A detecção de anomalias é importante em séries temporais por alguns motivos:

• Pode ser útil remover valores discrepantes ao ajustar modelos que não sejam suficientemente robustos para tais valores discrepantes.

• Pode ser útil identificar valores discrepantes se quisermos construir um modelo de previsão especificamente para prever a extensão de tais eventos discrepantes, dependendo do conhecimento de que ocorrerão.

A seguir, discutiremos a abordagem adotada pelo Twitter em seu trabalho de código aberto sobre detecção de anomalias.

Pacote de detecção de anomalias de código aberto do Twitter

O Twitter abriu o código-fonte de um pacote de detecção de valores discrepantes, AnomalyDetection,1 quatro anos

atrás, e o pacote continua útil e com bom desempenho. Este pacote implementa o ESD Híbrido Sazonal (Extreme Studentized Deviant), que constrói um modelo mais elaborado do que o ESD Generalizado para identificar valores discrepantes. O próprio teste ESD generalizado é baseado em outro teste estatístico, o teste de Grubbs , que define uma estatística para testar a hipótese de que há um único outlier em um conjunto de dados. O ESD Generalizado aplica este teste repetidamente, primeiro ao valor discrepante mais extremo e depois para valores

discrepantes sucessivamente menores, enquanto ajusta os valores críticos no teste para levar em conta vários testes sequenciais. O ESD Híbrido Sazonal baseia-se no ESD Generalizado para levar em conta a sazonalidade no comportamento por meio da decomposição de séries temporais.

Podemos ver o uso simples deste pacote no código R a seguir. Primeiro carregamos alguns dados de amostra fornecidos pelo pacote do Twitter:

##R

> biblioteca (Detecção de Anomalias)

> dados(dados_brutos)

> cabeça(dados_brutos)

contagem de carimbo de data/hora

125/09/1980 14:01:00 182.478

2 25/09/1980 14:02:00 176.231

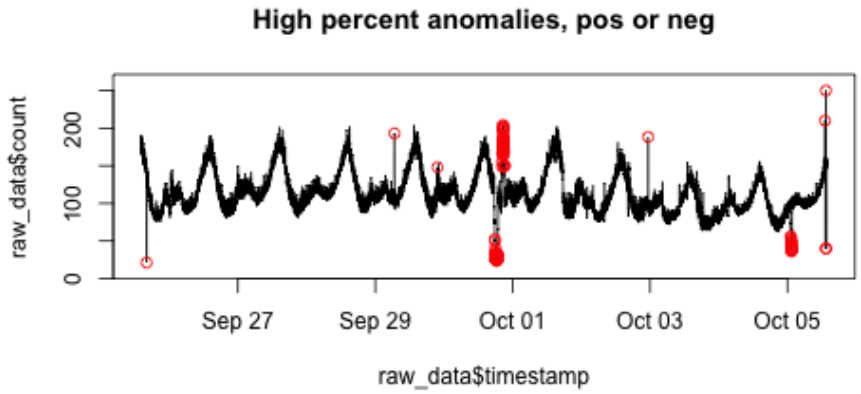
3 25/09/1980 14:03:00 183.917

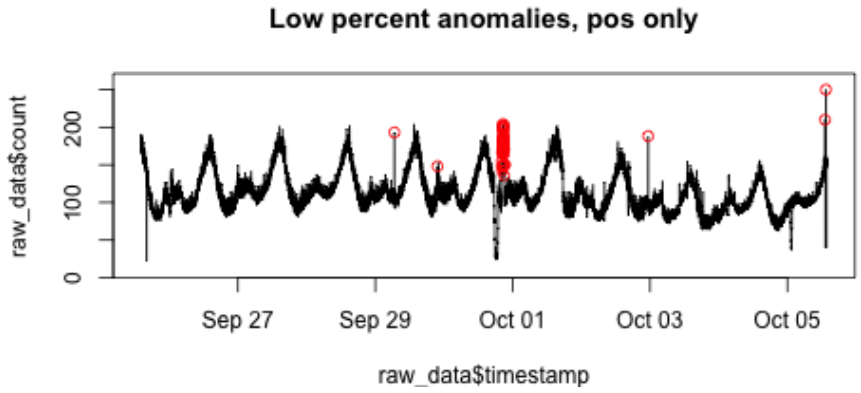
4 25/09/1980 14:04:00 177.798

5 25/09/1980 14:05:00 165.469

6 25/09/1980 14:06:00 181.878

1 Leia mais no repositório GitHub do projeto e no blog do Twitter





Em seguida, usamos a função automatizada de detecção de anomalias do Twitter com dois conjuntos de

parâmetros:

1. Procuramos uma grande parte de anomalias na direção positiva ou negativa.

2. Procuramos apenas uma pequena porção de anomalias na faixa positiva.

Esses casos de uso são demonstrados aqui:

##R

> ## detecta uma alta porcentagem de anomalias em qualquer direção

> general_anoms = AnomalyDetectionTs(raw_data, max_anoms=0,05,

> direção='ambos')

>

> *## detecta uma porcentagem menor de anomalias apenas na direção pos*

> high_anoms = AnomalyDetectionTs(raw_data, max_anoms=0,01,

> direção='pos')

Representamos graficamente os resultados em ambos os casos na Figura 16-3 .

Figura 16-3. As anomalias relatadas pelo AnomalyDetectionTs() do Twitter funcionam tanto com configurações muito inclusivas (parte superior) quanto com configurações mais limitadas (parte inferior).

Vemos que um grande número de anomalias são relatadas em uma parte da série temporal, então também cortamos nosso gráfico para esta parte da série temporal para obter uma

melhor compreensão do que está acontecendo (Figura 16-4).

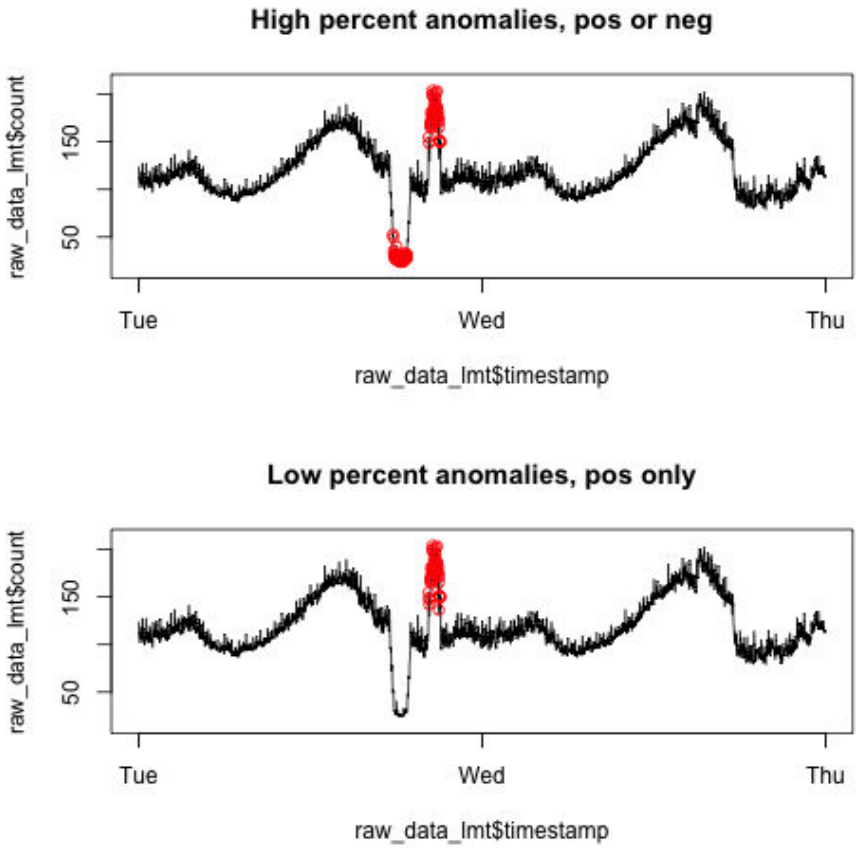


Figura 16-4. As mesmas anomalias identificadas, agora focando nas anomalias que ocorreram todas em um cluster no mesmo dia.

Podemos entender melhor por que esses pontos são anomalias depois de olhar mais de perto. Eles se desviam do padrão diário que de outra forma estaria presente. Quanto ao motivo pelo qual podemos querer procurar apenas desvios positivos, imagine que estamos a construir uma infra-estrutura para um website de elevado tráfego e com utilização intensiva de dados. Embora as anomalias descendentes/negativas possam ser interessantes, as anomalias de negócios decisivas para nós serão os momentos em que a nossa infraestrutura não consegue lidar com uma oportunidade de tráfego intenso. Estamos mais interessados em identificar picos, por vários motivos:

• Se esses números forem falsos, gostaríamos de eliminá-los para que possamos saber o verdadeiro limite de uso realista. Números anormalmente altos nos levarão a comprar recursos computacionais de que não precisamos, o que não acontece com números anormalmente baixos.

Usar a detecção de anomalias pode nos ajudar a limpar nossos dados como uma etapa de pré-processamento.

• Se o equipamento informático for barato, preferiríamos comprar mais, mesmo para acomodar estas anomalias. Se pudermos rotular anomalias, esse será o primeiro passo para gerar dados rotulados para tentar prever essas anomalias. No entanto, por

anomalias de definição são difíceis de prever, portanto, você não deve ter grandes expectativas em relação a tais esforços na maioria das vezes!

Detecção de anomalias | 459

Existem muitos parâmetros que você pode usar com a detecção automatizada de anomalias do Twitter, e é uma boa ferramenta disponível quando você está explorando um novo conjunto de dados, tanto para

limpeza e modelagem de dados.2

Outros pacotes de série temporal

Neste capítulo, concentrámo-nos em grande parte nos pacotes amplamente utilizados desenvolvidos por algumas das maiores empresas tecnológicas, em conjunto com os enormes conjuntos de dados e previsões relacionadas que estas empresas produzem como parte das suas operações comerciais principais. No entanto, estas empresas estão longe de ser os principais ou mais seniores fornecedores de pacotes de séries temporais. Existe um enorme ecossistema de pacotes de séries temporais dedicados a:

• Armazenamento e infraestrutura de série temporal

• Conjuntos de dados de série temporal

• Detecção de ponto de interrupção

• Previsão

• Análise no domínio da frequência3

• Séries temporais não lineares

• Previsão automatizada de séries temporais

Isto não é uma lista exaustiva. Existem pacotes para tudo, literalmente dezenas ou

até mesmo centenas de pacotes de séries temporais. A lista mais extensa de código aberto

packages é mantido pelo professor Rob Hyndman na página oficial do repositório CRAN do R. Vale a pena dar uma olhada nesta lista, tanto para encontrar pacotes específicos que possam atender às suas necessidades analíticas para um projeto específico, mas também, de forma mais geral, para se informar sobre a variedade de métodos de análise de série temporal disponíveis e sendo ativamente implantados na comunidade. Assim como este livro, essa página oferece uma visão geral e um delineamento das tarefas associadas aos dados de séries temporais. Em Python, não há uma lista igualmente extensa e consolidada de módulos de séries temporais, mas a ciência de dados

O artista Max Christ compilou uma lista muito útil deles.

2 Vale ressaltar que o Twitter também lançou um pacote de detecção de alterações de nível, BreakoutDetection, ao mesmo tempo em que lançou o pacote AnomalyDetection. No caso do BreakoutDetection, o pacote serve para identificar locais onde houve uma mudança de nível em uma série temporal. Este pacote é igualmente acessível e fácil de usar, embora não tenha ganhado tantos seguidores e não seja tão destacado quanto o pacote AnomalyDetection. Existem muitos pacotes alternativos de detecção de pontos de interrupção que foram testados e implantados de forma mais extensiva.

3 A análise no domínio da frequência é mencionada apenas brevemente neste livro, mas continua a ser uma área importante da análise de séries temporais e amplamente utilizada em algumas disciplinas, como física e climatologia.

Mais recursos

StatsNewbie123, "É possível automatizar a previsão de séries temporais?" postar na cruz Validado em 6 de dezembro de 2019, https://perma.cc/E3C4-RL4L.

Esta postagem recente do StackExchange pergunta se é possível automatizar a previsão de séries temporais para qualquer série temporal. Existem duas respostas muito úteis e detalhadas com uma visão geral dos pacotes automatizados, bem como uma discussão de todos os desafios inerentes a tal tarefa.

CausalImpact, pacote de código aberto do Google para inferência causal, https://perma.cc/

Y72Z-2SFD.

Este pacote de código aberto do Google é construído sobre outra versão do Google, bsts, o pacote Bayesian Structural Time Series, que usamos no Capítulo 7 . O pacote CausalImpact usa bsts para ajustar modelos e então, uma vez ajustados, para construir exemplos de controle contrafactuais para avaliar a causalidade e o tamanho do efeito em dados de séries temporais. O repositório GitHub do pacote contém links para artigos de pesquisa relevantes e uma visão geral em vídeo útil de um dos criadores do pacote. Também vale a pena conferir

o artigo de pesquisa associado .

Murray Stokely, Farzan Rohani e Eric Tassone, "Estatística Paralela em Grande Escala

Previsão de cálculos em R," em JSM Proceedings, Seção de Ciências Físicas e de Engenharia (Alexandria, VA: American Statistical Association, 2011), https://

perma.cc/25D2-RVVA.

Este documento oferece uma explicação detalhada e de alto nível de como os pacotes R foram escritos para a previsão interna de séries temporais do Google para fazer séries temporais de maneira altamente paralela e escalonável.

Esta é uma boa leitura não apenas para os detalhes relacionados à previsão, mas também para entender melhor como construir pipelines de dados de séries temporais para grandes organizações com grandes quantidades e tipos variados de dados de séries temporais.

Danilo Poccia, "Amazon Forecast: previsão de séries temporais facilitada", Notícias da AWS Blog, 28 de novembro de 2018, https://perma.cc/Y2PE-EUDV.

Um modelo recente de série temporal automatizada que não abordamos é o novo modelo de previsão da Amazon.

serviço de elenco, Amazon Forecast. Não é de código aberto, mas há muitas análises promissoras. Oferece uma maneira de usar os modelos desenvolvidos pela Amazon com sua experiência em varejo para ajudar as empresas a fazer previsões de negócios. Embora seja um serviço pago, você pode experimentar o nível gratuito, que oferece opções bastante generosas. O serviço foi projetado para enfatizar tanto a precisão quanto a usabilidade, e é uma boa alternativa para aquelas organizações que procuram um modelo "bom o suficiente" para grandes volumes.

situações de previsão. O pacote da Amazon usa uma abordagem mista de aprendizagem profunda e modelos estatísticos tradicionais, semelhante a como o modelo LSTNET simples de

O Capítulo 10 combinou um modelo de aprendizagem profunda com um componente autorregressivo.

Vale a pena ler sobre a arquitetura de rede neural exclusiva da Amazon para

previsão, DeepAR.

CAPÍTULO 17

Previsões sobre previsões

Há muitas citações boas sobre a impossibilidade de prever o futuro, mas não posso deixar de querer concluir este livro com algumas reflexões sobre o que está por vir.

Previsão como serviço

Como a previsão de séries temporais tem menos profissionais especializados do que outras áreas da ciência de dados, tem havido um impulso para desenvolver a análise e previsão de séries temporais como um serviço que pode ser facilmente empacotado e implementado de forma eficiente. Por exemplo, e

conforme observado no Capítulo 16, a Amazon lançou recentemente um serviço de previsão de séries temporais,

e não é a única empresa a fazer isso. O modelo da empresa parece deliberadamente gen-

geral, e enquadra a previsão como apenas uma etapa em um pipeline de dados (veja a Figura 17-1).

Esses esforços de modelagem de previsão como serviço visam um modelo geral suficientemente bom que possa acomodar uma variedade de campos sem fazer previsões terrivelmente imprecisas. A maioria deles descreve seus modelos como uma mistura de aprendizado profundo e modelos estatísticos tradicionais. No entanto, como o serviço é, em última análise, uma caixa negra, será difícil compreender o que pode fazer com que as previsões corram mal ou mesmo investigar retrospetivamente como podem ser melhoradas. Isto significa que existe um nível de qualidade razoavelmente elevado para as previsões, mas provavelmente também um limite máximo de desempenho.

Este serviço pode ser valioso para empresas que necessitam de muitas previsões, mas não possuem pessoal disponível para gerá-las individualmente. No entanto, para empresas que possuem quantidades substanciais de dados históricos onde heurísticas e “leis” mais gerais

de seus dados pudessem ser descobertos, é provável que um analista pudesse superar esses algoritmos, dada a familiaridade com o domínio.

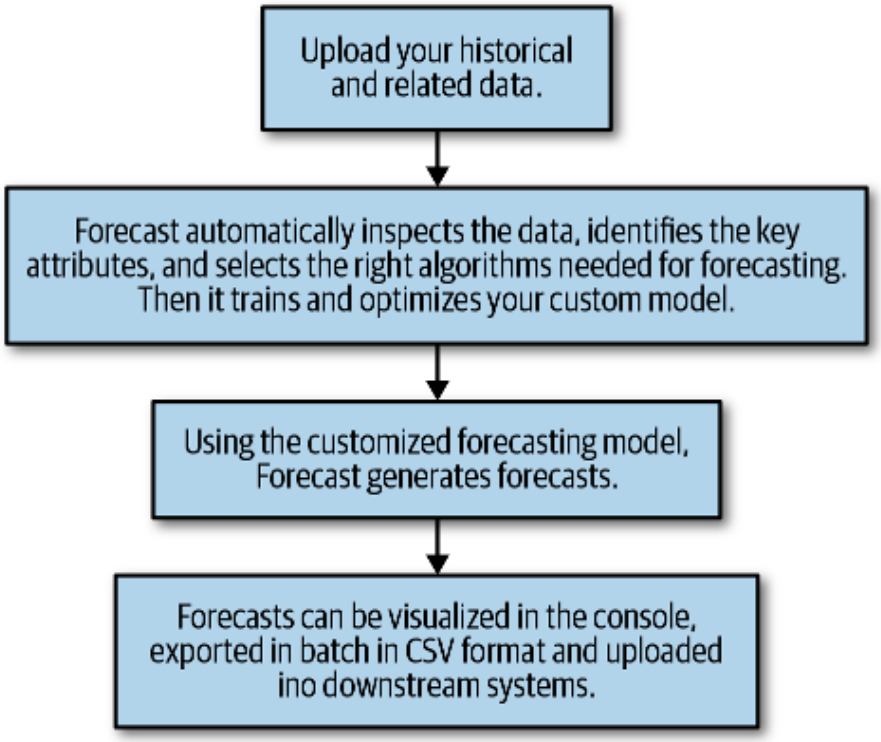


Figura 17-1. Um exemplo de pipeline de previsão como serviço de série temporal. Tais serviços são sendo oferecido por uma variedade de pequenas startups e gigantes da tecnologia, principalmente a Amazon, que lançou um conjunto de produtos adaptados para prever dados de séries temporais de forma automática e em escala por meio de um conjunto de produtos impulsionados por modelos estatísticos e de aprendizado profundo.

Observe que uma boa quantidade do produto que está sendo vendido na área de previsão como serviço também tem a ver com boas visualizações das previsões e utilitários de pipeline para revisar facilmente as previsões, alterar a frequência da previsão e assim sobre. Mesmo que a sua organização acabe por desenvolver as suas próprias análises de previsão, pode ser útil ver o que está a emergir como um padrão da indústria.

Deep Learning aumenta possibilidades probabilísticas

Nos últimos anos, muitas das maiores empresas de tecnologia tornaram públicas algumas informações sobre como fazem as suas próprias previsões para os seus serviços mais importantes. Neste caso não estamos falando da necessidade de muitas previsões paralelas do grande número de métricas que afetam o negócio da empresa, mas sim de preocupações centrais.

Nestes casos, onde a qualidade da previsão é primordial, as empresas dão frequentemente indicações de que estão a utilizar aprendizagem profunda com uma componente probabilística.

Por exemplo, a Uber publicou um blog sobre a previsão da demanda por viagens de carro, e a Amazon

desenvolveu uma rede neural recorrente autorregressiva bem conceituada, inspirada em

pensamento estatístico para fazer previsões sobre as demandas dos produtos. Quanto mais os pesquisadores forem capazes de integrar metodologias estatísticas, como a injeção de antecedentes para o conhecimento do domínio e a quantificação da incerteza, menos razão haverá .

será procurar modelos estatísticos quando um modelo de aprendizagem profunda puder oferecer os pontos fortes da estatística e da aprendizagem profunda.

No entanto, criar modelos de aprendizagem profunda razoavelmente interpretáveis – para que possamos saber o quão “errada” ou extrema uma previsão pode ser – continua a ser uma tarefa difícil, por isso é improvável que os modelos estatísticos tradicionais, com maior compreensão teórica e clareza mecanicista, sejam descartados. . Para previsões críticas, onde a saúde e a segurança podem estar em risco, as pessoas podem razoavelmente continuar a confiar no que funcionou durante décadas até que métodos mais transparentes e inspecionáveis possam ser desenvolvidos para

previsões de aprendizado de máquina.

Aumentando a importância do aprendizado de máquina em vez de

Estatisticas

Empiricamente, parece haver cada vez menos utilização de estatísticas adequadas na modelação de dados e na geração de previsões. Não se desespere: o campo da estatística continua a prosperar e a responder questões interessantes relacionadas à estatística. E ainda assim - especialmente para previsões de baixo risco que apenas precisam ser boas o suficiente - as técnicas de aprendizado de máquina e os métodos estatísticos orientados para resultados, em vez de teorias sofisticadas e soluções fechadas ou provas de convergência, estão vencendo na implantação real e na implementação real. casos de uso mundiais.

Do ponto de vista do profissional, isso é uma coisa boa. Se você felizmente deixou seus conjuntos de problemas para trás há muito tempo, sem nenhum desejo de provar coisas, não precisa temer o retorno de provas adequadas e coisas do gênero. Por outro lado, esta é uma tendência preocupante, à medida que estas tecnologias penetram em aspectos cada vez mais fundamentais da vida. Não me importo de navegar no site de um varejista que usa aprendizado de máquina para estimar minhas prováveis ações futuras como comprador. Mas gostaria de saber que as previsões de séries temporais que modelaram os meus resultados de saúde ou as progressões académicas do meu filho foram mais completas e validadas estatisticamente, porque um modelo tendencioso pode realmente prejudicar alguém nestas áreas centrais.

Por enquanto, os líderes no pensamento de séries temporais para fins industriais estão a trabalhar em áreas de baixo risco. Para problemas como a previsão de receitas de uma campanha publicitária ou do lançamento de um produto nas redes sociais, não é importante se as previsões são totalmente validadas. À medida que aspectos mais fundamentais do cuidado e da alimentação dos seres humanos entram no domínio da modelagem, esperemos que as estatísticas desempenhem um papel mais fundamental nas previsões de alto risco.

Combinação crescente de estatística e máquina

Metodologias de Aprendizagem

Uma série de indicações apontam para a combinação de aprendizado de máquina e estatística

metodologias1 em vez de simplesmente procurar o “melhor” método de previsão.

Esta é uma extensão da crescente aceitação e uso de métodos conjuntos para previsão, um fenômeno que discutimos ao longo do livro.

Um exemplo de teste extraordinariamente robusto com muitos conjuntos de dados do mundo real é a recente competição M4, uma competição de séries temporais que mede a precisão das previsões em

100.000 conjuntos de dados de série temporal, conforme mencionado brevemente nos Capítulos 2 e 9 . A inscrição vencedora desta competição combinou elementos de um modelo estatístico e uma rede neural. Da mesma forma, o segundo colocado incorporou aprendizado de máquina e estatística, neste caso usando um conjunto de modelos estatísticos, mas depois usando uma árvore com gradiente reforçado (XGBoost) para escolher os pesos relativos de cada modelo no conjunto. Neste exemplo, vemos duas maneiras distintas pelas quais o aprendizado de máquina e as abordagens estatísticas podem ser combinadas: como modelos alternativos montados juntos (como no caso da inscrição vencedora) ou com um método determinando como definir os metaparâmetros do outro método (como em o caso do vice-campeão). Um documento abrangente e altamente

resumo acessível dos resultados da competição foi posteriormente publicado no

Jornal Internacional de Previsão .

`

A medida que essas combinações ganham força, provavelmente veremos o desenvolvimento de pesquisas na área de determinação de quais problemas são mais passíveis de combinação de modelos estatísticos e de aprendizado de máquina, bem como o surgimento de melhores práticas para ajustar o desempenho desses modelos e selecionar arquiteturas. Esperamos ver o mesmo refinamento que em outras arquiteturas complexas, como redes neurais, em que paradigmas de design padrão emergem ao longo do tempo com pontos fortes, fracos e técnicas de treinamento conhecidos.

Mais previsões para a vida cotidiana

Mais empresas voltadas para o consumidor, como aplicativos móveis de saúde e bem-estar, lançaram ou foram solicitadas a lançar previsões personalizadas. À medida que as pessoas crescem

mais conscientes da quantidade de dados que seus aplicativos armazenam sobre eles e outras pessoas, eles procuram tirar vantagem obtendo previsões personalizadas para métricas como metas de saúde e condicionamento físico. Da mesma forma, as pessoas muitas vezes procuram previsões para cada

varia desde valores imobiliários futuros (difíceis de prever) até as datas prováveis de chegada de espécies de aves migratórias.

1 Agradecimentos especiais ao revisor técnico Rob Hyndman que sugeriu este tópico (além de muitas outras sugestões úteis ao longo do livro).

Mais produtos serão explicitamente impulsionados pela procura de fazer previsões, seja sobre assuntos esotéricos ou para métricas individuais. Isto significa que mais pipelines de previsão serão integrados em locais onde antes não eram muito prováveis, como aplicações móveis e websites para leitores casuais, em vez de especialistas do setor.

Quanto mais comum isso se tornar, maior será a probabilidade de o jargão das séries temporais fazer parte do discurso cotidiano. Esperamos que as pessoas também adquiram educação suficiente sobre séries temporais para compreender os limites e suposições das previsões, de modo que não dependam muito desses produtos.”

Mais previsões para a vida cotidiana | 467

Índice

A

Teste aumentado de Dickey-Fuller (ADF), 84

tipo de dados abstrato, 131

autocorrelação, 91

precisão, desempenho versus, 357

função de autocorrelação (ACF), 91 -92

ACF (ver função de autocorrelação)

Modelo AR(p) e, 171

tabelas atuariais, 3

de resíduos ARIMA, 190-193

modelo de regressão aditiva, 453

Processos MA/AR versus, 183

modelagem baseada em agente, 134

de dados não estacionários, 100 - 101

agregação

PACF versus , 94-101

A estrutura de previsão interna do Google,
de dados estacionários, 95
451
codificador automático, 334
de dados governamentais, 431
diferenciação automática, 298
Conjunto de dados AirPassengers, 82, 101
modelo autorregressivo (AR), 12 , 166-181
Visualizações 2D, 105-113
(veja também autoregressão vetorial (VAR))
Visualizações 3D, 113-116
ACF/PACF versus, 183
suavização de dados, 57
escolhendo parâmetros para o modelo AR(p);
Critério de informação de Akaike (AIC), 173
171 -176
parâmetro alfa (fator de suavização), 58
prevendo muitos passos no futuro,
AlphaVantage, 404
178 -180
Serviço de previsão de série temporal da Amazon, 461, 463

previsão de um passo à frente, 176 -178
Detecção de anomalias (pacote Twitter), 457 - 460
previsão com um processo AR(p), 176 - 180
Modelo AR(p), 167
Processo MA como processo AR de ordem infinita,
escolhendo parâmetros para, 171 -176
181
previsão com, 176-180
usando álgebra para entender as restrições em
como função de janela móvel, 177
Processos AR, 166 -171
ARCH (heteroske-
média móvel integrada autoregressiva
dasticidade), 201
(ARIMA) modelo, 186 -196
ARIMA (ver movimento integrado autoregressivo
cerca de, 186 - 189
média)
Modelo ARMA e, 187 - 189
Aristóteles, 6
ajuste automatizado de modelo, 194

suposições, em modelos de aprendizado de máquina, 291
exemplo de previsão de gripe, 371-377
astronomia, 9
ajuste manual do modelo, 190-194
mecanismo de atenção, para feed forward neural
seleção de parâmetro, 189 -196
rede, 322 - 323
SARIMA, 201
pesos de atenção, 322

modelo de média móvel autorregressiva (ARMA),
Classificação de série temporal UEA e UCR
187-189
Repositório, 21
limpeza de dados, 40 - 69

B

exemplo de previsão do nível de glicose no sangue,
operador de retrocesso, 182
385 -389
backtesting, 347
exemplo de previsão de gripe, 368-371

preenchimento para trás, 46
A estrutura de previsão interna do Google,
ensacamento (agregação de bootstrap), 268
450
Algoritmo Baum Welch, 220
tratamento de dados ausentes, 40 -52
Série temporal estrutural bayesiana (BSTS), 229 - 235
impedindo a antecipação, 67
código para, 230 -235
suavização, 55 - 63
recursos de aprendizagem, 236
fusos horários , 63-67
curva de sino, 171
aumento e redução da resolução , 52-55
Procedimento Benjamini – Yekutieli, 252
agrupamento, 272 -287
compensação entre viés e variância, 165
código para, 285-287
formato de dados binários
gerando recursos a partir de dados, 273-280
desempenho computacional e, 361

agrupamento hierárquico de recursos normalizados
NumPy e, 158
turs, 285
R e, 158, 362
agrupamento hierárquico com o dis-
do nível de glicose no sangue, 384-400
matriz de resistência, 286
limpeza/exploração de dados, 385 -389
reduzindo o atraso entre a medição e
geração de recursos, 251
previsão, 360
ajustando um modelo, 396 -400
métricas de distância com reconhecimento temporal, 280-285
recursos de geração, 390 -396
CNNs (ver redes neurais convolucionais)
impulsionando, 267
cointegração, 103
(veja também árvores com gradiente aumentado)
ordem da coluna principal, 106
agregação de bootstrap (ensacamento), 268
mercados de commodities, 9

Transformação de Box Cox, 86

CompEngine, 24

Caixa, George, 10

desempenho computacional, 357-365

Método Box-Jenkins, 10, 190

precisão versus, 357

Pacote BreakoutDetection, 460

ferramentas alternativas de alto desempenho, 364

BSTS (ver série temporal estrutural bayesiana)

evitando dados sobrepostos, 359

pacote bsts, 230-235

formatos de armazenamento de dados e, 361

ciclos de negócios, 8

empregando o paradigma do tipo gerador para iterar

comeu o conjunto de dados, 359

C

para aplicações financeiras, 422

recursos de aprendizagem, 365

C++, melhorando o desempenho usando 364

quantidade limite de dados de treinamento, 363

efeitos relacionados ao calendário, 451

modificando sua análise para se adequar ao desempenho
CatBoost, 271
considerações, 362
Distribuição de Cauchy, 170
problemas com modelos construídos para cross-
convoluções causais, 327 -328
dados seccionais, 358 -360
censura, 445
problemas com modelos que não pré-
pacote de césio, 245 -247, 260 - 264
calcular, 360
Fluxo de dinheiro Chaikin (CMF), 250
riscos com modelos novos/complexos, 363
classificação, 260 -272
trabalhando com ferramentas criadas para uso mais geral
agrupamento e, 272
casos de uso, 358-361
métodos de árvore de decisão, 264-272
desagregação conceitual, 451
seleção/geração de recurso, 260-264
probabilidades condicionais, 167

vetor de contexto, 322

bancos de dados NoSQL de uso geral, 154-156

conjunto de dados do monitor contínuo de glicose (CGM)

InfluxoDB, 153

(veja previsão do nível de glicose no sangue)

recursos de aprendizagem, 160

convolução, definida, 324

banco de dados de série temporal popular e solução de arquivo

redes neurais convolucionais (CNNs),

ções, 152-156

324-329

Prometeu, 154

modelos alternativos, 327 -329

SQL versus NoSQL, 148-156

convoluções causais, 327-328

armazenamento temporal de dados com, 148-156

convertendo uma série temporal em imagens, 329

bancos de dados específicos de séries temporais e relacionados

recursos de aprendizagem, 342

ferramentas de monitoramento, 152 - 154
modelo simples, 325 - 327
horário de verão, 66
correlação
árvores de decisão, 264 -272
cointegração, 103
classificação versus regressão, 271
em um sistema determinístico, 92
exemplo de código, 268 - 271
de autocorrelação parcial, 94-101
árvores com gradiente aumentado, 267
espúrio, 102-103
florestas aleatórias, 265
Arquivos CSV, desempenho e 361
decomposição, 62
interface cuDNN, 332
aprendizagem profunda, 289 -342
funções de rolagem personalizadas, 91
suposições, 291
CVAR (autoregressão vetorial cointegrada)
diferenciação automática, 298

modelo, 201

básico, 289 -292

série temporal cíclica, 63

CNN , 324-329

arquiteturas combinadas, 335-339

D

conceitos, 292-294

dados

dados, símbolos, operações, camadas e

montando uma coleção de dados de série temporal

gráficos, 294 - 298

de fontes díspares , 27-32

aprimoramento das possibilidades probabilísticas,

características, 150

464

limpeza de , 40-69

rede neural feed forward, 318-323

descartando, 435

recursos de aprendizagem, 340 -342

encontrou construção de série temporal , 33-35

aprendizado de máquina versus, 290

conjuntos de dados governamentais, 23

aprendizagem multitarefa aplicada a séries temporais,

conjuntos de dados preparados, 18 - 24

354

modernizando uma coleta de dados de série temporal

estruturas populares, 295

de uma coleção de tabelas , 26-35

pré-processamento de dados financeiros para, 410-416

simulação (veja simulação de dados de série temporal)

RNNs, 329-334

armazenamento (consulte armazenamento de dados temporais)

função softmax, 322

símbolos versus, 295

simulações de dados de séries temporais, 141

dificuldades de carimbo de data / hora , 35-39

para séries temporais, 289-342

onde encontrar , 18-26

pipeline de treinamento para, 298 -318

imputação de dados (ver dados faltantes, métodos para

demografia, origens de, 3

manuseio)

Dennis, Ricardo, 9

conjuntos de dados (ver conjuntos de dados específicos)

pacote depmixS4, 224 - 229

formatos de armazenamento de dados, desempenho computacional

diabetes (ver previsão do nível de glicose no sangue)

mance e, 361

Teste Dickey-Fuller, 84

(veja também armazenamento de dados temporais)

diferenciando, 186

pacote data.table, 42, 43

diferenciação, automática, 298

bancos de dados

convolução causal dilatada, 327

características dos dados de série temporal, 150

diluindo a tarefa de previsão, 414

desagregação, 451

pegadinhas de validação de modelo, 355

descartando dados, 435

considerações específicas do modelo para backtest

distância

ing, 347
recursos de aprendizagem para, 287
aprendizagem multitarefa aplicada a séries temporais,
similaridade versus, 283
354
métricas de distância
prevendo vários passos à frente, 353
para agrupamento, 273
abordagem recursiva para horizontes temporais distantes
temporalmente consciente, 280-285
zonas, 354
distribuição, definida, 171
mudanças estruturais no modelo, 355
conhecimento de domínio, geração de recursos e, 242
testando previsões, 344-348
recursos específicos de domínio, 249-251 , 257
estimativas, estimadores imparciais e, 165
redução da resolução, 52 -53
Distância euclidiana, 284
definido, 52
regressores harmônicos exógenos, 377-383

focar na parte do ciclo sazonal, 52

variáveis exógenas, 197

para corresponder a outros dados de baixa frequência, 53

expandindo janelas, 89

quando a resolução original dos dados não é sensível

gradientes explosivos, 331

capaz, 52

análise exploratória de dados , 73-118

métodos dunder (Python), 131

Visualizações 1D (gráficos de Gantt), 104

regressão harmônica dinâmica, 377 - 383

Visualizações 2D, 105 - 113

programação dinâmica, 223

Visualizações 3D, 113 - 116

distorção temporal dinâmica (DTW), 280-284 , 286

aplicando funções de janela , 86-91

operações de agrupamento, 74

E

histogramas, 77

previsão econômica, 7 -9

recursos de aprendizagem, 117

(veja também aplicações financeiras)
processos não-séries temporais , 73-81
geração de recursos para mercado de estoque técnico
plotagem , 74-77
indicadores de mercado, 249
gráficos de dispersão , 78-81
conjuntos de dados governamentais e, 7
autocorrelação, 91 -95
mercados comerciais, 9
correlações espúrias, 102-103
editar distância, 283
estacionariedade e, 82 - 86
de dados de uso de eletricidade, 298-318 , 332-333
métodos exploratórios específicos de séries temporais,
dados de eletrocardiogramas (ECG), 4, 250 -251
81-103
dados de eletroencefalogramas (EEG), 4
visualizações úteis, 104 -116
seleção/geração de recurso, 260-264
suavização exponencial, 56 -59
visualização, 262

Filtro Kalman Estendido (EKF), 217

tarefas embaraçosamente paralelas de processamento de dados,

403

F

variáveis endógenas, 197

Pacote Profeta do Facebook, 452-455

métodos de conjunto, 13 , 451

formato de pena, 158

características ergódicas, 264

geração de recursos, 239-251

série temporal ergódica, 242

exemplo de previsão do nível de glicose no sangue,

medição de erro, 343 - 356

390 -396

determinar a adequação das previsões, 348

agrupamento e, 273 - 280

estimar a incerteza em um modelo com um

conhecimento de domínio, 242

simulação, 350 -353

exemplos específicos de domínio, 249-251

ajustando a previsão diretamente ao horizonte de inter-

série temporal ergódica, 242
est, 353
considerações externas, 243
recursos de aprendizagem, 355
considerações gerais ao calcular
olhar para frente, 355
características, 241

série temporal de saúde, 250-251
mercados financeiros como passeio aleatório, 408 -409
exemplo introdutório, 240
formatação de dados para uma rede profunda, 413-416
duração da série temporal, 242
obtenção/exploração de dados, 404-409
codificação manual para, 264
considerações de desempenho, 422
natureza da série temporal, 242
pré-processamento de dados para aprendizagem profunda,
geração de recursos de série temporal de código aberto
410 -416
bibliotecas, 244 -249

dimensionando quantidades de interesse sem olhar
periodogramas e, 246
à frente, 411 -413
propósito de, 239
usando escala de tempo para entradas semelhantes a tempo
fontes para, 243 -251
escala para metas, 413
estacionariedade, 242
séries temporais financeiras, como disciplina separada, 410
indicadores técnicos do mercado de ações, 249
tipo de dados primeiro a entrar, primeiro a sair (FIFO), 131
classificação de série temporal, 260 -264
FitzRoy, Robert, 6
para séries temporais, 239-258
arquivos simples, 157 - 160
tsfeatures (pacote R), 248
previsão de gripe, 367-384
módulo tsfresh, 244
Modelo ARIMA usando harmônico exógeno
bibliotecas de geração de recursos, 244-249
regressores, 377 - 383

Plataforma de análise de série temporal de césio,
estudo de caso em uma área metropolitana, 367
245 - 247
exploração/limpeza de dados, 368-371
Módulo Python tsfresh, 244
ajustando um modelo ARIMA sazonal, 371-377
seleção de recursos, 252-255
pesquisa em previsões de gripe, 383
características ergódicas, 264
abordagens de última geração, 383
recursos de aprendizagem, 256
previsão
eliminação de recursos recursivos, 254
na escala, 449 - 455 , 450-452
classificação de série temporal, 260 -264
agrupamento e, 272
características
definido, 208
gerando/selecionando séries temporais, 239-258
diluindo a tarefa de previsão, 414
agrupamento hierárquico de recursos normalizados

futuro da análise de séries temporais, 463 -467
turs, 285
erros de medição em (ver medição de erro
rede neural feed forward, 318 -323
mento)
mecanismo de atenção para, 322-323
prevendo vários passos à frente, 353
exemplo, 319 -321
simulação versus, 121
RNN versus, 330
previsão como serviço, 463
Sequência de Fibonacci, 223
previsões
Conjunto de dados FiftyWords, 111
determinar a adequação de, 348
Conjunto de dados FiftyWords, 274-280
ajustando-se diretamente ao horizonte de interesse, 353
arquivos, para armazenamento de dados temporais
aprendizagem multitarefa aplicada a séries temporais,
Matrizes NumPy, 158
354

Quadro de dados do Pandas, 158
abordagem recursiva para horizontes temporais distantes
equivalentes R padrão, 158
zonas, 354
para armazenar dados temporais, 157-160
teste, 344 -348
Xarray, 159
preenchimento direto, 44 - 46
filtragem, 208
série temporal encontrada, 25
aplicações financeiras, 403 -423
construção, 33 - 35
adicionando quantidades de interesse aos valores brutos,
em conjuntos de dados governamentais, 24
410
Série de Fourier, 378
construindo/treinando uma RNN, 416 - 423
Distância Fréchet, 282
diluindo a tarefa de previsão, 414
Algoritmo FRESCO, 252 -255
previsão econômica, 7-9 , 249

futuro da análise de séries temporais, 463 - 467

previsão voltada para o consumidor, 466

pesquisa de grade, 56

aprimoramento de aprendizagem profunda de probabilística

operações de agrupamento, 74

possibilidades, 464

Distribuição guassiana (normal), 171

previsão como serviço, 463

combinações crescentes de dados estatísticos e

H

metodologias de aprendizado de máquina, 466

saúde, população, 2 - 4

importância crescente do aprendizado de máquina

aplicações de saúde, 367-401

sobre estatísticas, 465

previsão do nível de glicose no sangue, 384-400

causas de dados faltantes em, 40

G

geração de recursos, 250 - 251

GARCH (condição autorregressiva generalizada

seleção/geração de recursos para dados de EEG,

modelo de heterocedasticidade nacional), 201

260 -264

unidade recorrente fechada (GRU), 330 -332

previsão de gripe, 367 - 384

bancos de dados NoSQL de uso geral, 154 - 156

recursos de aprendizagem, 401

ESD generalizada, 457

mapas de calor, 111

geradores, 128-134 , 359

ocultos de Markov (HMMs), 218-229

desagregação geográfica, 451

Algoritmo Baum Welch e, 220

média geométrica, 47

codificação para, 224 - 229

Estrutura interna do Google para estatísticas em grande escala

ajustando o modelo, 220-222

cálculos de previsão estatística em R,

como o modelo funciona, 218

450-452

problemas de inferência, 229

conjuntos de dados governamentais, 23
recursos de aprendizagem, 236
previsão econômica e, 7
Algoritmo de Viterbi e, 222
explorando grandes dados de séries temporais, 428 -436
estado oculto, 322, 323, 334
encontrou séries temporais em, 24
agrupamento hierárquico
recursos de aprendizagem, 447
de recursos normalizados, 285
obtendo, 426
com a matriz de distância DTW, 286
análise estatística on-line de dados de séries temporais,
modelos hierárquicos de séries temporais, 201
436 - 447
dados de alta frequência, 147
preparar um conjunto de dados para testar metodologias,
análise de série temporal altamente comparativa (hctsa),
41-44
24
problemas com, 425

histogramas, 77 , 262
classificação, 432 -436
história da análise de série temporal
upsampling/agregação de dados ao iterar
astronomia, 9
através dele, 431
previsão econômica, 7-9
aplicações governamentais, 425-448
recursos de aprendizagem, 13
descartando dados, 435
medicina como problema de série temporal, 2-6
explorando grandes dados de séries temporais, 428 -436
origens das séries temporais de aprendizado de máquina
recursos de aprendizagem, 447
análise, 13
obtenção de dados governamentais, 426
desenvolvimentos pós-1960, 10
análise estatística on-line de dados de séries temporais,
previsão do tempo, 6
436 - 447
Método de Holt, 58

problemas com conjuntos de dados governamentais, 425
Suavização de Holt-Winters, 58
classificação de dados, 432 -436
horizonte de interesse, 353
upsampling/agregação de dados ao iterar
hiperparâmetros, 56
através dele, 431
árvores com gradiente aumentado, 267 , 271
EU
gráficos, aprendizagem profunda e, 289 , 296
iid (distribuído de forma independente e idêntica)
Graunt, John, 2
dados, 163 , 182

estilo de programação imperativo, 294
hora local, hora UTC versus, 37
imputação, 40
LOESS (gráfico de dispersão estimado localmente suave
(veja também dados faltantes, métodos para
ing), 58, 62
ling)

Periodograma Lomb-Scargle, 247

comparação de metodologias, 50

memória de longo prazo (LSTM), 331

dados de cuidados de saúde, 393

subsequência comum mais longa, 283

móvel para, 46-48

olhe para frente

preparar um conjunto de dados para testar metodologias,

preenchimento para trás e, 46

41-44

definido, 28

usando a média do conjunto de dados para, 48

validação do modelo e, 355

gripe (ver previsão de gripe)

prevenção, 67

InfluxoDB, 153

previsão informada, 185

M

interpolação, 40 , 49

Competição M4, 466

censura de intervalo, 445

Processo MA(q)

Modelo de ising, 134, 136

previsão, 184-186

iteradores, para pipeline de treinamento, 308

seleção de parâmetro para, 182 -184

aprendizado de máquina, 290

J.

(veja também aprendizado profundo)

Java, melhorando o desempenho computacional por

suposições, 291

usando, 364

agrupamento, 272-287

aprendizagem profunda versus, 290

K

futuras combinações de metodologia estatística

Filtro Kalman, 58, 210 -217

ogias com, 466

código para, 212 -217

recursos de aprendizagem para, 287

recursos de aprendizagem, 235

estatísticas versus, 465

visão geral da matemática, 210 - 212
origens da análise de série temporal, 13
estimativa de densidade do kernel, 262
classificação de série temporal, 260-272
Kwiatkowski–Phillips–Schmidt– Shin (KPSS)
para séries temporais, 259-288
teste, 85
visualização e, 262
ímã, modelagem de, 134 - 140
eu
Simulação de Cadeia de Markov Monte Carlo (MCMC)
ção, 135 , 232
operador de atraso (backshift), 182
Processo de Markov, 218
atraso(), 81
(veja também Modelos Ocultos de Markov (HMMs))
camadas, em aprendizagem profunda, 296
multiplicação de matrizes, 197
censura à esquerda, 445
Mcomp, 24
questões legais, no armazenamento de dados, 148

negociação mecânica, 9

tabelas de vida (atuariais), 3

remédio, 2

LuzGBM, 271

(veja também aplicações de saúde)

modelos lineares de espaço de estados gaussianos, 210

instrumentos médicos, 4 -6

(veja também filtro Kalman)

saúde da população, 2 -4

interpolação linear, 49

como problema de série temporal, 2

média móvel versus 50

memorização, 223

quando não usar, 50

meteorologia (ver previsão do tempo)

quando usar, 50

dados faltantes, métodos de tratamento, 40 - 52

regressão linear, 163

preenchimento para trás, 46

Ferramenta de classificação de linha de comando do Linux, 432

pacote data.table em R, 42

Teste de Ljung-Box, 176 , 184

preenchimento direto, 44 -46

metodologias de imputação comparadas, 50

Leis do movimento de Newton, 212 - 217

interpolação, 49

dados ruidosos, 147

recursos de aprendizagem, 69

organizações sem fins lucrativos, estatísticas de e-mail/doações

média móvel , 46-48

tiques, 122-128

preparando um conjunto de dados para testar a imputação

distribuição normal (guassiana), 171

metodologias , 41-44

função exponencial normalizada (softmax),

validação do modelo

322

problemas de teste de erro, 355

Bancos de dados NoSQL

olhar para frente, 355

escolhendo entre SQL e NoSQL, 151

mudanças estruturais, 355
uso geral, 154 -156
LSTNet modificado, 335
SQL versus, 148 - 156
MongoDB, 155
características de dados de série temporal, 150
Simulação de Monte Carol
Formato de dados NTC, 307 , 325
estimando a incerteza com, 350
produzindo em código, 311
Simulação de Cadeia de Markov Monte Carlo (MCMC)
TNC versus, 307
simulação, 135
Matrizes NumPy, 158
Lei de Moore, 11
vantagens, 158
média móvel , 46-48
desvantagens, 158
formas alternativas de, 47
como linha principal, 106
interpolação linear versus, 50

Modelos MA versus, 181

Ó

para suavizar, 55

análise estatística on-line, 436 -447

modelos de média móvel (MA), 181 - 186

valores discrepantes, 349

ACF/PACF versus, 183

(veja também Detecção de Anomalias)

como processo AR de ordem infinita, 181

sobreajuste

médias móveis versus, 181

backtesting e, 345

seleção de parâmetros para processo MA(q),

florestas aleatórias e, 266

182 - 184

convergência/divergência de média móvel

(MACD), 249

P

aprendizagem multitarefa, 354

Algoritmo P-quadrado, 436-447

série temporal multivariada, 21

pacotes (veja pacotes de séries temporais)

pacote mxnet, 295-298 , 314

Pandas, 31 , 158

função de autocorrelação parcial (PACF),

N

94-101

Modelo AR(p) e, 171 - 176

Formato de dados NC, 305, 310

de resíduos ARIMA, 190-193

valores negativos, impedindo, 377

Processos MA/AR versus, 183

redes neurais, 291

de dados não estacionários, 100 - 101

(veja também aprendizado profundo)

Correlação de Pearson, 283

suposições, 291

perceptron, 292-294

CNN, 324-329

desempenho (veja desempenho computacional)

arquiteturas combinadas, 335 -339

periodogramas, 246 , 258

rede feed forward , 318-323

sistema físico, simulação de, 134 -140

recursos de aprendizagem , 340-342

decapagem, 64 , 362

programação, 294-298

pipeline, para treinamento de aprendizagem profunda, 298-318

RNNs, 329 -334

plotagem, 74-77

programação simbólica e imperativa

saúde da população, 2 - 4

estilos, 294

Teste de Portmanteau, 200

Cidade de Nova York, conjunto de dados de chamadas 311, 425-448

conjuntos de dados preparados, 18 - 24

CompEngine, 24

objetos de zoológico, 89

conjuntos de dados governamentais, 23

florestas aleatórias, 265

Mcomp, 24

números aleatórios, 133 , 135

Repositório de aprendizado de máquina UCI, 18 -21

passeio aleatório, mercados financeiros como, 408 -409

Classificação de série temporal UEA e UCR

redes neurais recorrentes (RNNs), 329 -334

Repositório, 21

codificadores automáticos e, 334

séries temporais univariadas versus multivariadas,

construção/treinamento para aplicações financeiras,

21

416 -423

pré-processamento, 362

exemplo de previsão elétrica, 332 -333

(veja também procedimentos específicos, por exemplo,
suavização

recursos de aprendizagem, 341

ing)

pré-computação para, 360

de dados financeiros para aprendizagem profunda, 410-416

eliminação de recursos recursivos (RFE), 254

para melhorar o desempenho da computação, 362

mudanças de regime, 451

privacidade, aprendizagem profunda e, 141
índice de força relativa (RSI), 249
probabilidade, 167
Renascença, 6
Prometeu, 154 , 161
censura à direita, 445
PromQL, 154
buffer de anel, 314
Profeta (pacote Facebook), 452-455
RNNs (ver redes neurais recorrentes)
distância psicológica, desconto de tempo e,
junção rolante, 43, 54
39
média móvel , 46-48
sistemas baseados em pull, 154
rolamento significa suavização, 55 , 87
sistemas baseados em push, 153
janelas de enrolar, 86-88 , 346
Pitão
ordem de linha principal, 106
e formato de dados binários, 158, 362

métodos dunder, 131

S

Algoritmo P-quadrado, 437

Dados de preços de ações S&P 500, 403-423

divisão rolante com, 346

gráficos de dispersão, 78 -81

suavizando, 57

ARIMA sazonal (SARIMA), 201

fusos horários em , 63-67

exemplo de previsão de gripe, 371-377

módulo tsfresh, 244

ESD híbrido sazonal, 457

escrever código que é compilado em C/C++,

série temporal sazonal, 63

364

sazonalidade

Xarray, 159

redução da resolução e, 52

e previsão de gripe, 370

R

A estrutura de previsão interna do Google,

R

451

e formato de dados binários, 158, 362

comportamento humano e, 61

como coluna principal, 106

recursos de aprendizagem, 70

pacote data.table, 42 , 43

suavização de, 60 - 63

A estrutura interna do Google para projetos em larga escala

semente, definindo a, 133

cálculos de previsão estatística em R,

autocorrelação, 82, 91 - 95

450 - 452

de autocorrelação, 91-92

atraso(), 81

função de autocorrelação parcial, 94-101

formatos nativos para armazenar objetos, 158

modelo seq2seq, 334

divisão rolante com, 346

(veja também codificador automático)

pacote tsfeatures, 248

correlação serial (autocorrelação), 91
escrever código que é compilado em C/C++,
dados fragmentados, 146
365
semelhança
objetos xts, 89
distância versus, 283

Distância euclidiana como medida ruim de, 284
e teste Dickey-Fuller aumentado, 84
recursos de aprendizagem para, 287
definido, 81
simulando dados de séries temporais , 119-142
série temporal ergódica versus, 242
modelagem baseada em agente, 134
análise exploratória de dados e, 82 -86
construindo um universo de simulação que leva
seleção de recursos e, 242
cuidar de si mesmo, 128-134
na prática, 85
exemplos de código, 121-140

e Kwiatkowski-Phillips-Schmidt-Shin

previsão versus, 121

teste, 85

recursos de aprendizagem, 142

dados sazonais e cíclicos, 63

codificação manual, 122-128

fraco versus forte, 170

simulação de física, 134-140

processo estacionário, definido, 83

problemas exclusivos de, 120

modelos estatísticos para séries temporais, 163 - 205

simulações estatísticas, 141

vantagens e desvantagens, 203

suavização , 55-63

ARCO, 201

definido, 208

ARIMA, 186-196

exponencial, 56 - 59

modelos autorregressivos, 166-181

A estrutura de previsão interna do Google,

GARCH, 201

450
modelos hierárquicos de séries temporais, 201
hiperparâmetros e, 56
recursos de aprendizagem, 204
em Python, 57
problemas/limitações da regressão linear, 163
Filtro Kalman para, 217
modelos desenvolvidos para séries temporais, 166 - 202
recursos de aprendizagem, 70
modelos de média móvel, 181-186
olhar para frente e, 345
ARIMA sazonal (SARIMA), 201
propósitos de, 55
estimadores imparciais, 165
começando, 59
autoregressão vetorial, 196-201
fator de suavização (parâmetro alfa), 58
estacionariedade fraca versus forte, 170
função softmax, 322
I e, 167
classificação, 432-436

análise estatística de série temporal, 12

interpretação de spline, 50

simulações estatísticas de dados de séries temporais, 141

(veja também interpolação linear)

Estatisticas

correlações espúrias, 102 - 103

combinações de método de aprendizado de máquina

cointegração versus, 103

ologias com, 466

recursos de aprendizagem, 117

aprendizado de máquina versus 465

Bancos de dados SQL

mercado de ações

características dos dados de série temporal, 150

previsão econômica, 9

escolhendo entre SQL e NoSQL, 151

geração de recursos para mercado de estoque técnico

características de dados dos primeiros bancos de dados SQL,

indicadores de mercado, 249

150

armazenando dados temporais, 143-162

séries temporais encontradas e, 26

evitando gargalos, 361

NoSQL versus, 148-156

formato binário e desempenho computacional

dados obsoletos, 148

mance, 361

modelos de espaço de estado, 207 - 237

desempenho computacional e armazenamento

vantagens e desvantagens, 209

formatos, 361

temporal estrutural bayesiana, 229-235

soluções de banco de dados, 148 -156

Modelos ocultos de Markov , 218-229

definindo requisitos para, 145-148

Filtro Kalman, 210-217

soluções de arquivo, 157 -160

recursos de aprendizagem, 235

recursos de aprendizagem, 160

variáveis de estado, armazenamento, 147

considerações legais, 148

estacionariedade

dados ao vivo versus dados armazenados, 146-148

dados barulhentos e de alta frequência, 147

pacotes de série temporal, 449-461

Matrizes NumPy, 158

detecção de anomalia, 457 -460

Quadro de dados do Pandas, 158

ecossistema de, 460

banco de dados de série temporal popular e solução de arquivo

Pacote Profeta do Facebook, 452 -455

ções, 152 -156

previsão em escala, 449-455

pré-processamento e, 362

A estrutura interna do Google para projetos em larga escala

variáveis mudando lentamente, 147

cálculos de previsão estatística em R,

dados obsoletos, 148

450 -452

equivalentes R padrão, 158

recursos de aprendizagem, 461

casos de uso, 143

fusos horários , 63-67

Xarray, 159

horário de verão, 66

estacionariedade forte, 170

recursos de aprendizagem, 69

subsérie, redução da resolução para criar, 52

hora local versus hora universal , 37-38

estatísticas resumidas

tempo, resposta humana à passagem de, 39

expandindo janelas ao estimar, 89

bancos de dados específicos de séries temporais, 152 - 154

Filtros Kalman e, 217

InfluxoDB, 153

Aproximação Agregada Simbólica (SAX), 285

Prometeu, 154

estilo de programação simbólica, 294

carimbos de data / hora

símbolos, em aprendizagem profunda, 295

dificuldades com, 35 - 39

discrepâncias entre o momento do evento e

T

hora da gravação, 35

tabelas, modernizando a coleta de dados de séries temporais

e encontrei séries temporais, 25

de, 26 a 35

estimativa para dar sentido aos dados, 36 -38

simulação de táxi, 128 -134

hora local versus hora universal, 37, 38

agregação temporal, 451

junção contínua no pacote data.table, 43

dados temporais, armazenamento (consulte armazenamento de dados temporais

resolução temporal de, 39

dados)

dados de teste e, 348

resolução temporal, de carimbos de data/hora, 39

fusos horários e, 63 -67

análise de sensibilidade temporal, 349

aumento e redução da resolução , 52-55

Pacote TensorFlow, 417

comportamento do usuário versus comportamento da rede, 38

formato de dados de texto, desempenho computacional

Formato de dados TNC, 307

e, 361

vestígios, 26

desconto de tempo, 39

mercados de negociação (ver mercado de ações)

série temporal (geralmente)

dados de treinamento, quantidade limite de, 363

convertendo em imagens, 329

pipeline de treinamento

modelos estatísticos para, 163-205

montagem de componentes de pipeline, 317

univariada versus multivariada, 21

construindo iteradores, 308

análise de série temporal (geralmente)

para aprendizagem profunda, 298-318

astronomia, 9

métricas de avaliação, 316

breve história de, 2 -10

inspecionando o conjunto de dados, 299 -302

definido, 1

tornando o código facilmente configurável, 303-305

previsão econômica, 7-9
Formato de dados NC, 305 , 310
medicina como problema de série temporal, 2-6
Formato de dados NTC, 307, 311
origens das séries temporais de aprendizado de máquina
preparando dados de entrada, 305-309
análise, 13
definir parâmetros de treinamento e estabelecer
origens da análise estatística de séries temporais, 12
um sistema de manutenção de registros, 312-316
desenvolvimentos pós-1960, 10
moldar dados em código, 309 -312
previsão do tempo, 6
moldar dados de entrada, 305 -308
dados de série temporal (ver dados)
etapas de , 303-318

dados transacionais, 150
1D (gráficos de Gantt), 104
transformações
2D, 105-113

suposições com, 86

3D, 113-116

tornando as séries temporais estacionárias com, 85

convertendo uma série temporal em imagens, 329

objeto ts, 89

análise exploratória de dados, 104 -116

tsfeatures (pacote R), 248

recursos de aprendizagem, 118

tsfresh (módulo Python), 244, 252 -255

aprendizado de máquina e, 262

Twitter (pacote AnomalyDetection), 457 - 460

Algoritmo de Viterbi, 222

volatilidade, 410

de aprendizado de máquina **U** UCI, 18-21

C

Classificação de série temporal UEA e UCR

estacionariedade fraca, 170

Repositório, 21

previsão do tempo, 6

estimadores imparciais, 165

funções de janela

incerteza, estimativa , 350-353

funções de rolagem personalizadas, 91

dados de desemprego , 41-44

expandindo janelas, 89

raiz unitária, 83

análise exploratória de dados e, 86 -91

série temporal univariada, 21

janelas rolantes, 86 -88

horário universal (UTC), 37

Teorema de Wold, 187

aumento da resolução, 53

para alinhar entradas no modelo, 54

X

convertendo séries amostradas irregularmente em séries regulares

Xarray, 159

séries normalmente cronometradas, 54

XGBoost, 267 - 272 , 397, 466

definido, 52

objetos xts, 89

de dados governamentais, 431

interpolação baseada no habitual da variável

comportamento, 55

Y Yahoo Finanças, 404

V

Yule, Udny, 12 , 102

gradientes de fuga, 331

variáveis, exógenas versus endógenas, 197

Z

autoregressão vetorial (VAR), 196-201

objetos de zoológico, 88

visualizações

Sobre o autor

Aileen Nielsen é engenheira de software e analista de dados baseada na cidade de Nova York. Ela trabalhou com séries temporais e outros dados em diversas disciplinas, desde uma startup de saúde até uma campanha política, desde um laboratório de pesquisa física até uma empresa de comércio financeiro.

Atualmente desenvolve redes neurais para aplicações de previsão.

Colofão

O animal da capa da *Practical Time Series Analysis* é uma ovelha Bluefaced Leicester (*Ovis aries*). O Bluefaced Leicester é uma raça britânica também conhecida como *Dishley Leicester* ; foi criado pela primeira vez em Dishley, Leicestershire, no século XVIII.

Hoje, os agricultores de todo o Reino Unido e Canadá criam esta linhagem especializada de ovelhas.

As ovelhas Bluefaced Leicester têm lã branca que produz um velo fino e brilhante e fios fiados à mão de qualidade. Seus olhos castanhos são proeminentes contra seus rostos azul-acinzentados. As ovelhas são resistentes, com costas longas, ombros e pescoço fortes, focinhos largos e cascos escuros. Eles não têm lã nas pernas ou no rosto.

Quando adulto, um Leicester Bluefaced pode pesar entre 200 e 240 libras. A altura média do casco ao ombro é de 36 polegadas para carneiros e 33 polegadas para ovelhas.

Como a maioria das ovelhas, o Bluefaced Leicester pasta grama, ervas daninhas e flores. Pastam em bandos de 20 ovelhas, em média.

As ovelhas Bluefaced Leicester são muito procuradas porque os carneiros têm um talento especial para produzir o que a indústria chama de *mulas* . As mulas são ovelhas com altas taxas de reprodução que os criadores cruzam com outras raças para criar cordeiros para o mercado.

Muitos dos animais nas capas da O'Reilly estão ameaçados de extinção; todos eles são importantes para o mundo.

A ilustração da capa é de Karen Montgomery, baseada em uma gravura em preto e branco do *Meyers Kleines Lexicon* . As fontes da capa são Gilroy Semibold e Guardian Sans. A fonte do texto é Adobe Minion Pro; a fonte do título é Adobe Myriad Condensed; e a fonte do código é Ubuntu Mono da Dalton Maag.



Há muito mais

de onde isso veio.

Experimente livros, vídeos, ao vivo online

cursos de treinamento e muito mais da O'Reilly

e nossos mais de 200 parceiros – tudo em um só lugar.

Saiba mais em oreilly.com/online-learning

eilly Media, Inc'Rf O

ademarca o

ed trter

égide

eilly é um r'R. Ó

eilly Media, Inc'R

19O20©

Esboço do documento

- direito autoral
- Índice
- Prefácio
  - Quem deveria ler esse livro
    - Antecedentes Esperados
  - Por que escrevi este livro
  - Navegando neste livro
  - Recursos online
  - Convenções utilizadas neste livro
  - Usando exemplos de código
  - Aprendizagem on-line O'Reilly
  - Como entrar em contato conosco
  - Agradecimentos
- Capítulo 1. Séries Temporais: Uma Visão Geral e um Histórico Rápido
  - A história das séries temporais em diversas aplicações
    - Medicina como um problema de série temporal
    - Previsão do tempo
    - Previsão de crescimento econômico
    - Astronomia
  - A análise de série temporal decola
  - As origens da análise estatística de séries temporais
  - As origens da análise de série temporal de aprendizado de máquina
  - Mais recursos
- Capítulo 2. Encontrando e organizando dados de séries temporais
  - Onde encontrar dados de série temporal
    - Conjuntos de dados preparados
    - Série temporal encontrada
  - Adaptando uma coleção de dados de série temporal a partir de uma coleção de tabelas
    - Um exemplo prático: montando uma coleção de dados de série temporal
    - Construindo uma série temporal encontrada
  - Problemas de carimbo de data/hora
    - De quem é o carimbo de data/hora?

        - Adivinhando carimbos de data e hora para entender os dados
        - O que é uma escala de tempo significativa?
    - Limpando seus dados
        - Tratamento de dados ausentes
        - Upsampling e Downsampling
        - Suavização de dados
    - Dados sazonais
    - Fusos horários
    - Prevenindo a antecipação
    - Mais recursos
- Capítulo 3. Análise Exploratória de Dados para Séries Temporais
    - Métodos familiares
        - Plotagem
        - Histogramas
        - Gráficos de dispersão
    - Métodos Exploratórios Específicos de Séries Temporais
        - Compreendendo a estacionariedade
        - Aplicando funções de janela
        - Compreendendo e identificando a autocorrelação
        - Correlações Espúrias
    - Algumas visualizações úteis
        - Visualizações 1D
        - Visualizações 2D
        - Visualizações 3D
    - Mais recursos
- Capítulo 4. Simulação de dados de série temporal
    - O que há de especial na simulação de séries temporais?
        - Simulação versus previsão
    - Simulações em código
        - Fazendo o trabalho sozinho
        - Construindo um universo de simulação que funciona sozinho
        - Uma simulação de física
    - Notas Finais sobre Simulações
        - Simulações Estatísticas
        - Simulações de aprendizagem profunda
    - Mais recursos
- Capítulo 5. Armazenando Dados Temporais
    - Definindo Requisitos

        - Dados ativos versus dados armazenados
    - Soluções de banco de dados
        - SQL versus NoSQL
        - Soluções populares de banco de dados e arquivos de séries temporais
    - Soluções de arquivos
        - NumPy
        - Pandas
        - Equivalentes R padrão
        - Matriz X
    - Mais recursos
- Capítulo 6. Modelos Estatísticos para Séries Temporais
    - Por que não usar uma regressão linear?
    - Métodos estatísticos desenvolvidos para séries temporais
        - Modelos Autoregressivos
        - Modelos de média móvel
        - Modelos de média móvel integrados autorregressivos
        - Autoregressão vetorial
        - Variações em modelos estatísticos
    - Vantagens e desvantagens dos métodos estatísticos para séries temporais
    - Mais recursos
- Capítulo 7. Modelos de Espaço de Estados para Séries Temporais
    - Modelos de espaço de estado: vantagens e desvantagens
    - O Filtro de Kalman
        - Visão geral
        - Código para o Filtro Kalman
    - Modelos ocultos de Markov
        - Como funciona o modelo
        - Como ajustamos o modelo
        - Ajustando um HMM no código
    - Série temporal estrutural bayesiana
        - Código para bsts
    - Mais recursos
- Capítulo 8. Gerando e selecionando recursos para uma série temporal
    - Exemplo introdutório
    - Considerações gerais ao calcular recursos
        - A natureza da série temporal
        - Conhecimento de Domínio

        - Considerações Externas
    - Um catálogo de lugares para encontrar recursos de inspiração
        - Bibliotecas de geração de recursos de série temporal de código aberto
        - Exemplos de recursos específicos de domínio
    - Como selecionar recursos depois de gerá-los
    - Considerações finais
    - Mais recursos
- Capítulo 9. Aprendizado de Máquina para Séries Temporais
    - Classificação de série temporal
        - Selecionando e gerando recursos
        - Métodos de árvore de decisão
    - Agrupamento
        - Gerando recursos a partir dos dados
        - Métricas de distância com reconhecimento temporal
        - Código de agrupamento
    - Mais recursos
- Capítulo 10. Aprendizado profundo para séries temporais
    - Conceitos de aprendizagem profunda
    - Programando uma rede neural
        - Dados, símbolos, operações, camadas e gráficos
    - Construindo um pipeline de treinamento
        - Inspecionando nosso conjunto de dados
        - Etapas de um pipeline de treinamento
    - Redes Feed Forward
        - Um exemplo simples
        - Usando um mecanismo de atenção para tornar as redes feed forward mais conscientes do tempo
    - CNN
        - Um modelo convolucional simples
        - Modelos Convolucionais Alternativos
    - RNNs
        - Continuando nosso exemplo elétrico
        - A inovação do codificador automático
    - Arquiteturas de Combinação
    - Resumindo
    - Mais recursos
- Capítulo 11. Erro de Medição

    - O básico: como testar previsões
        - Considerações específicas do modelo para backtesting
    - Quando sua previsão é boa o suficiente?
    - Estimando a incerteza em seu modelo com uma simulação
    - Prevendo vários passos à frente
        - Ajustar-se diretamente ao horizonte de interesse
        - Abordagem recursiva para horizontes temporais distantes
        - Aprendizagem multitarefa aplicada a séries temporais
    - Dicas de validação de modelo
    - Mais recursos
- Capítulo 12. Considerações de desempenho no ajuste e exibição de modelos de série temporal
    - Trabalhando com ferramentas desenvolvidas para casos de uso mais gerais
        - Modelos construídos para dados transversais não “compartilham” dados entre amostras
        - Modelos que não pré-calculam criam um atraso desnecessário entre a medição dos dados e a elaboração de uma previsão
    - Formatos de armazenamento de dados: vantagens e desvantagens
        - Armazene seus dados em formato binário
        - Pré-processe seus dados de uma forma que permita “deslizar” sobre eles
    - Modificando sua análise para atender às considerações de desempenho
        - Usar todos os seus dados não é necessariamente melhor
        - Modelos complicados nem sempre funcionam melhor o suficiente
        - Uma breve menção a ferramentas alternativas de alto desempenho
    - Mais recursos
- Capítulo 13. Aplicações de Saúde
    - Prevendo a gripe
        - Um estudo de caso de gripe em uma área metropolitana
        - O que é o estado da arte na previsão da gripe?

    - Previsão dos níveis de glicose no sangue
        - Limpeza e exploração de dados
        - Gerando Recursos
        - Ajustando um modelo
    - Mais recursos
- Capítulo 14. Aplicações Financeiras
    - Obtenção e exploração de dados financeiros
    - Pré-processamento de dados financeiros para aprendizado profundo
        - Adicionando Quantidades de Interesse aos Nossos Valores Brutos
        - Dimensionando quantidades de interesse sem previsão
        - Formatando nossos dados para uma rede neural
    - Construindo e treinando um RNN
    - Mais recursos
- Capítulo 15. Séries Temporais para o Governo
    - Obtenção de dados governamentais
    - Explorando grandes dados de séries temporais
        - Faça um upsample e agregue os dados à medida que os iteramos
        - Classifique os dados
    - Análise estatística on-line de dados de séries temporais
        - Perguntas restantes
        - Melhorias adicionais
    - Mais recursos
- Capítulo 16. Pacotes de Séries Temporais
    - Previsão em escala
        - Previsão industrial interna do Google
        - Pacote Profeta de Código Aberto do Facebook
    - Detecção de anomalia
        - Pacote de detecção de anomalias de código aberto do Twitter
    - Outros pacotes de série temporal
    - Mais recursos
- Capítulo 17. Previsões sobre previsões
    - Previsão como serviço
    - Deep Learning aumenta possibilidades probabilísticas
    - Importância crescente do aprendizado de máquina em vez da estatística

    - Combinação crescente de metodologias estatísticas e de aprendizado de máquina
    - Mais previsões para a vida cotidiana
- Índice
- Sobre o autor
- Colofão